
Agentic AI using LangGraph

From LLM Workflows to Autonomous Agents

*A complete, worked treatment of stateful agent engineering —
state and reducers, conditional and iterative graphs, persistence, streaming,
tools, RAG, MCP, human-in-the-loop, memory, and observability.*

CampusX

End-to-End GenAI Course

25 chapters • 232 figures • 213 tables • 535 code listings

About this volume

This book is a typeset edition of the course notes for the *Agentic AI using LangGraph* series. Each chapter corresponds to one session of the course, reproduced in full: every diagram, comparison table and code listing from the source notes appears here.

The diagrams are not screenshots. Every flow diagram was re-rendered from its original mermaid source as a vector graphic, so it stays sharp at any zoom level and its text remains selectable and searchable. The chapter overview plates were likewise embedded as vectors.

Cross-references are live: entries in the table of contents, the list of figures and the list of tables are all clickable, as are the PDF bookmarks.

Typeset with Xe^LA_TE_X. Source notes converted via pandoc; diagrams rendered with mermaid-cli.

Contents

Agentic AI using LangGraph - Complete Playlist Introduction	xxv
Introduction	xxv
Why This Playlist?	xxv
Vision Behind the Playlist	xxvi
Curriculum Overview	xxvii
Module-wise Detailed Breakdown	xxviii
Prerequisites	xxxi
Frequently Asked Questions	xxxii
Key Takeaways	xxxiii
Glossary of Terms	xxxiii
1 Generative AI vs Agentic AI - Complete Notes	1
1.1 What is Generative AI?	1
1.2 Generative AI vs Traditional AI	1
1.3 Applications of Generative AI	3
1.4 Practical Scenario: HR Recruitment Process	4
1.5 Evolution: 4 Stages of AI Implementation	4
1.6 Stage 1: Simple LLM Chatbot	4
1.7 Stage 2: RAG-Based Chatbot (Retrieval Augmented Generation)	6
1.8 Stage 3: Tool-Augmented Chatbot	6
1.9 Stage 4: Agentic AI	7
1.10 Final Comparison	9
1.11 Key Insights	11
1.12 Summary	11
1.13 Next Steps	12
1.14 Key Takeaway	12
2 Agentic AI - Complete Study Notes	15
2.1 Overview	15
2.2 What is Agentic AI?	15
2.3 Real-World Example: AI HR Recruiter	15
2.4 ★ Six Key Characteristics of Agentic AI	17
2.5 Five Core Components of Agentic AI	21
2.6 Summary Table	23
2.7 Key Takeaways	23
2.8 What's Next?	23
3 LangChain Vs LangGraph Agentic AI using LangGraph Video 3 CampusX	27
3.1 Current Video Goals	27
3.2 Framework Landscape	27
3.3 Prerequisites	27
3.4 Key Questions to be Addressed	28
3.5 Video Structure Breakdown	28
3.6 Learning Approach	28
3.7 Development Challenges	28
3.8 LangChain Recap - Foundation for LangGraph Understanding	30
3.9 Quick Recap Context	30

3.10	What is LangChain?	30
3.11	LangChain Components (Modular Building Blocks)	30
3.12	What Can You Build with LangChain?	33
3.13	Learning Path Recommendations	37
3.14	LangChain Strengths Summary	37
3.15	Key Takeaways	37
3.16	Automated Hiring Workflow - LangChain Implementation Analysis	37
3.17	Workflow Overview	38
3.18	Critical Distinction: Workflow vs Agentic AI	38
3.19	Detailed Workflow Breakdown	39
3.20	Workflow Characteristics Analysis	43
3.21	! LangChain Implementation Challenges Preview	44
3.22	Upcoming Analysis Framework	44
3.23	Key Takeaways	46
3.24	Challenge #2: State Handling - LangChain vs LangGraph	46
3.25	Understanding State in Complex Workflows	46
3.26	State Evolution Through Workflow	47
3.27	✗ LangChain's State Handling Limitations	47
3.28	Manual State Implementation in LangChain	48
3.29	✓ LangGraph's State Handling Solution	49
3.30	LangGraph State Architecture	50
3.31	Comparative Analysis	52
3.32	Real-World Implications	52
3.33	Key Terminology	53
3.34	Decision Framework	53
3.35	Key Takeaways	53
3.36	Next Challenge Preview	54
3.37	Challenge #3: Event Driven Execution - LangChain vs LangGraph	54
3.38	Understanding Execution Models	54
3.39	Sequential Execution Model	54
3.40	Event-Driven Execution Model	55
3.41	Real-World Event-Driven Examples	55
3.42	✗ LangChain's Sequential Limitation	57
3.43	✓ LangGraph's Event-Driven Solution	58
3.44	Event-Driven Flow Comparison	59
3.45	Technical Implementation Comparison	59
3.46	Real-World Scenarios Requiring Event-Driven Execution	59
3.47	Decision Framework	62
3.48	Key Insights	62
3.49	Next Challenge Preview	62
3.50	Nested Workflows Feature & Challenge #6: Observability	63
3.51	LangGraph Feature: Nested Workflows (Subgraphs)	63
3.52	Real-World Example: Interview Process	63
3.53	Two Major Use Cases	63
3.54	✗ LangChain's Nested Workflow Limitation	67
3.55	Challenge #6: Observability	69
3.56	Observability Solutions	69
3.57	✗ LangChain's Observability Limitation	70
3.58	✓ LangGraph's Complete Observability	71
3.59	Observability Comparison	72
3.60	Production Impact	73

3.61	Key Insights	73
3.62	Final Framework Decision Matrix	74
3.63	Complete Challenge Summary	74
3.64	Conclusion & Revision: LangChain vs LangGraph Complete Summary	74
3.65	Quick Revision Questions	75
3.66	When to Use What? (Decision Framework)	75
3.67	Critical Question: Should We Abandon LangChain?	77
3.68	How They Work Together	78
3.69	Key Insights & Learning Value	78
3.70	Big Picture Understanding	79
3.71	What's Coming Next	79
3.72	Complete Learning Checklist	80
3.73	Inspiration & Motivation	80
3.74	Key Takeaways	81
4	LangGraph Core Concepts - Complete Guide	83
4.1	1. What is LangGraph?	83
4.2	2. LLM Workflows	84
4.3	3. Common LLM Workflow Patterns	85
4.4	4. Graphs, Nodes, and Edges	92
4.5	5. State Management	95
4.6	6. Reducers	97
4.7	7. LangGraph Execution Model	100
4.8	8. Summary and Key Takeaways	103
5	Sequential Workflows in LangGraph	107
5.1	Course Overview & Context	107
5.2	Sequential Workflows Explained	107
5.3	Development Environment Setup	107
5.4	Installation Verification	109
5.5	Quick Reference Commands	109
5.6	LangGraph Sequential Workflows - Detailed Study Notes	109
5.7	Course Overview & Context	110
5.8	Sequential Workflows Explained	110
5.9	Development Environment Setup	110
5.10	Key Learning Points	111
5.11	First Workflow: BMI Calculator	112
5.12	Quick Reference Commands	116
5.13	Complete BMI Workflow Code	116
5.14	LangGraph Tutorial Notes: Simple LLM Workflow	118
5.15	Overview	118
5.16	Setup and Imports	118
5.17	State Definition	118
5.18	Core Node Function	119
5.19	Graph Construction	119
5.20	Execution	119
5.21	Comparison with Direct LLM Call	120
5.22	Learning Objectives	120
5.23	Key Takeaways	120
5.24	LangGraph Tutorial Notes: Prompt Chaining Workflow	120
5.25	Overview	120
5.26	Concept: Prompt Chaining	120

5.27	State Definition	120
5.28	Implementation	121
5.29	Sample Output	122
5.30	Key Advantages of LangGraph vs Traditional Chains	123
5.31	Homework Assignment	123
5.32	Learning Objectives	124
5.33	Key Takeaways	124
6	Parallel Workflows in LangGraph	127
6.1	Code	127
6.2	LangGraph Parallel Workflow Notes	127
6.3	Overview	127
6.4	Workflow Structure	127
6.5	Node Functions	127
6.6	Graph Construction	128
6.7	Key Concepts	128
6.8	Sample Execution	130
6.9	Best Practices	130
6.10	Common Pitfalls	130
6.11	LLM-Based Parallel Workflow Notes	131
6.12	Essay Evaluation System Overview	131
6.13	State Structure	131
6.14	Key Concepts Integration	131
6.15	LLM Setup for Structured Output	133
6.16	Three Core Technical Challenges	133
6.17	Implementation Strategy	134
6.18	Benefits of This Architecture	134
6.19	Real-World Applications	135
6.20	Complete UPSC Essay Evaluation Workflow - Implementation Notes	135
6.21	Overview	135
6.22	Code Structure Analysis	135
6.23	Node Implementation Details	137
6.24	Graph Construction	138
6.25	Real-World Testing Results	138
6.26	Technical Architecture Benefits	139
6.27	Advanced State Management	139
6.28	Production Considerations	139
6.29	Integration Possibilities	140
7	Conditional Workflows in LangGraph - Complete Guide	143
7.1	Code	143
7.2	1. Introduction	143
7.3	2. Understanding Workflow Types in LangGraph	143
7.4	3. Key Differences: Parallel vs Conditional Workflows	145
7.5	4. How Conditional Workflows Work	145
7.6	5. Example 1: Quadratic Equation Solver (Non-LLM Based)	147
7.7	6. Example 2: Customer Review Handling System (LLM-Based)	153
7.8	7. Key Components of Conditional Workflows	161
7.9	8. Best Practices	161
7.10	9. Summary	162

8	Iterative (Looping) Workflows in LangGraph	165
8.1	Code	165
8.2	1. Introduction and Context	165
8.3	2. Types of Workflows Recap	165
8.4	3. Understanding Iterative Workflows	169
8.5	4. Real-World Use Case: Social Media Post Generator	170
8.6	5. Workflow Architecture Deep Dive	171
8.7	6. Implementation Guide	172
8.8	7. State Design with TypedDict	173
8.9	8. Building the Three Core Nodes	174
8.10	9. Structured Output with Pydantic	176
8.11	10. Creating the Loop with Conditional Edges	178
8.12	11. Complete Code Implementation	179
8.13	12. Testing and Execution	182
8.14	13. Key Takeaways and Best Practices	183
9	How to build a Chatbot using LangGraph	187
9.1	Overview	187
9.2	Today's Objective	187
9.3	Chatbot Architecture	187
9.4	Implementation	188
9.5	The Memory Problem	189
9.6	Solution: Persistence	189
9.7	Behind the Scenes: How Persistence Works	191
9.8	Key Concepts Introduced	191
9.9	State Inspection	191
9.10	Production Considerations	192
9.11	Next Steps Teased	192
9.12	Summary	192
10	LangGraph Persistence: Complete Comprehensive Guide	195
10.1	Code	195
10.2	1. Introduction	195
10.3	2. What is Persistence?	195
10.4	3. How Persistence Works	198
10.5	4. Checkpointers: The Implementation Mechanism	199
10.6	5. Threads: Managing Multiple Executions	201
10.7	6. Implementation in Code	202
10.8	7. Four Benefits of Persistence	204
10.9	8. Practical Code Examples	206
10.10	9. Best Practices and Production Considerations	209
10.11	10. Summary and Quick Reference	210
10.12	Conclusion	212
11	Building a LangGraph Chatbot with Streamlit UI	213
11.1	Code	213
11.2	1. Introduction	213
11.3	2. Understanding the Problem	213
11.4	3. Architecture Overview	214
11.5	4. Prerequisites	214
11.6	5. Project Setup	215
11.7	6. Streamlit Chat Interface Fundamentals	216

11.8	7. Building a Basic Copy-Cat Chatbot	218
11.9	8. Understanding Streamlit's Execution Model	218
11.10	9. Session State: Persisting Conversation History	220
11.11	10. Integrating LangGraph with Streamlit	221
11.12	11. Complete Implementation	222
11.13	12. Common Pitfalls and Solutions	224
11.14	13. Summary	226
12	Streaming in LLM Applications using LangGraph	229
12.1	Code	229
12.2	Introduction	229
12.3	What is Streaming?	229
12.4	Why Streaming is Important	230
12.5	How Streaming Works - Technical Foundation	232
12.6	Python Generators - The Building Block	232
12.7	Implementing Streaming in LangGraph	234
12.8	Stream Modes in LangGraph	234
12.9	Frontend Integration with Streamlit	236
12.10	Complete Code Implementation	237
12.11	Key Takeaways	239
12.12	Further Learning	240
13	LangGraph Chatbot: Resume Chat Feature - Comprehensive Guide	243
13.1	Code	243
13.2	Introduction	243
13.3	Feature Overview	243
13.4	Architecture Understanding	244
13.5	Implementation Roadmap	245
13.6	Detailed Implementation	245
13.7	Code Walkthrough	251
13.8	Key Concepts Explained	254
13.9	Current Limitations and Future Improvements	256
13.10	Summary	257
14	LangGraph Persistent Storage with SQLite Database	259
14.1	Code	259
14.2	1. Introduction and Problem Statement	259
14.3	2. Understanding Checkpointers in LangGraph	261
14.4	3. SQLite Saver Implementation	262
14.5	4. Backend Code Implementation	262
14.6	5. Database Connection Deep Dive	263
14.7	6. Thread Management and Retrieval	264
14.8	7. Frontend Integration	265
14.9	9. Complete Code Reference	268
14.10	10. Summary and Best Practices	272
15	LangSmith: Complete Guide to LLM Observability and Debugging	275
15.1	Code	275
15.2	1. Introduction	275
15.3	2. Why Do We Need LangSmith?	275
15.4	3. The Core Problem: Non-Deterministic Nature of LLMs	280
15.5	4. Understanding Observability	281

15.6	5. What is LangSmith?	282
15.7	6. What Does LangSmith Trace?	283
15.8	7. Core Concepts: Project, Trace, and Run	283
15.9	8. Setup and Configuration	285
15.10	9. Practical Implementation	287
15.11	10. Solving the Initial Scenarios	291
15.12	11. Best Practices	292
15.13	Summary	293
15.14	1. Introduction to RAG and LangSmith Integration	294
15.15	2. Understanding RAG Applications	294
15.16	3. The Two Critical Failure Modes in RAG	295
15.17	4. Why LangSmith is Essential for RAG Applications	296
15.18	5. Basic RAG Implementation (Version 1)	297
15.19	6. Adding Tracing with @traceable Decorator (Version 2)	298
15.20	7. Solving the Latency Problem with Caching (Version 4)	301
15.21	8. Index Rebuild Trigger Conditions	303
15.22	9. Adding Tags and Metadata for Rich Tracing	303
15.23	10. Tracing Agentic Applications	304
15.24	11. Complete Code Reference	306
15.25	12. Best Practices and Summary	309
15.26	Key Takeaways	310
15.27	LangGraph Fundamentals	310
15.28	LangGraph + LangSmith Integration	311
15.29	Other Features of LangSmith	315
15.30	LLMOps Overview	323
15.31	Best Practices Summary	323
15.32	Conclusion	324
16	LangSmith Observability for LangGraph Chatbot	327
16.1	Code	327
16.2	1. Introduction and Context	327
16.3	2. What is Observability?	327
16.4	3. Why LangSmith for Observability?	328
16.5	4. Prerequisites	329
16.6	5. LangSmith Setup Guide	329
16.7	6. Understanding LangSmith Architecture	331
16.8	7. Basic Tracing Implementation	331
16.9	8. The Thread Organization Problem	332
16.10	9. Implementing Thread-Based Organization	332
16.11	10. Complete Code Implementation	333
16.12	11. LangSmith Dashboard Navigation	335
16.13	12. Benefits and Use Cases	336
16.14	13. Quick Reference	336
16.15	Summary	337
16.16	Additional Resources	337
17	Tools in LangGraph: Comprehensive Guide	339
17.1	Code	339
17.2	1. Introduction	339
17.3	2. Understanding the Problem	339
17.4	3. What are Tools?	340
17.5	4. Core Concepts	340

17.6	5. Architecture and Workflow	344
17.7	6. Types of Tools	348
17.8	7. Complete Code Implementation	349
17.9	8. Execution Flow Analysis	354
17.10	9. Common Problems and Solutions	354
17.11	10. Frontend Integration	356
17.12	11. Key Takeaways	356
17.13	Quick Reference Code	357
17.14	Glossary	359
18	Agentic AI using LangGraph: Adding MCP (Model Context Protocol)	361
18.1	Code	361
18.2	Playlist Recap	361
18.3	What is MCP?	361
18.4	! Problems with Traditional Tools	362
18.5	✓ MCP Solution	363
18.6	Converting to Async	365
18.7	Building MCP Client in LangGraph	366
18.8	Demo & Implementation	368
18.9	Key Takeaways	369
18.10	Next Steps	370
18.11	Complete Architecture	370
18.12	Pro Tips	371
19	RAG (Retrieval Augmented Generation) Integration with LangGraph	373
19.1	Code	373
19.2	Introduction and Context	373
19.3	Understanding the Problem: Why RAG?	374
19.4	What is RAG? The Core Concept	375
19.5	RAG Architecture: How It Works	376
19.6	Implementation Strategy: RAG as a Tool	378
19.7	Step-by-Step Code Implementation	379
19.8	Production-Ready Integration	383
19.9	Observability and Debugging	385
19.10	Key Takeaways	386
20	Human-in-the-Loop (HITL) in LangGraph - Complete Guide	389
20.1	Code	389
20.2	1. Introduction to HITL	389
20.3	2. What is HITL?	389
20.4	3. Why HITL Exists in Agentic AI Systems	390
20.5	4. Benefits of HITL	390
20.6	5. Common HITL Patterns	393
20.7	6. HITL in LangGraph - Architecture Overview	395
20.8	7. Core Components for HITL Implementation	395
20.9	8. Step-by-Step HITL Workflow in LangGraph	397
20.10	9. Code Implementation - Basic Example	398
20.11	10. Code Implementation - Advanced Chatbot Example	402

21 LangGraph Subgraphs - Complete Guide	407
21.1 Video Overview	407
21.2 Plan of Action	407
21.3 Part 1: What Are Subgraphs?	407
21.4 Part 2: Why Are Subgraphs Needed?	408
21.5 Real-World Example: Software Development Agent	409
21.6 Core Benefits of Subgraphs	409
21.7 LangGraph-Specific Benefits	411
21.8 Part 3: Implementation Mechanisms	413
21.9 Comparison: Two Methods	414
21.10 Practical Example: Translation Workflow	415
21.11 Implementation: Method 1 (Separate States)	415
21.12 Implementation: Method 2 (Shared State)	419
21.13 Complete Comparison	421
21.14 Best Practices & Additional Features	422
21.15 Documentation Resources	422
21.16 When to Use Which Method?	423
21.17 Future Applications	423
21.18 Key Takeaways	424
21.19 Architecture Patterns Summary	424
21.20 Example Architectures	425
21.21 Debugging Tips	425
21.22 Performance Considerations	425
21.23 Quick Reference	426
21.24 Additional Study Resources	427
22 Memory in Generative AI — Complete Notes	429
22.1 1. Introduction: Why Memory Matters in GenAI	429
22.2 2. How LLMs Work at Inference — The Mathematical View	429
22.3 3. LLMs Are Stateless by Design	431
22.4 4. The Core Problem: LLMs Have No Intrinsic Memory	431
22.5 5. Two Prerequisite Concepts for Building Memory	432
22.6 6. Short-Term Memory — The First Solution	432
22.7 7. Problems with Short-Term Memory	434
22.8 8. The Need for Long-Term Memory (LTM)	435
22.9 9. Types of Long-Term Memory	436
22.10 10. How Long-Term Memory Works — High-Level Architecture	437
22.11 11. Complete Memory Architecture — End-to-End Flow	438
22.12 12. Challenges in Building Memory Systems	440
22.13 13. Tools and Libraries for Memory Management	440
22.14 14. The Future: Intrinsic Memory in LLMs	440
22.15 15. Summary — Complete Memory Landscape	441
23 Short-Term Memory in LangGraph — Detailed Notes	443
23.1 Table of Contents	443
23.2 1. Prerequisites & Recap: LLMs Are Stateless	443
23.3 2. What is Short-Term Memory (STM)?	444
23.4 3. Implementing STM in LangGraph — Checkpointers & Threads	444
23.5 4. Without Short-Term Memory (Code Example)	445
23.6 5. With Short-Term Memory Using InMemorySaver (Code Example)	446
23.7 6. Limitation of InMemorySaver — Volatile Storage	447
23.8 7. Persistence Using PostgreSQL	447

23.9	8. Setting Up PostgreSQL via Docker	447
23.10	9. Persistent Memory with PostgresSaver (Code Example)	448
23.11	10. Context Overflow Problem	449
23.12	11. Technique 1: Trimming	450
23.13	12. Technique 2: Deletion	452
23.14	13. Technique 3: Summarization	453
23.15	14. Summarization Workflow Implementation (Code Example)	454
23.16	15. Comparison Table — Trimming vs Deletion vs Summarization	457
23.17	16. Key Takeaways	458
24	Long Term Memory in LangGraph — Complete Notes	459
24.1	1. Introduction: Why Long Term Memory Matters	459
24.2	2. LangGraph Memory Store Architecture	460
24.3	3. Namespaces — Organizing Memories	461
24.4	4. Working with InMemoryStore — CRUD Operations	462
24.5	5. Semantic Search in Memory Store	463
24.6	6. Part 1 — Chatbot That READS Existing Memories	464
24.7	7. Part 2 — Chatbot That CREATES New Memories	466
24.8	8. De-Duplication Strategy	468
24.9	9. Merged Workflow — Read + Write Memories Together	469
24.10	10. Production Setup with PostgresStore	472
24.11	11. Summary — Complete Mental Model	473
25	Planning AI Agent — Blog Writing Agent with LangGraph	475
25.1	Table of Contents	475
25.2	1. What is a Planning Agent?	475
25.3	2. Planning Agent vs Direct Execution	475
25.4	3. Project Overview — Blog Writing Agent	476
25.5	4. High-Level Architecture	477
25.6	5. Orchestrator-Worker Pattern	477
25.7	6. Stage 1 — Basic Blog Writing Agent	479
25.8	7. Improving with Better Prompts & Pydantic Models	482
25.9	8. Stage 2 — Adding Internet Research	482
25.10	9. Tavily — Search Engine for LLMs	484
25.11	10. Complete Data Flow Summary	485
25.12	Key Concepts Recap	486
25.13	Blog Writing Agent — Stage 3 (Image Generation) & Stage 4 (Streamlit UI)	486
25.14	Table of Contents	486
25.15	1. Research-Aware Code Implementation (Recap)	486
25.16	2. Stage 3 — Adding Automatic Image Generation	489
25.17	3. Reducer Subgraph — The Three-Node Pipeline	490
25.18	4. Pydantic Schemas for Image Generation	492
25.19	5. Gemini API Setup for Image Generation	493
25.20	6. Stage 4 — Building the Streamlit GUI	494
25.21	7. Complete System Summary	495

List of Figures

1	Why This Playlist?	xxvi
2	Build Up	xxvi
3	Vision Behind the Playlist	xxvii
4	Curriculum Overview	xxviii
5	Module 2: LangGraph Fundamentals	xxix
6	Module 3: Advanced LangGraph Concepts	xxx
7	Module 4: AI Agents	xxxi
8	Module 5: Agentic RAG	xxxi
9	Module 6: Productionization	xxxi
10	Glossary of Terms	xxxiv
1.1	Key Characteristics	2
1.2	Generative AI	3
1.3	Design & Graphics	5
1.4	Practical Scenario: HR Recruitment Process	5
1.5	Adaptable	8
1.6	Agent's Autonomous Actions	10
1.7	Key Insights	11
1.8	Key Takeaway	13
2.1	Key Difference from Generative AI	16
2.2	How it Works	17
2.3	Autonomy	18
2.4	Context Awareness	20
2.5	Context Awareness	21
2.6	Five Core Components of Agentic AI	21
2.7	Summary Table	23
2.8	What's Next?	25
3.1	Video Structure Breakdown	29
3.2	Why Not Code from Scratch?	29
3.3	Model Component	31
3.4	Retrievers Component	32
3.5	Chains Component (Most Important)	32
3.6	Simple Conversational Workflows	33
3.7	Multistep Workflows	34
3.8	RAG (Retrieval Augmented Generation) Applications	35
3.9	Basic Level Agents	36
3.10	Key Differences Explained	38
3.11	Phase 1: Job Creation & Approval	40
3.12	Phase 2: Application Monitoring & Optimization	41
3.13	Phase 3: Candidate Processing	42
3.14	Phase 4: Interview & Selection Process	43
3.15	Phase 5: Offer & Onboarding	45
3.16	Complexity Spectrum	46
3.17	Critical Question: What Data is Important?	46
3.18	Available Memory vs Required Memory	48
3.19	Built-in State Management	49

3.20	Automatic State Management	51
3.21	State Complexity Scaling	53
3.22	Two Fundamental Execution Types	54
3.23	Traditional LangChain Workflow	54
3.24	Definition	56
3.25	Post JD $\rightarrow$ 7-Day Wait	56
3.26	Modify JD $\rightarrow$ 48-Hour Wait	57
3.27	Send Offer $\rightarrow$ Wait for Response	57
3.28	State Persistence Options	58
3.29	LangChain Approach (Complex & Error-Prone)	60
3.30	LangGraph Approach (Clean & Elegant)	61
3.31	Core Concept: Graph within Graph	64
3.32	Before Nested Workflows	65
3.33	After Nested Workflows	66
3.34	Self-Driving Car Example	67
3.35	Approval Workflow Example	68
3.36	Real-World Example: Costly Mistake	69
3.37	Partial Observability Problem	70
3.38	What Gets Tracked	71
3.39	Complete Timeline Creation	72
3.40	Simplified Mental Model	75
3.41	Use LangChain When	76
3.42	Use LangGraph When	76
3.43	The Relationship Explained	77
3.44	Different Purposes, Same Ecosystem	79
3.45	Learning Progression	80
3.46	Key Takeaways	82
4.1	How LangGraph Works	83
4.2	Example: Automated Hiring Workflow	84
4.3	1 Prompt Chaining	86
4.4	Real-World Example: Report Generation	86
4.5	2 Routing	87
4.6	3 Parallelization	88
4.7	Real-World Example: YouTube Content Moderation	88
4.8	4 Orchestrator Workers	89
4.9	Real-World Example: Research Assistant	91
4.10	5 Evaluator Optimizer	91
4.11	When to Use Evaluator Optimizer	92
4.12	Graph Visualization	94
4.13	The Problem Reducers Solve	98
4.14	When Default Behavior Fails	99
4.15	Execution Phases	101
4.16	Execution Flow in Detail	103
4.17	Best Practices	105
5.1	BMI Calculator Workflow Structure	112
5.2	Key Learning Points	115
5.3	Execution	122
5.4	Key Takeaways	125
6.1	Parallel Flow Architecture	129

6.2	Edge Definitions	129
6.3	Workflow Architecture	132
6.4	Code Structure Analysis	136
6.5	Assessment Tools	141
7.1	1 Sequential Workflows	143
7.2	2 Parallel Workflows	144
7.3	3 Conditional Workflows	146
7.4	3 Conditional Workflows	147
7.5	3 Execution Flow	148
7.6	2 Workflow Design	149
7.7	1 System Overview	154
7.8	2 Workflow Design	155
7.9	What's Next?	164
8.1	1 Sequential Workflows	165
8.2	2 Parallel Workflows	166
8.3	3 Conditional Workflows	167
8.4	4 Iterative (Looping) Workflows	168
8.5	2 The Improvement Cycle	170
8.6	1 Visual Representation	171
8.7	5 The Big Picture	185
8.8	5 The Big Picture	185
9.1	Basic Design	188
9.2	Storage Options	190
9.3	Summary	193
10.1	1 Definition	196
10.2	Principle 1: Concept of Graph	197
10.3	What is a Super Step?	199
10.4	3 Types of Checkpointers	200
10.5	4 Detailed Checkpoint Example	201
10.6	Four Benefits of Persistence	204
10.7	1 Benefit 1: Short-Term Memory	205
10.8	3 Benefit 3: Human-in-the-Loop (HITL)	207
10.9	Four Benefits Summary	211
10.10	Conclusion	212
11.1	Architecture Overview	214
11.2	1 Chat Message Component	217
11.3	2 Chat Input Component	218
11.4	Building a Basic Copy-Cat Chatbot	219
11.5	Building a Basic Copy-Cat Chatbot	219
11.6	Summary	227
11.7	Summary	228
12.1	Visual Comparison: With vs Without Streaming	230
12.2	Interleave UI Updates (For AI Agents)	232
12.3	The Core Mechanism	233
12.4	Step-by-Step Breakdown	235
12.5	When to Use Which Mode	236
12.6	Architecture Overview	241

12.7 Further Learning	242
13.1 User Experience Flow	244
13.2 Component Interaction	246
13.3 Complete Frontend Code Structure	252
13.4 Complete Frontend Code Structure	253
13.5 Session State in Streamlit	255
13.6 Summary	258
14.1 The Evolution of the Chatbot	260
14.2 The Core Problem	260
14.3 Types of Checkpointers Available	261
14.4 Application Flow with Database	269
14.5 Potential Improvements	273
15.1 The Problem Context	276
15.2 The Problem Context	279
15.3 Traditional Software vs LLM Systems	280
15.4 The Black Box Problem	280
15.5 Breaking Down the Definition	281
15.6 What LangSmith Does	282
15.7 Visual Hierarchy	285
15.8 Understanding the Magic	288
15.9 Introduction to RAG and LangSmith Integration	294
15.10 The Debugging Dilemma in Production	296
15.11 Understanding the Chain Architecture	299
15.12 What LangSmith Traces for Agents	305
15.13 Application Overview	312
15.14 What You See in LangSmith Dashboard	314
15.15 Alerting System	316
15.16 The Problem with LLM Behavior	317
15.17 Evaluation Approaches in LangSmith	318
15.18 Conclusion	325
16.1 What Gets Traced?	328
16.2 Recommended Prior Viewing	329
16.3 Step 2: Generate API Key	330
16.4 Hierarchical Organization	331
16.5 Additional Resources	338
17.1 How Tool Calling Works	342
17.2 ToolNode as Tool Executor	342
17.3 Decision Logic	343
17.4 1 Simple Workflow (Without Tools)	344
17.5 2 Enhanced Workflow (With Tools)	345
17.6 Problem with Direct Tool $\rightarrow$ END Flow	346
17.7 The Correct Pattern: Feedback Loop	347
17.8 Multi-Step Tool Usage Flow	348
17.9 8 Visual Representation of Compiled Graph	353
17.10 Architecture Diagram Summary	358
17.11 Glossary	360
18.1 The Brittleness Problem	363

18.2	Core Principle: Separation of Concerns	364
18.3	Complete Architecture	370
19.1	Evolution of the Chatbot Project	373
19.2	Outdated Knowledge (Knowledge Cutoff)	374
19.3	Complete RAG Pipeline	377
19.4	Using LangSmith for Tracing	385
19.5	Best Practices	388
20.1	3 Visual Representation	391
20.2	2 Two Primary Reasons for HITL	391
20.3	1 Comprehensive Benefits Overview	392
20.4	1 Pattern Overview	393
20.5	2 The interrupt() Function	396
20.6	1 Complete Workflow Diagram	397
20.7	3 State Flow During HITL	399
20.8	3 Graph Visualization	402
20.9	3 Complete Code	405
21.1	Plan of Action	407
21.2	Visual Representation	408
21.3	Development Workflow	410
21.4	Two Ways to Implement Subgraphs	413
21.5	Architecture	414
21.6	Architecture	415
21.7	Use Case	415
21.8	Architecture Design	416
21.9	Architecture Patterns Summary	424
21.10	Additional Study Resources	428
22.1	2 Problem 2: Context Window Overflow	435
22.2	How Long-Term Memory Works — High-Level Architecture	437
22.3	Complete Memory Architecture — End-to-End Flow	439
23.1	Without Short-Term Memory (Code Example)	445
23.2	Persistence Using PostgreSQL	447
23.3	Context Overflow Problem	450
23.4	Deletion Workflow	452
23.5	Workflow Structure	455
24.1	Class Hierarchy	460
24.2	How Semantic Search Works Internally	464
24.3	Workflow Diagram	465
24.4	Workflow Diagram	467
24.5	Workflow Diagram	470
24.6	Execution Flow for a Single Message	471
24.7	Summary — Complete Mental Model	474
25.1	Node-by-Node Breakdown	478
25.2	Complete Schema Relationship	493
25.3	Full Architecture — All Four Stages Combined	496

List of Tables

1	What Makes This Playlist Special?	xxv
2	Curriculum Summary Table	xxviii
3	Module 1: Foundations of Agentic AI	xxviii
4	Module 2: LangGraph Fundamentals	xxix
5	Module 3: Advanced LangGraph Concepts	xxix
6	Module 4: AI Agents	xxx
7	Module 5: Agentic RAG	xxx
8	Module 6: Productionization	xxxi
9	Python (Intermediate Level)	xxxi
10	Timeline Calculation	xxxii
11	Glossary of Terms	xxxiii
1.1	Popular Examples	1
1.2	Tools Integrated	6
1.3	Final Comparison	9
1.4	Problems Solved at Each Stage	12
2.1	Key Difference from Generative AI	15
2.2	Summary Table	23
3.1	LangChain Strengths Summary	37
3.2	Complexity Indicators	43
3.3	Dynamic State Changes	47
3.4	State Management Comparison	52
3.5	Technical Implementation Comparison	59
3.6	Observability Comparison	73
3.7	Complementary Roles	78
4.1	Key Characteristics of a LangGraph Graph	84
4.2	Advanced Features	84
4.3	Key Properties of LLM Workflows	85
4.4	Workflow Structure Types	85
4.5	Real-World Example: Customer Support Chatbot	87
4.6	Real-World Example: YouTube Content Moderation	87
4.7	Key Difference from Parallelization	89
4.8	Real-World Examples	91
4.9	Nodes	93
4.10	Edges	93
4.11	Scoring System	93
4.12	Types of Flow Expressed Through Edges	95
4.13	Characteristics of State	95
4.14	The Reducer Solution	99
4.15	Key Takeaways on Reducers	100
4.16	Phase 1: Graph Definition	100
4.17	Key Execution Concepts Summary	102
4.18	LangGraph at a Glance	103
4.19	Common Workflow Patterns Quick Reference	103
4.20	State Management Essentials	104

7.1	Key Differences: Parallel vs Conditional Workflows	145
7.2	The Discriminant	148
7.3	1 Component Summary Table	161
8.1	Workflow Comparison Table	169
8.2	3 The Three-Component Architecture	171
8.3	2 LLM Configuration	173
8.4	3 State Variables Explained	173
8.5	2 Node 2: Evaluate Tweet	175
8.6	2 The TweetEvaluation Schema	177
8.7	1 Core Concepts Summary	183
10.1	Principle 2: Concept of State	196
10.2	Example: State Tracking Through Execution	198
10.3	What is a Super Step?	199
10.4	3 Types of Checkpointers	200
10.5	4 Detailed Checkpoint Example	201
10.6	2 Thread ID Mechanism	202
10.7	3.2 getstatehistory() - Get All Intermediate States	204
10.8	3.4 Summary of API Methods	204
10.9	4 Benefit 4: Time Travel (Debugging)	206
10.10	1 Checkpointer Selection	209
10.11	Key Concepts	210
11.1	Understanding the Problem	213
11.2	Prerequisites	214
11.3	1 Chat Message Component	216
11.4	Understanding Streamlit's Execution Model	220
11.5	Summary	226
12.1	Two Ways of Response Delivery	229
12.2	Better UX for Long Outputs (Code, Documents)	231
12.3	Key Differences	233
12.4	Stream Modes in LangGraph	234
12.5	Streamlit's Chat Elements	236
12.6	Quick Reference Card	239
12.7	Common Mistakes to Avoid	239
13.1	Data Flow Overview	245
13.2	Complete Click Handler Implementation	250
13.3	Backend Code (langgraph_backend.py)	251
13.4	Message Types in LangChain	256
14.1	The Evolution of the Chatbot	259
14.2	Types of Checkpointers Available	261
14.3	Code Comparison: Before vs After	263
14.4	What Changes in Frontend?	265
14.5	What You'll See in the Database	267
14.6	Understanding Checkpoint Count	268
14.7	When to Use Each Checkpointer	272
15.1	Prerequisites	275
15.2	The Latency Problem Emerges	276

15.3 The Cost Problem Emerges	277
15.4 The Problem Context	278
15.5 The Hallucination Problem Emerges	278
15.6 The Hallucination Problem Emerges	278
15.7 Why This Matters for Debugging	280
15.8 Observability in LLM Context	281
15.9 Key Capabilities	282
15.10 Detailed Breakdown	283
15.11 Project	283
15.12 Trace	284
15.13 Run	284
15.14 Relationship Summary	285
15.15 Step 3: Install Dependencies	286
15.16 Configuration Parameters Explained	287
15.17 Metrics Summary	289
15.18 Understanding Custom Configuration	290
15.19 Key Observations	291
15.20 Project Organization	292
15.21 LangSmith Dashboard Navigation	293
15.22 Why RAG Sounds Simple But Is Complex in Production	295
15.23 What LangSmith Traces Automatically	296
15.24 Issues with Basic Implementation	298
15.25 The Latency Problem	301
15.26 Performance Comparison	301
15.27 When Index is Rebuilt (Slow Path)	303
15.28 Practical Use Cases for Tags and Metadata	304
15.29 What LangSmith Traces for Agents	305
15.30 Benefits of Agent Tracing	306
15.31 Summary of Key Concepts	309
15.32 Key Characteristics	310
15.33 What LangSmith Captures	311
15.34 Key Insight: Structured vs Regular LLM	315
15.35 What Information is Available Per Node	315
15.36 Key Metrics Tracked	316
15.37 Alerting System	316
15.38 Monitoring Dashboard in LangSmith	316
15.39 The Problem with LLM Behavior	318
15.40 Evaluation Types	319
15.41 RAG Chatbot Evaluation Example	319
15.42 Pre-built Evaluators in LangSmith	319
15.43 Playground Features	320
15.44 Prompt Versioning	320
15.45 Example: Customer Chatbot Dataset	321
15.46 Dataset Features	321
15.47 Feedback Types	321
15.48 Collaboration Features	322
15.49 LLM Ops Definition	323
15.50 LangSmith's Role in LLM Ops	323
15.51 LangSmith Navigation Quick Reference	324
16.1 Journey Till Now (Playlist Recap)	327
16.2 Key Benefits	328

16.3	Environment Variables Explained	330
16.4	Terminology	331
16.5	What Gets Captured in Each Trace	332
16.6	Understanding the Changes	333
16.7	Traces View vs Threads View	336
16.8	Trace Details Information	336
16.9	Production Use Cases	336
16.10	LangSmith URLs	337
17.1	Example Scenarios	340
17.2	Key Characteristics of ToolNode	340
17.3	Problem with Direct Tool $\rightarrow$ END Flow	347
17.4	Comparison Table	349
17.5	1 Required Imports	350
17.6	7 Building the Graph	352
17.7	4 Internet Search Flow	354
17.8	2 Status Container for Tool Usage	356
17.9	Best Practices	357
17.10	Glossary	359
18.1	Progress So Far: 18 Videos	361
18.2	Key Concept	361
18.3	Maintenance Nightmare	362
18.4	Key Benefits	364
18.5	Key Changes for Async	365
18.6	Transport Types	368
18.7	Usage Examples	369
18.8	MCP Advantages	369
19.1	Evolution of the Chatbot Project	374
19.2	Summary of RAG Benefits	375
19.3	The Context Window Limitation	376
19.4	Similarity Search	378
19.5	Step 3: Split into Chunks	380
19.6	Step 5: Create the Retriever	380
19.7	Example Trace Analysis	386
19.8	Best Practices	387
20.1	4 Real-World Analogy	389
20.2	Reason 1: To Help Agentic Systems	390
20.3	2 Detailed Benefits Table	392
20.4	Pattern 1: Action Approval Pattern (Most Common)	393
20.5	Pattern 3: Ambiguity Clarification Pattern	394
20.6	1 Essential Components	395
20.7	4 Checkpointer (Memory Saver)	396
21.1	Comparison: Two Methods	414
22.1	4 Applying This to LLMs	430
22.2	2 In-Context Learning (ICL)	432
22.3	3 Procedural Memory — “How to Do Things”	437
22.4	Step 2: Storage	438
22.5	Tools and Libraries for Memory Management	440

22.6 Summary — Complete Memory Landscape	441
23.1 1 Checkpointer	444
23.2 Persistence Using PostgreSQL	447
23.3 Persistent Memory with PostgresSaver (Code Example)	449
23.4 Trimming — Visual Walkthrough	451
23.5 Why Summarization is Better Than Trimming	454
23.6 The Summarize Node	456
23.7 Testing the Summarization Workflow	457
23.8 Comparison Table — Trimming vs Deletion vs Summarization	458
24.1 How Long Term Memory Differs from Short Term Memory	459
24.2 Namespace Examples	461
24.3 Step 3: Create Memories (put)	462
24.4 Key Concept: Where Do config and store Come From?	466
24.5 Before vs After De-Duplication	469
24.6 Two Separate LLMs	470
24.7 InMemoryStore vs PostgresStore	473
25.1 Planning Agent vs Direct Execution	476
25.2 Building the Graph	481
25.3 Router Node	483
25.4 State at Each Stage	485
25.5 All Pydantic Schemas in One Place	486
25.6 What Changes and What Stays the Same	489
25.7 GUI Features	495
25.8 What Each Stage Added	495
25.9 Technology Stack	496

Agentic AI using LangGraph - Complete Playlist Introduction

- Understand the core ideas behind agentic workflows
- Use LangGraph to define stateful agents and tasks
- Coordinate multiple tools, APIs, and LLM calls
- Handle memory, branching, and real-time decision logic
- Build real applications like assistants, planners, and autonomous agents.

Introduction

This document provides a comprehensive overview of the **Agentic AI using LangGraph** playlist. The playlist represents a carefully planned educational journey designed after approximately 3-4 months of intensive research, curriculum development, and documentation study.

The instructor emphasizes that this playlist is designed to be **comprehensive and end-to-end**, meaning learners who complete the entire playlist will gain complete knowledge of building agentic applications using LangGraph.

What Makes This Playlist Special?

The playlist fills a gap in existing educational content. Upon researching available content on YouTube, two types of existing materials were identified:

Table 1. *What Makes This Playlist Special?*

Content Type	Description	Limitation
Project-Based	Directly teaches building projects using LangGraph	Fundamentals are not properly discussed
Fundamentals-Based	Teaches basic LangGraph concepts	Too short, ends quickly without depth

This playlist aims to combine both approaches—providing **strong fundamentals** while also teaching how to **build real-world AI agents and workflows**.

Why This Playlist?

Three primary reasons drive the creation of this playlist:

1. Timing

Current State of Technology (2024-2025):

Agentic AI is everywhere—on YouTube, Twitter, Instagram, and across all major platforms. The term is being discussed and promoted by:

- Major technology companies
- Thought leaders in the AI industry
- CEOs of large corporations
- People in powerful positions in the tech world

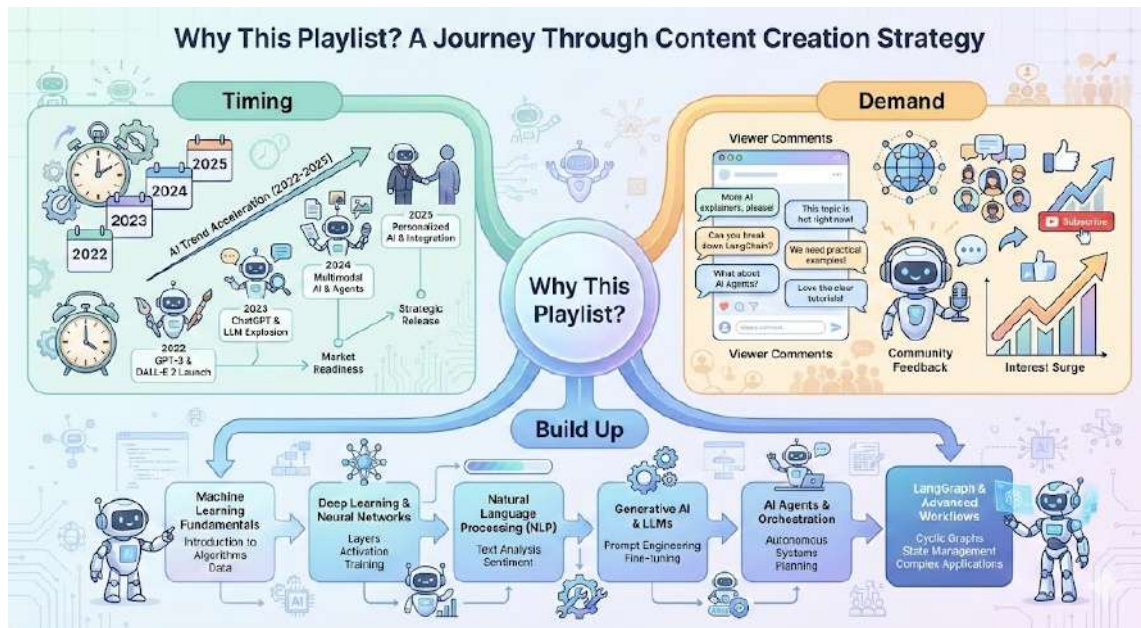


Figure 1. Why This Playlist?

Why Timing Matters:

The evolution of AI has followed a clear trajectory:

- **2022:** ChatGPT was released, starting a new era of Generative AI in Computer Science
- **Present:** Generative AI tools have matured significantly
- **Future (Next 5 Years):** Powerful AI agents can now be built using these mature tools

Key Insight: Those who learn to build agentic applications now will be in a valuable position in the future as AI agents are expected to create enormous value.

2. Demand

Over the past 3-4 months, there has been overwhelming demand from the community:

- Every third comment on the channel requested LangGraph content
- Viewers consistently asked for a comprehensive playlist on this topic
- The demand reflected the industry's increasing focus on agentic AI

3. Build Up

The channel has followed a **sequential, organized learning approach**:



Figure 2. Build Up

This progression means learners are now ready to understand LangGraph and how to build AI agents.

Vision Behind the Playlist

The playlist is designed with three specific, actionable goals:



Figure 3. Vision Behind the Playlist

Goal 1: Simple to Understand

Create a playlist so simple that:

- Even complete beginners can follow along
- Anyone can learn to build agentic applications easily
- No learner faces difficulty in building any type of AI agent

Goal 2: Mastering LangGraph Fundamentals

Teach LangGraph fundamentals in such a way that:

- Learners gain a **strong command** over LangGraph
- Every concept is thoroughly understood
- Practical implementation is clear

Goal 3: Transferable Fundamentals

Provide such deep conceptual understanding that:

- If LangGraph is replaced by a new framework in the future
- Learners can easily adapt and learn that new framework
- The core concepts remain applicable regardless of the tool

***Important Note:** The AI ecosystem is evolving rapidly. New tools emerge and old ones become obsolete. Having **framework-agnostic conceptual understanding** ensures long-term value.*

Curriculum Overview

The curriculum is divided into **6 comprehensive modules**. The structure is designed to progressively build knowledge from foundational concepts to production-ready applications.

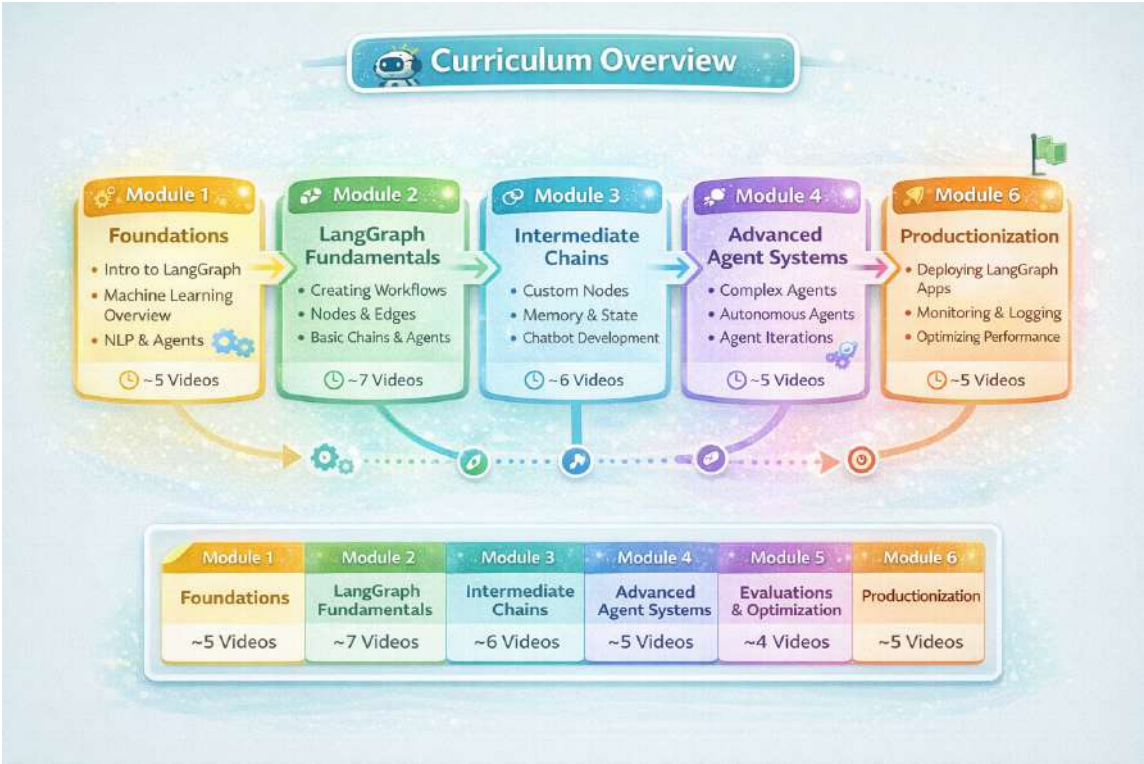


Figure 4. Curriculum Overview

Curriculum Summary Table

Table 2. Curriculum Summary Table

Module	Name	Focus Area	Estimated Videos
1	Foundations of Agentic AI	Conceptual understanding, terminology	5-6 videos
2	LangGraph Fundamentals	Core concepts, basic workflows	Multiple videos
3	Advanced LangGraph	Industry-grade features	Multiple videos
4	AI Agents	Design patterns, agent types	Multiple videos
5	Agentic RAG	Advanced RAG architectures	Multiple videos
6	Productionization	Deployment, debugging, observability	Multiple videos

Disclaimer: The curriculum is tentative and may change. Agentic AI is rapidly evolving, so some topics might be added or removed based on industry developments.

Module-wise Detailed Breakdown

Module 1: Foundations of Agentic AI

Duration: First 5-6 videos of the playlist
Objective: Provide an in-depth overview of Agentic AI and related terminology.
Topics Covered:

Table 3. Module 1: Foundations of Agentic AI

Topic	Description
What is Agentic AI?	Core definition and understanding

Topic	Description
Agentic AI vs AI Agent	Key differences between the terms
Agentic AI vs Generative AI	How they relate and differ
What is Agentic RAG?	Introduction to agentic retrieval systems
Traditional RAG vs Agentic RAG	Comparison and evolution
Top Frameworks	Overview of frameworks for building agentic applications

Outcome: After this module, learners will have a solid high-level understanding of what will be covered throughout the playlist.

Module 2: LangGraph Fundamentals

Objective: Start the LangGraph journey with core fundamentals.

Topics Covered:

Table 4. Module 2: LangGraph Fundamentals

Concept	Description
Graph Building	How to construct graphs in LangGraph
State	The concept of state management
Nodes	Understanding what nodes are and how they work
Edges	Connecting nodes with edges
Conditional Edges	Implementing conditional routing
Looping	How to implement loops in LangGraph

Practical Application: Using these fundamental concepts to build popular AI workflows.

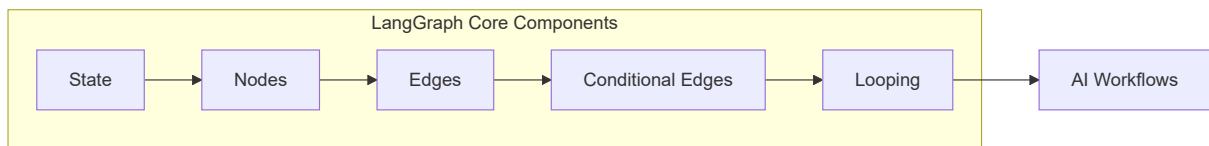


Figure 5. Module 2: LangGraph Fundamentals

Module 3: Advanced LangGraph Concepts

Objective: Learn advanced features to build industry-grade AI agents.

When to Start: After completing fundamentals and building basic AI workflows with confidence.

Advanced Concepts:

Table 5. Module 3: Advanced LangGraph Concepts

Concept	Purpose
Persistence	Maintaining state across sessions
Memory	Enabling agents to remember past interactions
Human-in-the-Loop	Allowing human intervention in agent workflows
Breakpoints	Pausing execution at specific points
Checkpointers	Saving and restoring agent state
Time Travel	Going back to previous states in execution

Outcome: Ability to implement these advanced features to make AI agents truly **industry-grade**.

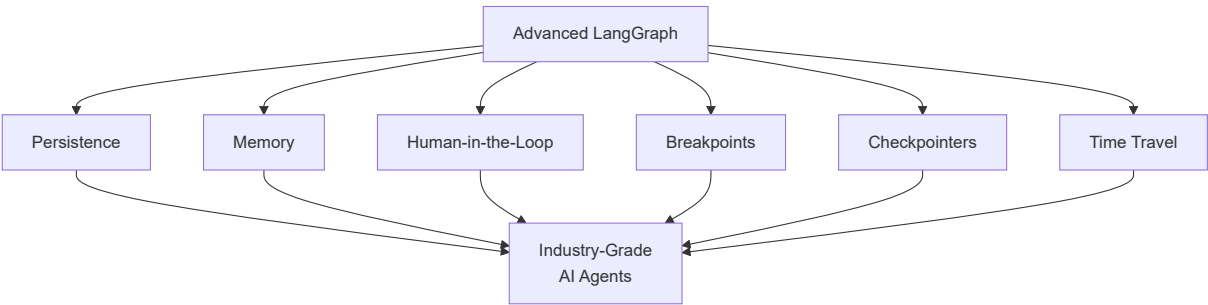


Figure 6. Module 3: Advanced LangGraph Concepts

Module 4: AI Agents

Objective: Build different types of AI agents using the knowledge gained.

Module Structure:

- 1. Theoretical Understanding
 - What are AI Agents?
 - Popular design patterns used in the industry today
- 2. Practical Implementation of various agent types:

Table 6. Module 4: AI Agents

Agent Type/Pattern	Description
ReAct Agent	Reasoning + Acting pattern for problem-solving
Reflection	Agents that can reflect on their own outputs
Self-Ask with Search	Pattern for self-questioning with external search
Planning	Agents that plan before executing
Multi-Agent Systems	Systems with multiple cooperating agents

Outcome: After completing this module, learners will be able to build **any type of AI agent**.

Module 5: Agentic RAG

Objective: Learn to build advanced RAG applications using agentic approaches.

Background: Traditional RAG was covered in the LangChain playlist. This module covers the **advanced/evolved version** that merges AI agents with RAG concepts.

Architectures Covered:

Table 7. Module 5: Agentic RAG

Architecture	Description
CRAG (Corrective RAG)	RAG with self-correction capabilities
Self-RAG	RAG that can evaluate and improve its own retrieval
Other Architectures	Various advanced RAG patterns

Key Concept: Agentic RAG = AI Agent + RAG (merged together)

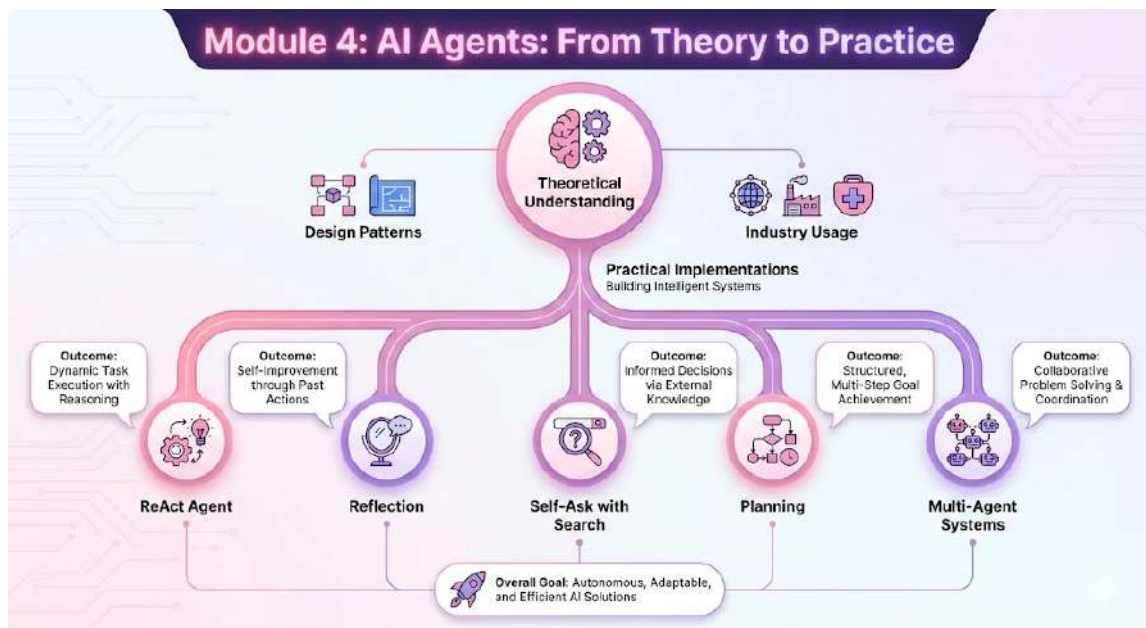


Figure 7. Module 4: AI Agents

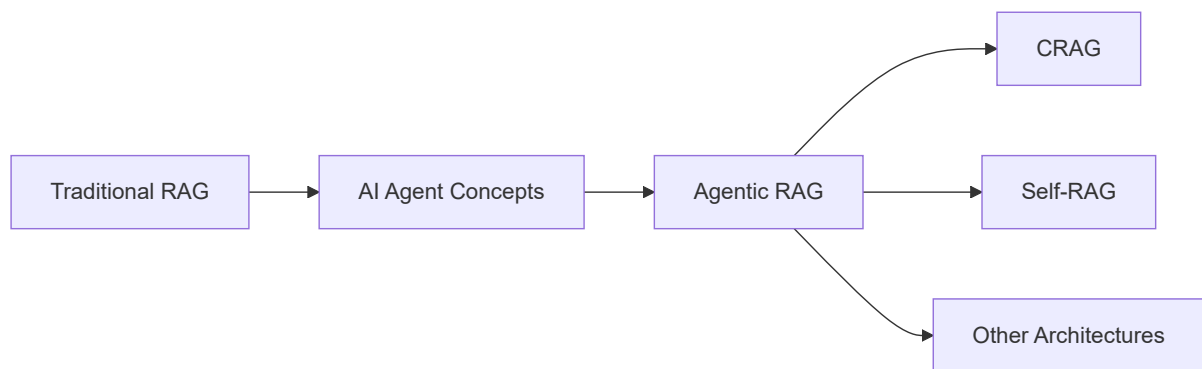


Figure 8. Module 5: Agentic RAG

Module 6: Productionization

Objective: Build a complete, production-ready project using all knowledge from the playlist.

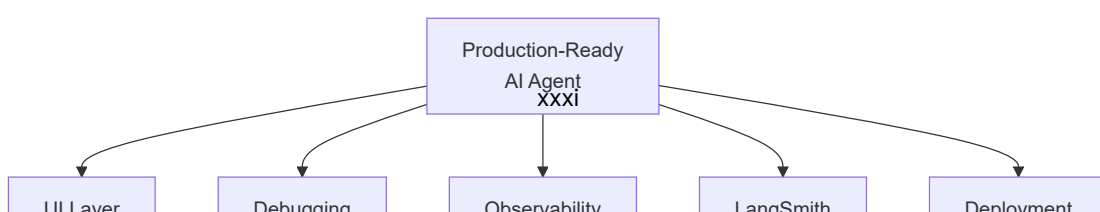
Goal: Create a project that can be:

- Added to your resume
- Showcased in interviews
- Demonstrated to potential employers

Topics Covered:

Table 8. Module 6: Productionization

Topic	Description
UI Development	Providing a user interface for the agent
Debugging Support	Adding debugging capabilities
Observability	Monitoring and understanding agent behavior
LangSmith Integration	Using LangSmith for tracing and evaluation
Deployment	Deploying the agent to production



2. Familiarity with LLMs

Requirement: Some experience working with Large Language Models.

What This Means:

- Understanding how to interact with LLMs
- Basic knowledge of prompting
- Experience with LLM APIs (OpenAI, etc.)

Recommendation: The instructor's LangChain playlist covers this.

3. LangChain

Critical Requirement: LangGraph is built **on top of LangChain**.

Implications:

- Every code written in this playlist will have some LangChain dependency
- Without LangChain knowledge, the content will be very difficult to follow
- The playlist will “go over your head” without this foundation

Strong Recommendation: Watch the LangChain playlist first (approximately 18 videos, somewhat lengthy but essential).

Frequently Asked Questions

Q1: How many videos will be in this playlist?

Answer: Approximately **35 to 50 videos**.

Note: An exact count cannot be provided because:

- Agentic AI is evolving rapidly
- LangGraph itself is undergoing many changes
- The curriculum may need to adapt to new developments

Q2: What will be the video upload frequency?

Answer: Target is **3 videos per week**.

Details:

- Maximum effort will be made to maintain this schedule
- If fewer videos are uploaded, there will be a valid reason (personal life circumstances)
- The instructor is committed to maintaining consistency

Timeline Calculation

Table 10. *Timeline Calculation*

Scenario	Videos	Duration at 3/week
Minimum	35 videos	~12 weeks (3 months)
Maximum	50 videos	~17 weeks (4+ months)

Key Takeaways

Why Learn Agentic AI Now?

1. **Industry Momentum:** All major companies and thought leaders are investing in this space
2. **Career Value:** Early expertise in agentic AI will be highly valuable in the future
3. **Technology Maturity:** Generative AI tools have matured enough to build powerful agents
4. **Next 5 Years:** AI agents are expected to create tremendous value and potentially transform the world

What Makes This Playlist Different?

1. **Comprehensive Coverage:** 35-50 videos covering everything from basics to production
2. **Organized Structure:** 6 well-planned modules with logical progression
3. **Conceptual Depth:** Focus on understanding, not just implementation
4. **Future-Proof Learning:** Framework-agnostic principles that remain valuable even if tools change

Final Notes

Instructor's Commitment:

- Personal promise to create the **best playlist on LangGraph**
- Will dedicate 100% effort over the next 3-4 months
- Excited about the potential impact on learners

Learner's Action Items:

1. Ensure prerequisites are met (Python, LLMs, LangChain)
2. Follow the playlist sequentially
3. Practice building agents alongside videos
4. Provide feedback for curriculum improvements

Glossary of Terms

Table 11. *Glossary of Terms*

Term	Definition
Agentic AI	AI systems that can act autonomously, make decisions, and perform tasks with minimal human intervention
AI Agent	A specific implementation of an autonomous AI system that can perceive, decide, and act
LangGraph	A framework built on top of LangChain for building stateful, multi-step AI applications
RAG (Retrieval Augmented Generation)	A technique that combines retrieval of relevant documents with LLM generation
Agentic RAG	Advanced RAG systems that incorporate agent-like behaviors for improved retrieval and generation
ReAct	A prompting paradigm that combines reasoning and acting in language models
Multi-Agent Systems	Systems where multiple AI agents work together to accomplish complex tasks
Human-in-the-Loop	Design pattern where human oversight or intervention is incorporated into AI workflows

Term	Definition
Persistence	The ability to maintain state across multiple interactions or sessions
Checkpointers	Mechanisms for saving and restoring the state of an agent during execution

CX

Agentic AI using LangGraph

Complete Playlist • 35-50 Videos • by CampusX

Build Stateful Agents • Multi-Tool Coordination • Real-time Decision Logic

LEARNING OBJECTIVES

Understand Agentic Workflows • Define Stateful Agents • Coordinate Tools & APIs • Handle Memory & Branching • Build Assistants & Planners

WHY NOW?

- 2024-2025: Agentic AI is everywhere
- Major tech companies investing heavily
- GenAI tools have matured significantly
- Next 5 years: AI agents create huge value

"Early expertise = valuable career position"

LEARNING PATH

ML → Deep Learning → LangChain

↓

LangGraph

PLAYLIST GOALS

- 1 Simple to Understand
- 2 Master LangGraph Fundamentals
- 3 Transferable Knowledge

Framework agnostic concepts for long-term value

6 COMPREHENSIVE MODULES

1 Foundations of Agentic AI

5/6 Videos • Conceptual Understanding

- What is Agentic AI?
- Agentic AI vs AI Agent
- Agentic AI vs Generative AI
- Traditional RAG vs Agentic RAG
- Top Frameworks Overview

2 LangGraph Fundamentals

Core Concepts • Basic Workflows

- Graph Building
- State Management
- Nodes & Edges
- Conditional Edges
- Looping Mechanisms

3 Advanced LangGraph

Industry-Grade Features

- Persistence & Memory
- Human-in-the-Loop
- Breakpoints
- Checkpointers
- Time Travel

4 AI Agents

Design Patterns • Agent Types

- ReAct Agent Pattern
- Reflection Agents
- Self-Ask with Search
- Planning Agents
- Multi-Agent Systems

5 Agentic RAG

Advanced RAG Architectures

- CRAG (Corrective RAG)
- Self-RAG
- AI Agent • RAG Fusion
- Advanced Retrieval Patterns

Formula: Agentic RAG = AI Agent • RAG

6 Productionization

Deployment • Debugging • Observability

- UI Development
- Debugging Support
- Observability & Monitoring
- LangSmith Integration
- Resume-Ready Project

PREREQUISITES

Python (Intermediate)

- Object-Oriented Programming
- Typing Module
- Pydantic
- AsyncIO

⚠ Basic Python is NOT sufficient

LLM Familiarity

- Understanding LLM Interactions
- Basic prompting knowledge
- Experience with LLM APIs

OpenAI, Anthropic, etc.

LangChain (Critical)

- LangGraph is built ON TOP of LangChain
- Every code has LangChain dependency
- ~18 videos in LangChain playlist

⚠ Watch LangChain playlist first

LANGGRAPH CORE COMPONENTS

State → Nodes → Edges → Conditional → Looping → AI Workflows

TIMELINE & STATS

- 35-50 Videos Total
- 3 Videos/Week Upload Frequency
- ~3-4 Months Duration
- 6 Comprehensive Modules

KEY TAKEAWAY

"Those who learn to build agentic applications NOW will be in a valuable position as AI agents are expected to create enormous value in the next 5 years."

Start Your Agentic AI Journey with CampusX

Comprehensive • End-to-End • Industry-Ready • Future-Proof Learning

Figure 10. Glossary of Terms

xxxiv

Generative AI vs Agentic AI - Complete Notes

What Generative AI does (text, images, code) What makes Agentic AI different (goals, memory, decisions) Real-world use cases of Agentic AI How LangGraph helps you build these systems step by step

1.1 What is Generative AI?

1.1.1 Definition

Generative AI refers to a class of AI models that can create new content such as text, images, audio, code, and video that resembles human-created data.

1.1.2 Timeline

- **Started:** ~3 years ago (around 2021)
- **Impact:** Transformed the world in just 3 years
- **Current Status:** Continuously evolving and improving

1.1.3 Key Characteristics

- Creates new content across multiple modalities
 - Mimics human creativity - output feels human-made
 - Rapidly improving - getting better every year

1.1.4 Popular Examples

Table 1.1. Popular Examples

Category	Examples	Use Case
LLM Chatbots	ChatGPT, Gemini, Claude, Grok	Human-like text generation
Image Generation	DALL-E, Midjourney	Text-to-image creation
Code Generation	Code Llama	Software code writing
Text-to-Speech	ElevenLabs	Natural voice synthesis
Video Generation	Sora	Text-to-video creation

1.2 Generative AI vs Traditional AI

1.2.1 Traditional AI

- **Goal:** Find patterns in data

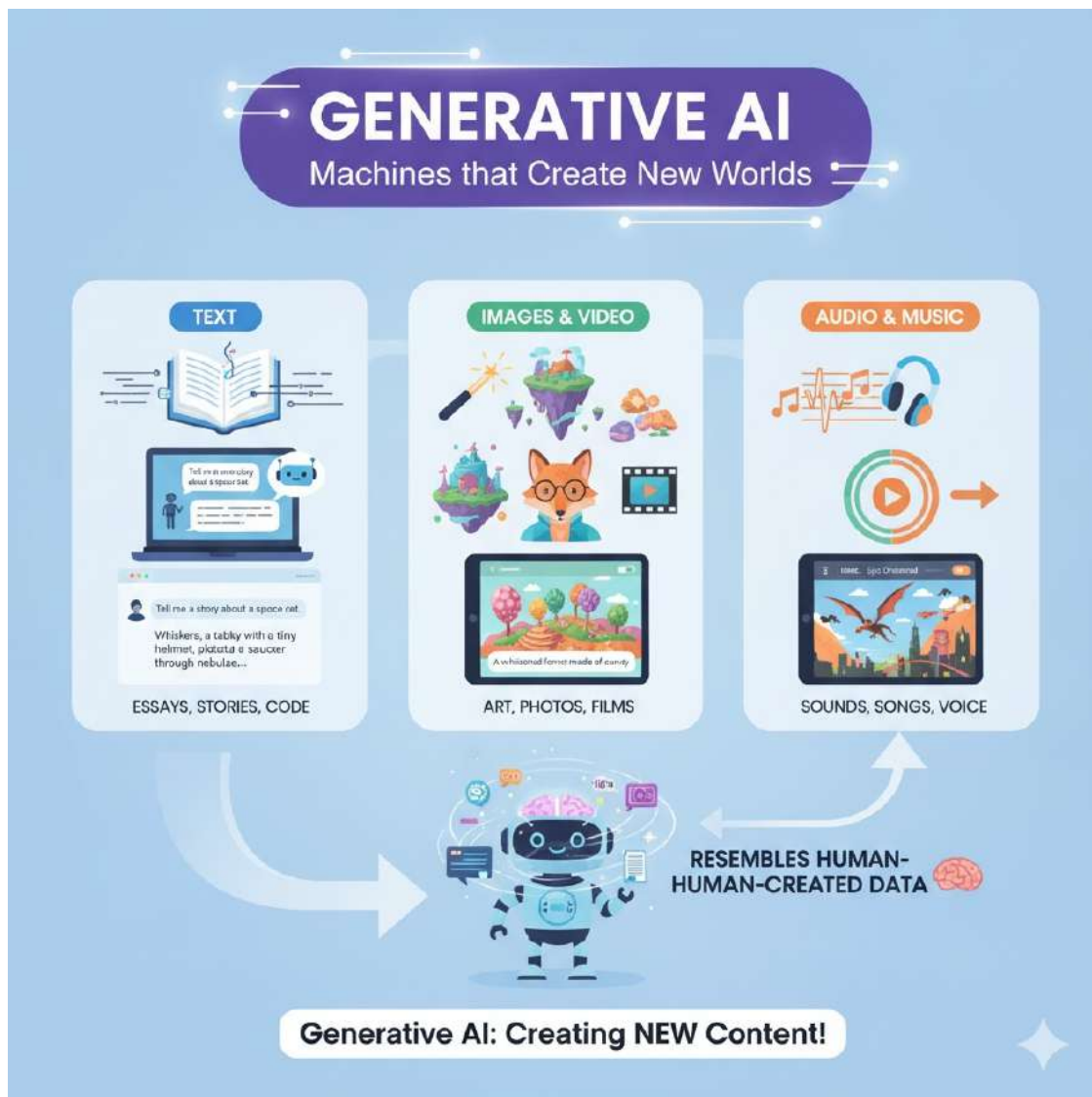


Figure 1.1. Key Characteristics

- **Approach:** Identify relationship between input and output
- **Use Cases:**
 - Classification (spam detection)
 - Medical diagnosis (cancer detection)
 - Regression (stock price prediction)
 - Weather forecasting

1.2.2 Generative AI

- **Goal:** Learn data distribution
- **Approach:** Understand the nature/characteristics of data
- **Outcome:** Generate new samples from learned distribution
- **Example:** Learn what cats look like → Generate new cat images

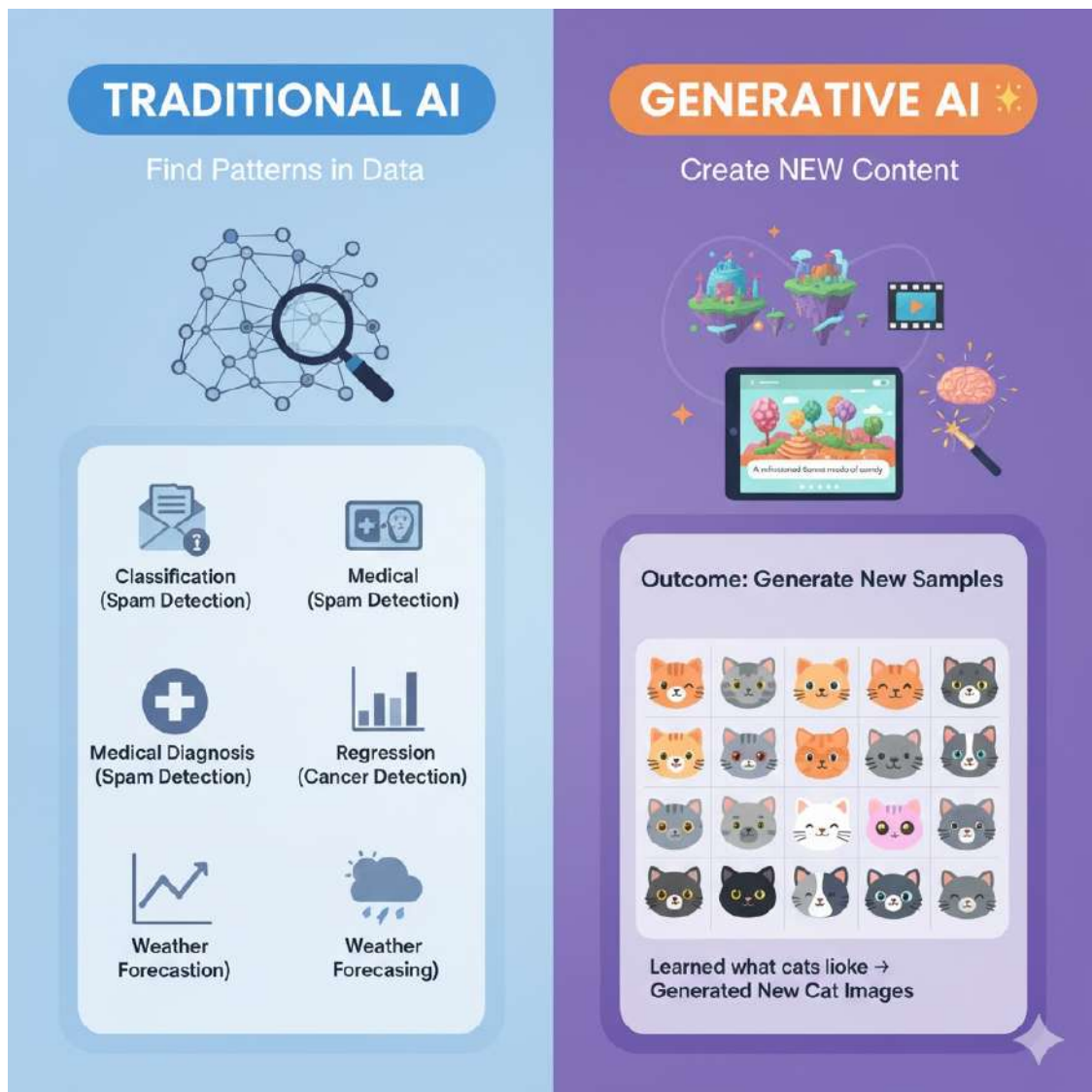


Figure 1.2. Generative AI

1.3 Applications of Generative AI

1.3.1 1. Creative & Business Writing

- Blog post generation
- Email drafting & grammar correction
- Content creation with proper tone
- **Tools:** ChatGPT, Gemini integrated in Gmail, etc.

1.3.2 2. Software Development

- Auto-completion tools
- Bug detection and debugging
- Error resolution
- **Impact:** Less manual coding required

1.3.3 3. Customer Support

- AI Chatbots for large-scale support
- Automatic query resolution
- Human escalation when needed
- **Examples:** Ola, Uber, Zomato, Swiggy chatbots

1.3.4 4. Education

- Doubt clearing from videos
- Personalized curriculum creation
- Content summarization
- Simplified explanations

1.3.5 5. Design & Graphics

- Thumbnail generation
- Infographics creation
- Advertisement video clips
- **Tools:** Sora, Runway

1.4 Practical Scenario: HR Recruitment Process

1.4.1 Problem Statement

Role: HR Recruiter **Task:** Hire a Backend Engineer

1.4.2 Steps Required

1. Draft Job Description (JD)
2. Post on job platforms (Naukri.com, LinkedIn)
3. Shortlist applications (from 1000 → 25 candidates)
4. Conduct interviews
5. Send offer letter
6. Handle onboarding

1.5 Evolution: 4 Stages of AI Implementation

1.6 Stage 1: Simple LLM Chatbot

1.6.1 Capabilities

- ✓ Draft JD based on requirements
- ✓ Suggest job platforms
- ✓ Provide generic hiring advice
- ✓ Draft email templates
- ✓ Generate interview questions
- ✓ Create offer letters

APPLICATIONS OF GENERATIVE AI



Figure 1.3. Design & Graphics

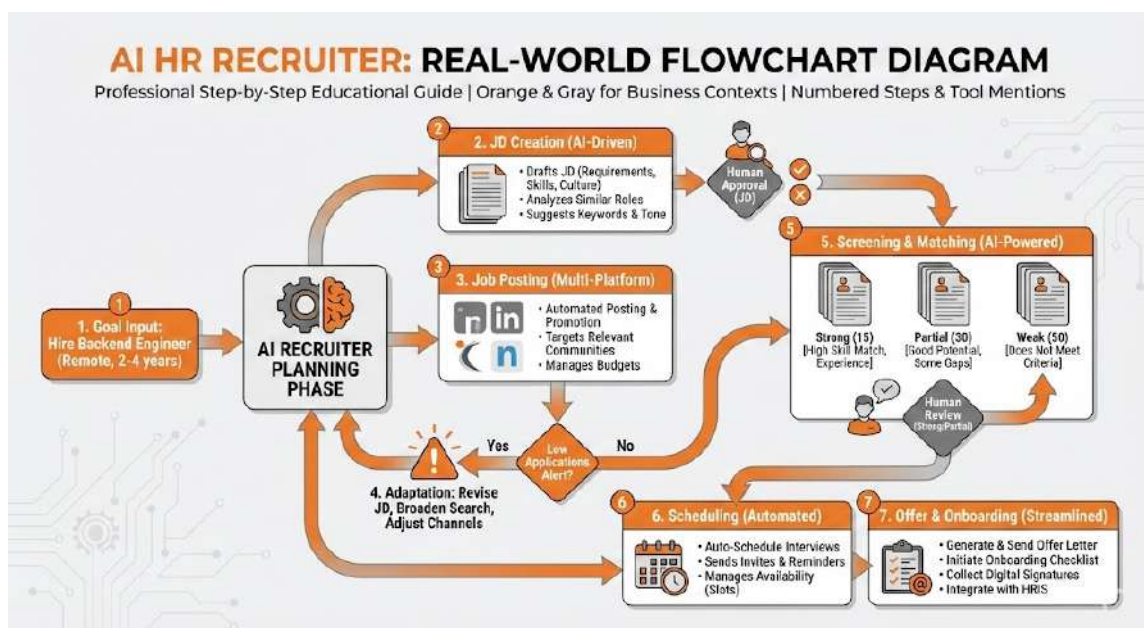


Figure 1.4. Practical Scenario: HR Recruitment Process

1.6.2 ✗ Limitations

1. **Reactive** - waits for human prompts
2. **No Memory** - forgets past conversations
3. **Generic Advice** - not company-specific
4. **No Actions** - can't execute tasks

1.7 Stage 2: RAG-Based Chatbot (Retrieval Augmented Generation)

1.7.1 Improvement: Company-Specific Knowledge

1.7.2 Documents Fed to Chatbot

- Past JD templates (high-performing ones)
- JD variations (remote vs in-office, junior vs senior)
- Hiring strategy/playbook
- Internal salary bands
- Past interview question banks
- Onboarding documents
- Email templates
- Employee policies

1.7.3 ✓ Enhanced Capabilities

- **Tailored advice** specific to company
 - **Data-driven suggestions** based on past success
 - **Company culture alignment** in responses
 - **Better recommendations** using historical data

1.7.4 ✗ Still Limited

1. Still **Reactive**
2. Still **No Memory**
3. ✓ **Generic advice** → **Specific advice**
4. Still **No Actions**

1.8 Stage 3: Tool-Augmented Chatbot

1.8.1 Tools Integrated

Table 1.2. *Tools Integrated*

Tool	Purpose
LinkedIn API	Post jobs, boost ads
Resume Parser	Extract & analyze PDF content
Calendar API	Check availability, schedule
Email API	Send & receive emails
HRMS Software	Handle onboarding process

1.8.2 New Capabilities

- ✓ **Autonomous actions** - not just suggestions
- Automatically post JDs on platforms
- Monitor application count
- Parse and analyze resumes
- Send emails automatically
- Update JD based on performance
- Boost job posts with ads
- Trigger onboarding workflows

1.8.3 X Still Missing

1. Still **Reactive**
2. Still **No Memory**
3. ✓ **Generic advice** → **Specific advice**
4. ✓ **No actions** → **Can take actions** 5. **Cannot adapt** to changing situations

1.9 Stage 4: Agentic AI

1.9.1 Key Characteristics

1. Proactive (Not Reactive)

- Takes initiative
- Understands goals automatically
- Plans the entire workflow
- Executes without constant guidance

2. Context-Aware

- **Has Memory** - remembers past interactions
- Knows what was done previously
- Understands what needs to be done next
- Maintains conversation history

3. Adaptable

- Identifies problems automatically
- Suggests solutions
- Adjusts strategy when needed
- Chooses alternate paths

1.9.2 Complete Workflow Example

Human: "I want to hire a Backend Engineer"

Agent's Autonomous Actions:

1. Planning Phase

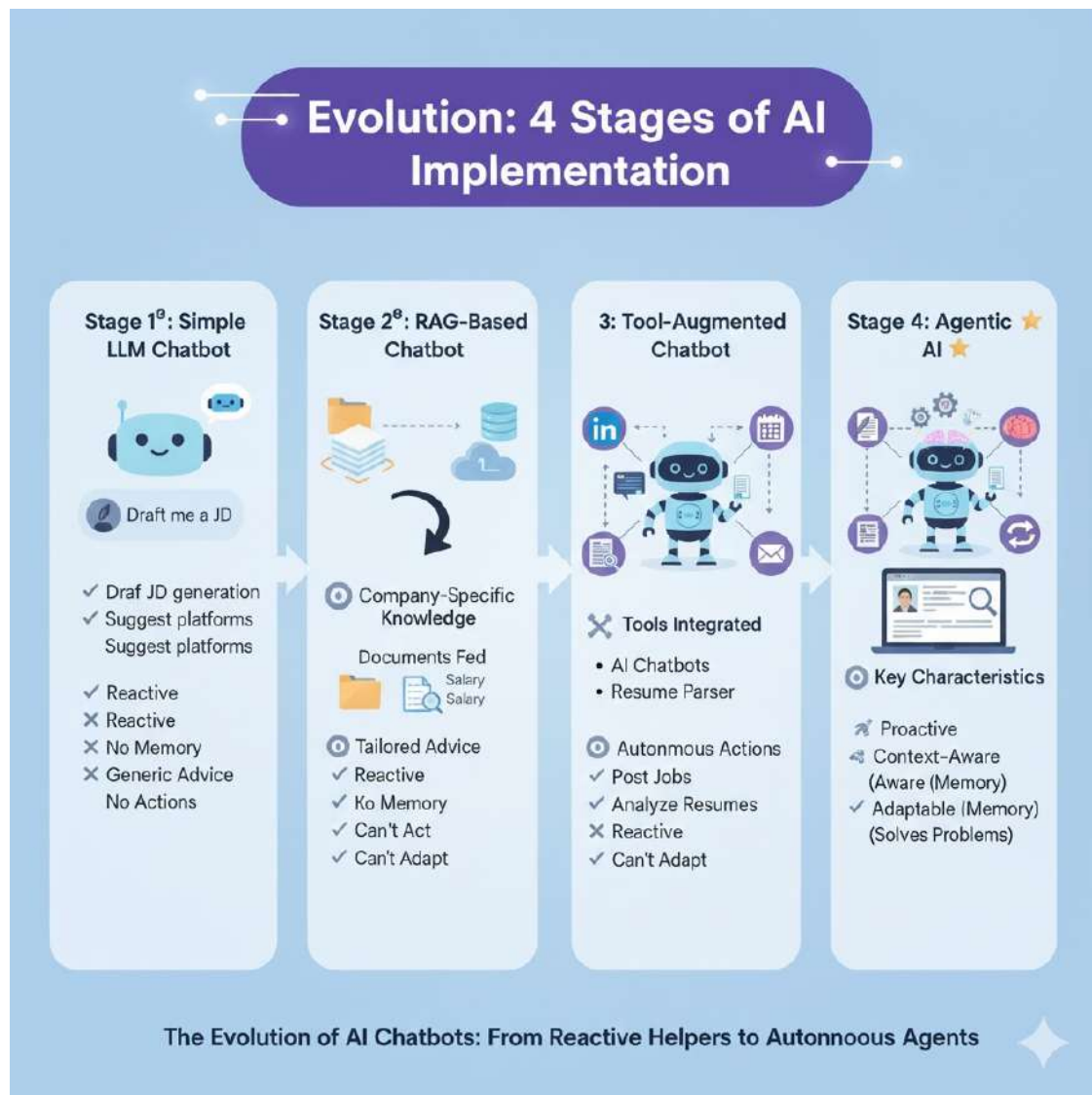


Figure 1.5. Adaptable

Understand goal
Create execution plan:

- Draft JD
- Post on platforms
- Monitor applications
- Screen candidates
- Schedule interviews
- Send offers
- Handle onboarding

2. Execution Phase

- ✓ Drafts JD (using company knowledge)
- Posts on LinkedIn & Naukri automatically
- Continuously monitors applications

3. Problem Detection

Alert: "Only 2 applications received, below expectations"

4. Adaptive Response

Suggested Actions:

- Broaden JD (Backend → Full Stack)
- Boost job on LinkedIn with ads

Waits for approval → Executes automatically

5. Smart Screening

- Downloads 8 resumes
- Analyzes using resume parser
- **Results:** 2 strong, 3 partial, 3 weak matches
- Suggests next steps

6. Intelligent Scheduling

- Checks calendar availability
- Suggests interview slots
- Drafts invitation emails
- Sends to candidates + HR

7. Interview Support

- Sends reminder on interview day
- Provides question bank
- Tracks interview completion

8. Offer Management

- Drafts offer letter
- Sends to candidate
- Tracks acceptance status

9. Onboarding Automation

When offer accepted:

- Send welcome email
- Generate employment contract
- Create official email ID
- Assign laptop
- Schedule KT sessions
- Setup intro meetings

1.10 Final Comparison

Table 1.3. *Final Comparison*

Aspect	Generative AI	Agentic AI
Focus	Content Creation	Goal Achievement
Behavior	Reactive	Proactive
Autonomy	✗ Human guides each step	✓ Self-driven after goal
Memory	✗ Forgets context	✓ Maintains context
Tools	✗ No external actions	✓ Integrates tools
Adaptability	✗ Cannot adjust	✓ Adapts to problems
Human Role	Active guidance	✓ Approval & monitoring

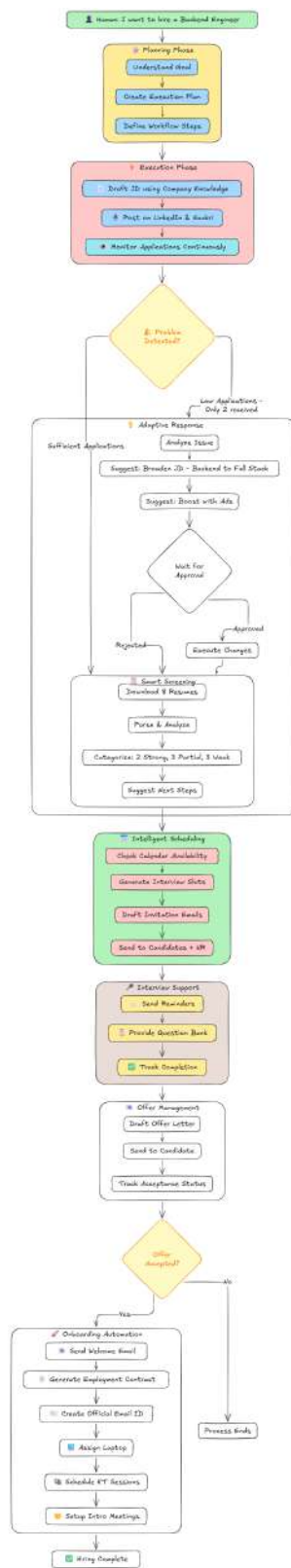


Figure 1.6. Agent's Autonomous Actions

1.11 Key Insights

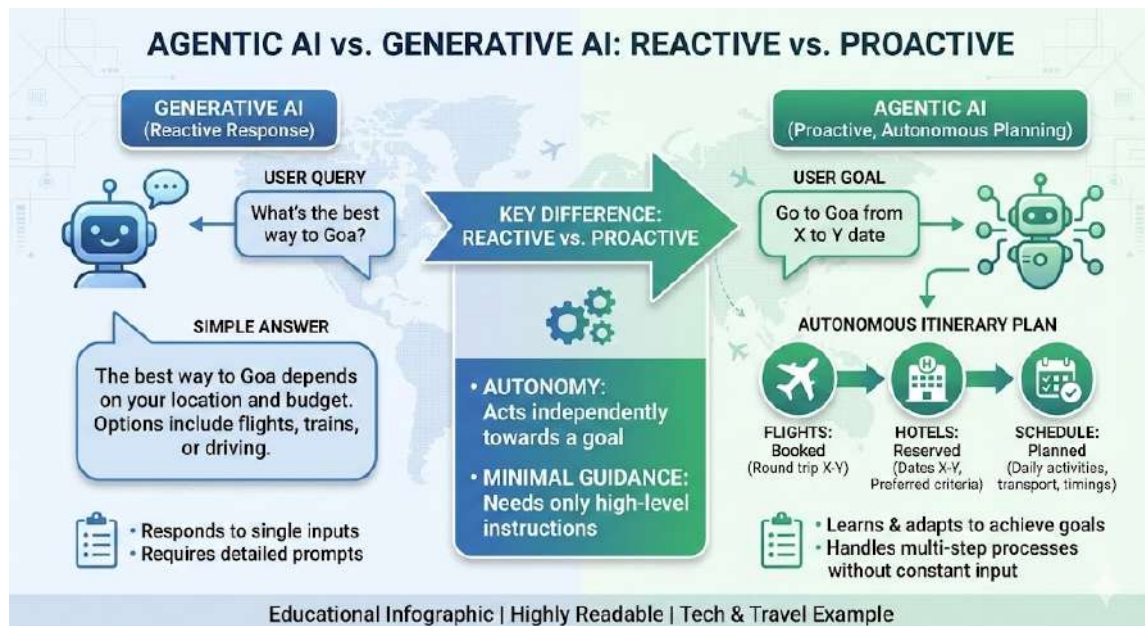


Figure 1.7. Key Insights

1.11.1 Main Differences

1. Purpose

- **GenAI:** Creates content
- **Agentic AI:** Achieves goals

2. Interaction Style

- **GenAI:** Reactive (waits for prompts)
- **Agentic AI:** Proactive (takes initiative)

3. Relationship > Generative AI is a building block of Agentic AI

Agentic AI uses:

- └ LLMs (GenAI) for planning & reasoning
- └ Tools for actions
- └ Memory for context
- └ RAG for knowledge

4. Philosophy > "Generative AI is a **capability**, > Agentic AI is a **behavior** (using multiple capabilities)"

1.12 Summary

1.12.1 Evolution Journey

1.12.2 Problems Solved at Each Stage

Table 1.4. *Problems Solved at Each Stage*

Stage	Reactive	Memory	Generic	No Actions	Adaptability
1. Simple	✗	✗	✗	✗	✗
2. RAG	✗	✗	✓	✗	✗
3. Tool	✗	✗	✓	✓	✗
4. Agentic	✓	✓	✓	✓	✓

1.13 Next Steps

Next Video: What is Agentic AI? (Formal Definition)

This Video's Purpose:

- Build deep intuition
- Understand the “why” behind Agentic AI
- See practical evolution
- Feel at home with concepts

1.14 Key Takeaway

Agentic AI represents the future where AI systems don't just assist but autonomously execute complex workflows, adapting to challenges and achieving goals with minimal human intervention.

The magic: You provide the goal → AI handles everything → You just monitor and approve!



Figure 1.8. Key Takeaway

Agentic AI - Complete Study Notes

2.1 Overview

This is the second video in the Agentic AI using LangGraph playlist, focusing on the formal definition and study of Agentic AI.

2.2 What is Agentic AI?

***Definition:** Agentic AI is a type of AI that can take up a task and goal from a user and then work towards completing it on its own with minimal human guidance. It plans, takes action, adapts to changes, and seeks help only when necessary.*

2.2.1 Key Difference from Generative AI

Table 2.1. Key Difference from Generative AI

Generative AI	Agentic AI
Reactive - Responds to specific questions Requires step-by-step guidance Example: ChatGPT answering queries	Proactive - Takes initiative Works autonomously Example: AI agent completing complex tasks

Travel Example:

- **Generative AI:** You ask “What’s the best way to reach Goa on the 15th?” → It answers only that question
- **Agentic AI:** You say “I want to go to Goa from X date to Y date” → It plans flights, hotels, itinerary automatically

2.3 Real-World Example: AI HR Recruiter

2.3.1 Scenario

An HR recruiter needs to hire a Backend Engineer using an Agentic AI chatbot.

2.3.2 How it Works:

1. Goal Setting

- User: “I want to hire a Backend Engineer (Remote, 2-4 years experience)”

2. Planning Phase

- Draft Job Description (JD)
- Post on best job platforms (LinkedIn, Naukri)

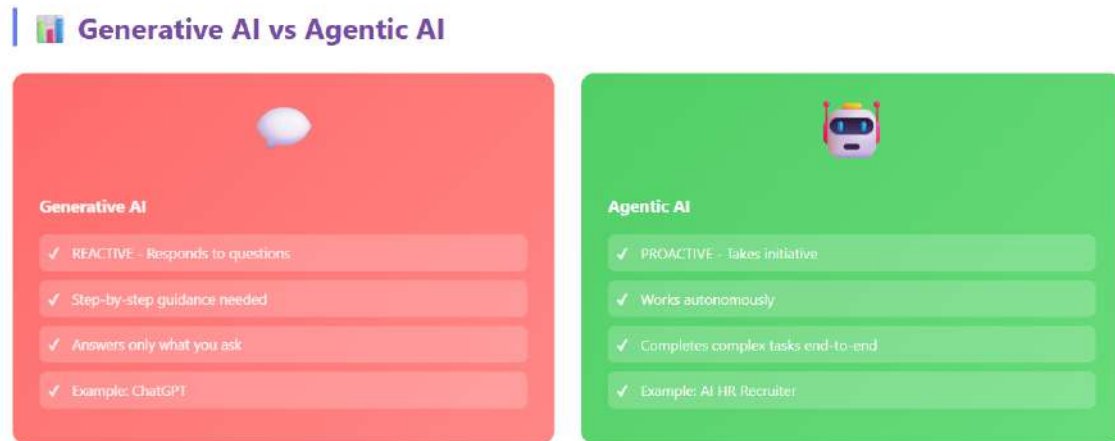


Figure 2.1. Key Difference from Generative AI

- Monitor applications constantly
- Screen candidates
- Schedule interviews
- Send offer letters
- Start onboarding process

3. Execution Phase

Step 1: JD Creation

- Accesses company documents
- Determines tech stack, salary, responsibilities
- Creates and presents JD for approval

Step 2: Job Posting

- Posts on LinkedIn and Naukri automatically
- Monitors applications continuously

Step 3: Adaptation

- Notices low applications after 3 days
- Suggests changes: Change to "Full Stack Engineer", Run LinkedIn ads
- Waits for approval and implements

Step 4: Candidate Screening

- Uses resume parser tool
- Analyzes all applicants
- Categorizes: 2 strong, 3 partial, 3 weak matches

Step 5: Interview Scheduling

- Checks HR recruiter's calendar
- Sends interview invitations
- Provides reminder and question list

Step 6: Offer & Onboarding

- Drafts offer letter from company documents
- Sends via email
- Monitors acceptance
- Initiates onboarding (IT access, laptop, welcome email)



Figure 2.2. How it Works

2.4 ★ Six Key Characteristics of Agentic AI

2.4.1 1. Autonomy

Definition: The ability to make decisions and take actions independently without step-by-step human instructions.

How it's Implemented:

- **Execution Autonomy:** Automatically executes plan steps
- **Decision Making:** Decides which candidates to shortlist
- **Tool Usage:** Selects appropriate tools for each task

Controlling Autonomy:

- **Scope Definition:** Limit which tools/actions agent can perform independently
- **Human-in-the-Loop:** Insert checkpoints requiring human approval
- **Override Controls:** Allow users to stop, pause, or change behavior
- **Guard Rails & Policies:** Define hard rules and ethical boundaries

! Risks of Uncontrolled Autonomy:

- Rolling out job offers with incorrect salary/terms
- Biased hiring based on nationality or age
- Overspending on ads without permission

2.4.2 2. Goal-Oriented

Definition: Operates with a persistent objective in mind and continuously directs actions to achieve that objective.

Goal serves as a compass for the system's autonomy.

Types of Goals:

- **Independent Goals:** "Hire a Backend Engineer"



Figure 2.3. Autonomy

- **Goals with Constraints:** “Hire a Backend Engineer from India, Remote, under \$X budget”

Goal Storage (JSON format):

```
{
  "main_goal": "Hire a Backend Engineer",
  "constraints": {
    "experience": "2-4 years",
    "remote": true,
    "tech_stack": ["Python", "Django", "AWS"]
  },
  "status": "active",
  "created_at": "2025-01-15",
  "progress": {
    "jd_created": true,
    "posted_on": ["LinkedIn", "Naukri"],
    "applications": 8,
    "interviews_scheduled": 2
  }
}
```

Goals Can Be Altered Mid-Way: Example: Change from “Hire Backend Engineer” → “Find a Freelancer for project”

2.4.3 3. Planning

Definition: The ability to break down a high-level goal into a structured sequence of actions and sub-goals.

***Most Important Concept:** Agentic AI operates in 2 steps: 1. **Planning** - Break down goal and create plan 2. **Execution** - Execute the plan step-by-step*

*This is an **iterative loop***

Three-Step Planning Process:

Step 1: Generate Multiple Candidate Plans

- Planning is a **search problem**
- Initial State → Final State (multiple paths exist)
- System generates multiple approaches

Example Plans:

- **Plan A:** Post on job portals (LinkedIn, Naukri, AngelList) + Promote
- **Plan B:** Internal referrals + Hiring agency

Step 2: Evaluate Plans

Evaluation criteria:

- **Efficiency:** Which is fastest?
- **Tool Availability:** Do we have required tools?
- **Cost:** Budget constraints
- **Risk !:** Failure probability
- **Alignment with Constraints ✓:** Matches requirements?

Step 3: Select Best Plan

Selection methods:

- **Human-in-the-Loop Input:** Ask human supervisor
- **Pre-programmed Policy:** Automated decision based on rules

2.4.4 4. Reasoning

Definition: The cognitive process through which an AI system interprets information, draws conclusions, and makes decisions (both while planning and executing).

Why Important?: Reasoning is needed in BOTH planning and execution stages.

Example: Phone Theft Scenario

1. **Interpret Information:** Phone is stolen
2. **Draw Conclusions:** Thief might misuse my number
3. **Make Decisions:** Block number with Airtel

Reasoning in Planning Stage:

- **Goal Decomposition:** Break “Hire Backend Engineer” into steps
- **Tool Selection:** Determine which tool for each step (e.g., Google search for salary info)
- **Resource Estimation:** Time, dependencies, risks

Reasoning in Execution Stage:

- **Decision Making:** Interview 2 or 3 candidates?
- **Human-in-the-Loop:** When to ask human vs. decide independently
- **Error Handling:** LinkedIn server down → Wait? Notify human? Use alternative platform?

2.4.5 5. Adaptability

Definition: The ability to modify plans, strategies, and actions in response to unexpected conditions while staying aligned with the goal.

Example from AI Recruiter:

- Low applications after 3 days
- Adapts by: Running LinkedIn ads + Modifying JD to “Full Stack Engineer”

Reasons for Adaptation:

1. **Failures ✗**
 - Tool/API fails (e.g., Calendar API down)
 - Solution: Ask human directly for availability
2. **External Feedback**
 - Environment provides unexpected data
 - Example: Low application numbers from LinkedIn

3. Goal Changes

- Mid-way goal modification
- Example: “Hire Backend Engineer” → “Find Freelancer”

2.4.6 6. Context Awareness

Definition: The ability to understand, retain, and utilize relevant information from ongoing tasks, past interactions, user preferences, and environmental cues to make better decisions throughout a multi-step process.

Types of Context Stored:

1. Original Goal

- Core objective: “Hire Backend Engineer”

2. Progress & Conversation History

- JD finalized and posted on LinkedIn
- All chat history with human supervisor

3. Environment State

- Job posted on LinkedIn
- 8 applications received
- Ad budget: \$X remaining for 2 days

4. Tool Responses

- Resume parser: “Candidate B has 3 years Django + AWS experience”
- Calendar API: “Human is free at 2:00 PM”

5. User Preferences

- Company prefers remote candidates
- Human likes interview questions in Google Docs

6. Policies & Guard Rails

- “Don’t send offer letter without approval”
- “Never use paid ad platforms without permission”

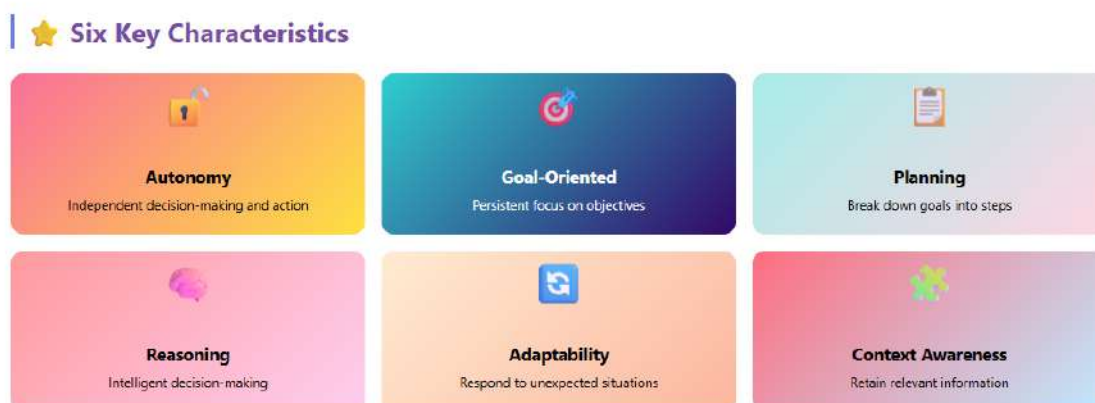


Figure 2.4. Context Awareness

Memory Types:

- **Short-term Memory:** Current session info (tool responses, immediate decisions)

- **Long-term Memory:** Goals, past interactions, user preferences, cross-session decisions

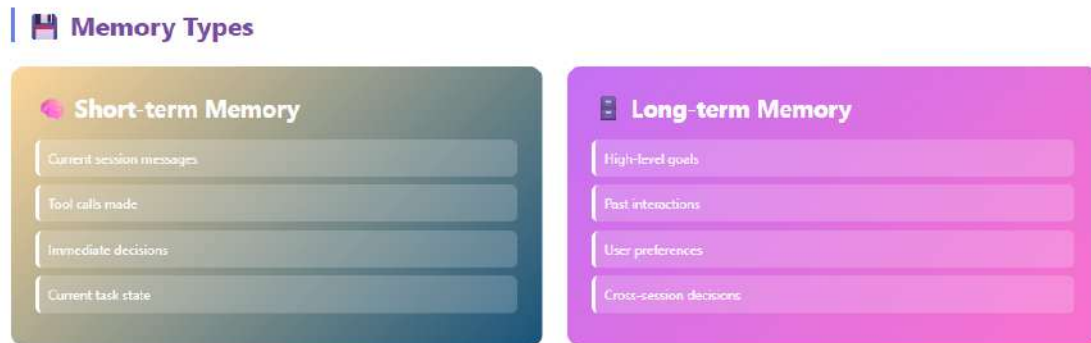


Figure 2.5. Context Awareness

2.5 Five Core Components of Agentic AI

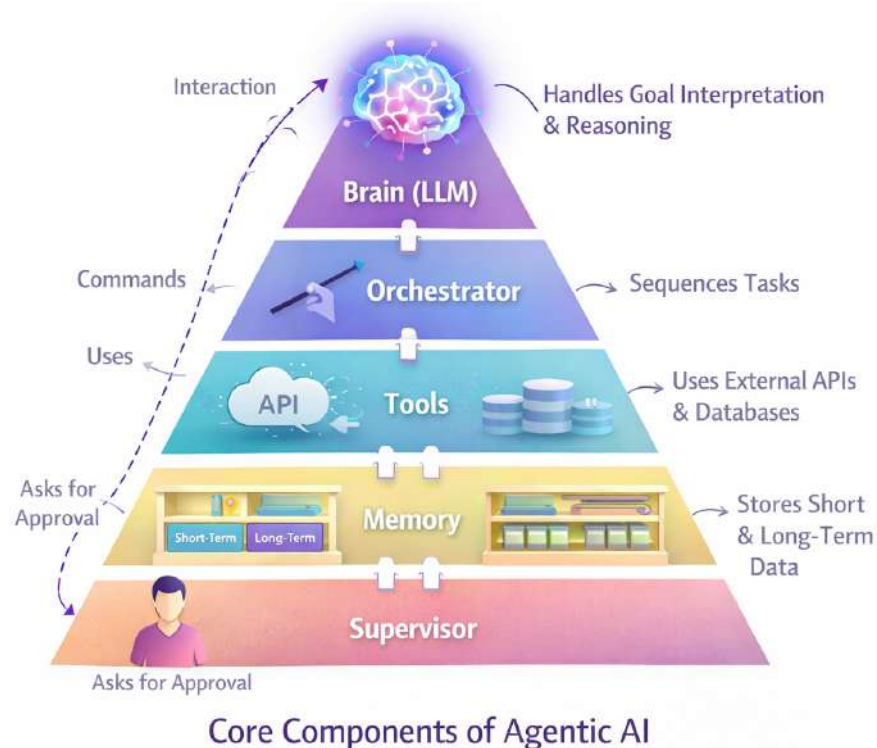


Figure 2.6. Five Core Components of Agentic AI

2.5.1 1. Brain (LLM)

The backbone - Does most of the heavy lifting.

Responsibilities:

- Goal Interpretation:** Understand what user wants
- Planning:** Break down goals into sub-goals
- Reasoning:** Both in planning and execution
- Tool Selection:** Choose appropriate tools for tasks

-**Communication:** Generate and understand natural language

2.5.2 2. Orchestrator

The project manager - Executes the plan step-by-step.

Responsibilities:

- Task Sequencing:** Order of step execution
- Conditional Routing:** If-else logic (Step 2 → Step 3 or 4?)
- Retry Logic:** Handle tool failures
- Looping & Iteration:** Repeat steps when needed
- Delegation:** Task assignment (human vs. LLM)

Built using frameworks: LangGraph, CrewAI, AutoGen

2.5.3 3. Tools

Hands and legs - Interact with external world.

Examples:

- API calls (LinkedIn, Naukri, Calendar, Email)
- Database operations
- Resume parser
- **Knowledge Base (RAG):** Company documents, policies
- Google search

2.5.4 4. Memory

Types:

Short-term Memory:

- Current session messages
- Tool calls made
- Immediate decisions
- Current task state

Long-term Memory:

- High-level goals
- Past interactions
- User preferences
- Cross-session decisions

Additional Function:

- **State Tracking:** How much work done, how much remaining

2.5.5 5. Supervisor

Implements Human-in-the-Loop concept.

Responsibilities:

- **Approval Workflows ✓:** High-risk actions (offer letter, paid ads)
- **Guard Rail Enforcement:** Ensure policies followed
- **Escalations !:** Handle edge cases
- Example: Non-IIT candidate with excellent resume → Alert human

2.6 Summary Table

Table 2.2. Summary Table

Characteristic	Icon	Key Point
Autonomy		Independent operation with minimal human guidance
Goal-Oriented		Persistent focus on achieving objectives
Planning		Structured breakdown of goals into actionable steps
Reasoning		Intelligent decision-making throughout process
Adaptability		Responds to unexpected situations
Context Awareness		Retains and utilizes relevant information

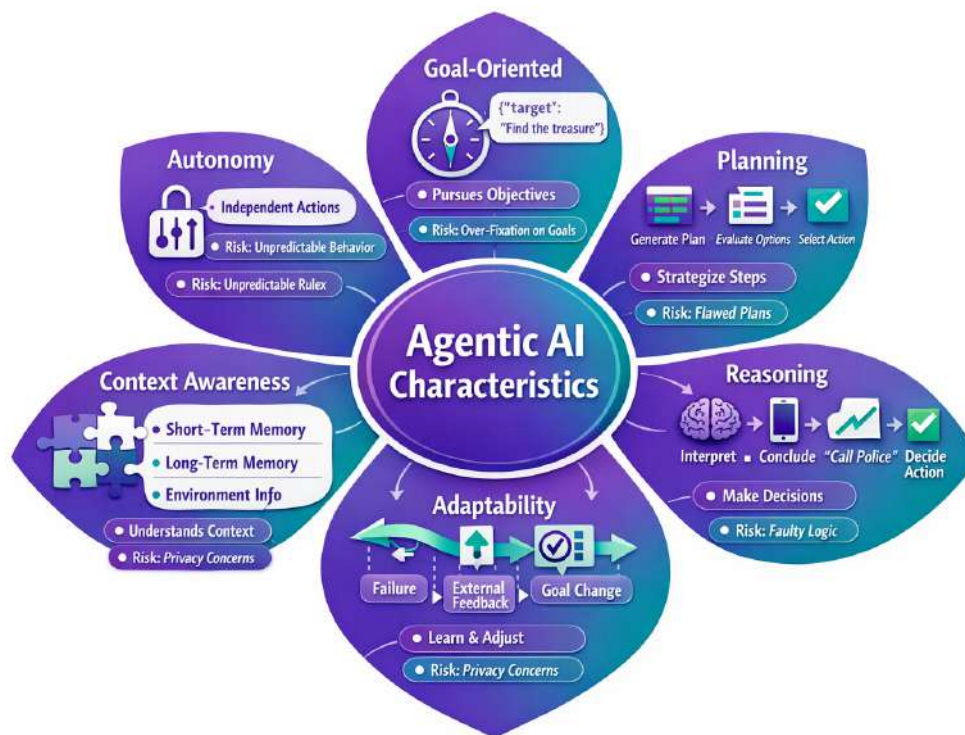


Figure 2.7. Summary Table

2.7 Key Takeaways

- ✓ **Agentic AI $\neq$ Generative AI:** Proactive vs. Reactive
- ✓ **Two-Phase Operation:** Planning $\rightarrow$ Execution (Iterative Loop)
- ✓ **Control is Essential:** Use guard rails, human-in-the-loop, policies
- ✓ **Memory is Critical:** Both short-term and long-term required
- ✓ **Components Work Together:** Brain, Orchestrator, Tools, Memory, Supervisor

2.8 What's Next?

Going forward, this knowledge will be refined further as we:

- Build practical agentic systems

- Implement these concepts in code
- Use LangGraph framework
- Create real-world applications

Pro Tip: When evaluating if a system is “Agentic AI”, check all 6 characteristics. If all are present, it’s an Agentic AI system!



Figure 2.8. What's Next?

LangChain Vs LangGraph | Agentic AI using LangGraph | Video 3 | CampusX

3.1 Current Video Goals

3.1.1 Primary Learning Objectives:

1. **Deep Level Intuition:** Understanding why LangGraph exists
2. **Problem Analysis:** What problems LangChain cannot solve that require LangGraph
3. **Technical Overview:** What exactly is LangGraph
4. **Comparative Analysis:** LangChain vs LangGraph differences

3.1.2 Expected Outcomes:

- Ability to determine when to use LangChain vs LangGraph for any application
- Develop intuition for framework selection based on use case

3.2 Framework Landscape

3.2.1 Popular Agentic AI Frameworks:

- **Crew AI:** Collaborative AI agents framework
- **Microsoft AutoGen:** Multi-agent conversation framework
- **LangGraph:** Focus of this series (by LangChain team)
- **Other emerging frameworks**

3.2.2 Why LangGraph?

- Developed by the LangChain team
- LangChain is already a well-established and famous product
- Positive community feedback
- Considered one of the top frameworks for building Agentic AI
- Will be used throughout the entire playlist

3.3 Prerequisites

3.3.1 Required Knowledge:

- **LangChain Fundamentals:** Essential understanding needed
- **Basic LangChain Concepts:** What it is and what you can do with it
- **LangChain Coding:** Basic level code writing experience

3.3.2 Recommended Preparation:

- Watch LangChain playlist
- Specifically focus on first two videos:
 1. Introduction to LangChain
 2. LangChain Components Overview

3.4 Key Questions to be Addressed

3.4.1 Central Problem Statement:

"What specific problems exist that LangChain cannot solve, which necessitated the creation of LangGraph?"

3.4.2 Framework Comparison Focus:

- Both are libraries from the same ecosystem
- Understanding their distinct use cases
- Decision framework for choosing between them

3.5 Video Structure Breakdown

3.6 Learning Approach

3.6.1 Instructional Method:

- **Deep Level Examples:** Detailed practical scenarios
- **Visual Demonstrations:** Step-by-step breakdowns
- **Technical Deep Dive:** Comprehensive understanding
- **Interview Preparation:** Ability to answer any related questions

3.6.2 Time Investment:

- **Warning:** This is a longer video than usual
- **Justification:** Could be explained in 10-15 minutes, but detailed examples require more time
- **Benefit:** Complete understanding guaranteed with full video completion

3.7 Development Challenges

3.7.1 Why Not Code from Scratch?

3.7.2 Framework Benefits:

- **Simplified Development:** Reduced complexity
- **Proven Patterns:** Battle-tested approaches
- **Community Support:** Extensive documentation and examples
- **Rapid Prototyping:** Faster time to working solution

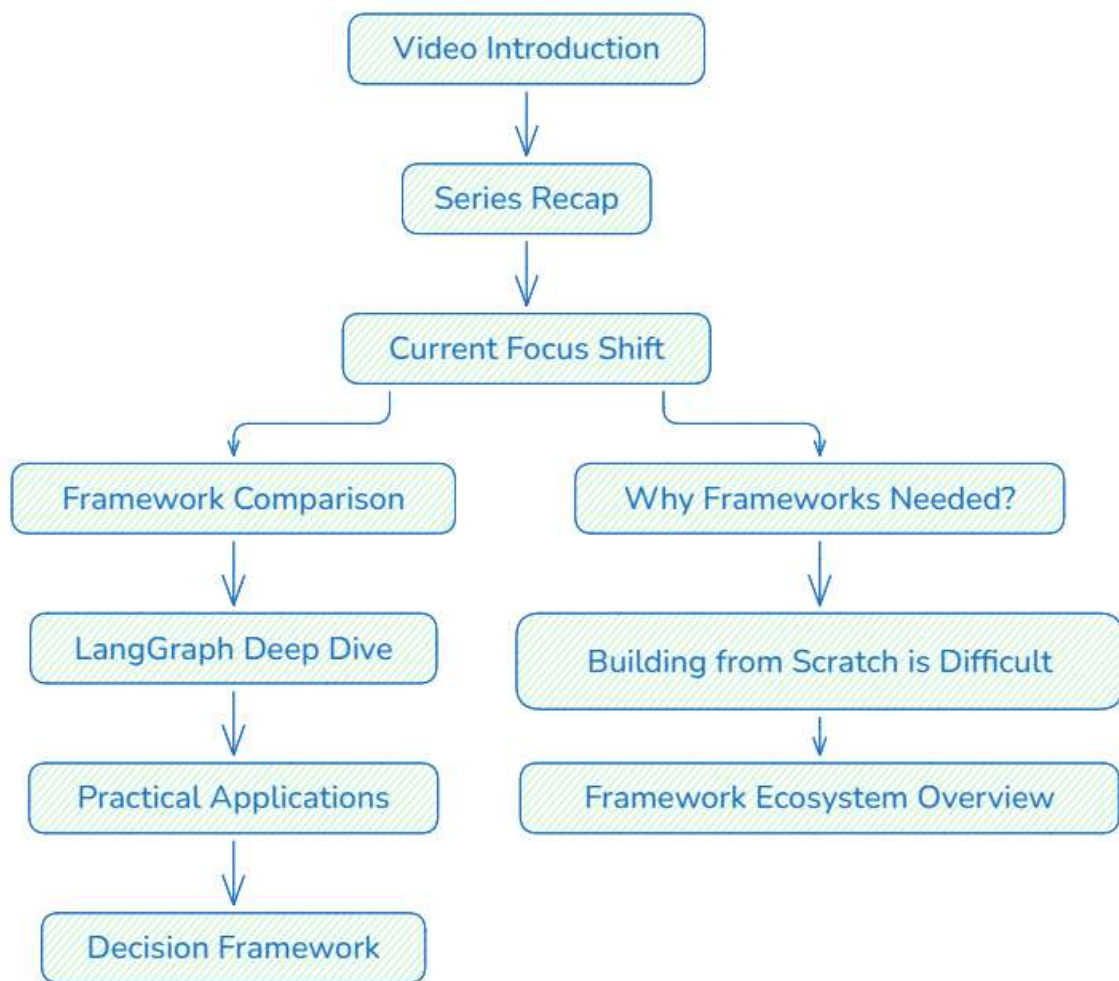


Figure 3.1. Video Structure Breakdown

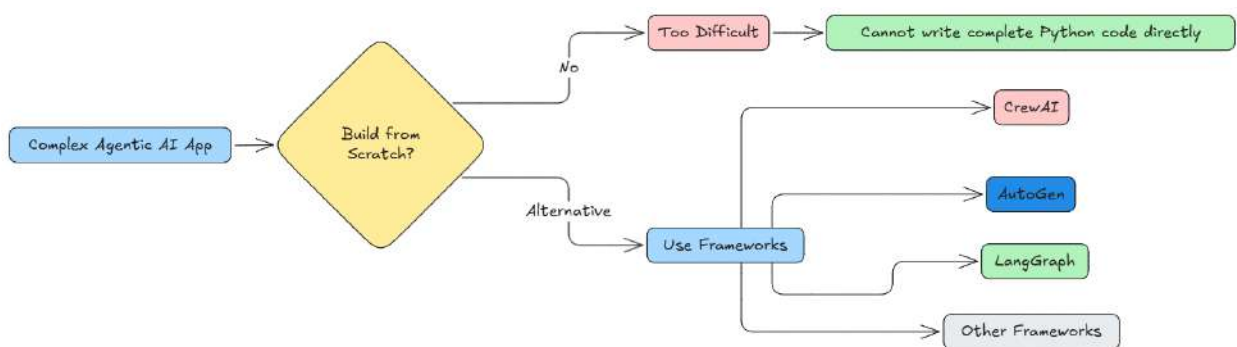


Figure 3.2. Why Not Code from Scratch?

3.8 LangChain Recap - Foundation for LangGraph Understanding

3.9 Quick Recap Context

Purpose: Essential refresher before diving into LangGraph concepts **Target Audience:** Those who may need a quick review of LangChain fundamentals **Next Step:** Understanding complex workflows and LangChain limitations

3.10 What is LangChain?

3.10.1 Core Definition

LangChain is an open-source library designed to simplify the process of building LLM-based applications.

3.10.2 Market Context

- **Current Trend:** Integration of LLMs into existing software everywhere
- **Examples of LLM Integration:**
 - Chatbots in websites
 - Chrome plugins for YouTube video interaction
 - AI assistants in various applications

3.10.3 The Challenge

Building LLM-based applications is **inherently difficult** because:

- Multiple components need to be connected
- Complex integration requirements
- Various services must work together seamlessly

3.10.4 LangChain's Solution

Simplifies the entire process by providing modular building blocks for any LLM-based workflow

3.11 LangChain Components (Modular Building Blocks)

3.11.1 1. Model Component

Key Features:

- **Unified Interface:** Single interface for multiple LLM providers
- **Provider Flexibility:** OpenAI, Anthropic, Hugging Face, Ollama support
- **Easy Switching:** Minimal code changes when replacing LLMs
- **Consistent API:** Same methodology across different LLMs

Benefits:

- Vendor independence
- Simplified code maintenance
- Easy model experimentation

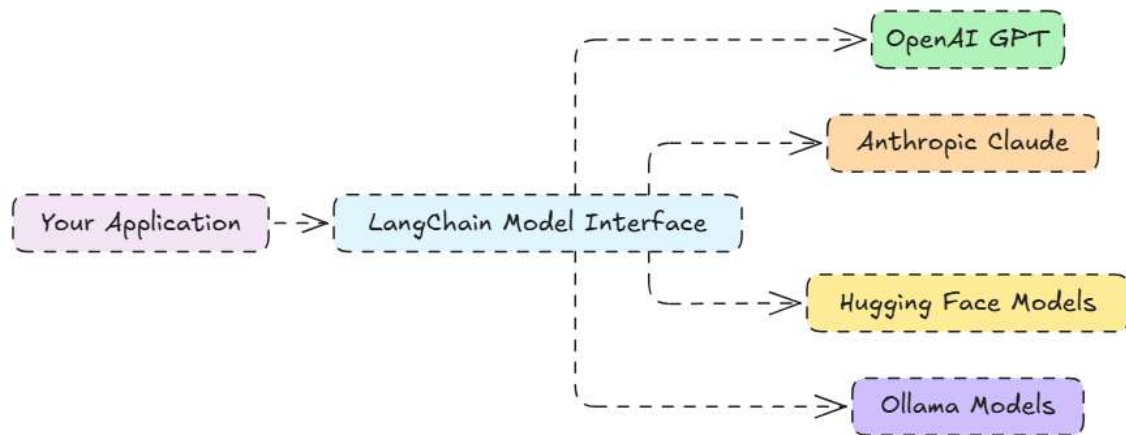


Figure 3.3. Model Component

3.11.2 2. Prompts Component

Purpose: Facilitates prompt engineering across different scenarios

Capabilities:

- Template management
- Dynamic prompt generation
- Prompt optimization tools
- Structured prompt design

Why Important:

- LLMs operate entirely based on prompts
- Prompts require careful design and engineering
- Different use cases need different prompt strategies

3.11.3 3. Retrievers Component

Core Function: Fetch relevant documents from vector stores/knowledge bases

Features:

- Multiple search strategies
- Various algorithms for document retrieval
- Support for large and diverse knowledge bases
- Foundation for RAG (Retrieval Augmented Generation) applications

Use Cases:

- Document search systems
- Company knowledge bases
- Personal document libraries

3.11.4 4. Chains Component (Most Important)

The biggest offering of LangChain - that's why "Chain" is in the name!

Chain Concept:

- **Component Connection:** Link different components together
- **Automatic Flow:** First block's output becomes second block's input
- **Scalability:** Create chains of any length

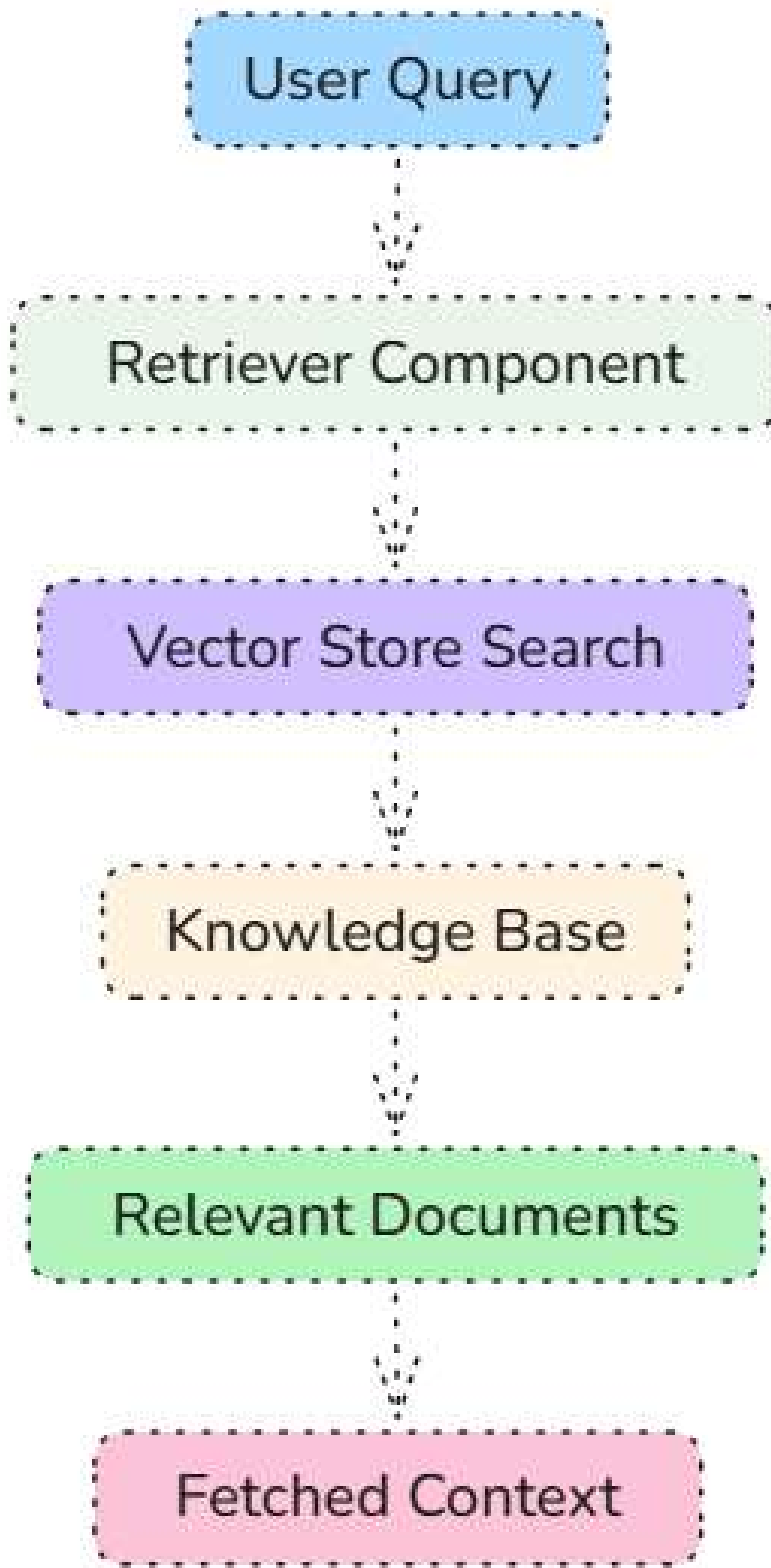


Figure 3.4. Retrievers Component



Figure 3.5. Chains Component (Most Important)

- **No Manual Work:** LangChain handles the data flow automatically

Example Chain Flow:

1. **Prompt** generates structured input
2. **Model** processes the prompt
3. **Output Parser** formats the response
4. **Next Component** continues the workflow

3.12 What Can You Build with LangChain?

3.12.1 1. Simple Conversational Workflows

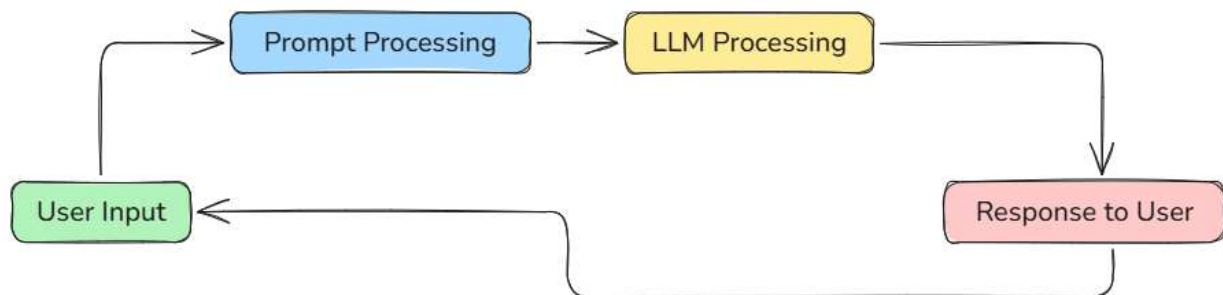


Figure 3.6. Simple Conversational Workflows

Applications:

- **Chatbots:** Interactive conversation systems
- **Text Summarizers:** Content condensation tools
- **Q&A Systems:** Direct question-answer workflows

Workflow Pattern:

1. Take user prompt
2. Send to LLM
3. Return output to user
4. Optionally loop for continuous conversation

3.12.2 2. Multistep Workflows

Example Use Case: Report Generation + Summarization

1. **Step 1:** User provides topic → Generate detailed report
2. **Step 2:** Take detailed report → Create summary
3. **Output:** Deliver final summary to user

Implementation Strategy:

- Chain multiple prompt-model combinations
- Each step builds on previous output
- Automatic data flow between components

3.12.3 3. RAG (Retrieval Augmented Generation) Applications

Real-World Example: Company Policy Chatbot

- **Question:** "What is our company's leave policy?"

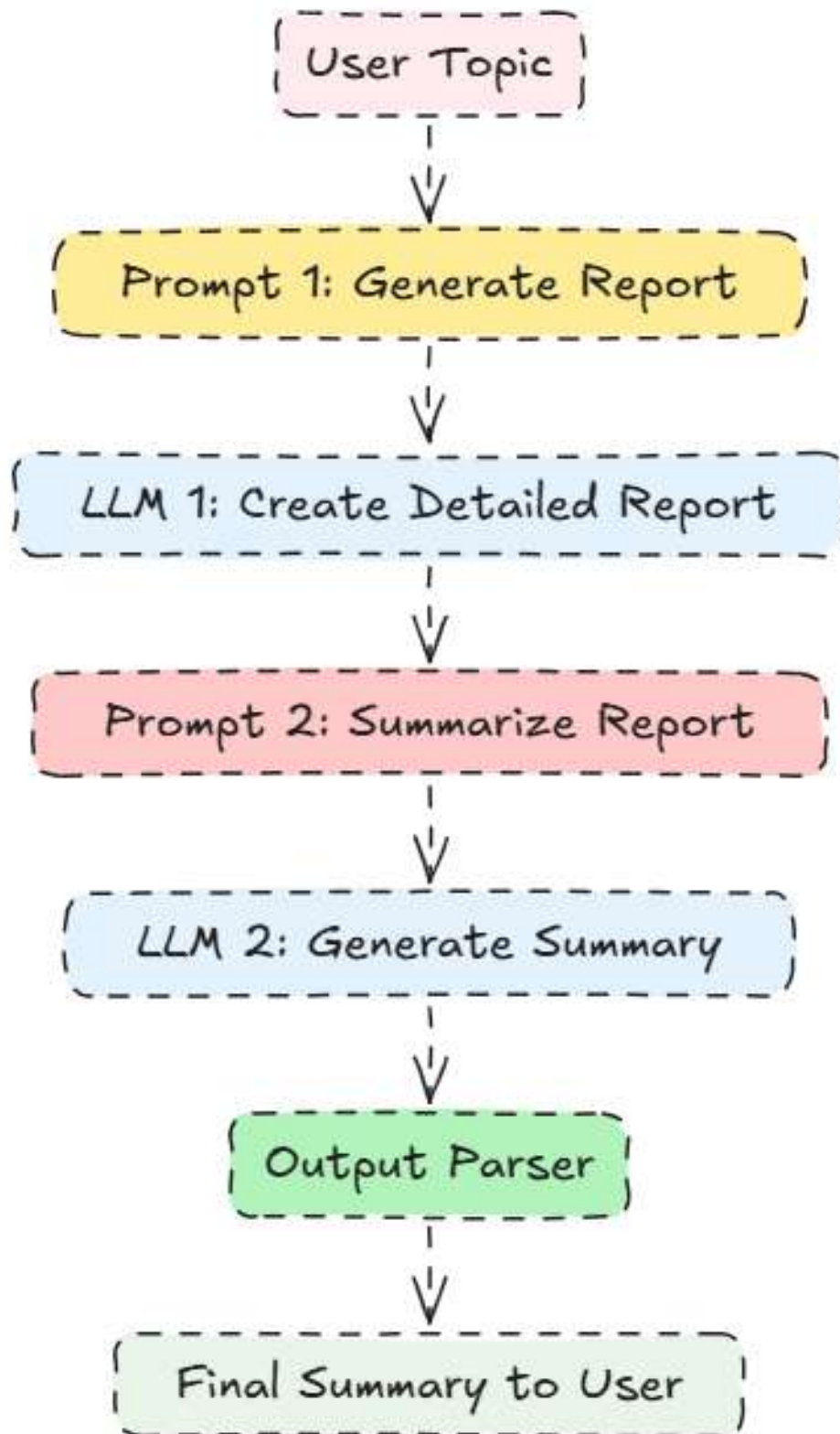


Figure 3.7. Multistep Workflows

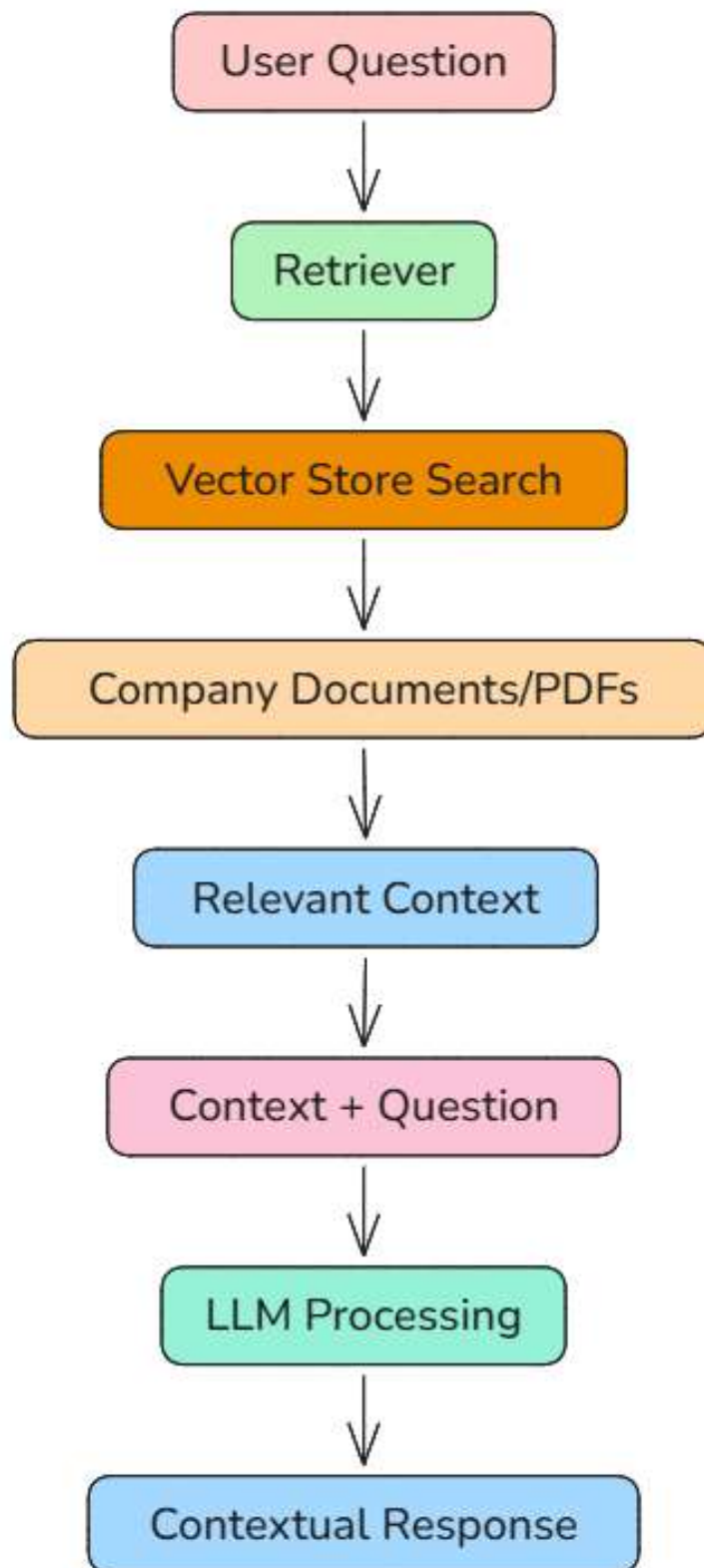


Figure 3.8. RAG (Retrieval Augmented Generation) Applications

- **Process:** Search company documents → Find relevant policy → Generate answer
- **Benefit:** Accurate, company-specific responses

Workflow Components:

1. **User Query** → Retriever component
2. **Retriever** → Searches vector store
3. **Context Extraction** → Relevant documents fetched
4. **LLM Processing** → Question + context → Response

3.12.4 4. Basic Level Agents

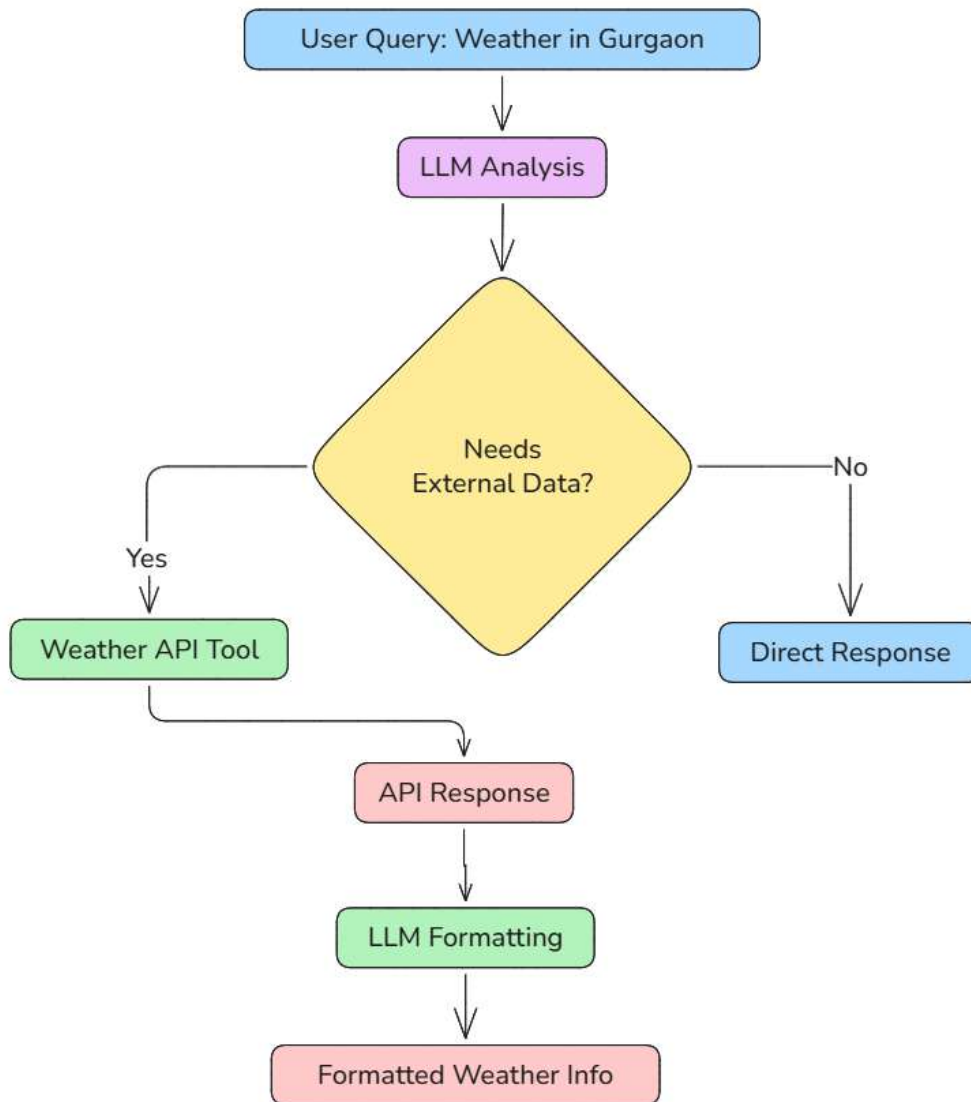


Figure 3.9. Basic Level Agents

Tools Integration Concept:

- **Connect LLMs to external tools:** APIs, Python functions
- **Intelligent Decision Making:** LLM decides when to use which tool
- **Dynamic Input Handling:** LLM determines appropriate tool inputs

Example Workflow:

1. **User:** "What's the weather in Gurgaon?"

2. **LLM:** Recognizes need for real-time data
3. **Tool Trigger:** Calls weather API with location
4. **Response:** LLM formats API response for user

3.13 Learning Path Recommendations

3.13.1 If This Content Seems New/Confusing:

Recommended Action: Watch the detailed LangChain playlist first

- Covers all components in detail
- Provides hands-on coding examples
- Essential foundation for advanced topics

3.13.2 Prerequisites Confirmed:

- Understanding of LLM applications
- Familiarity with prompt engineering
- Basic knowledge of component-based architecture

3.14 LangChain Strengths Summary

Table 3.1. *LangChain Strengths Summary*

Component	Primary Benefit	Use Case
Model	Unified LLM interface	Multi-provider flexibility
Prompts	Structured prompt engineering	Consistent prompt management
Retrievers	Smart document search	RAG applications
Chains	Automated workflow	Complex multi-step processes

3.14.1 Overall Value Proposition:

- **Simplification:** Complex LLM workflows made manageable
- **Modularity:** Reusable components for different applications
- **Flexibility:** Support for various LLM providers and use cases
- **Automation:** Reduced manual coding for workflow management

3.15 Key Takeaways

1. **LangChain Excellence:** Perfect for linear, predictable workflows
2. **Component Power:** Modular approach enables rapid development
3. **Chain Magic:** Automatic data flow between components
4. **Current Limitations:** (To be explored with complex workflow example)

3.16 Automated Hiring Workflow - LangChain Implementation Analysis

3.17 Workflow Overview

3.17.1 Visual Complexity Assessment

- **Flowchart Detail:** Extremely comprehensive and detailed
- **Zoom Requirement:** Had to zoom out to fit entire flowchart on screen
- **Complexity Level:** Far beyond basic LangChain workflows
- **Implementation Challenge:** Not straightforward linear process

3.18 Critical Distinction: Workflow vs Agentic AI

3.18.1 Essential Clarification

What we're looking at is a WORKFLOW, not an Agentic AI application

3.18.2 Key Differences Explained

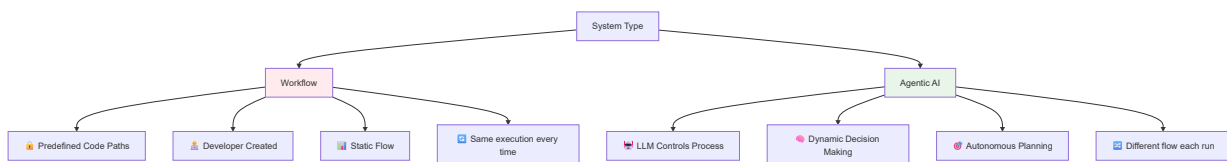


Figure 3.10. Key Differences Explained

3.18.3 Anthropic's Definition Reference

Recommended Reading: [Building Effective Agents - Anthropic Blog]

Workflows:

- **Systems where LLMs and tools are orchestrated through predefined code paths**
- Developer creates the entire flow structure
- Static execution pattern
- Same sequence every time

Agents:

- **Systems where LLMs dynamically direct their own processes and tool usage**
- Maintaining control over how they accomplish tasks
- Dynamic planning and execution
- Variable flow based on context

3.18.4 This Example Context

- **Previous Video:** Presented as agentic AI example
- **Reality:** Human recruiter provides initial input → system follows predefined steps
- **Developer Role:** Created the entire flowchart structure
- **Agent Role:** Would create its own flow dynamically each time

3.19 Detailed Workflow Breakdown

3.19.1 Phase 1: Job Creation & Approval

Step 1: Hiring Request

- **Input:** “We need a backend engineer, remote work, 2-4 years experience”
- **Format:** Prompt-based request

Step 2: JD Creation

- **Process:** LLM generates detailed job description
- **Output:** Comprehensive job posting content

Step 3: Human Approval Loop

- **Decision Point:** Human supervisor reviews JD
- **If Rejected:** Return to JD creation with feedback
- **If Approved:** Proceed to posting

Step 4: Job Posting

- **Tools Required:** LinkedIn API, Naukri.com API
- **Action:** Automated posting across platforms

3.19.2 Phase 2: Application Monitoring & Optimization

Waiting Period

- **Duration:** 7 days initial wait
- **Purpose:** Allow maximum applications

Application Monitoring

- **Tool:** LinkedIn API integration
- **Check:** Count of received applications
- **Threshold:** 20 applications minimum

Optimization Loop

- **If Insufficient Applications:**
- Modify eligibility criteria (include freshers)
- Expand role scope (backend → full-stack)
- Increase salary offering
- Wait 48 hours and re-check
- **Loop Continues:** Until threshold met

3.19.3 Phase 3: Candidate Processing

Resume Analysis

- **Tool:** Resume parser for automated download
- **Processing:** Parse and structure resume data
- **Scoring:** LLM generates score for each candidate
- **Result:** Top 5 candidates selected

Interview Scheduling

- **Tools Required:**
- Calendar API (availability checking)
- Email API (invitation sending)
- **Process:** Automated scheduling and communication

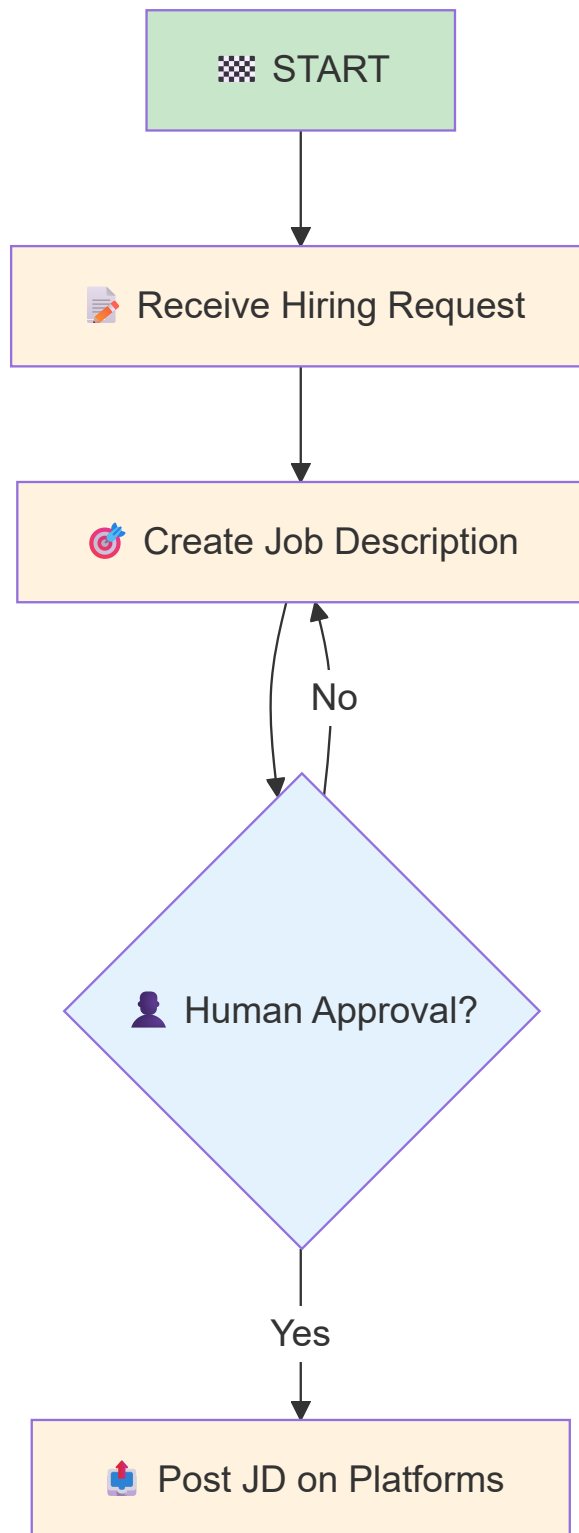


Figure 3.11. *Phase 1: Job Creation & Approval*

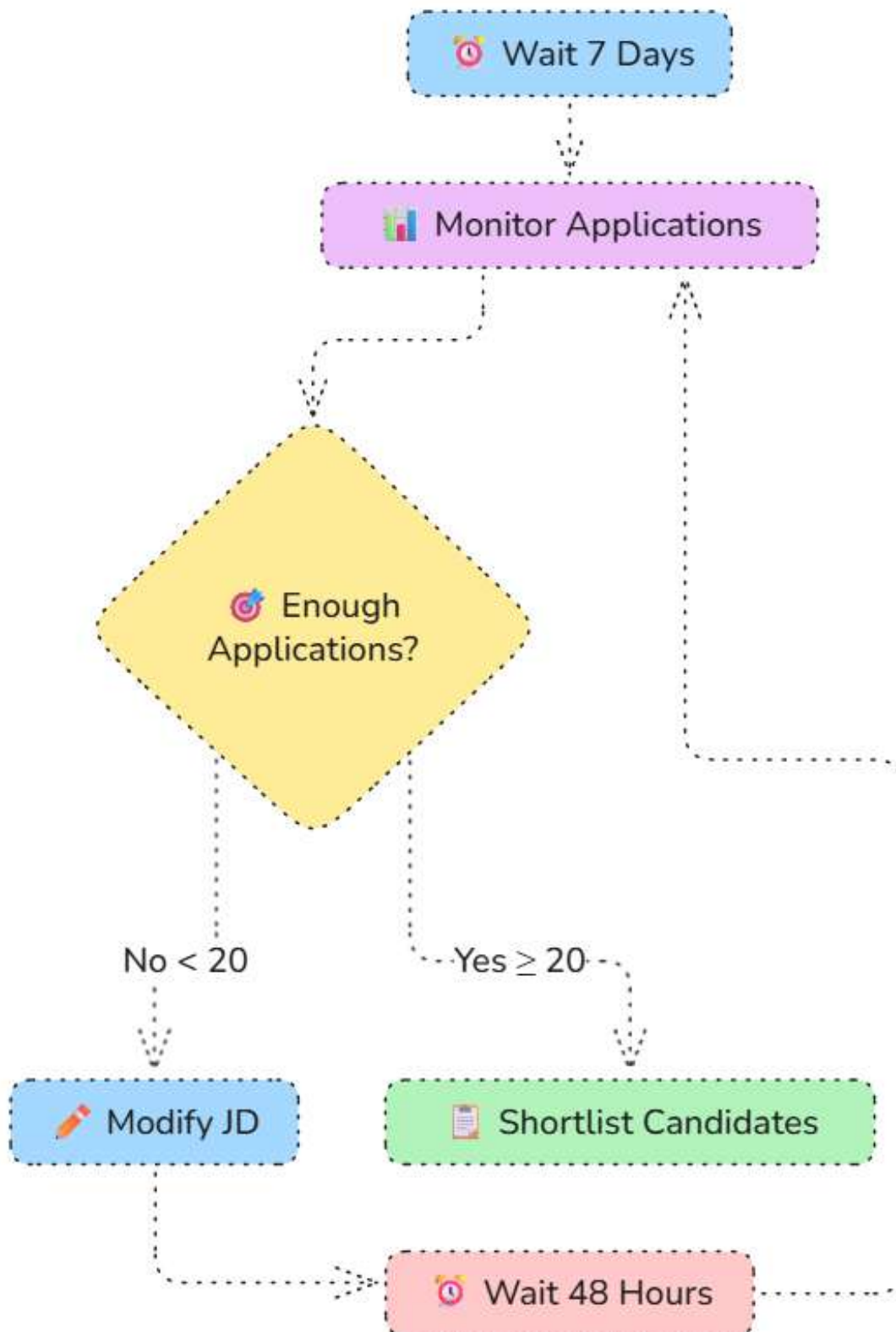


Figure 3.12. Phase 2: Application Monitoring & Optimization



Figure 3.13. Phase 3: Candidate Processing

3.19.4 Phase 4: Interview & Selection Process

Interview Execution

- **Support Tools:** Question bank generation
- **Communication:** Automated reminder emails
- **Multiple Candidates:** Repeat process for each

Decision Processing

- **Per Candidate:** Individual selection decision
- **If Rejected:** Automated regret email
- **If Selected:** Proceed to offer process

3.19.5 Phase 5: Offer & Onboarding

Offer Management

- **Tools:** LLM for offer letter creation, Email API
- **Tracking:** Monitor acceptance/rejection
- **Negotiation Loop:** Human-handled renegotiation process

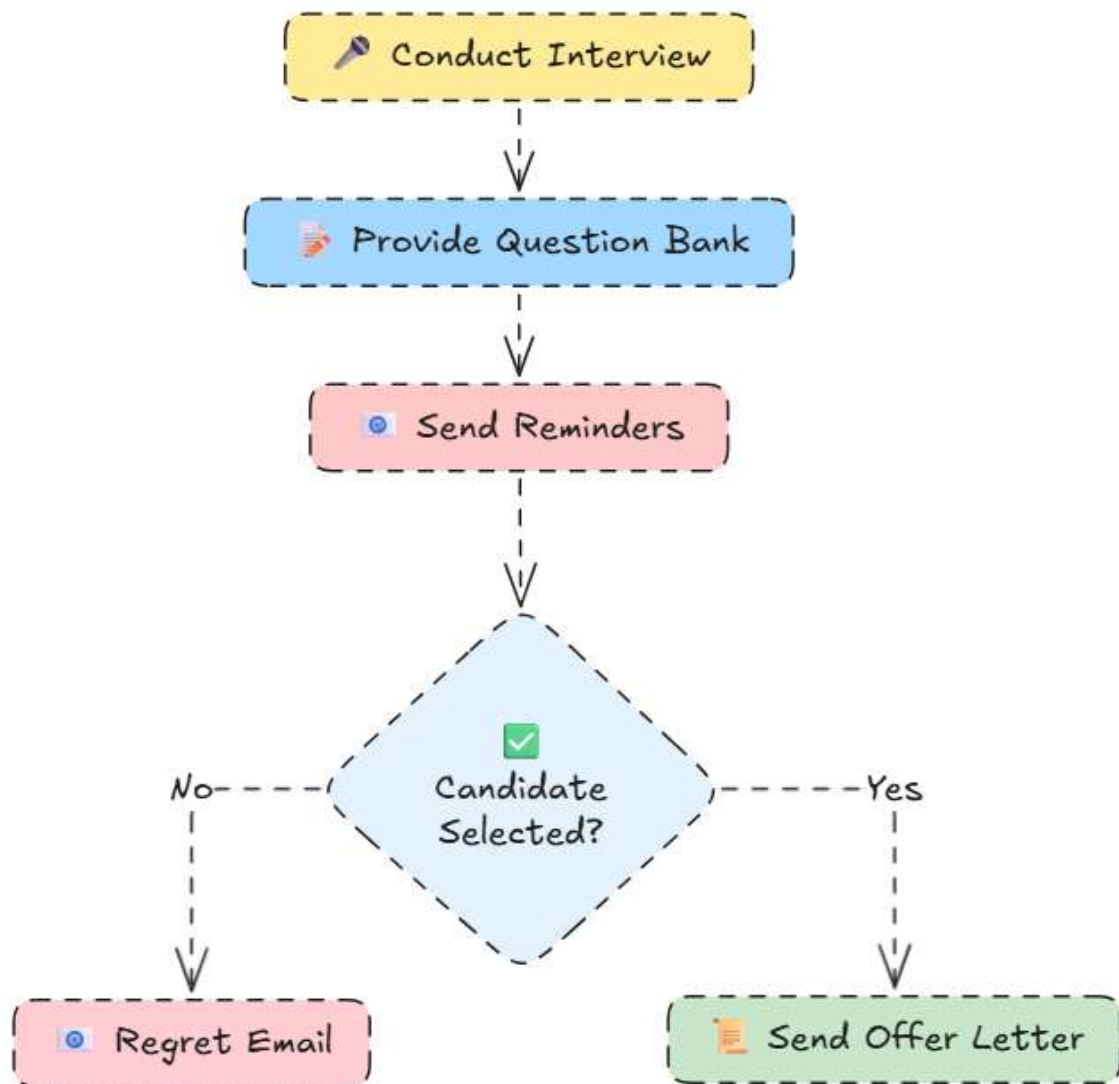


Figure 3.14. Phase 4: Interview & Selection Process

Onboarding Automation

- **HRMS Integration:** Human Resource Management System
- **Automated Tasks:**
 - Welcome email dispatch
 - Knowledge Transfer session scheduling
 - Equipment provisioning
 - System access setup

3.20 Workflow Characteristics Analysis

3.20.1 Complexity Indicators

Table 3.2. Complexity Indicators

Aspect	Basic Workflow	This Workflow
Flow Type	Linear	Branching + Loops

Aspect	Basic Workflow	This Workflow
Decision Points	Minimal	Multiple Critical
External Tools	1-2	6+ APIs/Tools
Human Interaction	Optional	Required
Time Dependencies	None	Multiple Waits
State Management	Simple	Complex

3.20.2 Tool Dependencies

1. **LinkedIn API** - Job posting and application monitoring
2. **Naukri.com API** - Additional job platform
3. **Resume Parser** - Document processing
4. **Calendar API** - Interview scheduling
5. **Email API** - Communication automation
6. **HRMS Integration** - Onboarding workflow

3.20.3 Critical Features Required

- **Conditional Branching:** Multiple if/else decision points
- **Looping Mechanisms:** Retry logic with conditions
- **Time-based Triggers:** Wait periods and scheduling
- **State Persistence:** Remember workflow progress
- **Human-in-the-loop:** Approval and decision points
- **Error Handling:** Tool failure recovery
- **Parallel Processing:** Multiple candidate handling

3.21 ! LangChain Implementation Challenges Preview

3.21.1 Complexity Assessment

- **Basic LangChain Workflows:** ✓ Very manageable
- **Linear Workflows:** ✓ Straightforward implementation
- **This Workflow:** ✗ Significantly challenging

3.21.2 Why This Is Difficult

“Creating basic level workflows in LangChain is quite easy, especially linear workflows. What you see on screen is not basic level - this is a complex workflow.”

3.21.3 Emerging Intuition

Most viewers with LangChain experience will sense this won't be straightforward to implement.

3.22 Upcoming Analysis Framework

3.22.1 Discussion Structure

8 Specific Points will be covered to analyze implementation challenges:

1. **Point-by-point breakdown** of implementation difficulties
2. **LangChain limitations** for each challenge
3. **LangGraph solutions** for each problem

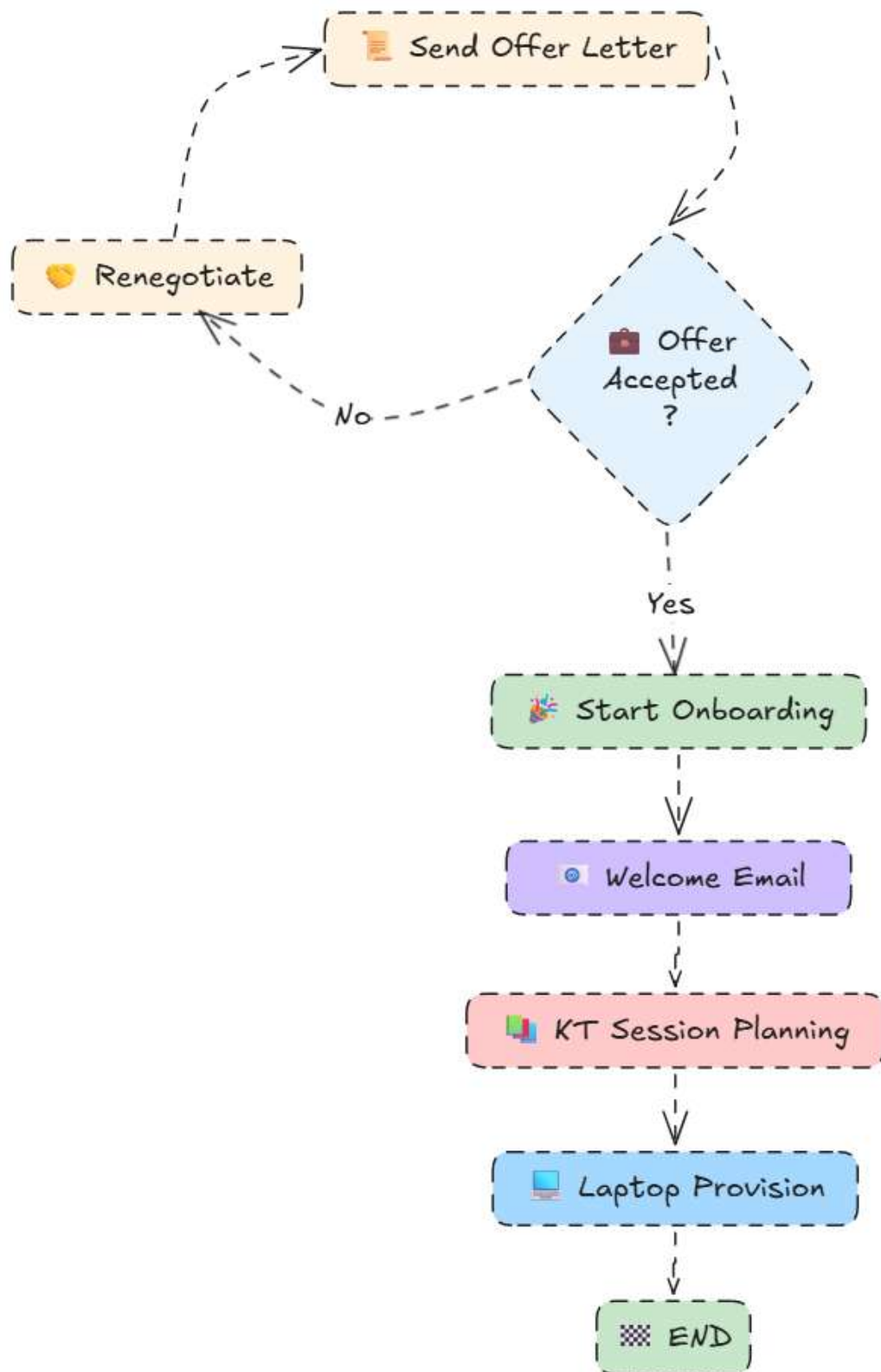


Figure 3.15. Phase 5: Offer & Onboarding

4. Side-by-side comparison of approaches

3.22.2 Learning Outcome

By the end of this analysis, you'll have:

- **Clear understanding** of LangChain vs LangGraph differences
- **Practical intuition** for framework selection
- **Technical insight** into why LangGraph was necessary

3.23 Key Takeaways

3.23.1 Workflow vs Agent Distinction

- **Static vs Dynamic:** Predetermined paths vs autonomous planning
- **Developer vs AI Control:** Human-designed vs AI-designed execution
- **Predictable vs Adaptive:** Same flow vs context-dependent flow

3.23.2 Complexity Spectrum

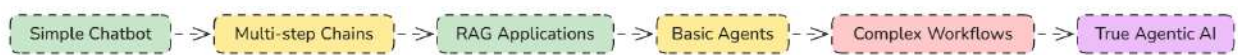


Figure 3.16. Complexity Spectrum

- **LangChain Sweet Spot:** A through D
- **Challenge Zone:** E (This workflow)
- **LangGraph Territory:** E and F

3.24 Challenge #2: State Handling - LangChain vs LangGraph

3.25 Understanding State in Complex Workflows

3.25.1 What is “State”?

State = Collection of important data points that must be maintained and tracked throughout workflow execution

3.25.2 Critical Question: What Data is Important?

Looking at our automated hiring workflow, multiple data points are crucial:

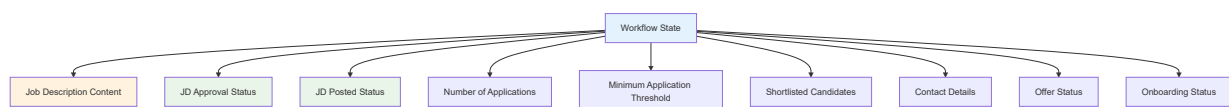


Figure 3.17. Critical Question: What Data is Important?

3.26 State Evolution Through Workflow

3.26.1 Dynamic State Changes

State values evolve as workflow progresses through different stages:

Table 3.3. *Dynamic State Changes*

Workflow Stage	State Updates
Hiring Request	jd: null
Create JD	jd: "Generated content"
JD Approval	jd_approved: true/false
Post JD	jd_posted: true
Monitor Applications	num_applications: count
Shortlisting	shortlisted_candidates: [...]
Interview	interview_status: "scheduled"
Offer	offer_status: "sent/accepted/rejected"
Onboarding	onboarding_status: "in_progress"

3.26.2 Example State Object Structure

```
{
  "goal": "Hire a backend software engineer",
  "jd": "",
  "jd_approved": false,
  "jd_posted": false,
  "min_applicants": 20,
  "num_applications": 0,
  "shortlisted_candidates": [],
  "interview_questions": [],
  "offer_status": {
    "sent": false,
    "accepted": false,
    "renegotiated": false
  },
  "onboarding_status": {
    "completed": false,
    "start_date": null,
    "employee_id": null
  }
}
```

3.27 X LangChain's State Handling Limitations

3.27.1 Core Problem: Stateless Architecture

LangChain workflows are inherently STATELESS

3.27.2 Available Memory vs Required Memory

3.27.3 What LangChain Provides:

- ✓ **Conversational Memory:** Store and pass chat history
- ✓ **Context Continuity:** LLM remembers past conversations

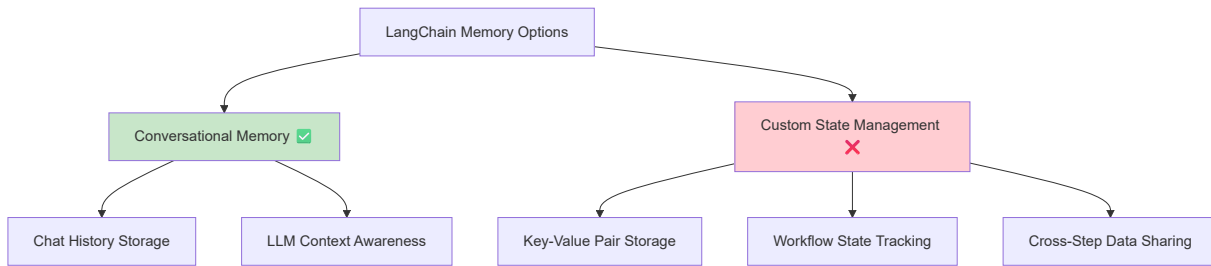


Figure 3.18. Available Memory vs Required Memory

3.27.4 What LangChain DOESN'T Provide:

- ✗ **Key-Value State Storage:** No built-in mechanism
- ✗ **Workflow State Tracking:** Manual implementation required
- ✗ **Cross-Step Data Persistence:** No automatic state passing

3.28 Manual State Implementation in LangChain

3.28.1 Required Approach (Manual & Error-Prone)

```

# Step 1: Create global dictionary (MANUAL)
workflow_state = {
    "jd": None,
    "jd_approved": False,
    "jd_posted": False,
    "num_applications": 0,
    "shortlisted_candidates": []
}

# Step 2: Manual state updates throughout workflow
def create_jd_step():
    global workflow_state
    jd_content = jd_chain.invoke(...)
    workflow_state["jd"] = jd_content # MANUAL UPDATE
    return jd_content

def approval_step():
    global workflow_state
    approved = check_approval(workflow_state["jd"])
    workflow_state["jd_approved"] = approved # MANUAL UPDATE
    return approved

def post_jd_step():
    global workflow_state
    if workflow_state["jd_approved"]:
        post_result = post_jd(workflow_state["jd"])
        workflow_state["jd_posted"] = True # MANUAL UPDATE

# This pattern repeats for EVERY step...

```

3.28.2 Problems with Manual Approach:

- Error-Prone:** Easy to forget state updates
- Maintenance Nightmare:** Global dictionary management
- Complex Workflows:** More steps = more manual work
- Team Collaboration:** Difficult coordination on state structure
- Debugging:** Hard to track state changes

3.28.3 Alternative: Conversational State (Hacky)

```
# Option: Store entire state as text in conversational memory
state_as_text = """
Current state:
- JD: "Software Engineer position..."
- JD Approved: True
- Applications: 15
- Threshold: 20
"""
# Problems: Text parsing, formatting, unreliable
```

3.29 ✓ LangGraph's State Handling Solution

3.29.1 Core Philosophy: Stateful Architecture

LangGraph execution is STATEFUL by design

3.29.2 Built-in State Management

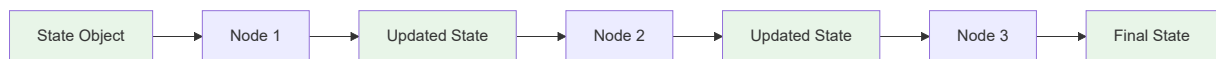


Figure 3.19. Built-in State Management

3.29.3 State Object Creation

```
# Define state structure (Pydantic or TypedDict)
from typing import TypedDict, List

class WorkflowState(TypedDict):
    goal: str
    jd: str
    jd_approved: bool
    jd_posted: bool
    min_applicants: int
    num_applications: int
    shortlisted_candidates: List[dict]
    offer_status: dict
    onboarding_status: dict

# Or simple dictionary approach
workflow_state = {
    "goal": "Hire backend engineer",
    "jd": "",
    "jd_approved": False,
    # ... other fields
}
```

3.29.4 Node Implementation with State Access

```
def create_jd_node(state: WorkflowState) -> WorkflowState:
    """Every node receives and returns state"""
    # Access current state
    hiring_goal = state["goal"]

    # Perform processing
    jd_content = jd_chain.invoke({"request": hiring_goal})
```

```
# Update state (automatic propagation)
return {
    **state,
    "jd": jd_content
}

def approval_node(state: WorkflowState) -> WorkflowState:
    """State automatically passed from previous node"""
    # Access JD from state
    jd_content = state["jd"]

    # Check approval
    approved = check_approval(jd_content)

    # Return updated state
    return {
        **state,
        "jd_approved": approved
    }

def post_jd_node(state: WorkflowState) -> WorkflowState:
    """Conditional logic based on state"""
    if state["jd_approved"]:
        post_jd(state["jd"])
        return {
            **state,
            "jd_posted": True
        }
    return state
```

3.30 LangGraph State Architecture

3.30.1 Automatic State Management

3.30.2 Key Features:

1. **Universal Access:** Every node can read entire state
2. **Mutable State:** Nodes can modify state values
3. **Automatic Propagation:** State updates flow to next nodes
4. **Type Safety:** Pydantic/TypedDict support
5. **No Manual Management:** Framework handles state lifecycle

3.30.3 State Flow Example:

```
# Node execution flow with state
graph = create_graph()

# Initial state
initial_state = {
    "goal": "Hire backend engineer",
    "jd": None,
    "jd_approved": False
}

# Execution flow
result = graph.invoke(initial_state)
# State automatically flows:
# initial_state → create_jd_node → approval_node → post_jd_node
```

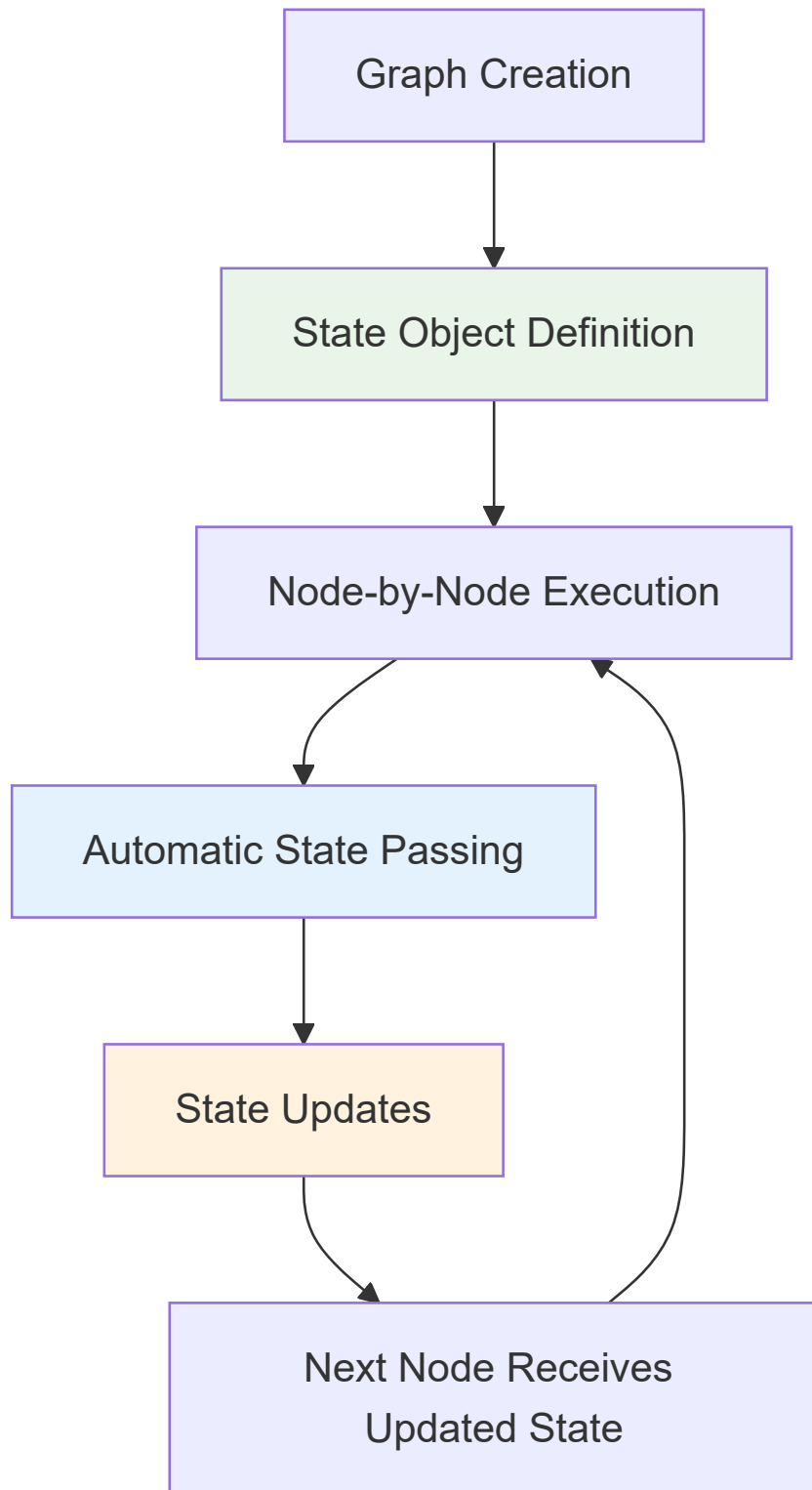


Figure 3.20. *Automatic State Management*

3.31 Comparative Analysis

3.31.1 State Management Comparison

Table 3.4. State Management Comparison

Aspect	LangChain	LangGraph
Architecture	Stateless	Stateful
State Storage	Manual dictionary	Built-in state object
State Access	Global variables	Node parameters
State Updates	Manual modification	Return-based updates
Error Handling	Prone to bugs	Framework managed
Maintenance	High complexity	Low complexity
Team Collaboration	Difficult coordination	Standardized patterns

3.31.2 Code Complexity Comparison

```
# LangChain Approach
global_state = {} # Manual management

def step1():
    global global_state
    result = process()
    global_state["field1"] = result # Manual update

def step2():
    global global_state # Access global
    input_data = global_state["field1"]
    result = process(input_data)
    global_state["field2"] = result # Manual update

# LangGraph Approach
def step1(state):
    result = process()
    return {"**state", "field1": result} # Automatic management

def step2(state):
    input_data = state["field1"] # Direct access
    result = process(input_data)
    return {"**state", "field2": result} # Automatic update
```

3.32 Real-World Implications

3.32.1 For Simple Workflows:

- **LangChain:** ✓ Adequate (minimal state needed)
- **LangGraph:** ✓ Capable but potentially overkill

3.32.2 For Complex Workflows:

- **LangChain:** ✗ Becomes unmanageable
- **LangGraph:** ✓ Designed for complexity

3.32.3 State Complexity Scaling:

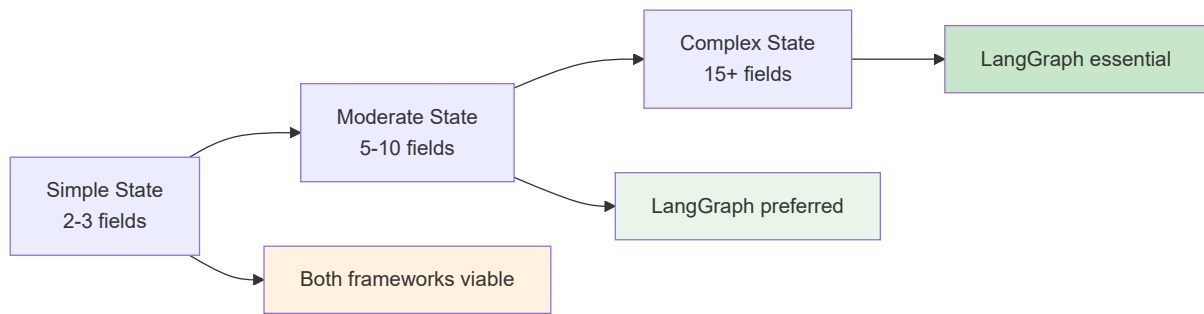


Figure 3.21. State Complexity Scaling

3.33 Key Terminology

3.33.1 Critical Distinction:

LangChain = STATELESS *LangGraph* = STATEFUL

3.33.2 Why This Matters:

- **Complex Applications:** Require stateful execution
- **Enterprise Workflows:** Multiple interconnected data points
- **Long-Running Processes:** State persistence across steps
- **Error Recovery:** State-aware debugging and resumption

3.34 Decision Framework

3.34.1 Choose LangChain When:

- ✓ Linear workflows with minimal data sharing
- ✓ Simple conversational applications
- ✓ Prototype or proof-of-concept projects
- ✓ Limited state requirements

3.34.2 Choose LangGraph When:

- ✓ Complex workflows with extensive state management
- ✓ Multiple conditional branches requiring state checks
- ✓ Long-running processes with persistent data
- ✓ Enterprise applications with robust state requirements
- ✓ Teams needing standardized state management patterns

3.35 Key Takeaways

1. **State is Critical:** Complex workflows cannot function without proper state management
2. **Manual vs Automatic:** LangChain requires manual state handling, LangGraph provides built-in support
3. **Scalability Factor:** As workflow complexity increases, state management becomes exponentially more important
4. **Development Experience:** Stateful architecture dramatically improves code maintainability and team collaboration

3.36 Next Challenge Preview

With control flow complexity and state handling understood, the next challenge will explore another critical limitation of LangChain when dealing with complex workflows. Each challenge builds upon the previous understanding, creating a comprehensive framework for choosing the right tool.

3.37 Challenge #3: Event Driven Execution - LangChain vs LangGraph

3.38 Understanding Execution Models

3.38.1 Two Fundamental Execution Types

Any workflow, simple or complex, can execute in two ways:

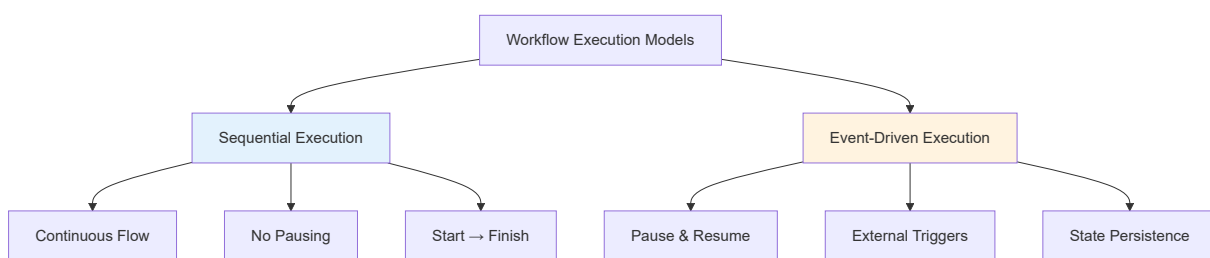


Figure 3.22. Two Fundamental Execution Types

3.39 Sequential Execution Model

3.39.1 Traditional LangChain Workflow



Figure 3.23. Traditional LangChain Workflow

3.39.2 Characteristics:

- ✓ **Continuous Flow:** Left to right execution without stopping
- ✓ **Immediate Transitions:** When Block 1 finishes, Block 2 starts immediately
- ✓ **No Interruptions:** Execution flow never pauses or waits
- ✓ **Start-to-Finish:** Once started, runs until completion

3.39.3 Example Sequential Chain:

```

# Traditional LangChain sequential execution
user_prompt = "Analyze this document"
response1 = llm1.invoke(user_prompt) # Executes immediately
processed = process_response(response1) # Executes immediately
second_prompt = create_prompt(processed) # Executes immediately
  
```

```
final_response = llm2.invoke(second_prompt) # Executes immediately
# No waiting, no pausing, continuous execution
```

3.40 Event-Driven Execution Model

3.40.1 Definition

Workflow pauses at specific points and waits for external triggers before resuming execution

3.40.2 Key Components:

1. **Pause Points:** Specific nodes where execution stops
2. **External Triggers:** Events from outside the workflow
3. **State Persistence:** Maintaining workflow state during pause
4. **Resume Capability:** Continuing from exact pause point

3.41 Real-World Event-Driven Examples

3.41.1 Hiring Workflow Pause Points

1. Post JD → 7-Day Wait

Scenario:

- Post job on LinkedIn today
- **Pause workflow** for 7 days
- **External trigger:** Timer event after 7 days
- **Resume:** Monitor applications

2. Modify JD → 48-Hour Wait

Scenario:

- Modify job description for more applications
- **Pause workflow** for 48 hours
- **External trigger:** Timer event after 48 hours
- **Resume:** Check application count again

3. Send Offer → Wait for Response

Scenario:

- Send offer letter to candidate
- **Pause workflow** indefinitely
- **External trigger:** Candidate replies (accept/reject)
- **Resume:** Process acceptance or rejection

3.41.2 Common Trigger Types:

- Time-based:** Wait X days/hours
- Response-based:** Wait for email reply
- Threshold-based:** Wait for minimum applications
- Human approval:** Wait for manual decision

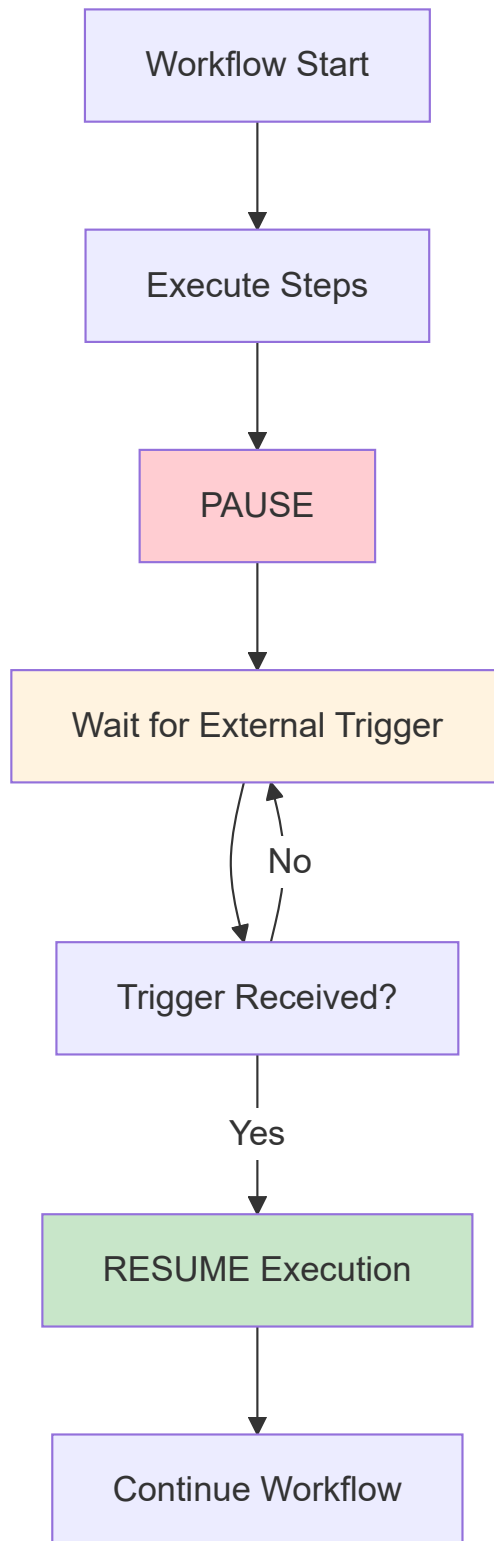


Figure 3.24. Definition

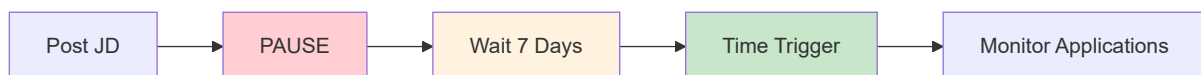
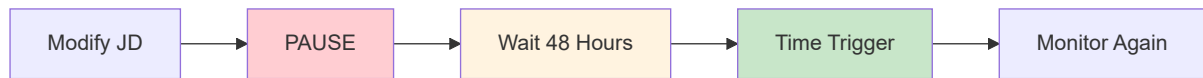
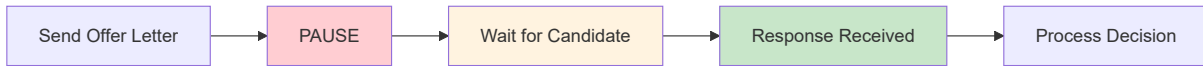


Figure 3.25. Post JD $\rightarrow$ 7-Day Wait

Figure 3.26. Modify JD $\rightarrow$ 48-Hour WaitFigure 3.27. Send Offer $\rightarrow$ Wait for Response

-API callback: Wait for external system response

3.42 ✗ LangChain's Sequential Limitation

3.42.1 Core Design Problem

LangChain is NOT built for event-driven execution

3.42.2 Why LangChain Struggles:

- ✗ **Sequential Design**: Built for start-to-finish execution
- ✗ **No Pause Capability**: Chains cannot stop mid-execution
- ✗ **No Resume Feature**: Cannot continue from pause points
- ✗ **Stateless Nature**: Cannot maintain state during waits

3.42.3 Required Workaround (Problematic):

```

# LangChain Event-Driven Attempt (BAD APPROACH)

# Chain 1: Up to pause point
def chain_1():
    jd = create_jd_chain.invoke(request)
    post_jd(jd)
    # Chain ends here - cannot pause for 7 days
    save_state_manually({"jd": jd, "posted": True})

# External wait logic (GLUE CODE)
def wait_7_days():
    import time
    time.sleep(7 * 24 * 60 * 60) # 7 days in seconds
    # OR use external scheduler/cron job

# Chain 2: After wait period
def chain_2():
    state = load_state_manually() # MANUAL STATE TRANSFER
    applications = monitor_applications()
    # Continue workflow...

# Manual orchestration (MORE GLUE CODE)
chain_1()
wait_7_days()
chain_2()
  
```

3.42.4 Problems with LangChain Approach:

1. **Multiple Separate Chains**: Break workflow into pieces
2. **External Wait Logic**: Custom Python timing code

3. **Manual State Transfer:** Save/load state between chains
4. **Complex Orchestration:** Custom coordination code
5. **Error-Prone:** Many failure points
6. **Maintenance Nightmare:** Lots of glue code

3.43 ✓ LangGraph's Event-Driven Solution

3.43.1 Built-in Event-Driven Architecture

LangGraph inherently provides event-driven execution capabilities

3.43.2 Key Features:

1. Checkpoint System

```
# LangGraph checkpoint feature
graph = create_graph()

# Execute until pause point
state = graph.invoke(initial_state)

# Save current state (checkpoint)
checkpoint_id = graph.save_checkpoint(state)

# Pause workflow - can wait days/weeks/months

# Resume from exact point
graph.resume_from_checkpoint(checkpoint_id, new_trigger_data)
```

2. State Persistence Options

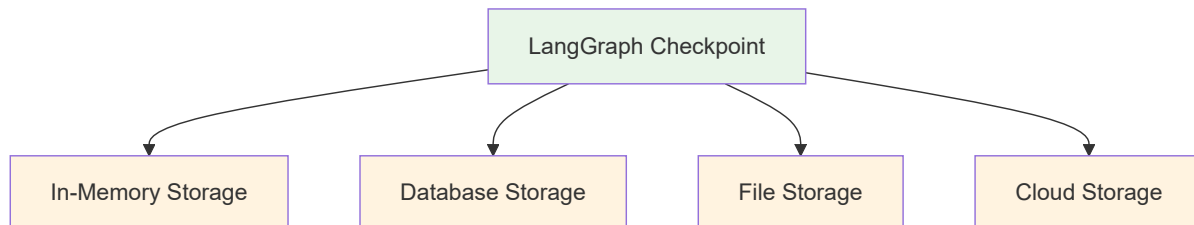


Figure 3.28. State Persistence Options

3. Seamless Resume Capability

```
# LangGraph event-driven implementation
def hiring_workflow_node(state):
    if state["step"] == "post_jd":
        # Post the job
        post_jd(state["jd"])

        # Set up wait for 7 days
        return {
            **state,
            "jd_posted": True,
            "wait_until": datetime.now() + timedelta(days=7),
            "next_step": "monitor_applications"
        }
```

```
elif state["step"] == "monitor_applications":
    # Resume after 7 days
    applications = check_applications()
    return {
        **state,
        "applications": applications,
        "step": "evaluate_applications"
    }

# Workflow naturally pauses and resumes
# No manual chain splitting needed
# No glue code required
```

3.44 Event-Driven Flow Comparison

3.44.1 LangChain Approach (Complex & Error-Prone):

3.44.2 LangGraph Approach (Clean & Elegant):

3.45 Technical Implementation Comparison

Table 3.5. Technical Implementation Comparison

Aspect	LangChain	LangGraph
Pause Capability	✗ Not supported	✓ Built-in checkpoints
State Persistence	✗ Manual implementation	✓ Automatic management
Resume Feature	✗ Manual coordination	✓ Native support
Wait Handling	✗ External logic required	✓ Framework-managed
Code Complexity	✗ High (lots of glue code)	✓ Low (clean patterns)
Error Handling	✗ Manual at each break	✓ Framework-level
Scaling	✗ Becomes unmanageable	✓ Handles any complexity

3.46 Real-World Scenarios Requiring Event-Driven Execution

3.46.1 Enterprise Applications:

- **Approval Workflows:** Wait for human decisions
- **Integration Processes:** Wait for external system callbacks
- **Batch Processing:** Wait for scheduled triggers
- **Monitoring Systems:** Wait for threshold conditions

3.46.2 AI Agent Behaviors:

- **Long-running Tasks:** Multi-day research projects
- **Interactive Systems:** Wait for user input
- **Scheduled Operations:** Time-based automations
- **Callback Handling:** API response processing

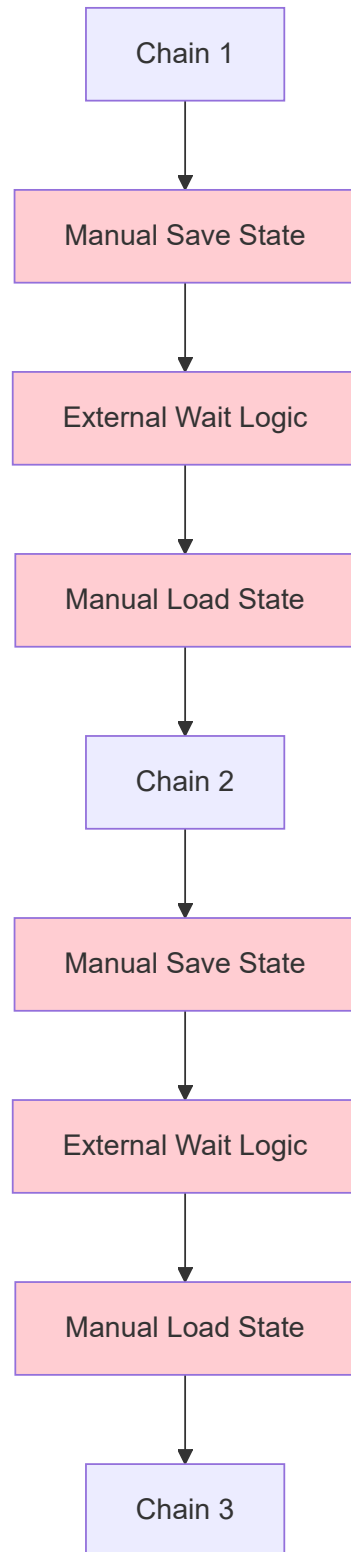


Figure 3.29. *LangChain Approach (Complex & Error-Prone)*

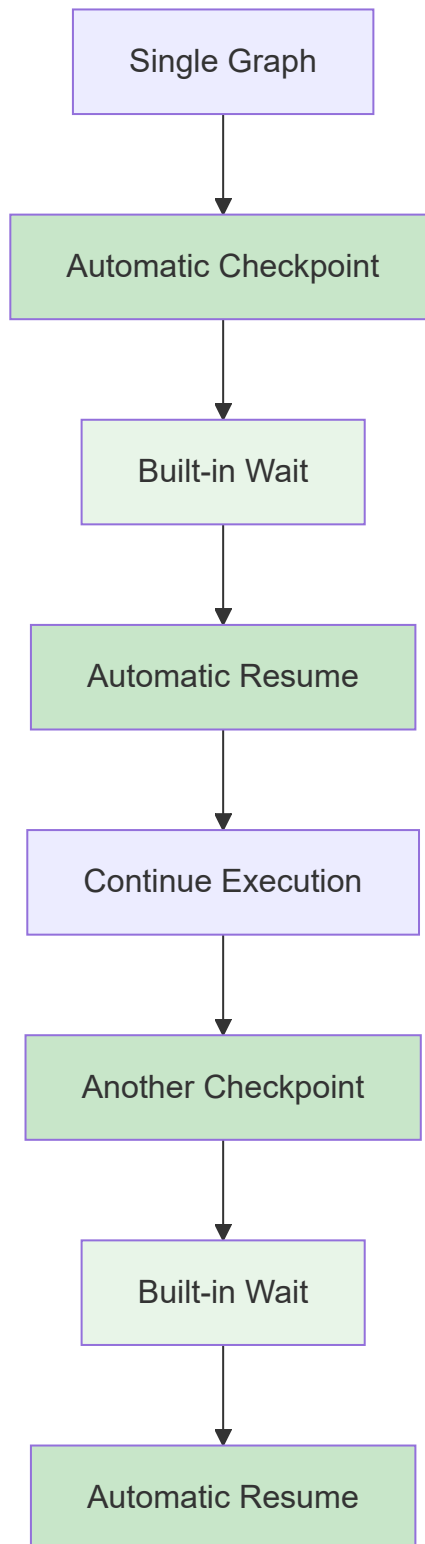


Figure 3.30. *LangGraph Approach (Clean & Elegant)*

3.46.3 Complex Workflows:

- **Multi-stage Processes:** Manufacturing, recruitment, onboarding
- **Compliance Workflows:** Review and approval cycles
- **Data Pipelines:** Wait for data availability
- **Customer Journeys:** Response-dependent paths

3.47 Decision Framework

3.47.1 Use Sequential Execution (LangChain) When:

- ✓ Simple, linear workflows
- ✓ No waiting periods required
- ✓ All processing can happen immediately
- ✓ No external dependencies with delays

3.47.2 Use Event-Driven Execution (LangGraph) When:

- ✓ Workflows have natural pause points
- ✓ External triggers are required
- ✓ Long waiting periods are involved
- ✓ Human-in-the-loop interactions
- ✓ Multi-day/week processes
- ✓ Complex enterprise workflows

3.48 Key Insights

3.48.1 Architectural Implications:

- **Sequential $\neq$ Real World:** Most real workflows have waits and dependencies
- **State Persistence:** Event-driven requires robust state management
- **Framework Design:** Tool choice impacts implementation complexity dramatically

3.48.2 Development Impact:

- **LangChain:** Forces artificial breaking of natural workflows
- **LangGraph:** Allows natural workflow modeling
- **Maintenance:** Event-driven capabilities reduce technical debt

3.48.3 Business Value:

- **Realistic Modeling:** Workflows match real business processes
- **Scalability:** Handle enterprise-level complexity
- **Reliability:** Built-in state management reduces failures

3.49 Next Challenge Preview

With execution models understood, the next challenge will explore another critical limitation when building complex workflows. Each challenge builds upon previous understanding, creating a comprehensive picture of when and why to choose each framework.

3.50 Nested Workflows Feature & Challenge #6: Observability

3.51 LangGraph Feature: Nested Workflows (Subgraphs)

3.51.1 Definition

Nested Workflows = Ability to create workflows within workflows, where individual nodes can themselves be complete graphs

3.51.2 Core Concept: Graph within Graph

3.51.3 Official Definition:

“A subgraph is a graph that is used as a node in another graph”

3.51.4 Encapsulation Concept Applied to Workflows

- **Main Graph:** High-level workflow
- **Subgraph:** Detailed implementation of complex nodes
- **Encapsulation:** Hide complexity while maintaining functionality

3.52 Real-World Example: Interview Process

3.52.1 Before Nested Workflows:

Problem: “Conduct Interview” is oversimplified - it’s actually very complex!

3.52.2 After Nested Workflows:

Benefits:

- **Clarity:** Main workflow remains readable
- **Detail:** Complex process properly modeled
- **Maintainability:** Changes to interview process isolated
- **Reusability:** Interview subgraph can be reused elsewhere

3.53 Two Major Use Cases

3.53.1 1. Multi-Agent Systems

Self-Driving Car Example:

Architecture Benefits:

- **Specialized Agents:** Each handles specific domain
- **Hierarchical Control:** CEO agent orchestrates others
- **Scalable Design:** Add new agents without disrupting existing ones
- **Fault Isolation:** Agent failures don’t crash entire system

Multi-Agent System Applications:

- **Enterprise Automation:** Different agents for different departments
- **Content Creation:** Writer agent, editor agent, fact-checker agent
- **Trading Systems:** Market analysis agent, risk assessment agent, execution agent

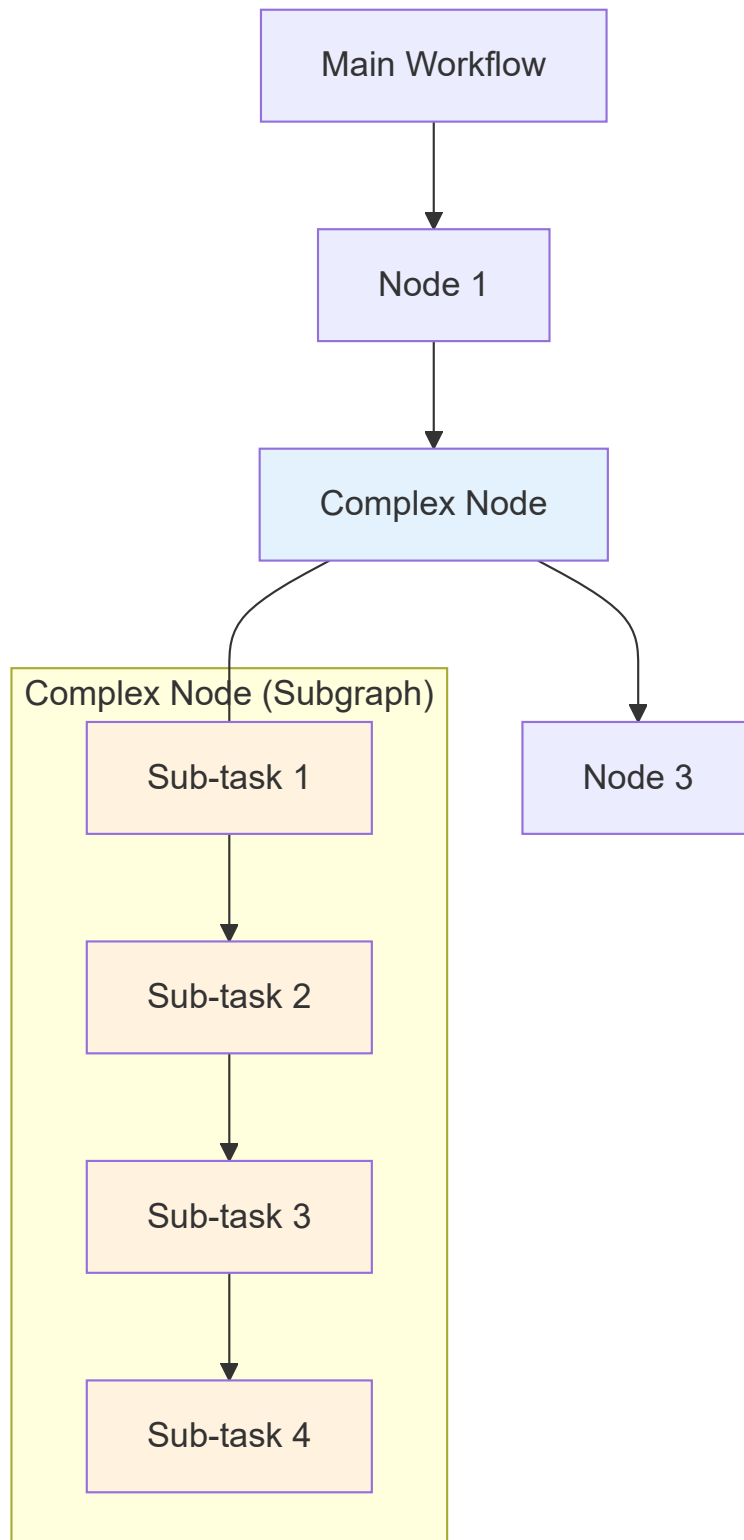


Figure 3.31. Core Concept: Graph within Graph

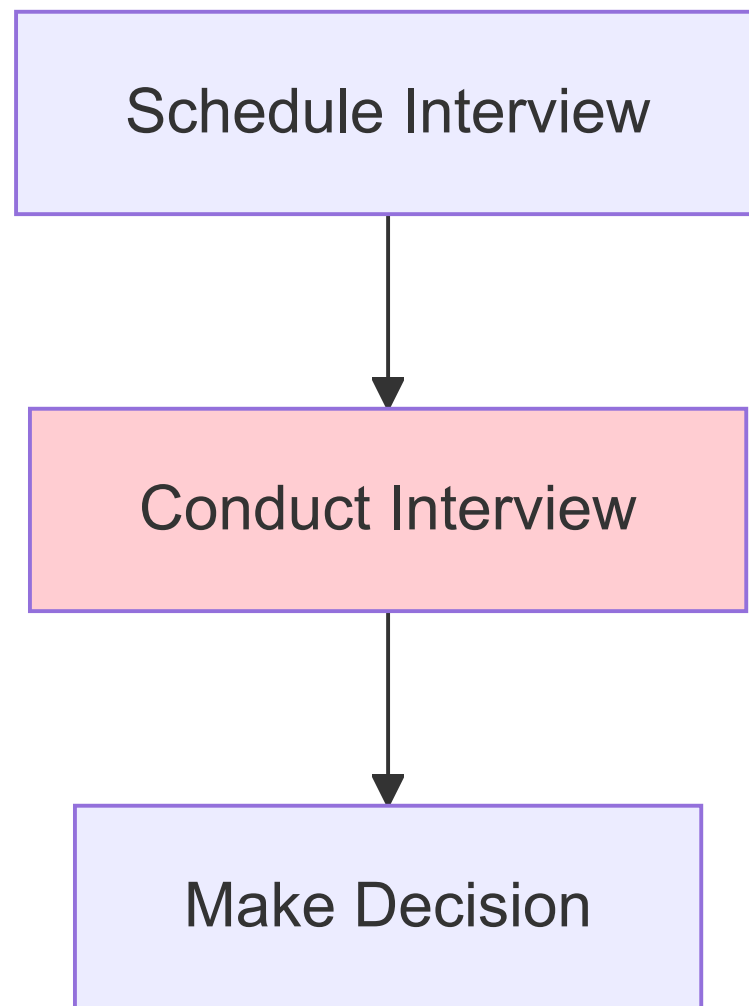


Figure 3.32. Before Nested Workflows

- **Customer Service:** Routing agent, technical support agent, billing agent

3.53.2 2. Reusability

Approval Workflow Example:

Reusability Benefits:

- **DRY Principle:** Don't Repeat Yourself for workflows
- **Consistent Logic:** Same approval process everywhere
- **Easy Updates:** Change approval logic in one place
- **Quality Assurance:** Well-tested subgraph reduces bugs

Programming Analogy:

```
# Traditional Programming
def function_a():
    # Reusable logic
    return result
```

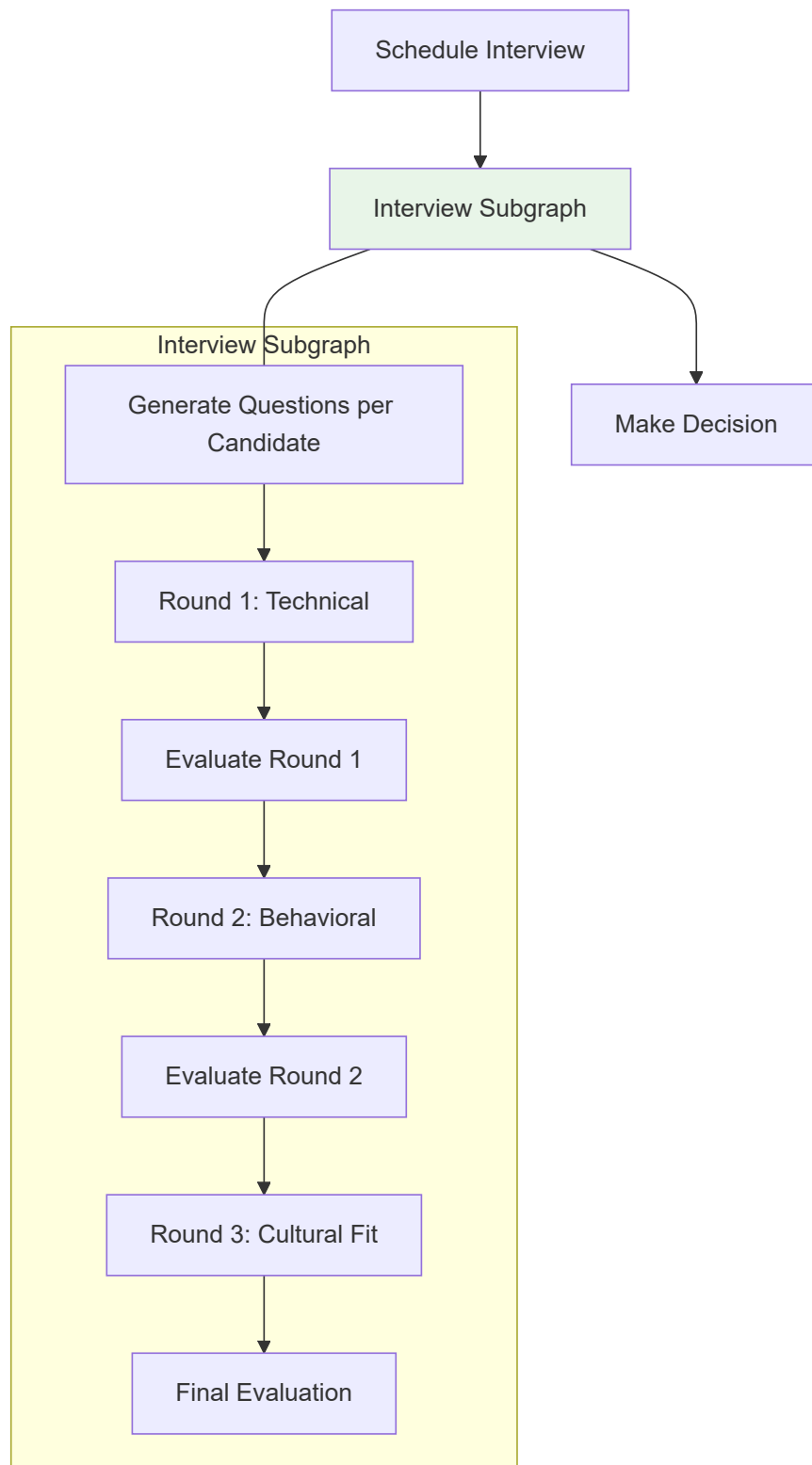


Figure 3.33. *After Nested Workflows*

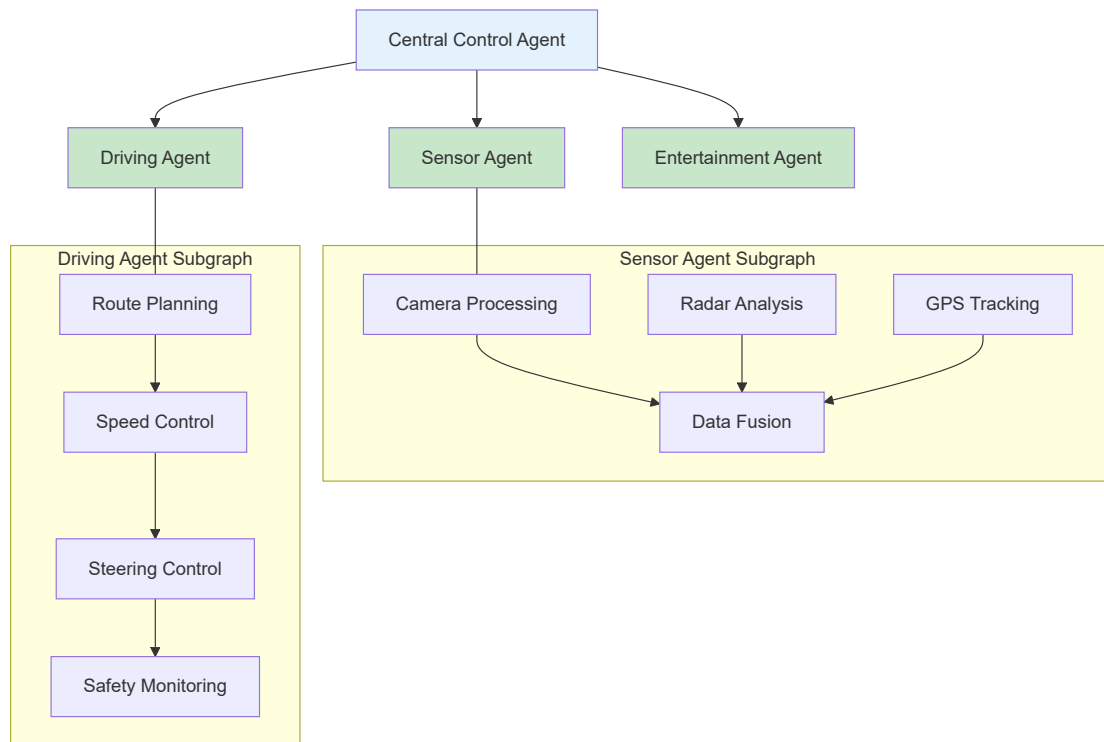


Figure 3.34. Self-Driving Car Example

```

def function_b():
    result = function_a() # Reuse
    # Continue processing

# Workflow Programming (LangGraph)
approval_subgraph = create_approval_workflow()

main_workflow.add_subgraph("approval_1", approval_subgraph)
main_workflow.add_subgraph("approval_2", approval_subgraph) # Reuse
main_workflow.add_subgraph("approval_3", approval_subgraph) # Reuse
  
```

3.54 X LangChain's Nested Workflow Limitation

3.54.1 The Fundamental Problem:

"If you're struggling to build this simple workflow in LangChain, then building a workflow within a workflow is absolutely not possible"

3.54.2 Why LangChain Cannot Support Nested Workflows:

1. **Struggles with Single Workflows:** Already difficult to build complex workflows
2. **No Graph Concept:** Linear chains cannot contain other chains naturally
3. **State Management Issues:** Cannot handle nested state interactions
4. **Glue Code Explosion:** Would require exponentially more custom code

3.54.3 LangChain Reality Check:

```

# LangChain approach would require
main_chain = create_complex_chain() # Already difficult
  
```

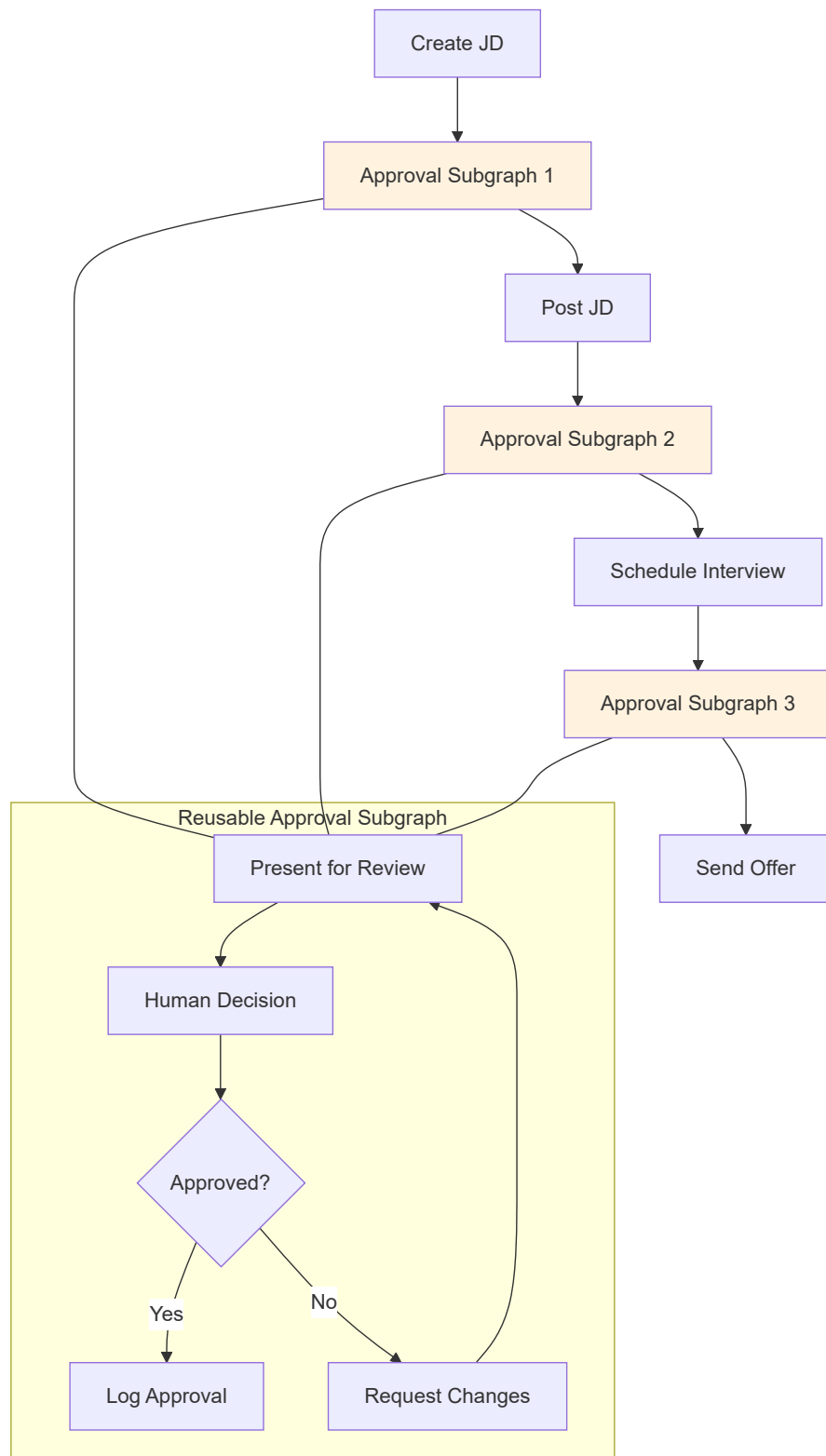


Figure 3.35. *Approval Workflow Example*


```
nested_logic = custom_python_code() # Manual implementation
state_coordination = more_custom_code() # Manual state handling
# Result: Maintenance nightmare
```

3.55 Challenge #6: Observability

3.55.1 Definition

Observability = How easily you can monitor, debug, and understand what your workflow is doing at runtime

3.55.2 Why Observability is Critical:

Production Scenarios:

- **Error Tracking:** What went wrong and why?
- **Performance Monitoring:** Which steps are slow?
- **Decision Auditing:** Why did the agent make specific choices?
- **Cost Monitoring:** How much are LLM calls costing?
- **Compliance:** Regulatory audit trail requirements

Real-World Example: Costly Mistake

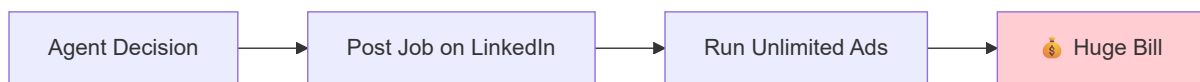


Figure 3.36. Real-World Example: Costly Mistake

Scenario: Agent posts job and runs unlimited ads, creating massive costs **Need:** Audit trail to understand decision-making process

3.56 Observability Solutions

3.56.1 LangSmith: The Monitoring Tool

LangSmith = Specialized tool for monitoring LLM-based applications

LangSmith Capabilities:

- **LLM Call Tracking:** Every prompt and response
- **Token Counting:** Usage and cost monitoring
- **Response Timing:** Performance analysis
- **Error Logging:** Failure point identification
- **Chain Visualization:** Workflow execution flow

3.57 X LangChain's Observability Limitation

3.57.1 Partial Observability Problem:

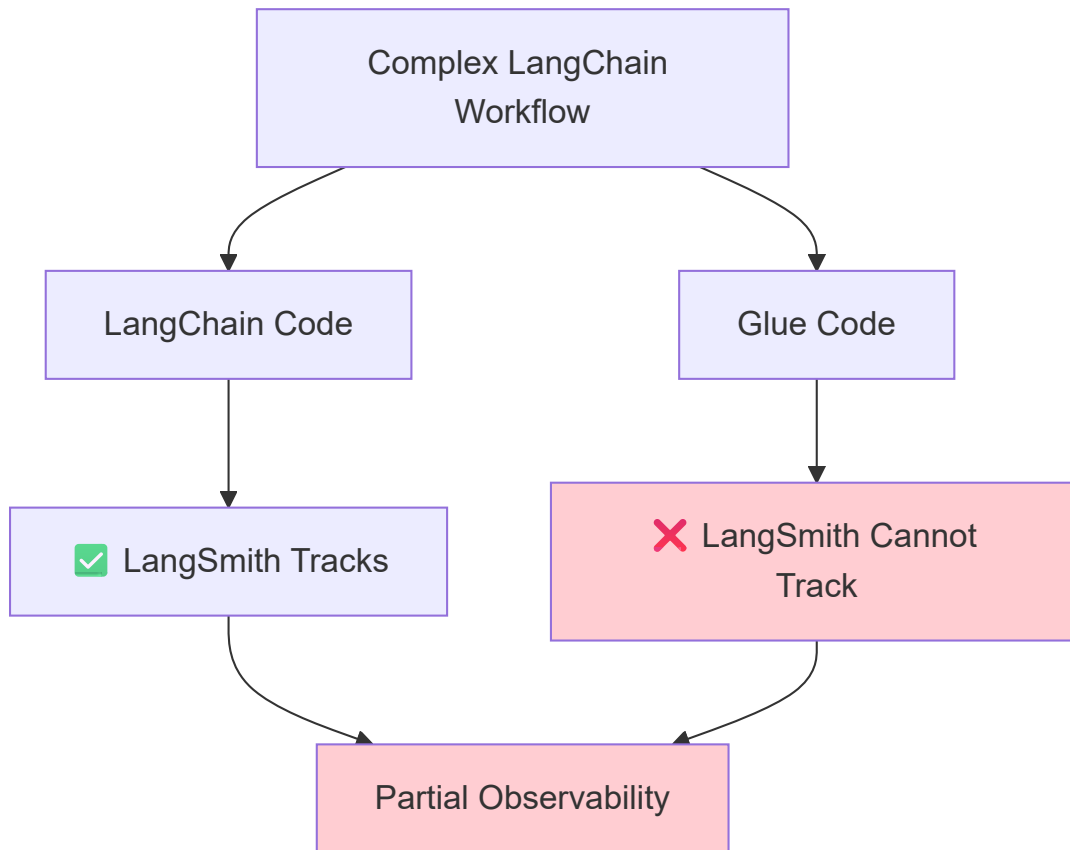


Figure 3.37. Partial Observability Problem

3.57.2 The Core Problem:

```

# LangChain with glue code
approved = False
iteration = 0

while not approved: # GLUE CODE - Not tracked
    iteration += 1 # GLUE CODE - Not tracked

    # This LLM call IS tracked by LangSmith
    jd_output = jd_chain.invoke(input)

    # This logic is NOT tracked
    approved = check_approval(jd_output) # GLUE CODE - Not tracked

    if not approved: # GLUE CODE - Not tracked
        print(f"Iteration {iteration} failed") # GLUE CODE - Not tracked

# LangSmith sees: LLM calls only
# LangSmith misses: Loop iterations, approval logic, decision flow
  
```

3.57.3 What LangSmith Misses in LangChain:

- ✗ **Loop Iterations:** How many times did we retry?
- ✗ **Custom Logic:** Why did approval fail?
- ✗ **State Changes:** How did variables evolve?
- ✗ **Control Flow:** Why did we take specific branches?
- ✗ **Complete Timeline:** What was the full execution sequence?

3.57.4 Result: Partial Observability

- ✓ Can see LLM interactions
- ✗ Cannot see complete workflow logic
- ✗ Cannot understand full decision-making process
- ✗ Cannot debug glue code issues

3.58 ✓ LangGraph's Complete Observability

3.58.1 Tight LangSmith Integration:

LangGraph has very tight integration with LangSmith

3.58.2 What Gets Tracked:

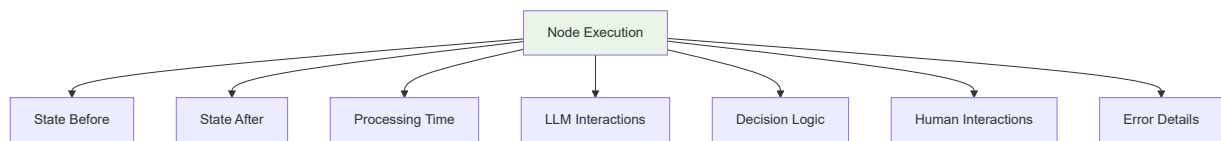


Figure 3.38. What Gets Tracked

3.58.3 Complete Timeline Creation:

3.58.4 Comprehensive Tracking:

1. **Node-to-Node Flow:** Complete execution path
2. **State Evolution:** How state changes at each step
3. **Human Interactions:** When humans intervened and what they decided
4. **LLM Communications:** All prompts and responses
5. **API Calls:** External service interactions
6. **Error Contexts:** Exactly where and why failures occurred
7. **Performance Metrics:** Timing for each operation

3.58.5 No Glue Code = Complete Visibility:

```

# LangGraph - Everything is tracked
def approval_node(state):
    # LangSmith tracks: Node entry, state input
    jd_content = state["jd"]

    # LangSmith tracks: Human interaction request
    approval = request_human_approval(jd_content)
  
```

Workflow Execution Timeline

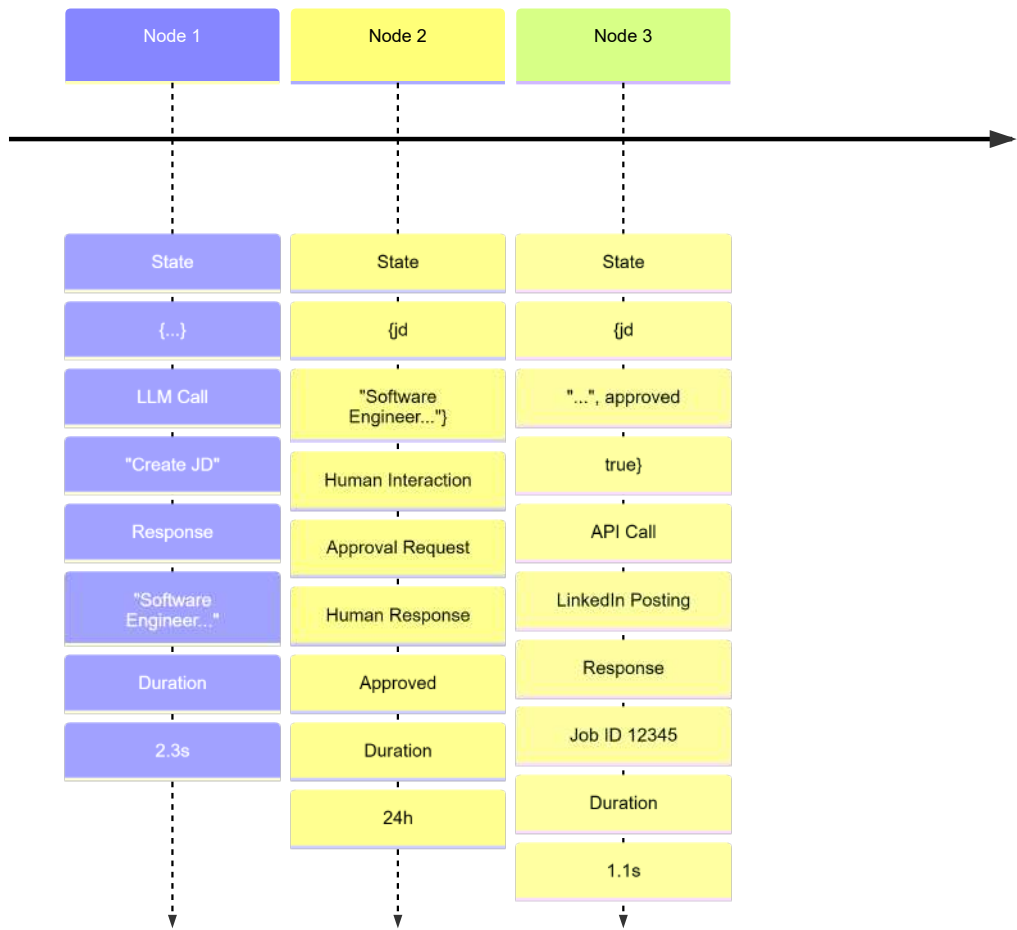


Figure 3.39. Complete Timeline Creation

```
# LangSmith tracks: State update, node exit
return {**state, "approved": approval}

# LangSmith sees: EVERYTHING
# - When node started/ended
# - Input state values
# - Human interaction details
# - Output state changes
# - Complete workflow context
```

3.59 Observability Comparison

Table 3.6. *Observability Comparison*

Aspect	LangChain + LangSmith	LangGraph + LangSmith
LLM Call Tracking	✓ Complete	✓ Complete
Custom Logic Tracking	✗ Not visible	✓ Complete
State Evolution	✗ Manual tracking	✓ Automatic
Control Flow Visibility	✗ Partial	✓ Complete
Human Interaction Logs	✗ Manual implementation	✓ Built-in
Error Context	! Limited	✓ Full context
Timeline Reconstruction	✗ Impossible	✓ Complete
Production Debugging	! Difficult	✓ Straightforward

3.60 Production Impact

3.60.1 For Complex Applications:

- **LangChain:** Partial observability = Difficult debugging
- **LangGraph:** Complete observability = Easy debugging

3.60.2 Business Consequences:

- **Audit Requirements:** Regulatory compliance needs complete trails
- **Error Resolution:** Faster debugging = Less downtime
- **Cost Management:** Better monitoring = Cost optimization
- **Quality Assurance:** Complete visibility = Better reliability

3.60.3 Development Experience:

- **LangChain:** “Why did this fail?” → Hard to answer
- **LangGraph:** “Why did this fail?” → Complete audit trail available

3.61 Key Insights

3.61.1 The Observability Gap:

1. **Glue Code Creates Blind Spots:** What you can’t see, you can’t debug
2. **Production Requires Visibility:** Real applications need complete monitoring
3. **Framework Choice Affects Debuggability:** Architecture impacts observability

3.61.2 Advanced Concept Preview:

- **Integration Details:** How to set up LangSmith with LangGraph
- **Custom Monitoring:** Adding additional observability layers
- **Performance Optimization:** Using observability data for improvements

3.62 Final Framework Decision Matrix

3.62.1 Choose LangChain When:

- ✓ Simple, linear workflows
- ✓ Prototype/demo applications
- ✓ Short-running processes
- ✓ Minimal observability needs
- ✓ Single-user applications

3.62.2 Choose LangGraph When:

- ✓ Complex, non-linear workflows
 - ✓ Production applications
 - ✓ Long-running processes
 - ✓ Multi-agent systems needed
 - ✓ Complete observability required
 - ✓ Human-in-the-loop workflows
 - ✓ Fault tolerance essential
 - ✓ Nested workflows beneficial
 - ✓ Enterprise-grade reliability needed
-

3.63 Complete Challenge Summary

3.63.1 All Challenges Covered:

1. ✓ **Control Flow Complexity:** Branching, loops, jumps
2. ✓ **State Handling:** Stateless vs stateful execution
3. ✓ **Event-Driven Execution:** Pause and resume capabilities
4. ✓ **Fault Tolerance:** Recovery from failures
5. ✓ **Human-in-the-Loop:** Extended human interaction support
6. ✓ **Observability:** Complete monitoring and debugging

3.63.2 Plus Key Feature:

- ✓ **Nested Workflows:** Multi-agent systems and reusability

3.63.3 The Pattern:

Every challenge reveals the same truth: **LangGraph's stateful, graph-based architecture enables capabilities that are impossible or impractical with LangChain's stateless, sequential approach.**

3.64 Conclusion & Revision: LangChain vs LangGraph Complete Summary

3.65 Quick Revision Questions

3.65.1 1. What is LangGraph? (Complete Definition)

LangGraph is an orchestration framework that enables you to build stateful, multi-step, and event-driven workflows using LLMs. It's ideal for designing both single-agent and multi-agent agentic AI applications.

3.65.2 Key Terms Breakdown:

- ✓ **Orchestration Framework:** Coordinates complex workflows
- ✓ **Stateful:** Maintains and tracks state throughout execution
- ✓ **Multi-step:** Handles complex, sequential processes
- ✓ **Event-driven:** Supports pause/resume and external triggers
- ✓ **Single & Multi-agent:** Supports various AI architectures

3.65.3 Simplified Mental Model:

Think of LangGraph as a flowchart engine for LLMs

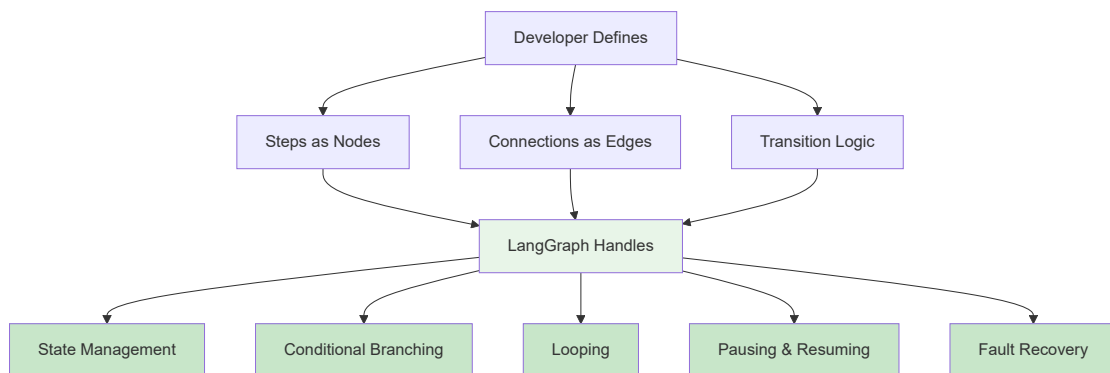


Figure 3.40. Simplified Mental Model

You Define: The workflow structure and logic **LangGraph Handles:** All the complex execution features essential for robust, production-grade AI systems

3.66 When to Use What? (Decision Framework)

3.66.1 Use LangChain When:

Characteristics:

- ✓ Linear, sequential processing
- ✓ Simple input → output transformations
- ✓ Minimal state requirements
- ✓ Short execution times
- ✓ Prototype/demo applications

Example Use Cases:

- Document summarization

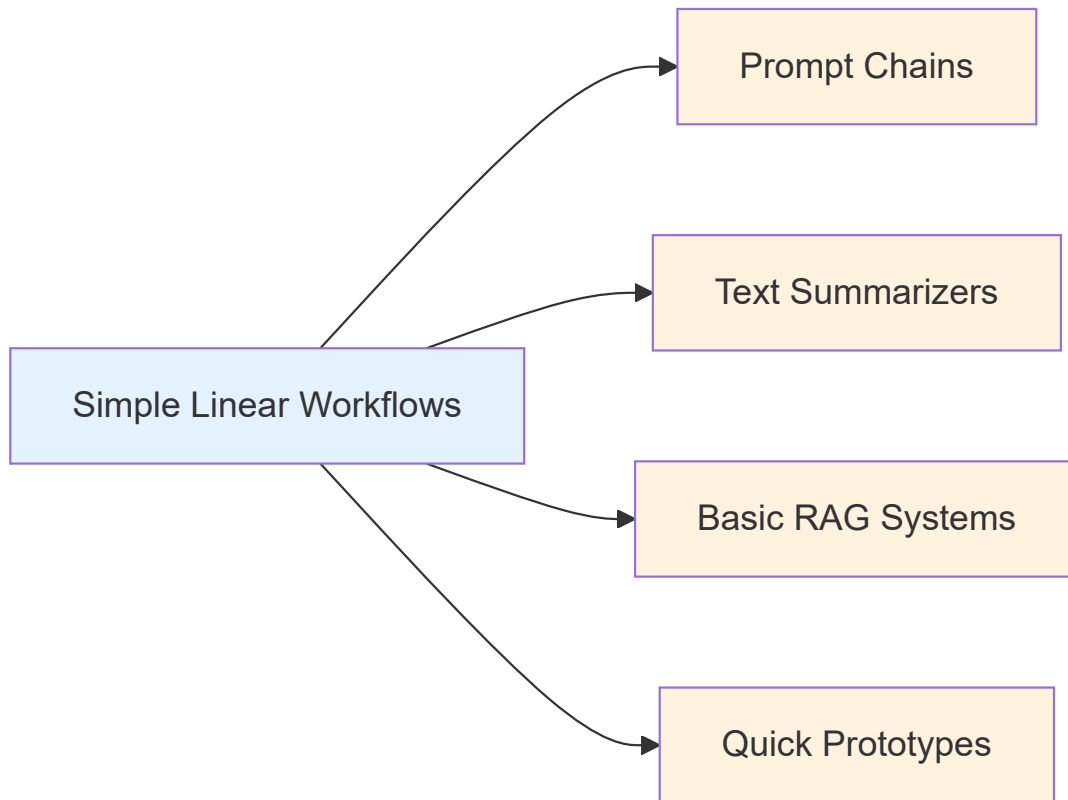


Figure 3.41. Use LangChain When

- Q&A chatbots
- Content generation
- Basic document search
- Simple data processing chains

3.66.2 Use LangGraph When:

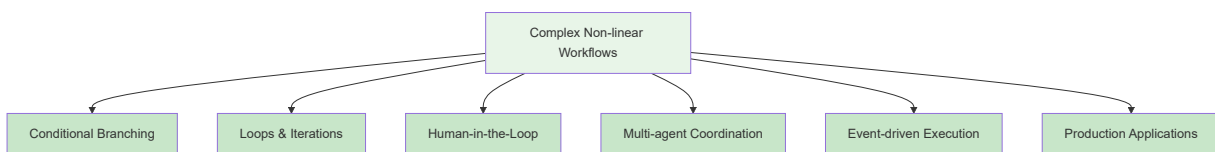


Figure 3.42. Use LangGraph When

Characteristics:

- ✓ Non-linear, branching workflows
- ✓ Complex state management needed
- ✓ Long-running processes
- ✓ Human approval requirements
- ✓ Fault tolerance essential
- ✓ Production-grade reliability

Example Use Cases:

- Automated hiring workflows

- Complex business process automation
- Multi-agent AI systems
- Enterprise applications
- Customer journey automation
- Research and analysis workflows

3.67 Critical Question: Should We Abandon LangChain?

3.67.1 X Answer: NO! LangChain is Still Essential

“LangGraph is built ON TOP OF LangChain”

3.67.2 The Relationship Explained:

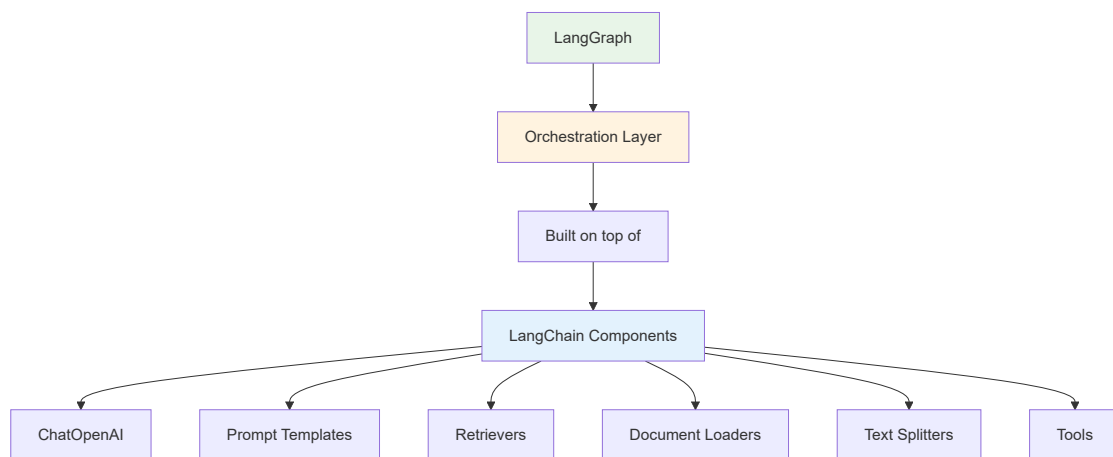


Figure 3.43. The Relationship Explained

3.67.3 LangChain Still Provides:

- LLM Integrations: ChatOpenAI, Claude, etc.
- Prompt Templates: Structured prompt management
- Retrievers: Document search capabilities
- Document Loaders: File processing tools
- Text Splitters: Content chunking utilities
- Tools: API integrations and function calling

3.67.4 LangGraph Provides:

- Orchestration: Workflow coordination
- Complex Flow Control: Branching, loops, conditions
- State Management: Stateful execution
- Event-driven Features: Pause/resume capabilities
- Fault Tolerance: Recovery mechanisms

3.68 How They Work Together

3.68.1 Complementary Roles:

Table 3.7. *Complementary Roles*

Aspect	LangChain	LangGraph
Purpose	Component Library	Orchestration Framework
Focus	Individual Tools	Workflow Coordination
Scope	Building Blocks	System Architecture
Use Case	Simple Chains	Complex Workflows

3.68.2 Real Implementation Example:

```
# Using Both Together
from langchain.llms import ChatOpenAI          # LangChain component
from langchain.prompts import PromptTemplate    # LangChain component
from langchain.tools import DuckDuckGoSearch   # LangChain component
from langgraph import StateGraph               # LangGraph orchestration

# LangChain provides the components
llm = ChatOpenAI(model="gpt-4")
prompt = PromptTemplate(template="...")
search_tool = DuckDuckGoSearch()

# LangGraph orchestrates them into complex workflows
def research_node(state):
    # Using LangChain components within LangGraph nodes
    search_results = search_tool.run(state["query"])
    analysis = llm.invoke(prompt.format(results=search_results))
    return {"research": analysis}

# LangGraph coordinates the overall workflow
workflow = StateGraph(WorkflowState)
workflow.add_node("research", research_node)
# ... more complex orchestration
```

3.68.3 The Division of Labor:

- **LangChain:** “Here are the tools you need”
- **LangGraph:** “Here’s how to coordinate them in complex ways”

3.69 Key Insights & Learning Value

3.69.1 Your LangChain Knowledge is NOT Wasted:

- ✓ **Components Still Used:** All LangChain components remain relevant
- ✓ **Skills Transfer:** LangChain knowledge enhances LangGraph usage
- ✓ **Progressive Learning:** LangChain → LangGraph is natural progression
- ✓ **Combined Power:** Best results come from using both together

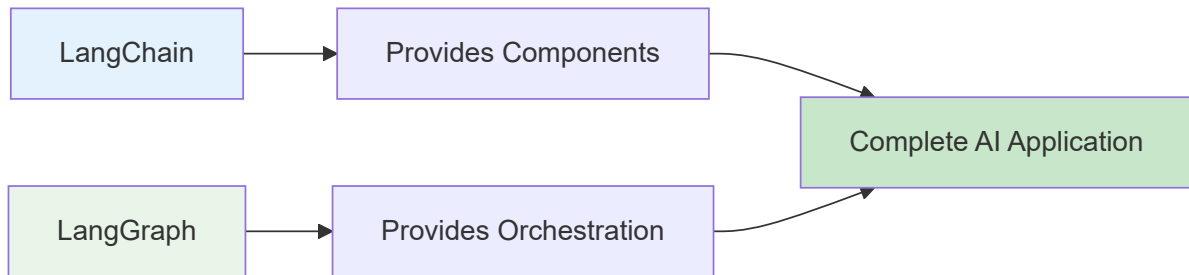


Figure 3.44. *Different Purposes, Same Ecosystem*

3.69.2 Different Purposes, Same Ecosystem:

3.69.3 Evolution Path:

1. **Learn LangChain:** Understand components and simple chains
2. **Recognize Limitations:** Hit complexity walls with real applications
3. **Adopt LangGraph:** Use for complex orchestration needs
4. **Combine Both:** Leverage strengths of each framework

3.70 Big Picture Understanding

3.70.1 Why This Framework Exists:

- **Real-world Complexity:** Business processes are inherently complex
- **AI System Requirements:** Production AI needs robust orchestration
- **Scalability Needs:** Simple chains don't scale to enterprise requirements
- **Developer Experience:** Need tools that match problem complexity

3.70.2 The “Why” Matters:

“Until you have the answer to ‘why’, you won’t enjoy the upcoming videos”

Understanding the fundamental need for LangGraph provides:

- Clear Purpose:** Know when and why to use each tool
- Motivation:** Excitement for building real applications
- Context:** Framework for interpreting advanced features
- Confidence:** Make informed architectural decisions

3.71 What’s Coming Next

3.71.1 Upcoming Video Topics:

- **Hands-on LangGraph Development:** Building real workflows
- **Advanced Features Deep Dive:** Exploring each capability in detail
- **Production Deployment:** Enterprise-ready implementations
- **Best Practices:** Proven patterns and techniques

3.71.2 Learning Progression:

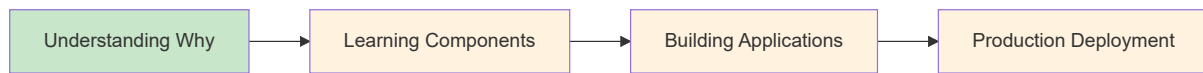


Figure 3.45. Learning Progression

3.71.3 Expected Outcomes:

- Build Complex AI Applications: Real-world problem solving
 - Make Informed Decisions: Choose right tools for each situation
 - Enterprise Readiness: Production-grade AI system development
 - Deep Understanding: Master both frameworks and their synergies
-

3.72 Complete Learning Checklist

3.72.1 ✓ Completed in This Video:

- ☑ **LangChain Recap:** Refreshed on core concepts
- ☑ **Complex Workflow Analysis:** Understood real-world requirements
- ☑ **6 Key Challenges:** Identified LangChain limitations
- ☑ **LangGraph Solutions:** Learned how each challenge is addressed
- ☑ **Decision Framework:** Know when to use each tool
- ☑ **Relationship Understanding:** Both frameworks work together

3.72.2 Next Steps:

- ☐ **Hands-on Practice:** Start building with LangGraph
 - ☐ **Advanced Features:** Deep dive into specific capabilities
 - ☐ **Real Projects:** Apply knowledge to actual use cases
 - ☐ **Production Skills:** Learn deployment and monitoring
-

3.73 Inspiration & Motivation

3.73.1 The Promise Delivered:

“By the end of this video, you’ll automatically understand whether to implement LangChain or LangGraph for any new project”

3.73.2 Your New Superpower:

- Pattern Recognition: Spot complexity requirements instantly
- Tool Selection: Choose appropriate framework confidently
- Architecture Planning: Design systems that scale
- Problem Solving: Tackle complex AI challenges effectively

3.73.3 Ready for Advanced Topics:

With this foundation, you're prepared to:

- Build production-grade AI applications
- Navigate complex workflow requirements
- Make informed architectural decisions
- Contribute to enterprise AI initiatives

3.74 Key Takeaways

1. **LangChain & LangGraph are Complementary:** Different purposes, work together
2. **Choose Based on Complexity:** Simple workflows → LangChain; Complex workflows → LangGraph
3. **Your Knowledge Compounds:** LangChain skills enhance LangGraph usage
4. **Understanding "Why" Enables "How":** Foundation enables advanced learning
5. **Production Reality:** Real applications need robust orchestration frameworks



Figure 3.46. Key Takeaways

LangGraph Core Concepts - Complete Guide

How LangGraph structures workflows with nodes and edges How state, reducers, and orchestrators work together The full execution model that powers agentic AI systems

4.1 1. What is LangGraph?

LangGraph is an **orchestration framework** for building **intelligent, stateful, and multi-step LLM workflows**. In simple terms, when you give LangGraph an LLM workflow to execute, it first represents that workflow in the form of a **graph** and then executes it systematically.

4.1.1 Core Definition

LangGraph converts any LLM workflow into a graph structure where each node represents a task, and edges define the flow of execution between tasks.

4.1.2 How LangGraph Works

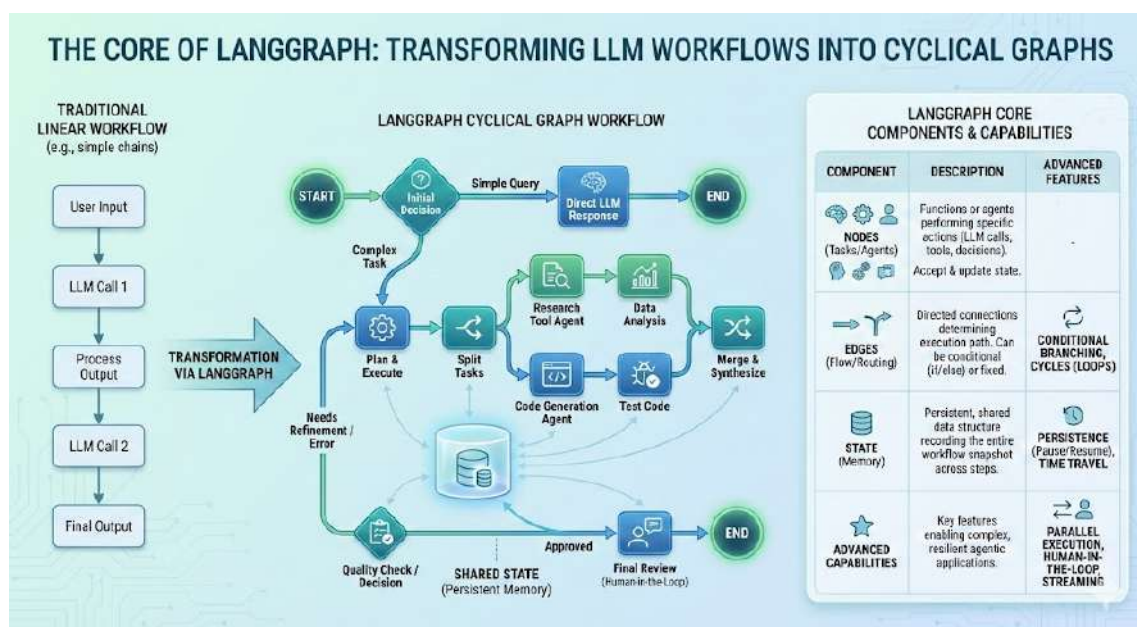


Figure 4.1. How LangGraph Works

When you provide input to the first node and trigger the graph, all nodes automatically execute in their correct order, completing your workflow.

4.1.3 Key Characteristics of a LangGraph Graph

Table 4.1. Key Characteristics of a LangGraph Graph

Component	Description
Nodes	Each node represents a single task in your workflow
Edges	Connections between nodes that define execution order
Task Types	Can include LLM calls, tool calls, decision making, etc.

4.1.4 Advanced Features

LangGraph provides several advanced capabilities that make it ideal for **agentic and production-grade AI applications**:

Table 4.2. Advanced Features

Feature	Description
Parallelism	Execute multiple tasks simultaneously
Loops	Return to previous nodes and execute in cycles
Branching	Conditional execution based on certain conditions
Memory	Record and maintain conversation history and task outputs
Resumability	Resume from a specific point if workflow breaks down

4.1.5 Why Use LangGraph?

LangGraph is particularly suited for:

- Building complex AI agents
- Creating production-grade AI applications
- Implementing workflows that require state management
- Developing systems that need human-in-the-loop interactions

4.2 2. LLM Workflows

4.2.1 What is a Workflow?

A **workflow** is defined as a **series of tasks** that you execute **in order to achieve a goal**.

Workflow = Series of Tasks → Executed in Order → Goal Achievement

4.2.2 Example: Automated Hiring Workflow

Consider an automated hiring system with these sequential steps:

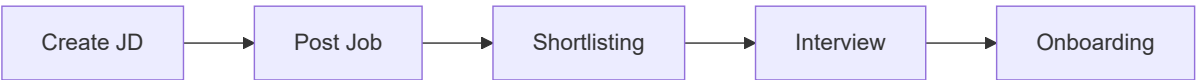


Figure 4.2. Example: Automated Hiring Workflow

Each step must be executed in the correct order to achieve the final goal of hiring a candidate.

4.2.3 What is an LLM Workflow?

An **LLM Workflow** is a workflow where **multiple tasks depend on LLMs** (Large Language Models) for their execution.

In the hiring example:

- **JD Creation** → Requires LLM to generate job description
- **Shortlisting** → May use LLM for resume screening
- **Interviews** → Could use LLM-based assessment

4.2.4 Key Properties of LLM Workflows

Table 4.3. *Key Properties of LLM Workflows*

Property	Description
Step-by-step Process	Sequential execution of distinct tasks
LLM Dependency	Multiple steps require LLM capabilities
Complex Applications	Enable building sophisticated AI systems

4.2.5 Types of Tasks in LLM Workflows

Each step in a workflow can perform distinct tasks such as:

- **Prompting** - Sending queries to LLM
- **Reasoning** - Logical decision making
- **Tool Calling** - Invoking external tools/APIs
- **Memory Access** - Retrieving stored information
- **Decision Making** - Conditional logic execution

4.2.6 Workflow Structure Types

Workflows can be structured in different ways:

Table 4.4. *Workflow Structure Types*

Type	Description	Use Case
Linear	Sequential execution $A \rightarrow B \rightarrow C$	Simple pipelines
Parallel	Simultaneous execution	Independent subtasks
Branched	Conditional paths	Decision-based flows
Looped	Iterative execution	Refinement processes

These structures allow for complex behaviors like:

- Retries on failure
- Multi-agent communication
- Tool-augmented reasoning

4.3 3. Common LLM Workflow Patterns

While every application has its unique workflow, certain patterns appear frequently across different use cases. Understanding these patterns helps in designing effective LLM applications.

4.3.1 3.1 Prompt Chaining

Prompt Chaining is one of the most common LLM workflow patterns where you **call the LLM multiple times in sequence**, with each call building upon the previous one.

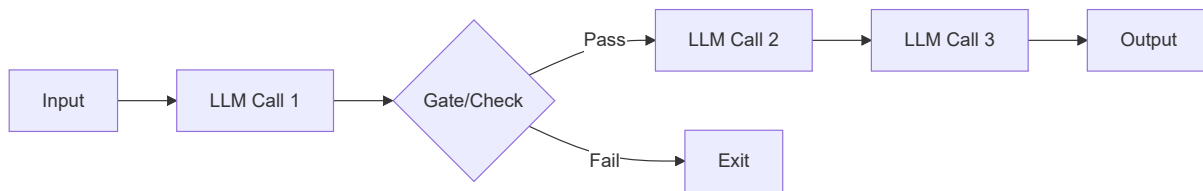


Figure 4.3. 1 Prompt Chaining

4.3.2 How Prompt Chaining Works

1. **Input** arrives at the first LLM
2. **First LLM** processes and generates output
3. **Gate/Check** validates the output (optional)
4. If **Pass**: Output moves to next LLM
5. If **Fail**: Workflow exits or retries
6. Process continues through all chained LLMs

4.3.3 Real-World Example: Report Generation

Scenario: User provides a topic, and you need to create a detailed report.

Instead of directly generating a report (which may be suboptimal), you break it down:

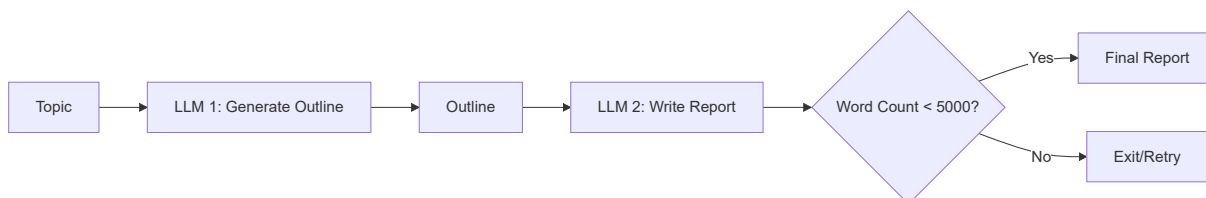


Figure 4.4. Real-World Example: Report Generation

Step 1: Topic → LLM generates outline **Step 2:** Outline → LLM generates detailed report

Validation: Check if report meets criteria (e.g., word count)

4.3.4 When to Use Prompt Chaining

- **Complex tasks** that need to be broken into subtasks
- Tasks requiring **intermediate validation**
- When you want **better control** over output quality
- **Multi-stage content generation** (outline → draft → final)

4.3.5 3.2 Routing

Routing is a pattern where you **analyze a task and decide which specialized handler** should process it.

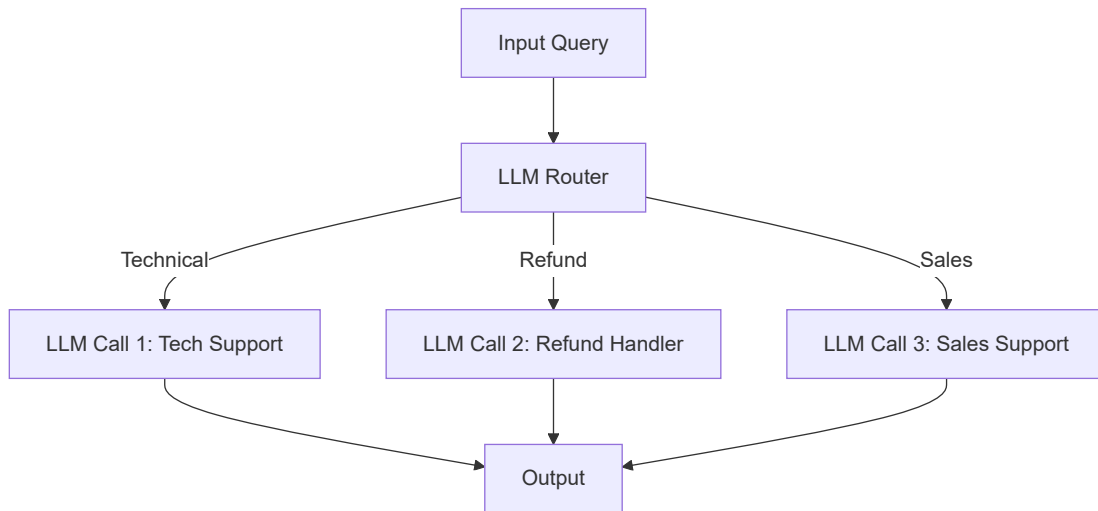


Figure 4.5. 2 Routing

4.3.6 How Routing Works

1. **Query arrives** at the router LLM
2. **Router analyzes** the query to understand its nature
3. **Decision made** about which specialized LLM should handle it
4. **Query forwarded** to the appropriate handler
5. **Handler processes** and returns response

4.3.7 Real-World Example: Customer Support Chatbot

Scenario: Building a customer support chatbot that handles various query types.

Table 4.5. Real-World Example: Customer Support Chatbot

Query Type	Handler	Example Query
Technical	Tech Support LLM	"How do I configure the API?"
Refund	Refund Handler LLM	"I want to return my order"
Sales	Sales Support LLM	"What pricing plans do you offer?"

The **Router LLM** acts as a **decision maker** that determines the most capable LLM for solving each specific query.

4.3.8 When to Use Routing

- **Multi-domain applications** with specialized handlers
- **Customer support systems** with varied query types
- When different queries need **different expertise**
- To **optimize response quality** by using specialized models

4.3.9 3.3 Parallelization

Parallelization involves **breaking a task into multiple subtasks** that execute **simultaneously**, then **merging their results** to produce the final output.

4.3.10 How Parallelization Works

1. **Task divided** into independent subtasks
2. **All subtasks execute** simultaneously (in parallel)
3. **Results collected** from all parallel executions⁸⁷
4. **Aggregator merges** all results
5. **Final outcome** produced from merged results

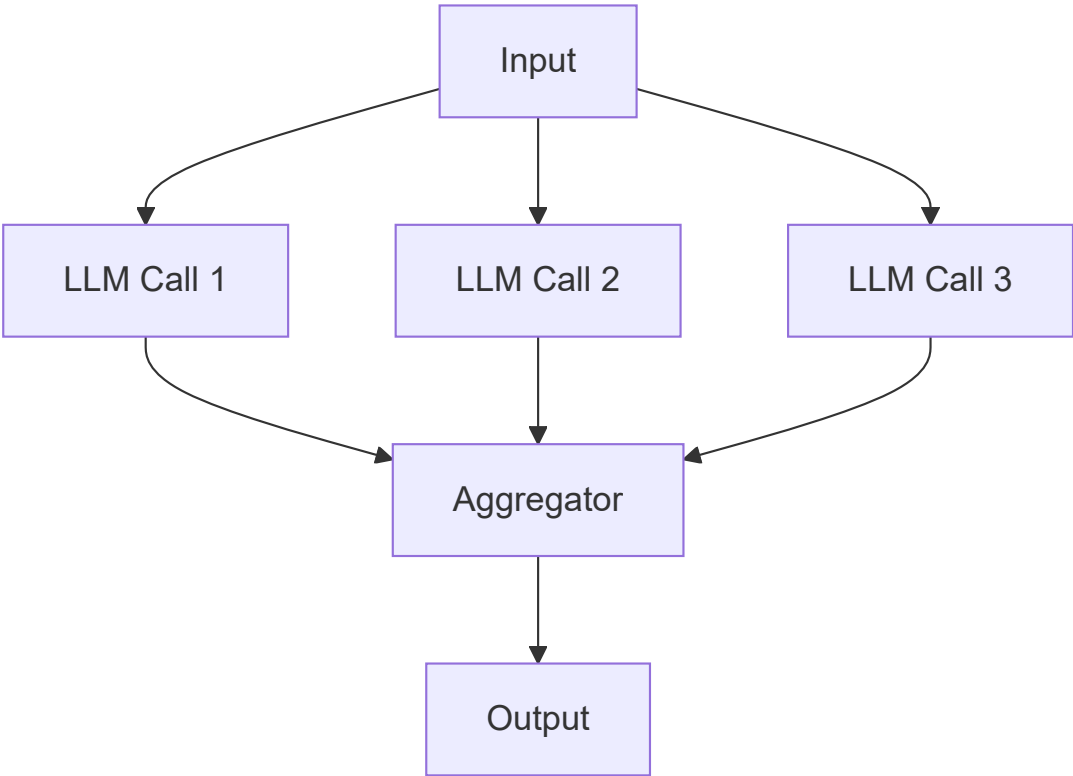


Figure 4.6. 3 Parallelization

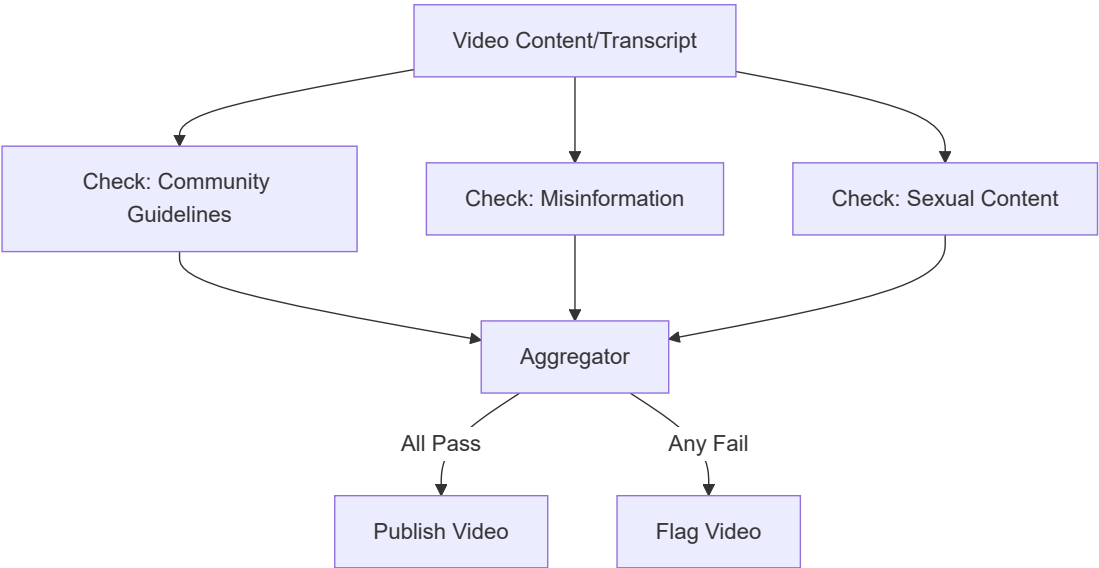


Figure 4.7. Real-World Example: YouTube Content Moderation

Key Insight: All three checks can run **independently and simultaneously** because none of them depends on the others' results.

4.3.12 When to Use Parallelization

- Tasks with **independent subtasks**
- When **speed is important** (parallel = faster)
- **Multi-criteria evaluation** scenarios
- Content that needs **analysis from multiple angles**

4.3.13 3.4 Orchestrator Workers

Orchestrator Workers is similar to Parallelization but with one key difference: **the subtasks are dynamically determined** based on the input, rather than being predefined.

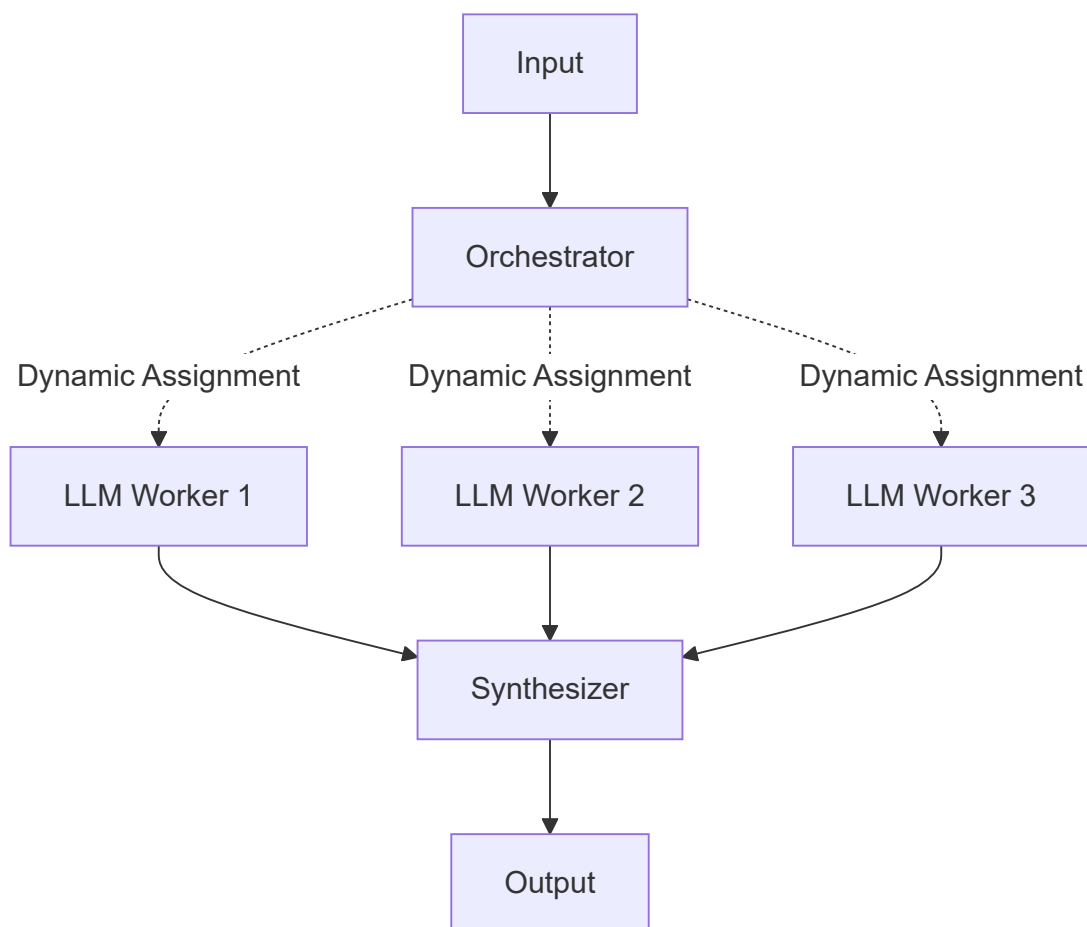


Figure 4.8. 4 Orchestrator Workers

4.3.14 Key Difference from Parallelization

Table 4.7. Key Difference from Parallelization

Aspect	Parallelization	Orchestrator Workers
Subtasks	Predefined (known beforehand)	Dynamic (determined at runtime)

Aspect	Parallelization	Orchestrator Workers
Worker Roles	Fixed roles	Roles assigned by orchestrator
Flexibility	Less flexible	Highly flexible

4.3.15 How Orchestrator Workers Works

1. **Input arrives** at the Orchestrator
2. **Orchestrator analyzes** the input
3. **Dynamically decides** what each worker should do
4. **Workers execute** their assigned tasks
5. **Synthesizer combines** all results
6. **Final output** produced

4.3.16 Real-World Example: Research Assistant

Scenario: Building a research assistant that creates detailed reports on any given query.

Dynamic Decision Making:

- If query is a **scientific term** → Orchestrator assigns worker to search Google Scholar
- If query is a **political incident** → Orchestrator assigns worker to search Google News
- If query is a **technical topic** → Orchestrator assigns worker to search documentation

The nature of subtasks **varies based on input query**, making it dynamic.

4.3.17 When to Use Orchestrator Workers

- **Variable task nature** depending on input
- **Research and analysis** applications
- **Multi-source information gathering**
- When subtask requirements are not predictable

4.3.18 3.5 Evaluator Optimizer

Evaluator Optimizer is an iterative pattern where a **Generator LLM creates solutions** and an **Evaluator LLM provides feedback** until the output meets quality criteria.

4.3.19 How Evaluator Optimizer Works

1. **Task provided** to Generator LLM
2. **Generator creates** initial solution
3. **Evaluator assesses** against criteria
4. If **Accepted** → Output final solution
5. If **Rejected** → Feedback sent to Generator
6. **Loop continues** until Evaluator accepts

4.3.20 The Creative Iteration Concept

This pattern mimics how human writers work:

Draft 1 → Review → Find Issues → Draft 2 → Review → ... → Final Version

A poet or writer doesn't produce perfect content in one go. They:

1. Write first draft

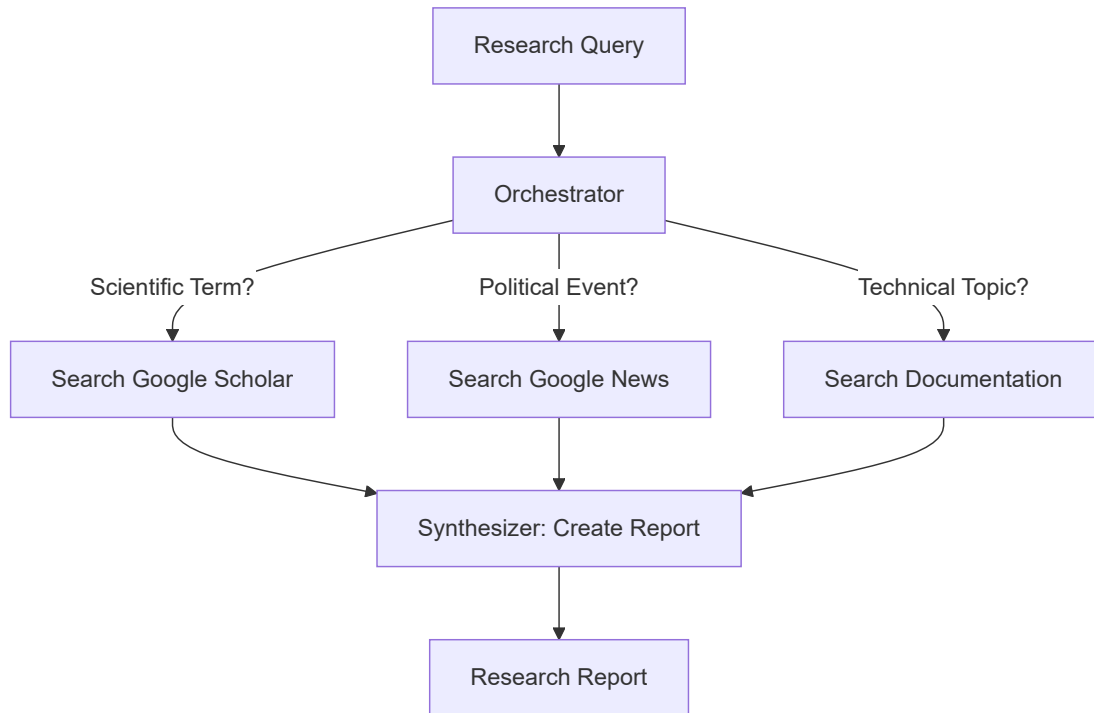


Figure 4.9. Real-World Example: Research Assistant

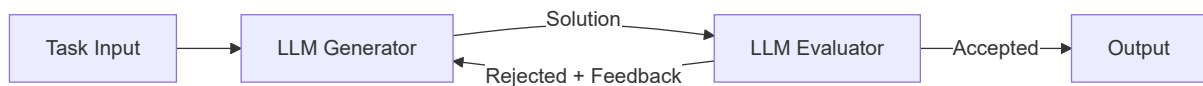


Figure 4.10. 5 Evaluator Optimizer

2. Identify weaknesses
3. Incorporate feedback
4. Create improved draft
5. Repeat until satisfied

4.3.21 Real-World Examples

Table 4.8. Real-World Examples

Use Case	Generator Role	Evaluator Criteria
Email Drafting	Create email content	Tone, clarity, professionalism
Blog Writing	Write blog posts	SEO, engagement, accuracy
Code Generation	Write code	Correctness, efficiency, style
Poetry/Creative Writing	Generate creative content	Artistic quality, originality

4.3.22 Evaluation Criteria Example

For an email drafting task, evaluator might check:

- Is the tone appropriate?
- Is the message clear?
- Is grammar correct?

- Is length appropriate?

4.3.23 When to Use Evaluator Optimizer

- **Creative content generation** (writing, poetry, stories)
- **Quality-critical outputs** that need refinement
- Tasks with **subjective quality criteria**
- When **first-attempt perfection is unlikely**
- **Self-improving systems**

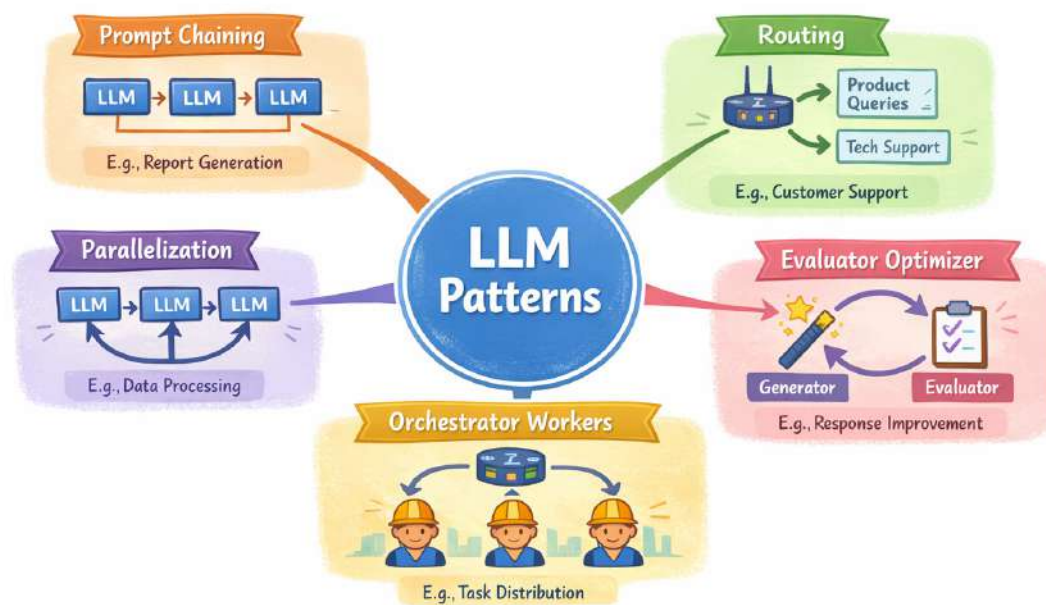


Figure 4.11. When to Use Evaluator Optimizer

4.4 4. Graphs, Nodes, and Edges

This is the **most fundamental concept** in LangGraph. Understanding how workflows are represented as graphs is essential for building effective LLM applications.

4.4.1 The Graph Representation

When LangGraph processes a workflow, it converts it into a graph structure:

LLM Workflow → Graph Representation → Nodes + Edges

4.4.2 Components of a LangGraph Graph

Nodes

Nodes represent individual **tasks** in your workflow.

Table 4.9. *Nodes*

Property	Description
What they are	Python functions
What they do	Execute a single task
Examples	LLM call, tool call, decision making

```
# A node is simply a Python function
def generate_topic(state):
    # Task: Generate a topic
    topic = llm.generate("Create a UPSC essay topic")
    state["topic"] = topic
    return state
```

Edges

Edges connect nodes and define the **flow of execution**.

Table 4.10. *Edges*

Edge Type	Description	Visual Representation
Sequential	One after another	$A \rightarrow B \rightarrow C$
Parallel	Multiple simultaneous	$A \rightarrow [B, C, D]$
Conditional	Based on condition	$A \rightarrow \{B \text{ or } C\}$
Loop	Back to previous	$A \rightarrow B \rightarrow A$

4.4.3 Understanding Through Example: UPSC Essay Practice Platform

Scenario: Building a website where UPSC aspirants practice writing essays.

Workflow Steps:

1. Generate topic for the student
2. Collect student's essay
3. Evaluate essay on multiple criteria
4. Aggregate scores
5. Provide pass/fail result
6. Give feedback if failed
7. Allow retry option

Graph Visualization

Scoring System

Table 4.11. *Scoring System*

Criteria	Max Score	What is Evaluated
Clarity of Thought	5	Clear expression of ideas
Depth of Analysis	5	Thoroughness of arguments
Language	5	Grammar, vocabulary, tone
Total	15	Threshold: 10

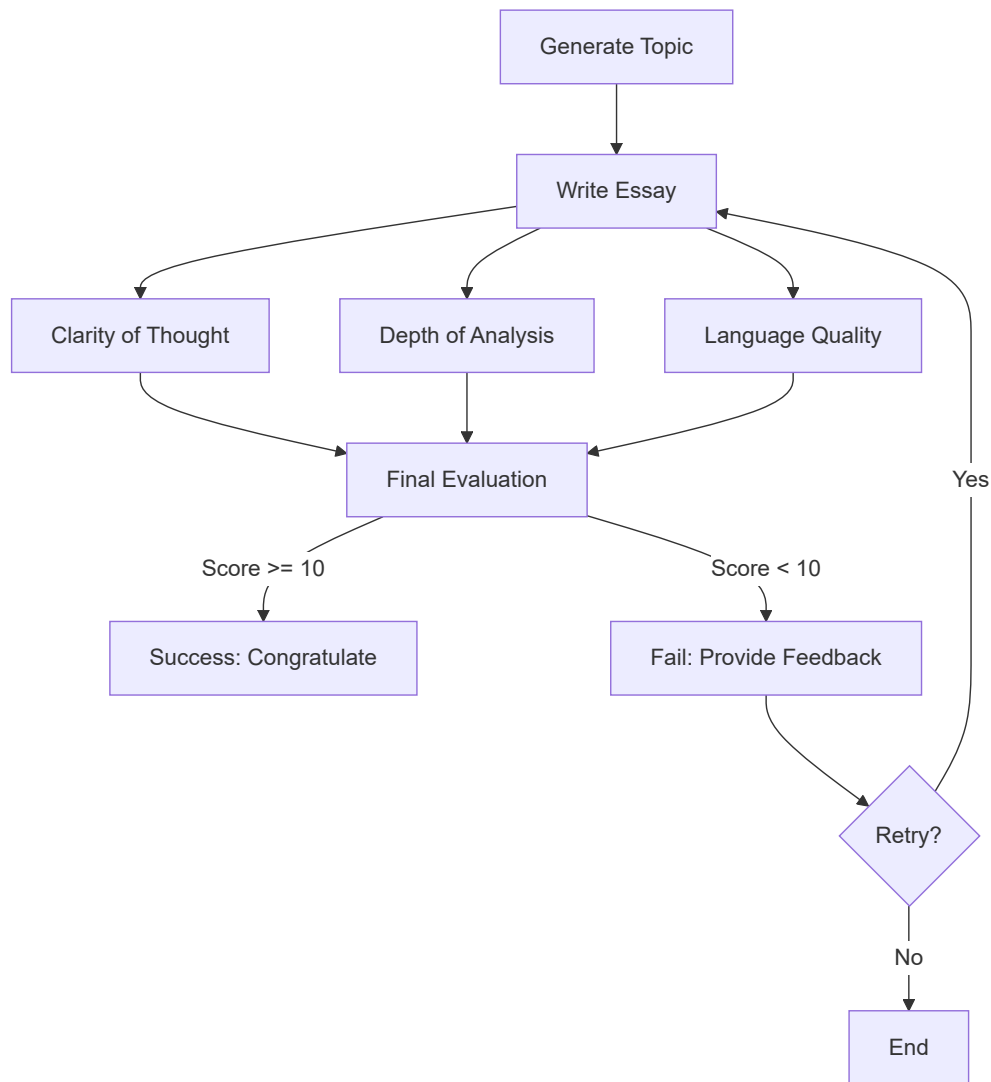


Figure 4.12. Graph Visualization

4.4.4 Key Insights

Nodes tell us **WHAT** to do:

- Generate Topic → One task (Python function)
- Write Essay → One task (Python function)
- Evaluate Clarity → One task (Python function)

Edges tell us **WHEN** to do it:

- After Generate Topic → Execute Write Essay
- After Write Essay → Execute all three evaluators
- After evaluators → Execute Final Evaluation

4.4.5 Types of Flow Expressed Through Edges

Table 4.12. *Types of Flow Expressed Through Edges*

Flow Type	Example in Diagram	Description
Sequential	Topic → Essay	One follows another
Parallel	Essay → [Clarity, Depth, Language]	All execute together
Conditional	Evaluation → Success OR Fail	Branch based on condition
Loop	Fail → Retry → Essay	Return to earlier node

4.5 5. State Management

State is one of the most important concepts in LangGraph. It represents the **shared memory** that flows through your entire workflow.

4.5.1 What is State?

State is the collection of data points that your workflow needs throughout its execution, and this data evolves over time as the workflow progresses.

4.5.2 Characteristics of State

Table 4.13. *Characteristics of State*

Property	Description
Shared	Accessible by all nodes in the graph
Mutable	Can be modified by any node
Evolving	Changes as execution progresses
Essential	Required for workflow execution

4.5.3 Formal Definition

*In LangGraph, **State** is a shared memory that flows through your workflow — it holds all the data being passed between nodes as your graph runs.*

4.5.4 How State Works

1. **Node receives** current state as input
2. **Node executes** its task
3. **Node updates** relevant parts of state
4. **Updated state** passed to next node(s)

4.5.5 State Example: UPSC Essay Platform

For the essay evaluation workflow, state would contain:

```
from typing import TypedDict, Annotated
from operator import add

class EssayState(TypedDict):
    essay_text: str          # The essay content
    topic: str               # Essay topic
    depth_score: int         # Score for depth (0-5)
    language_score: int      # Score for language (0-5)
    clarity_score: int       # Score for clarity (0-5)
    total_score: int         # Combined score
```

```
feedback: Annotated[List[str], add] # List of feedback comments
evaluation_round: int # Which attempt number
```

4.5.6 State Evolution Example

Initial State:

```
{
  "essay_text": "",
  "topic": "",
  "depth_score": 0,
  "language_score": 0,
  "clarity_score": 0,
  "total_score": 0,
  "feedback": [],
  "evaluation_round": 1
}
```

After Topic Generation:

```
{
  "topic": "Impact of AI on Employment",
  # ... other fields unchanged
}
```

After Essay Submission:

```
{
  "essay_text": "The advent of artificial intelligence...",
  "topic": "Impact of AI on Employment",
  # ... other fields unchanged
}
```

After Evaluation:

```
{
  "essay_text": "The advent of artificial intelligence...",
  "topic": "Impact of AI on Employment",
  "depth_score": 4,
  "language_score": 3,
  "clarity_score": 4,
  "total_score": 11,
  "feedback": ["Good depth", "Minor grammar issues"],
  "evaluation_round": 1
}
```

4.5.7 State Implementation

State in LangGraph is typically implemented as a **TypedDict**:

```
from typing import TypedDict

class MyState(TypedDict):
    key1: str
    key2: int
    key3: list
```

You can also use **Pydantic** models for more complex validation:

```
from pydantic import BaseModel

class MyState(BaseModel):
    key1: str
    key2: int
```

key3: `list`

4.5.8 Key Points About State

1. **Every node has access** to the complete state
2. **Nodes can read** any value from state
3. **Nodes can update** any value in state
4. **Updates propagate** to subsequent nodes
5. State acts as the **communication channel** between nodes

4.6 6. Reducers

Reducers are closely related to State and define **how updates are applied** to state values.

4.6.1 The Problem Reducers Solve

Recall two properties of State:

1. **Accessible** to all nodes
2. **Mutable** by all nodes

Consider this scenario:

State:

```
{
  "first_num": 5,
  "second_num": 6,
  "result": ?
}
```

Execution:

1. Node 1: Sets first_num=5, second_num=6
2. Node 2: Calculates sum (5+6=11), sets result=11
3. Node 3: Multiplies (11×2=22), sets result=22

Here, Node 3 **replaces** the value Node 2 set. This is the **default update behavior**.

4.6.2 When Default Behavior Fails

Chatbot Example:

State:

```
{
  "message": ?
}
```

Conversation Flow:

1. Human: "Hi, my name is Nitish" → message = "Hi, my name is Nitish"
2. LLM: "How can I help you?" → message = "How can I help you?"
3. Human: "Can you tell me my name?" → message = "Can you tell me my name?"

Problem: LLM cannot answer because the message containing the name was **replaced**!

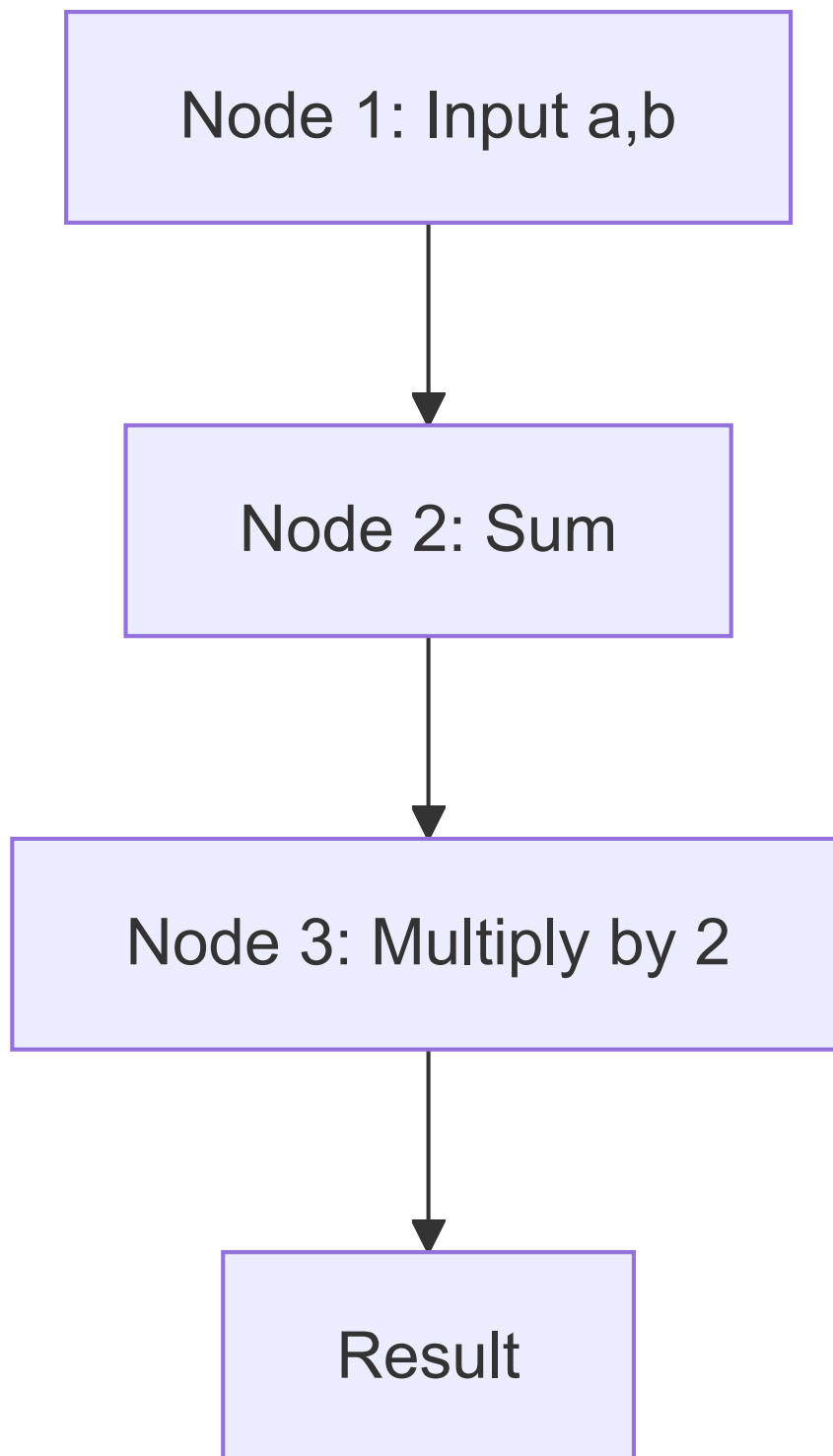


Figure 4.13. *The Problem Reducers Solve*

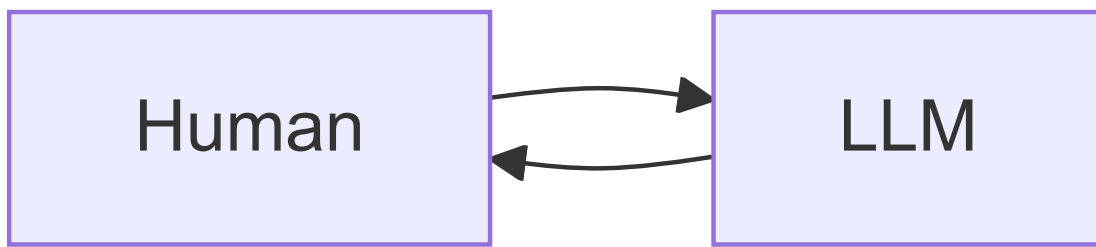


Figure 4.14. When Default Behavior Fails

4.6.3 The Reducer Solution

Reducer defines how updates should be applied to each state key.

Table 4.14. The Reducer Solution

Update Policy	Behavior	Use Case
Replace (default)	New value replaces old	Simple values that change
Add/Append	New value added to list	Message history, logs
Merge	Combine old and new	Dictionary updates

4.6.4 Implementing Reducers

```
from typing import TypedDict, Annotated
from operator import add

class ChatState(TypedDict):
    messages: Annotated[list[str], add] # ADD instead of replace
```

With this reducer, messages **accumulate**:

```
# Message 1: "Hi, my name is Nitish"
messages = ["Hi, my name is Nitish"]

# Message 2: "How can I help you?"
messages = ["Hi, my name is Nitish", "How can I help you?"]

# Message 3: "Can you tell me my name?"
messages = ["Hi, my name is Nitish", "How can I help you?", "Can you tell me my name?"]
```

Now the LLM has **access to complete conversation history**!

4.6.5 Reducer Example: Essay Platform

In the essay evaluation workflow, you might want to preserve all essay attempts:

```
class EssayState(TypedDict):
    essay_text: Annotated[list[str], add] # Keep all attempts
    # ... other fields
```

Evolution:

```
# First attempt
essay_text = ["First essay content..."]
```

```
# Second attempt (after feedback)
essay_text = ["First essay content...", "Improved essay content..."]

# Third attempt
essay_text = ["First essay content...", "Improved essay content...", "Final essay..."]
```

This allows students to **see their evolution** over multiple attempts.

4.6.6 Key Takeaways on Reducers

Table 4.15. *Key Takeaways on Reducers*

Aspect	Description
Purpose	Define how state updates are applied
Scope	Each key can have its own reducer
Options	Replace, Add, Merge, Custom functions
Use Case	Essential for parallel workflows and history preservation

4.7 7. LangGraph Execution Model

Understanding how LangGraph executes workflows behind the scenes helps in designing and debugging complex applications.

4.7.1 Inspiration: Google Pregel

LangGraph's execution model is inspired by **Google Pregel**, a system designed for **large-scale graph processing** used in many Google products.

4.7.2 Execution Phases

The execution follows four main phases:

4.7.3 Phase 1: Graph Definition

In this phase, you define three things:

Table 4.16. *Phase 1: Graph Definition*

Component	What You Define
State Schema	The TypedDict or Pydantic model for state
Nodes	Python functions that perform tasks
Edges	Which node connects to which

```
from langgraph.graph import StateGraph

# Define state
class MyState(TypedDict):
    value: str

# Create graph
graph = StateGraph(MyState)

# Add nodes
graph.add_node("node1", function1)
graph.add_node("node2", function2)
```

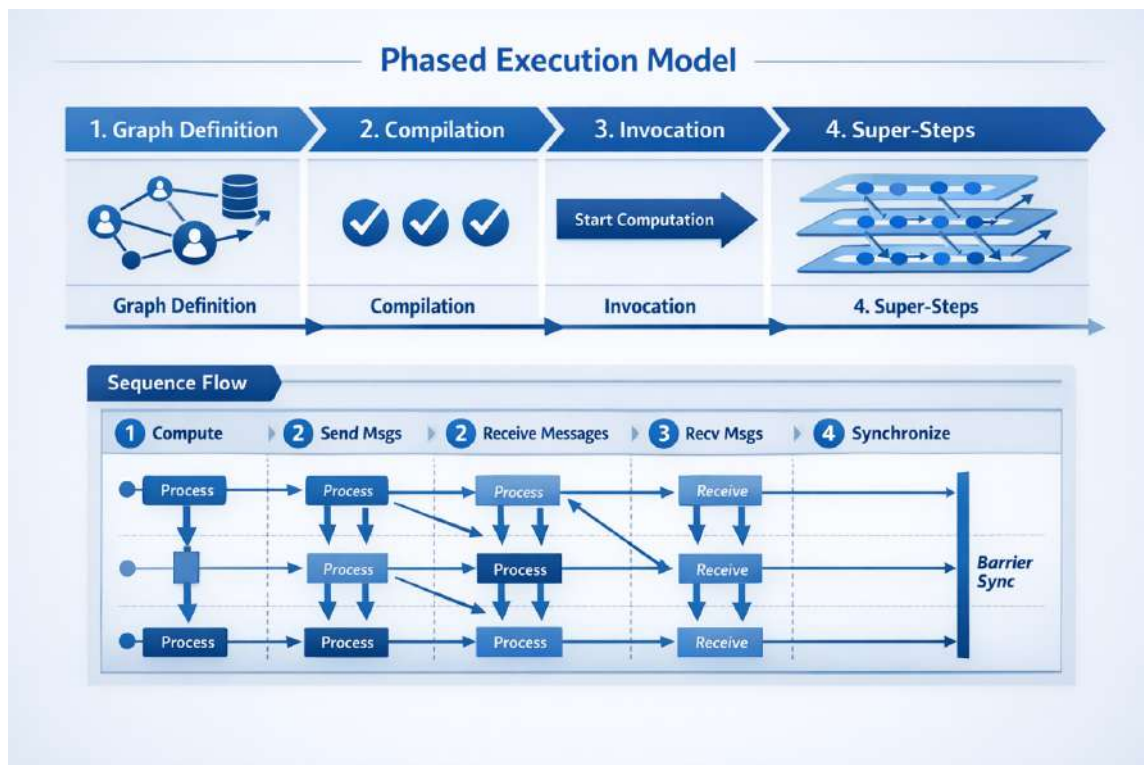



Figure 4.15. Execution Phases

```
# Add edges
graph.add_edge("node1", "node2")
```

4.7.4 Phase 2: Compilation

Compilation validates the graph structure:

```
# Compile the graph
compiled_graph = graph.compile()
```

What compilation checks:

- No orphaned nodes (disconnected from graph)
- Valid edge connections
- Proper state schema
- No structural inconsistencies

4.7.5 Phase 3: Invocation

Start execution by providing **initial state** to the entry node:

```
# Run the graph
result = compiled_graph.invoke(initial_state)
```

What happens:

1. Initial state passed to first node
2. First node **activates** (function executes)
3. Execution begins

4.7.6 Phase 4: Super-Steps Execution

Execution proceeds in **rounds called Super-Steps**.

What is a Super-Step?

A Super-Step is one round of execution that may include **one or more parallel steps**.

Why “Super-Step” instead of “Step”?

Because graph structure allows parallel execution. A single super-step might contain:

- 1 node (sequential)
- 3 nodes (parallel)
- Any number executing together

4.7.7 Message Passing

Message Passing is how state moves between nodes via edges.

Process:

1. Node executes its function
2. Node updates state (partial update)
3. Updated state **passes through edge** to next node
4. Next node receives state and activates

4.7.8 Execution Flow in Detail

4.7.9 Termination Conditions

Workflow execution **stops** when:

1. **No active nodes** remain
2. **No messages** being passed through edges
3. **End node** is reached

4.7.10 Key Execution Concepts Summary

Table 4.17. *Key Execution Concepts Summary*

Concept	Description
Graph Definition	Define nodes, edges, state
Compilation	Validate graph structure
Invocation	Start execution with initial state
Super-Step	One round of (possibly parallel) execution
Message Passing	State transfer via edges
Activation	Node becomes active when it receives state

4.7.11 Automatic Orchestration

Important: You don’t manually call each node. Once you invoke the graph:

- Nodes automatically execute in correct order
- State automatically passes between nodes
- Parallel nodes automatically run together
- The framework handles all orchestration

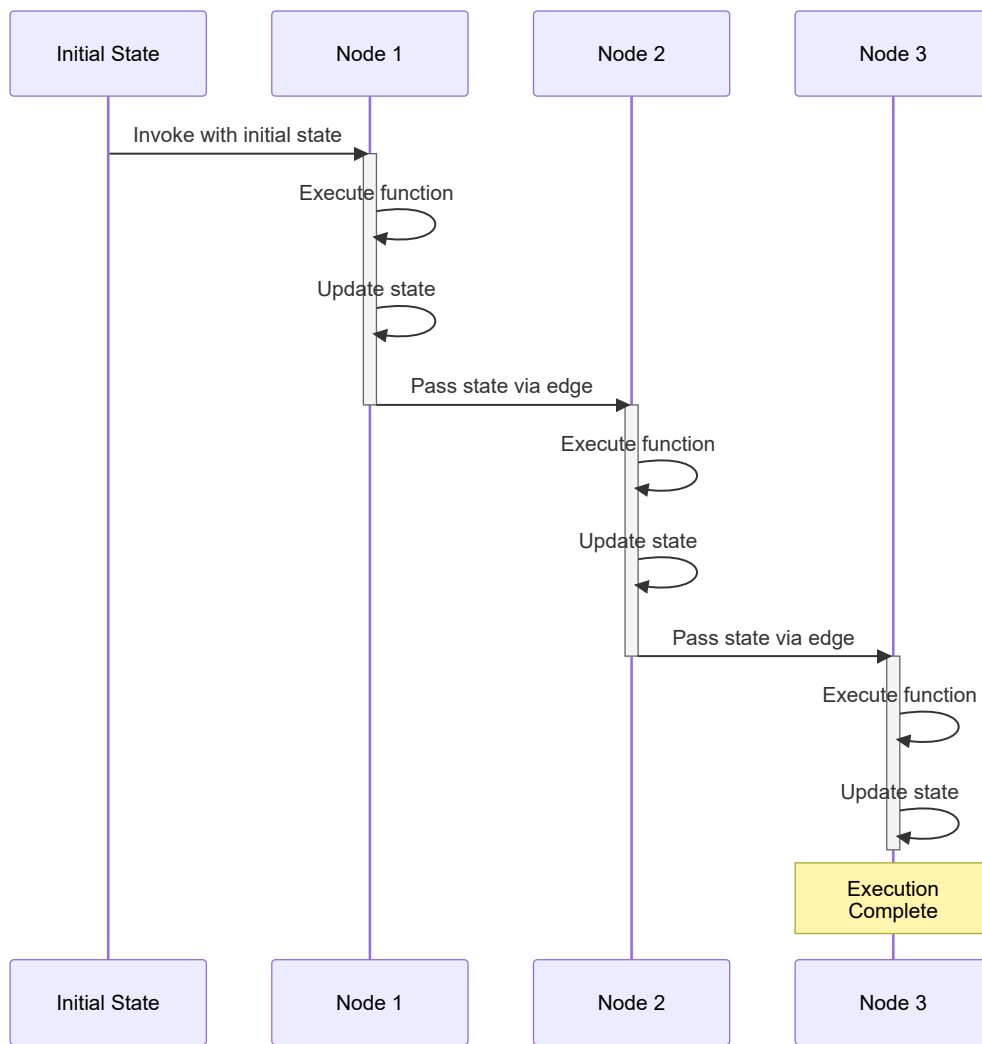


Figure 4.16. Execution Flow in Detail

4.8 8. Summary and Key Takeaways

4.8.1 LangGraph at a Glance

Table 4.18. LangGraph at a Glance

Aspect	Description
What	Orchestration framework for LLM workflows
How	Represents workflows as graphs with nodes and edges
Why	Enables complex, stateful, production-grade AI applications

4.8.2 Common Workflow Patterns Quick Reference

Table 4.19. Common Workflow Patterns Quick Reference

Pattern	Use When	Key Feature
Prompt Chaining	Complex tasks needing breakdown	Sequential LLM calls with validation

Pattern	Use When	Key Feature
Routing	Different query types need different handlers	Decision-making router
Parallelization	Independent subtasks	Simultaneous execution
Orchestrator Workers	Dynamic subtask assignment	Runtime task determination
Evaluator Optimizer	Quality refinement needed	Iterative improvement loop

4.8.3 State Management Essentials

Table 4.20. State Management Essentials

Concept	Purpose	Implementation
State	Shared data across nodes	TypedDict
Reducer	Control update behavior	Annotated[type, add]
Accessibility	All nodes can read/write	Built-in feature
Evolution	State changes over execution	Automatic propagation

4.8.4 Execution Model Key Points

- 1. **Define** → State, Nodes, Edges
- 2. **Compile** → Validate structure
- 3. **Invoke** → Start with initial state
- 4. **Execute** → Super-steps with message passing
- 5. **Complete** → No active nodes, no messages

4.8.5 Best Practices

- 1. **Start with workflow design** before coding
- 2. **Identify state requirements** early
- 3. **Choose appropriate reducers** for each state key
- 4. **Use the right pattern** for your use case
- 5. **Validate graph structure** through compilation
- 6. **Test each node** independently before integration



Figure 4.17. Best Practices

Sequential Workflows in LangGraph

5.1 Course Overview & Context

- <https://github.com/campusx-official/langgraph-tutorials>

5.1.1 Previous Videos Recap

This video is part of an **Agentic AI using LangGraph** playlist. Previous coverage included:

1. **Video 1:** Core differences between Generative AI and Agentic AI
2. **Video 2:** Deep dive into understanding Agentic AI with use cases
3. **Video 3:** Why LangGraph was needed when LangChain already exists + core differences between libraries
4. **Video 4:** Core LangGraph concepts required for coding

5.1.2 Current Video Goals

Transition from Theory to Practice - This marks the beginning of hands-on coding sessions.

Two Primary Objectives:

1. **First Coding Experience:** Learn basic LangGraph code structure
2. **Sequential Workflow Mastery:** Build any type of sequential workflow independently

5.2 Sequential Workflows Explained

5.2.1 Definition

Sequential Workflow: A workflow where all tasks are connected in linear fashion

- Task 1 → Task 2 → Task 3 → and so on
- **No branching** or parallel paths
- Simple, linear execution flow

5.2.2 Characteristics

- **Linear Task Connection:** Each task follows the previous one
- **Single Path Execution:** No conditional branching
- **Predictable Flow:** Clear start-to-finish sequence

5.3 Development Environment Setup

5.3.1 Project Structure

```
Desktop/└─  
  langgraph_tutorials/  
    └─ 01_test_installation.ipynb  
      └─ [future tutorial files]
```

5.3.2 Virtual Environment Creation

```
# Create virtual environment  
python -m venv myenv  
  
# Activate virtual environment  
myenv\Scripts\activate
```

5.3.3 Required Libraries Installation

Core Dependencies

```
# Primary LangGraph library  
pip install langgraph  
  
# LangChain for LLM components  
pip install langchain  
  
# OpenAI integration  
pip install langchain-openai  
  
# Environment variables management  
pip install python-dotenv
```

Why Each Library?

- **LangGraph:** Main framework for building workflows
- **LangChain:** Provides LLM-related components (chat models, prompt templates, document loaders, text splitters)
- **LangChain-OpenAI:** Enables OpenAI model integration
- **Python-dotenv:** Handles environment variable management

5.3.4 Development Environment Choice

Jupyter Notebook Benefits

- **Graph Visualization:** Easy to print and visualize LangGraph workflows
- **Interactive Development:** Cell-by-cell execution for testing
- **Quick Prototyping:** Ideal for learning and experimentation

Future Development

- **Production Code:** Will use regular Python files for actual projects
- **Learning Phase:** Jupyter notebooks for understanding concepts

5.4 Installation Verification

5.4.1 Test Setup

Create: 01_test_installation.ipynb

5.4.2 Import Testing

```
# Test core LangGraph import
from langgraph.graph import StateGraph

# Verify auto-suggestions work for both libraries
# - LangGraph suggestions should appear
# - LangChain suggestions should appear
```

5.4.3 Environment Configuration

1. **Kernel Selection:** Choose the created virtual environment
2. **Package Installation:** Install ipykernel if prompted
3. **Import Verification:** Confirm all libraries import successfully

5.4.4 LangGraph-LangChain Relationship

- **Complementary Libraries:** Work together, not as replacements
- **LangGraph Role:** Workflow orchestration and graph creation
- **LangChain Role:** Provides building blocks (LLM components, tools, utilities)
- **Integration:** LangGraph workflows use LangChain components

5.4.5 Best Practices Established

- **Virtual Environment:** Isolated development environment
- **Version Control:** Git repository for code sharing
- **Documentation:** Comprehensive notes and examples
- **Testing:** Verify installations before proceeding

5.5 Quick Reference Commands

```
# Virtual Environment Setup
python -m venv myenv
myenv\Scripts\activate

# Library Installation
pip install langgraph langchain langchain-openai python-dotenv

# Test Import
from langgraph.graph import StateGraph
```

5.6 LangGraph Sequential Workflows - Detailed Study Notes

5.7 Course Overview & Context

5.7.1 Previous Videos Recap

This video is part of an **Agentic AI using LangGraph** playlist. Previous coverage included:

1. **Video 1:** Core differences between Generative AI and Agentic AI
2. **Video 2:** Deep dive into understanding Agentic AI with use cases
3. **Video 3:** Why LangGraph was needed when LangChain already exists + core differences between libraries
4. **Video 4:** Core LangGraph concepts required for coding

5.7.2 Current Video Goals

Transition from Theory to Practice - This marks the beginning of hands-on coding sessions.

Two Primary Objectives:

1. **First Coding Experience:** Learn basic LangGraph code structure
2. **Sequential Workflow Mastery:** Build any type of sequential workflow independently

5.8 Sequential Workflows Explained

5.8.1 Definition

Sequential Workflow: A workflow where all tasks are connected in linear fashion

- Task 1 → Task 2 → Task 3 → and so on
- **No branching** or parallel paths
- Simple, linear execution flow

5.8.2 Characteristics

- **Linear Task Connection:** Each task follows the previous one
- **Single Path Execution:** No conditional branching
- **Predictable Flow:** Clear start-to-finish sequence

5.9 Development Environment Setup

5.9.1 Project Structure

```
Desktop/└─  
  langgraph_tutorials/  
    ├── 01_test_installation.ipynb  
    └── [future tutorial files]
```

5.9.2 Virtual Environment Creation

```
# Create virtual environment  
python -m venv myenv  
  
# Activate virtual environment  
myenv\Scripts\activate
```

5.9.3 Required Libraries Installation

Core Dependencies

```
# Primary LangGraph library
pip install langgraph

# LangChain for LLM components
pip install langchain

# OpenAI integration
pip install langchain-openai

# Environment variables management
pip install python-dotenv
```

Why Each Library?

- **LangGraph:** Main framework for building workflows
- **LangChain:** Provides LLM-related components (chat models, prompt templates, document loaders, text splitters)
- **LangChain-OpenAI:** Enables OpenAI model integration
- **Python-dotenv:** Handles environment variable management

5.9.4 Import Testing

```
# Test core LangGraph import
from langgraph.graph import StateGraph

# Verify auto-suggestions work for both libraries
# - LangGraph suggestions should appear
# - LangChain suggestions should appear
```

5.10 Key Learning Points

5.10.1 LangGraph-LangChain Relationship

- **Complementary Libraries:** Work together, not as replacements
- **LangGraph Role:** Workflow orchestration and graph creation
- **LangChain Role:** Provides building blocks (LLM components, tools, utilities)
- **Integration:** LangGraph workflows use LangChain components

5.10.2 Best Practices Established

- **Virtual Environment:** Isolated development environment
- **Version Control:** Git repository for code sharing
- **Documentation:** Comprehensive notes and examples
- **Testing:** Verify installations before proceeding

5.11 First Workflow: BMI Calculator

5.11.1 Workflow Design Philosophy

Non-LLM Focus: Deliberately chosen to concentrate on LangGraph syntax rather than LLM complexities
Simple and Linear: Perfect introduction to sequential workflow concepts

5.11.2 BMI Calculator Workflow Structure

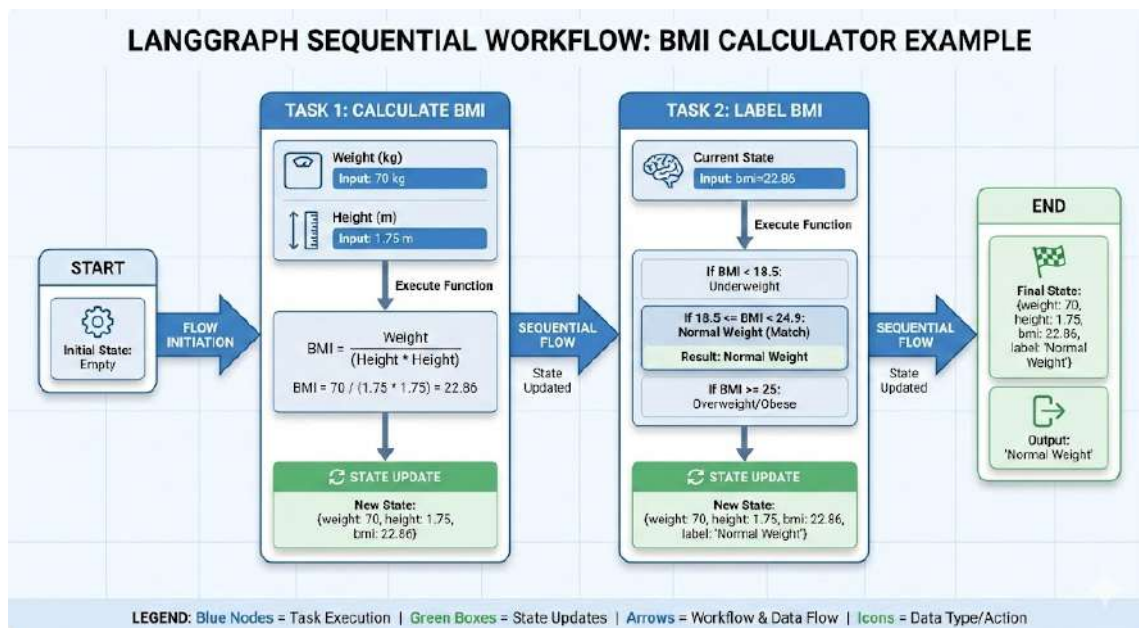


Figure 5.1. BMI Calculator Workflow Structure

5.11.3 Step-by-Step Implementation

Step 1: Define State

```
from typing import TypedDict

class BMIState(TypedDict):
    weight_kg: float # Weight in kilograms
    height_m: float # Height in meters
    bmi: float # Calculated BMI value
    category: str # BMI category (Added in extended version)
```

Key Concepts:

- State is a **TypedDict** - special dictionary with type hints
- Contains all data that flows through the workflow
- Each attribute has defined data type for clarity

Step 2: Create Graph Object

```
from langgraph.graph import StateGraph

graph = StateGraph(BMIState)
```

Important Points:

- Pass your state class as parameter
- This registers the state structure with the graph

Step 3: Define Node Functions

```
def calculate_bmi(state: BMIState) -> BMIState:
    # Extract values from state
    weight = state['weight_kg']
    height = state['height_m']

    # Calculate BMI using formula: weight / height^2
    bmi = weight / (height ** 2)

    # Update state with calculated value
    state['bmi'] = round(bmi, 2)

    # Return updated state
    return state

def label_bmi(state: BMIState) -> BMIState:
    bmi = state['bmi']

    # Categorize BMI value
    if bmi < 18.5:
        state["category"] = "Underweight"
    elif 18.5 <= bmi < 25:
        state["category"] = "Normal"
    elif 25 <= bmi < 30:
        state["category"] = "Overweight"
    else:
        state["category"] = "Obese"

    return state
```

Node Function Rules:

- **Input:** Always receives state object
- **Output:** Always returns state object
- **Type Hints:** Specify input and output types
- **Partial Updates:** Modify only relevant state attributes

Step 4: Add Nodes to Graph

```
graph.add_node('calculate_bmi', calculate_bmi)
graph.add_node('label_bmi', label_bmi)
```

Node Mapping:

- First parameter: Node name (string identifier)
- Second parameter: Python function to execute
- Names can be different but keeping them same aids clarity

Step 5: Define Edges (Flow Control)

```
from langgraph.graph import START, END

# Simple version edges
graph.add_edge(START, 'calculate_bmi')
graph.add_edge('calculate_bmi', END)

# Extended version edges
graph.add_edge(START, 'calculate_bmi')
graph.add_edge('calculate_bmi', 'label_bmi')
```

```
graph.add_edge('label_bmi', END)
```

Edge Components:

- **START:** Special node indicating workflow beginning
- **END:** Special node indicating workflow completion
- **Sequential Flow:** Each edge connects one node to the next

Step 6: Compile Graph

```
workflow = graph.compile()
```

Compilation Purpose:

- Validates graph structure
- Creates executable workflow object
- Returns runnable that can be invoked

Step 7: Execute Workflow

```
# Define initial state
initial_state = {
    'weight_kg': 80,
    'height_m': 1.73
}

# Execute workflow
final_state = workflow.invoke(initial_state)

print(final_state)
# Output: {'weight_kg': 80, 'height_m': 1.73, 'bmi': 26.73, 'category': 'Overweight'}
```

Execution Flow:

- **Input:** Dictionary with initial state values
- **Processing:** Nodes execute sequentially, updating state
- **Output:** Complete state object with all computed values

5.11.4 Graph Visualization

Jupyter Notebook Visualization

```
from IPython.display import Image
Image(workflow.get_graph().draw_mermaid_png())
```

Visualization Benefits:

- **Visual Understanding:** See workflow structure graphically
- **Debugging Aid:** Identify flow issues visually
- **Documentation:** Clear representation for team sharing
- **Jupyter Only:** This visualization code works only in Jupyter notebooks

5.11.5 Key Learning Points

LangGraph Workflow Pattern

1. **Define State:** TypedDict with all required attributes

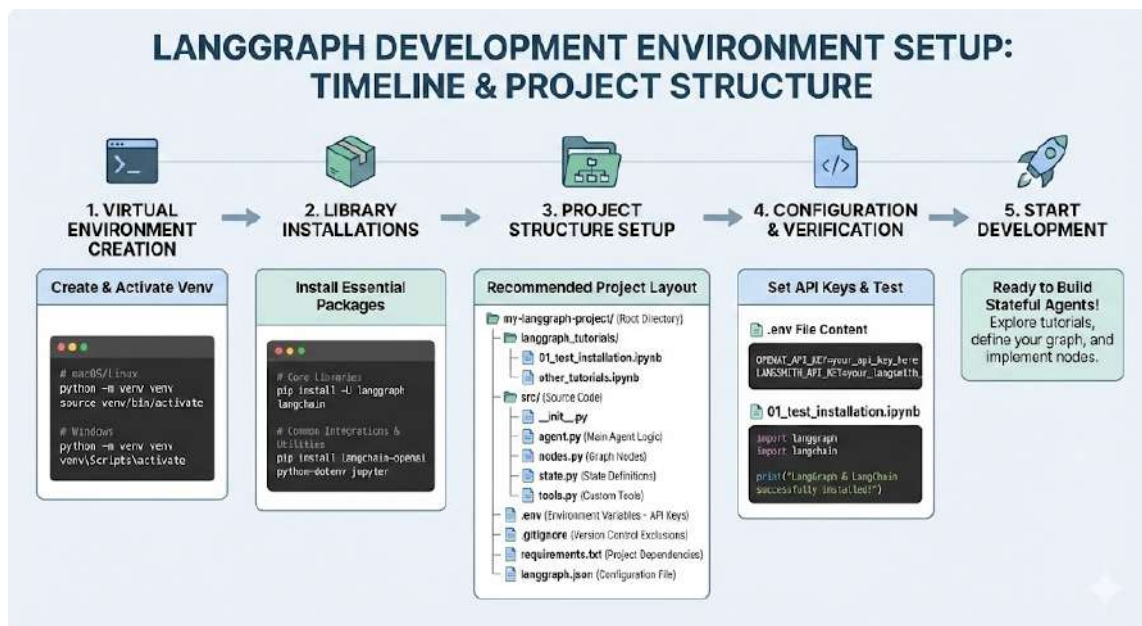


Figure 5.2. Key Learning Points

2. **Create Graph:** StateGraph instance with state type
3. **Add Nodes:** Map node names to Python functions
4. **Add Edges:** Define sequential flow between nodes
5. **Compile:** Validate and create executable workflow
6. **Execute:** Invoke with initial state, receive final state

State Management

- **Input State:** Flows into each node function
- **Partial Updates:** Nodes modify only relevant attributes
- **State Persistence:** All data maintained throughout workflow
- **Output State:** Final state contains all computed values

Sequential Workflow Characteristics

- **Linear Flow:** No branching or parallel paths
- **Predictable Execution:** Clear start-to-finish sequence
- **State Evolution:** Each node builds upon previous state
- **Simple Structure:** Easy to understand and debug

5.11.6 Extension Strategy

Adding New Functionality

1. **Update State:** Add new attributes to TypedDict
2. **Create Node Function:** Implement required logic
3. **Add Node:** Register function with graph
4. **Modify Edges:** Update flow to include new node
5. **Recompile:** Generate new executable workflow

Best Practices

- **Incremental Development:** Start simple, add complexity gradually

- **Clear Naming:** Use descriptive names for nodes and functions
- **Type Safety:** Always use type hints for clarity
- **State Design:** Include all necessary data in state definition

5.12 Quick Reference Commands

```
# Virtual Environment Setup
python -m venv myenv
myenv\Scripts\activate

# Library Installation
pip install langgraph langchain langchain-openai python-dotenv

# Test Import
from langgraph.graph import StateGraph, START, END
from typing import TypedDict
```

5.13 Complete BMI Workflow Code

```
# BMI Calculator Workflow - LangGraph Tutorial
# Complete implementation following the tutorial

from langgraph.graph import StateGraph, START, END
from typing import TypedDict

# Step 1: Define State
class BMIState(TypedDict):
    """
    State structure for BMI calculation workflow
    Contains all data that flows through the workflow
    """
    weight_kg: float  # Weight in kilograms
    height_m: float   # Height in meters
    bmi: float        # Calculated BMI value
    category: str     # BMI category classification

# Step 2: Define Node Functions

def calculate_bmi(state: BMIState) -> BMIState:
    """
    Calculate BMI using the formula: weight / height^2

    Args:
        state: Current state containing weight and height

    Returns:
        Updated state with calculated BMI value
    """
    # Extract values from state
    weight = state['weight_kg']
    height = state['height_m']

    # Calculate BMI using standard formula
    bmi = weight / (height ** 2)

    # Update state with calculated value (rounded to 2 decimal places)
    state['bmi'] = round(bmi, 2)

    # Return updated state
    return state

def label_bmi(state: BMIState) -> BMIState:
    """
    Categorize BMI value into standard health categories
```



```

    Args:
        state: Current state containing calculated BMI

    Returns:
        Updated state with BMI category
    """
    # Extract BMI value from state
    bmi = state['bmi']

    # Categorize BMI according to standard ranges
    if bmi < 18.5:
        state["category"] = "Underweight"
    elif 18.5 <= bmi < 25:
        state["category"] = "Normal"
    elif 25 <= bmi < 30:
        state["category"] = "Overweight"
    else:
        state["category"] = "Obese"

    return state

# Step 3: Build the Graph

# Create graph object with state type
graph = StateGraph(BMIState)

# Add nodes to the graph (mapping node names to functions)
graph.add_node('calculate_bmi', calculate_bmi)
graph.add_node('label_bmi', label_bmi)

# Add edges to define the workflow sequence
graph.add_edge(START, 'calculate_bmi')           # Start → calculate_bmi
graph.add_edge('calculate_bmi', 'label_bmi')      # calculate_bmi → label_bmi
graph.add_edge('label_bmi', END)                  # label_bmi → End

# Step 4: Compile the graph
workflow = graph.compile()

# Step 5: Execute the workflow

def run_bmi_workflow(weight_kg: float, height_m: float) -> dict:
    """
    Execute the BMI calculation workflow

    Args:
        weight_kg: Weight in kilograms
        height_m: Height in meters

    Returns:
        Complete state with BMI and category
    """
    # Define initial state
    initial_state = {
        'weight_kg': weight_kg,
        'height_m': height_m
    }

    # Execute workflow
    final_state = workflow.invoke(initial_state)

    return final_state

# Example usage
if __name__ == "__main__":
    # Test the workflow with sample data
    result = run_bmi_workflow(weight_kg=80, height_m=1.73)

    print("BMI Calculation Results:")
    print(f"Weight: {result['weight_kg']} kg")
    print(f"Height: {result['height_m']} m")
    print(f"BMI: {result['bmi']}")
    print(f"Category: {result['category']}")

```

```
# Expected output:
# Weight: 80 kg
# Height: 1.73 m
# BMI: 26.73
# Category: Overweight

# For Jupyter Notebook visualization (uncomment if using Jupyter):
# from IPython.display import Image
# Image(workflow.get_graph().draw_mermaid_png())

# Alternative test cases
test_cases = [
    {"weight_kg": 60, "height_m": 1.75, "expected_category": "Normal"},
    {"weight_kg": 50, "height_m": 1.70, "expected_category": "Underweight"},
    {"weight_kg": 90, "height_m": 1.65, "expected_category": "Obese"},
]

print("\nTesting multiple cases:")
for i, case in enumerate(test_cases, 1):
    result = run_bmi_workflow(case["weight_kg"], case["height_m"])
    print(f"Test {i}: BMI={result['bmi']}, Category={result['category']}")
```

5.14 LangGraph Tutorial Notes: Simple LLM Workflow

5.15 Overview

This tutorial demonstrates creating a simple LLM workflow using LangGraph, which is the simplest possible LLM workflow implementation.

5.16 Setup and Imports

```
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from typing import TypedDict
from dotenv import load_dotenv

load_dotenv()
model = ChatOpenAI()
```

Key Components:

- **StateGraph**: Core LangGraph component for building workflows
- **START/END**: Special nodes marking workflow boundaries
- **ChatOpenAI**: OpenAI integration from LangChain
- **TypedDict**: For type-safe state definitions

5.17 State Definition

```
class LLMState(TypedDict):
    question: str
    answer: str
```

Purpose: Defines the workflow state with two attributes:

- **question**: Input question (string)
- **answer**: LLM response (string)

5.18 Core Node Function

```
def llm_qa(state: LLMState) -> LLMState:
    # Extract question from state
    question = state['question']

    # Form a prompt
    prompt = f'Answer the following question {question}'

    # Ask LLM and get response
    answer = model.invoke(prompt).content

    # Update state with answer
    state['answer'] = answer

    return state
```

Workflow Logic:

1. Extract question from current state
2. Format prompt for LLM
3. Invoke LLM with prompt
4. Extract content from response
5. Update state with answer
6. Return updated state

5.19 Graph Construction

```
# Create graph instance
graph = StateGraph(LLMState)

# Add single node
graph.add_node('llm_qa', llm_qa)

# Define workflow path
graph.add_edge(START, 'llm_qa')
graph.add_edge('llm_qa', END)

# Compile the workflow
workflow = graph.compile()
```

Graph Structure:

- **Node:** llm_qa - processes questions using LLM
- **Edges:** START → llm_qa → END (linear flow)
- **Compilation:** Creates executable workflow

5.20 Execution

```
# Define initial state
initial_state = {'question': 'How far is moon from the earth?'}

# Execute workflow
final_state = workflow.invoke(initial_state)

# Access result
print(final_state['answer'])
```

Output: “The average distance between the Moon and Earth is about 384,400 kilometers (238,855 miles).”

5.21 Comparison with Direct LLM Call

The tutorial shows that this workflow achieves the same result as:

```
model.invoke('How far is moon from the earth?').content
```

Key Point: This simple example demonstrates LangGraph concepts, though direct LLM invocation would be more efficient for such basic tasks.

5.22 Learning Objectives

1. **LangGraph Fundamentals:** Understanding StateGraph, nodes, and edges
2. **State Management:** Using TypedDict for type-safe state handling
3. **Workflow Design:** Creating linear workflows with START/END nodes
4. **Integration:** Combining LangChain and LangGraph components

5.23 Key Takeaways

- LangGraph is overkill for simple linear workflows but becomes powerful for complex scenarios
- State management is central to LangGraph workflows
- The framework provides a structured approach to building AI workflows
- Understanding these basics is essential for more complex workflow implementations

5.24 LangGraph Tutorial Notes: Prompt Chaining Workflow

5.25 Overview

This tutorial demonstrates **Prompt Chaining** - a workflow where multiple LLM calls are made sequentially, with the output of one call becoming input to the next. Instead of generating a blog directly from a topic, we decompose the task into two steps: first generating an outline, then creating the blog using that outline.

5.26 Concept: Prompt Chaining

Definition: A workflow where you interact with the LLM multiple times in sequence, breaking down complex tasks into smaller, manageable steps.

Workflow Structure:

```
START → Generate Outline → Generate Blog → END
```

Why Use Prompt Chaining?

- Better structured output by breaking complex tasks into steps
- Intermediate results are preserved and accessible
- More control over the generation process

5.27 State Definition

```
class BlogState(TypedDict):
    title: str # Input topic for the blog
    outline: str # Generated outline (intermediate result)
    content: str # Final blog content
```

Key Points:

- Three attributes track the progression: input → intermediate → final
- All intermediate states are preserved and accessible
- State evolves through each node in the workflow

5.28 Implementation

5.28.1 Setup and Imports

```
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from typing import TypedDict
from dotenv import load_dotenv

load_dotenv()
model = ChatOpenAI()
```

5.28.2 Node 1: Create Outline

```
def create_outline(state: BlogState) -> BlogState:
    # Extract title from state
    title = state['title']

    # Create prompt for outline generation
    prompt = f'Generate a detailed outline for a blog on the topic - {title}'

    # Call LLM and get outline
    outline = model.invoke(prompt).content

    # Update state with outline
    state['outline'] = outline

    return state
```

5.28.3 Node 2: Create Blog

```
def create_blog(state: BlogState) -> BlogState:
    # Extract both title and outline from state
    title = state['title']
    outline = state['outline']

    # Create prompt using both title and outline
    prompt = f'Write a detailed blog on the title - {title} using the following outline \n {outline}'

    # Call LLM and get blog content
    content = model.invoke(prompt).content

    # Update state with final content
    state['content'] = content

    return state
```

5.28.4 Graph Construction

```
graph = StateGraph(BlogState)

# Add nodes
graph.add_node('create_outline', create_outline)
graph.add_node('create_blog', create_blog)

# Define sequential edges
graph.add_edge(START, 'create_outline')
graph.add_edge('create_outline', 'create_blog')
graph.add_edge('create_blog', END)

# Compile workflow
workflow = graph.compile()
```

5.28.5 Execution

```
initial_state = {'title': 'Rise of AI in India'}
final_state = workflow.invoke(initial_state)

# All states are accessible
print(final_state['title']) # Original topic
print(final_state['outline']) # Generated outline
print(final_state['content']) # Final blog
```

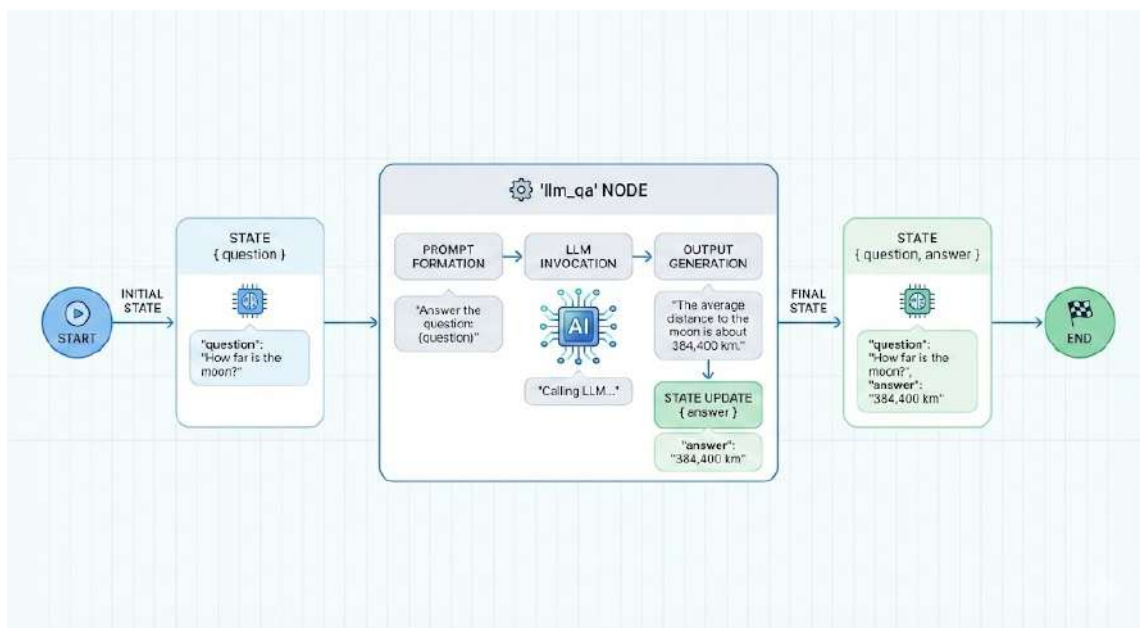


Figure 5.3. Execution

5.29 Sample Output

5.29.1 Generated Outline:

```
I. Introduction
  A. Brief explanation of what AI (Artificial Intelligence) is
  B. Overview of how AI has been steadily growing in India
  C. Purpose of the blog
```

- II. Historical Context of AI in India
 - A. Early developments in AI in India
 - B. Government initiatives and policies to promote AI
 - C. Rise of AI startups in India
- III. Current State of AI in India
 - A. Major industries leveraging AI technology
 - B. Indian companies leading AI innovation
 - C. Impact on Indian economy and job market
- IV. Challenges and Opportunities
 - A. Challenges faced by AI adoption in India
 - B. Opportunities for growth and development
 - C. Importance of AI education and skill development
- V. Future Outlook
 - A. Predictions for the future of AI in India
 - B. Potential areas of growth and innovation
 - C. Implications on society and culture
- VI. Conclusion
 - A. Recap of key points
 - B. Final thoughts on the rise of AI in India
 - C. Call to action for readers

5.30 Key Advantages of LangGraph vs Traditional Chains

5.30.1 State Persistence

- **Traditional Chains:** Only final output is accessible
- **LangGraph:** All intermediate states (title, outline, content) remain accessible

5.30.2 Flexibility

- **Traditional Chains:** Limited access to intermediate results
- **LangGraph:** Full control over state at each step

5.30.3 Debugging

- **Traditional Chains:** Difficult to inspect intermediate steps
- **LangGraph:** Easy to examine state at any point in the workflow

5.31 Homework Assignment

Task: Add a third node called `evaluate_blog` that:

1. Takes the outline and generated blog content
2. Evaluates the blog based on the outline
3. Generates an integer score

Required Changes:

1. **Update BlogState:**

```
class BlogState(TypedDict):
    title: str
    outline: str
    content: str
    score: int # Add this field
```

2. Create Evaluation Node:

```
def evaluate_blog(state: BlogState) -> BlogState:
    outline = state['outline']
    content = state['content']

    prompt = f'Based on this outline, rate my blog and generate an integer score: \nOutline: {outline}\nBlog: {
    ↪ content}'

    score = model.invoke(prompt).content
    state['score'] = int(score) # Convert to integer

    return state
```

3. Update Graph:

```
graph.add_node('evaluate_blog', evaluate_blog)
graph.add_edge('create_blog', 'evaluate_blog')
graph.add_edge('evaluate_blog', END)
```

5.32 Learning Objectives

1. **Sequential Processing:** Understanding how to chain multiple LLM calls
2. **State Evolution:** How state grows and changes through the workflow
3. **Intermediate Access:** Benefit of preserving all intermediate results
4. **Workflow Design:** Breaking complex tasks into manageable steps

5.33 Key Takeaways

- **Prompt Chaining** enables complex task decomposition
- **State management** is crucial for maintaining context across steps
- **LangGraph** provides superior control over workflow execution compared to traditional chains
- **Intermediate results** are preserved and accessible throughout the process
- This approach scales well to more complex, multi-step workflows

LangGraph Sequential Workflows

From Theory to Practice - Complete Tutorial Summary

Course Overview

- Video 1: Generative AI vs Agentic AI
- Video 2: Understanding Agentic AI with use cases
- Video 3: Why LangGraph vs LangChain
- Video 4: Core LangGraph concepts
- **Video 5: Sequential Workflows (This Tutorial)**

Environment Setup

```
python -m venv myenv
myenv\Scripts\activate
pip install langgraph langchain
pip install langchain-openai python-dotenv
```

Sequential Workflow Definition

A workflow where all tasks are connected in linear fashion: Task 1 → Task 2 → Task 3 → END

- No branching or parallel paths
- Single path execution
- Predictable flow
- Clear start-to-finish sequence

Three Progressive Examples

1. BMI Calculator (Non-LLM)

```
graph LR
    START((START)) --> calculate_bmi[calculate_bmi]
    calculate_bmi --> END((END))
```

State:

```
weight_kg: float
height_m: float
bmi: float
category: str
```

```
graph = StateGraph(BMISState)
graph.add_node('calculate_bmi', fn)
graph.add_edge(START, 'calculate_bmi')
workflow = graph.compile()
result = workflow.invoke(state)
```

2. Simple LLM Q&A

```
graph LR
    START((START)) --> llm_qa[llm_qa]
    llm_qa --> END((END))
```

State:

```
question: str
answer: str
```

Integrates ChatOpenAI from LangChain

```
model = ChatOpenAI()
prompt = f'Answer: {question}'
answer = model.invoke(prompt).content
state['answer'] = answer
```

3. Prompt Chaining (Blog)

```
graph LR
    START((START)) --> create_outline[create_outline]
    create_outline --> create_blog[create_blog]
    create_blog --> END((END))
```

State:

```
title: str # input
outline: str # intermediate
content: str # final output
```

- ✓ All intermediate states preserved
- ✓ Better than traditional chains

Homework: Add evaluate_blog node to score blog based on outline

Key LangGraph Concepts

State (TypedDict)	Nodes (Functions)	Edges (Flow)	Workflow Pattern
• Type-safe data container • Flows through workflow • Updated by each node • All data preserved	• Input: state object • Process: business logic • Output: updated state • Type hints required	• Define execution order • START → nodes → END • Sequential connections • No branching (yet)	1. Define State 2. Create Graph 3. Add Nodes & Edges 4. Compile & Execute

Next Steps in the Course

- Complex workflows with branching and conditional logic
- Error handling and robust workflow development
- Advanced LLM integrations and multi-agent systems
- Production deployment patterns and best practices

LangGraph = LangChain + Graph-based Workflows

"LangGraph provides structured approach to building AI workflows with full state management"

Remember: Start simple, add complexity gradually, always use type hints!

Figure 5.4. Key Takeaways

Parallel Workflows in LangGraph

6.1 Code

- <https://github.com/campusx-official/langgraph-tutorials>

6.2 LangGraph Parallel Workflow Notes

6.3 Overview

This tutorial covers creating a **parallel workflow** using LangGraph to analyze cricket batsman statistics. The workflow calculates multiple metrics simultaneously from initial batting data.

6.4 Workflow Structure

6.4.1 Input State (BatsmanState)

```
class BatsmanState(TypedDict):
    runs: int      # Total runs scored
    balls: int     # Total balls faced
    fours: int     # Number of boundaries (4 runs)
    sixes: int     # Number of sixes (6 runs)

    sr: float      # Strike rate
    bpb: float     # Balls per boundary
    boundary_percent: float # Percentage of runs from boundaries
    summary: str   # Final summary text
```

6.5 Node Functions

6.5.1 1. Calculate Strike Rate

```
def calculate_sr(state: BatsmanState):
    sr = (state['runs']/state['balls'])*100
    return {'sr': sr}
```

Formula: $(\text{Runs} \div \text{Balls}) \times 100$

6.5.2 2. Calculate Balls Per Boundary

```
def calculate_bpb(state: BatsmanState):
    bpb = state['balls']/(state['fours'] + state['sixes'])
    return {'bpb': bpb}
```

Formula: $\text{Total Balls} \div (\text{Fours} + \text{Sixes})$

6.5.3 3. Calculate Boundary Percentage

```
def calculate_boundary_percent(state: BatsmanState):  
    boundary_percent = (((state['fours'] * 4) + (state['sixes'] * 6))/state['runs'])*100  
    return {'boundary_percent': boundary_percent}
```

Formula: $((\text{Fours} \times 4 + \text{Sixes} \times 6) \div \text{Total Runs}) \times 100$

6.5.4 4. Generate Summary

```
def summary(state: BatsmanState):  
    summary = f"""  
Strike Rate - {state['sr']}  
Balls per boundary - {state['bpb']}  
Boundary percent - {state['boundary_percent']}  
"""  
    return {'summary': summary}
```

6.6 Graph Construction

6.6.1 Parallel Flow Architecture

6.6.2 Edge Definitions

```
# Parallel edges from START  
graph.add_edge(START, 'calculate_sr')  
graph.add_edge(START, 'calculate_bpb')  
graph.add_edge(START, 'calculate_boundary_percent')  
  
# Convergence edges to summary  
graph.add_edge('calculate_sr', 'summary')  
graph.add_edge('calculate_bpb', 'summary')  
graph.add_edge('calculate_boundary_percent', 'summary')  
  
# Final edge  
graph.add_edge('summary', END)
```

6.7 Key Concepts

6.7.1 ! Partial State Updates

Critical Rule: In parallel workflows, nodes must return **partial state updates**, not the entire state.

X Wrong Approach:

```
return state # Returns entire state - causes conflicts
```

✓ Correct Approach:

```
return {'sr': sr} # Returns only the calculated field
```

6.7.2 Why Partial Updates Are Necessary

When multiple nodes run in parallel and all return the complete state, LangGraph detects potential conflicts:

Parallel Workflow for Cricket Batsman Stats in LangGraph

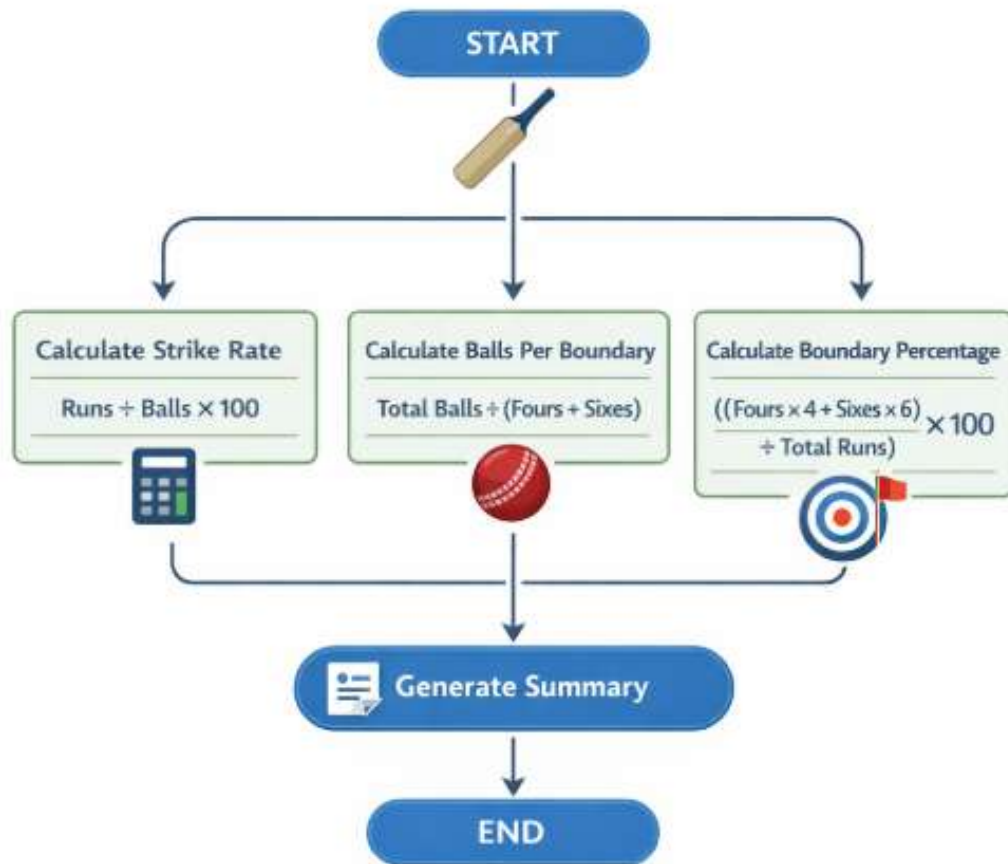


Figure 6.1. Parallel Flow Architecture

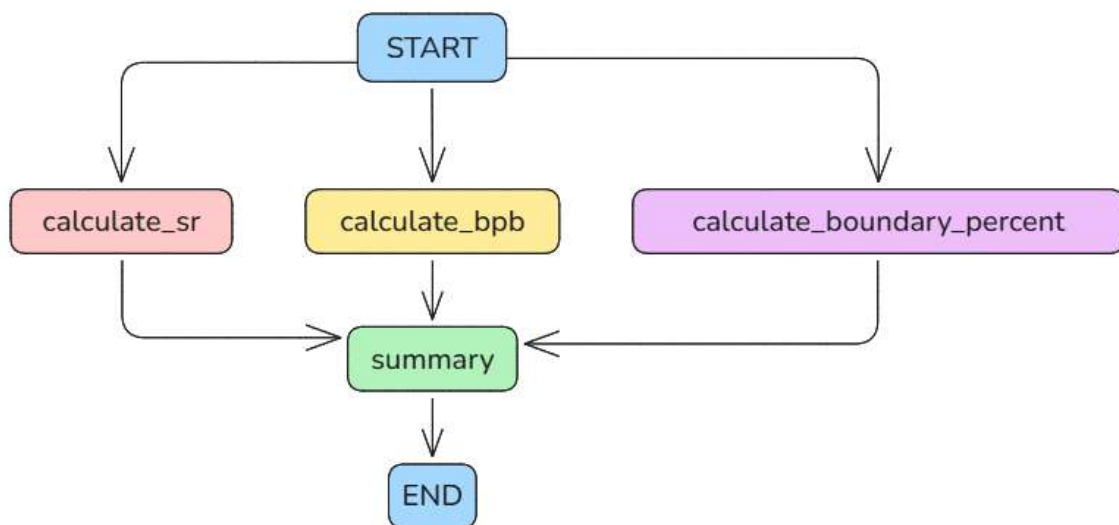


Figure 6.2. Edge Definitions

- Each node appears to modify ALL state fields
- Creates ambiguity about which values are authoritative
- Results in “Invalid Update Error”

6.7.3 Solution Strategy

- Each node returns only the fields it calculates/modifies
- Allows safe parallel execution
- Works for both sequential and parallel workflows
- **Recommended approach for all workflows**

6.8 Sample Execution

6.8.1 Input Data

```
initial_state = {  
    'runs': 100,  
    'balls': 50,  
    'fours': 6,  
    'sixes': 4  
}
```

6.8.2 Output Results

```
{  
    'runs': 100,  
    'balls': 50,  
    'fours': 6,  
    'sixes': 4,  
    'sr': 200.0,           # Strike rate  
    'bpb': 5.0,           # Balls per boundary  
    'boundary_percent': 48.0, # Boundary percentage  
    'summary': 'Strike Rate - 200.0...' # Summary text  
}
```

6.9 Best Practices

1. **Always use partial state updates** - safer and works universally
2. **Design for parallel execution** - even if starting sequential
3. **Clear node responsibilities** - each node calculates specific metrics
4. **Meaningful return values** - return only what the node computes

6.10 Common Pitfalls

- Returning entire state in parallel workflows
- Mixing sequential and parallel update patterns
- Not understanding state conflict resolution
- Forgetting to handle partial state merging

This parallel workflow demonstrates efficient concurrent processing while maintaining state consistency through proper update patterns.

6.11 LLM-Based Parallel Workflow Notes

6.12 Essay Evaluation System Overview

This tutorial creates a **parallel LLM workflow** for evaluating UPSC essays across multiple dimensions simultaneously.

6.12.1 Workflow Architecture

6.13 State Structure

6.13.1 Required State Attributes

```
class EssayState(TypedDict):
    essay: str  # Input essay text

    # Individual feedbacks
    cot_feedback: str  # Clarity of Thought feedback
    doa_feedback: str  # Depth of Analysis feedback
    language_feedback: str  # Language feedback

    # Final summarized feedback
    summarized_feedback: str

    # Scores management
    individual_scores: List[int] # List of 3 scores [COT, DOA, Lang]
    final_score: float  # Average of individual scores
```

6.14 Key Concepts Integration

6.14.1 1. Parallel LLM Processing

- **Three simultaneous evaluations** of the same essay
- Each LLM node evaluates different aspects independently
- Parallel execution improves performance over sequential processing

6.14.2 2. Structured Output Requirements

Problem: LLMs may return inconsistent formats

- Score as “seven” instead of 7
- Unpredictable response structure
- Unreliable data extraction

Solution: Pydantic Schema for Structured Output

```
from pydantic import BaseModel, Field

class EvaluationSchema(BaseModel):
    feedback: str = Field(description="Detailed feedback for the essay")
    score: int = Field(
        description="Score out of 10",
        ge=0, # Greater than or equal to 0
        le=10 # Less than or equal to 10
    )
```

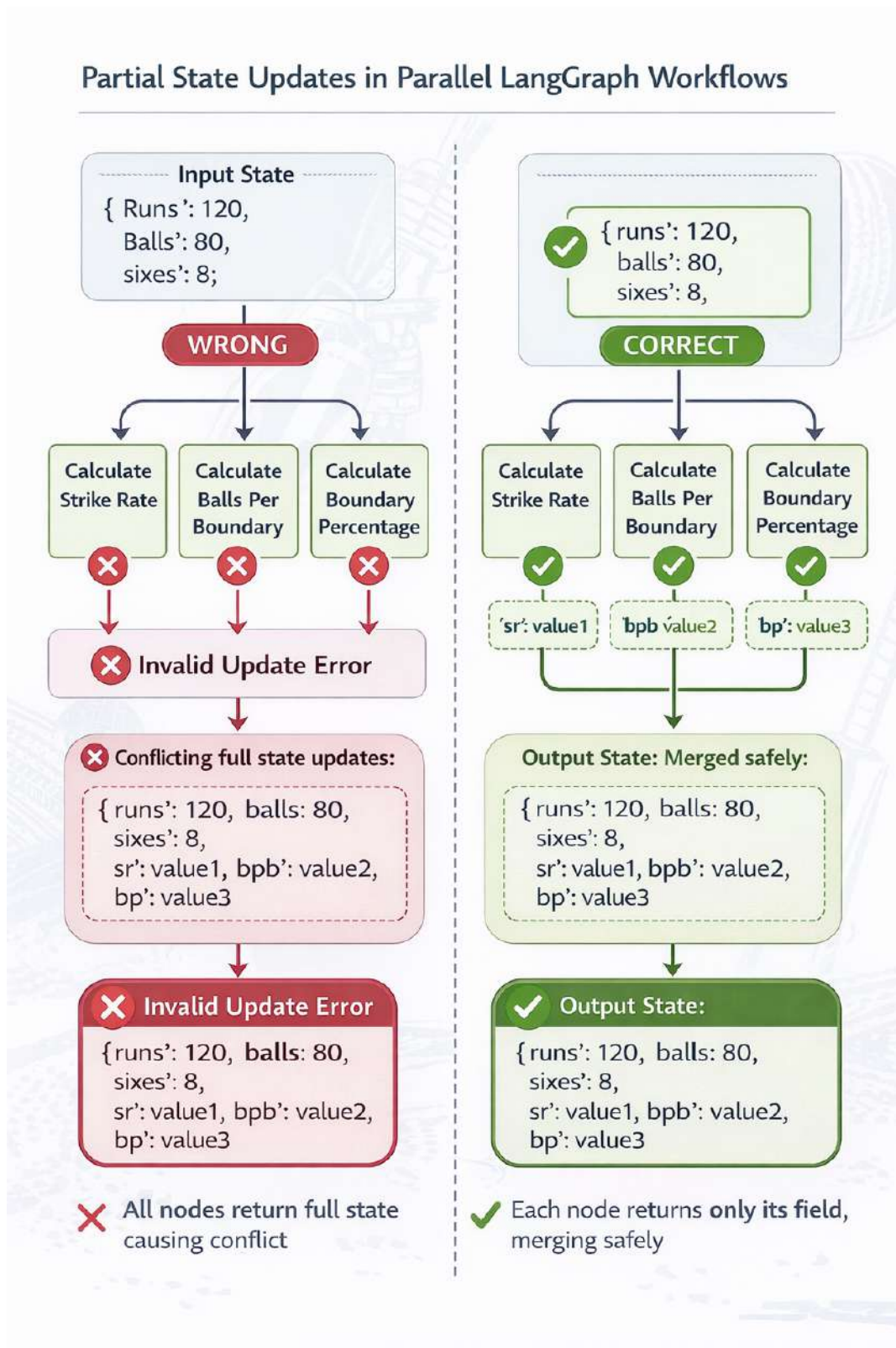


Figure 6.3. Workflow Architecture

6.14.3 3. Reducer Function Necessity

Challenge: Multiple parallel nodes updating same state attribute

Scenario:

- All 3 evaluation nodes need to add scores to `individual_scores` list
- Parallel execution causes conflicts
- Default behavior: replacement instead of merging

Solution: Reducer function for list merging

```
# Without reducer: Only last score survives
# With reducer: All three scores are collected
```

6.15 LLM Setup for Structured Output

6.15.1 Model Selection

```
from langchain_openai import ChatOpenAI

# Use GPT-4o-mini for built-in structured output support
model = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0 # For consistent outputs
)
```

6.15.2 Schema-Bound Model Creation

```
# Bind schema to model for guaranteed structure
structured_model = model.with_structured_output(EvaluationSchema)
```

6.16 Three Core Technical Challenges

6.16.1 1. ✓ Parallel Workflow Management

- **Concept:** Multiple nodes executing simultaneously
- **Implementation:** Proper edge configuration from START
- **Key:** Partial state updates to avoid conflicts

6.16.2 2. ✓ Structured Output Generation

- **Concept:** Consistent, parseable LLM responses
- **Implementation:** Pydantic schemas + model binding
- **Key:** Reliable data extraction from LLM responses

6.16.3 3. Reducer Functions (New Concept)

- **Concept:** Merging parallel updates to same state field
- **Problem:** Multiple nodes adding to same list/collection
- **Solution:** Custom merge logic for state attributes

6.17 Implementation Strategy

6.17.1 Phase 1: Structured Output Setup

1. Define Pydantic schema for evaluation format
2. Create schema-bound LLM model
3. Test structured output reliability

6.17.2 Phase 2: Individual Evaluation Nodes

1. Create three evaluation functions (COT, DOA, Language)
2. Each returns structured feedback + score
3. Use partial state updates for parallel safety

6.17.3 Phase 3: Final Evaluation Node

1. Merge three textual feedbacks using LLM
2. Calculate average from individual scores
3. Return final summarized results

6.17.4 Phase 4: State Management

1. Implement reducer function for score list
2. Configure proper state merging
3. Handle parallel state conflicts

6.18 Benefits of This Architecture

6.18.1 Performance

- **Parallel processing** reduces total evaluation time
- Three evaluations happen simultaneously vs. sequentially

6.18.2 Reliability

- **Structured output** ensures consistent data format
- **Schema validation** prevents parsing errors

6.18.3 Scalability

- Easy to add new evaluation dimensions
- Modular design allows independent node updates

6.18.4 Quality

- **Specialized evaluations** for each aspect
- **Comprehensive feedback** from multiple perspectives

6.19 Real-World Applications

6.19.1 Educational Platforms

- Automated essay grading systems
- Writing assessment tools
- Student feedback generation

6.19.2 Content Evaluation

- Article quality assessment
- Report evaluation workflows
- Multi-criteria content analysis

6.19.3 Business Processes

- Document review workflows
- Proposal evaluation systems
- Quality assurance pipelines

This workflow demonstrates advanced LangGraph concepts by combining parallel processing, structured AI outputs, and sophisticated state management in a practical evaluation system.

6.20 Complete UPSC Essay Evaluation Workflow - Implementation Notes

6.21 Overview

This is a **production-ready implementation** of the UPSC essay evaluation system discussed in the previous tutorial. It demonstrates advanced LangGraph concepts including structured output, reducer functions, and parallel LLM processing.

6.22 Code Structure Analysis

6.22.1 1. Core Imports and Setup

```
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from typing import TypedDict, Annotated
from pydantic import BaseModel, Field
import operator
```

6.22.2 2. Structured Output Schema

```
class EvaluationSchema(BaseModel):
    feedback: str = Field(description='Detailed feedback for the essay')
    score: int = Field(description='Score out of 10', ge=0, le=10)

# Create structured model
model = ChatOpenAI(model='gpt-4o-mini')
structured_model = model.with_structured_output(EvaluationSchema)
```

Key Benefits:

- **Guaranteed format consistency** - always returns feedback + score

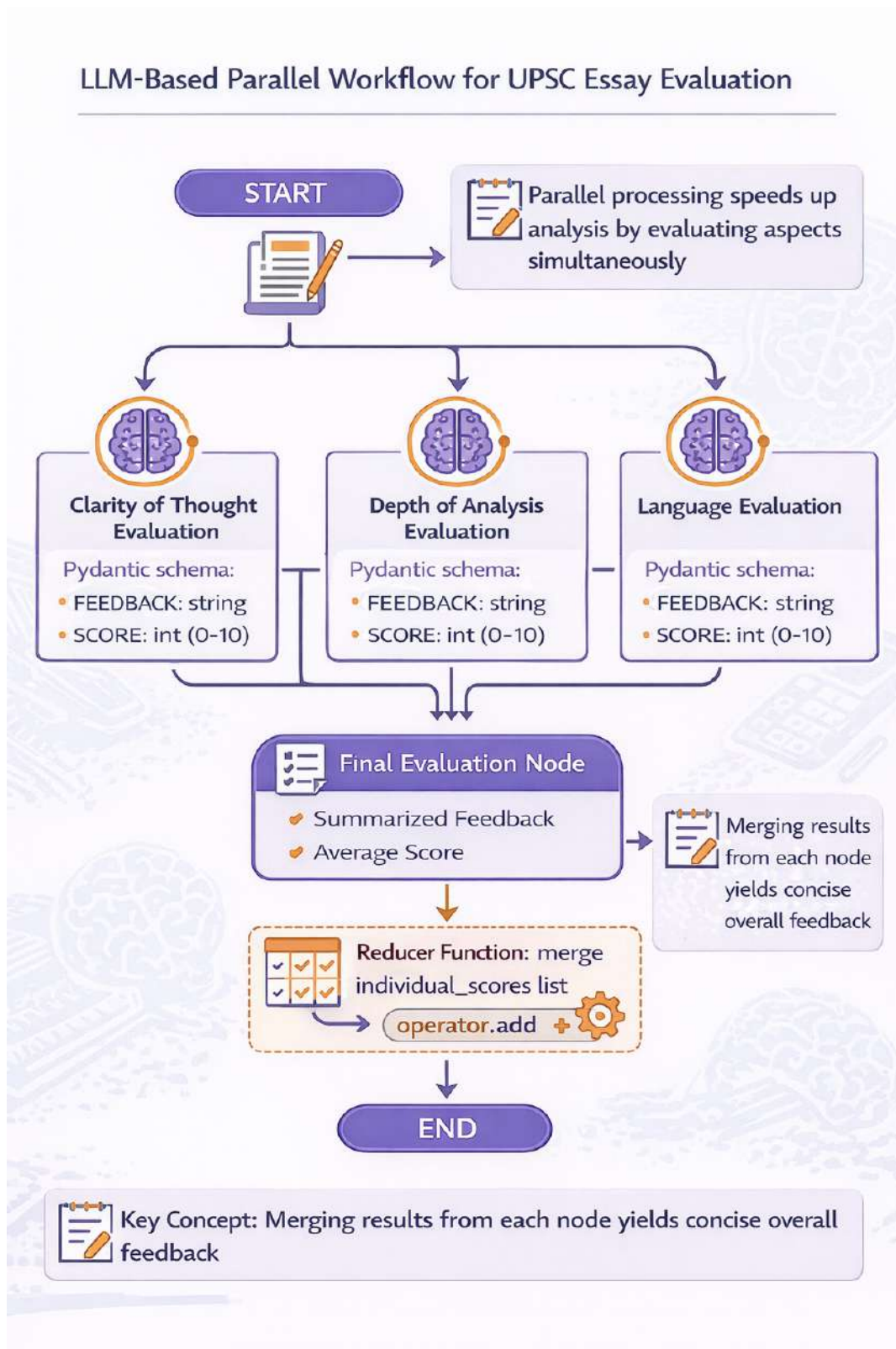


Figure 6.4. Code Structure Analysis

- **Type validation** - score automatically constrained to 0-10 range
- **Parse reliability** - eliminates “seven” vs 7 issues

6.22.3 3. State Definition with Reducer Function

```
class UPSCState(TypedDict):
    essay: str # Input essay text
    language_feedback: str # Language evaluation feedback
    analysis_feedback: str # Analysis evaluation feedback
    clarity_feedback: str # Clarity evaluation feedback
    overall_feedback: str # Summarized final feedback

    # Critical: Reducer function for parallel score collection
    individual_scores: Annotated[list[int], operator.add]
    avg_score: float # Final averaged score
```

Reducer Function Explained:

- Annotated[list[int], operator.add] enables **list merging**
- Without reducer: [8], [7], [6] $\rightarrow$ [6] (last overwrites)
- With reducer: [8], [7], [6] $\rightarrow$ [8, 7, 6] (all scores preserved)

6.23 Node Implementation Details

6.23.1 Individual Evaluation Nodes

```
def evaluate_language(state: UPSCState):
    prompt = f'Evaluate the language quality of the following essay and provide a feedback and assign a score out of \n {state["essay"]}'
    output = structured_model.invoke(prompt)

    return {
        'language_feedback': output.feedback,
        'individual_scores': [output.score] # Single-item list for reducer
    }
```

Pattern Analysis:

- Each node returns **partial state updates only**
- Score wrapped in [output.score] for reducer compatibility
- Structured model guarantees consistent output format

6.23.2 Final Evaluation Node

```
def final_evaluation(state: UPSCState):
    # Combine three feedbacks using regular LLM
    prompt = f'Based on the following feedbacks create a summarized feedback \n language feedback - {state["language_feedback"]} \n depth of analysis feedback - {state["analysis_feedback"]} \n clarity of thought \n {state["clarity_feedback"]}'
    overall_feedback = model.invoke(prompt).content # Note: regular model, not structured

    # Calculate average from collected scores
    avg_score = sum(state['individual_scores'])/len(state['individual_scores'])

    return {
        'overall_feedback': overall_feedback,
        'avg_score': avg_score
    }
```

Key Decisions:

- Uses **regular model** for summarization (no structured output needed)
- Calculates average from `individual_scores` list populated by reducer
- Returns both synthesized feedback and quantitative score

6.24 Graph Construction

6.24.1 Parallel Architecture Setup

```
graph = StateGraph(UPSCState)

# Add all nodes
graph.add_node('evaluate_language', evaluate_language)
graph.add_node('evaluate_analysis', evaluate_analysis)
graph.add_node('evaluate_thought', evaluate_thought)
graph.add_node('final_evaluation', final_evaluation)

# Parallel edges from START
graph.add_edge(START, 'evaluate_language')
graph.add_edge(START, 'evaluate_analysis')
graph.add_edge(START, 'evaluate_thought')

# Convergence edges to final evaluation
graph.add_edge('evaluate_language', 'final_evaluation')
graph.add_edge('evaluate_analysis', 'final_evaluation')
graph.add_edge('evaluate_thought', 'final_evaluation')

graph.add_edge('final_evaluation', END)
```

6.25 Real-World Testing Results

6.25.1 High-Quality Essay Results

```
# Professional essay about "India in the Age of AI"
{
    'individual_scores': [8, 8, 8], # Consistent high scores
    'avg_score': 8.0,
    'language_feedback': 'Well-structured, formal language...',
    'analysis_feedback': 'Comprehensive discussion...',
    'clarity_feedback': 'Clear flow of ideas...'
}
```

6.25.2 Low-Quality Essay Results

```
# Deliberately poor essay with grammar issues
{
    'individual_scores': [4, 3, 4], # Lower scores across dimensions
    'avg_score': 3.67,
    'language_feedback': 'Poor grammar, awkward phrasing...',
    'analysis_feedback': 'Shallow analysis, lacks evidence...',
    'clarity_feedback': 'Disorganized, lacks structure...'
}
```

6.26 Technical Architecture Benefits

6.26.1 1. Performance Optimization

- **3x speed improvement** through parallel evaluation
- Three LLM calls execute simultaneously vs. sequentially

6.26.2 2. Reliability Engineering

- **Structured output** eliminates parsing failures
- **Type validation** prevents runtime errors
- **Consistent scoring** through schema constraints

6.26.3 3. Scalable Design

- Easy to add new evaluation dimensions
- Modular node architecture
- State management handles complexity automatically

6.26.4 4. Quality Assurance

- **Multi-dimensional analysis** provides comprehensive feedback
- **Quantitative scoring** enables objective comparison
- **Synthesized feedback** gives holistic evaluation

6.27 Advanced State Management

6.27.1 Reducer Function Deep Dive

```
# How reducer function works internally:
# Node 1 returns: {'individual_scores': [8]}
# Node 2 returns: {'individual_scores': [7]}
# Node 3 returns: {'individual_scores': [6]}

# operator.add performs: [8] + [7] + [6] = [8, 7, 6]
# Final state: {'individual_scores': [8, 7, 6]}
```

Alternative Reducers Available:

- `operator.mul` - Multiply values
- `max` - Keep maximum value
- `min` - Keep minimum value
- Custom functions for complex merging logic

6.28 Production Considerations

6.28.1 Error Handling

- **Structured output** prevents most parsing errors
- **Schema validation** catches invalid scores automatically
- **Type hints** enable static analysis

6.28.2 Performance Monitoring

- Track individual node execution times
- Monitor LLM token usage across parallel calls
- Measure end-to-end evaluation latency

6.28.3 Quality Control

- **Consistent evaluation criteria** across all dimensions
- **Reproducible scoring** through temperature=0 setting
- **Audit trail** with detailed feedback for each dimension

6.29 Integration Possibilities

6.29.1 Educational Platforms

- **Automated essay scoring** for competitive exams
- **Detailed feedback generation** for student improvement
- **Bulk evaluation** of student submissions

6.29.2 Content Management

- **Article quality assessment** for publications
- **Report evaluation** in corporate settings
- **Multi-criteria content analysis** pipelines

6.29.3 Assessment Tools

- **Writing skill evaluation** across different domains
- **Language proficiency testing** with detailed breakdowns
- **Academic paper review** automation

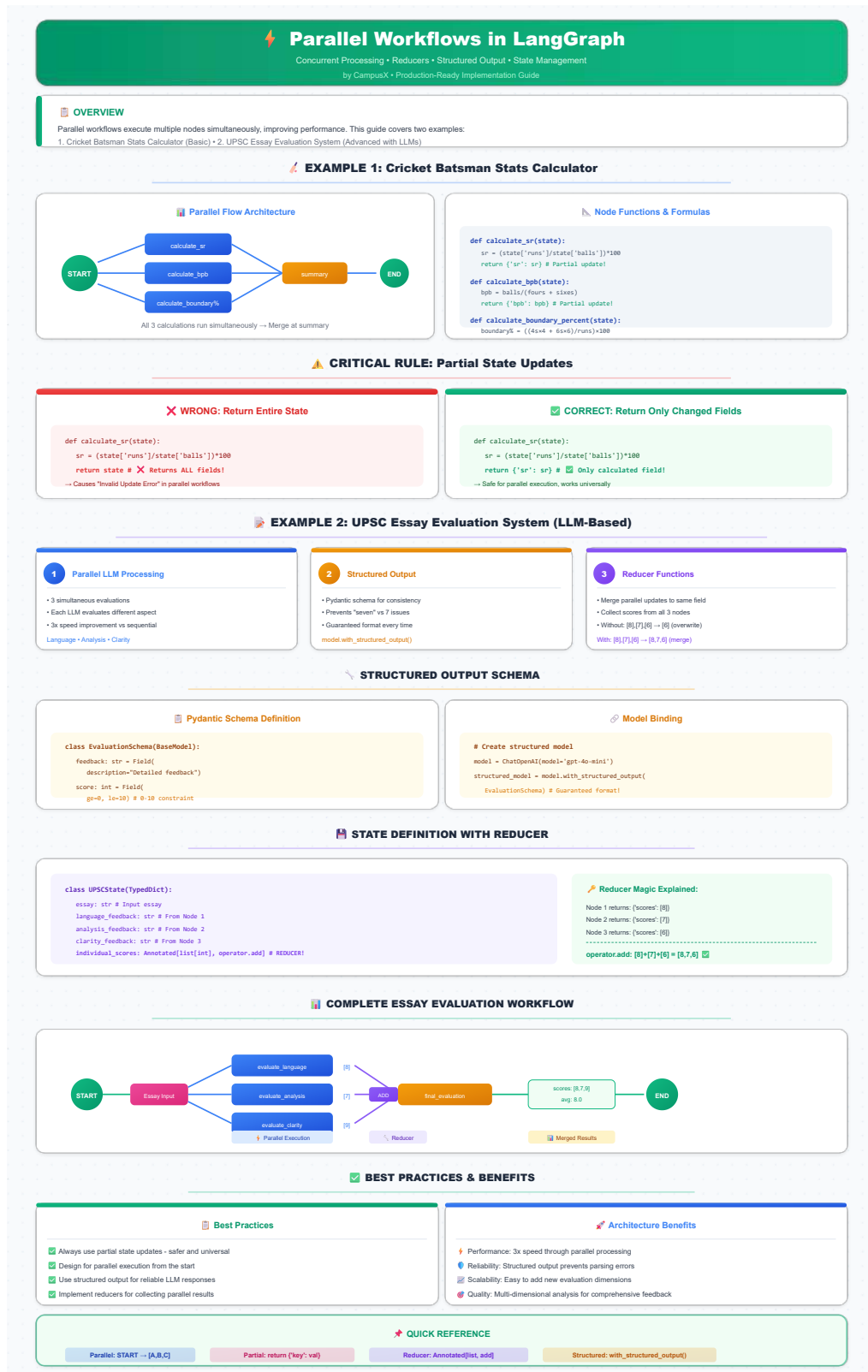


Figure 6.5. Assessment Tools

Conditional Workflows in LangGraph - Complete Guide

7.1 Code

- <https://github.com/campusx-official/langgraph-tutorials>

7.2 1. Introduction

Conditional workflows are one of the most important concepts in LangGraph, analogous to how if-else statements work in programming. They allow you to create intelligent systems that can make decisions and route execution flow based on specific conditions. This guide covers conditional workflows in depth with two practical examples: a mathematical quadratic equation solver and an LLM-based customer review handling system.

Why Conditional Workflows Matter:

- Enable decision-making capabilities in AI agents
- Allow dynamic routing based on runtime conditions
- Essential for building complex, real-world applications
- Foundation for more advanced workflow patterns

7.3 2. Understanding Workflow Types in LangGraph

Before diving into conditional workflows, let's understand the three main types of workflows in LangGraph.

7.3.1 2.1 Sequential Workflows

A sequential workflow executes tasks **one after another in a linear order**. Each task must complete before the next one begins.

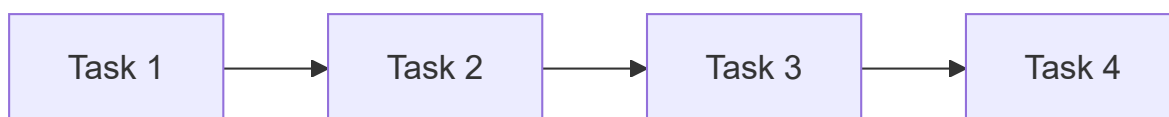


Figure 7.1. 1 Sequential Workflows

Characteristics:

- Tasks execute in a strict linear sequence
- Output of one task feeds into the next
- Simple and predictable execution order

- Best for step-by-step processes

Example Use Cases:

- Data preprocessing pipelines
- Document processing (read → analyze → summarize)
- Simple chatbot flows

7.3.2 2.2 Parallel Workflows

A parallel workflow allows **multiple tasks to execute simultaneously**. After a branch point, multiple paths run concurrently, and their outputs converge at a synchronization point.

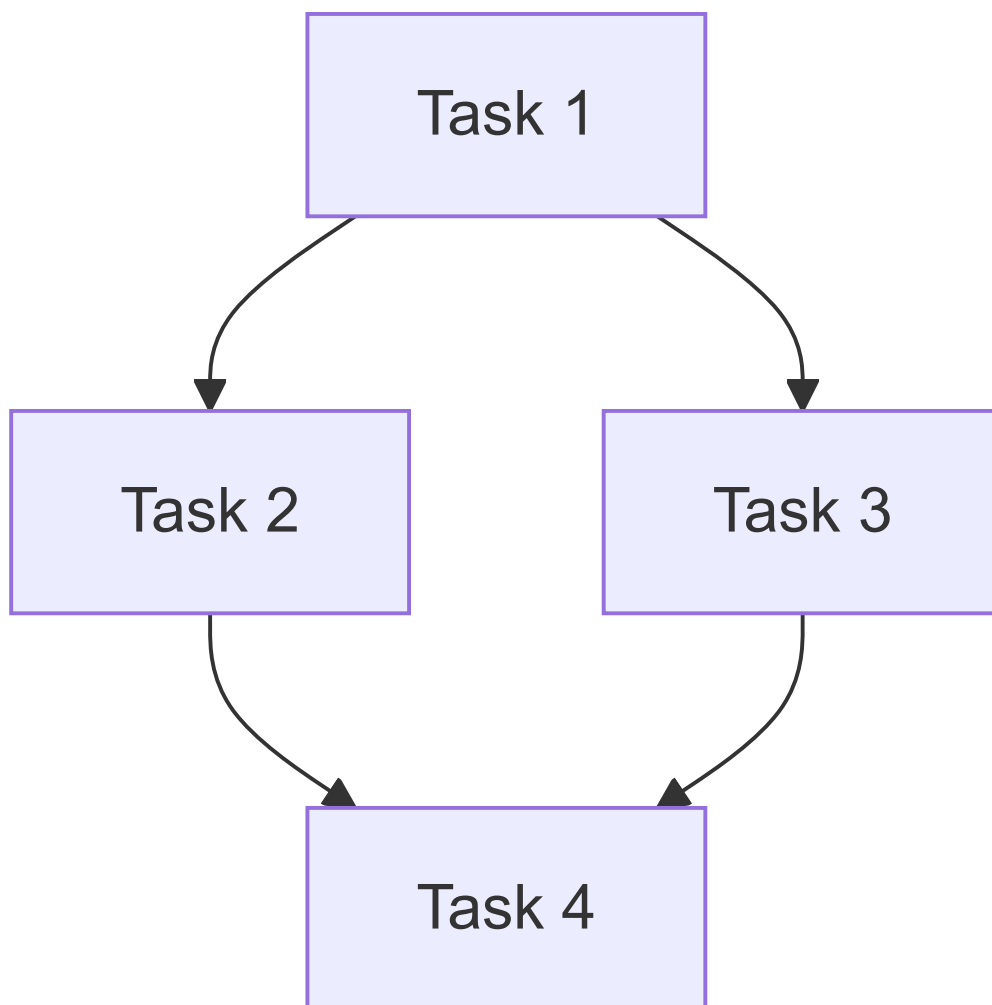


Figure 7.2. 2 Parallel Workflows

Characteristics:

- Multiple tasks run at the same time
- Branching occurs at specific points
- All branches converge before continuing
- Improves performance through parallelization

Example Use Cases:

- Fetching data from multiple APIs simultaneously
- Running multiple analysis tasks on the same input
- Parallel document processing

7.3.3 2.3 Conditional Workflows

A conditional workflow looks similar to parallel workflows but has a **critical difference**: you only enter **one branch** based on a condition, not all branches simultaneously.

Characteristics:

- Only one branch executes based on condition evaluation
- Similar to if-else statements in programming
- Enables intelligent decision-making
- Critical for building adaptive AI systems

Example Use Cases:

- Sentiment-based response routing
- Error handling and recovery paths
- Multi-class classification responses
- Customer support ticket routing

7.4 3. Key Differences: Parallel vs Conditional Workflows

Table 7.1. Key Differences: Parallel vs Conditional Workflows

Feature	Parallel Workflow	Conditional Workflow
Branch Execution	All branches execute simultaneously	Only ONE branch executes
Decision Mechanism	No condition needed	Based on condition evaluation
Use Case	Speed up processing via parallelization	Make decisions based on data
Analogy	Fork-Join pattern	If-Else statements
Example	Fetch from API 1 AND API 2	If positive sentiment THEN thank, ELSE diagnose
Edge Type	Regular edges (solid lines)	Conditional edges (dotted lines)

7.5 4. How Conditional Workflows Work

The mechanism of conditional workflows in LangGraph involves three main components:

7.5.1 4.1 The Routing Function

A routing function is a **regular Python function** that:

- Takes the current state as input
- Evaluates conditions based on state values
- Returns the **name of the next node** to execute

```
def check_condition(state: MyState) -> Literal["option_a", "option_b", "option_c"]:
    if state['value'] > 0:
        return "option_a"
    elif state['value'] == 0:
        return "option_b"
    else:
        return "option_c"
```

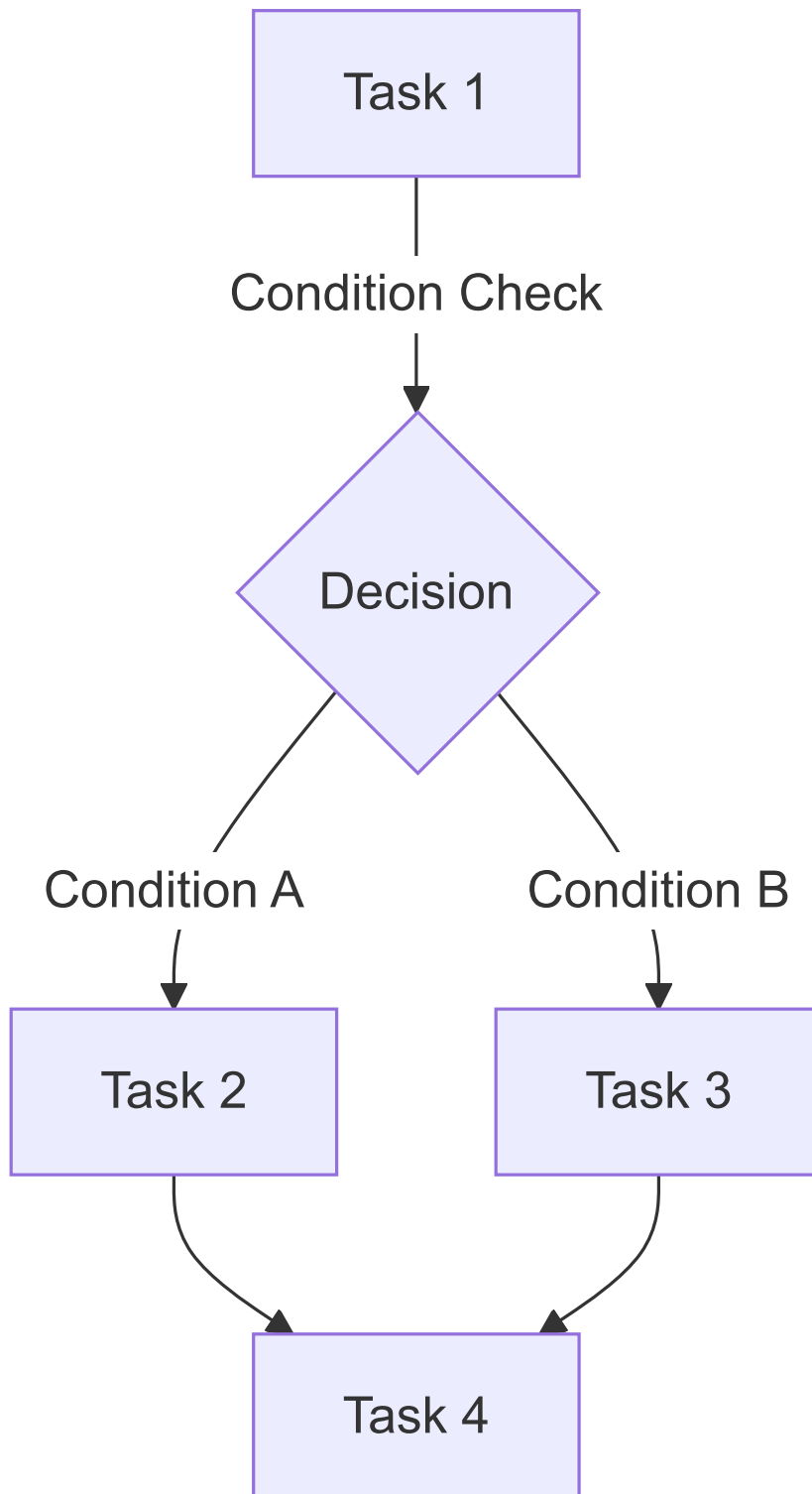


Figure 7.3. 3 Conditional Workflows

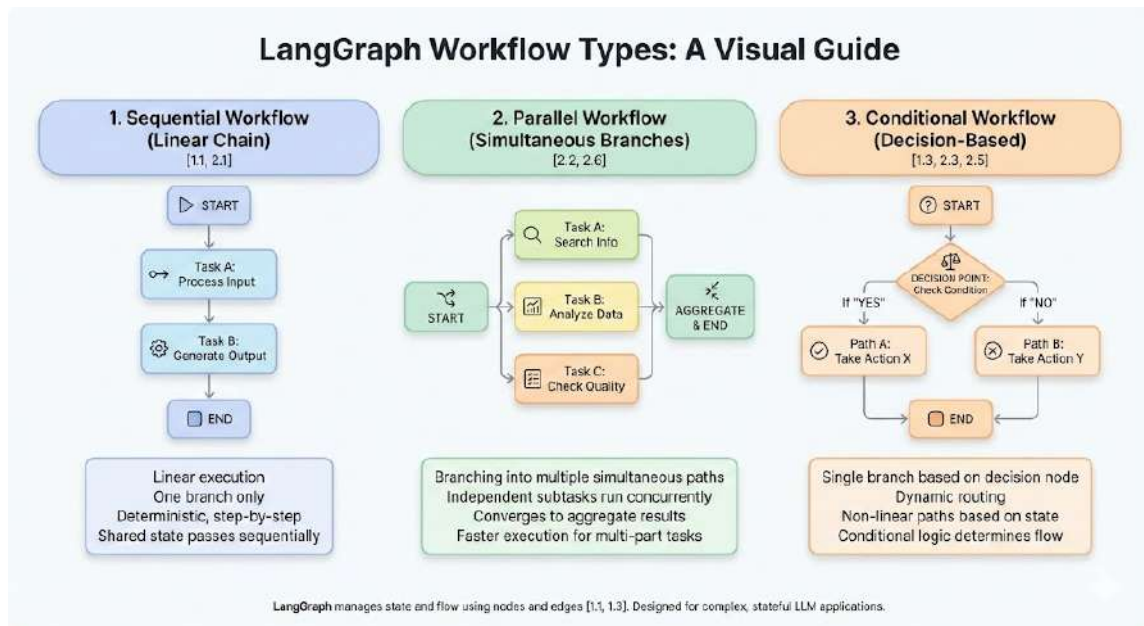


Figure 7.4. 3 Conditional Workflows

7.5.2 4.2 The `add_conditional_edges` Method

Instead of `add_edge`, you use `add_conditional_edges` to create conditional routing:

```
# Regular edge (always follows this path)
graph.add_edge('node_a', 'node_b')

# Conditional edge (path determined at runtime)
graph.add_conditional_edges('source_node', routing_function)
```

7.5.3 4.3 Execution Flow

1. Workflow reaches the source node of conditional edges
2. Source node completes execution
3. Routing function is called with current state
4. Function returns the name of the next node
5. Execution continues to that specific node

7.6 5. Example 1: Quadratic Equation Solver (Non-LLM Based)

This example demonstrates conditional workflows using a mathematical problem: solving quadratic equations. It's a pure Python implementation without any LLM involvement, making it perfect for understanding the core concepts.

7.6.1 5.1 Mathematical Background

A **quadratic equation** is a second-degree polynomial equation of the form:

$$ax^2 + bx + c = 0$$

Where:

- a , b , and c are coefficients (constants)

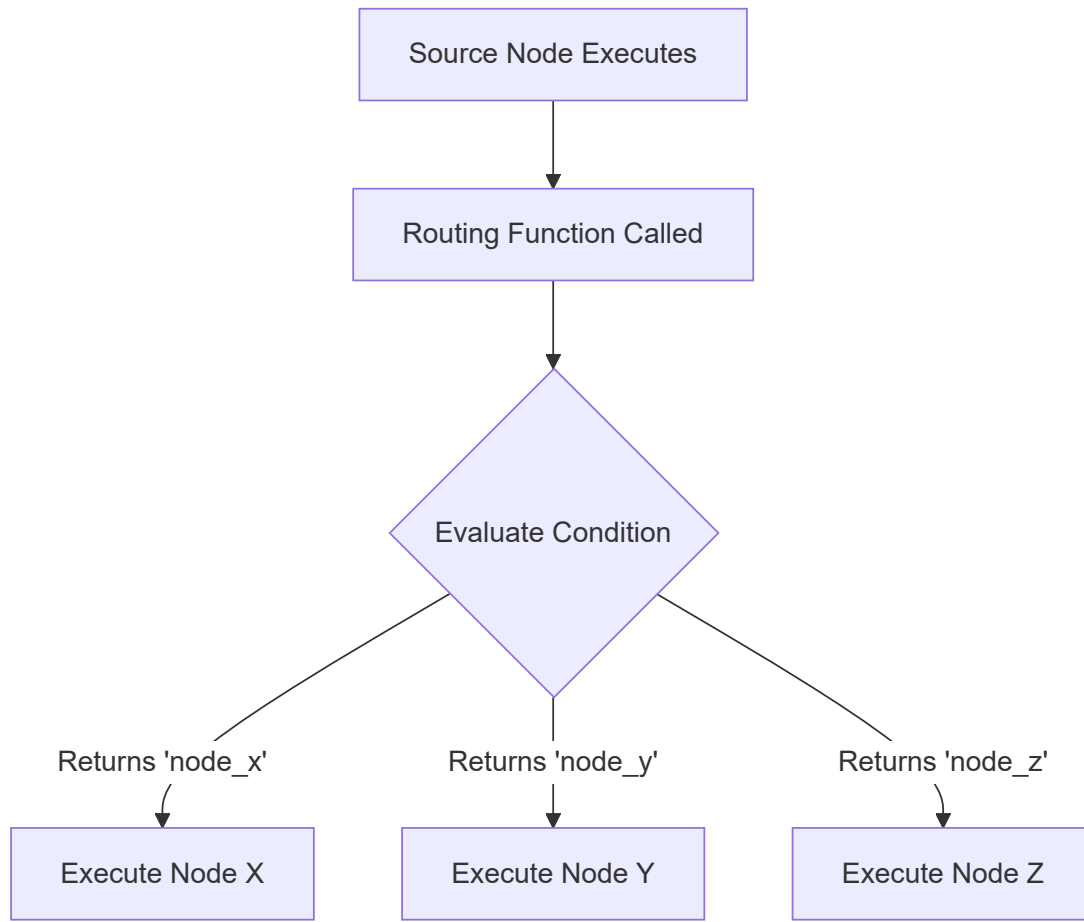


Figure 7.5. 3 Execution Flow

- $a \neq 0$ (otherwise it wouldn't be quadratic)
- x is the variable we want to solve for

The Discriminant

To find the roots (solutions) of a quadratic equation, we first calculate the **discriminant**:

$$D = b^2 - 4ac$$

The discriminant determines the **nature and number of roots**:

Table 7.2. The Discriminant

Condition	Discriminant Value	Type of Roots	Formula
$D > 0$	Positive	Two distinct real roots	$x = \frac{-b \pm \sqrt{D}}{2a}$
$D = 0$	Zero	One repeated real root	$x = \frac{-b}{2a}$
$D < 0$	Negative	No real roots	Complex roots only

Root Calculation Formulas

Case 1: Two Real Roots ($D > 0$)

$$x_1 = \frac{-b + \sqrt{D}}{2a}$$

$$x_2 = \frac{-b - \sqrt{D}}{2a}$$

Case 2: One Repeated Root ($D = 0$)

$$x = \frac{-b}{2a}$$

Case 3: No Real Roots ($D < 0$)

No real number solutions exist. The roots would be complex numbers.

7.6.2 5.2 Workflow Design

Based on the mathematical logic, our workflow needs conditional branching after calculating the discriminant:

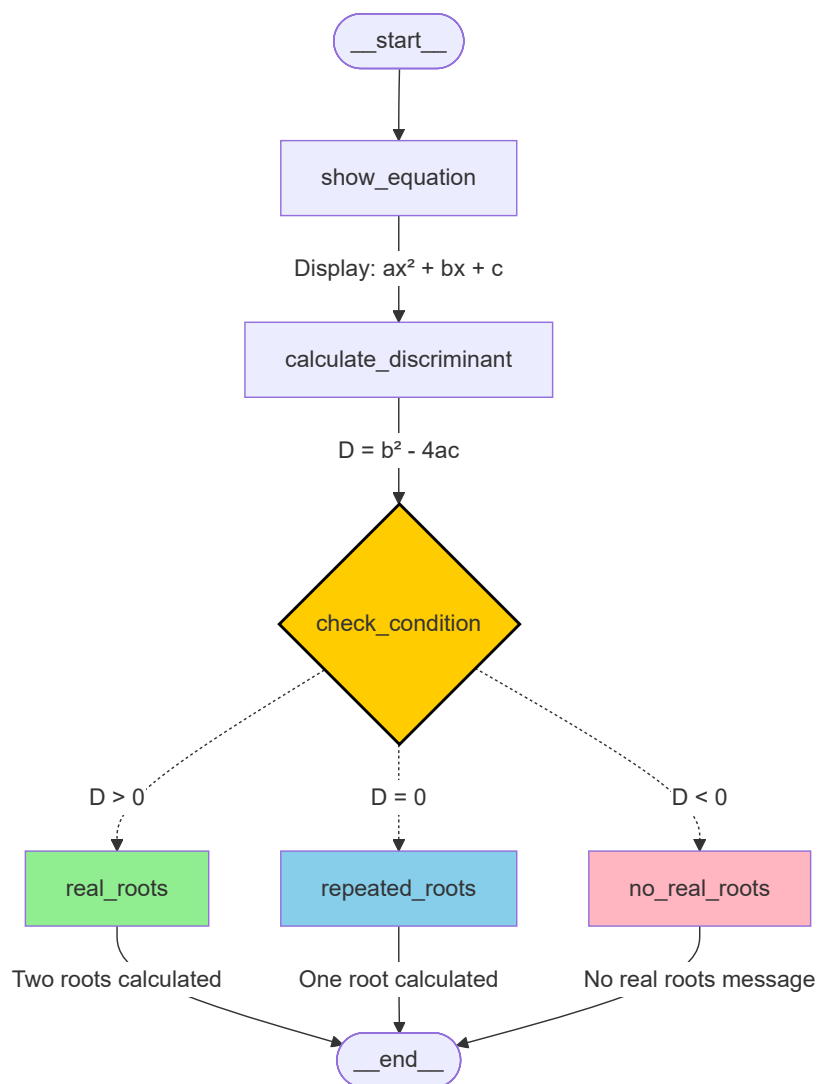


Figure 7.6. 2 Workflow Design

Workflow Steps:

1. **Start:** Receive coefficients a, b, c as input
2. **show_equation:** Display the quadratic equation in readable format

3. **calculate_discriminant**: Compute $D = b^2 - 4ac$
4. **Conditional Routing**: Based on discriminant value:
 - If $D > 0 \rightarrow$ **real_roots**: Calculate two distinct roots
 - If $D = 0 \rightarrow$ **repeated_roots**: Calculate single repeated root
 - If $D < 0 \rightarrow$ **no_real_roots**: Return “no real roots” message
5. **End**: Return the result

7.6.3 5.3 Complete Implementation

```

from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Literal

# =====
# STEP 1: Define the State Schema
# =====
class QuadState(TypedDict):
    """
    State schema for the Quadratic Equation Solver workflow.

    Attributes:
        a: Coefficient of x2 (must be non-zero)
        b: Coefficient of x
        c: Constant term
        equation: String representation of the equation
        discriminant: Calculated discriminant value (b2 - 4ac)
        result: Final result message with roots
    """
    a: int
    b: int
    c: int
    equation: str
    discriminant: float
    result: str

# =====
# STEP 2: Define Node Functions
# =====

def show_equation(state: QuadState) -> dict:
    """
    Node 1: Display the quadratic equation in readable format.

    Takes coefficients a, b, c and creates a string representation
    of the equation: ax2 + bx + c = 0
    """
    a, b, c = state["a"], state["b"], state["c"]

    # Build equation string with proper signs
    equation = f'{a}x2 '
    equation += f'+ {b}x ' if b >= 0 else f'- {abs(b)}x '
    equation += f'+ {c} = 0' if c >= 0 else f'- {abs(c)} = 0'

    return {'equation': equation}

def calculate_discriminant(state: QuadState) -> dict:
    """
    Node 2: Calculate the discriminant.

    Formula: D = b2 - 4ac

    The discriminant determines the nature of roots:
    - D > 0: Two distinct real roots
    - D = 0: One repeated real root
    - D < 0: No real roots (complex roots only)
    """
    b_squared = state["b"] ** 2
    four_ac = 4 * state["a"] * state["c"]
    discriminant = b_squared - four_ac

```

```

    return {'discriminant': discriminant}

def real_roots(state: QuadState) -> dict:
    """
    Node 3a: Calculate two distinct real roots.

    Called when discriminant > 0

    Formulas:
        root1 = (-b + √D) / 2a
        root2 = (-b - √D) / 2a
    """
    a = state["a"]
    b = state["b"]
    d = state["discriminant"]

    # Calculate both roots
    root1 = (-b + d**0.5) / (2 * a)
    root2 = (-b - d**0.5) / (2 * a)

    result = f'Two real roots: 1x = {root1:.4f} and 2x = {root2:.4f}'

    return {'result': result}

def repeated_roots(state: QuadState) -> dict:
    """
    Node 3b: Calculate single repeated root.

    Called when discriminant = 0

    Formula:
        root = -b / 2a
    """
    a = state["a"]
    b = state["b"]

    root = -b / (2 * a)

    result = f'One repeated root: x = {root:.4f}'

    return {'result': result}

def no_real_roots(state: QuadState) -> dict:
    """
    Node 3c: Return message for no real roots.

    Called when discriminant < 0

    When D < 0, square root of D is imaginary,
    so no real number solutions exist.
    """
    result = 'No real roots (discriminant is negative)'

    return {'result': result}

# =====
# STEP 3: Define Routing Function
# =====

def check_condition(state: QuadState) -> Literal["real_roots", "repeated_roots", "no_real_roots"]:
    """
    Routing function for conditional edges.

    Evaluates the discriminant and returns the name of the
    appropriate node to execute next.

    Returns:
        "real_roots" if discriminant > 0
        "repeated_roots" if discriminant == 0
        "no_real_roots" if discriminant < 0
    """
    d = state['discriminant']

```

```

    if d > 0:
        return "real_roots"
    elif d == 0:
        return "repeated_roots"
    else:
        return "no_real_roots"

# =====
# STEP 4: Build the Graph
# =====

# Initialize the graph with state schema
graph = StateGraph(QuadState)

# Add all nodes
graph.add_node('show_equation', show_equation)
graph.add_node('calculate_discriminant', calculate_discriminant)
graph.add_node('real_roots', real_roots)
graph.add_node('repeated_roots', repeated_roots)
graph.add_node('no_real_roots', no_real_roots)

# Add sequential edges
graph.add_edge(START, 'show_equation')
graph.add_edge('show_equation', 'calculate_discriminant')

# Add CONDITIONAL edges (the key part!)
graph.add_conditional_edges('calculate_discriminant', check_condition)

# Add edges from all conditional branches to END
graph.add_edge('real_roots', END)
graph.add_edge('repeated_roots', END)
graph.add_edge('no_real_roots', END)

# Compile the workflow
workflow = graph.compile()

# =====
# STEP 5: Test the Workflow
# =====

# Test Case 1: Two real roots (D > 0)
# Equation:  $x^2 - 5x + 6 = 0 \rightarrow$  roots: 2 and 3
test1 = {'a': 1, 'b': -5, 'c': 6}
result1 = workflow.invoke(test1)
print("Test 1:", result1)

# Test Case 2: One repeated root (D = 0)
# Equation:  $x^2 - 4x + 4 = 0 \rightarrow$  root: 2
test2 = {'a': 1, 'b': -4, 'c': 4}
result2 = workflow.invoke(test2)
print("Test 2:", result2)

# Test Case 3: No real roots (D < 0)
# Equation:  $x^2 + x + 1 = 0 \rightarrow$  no real roots
test3 = {'a': 1, 'b': 1, 'c': 1}
result3 = workflow.invoke(test3)
print("Test 3:", result3)

```

7.6.4 5.4 Code Walkthrough

Step 1: State Definition

```

class QuadState(TypedDict):
    a: int          # Coefficient of  $x^2$ 
    b: int          # Coefficient of  $x$ 
    c: int          # Constant term

```

```
equation: str    # String representation
discriminant: float #  $b^2 - 4ac$ 
result: str     # Final answer
```

The state acts as a **shared data container** that flows through all nodes. Each node can read from and write to this state.

Step 2: Node Functions

Each node function:

- Receives the current state as input
- Performs its specific computation
- Returns a dictionary with updated values

Important: Node functions don't modify state directly. They return a dictionary with keys to update, and LangGraph handles the state mutation.

Step 3: The Routing Function

```
def check_condition(state: QuadState) -> Literal["real_roots", "repeated_roots", "no_real_roots"]:
    d = state['discriminant']

    if d > 0:
        return "real_roots"      # String matches node name exactly
    elif d == 0:
        return "repeated_roots" # String matches node name exactly
    else:
        return "no_real_roots"  # String matches node name exactly
```

Key Points:

- Return type uses `Literal` to specify exact possible values
- Returns **strings** that **exactly match** the node names
- Evaluated after the source node completes
- Has access to full state for decision making

Step 4: Conditional Edges

```
# Regular edges - always follows this path
graph.add_edge(START, 'show_equation')
graph.add_edge('show_equation', 'calculate_discriminant')

# Conditional edges - path determined by check_condition function
graph.add_conditional_edges('calculate_discriminant', check_condition)
```

The `add_conditional_edges` method takes:

1. **Source node name:** Where the conditional branching starts
2. **Routing function:** Function that determines which node to go to

7.7 6. Example 2: Customer Review Handling System (LLM-Based)

This example demonstrates a more complex conditional workflow that uses LLMs for sentiment analysis and response generation.

7.7.1 6.1 System Overview

The system processes customer reviews and generates appropriate responses based on sentiment:

For Positive Reviews:

- Generate a warm thank-you message
- Ask for feedback on the website

For Negative Reviews:

- Diagnose the issue (type, tone, urgency)
- Generate an empathetic resolution message

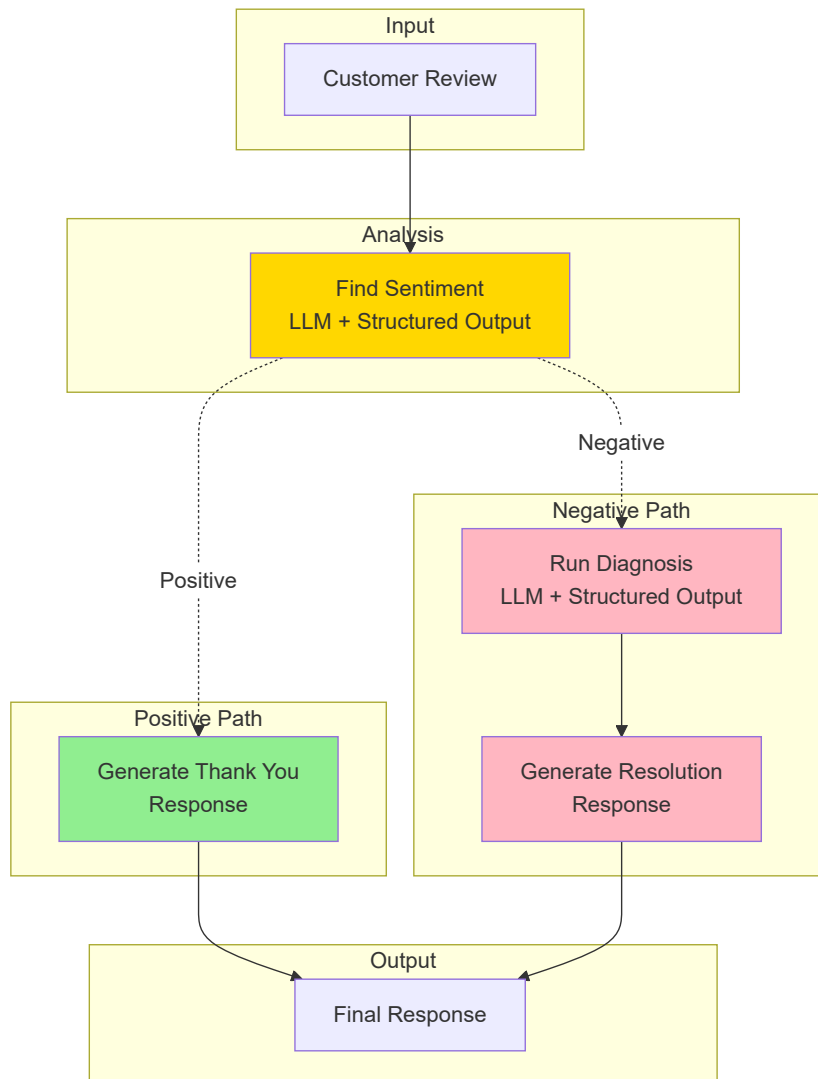


Figure 7.7. 1 System Overview

7.7.2 6.2 Workflow Design

Workflow Steps:

1. **find_sentiment:** LLM analyzes review and outputs structured sentiment (positive/negative)

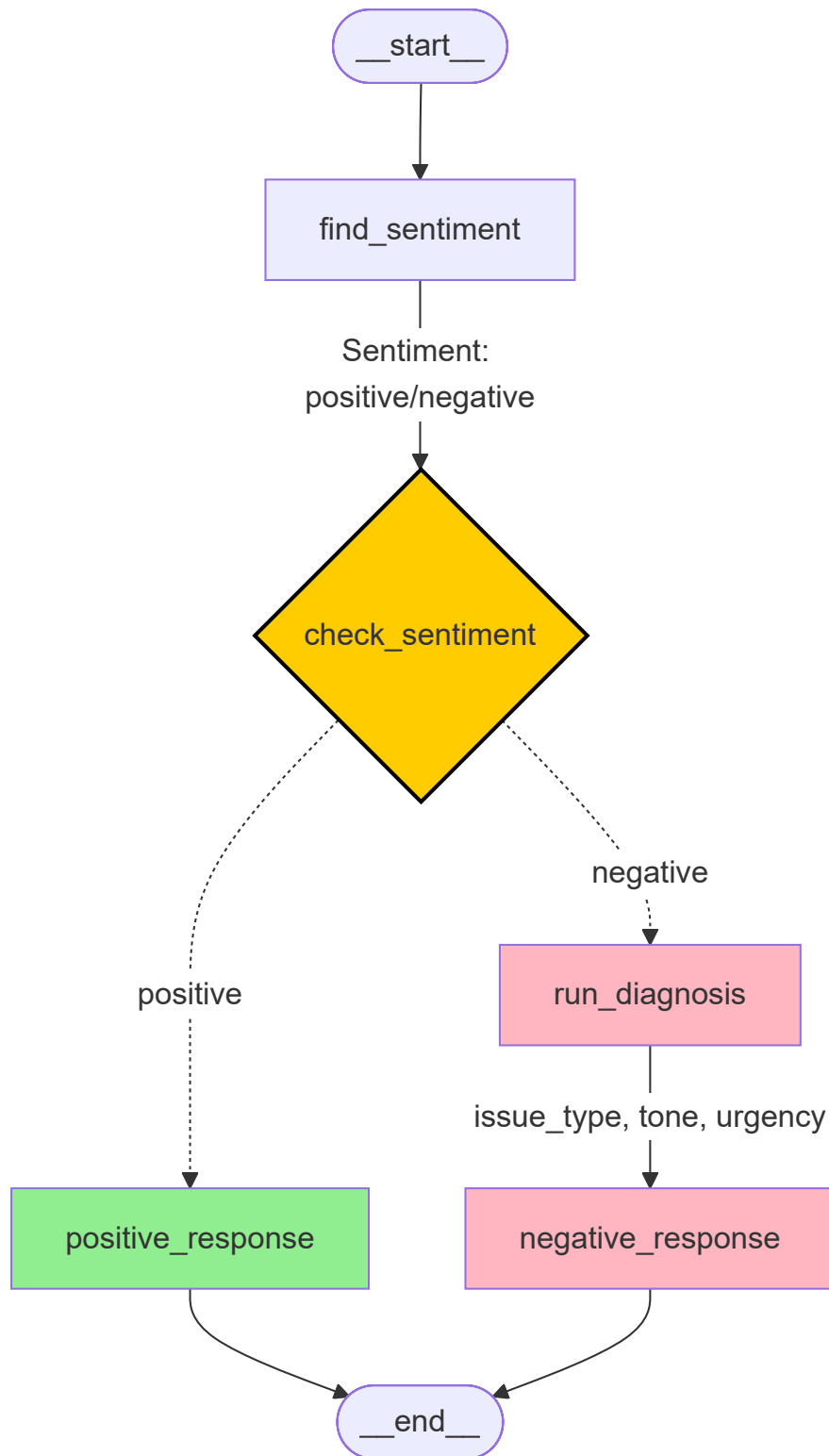


Figure 7.8. 2 Workflow Design

2. Conditional Routing: Based on sentiment:

- Positive → **positive_response**: Generate thank-you message
- Negative → **run_diagnosis**: Extract issue details

3. **run_diagnosis** (negative path only): LLM extracts:

- Issue Type (UX, Performance, Bug, Support, Other)
- Tone (angry, frustrated, disappointed, calm)
- Urgency (low, medium, high)

4. **negative_response**: Generate empathetic resolution based on diagnosis

5. **End**: Return final response

7.7.3 6.3 Structured Output Schemas

Structured outputs ensure LLM responses follow a specific format using Pydantic models.

Sentiment Schema

```
from pydantic import BaseModel, Field
from typing import Literal

class SentimentSchema(BaseModel):
    """
    Schema for sentiment analysis output.

    The LLM must return exactly one of two values:
    - "positive": Review expresses satisfaction
    - "negative": Review expresses dissatisfaction
    """
    sentiment: Literal["positive", "negative"] = Field(
        description='Sentiment of the review'
    )
```

Diagnosis Schema

```
class DiagnosisSchema(BaseModel):
    """
    Schema for negative review diagnosis.

    Extracts three dimensions from negative reviews:
    1. issue_type: What category of problem
    2. tone: How the user is feeling
    3. urgency: How critical is this
    """
    issue_type: Literal["UX", "Performance", "Bug", "Support", "Other"] = Field(
        description='The category of issue mentioned in the review'
    )
    tone: Literal["angry", "frustrated", "disappointed", "calm"] = Field(
        description='The emotional tone expressed by the user'
    )
    urgency: Literal["low", "medium", "high"] = Field(
        description='How urgent or critical the issue appears to be'
    )
```

7.7.4 6.4 Complete Implementation

```

from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from typing import TypedDict, Literal
from dotenv import load_dotenv
from pydantic import BaseModel, Field

# Load environment variables (for OpenAI API key)
load_dotenv()

# =====
# STEP 1: Initialize Models
# =====

# Base model for regular text generation
model = ChatOpenAI(model='gpt-4o-mini')

# =====
# STEP 2: Define Schemas for Structured Output
# =====

class SentimentSchema(BaseModel):
    """Schema for sentiment classification output."""
    sentiment: Literal["positive", "negative"] = Field(
        description='Sentiment of the review'
    )

class DiagnosisSchema(BaseModel):
    """Schema for negative review diagnosis output."""
    issue_type: Literal["UX", "Performance", "Bug", "Support", "Other"] = Field(
        description='The category of issue mentioned in the review'
    )
    tone: Literal["angry", "frustrated", "disappointed", "calm"] = Field(
        description='The emotional tone expressed by the user'
    )
    urgency: Literal["low", "medium", "high"] = Field(
        description='How urgent or critical the issue appears to be'
    )

# Create structured output models
sentiment_model = model.with_structured_output(SentimentSchema)
diagnosis_model = model.with_structured_output(DiagnosisSchema)

# =====
# STEP 3: Define State Schema
# =====

class ReviewState(TypedDict):
    """
    State schema for Review Handling workflow.

    Attributes:
        review: The original customer review text
        sentiment: Classified sentiment (positive/negative)
        diagnosis: Dictionary with issue_type, tone, urgency (for negative reviews)
        response: Generated response message
    """
    review: str
    sentiment: Literal["positive", "negative"]
    diagnosis: dict
    response: str

# =====
# STEP 4: Define Node Functions
# =====

def find_sentiment(state: ReviewState) -> dict:
    """
    Node 1: Analyze review sentiment using LLM.

    Uses structured output to ensure the response is
    """

```

```

    either "positive" or "negative".
    """
    prompt = f'''Analyze the sentiment of the following customer review.

Review: {state["review"]}

Classify as either "positive" or "negative".'''

    result = sentiment_model.invoke(prompt)

    return {'sentiment': result.sentiment}

def check_sentiment(state: ReviewState) -> Literal["positive_response", "run_diagnosis"]:
    """
    Routing function: Determine next node based on sentiment.

    Returns:
        "positive_response" for positive sentiment
        "run_diagnosis" for negative sentiment
    """
    if state['sentiment'] == 'positive':
        return 'positive_response'
    else:
        return 'run_diagnosis'

def positive_response(state: ReviewState) -> dict:
    """
    Node 2a: Generate thank-you response for positive reviews.

    Creates a warm, appreciative message and asks for
    feedback on the website.
    """
    prompt = f'''Write a warm thank-you message in response to this positive review:

"{state['review']}"

Also, kindly ask the user to leave feedback on our website.
Keep the response professional yet friendly, around 3-4 sentences.'''

    response = model.invoke(prompt).content

    return {'response': response}

def run_diagnosis(state: ReviewState) -> dict:
    """
    Node 2b: Diagnose negative review using LLM.

    Extracts three key dimensions:
    - issue_type: Category of the problem
    - tone: Emotional state of the user
    - urgency: How critical the issue is
    """
    prompt = f'''Diagnose this negative customer review:

"{state['review']}"

Analyze and return:
1. issue_type: The category (UX, Performance, Bug, Support, or Other)
2. tone: User's emotional tone (angry, frustrated, disappointed, or calm)
3. urgency: How urgent this is (low, medium, or high)'''

    result = diagnosis_model.invoke(prompt)

    # Convert Pydantic model to dictionary for state storage
    return {'diagnosis': result.model_dump()}

def negative_response(state: ReviewState) -> dict:
    """
    Node 3: Generate empathetic response for negative reviews.

    Uses the diagnosis information to craft a personalized,
    empathetic resolution message.
    """

```

```

diagnosis = state['diagnosis']

prompt = f'''You are a customer support specialist.

A customer submitted this review: "{state['review']}"

Analysis:
- Issue Type: {diagnosis['issue_type']}
- Emotional Tone: {diagnosis['tone']}
- Urgency Level: {diagnosis['urgency']}

Write an empathetic, helpful resolution message that:
1. Acknowledges their frustration
2. Apologizes for the inconvenience
3. Offers concrete next steps
4. Matches the urgency level in your response

Keep it professional, sincere, and around 4-5 sentences.'''

response = model.invoke(prompt).content

return {'response': response}

# =====
# STEP 5: Build the Graph
# =====

# Initialize graph with state schema
graph = StateGraph(ReviewState)

# Add all nodes
graph.add_node('find_sentiment', find_sentiment)
graph.add_node('positive_response', positive_response)
graph.add_node('run_diagnosis', run_diagnosis)
graph.add_node('negative_response', negative_response)

# Add start edge
graph.add_edge(START, 'find_sentiment')

# Add CONDITIONAL edges from sentiment analysis
graph.add_conditional_edges('find_sentiment', check_sentiment)

# Add remaining edges
graph.add_edge('positive_response', END)
graph.add_edge('run_diagnosis', 'negative_response')
graph.add_edge('negative_response', END)

# Compile the workflow
workflow = graph.compile()

# =====
# STEP 6: Test the Workflow
# =====

# Test with positive review
positive_review = {
    'review': """I've been using this app for about a month, and I must say the
user interface is incredibly clean and intuitive. Everything is exactly
where you'd expect it to be. It's rare to find something that just works
without needing a tutorial. Great job to the design team!"""
}

print("=" * 60)
print("POSITIVE REVIEW TEST")
print("=" * 60)
result_positive = workflow.invoke(positive_review)
print(f"Sentiment: {result_positive['sentiment']}")
print(f"Response: {result_positive['response']}")

# Test with negative review
negative_review = {
    'review': """I've been trying to log in for over an hour now, and the app
keeps freezing on the authentication screen. I even tried reinstalling it,

```

```
    but no luck. This kind of bug is unacceptable, especially when it affects
    basic functionality."""
}

print("\n" + "=" * 60)
print("NEGATIVE REVIEW TEST")
print("=" * 60)
result_negative = workflow.invoke(negative_review)
print(f"Sentiment: {result_negative['sentiment']}")
print(f"Diagnosis: {result_negative['diagnosis']}")
print(f"Response: {result_negative['response']}")
```

7.7.5 6.5 Code Walkthrough

Creating Structured Output Models

```
# Regular model for free-form text generation
model = ChatOpenAI(model='gpt-4o-mini')

# Model that forces output to match SentimentSchema
sentiment_model = model.with_structured_output(SentimentSchema)

# Model that forces output to match DiagnosisSchema
diagnosis_model = model.with_structured_output(DiagnosisSchema)
```

The `with_structured_output()` method wraps the model to:

1. Include schema information in the prompt
2. Parse the response into a Pydantic model
3. Validate the response matches the schema

Accessing Structured Output

```
# Returns a SentimentSchema object
result = sentiment_model.invoke(prompt)

# Access the sentiment value
sentiment_value = result.sentiment # "positive" or "negative"
```

Converting Pydantic to Dictionary

```
# In run_diagnosis function
result = diagnosis_model.invoke(prompt)

# Convert to dict for storing in state
return {'diagnosis': result.model_dump()}
# model_dump() returns: {'issue_type': '...', 'tone': '...', 'urgency': '...'}
```

Conditional Edges Pattern

```
# The routing function returns a string matching a node name
def check_sentiment(state: ReviewState) -> Literal["positive_response", "run_diagnosis"]:
    if state['sentiment'] == 'positive':
        return 'positive_response' # Must match node name exactly
    else:
        return 'run_diagnosis' # Must match node name exactly
```

```
# Connect with conditional edges
graph.add_conditional_edges('find_sentiment', check_sentiment)
```

7.8 7. Key Components of Conditional Workflows

7.8.1 7.1 Component Summary Table

Table 7.3. 1 Component Summary Table

Component	Purpose	Example
State Schema	Define data structure flowing through workflow	TypedDict with all required fields
Node Functions	Perform specific tasks	find_sentiment(), calculate_discriminant()
Routing Function	Decide which branch to take	Returns node name as string
add_conditional_edges()	Create conditional branching	Takes source node + routing function
Literal Type	Constrain return values	Literal["option_a", "option_b"]

7.8.2 7.2 Routing Function Requirements

Your routing function **must**:

1. **Accept state as parameter:** `def route(state: MyState)`
2. **Return a string:** The exact name of the target node
3. **Use Literal return type:** `-> Literal["node1", "node2", ...]`
4. **Cover all cases:** Handle every possible condition

```
# CORRECT
def route(state) -> Literal["a", "b", "c"]:
    if condition1:
        return "a"
    elif condition2:
        return "b"
    else:
        return "c"

# INCORRECT - missing return type hint
def route(state):
    return "a" if condition else "b"

# INCORRECT - returns node reference instead of string
def route(state):
    return node_a # Should be "node_a"
```

7.9 8. Best Practices

7.9.1 8.1 Design Principles

1. **Keep routing functions simple:** Only check conditions, don't perform complex logic
2. **Use descriptive node names:** Names should indicate what the node does
3. **Handle all edge cases:** Ensure routing function covers every possibility
4. **Use structured outputs:** For LLM-based decisions, use Pydantic schemas

7.9.2 8.2 Common Patterns

Pattern 1: Binary Decision

```
def binary_route(state) -> Literal["path_a", "path_b"]:
    return "path_a" if state['condition'] else "path_b"
```

Pattern 2: Multi-way Decision

```
def multi_route(state) -> Literal["opt1", "opt2", "opt3", "default"]:
    mapping = {
        "value1": "opt1",
        "value2": "opt2",
        "value3": "opt3"
    }
    return mapping.get(state['key'], "default")
```

Pattern 3: Threshold-based Decision

```
def threshold_route(state) -> Literal["low", "medium", "high"]:
    score = state['score']
    if score < 30:
        return "low"
    elif score < 70:
        return "medium"
    else:
        return "high"
```

7.9.3 8.3 Debugging Tips

1. **Print state in routing function:** Temporarily add print statements
2. **Visualize the graph:** Use `workflow.get_graph().draw_mermaid()`
3. **Test each branch:** Create test cases that exercise all paths
4. **Check node name matching:** Ensure routing function returns exact node names

7.10 9. Summary

7.10.1 Key Takeaways

1. **Conditional workflows** enable decision-making in LangGraph, similar to if-else in programming
2. **The mechanism involves three parts:**
 - Node functions that perform tasks
 - A routing function that evaluates conditions
 - `add_conditional_edges()` to connect them
3. **The routing function:**
 - Takes current state as input
 - Returns the exact name of the next node (as a string)
 - Should use `Literal` type hints for return values
4. **Conditional vs Parallel:**
 - Parallel: All branches execute simultaneously
 - Conditional: Only ONE branch executes based on condition
5. **For LLM-based decisions:**
 - Use structured outputs with Pydantic schemas

- Ensure reliable, parseable responses from LLMs

7.10.2 Quick Reference

```
# 1. Define routing function
def my_router(state: MyState) -> Literal["option_a", "option_b"]:
    if state['condition']:
        return "option_a"
    else:
        return "option_b"

# 2. Create conditional edges
graph.add_conditional_edges('source_node', my_router)

# 3. Connect all branches to their destinations
graph.add_edge('option_a', 'next_node')
graph.add_edge('option_b', END)
```

7.10.3 What's Next?

After mastering conditional workflows, you can explore:

- **Dynamic Workflows:** Using the Command function for more flexible routing
- **Human-in-the-Loop:** Adding human intervention points
- **Subgraphs:** Nesting workflows within workflows
- **Memory and Persistence:** Maintaining state across sessions

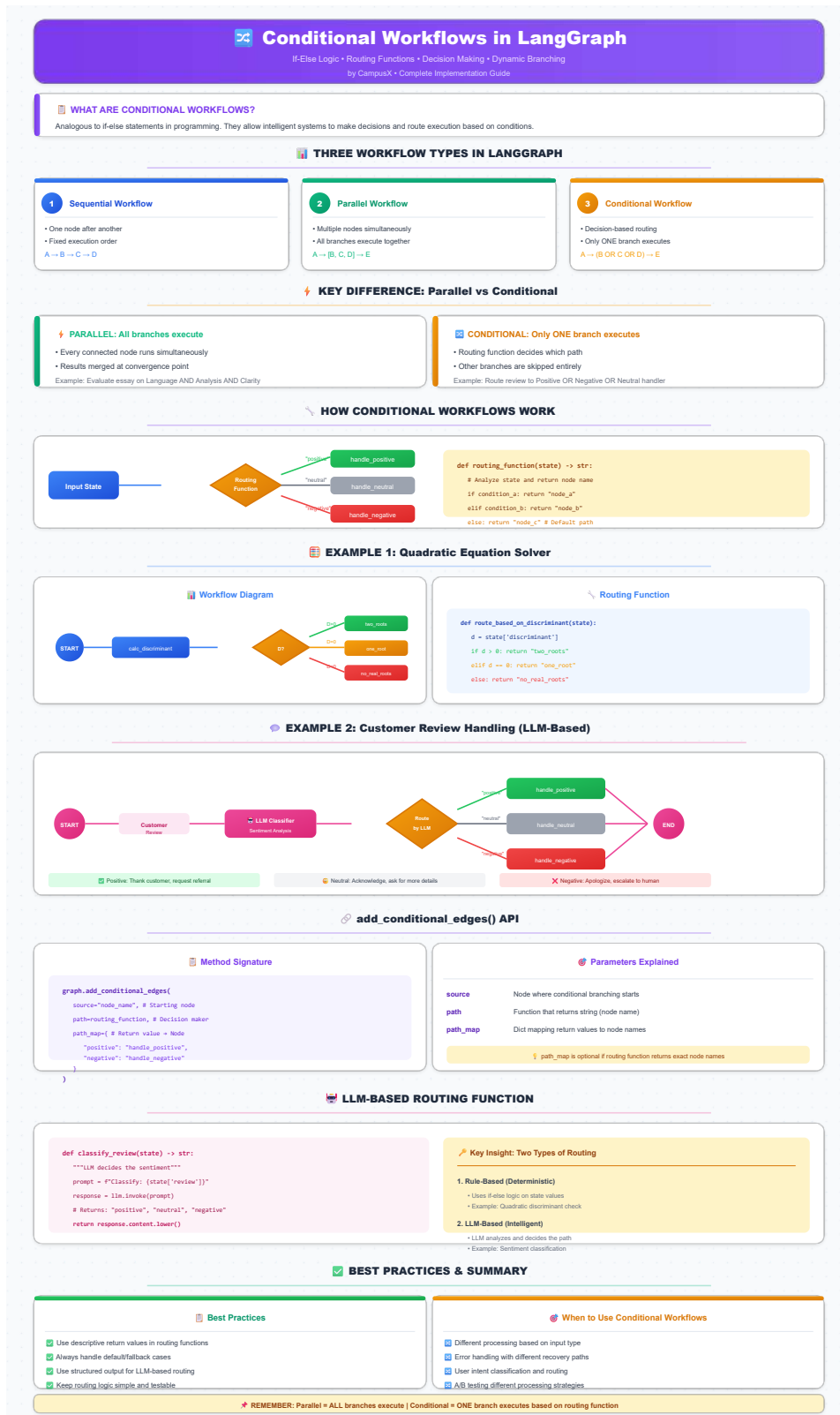


Figure 7.9. *What's Next?*

Iterative (Looping) Workflows in LangGraph

8.1 Code

- <https://github.com/campusx-official/langgraph-tutorials>

8.2 1. Introduction and Context

This document covers **Iterative (Looping) Workflows** in LangGraph, which represent the fourth fundamental type of workflow pattern used in building agentic AI applications. Iterative workflows are essential when you need to continuously improve an output until it meets certain quality criteria.

The core concept is simple: instead of producing an output in a single pass, you create a feedback loop where the output is evaluated, and if it doesn't meet the required standards, it's refined and re-evaluated until it either passes the quality check or reaches a maximum iteration limit.

8.3 2. Types of Workflows Recap

Before diving into iterative workflows, let's understand where they fit in the broader context of LangGraph workflow patterns.

8.3.1 2.1 Sequential Workflows

Sequential workflows execute tasks in a linear fashion—one after another. Task B starts only after Task A completes, Task C starts after Task B completes, and so on.

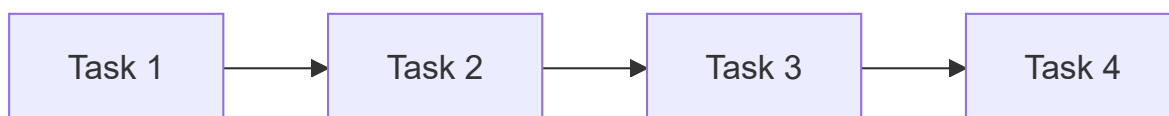


Figure 8.1. 1 Sequential Workflows

Characteristics:

- Linear execution flow
- Each task depends on the completion of the previous task
- Simple to understand and implement
- Predictable execution order

Use Cases:

- Data processing pipelines
- Multi-step document processing
- Sequential content generation (outline → draft → edit)

8.3.2 2.2 Parallel Workflows

Parallel workflows allow multiple tasks to execute simultaneously, improving efficiency when tasks are independent of each other.

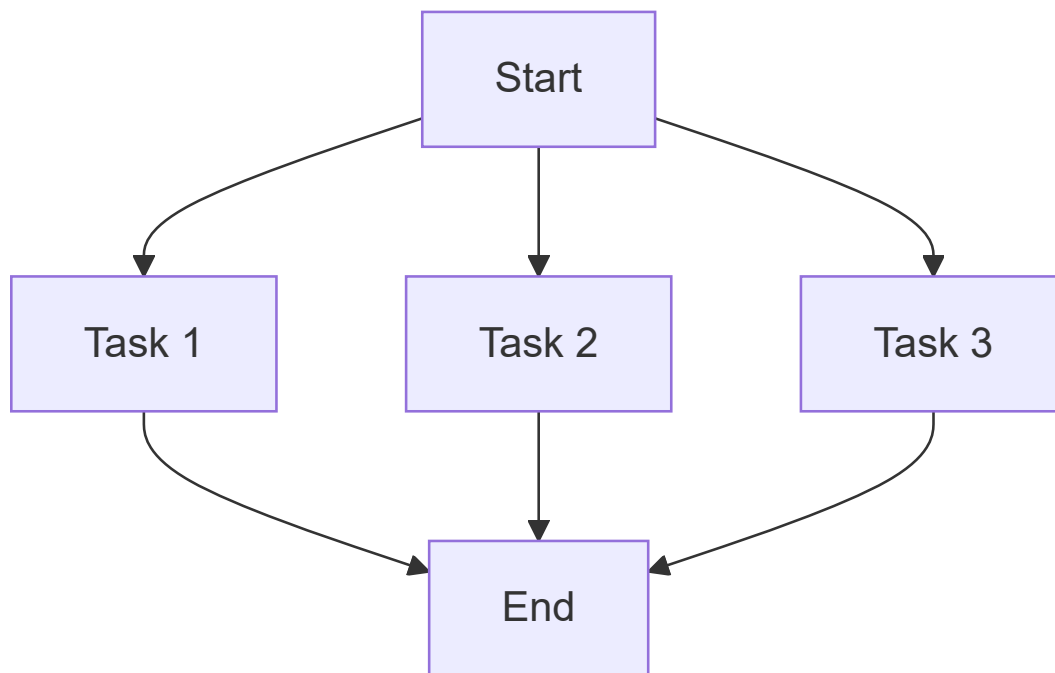


Figure 8.2. 2 Parallel Workflows

Characteristics:

- Multiple tasks execute concurrently
- Reduces total execution time
- Tasks must be independent
- Requires synchronization at the end

Use Cases:

- Fetching data from multiple APIs simultaneously
- Processing multiple documents in parallel
- Running multiple analysis tasks at once

8.3.3 2.3 Conditional Workflows

Conditional workflows allow branching based on some condition. The workflow chooses one path among multiple options depending on the evaluation result.

Characteristics:

- Dynamic path selection
- Only one branch executes based on condition
- Enables complex decision trees

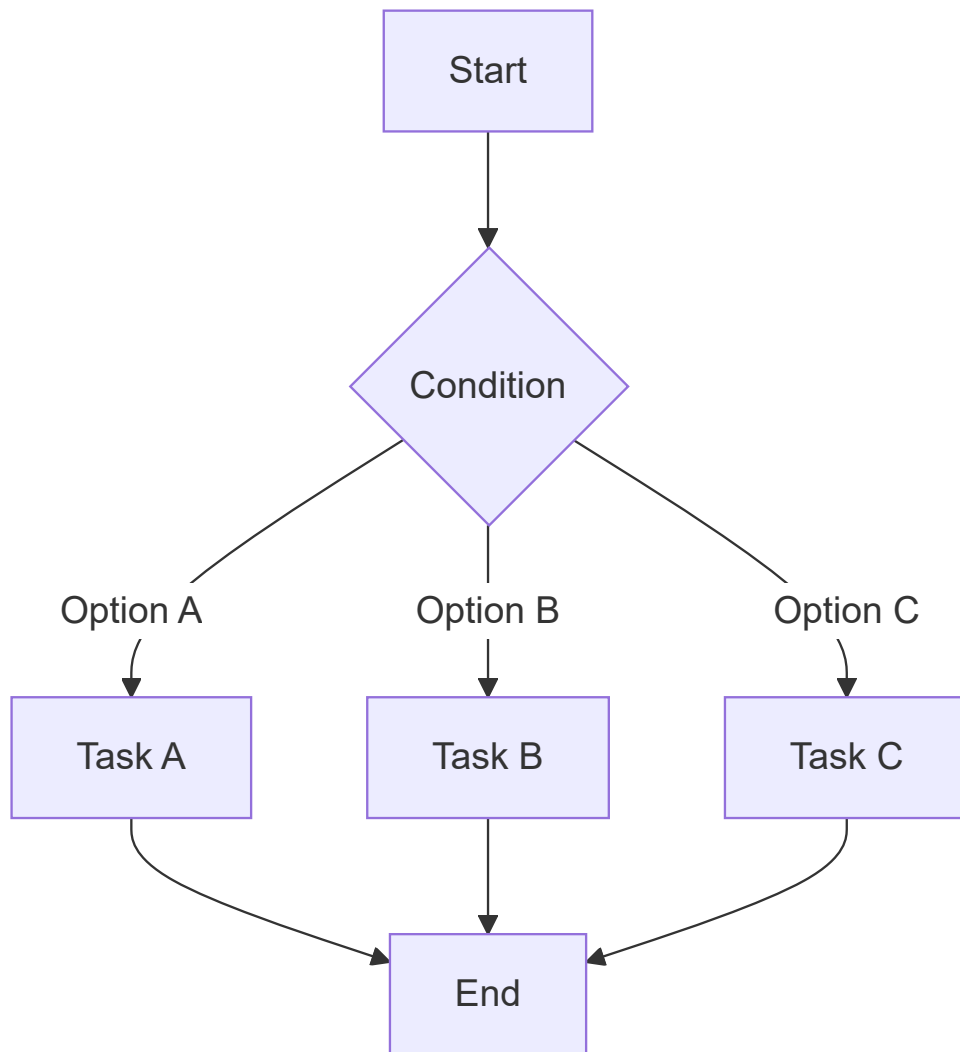


Figure 8.3. 3 *Conditional Workflows*

- Conditional edges determine routing

Use Cases:

- Customer support routing
- Content moderation decisions
- Task delegation based on input type

8.3.4 2.4 Iterative (Looping) Workflows

Iterative workflows create cycles where execution returns to a previous node based on some condition, enabling continuous improvement of outputs.

Characteristics:

- Creates feedback loops
- Enables continuous refinement
- Requires termination conditions to prevent infinite loops
- Combines conditional logic with repeated execution

Use Cases:

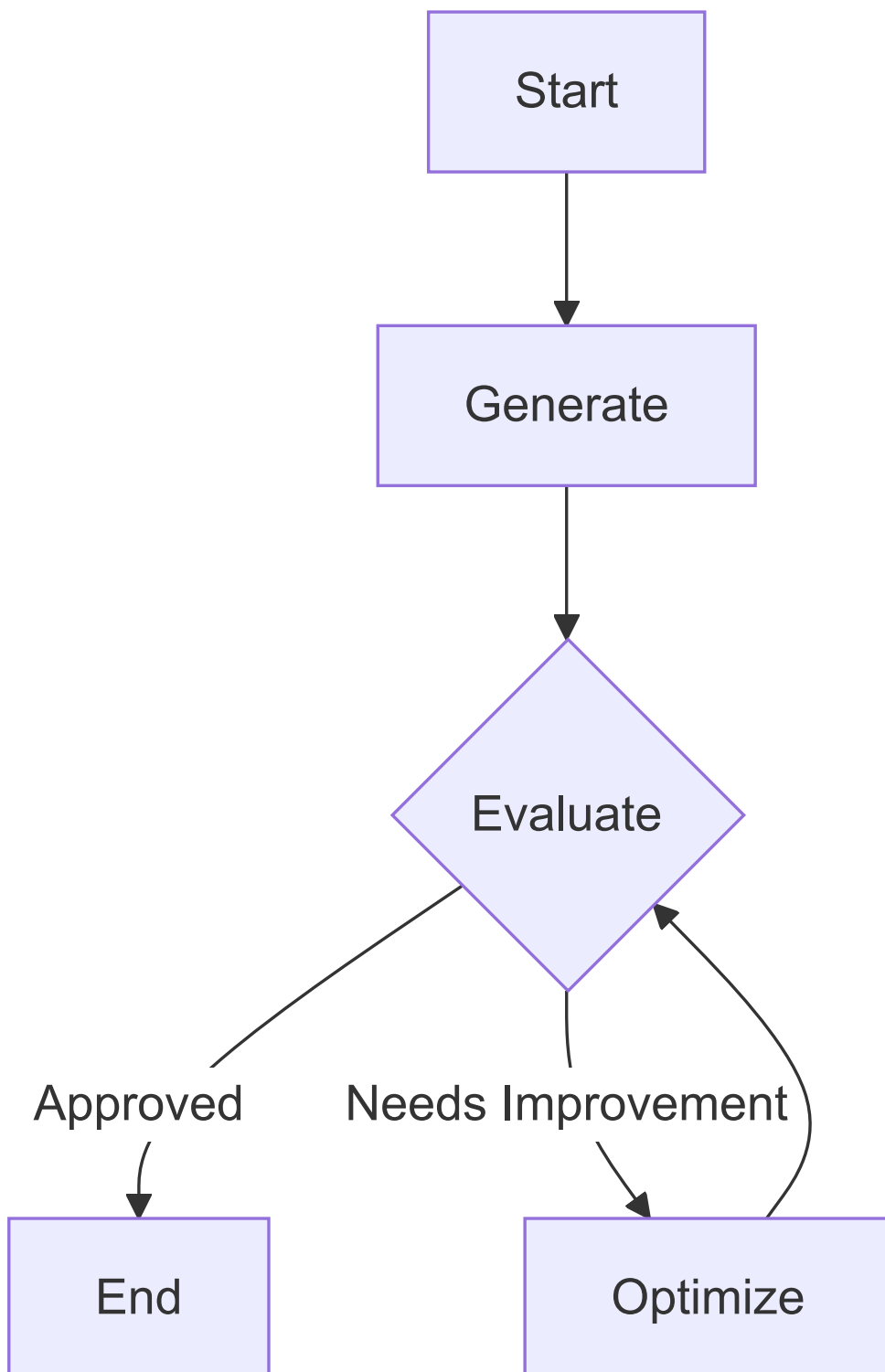


Figure 8.4. 4 Iterative (Looping) Workflows

- Content quality improvement
- Code refinement and optimization
- Iterative problem solving
- Self-correcting AI systems

8.3.5 Workflow Comparison Table

Table 8.1. *Workflow Comparison Table*

Aspect	Sequential	Parallel	Conditional	Iterative
Execution Pattern	One after another	Simultaneously	One of many	Repeated cycles
Task Dependencies	High (linear)	Low (independent)	Medium (branching)	Medium (cyclic)
Complexity	Low	Medium	Medium	High
Use Case	Pipelines	Concurrent operations	Decision trees	Quality improvement
Termination	End of chain	All tasks complete	Branch ends	Condition met or max iterations
Edge Types	Normal edges	Normal edges	Conditional edges	Conditional + Normal edges

8.4 3. Understanding Iterative Workflows

8.4.1 3.1 What Makes a Workflow Iterative?

An iterative workflow is characterized by:

1. **A Loop Structure:** Execution can return to a previous node
2. **Evaluation Criteria:** Clear standards for determining if the output is acceptable
3. **Refinement Mechanism:** A process to improve outputs that don't meet criteria
4. **Termination Condition:** Rules to exit the loop (success or max iterations)

8.4.2 3.2 The Improvement Cycle

The fundamental principle of iterative workflows is the **Generate** → **Evaluate** → **Improve** cycle:

8.4.3 3.3 Why Use Iterative Workflows?

Problem: Most LLM outputs in the first generation are not satisfactory. They may be:

- Too generic
- Missing key requirements
- Not aligned with specific guidelines
- Lacking creativity or impact

Solution: Instead of accepting the first output, iterative workflows allow the system to:

1. **Self-evaluate** against defined criteria
2. **Identify weaknesses** through feedback generation
3. **Refine the output** based on specific feedback
4. **Repeat** until quality standards are met

This mimics how humans work—we rarely get things perfect on the first try. We iterate, refine, and improve.

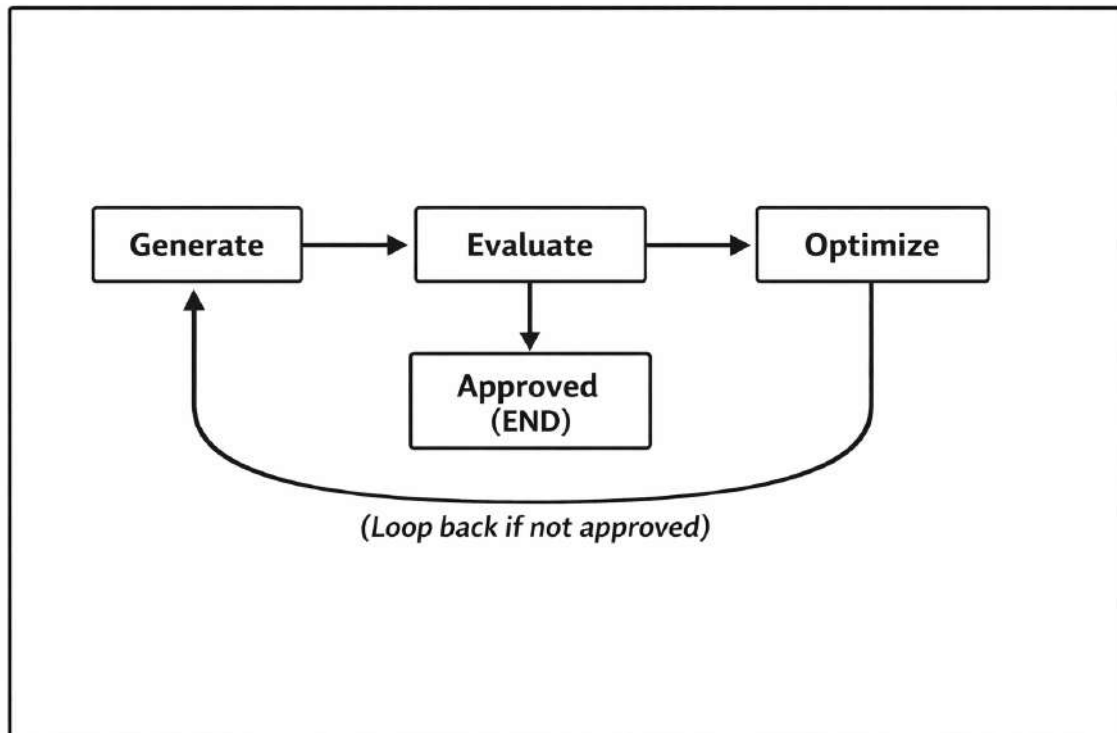


Figure 8.5. 2 The Improvement Cycle

8.5 4. Real-World Use Case: Social Media Post Generator

8.5.1 4.1 The Problem Statement

Consider a YouTuber who wants to maintain an active presence across multiple social platforms (Twitter/X, LinkedIn, Instagram) but lacks time to create quality content for each platform. The naive solution would be to:

1. Provide a topic to an LLM
2. Ask it to generate a post
3. Publish the post directly

However, this approach has significant problems:

- First-generation outputs are often mediocre
- Posts may be repetitive or generic
- No quality assurance mechanism
- May not create real value for the audience

8.5.2 4.2 The Solution: Iterative Quality Improvement

Instead of accepting whatever the LLM generates, we implement a quality control system:

8.5.3 4.3 The Three-Component Architecture

Our workflow consists of three specialized LLMs, each with a specific role:

Table 8.2. 3 *The Three-Component Architecture*

Component	Role	Description
Generator	Content Creation	Creates the initial post based on the given topic
Evaluator	Quality Assessment	Judges the post against strict criteria and provides feedback
Optimizer	Content Refinement	Improves the post based on evaluator feedback

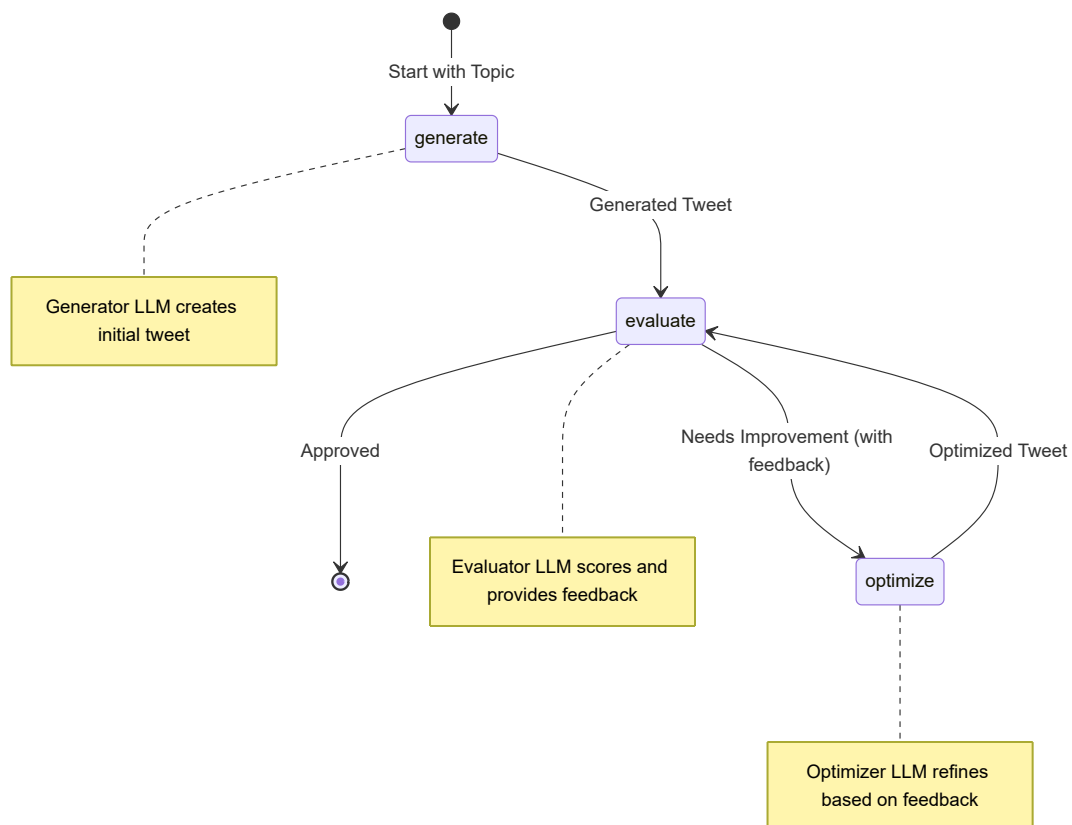
Key Insight: In production systems, these three LLMs should ideally be different models optimized for their specific tasks:

- **Generator:** A model with strong creative writing capabilities (e.g., GPT-4.5, Claude)
- **Evaluator:** A model that follows instructions precisely (e.g., GPT-4o)
- **Optimizer:** A model good at iterative refinement (e.g., GPT-4)

8.6 5. Workflow Architecture Deep Dive

8.6.1 5.1 Visual Representation

The complete workflow can be visualized as follows:

**Figure 8.6. 1** *Visual Representation*

8.6.2 5.2 Detailed Flow Explanation

Step 1: Generation

- Input: Topic (e.g., “Indian Railways”)
- Process: Generator LLM creates a tweet following specific guidelines
- Output: Initial tweet

Step 2: Evaluation

- Input: Generated tweet
- Process: Evaluator LLM judges against strict criteria
- Output:
- Evaluation result: “approved” OR “needs_improvement”
- Feedback: Detailed strengths and weaknesses

Step 3: Routing Decision

- If “approved” OR max_iterations reached → End workflow
- If “needs_improvement” → Go to Optimization

Step 4: Optimization

- Input: Current tweet + Feedback
- Process: Optimizer LLM improves the tweet based on feedback
- Output: Improved tweet
- Action: Increment iteration counter, return to Evaluation

8.6.3 5.3 The Loop Mechanism

The loop is created by having an edge from **Optimize** back to **Evaluate**:

Important: The key to creating loops in LangGraph is simply manipulating edges. You create:

1. A **conditional edge** from Evaluate to either End or Optimize
2. A **normal edge** from Optimize back to Evaluate

8.7 6. Implementation Guide

8.7.1 6.1 Required Imports

```
# Core LangGraph imports
from langgraph.graph import StateGraph, START, END

# Type hints
from typing import TypedDict, Literal, Annotated

# LLM integration
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage

# For state management with lists
import operator

# For structured output
from pydantic import BaseModel, Field
```

8.7.2 6.2 LLM Configuration

```
# Initialize three separate LLM instances
generator_llm = ChatOpenAI(model='gpt-4o-mini') # For content generation
evaluator_llm = ChatOpenAI(model='gpt-4o-mini') # For evaluation
optimizer_llm = ChatOpenAI(model='gpt-4o-mini') # For optimization
```


Production Recommendation: Use different models based on their strengths:

Table 8.3. 2 LLM Configuration

Role	Recommended Models	Reason
Generator	GPT-4.5, Claude 3, Gemini Pro	Strong creative writing
Evaluator	GPT-4o, Claude 3	Precise instruction following
Optimizer	GPT-4, Claude 3	Good at refinement tasks

8.8 7. State Design with TypedDict

8.8.1 7.1 Understanding State in LangGraph

State is the central data structure that flows through your graph. It contains all information needed by nodes to perform their tasks and stores all intermediate results.

8.8.2 7.2 The TweetState Class

```
class TweetState(TypedDict):
    # Input
    topic: str  # Topic for the tweet

    # Current tweet
    tweet: str  # Current version of the tweet

    # Evaluation results
    evaluation: Literal["approved", "needs_improvement"]  # Evaluation decision
    feedback: str  # Detailed feedback

    # Iteration tracking
    iteration: int  # Current iteration number
    max_iteration: int  # Maximum allowed iterations

    # History tracking (with reducer)
    tweet_history: Annotated[list[str], operator.add]  # All generated tweets
    feedback_history: Annotated[list[str], operator.add]  # All feedback messages
```

8.8.3 7.3 State Variables Explained

Table 8.4. 3 State Variables Explained

Variable	Type	Purpose
topic	str	The input topic provided by user
tweet	str	Current version of the generated tweet
evaluation	Literal[...]	Binary result: approved or needs improvement
feedback	str	Detailed feedback from evaluator
iteration	int	Tracks how many optimization cycles have occurred
max_iteration	int	Prevents infinite loops by setting a limit
tweet_history	list[str]	Stores all versions of the tweet
feedback_history	list[str]	Stores all feedback messages

8.8.4 7.4 The Reducer Pattern for Lists

When you want to **accumulate** values instead of **replacing** them, you use a reducer:

```
# Without reducer - new value replaces old value
tweet: str # If you set tweet="A" then tweet="B", result is "B"

# With reducer - new value is appended to list
tweet_history: Annotated[List[str], operator.add]
# If you add ["A"] then add ["B"], result is ["A", "B"]
```

How it works:

- Annotated tells LangGraph this field has special behavior
- operator.add is the reducer function that merges lists
- Each update appends to the existing list instead of replacing it

8.9 8. Building the Three Core Nodes

8.9.1 8.1 Node 1: Generate Tweet

The generator creates the initial tweet based on the provided topic.

```
def generate_tweet(state: TweetState):
    """
    Generate an initial tweet on the given topic.

    This function creates a prompt with specific guidelines for
    generating humorous, viral-worthy tweets and sends it to the
    generator LLM.

    Args:
        state: TweetState containing the topic

    Returns:
        Dictionary with 'tweet' and 'tweet_history' keys
    """

    # Create the prompt with System and Human messages
    messages = [
        SystemMessage(content="You are a funny and clever Twitter/X influencer."),
        HumanMessage(content=f"""
Write a short, original, and hilarious tweet on the topic: "{state['topic']}".

Rules:
- Do NOT use question-answer format.
- Max 280 characters.
- Use observational humor, irony, sarcasm, or cultural references.
- Think in meme logic, punchlines, or relatable takes.
- Use simple, day to day english
""")
    ]

    # Generate the tweet
    response = generator_llm.invoke(messages).content

    # Return both the current tweet and add to history
    return {
        'tweet': response,
        'tweet_history': [response] # List format for reducer
    }
```

Key Points:

- Uses SystemMessage to set the persona
- HumanMessage contains detailed instructions and rules
- Returns a dictionary matching state variables
- tweet_history is a list because of the reducer pattern

8.9.2 8.2 Node 2: Evaluate Tweet

The evaluator judges the tweet against strict criteria.

```
def evaluate_tweet(state: TweetState):
    """
    Evaluate the generated tweet against quality criteria.

    Uses a "ruthless critic" persona to ensure only high-quality
    tweets get approved. Returns both an evaluation decision and
    detailed feedback.

    Args:
        state: TweetState containing the current tweet

    Returns:
        Dictionary with 'evaluation', 'feedback', and 'feedback_history'
    """

    messages = [
        SystemMessage(content="You are a ruthless, no-laugh-given Twitter critic.
        You evaluate tweets based on humor, originality, virality, and tweet format."),
        HumanMessage(content=f"
        Evaluate the following tweet:

        Tweet: \"{state['tweet']}\"

        Use the criteria below to evaluate the tweet:

        1. Originality – Is this fresh, or have you seen it a hundred times before?
        2. Humor – Did it genuinely make you smile, laugh, or chuckle?
        3. Punchiness – Is it short, sharp, and scroll-stopping?
        4. Virality Potential – Would people retweet or share it?
        5. Format – Is it a well-formed tweet (not Q&A, under 280 characters)?

        Auto-reject if:
        - It's written in question-answer format
        - It exceeds 280 characters
        - It reads like a traditional setup-punchline joke
        - It ends with generic, throwaway lines that weaken the humor

        ### Respond ONLY in structured format:
        - evaluation: \"approved\" or \"needs_improvement\"
        - feedback: One paragraph explaining the strengths and weaknesses
        """)
    ]

    # Use structured output for consistent response format
    response = structured_evaluator_llm.invoke(messages)

    return {
        'evaluation': response.evaluation,
        'feedback': response.feedback,
        'feedback_history': [response.feedback]
    }
```

The Evaluation Criteria Breakdown:

Table 8.5. 2 Node 2: Evaluate Tweet

Criterion	Question Asked	Why It Matters
Originality	Is this fresh?	Repeated content doesn't engage
Humor	Did it make you laugh?	Core goal of funny tweets
Punchiness	Is it scroll-stopping?	Social media requires attention-grabbing content
Virality	Would people share it?	Measures potential reach
Format	Is it properly formatted?	Technical requirements

Auto-Reject Conditions:

- Question-answer format (lazy humor)
- Over 280 characters (Twitter limit)
- Traditional jokes (not native to Twitter)
- Weak endings (dilutes impact)

8.9.3 8.3 Node 3: Optimize Tweet

The optimizer improves the tweet based on feedback.

```
def optimize_tweet(state: TweetState):
    """
    Improve the tweet based on evaluator feedback.

    Takes the current tweet and specific feedback, then generates
    an improved version that addresses the identified weaknesses.

    Args:
        state: TweetState containing tweet and feedback

    Returns:
        Dictionary with 'tweet', 'iteration', and 'tweet_history'
    """

    messages = [
        SystemMessage(content="You punch up tweets for virality and humor based on given feedback."),
        HumanMessage(content=f"""
Improve the tweet based on this feedback:
"{state['feedback']}"

Topic: "{state['topic']}"
Original Tweet:
{state['tweet']}

Re-write it as a short, viral-worthy tweet. Avoid Q&A style and stay under 280 characters.
""")
    ]

    response = optimizer_llm.invoke(messages).content

    # Increment iteration counter
    iteration = state['iteration'] + 1

    return {
        'tweet': response,
        'iteration': iteration,
        'tweet_history': [response]
    }
```

Key Points:

- Receives both the current tweet AND the feedback
- Uses feedback to guide specific improvements
- Increments the iteration counter
- Adds new version to tweet_history

8.10 9. Structured Output with Pydantic

8.10.1 9.1 Why Structured Output?

When the evaluator returns a response, we need exactly two things:

1. An evaluation decision (approved/needs_improvement)
2. Detailed feedback

Without structured output, we'd need to:

- Parse free-form text
- Hope the LLM follows our format
- Handle parsing errors

With structured output:

- Response format is guaranteed
- Values are type-checked
- Easy to access as object attributes

8.10.2 9.2 The TweetEvaluation Schema

```
from pydantic import BaseModel, Field
from typing import Literal

class TweetEvaluation(BaseModel):
    """
    Schema for the evaluation response.

    This Pydantic model ensures the evaluator LLM always returns
    responses in a consistent, structured format.
    """

    evaluation: Literal["approved", "needs_improvement"] = Field(
        ..., # Required field
        description="Final evaluation result."
    )

    feedback: str = Field(
        ..., # Required field
        description="Feedback for the tweet."
    )
```

Schema Breakdown:

Table 8.6. 2 *The TweetEvaluation Schema*

Field	Type	Constraint	Description
evaluation	Literal	Only “approved” or “needs_improvement”	Binary decision
feedback	str	Required	Detailed explanation

8.10.3 9.3 Creating the Structured LLM

```
# Create a version of the evaluator that returns structured output
structured_evaluator_llm = evaluator_llm.with_structured_output(TweetEvaluation)
```

How It Works:

```
Standard LLM Call:
Input: Prompt
Output: "I think this tweet is good because..."
↓
(Need to parse this text manually)

Structured LLM Call:
Input: Prompt + Schema
Output: TweetEvaluation(evaluation="approved", feedback="...")
↓
(Direct attribute access: response.evaluation, response.feedback)
```

8.11 10. Creating the Loop with Conditional Edges

8.11.1 10.1 The Routing Function

The routing function determines which path to take based on the evaluation result:

```
def route_evaluation(state: TweetState):  
    """  
    Determine the next step based on evaluation result.  
  
    Returns 'approved' to end the workflow, or 'needs_improvement'  
    to continue the optimization loop.  
  
    Args:  
        state: TweetState with evaluation and iteration info  
  
    Returns:  
        String indicating next path: 'approved' or 'needs_improvement'  
    """  
  
    # Exit conditions:  
    # 1. Tweet was approved  
    # 2. Maximum iterations reached (prevent infinite loops)  
    if state['evaluation'] == 'approved' or state['iteration'] >= state['max_iteration']:  
        return 'approved'  
    else:  
        return 'needs_improvement'
```

8.11.2 10.2 Preventing Infinite Loops

The `max_iteration` parameter is crucial for preventing infinite loops. Consider this scenario:

Without `max_iteration`:

```
Iteration 1: Tweet rejected → Optimize  
Iteration 2: Tweet rejected → Optimize  
Iteration 3: Tweet rejected → Optimize  
... (continues forever if evaluator never approves)
```

With `max_iteration = 5`:

```
Iteration 1: Tweet rejected → Optimize  
Iteration 2: Tweet rejected → Optimize  
Iteration 3: Tweet rejected → Optimize  
Iteration 4: Tweet rejected → Optimize  
Iteration 5: max_iteration reached → END (regardless of evaluation)
```

Common Reasons for Infinite Loops:

1. Evaluation criteria too strict
2. Generator/Optimizer LLM not capable enough
3. Conflicting evaluation rules
4. Bug in routing logic

8.11.3 10.3 Building the Graph with Edges

```
# Create the graph  
graph = StateGraph(TweetState)  
  
# Add nodes  
graph.add_node('generate', generate_tweet)  
graph.add_node('evaluate', evaluate_tweet)  
graph.add_node('optimize', optimize_tweet)
```

```

# Add edges
graph.add_edge(START, 'generate')          # Start → Generate
graph.add_edge('generate', 'evaluate')      # Generate → Evaluate

# Conditional edge: Evaluate → End OR Optimize
graph.add_conditional_edges(
    'evaluate',                               # Source node
    route_evaluation,                         # Routing function
    {
        'approved': END,                    # If approved → End
        'needs_improvement': 'optimize'     # If needs improvement → Optimize
    }
)

# Normal edge: Optimize → Evaluate (creates the loop)
graph.add_edge('optimize', 'evaluate')

# Compile
workflow = graph.compile()

```

Edge Structure Visualization:

EDGE	TYPE	SOURCE	TARGET	CONDITION
				
Normal Edge		START	generate	Always
Normal Edge		generate	evaluate	Always
Conditional Edge		evaluate	END	If 'approved'
Conditional Edge		evaluate	optimize	If 'needs_improvement'
Normal Edge		optimize	evaluate	Always (creates loop)

8.12 11. Complete Code Implementation

Here is the complete, runnable code for the iterative tweet generator:

```

# =====
# ITERATIVE TWEET GENERATOR WORKFLOW
# =====
# This workflow generates, evaluates, and optimizes tweets until they meet
# quality criteria or reach the maximum iteration limit.
# =====

# -----
# IMPORTS
# -----
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Literal, Annotated
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage
import operator
from pydantic import BaseModel, Field

# -----
# LLM CONFIGURATION
# -----
# In production, use different models optimized for each task
generator_llm = ChatOpenAI(model='gpt-4o-mini')
evaluator_llm = ChatOpenAI(model='gpt-4o-mini')
optimizer_llm = ChatOpenAI(model='gpt-4o-mini')

# -----
# STRUCTURED OUTPUT SCHEMA
# -----
class TweetEvaluation(BaseModel):
    """Schema for evaluation results ensuring consistent output format."""
    evaluation: Literal["approved", "needs_improvement"] = Field(
        ...,
        description="Final evaluation result."
    )

```

```

feedback: str = Field(
    ...,
    description="Feedback for the tweet."
)

# Create structured evaluator
structured_evaluator_llm = evaluator_llm.with_structured_output(TweetEvaluation)

# -----
# STATE DEFINITION
# -----
class TweetState(TypedDict):
    """State object that flows through the graph."""
    topic: str
    tweet: str
    evaluation: Literal["approved", "needs_improvement"]
    feedback: str
    iteration: int
    max_iteration: int
    tweet_history: Annotated[list[str], operator.add]
    feedback_history: Annotated[list[str], operator.add]

# -----
# NODE FUNCTIONS
# -----
def generate_tweet(state: TweetState):
    """Generate the initial tweet on the given topic."""
    messages = [
        SystemMessage(content="You are a funny and clever Twitter/X influencer."),
        HumanMessage(content=f"""
Write a short, original, and hilarious tweet on the topic: "{state['topic']}".

Rules:
- Do NOT use question-answer format.
- Max 280 characters.
- Use observational humor, irony, sarcasm, or cultural references.
- Think in meme logic, punchlines, or relatable takes.
- Use simple, day to day english
""")
    ]

    response = generator_llm.invoke(messages).content
    return {'tweet': response, 'tweet_history': [response]}

def evaluate_tweet(state: TweetState):
    """Evaluate the tweet against quality criteria."""
    messages = [
        SystemMessage(content="""You are a ruthless, no-laugh-given Twitter critic.
You evaluate tweets based on humor, originality, virality, and tweet format."""),
        HumanMessage(content=f"""
Evaluate the following tweet:

Tweet: "{state['tweet']}"

Use the criteria below to evaluate the tweet:

1. Originality – Is this fresh, or have you seen it a hundred times before?
2. Humor – Did it genuinely make you smile, laugh, chuckle?
3. Punchiness – Is it short, sharp, and scroll-stopping?
4. Virality Potential – Would people retweet or share it?
5. Format – Is it a well-formed tweet (not Q&A, under 280 characters)?

Auto-reject if:
- It's written in question-answer format
- It exceeds 280 characters
- It reads like a traditional setup-punchline joke
- It ends with generic, throwaway lines that weaken the humor

### Respond ONLY in structured format:
- evaluation: "approved" or "needs_improvement"
- feedback: One paragraph explaining the strengths and weaknesses
""")
    ]

```



```

    ]

    response = structured_evaluator_llm.invoke(messages)
    return {
        'evaluation': response.evaluation,
        'feedback': response.feedback,
        'feedback_history': [response.feedback]
    }

def optimize_tweet(state: TweetState):
    """Improve the tweet based on feedback."""
    messages = [
        SystemMessage(content="You punch up tweets for virality and humor based on given feedback."),
        HumanMessage(content=f"""
Improve the tweet based on this feedback:
"{state['feedback']}"

Topic: "{state['topic']}"
Original Tweet:
{state['tweet']}

Re-write it as a short, viral-worthy tweet. Avoid Q&A style and stay under 280 characters.
""")
    ]

    response = optimizer_llm.invoke(messages).content
    iteration = state['iteration'] + 1
    return {'tweet': response, 'iteration': iteration, 'tweet_history': [response]}

def route_evaluation(state: TweetState):
    """Route based on evaluation result and iteration count."""
    if state['evaluation'] == 'approved' or state['iteration'] >= state['max_iteration']:
        return 'approved'
    else:
        return 'needs_improvement'

# -----
# GRAPH CONSTRUCTION
# -----
graph = StateGraph(TweetState)

# Add nodes
graph.add_node('generate', generate_tweet)
graph.add_node('evaluate', evaluate_tweet)
graph.add_node('optimize', optimize_tweet)

# Add edges
graph.add_edge(START, 'generate')
graph.add_edge('generate', 'evaluate')
graph.add_conditional_edges(
    'evaluate',
    route_evaluation,
    {'approved': END, 'needs_improvement': 'optimize'}
)
graph.add_edge('optimize', 'evaluate') # The loop!

# Compile
workflow = graph.compile()

# -----
# EXECUTION
# -----
if __name__ == "__main__":
    # Define initial state
    initial_state = {
        "topic": "Indian Railways",
        "iteration": 1,
        "max_iteration": 5
    }

    # Run the workflow

```

```
result = workflow.invoke(initial_state)

# Display results
print("=" * 60)
print("FINAL RESULT")
print("=" * 60)
print(f"Topic: {result['topic']}")
print(f"Final Tweet: {result['tweet']}")
print(f"Evaluation: {result['evaluation']}")
print(f"Total Iterations: {result['iteration']}")
print()
print("Tweet History:")
for i, tweet in enumerate(result['tweet_history'], 1):
    print(f"    Version {i}: {tweet}")
```

8.13 12. Testing and Execution

8.13.1 12.1 Running the Workflow

```
# Define the initial state
initial_state = {
    "topic": "Indian Railways", # Topic for the tweet
    "iteration": 1,             # Starting iteration
    "max_iteration": 5         # Maximum iterations allowed
}

# Invoke the workflow
result = workflow.invoke(initial_state)
```

8.13.2 12.2 Understanding the Result

The result dictionary contains all state variables:

```
{
    'topic': 'Indian Railways',

    'tweet': 'Boarding an Indian railway train is like going to Hogwarts...',

    'evaluation': 'approved',

    'feedback': 'This tweet scores well on originality...',

    'iteration': 1, # or higher if multiple iterations occurred

    'max_iteration': 5,

    'tweet_history': [
        'First version of the tweet...',
        'Second version (if optimization occurred)...',
        # ... more versions
    ],

    'feedback_history': [
        'Feedback for first version...',
        'Feedback for second version...',
        # ... more feedback
    ]
}
```

8.13.3 12.3 Analyzing Multiple Iterations

When the tweet goes through multiple optimization cycles:

```
# Print all tweet versions
print("Tweet Evolution:")
for i, tweet in enumerate(result['tweet_history'], 1):
    print(f"\nVersion {i}:")
    print(f"    {tweet}")

# Print all feedback
print("\nFeedback History:")
for i, feedback in enumerate(result['feedback_history'], 1):
    print(f"\nRound {i}:")
    print(f"    {feedback}")
```

8.13.4 12.4 Example Output

```
Topic: Indian Railways
Final Iterations: 3

Tweet History:
Version 1: "Indian Railways: where the journey is the destination...
          and also the workout, the meditation, and the test of patience."

Version 2: "Indian Railways taught me more about human patience than
          any self-help book ever could."

Version 3: "Boarding an Indian railway train is like going to Hogwarts.
          You have no clue when you'll get there, you'll make unexpected
          friends, and there's a slight slim chance you'll spot wild
          creatures like the ticket checker."

Final Evaluation: approved

Feedback History:
Round 1: "The tweet lacks punch and feels like a generic observation..."

Round 2: "Better concept but needs more specific humor..."

Round 3: "Excellent use of cultural reference, relatable humor, and
          good length. Approved!"
```

8.14 13. Key Takeaways and Best Practices

8.14.1 13.1 Core Concepts Summary

Table 8.7. 1 Core Concepts Summary

Concept	Implementation	Purpose
Loop Creation	Edge from Optimize → Evaluate	Enables iteration
Conditional Routing	<code>add_conditional_edges()</code>	Dynamic path selection
State Accumulation	<code>Annotated[list, operator.add]</code>	Track history
Structured Output	<code>Pydantic + with_structured_output()</code>	Consistent responses
Termination	<code>max_iteration</code> check	Prevent infinite loops

8.14.2 13.2 Best Practices

1. Always Set a Maximum Iteration Limit

```
# Prevents infinite loops
if state['iteration'] >= state['max_iteration']:
    return 'approved' # Force exit
```

2. Use Specialized LLMs for Each Role

```
# Production configuration
generator_llm = ChatOpenAI(model='gpt-4.5')    # Creative
evaluator_llm = ChatOpenAI(model='gpt-4o')    # Precise
optimizer_llm = ChatOpenAI(model='gpt-4')     # Balanced
```

3. Detailed Evaluation Criteria

- Be specific about what constitutes a good output
- Include auto-reject conditions
- Provide clear scoring guidelines

4. Track History for Debugging

```
tweet_history: Annotated[List[str], operator.add]
feedback_history: Annotated[List[str], operator.add]
```

5. Use Structured Output for Consistency

```
class EvaluationSchema(BaseModel):
    evaluation: Literal["approved", "needs_improvement"]
    feedback: str
```

8.14.3 13.3 When to Use Iterative Workflows

Good Use Cases:

- Content quality improvement
- Code generation and refinement
- Document drafting and editing
- Any task where “good enough” isn’t acceptable

Not Ideal For:

- Simple, deterministic tasks
- Real-time applications requiring instant responses
- Tasks with clear one-shot solutions

8.14.4 13.4 Future Enhancements

The workflow demonstrated here can be extended with:

1. **Human-in-the-Loop:** Add a human approval step before publishing
2. **Multi-Platform Support:** Generate content for different platforms (LinkedIn, Instagram)
3. **Tool Integration:** Connect to Twitter API for direct posting
4. **Memory:** Remember previously successful tweet styles
5. **A/B Testing:** Generate multiple versions and test performance

8.14.5 13.5 The Big Picture

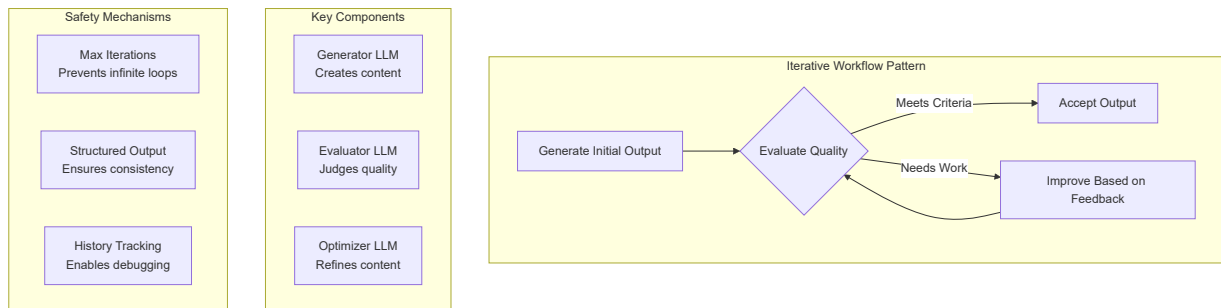


Figure 8.7. 5 The Big Picture

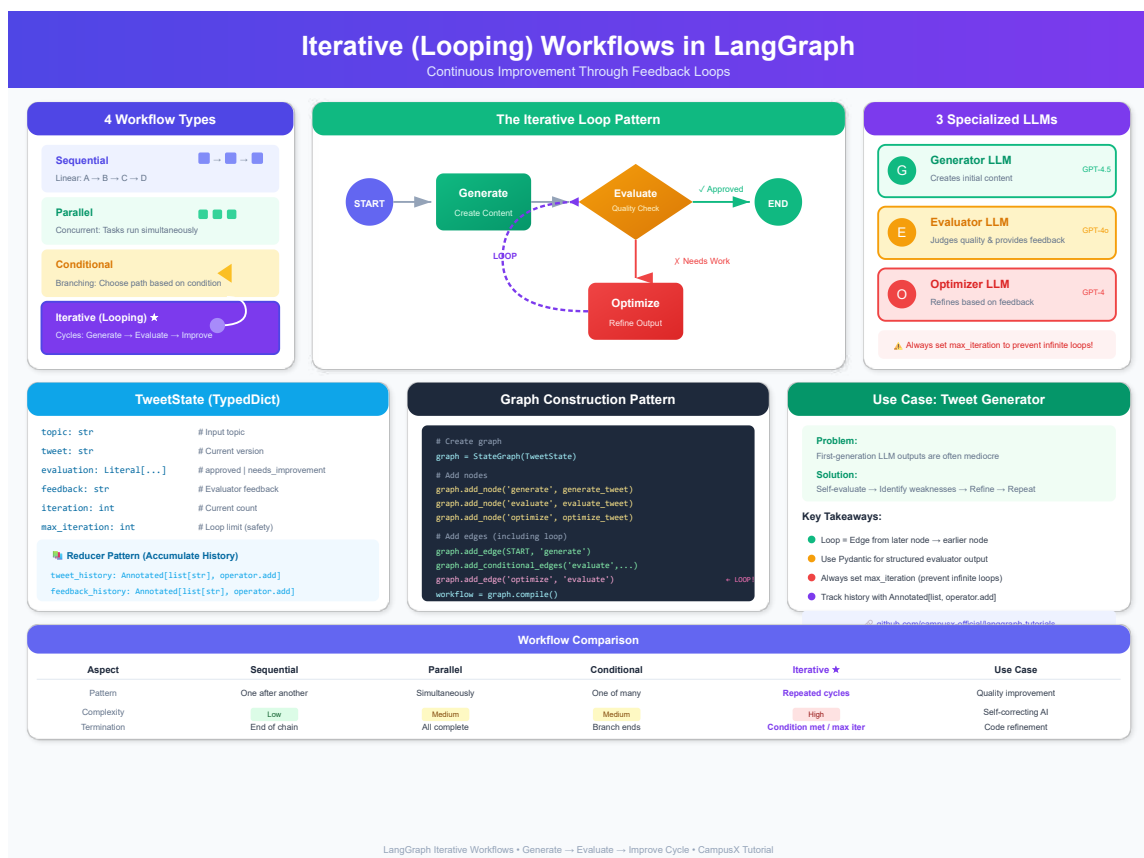


Figure 8.8. 5 The Big Picture

How to build a Chatbot using LangGraph

9.1 Overview

This lecture is part of an Agentic AI using LangGraph playlist, focusing on building a comprehensive chatbot with multiple features including normal chatting, RAG, tools, UI, and LangSmith integration.

- <https://github.com/campusx-official/langgraph-tutorials>

9.2 Today's Objective

Build a simple chatbot that can:

- Chat with users
- Remember previous conversation history
- Maintain context across multiple interactions

9.3 Chatbot Architecture

9.3.1 Basic Design

- **Type:** Sequential workflow with a single node
- **Flow:** User Message → Chat Node (LLM) → Response → End
- **Core Component:** One chat node containing an LLM

9.3.2 State Definition

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated
from langchain_core.messages import BaseMessage, HumanMessage
from langchain_openai import ChatOpenAI
from langgraph.graph.message import add_messages

class ChatState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]
```

Key Points:

- State contains all exchanged messages between user and LLM
- Uses BaseMessage instead of strings for flexibility
- Supports multiple message types: HumanMessage, AIMessage, SystemMessage, ToolMessage
- add_messages reducer function prevents message replacement

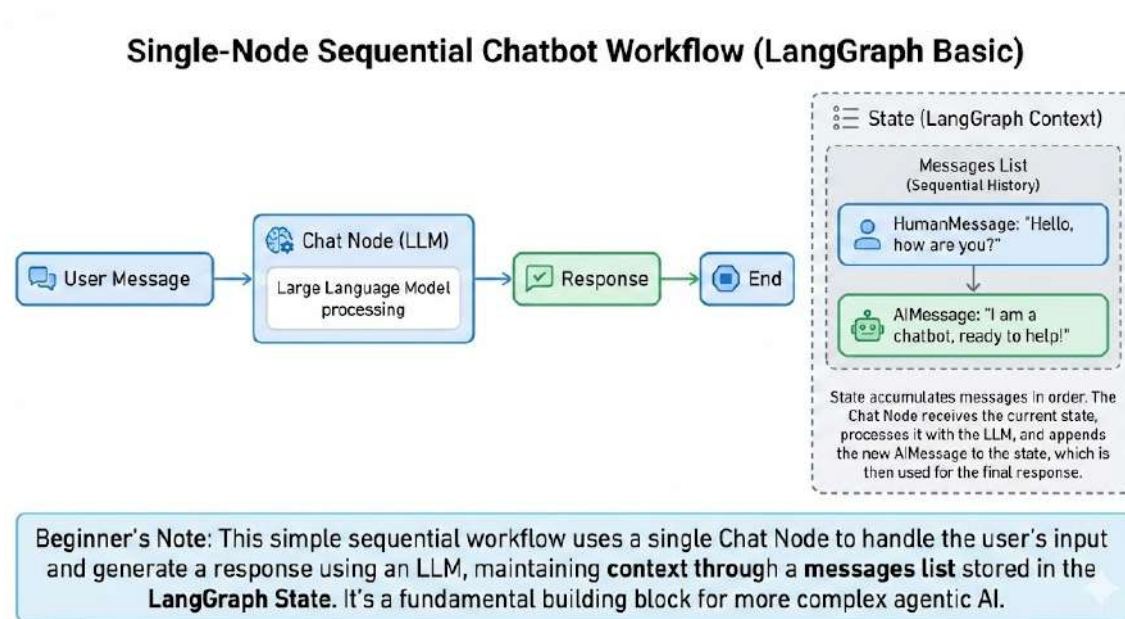


Figure 9.1. Basic Design

9.3.3 Why Use `add_messages` Reducer?

- Default state behavior: new values replace previous ones
- `add_messages`: specialized reducer for appending messages
- More optimized than `operator.add` for message handling
- Maintains complete conversation history

9.4 Implementation

9.4.1 1. Chat Node Function

```
llm = ChatOpenAI()

def chat_node(state: ChatState):
    # Extract messages from state
    messages = state['messages']

    # Send to LLM
    response = llm.invoke(messages)

    # Return response to be added to state
    return {'messages': [response]}
```

9.4.2 2. Graph Construction

```
graph = StateGraph(ChatState)

# Add nodes
graph.add_node('chat_node', chat_node)

# Add edges
graph.add_edge(START, 'chat_node')
graph.add_edge('chat_node', END)

# Compile
chatbot = graph.compile()
```


9.4.3 3. Basic Usage

```
initial_state = {
    'messages': [HumanMessage(content='What is the capital of India?')]
}

result = chatbot.invoke(initial_state)
answer = result['messages'][-1].content # "New Delhi"
```

9.5 The Memory Problem

9.5.1 Issue Identified

When implementing a chat loop, the chatbot loses memory between interactions:

```
while True:
    user_input = input("Type here: ")
    if user_input.strip().lower() in ['exit', 'quit', 'bye']:
        break

    response = chatbot.invoke({
        'messages': [HumanMessage(content=user_input)]
    })

    print("AI:", response['messages'][-1].content)
```

Problem: Each `invoke()` call starts with a fresh state, losing previous conversation context.

9.5.2 Why Memory Loss Occurs

- Each loop iteration calls `invoke()` separately
- Every `invoke()` starts with a clean state
- Previous messages are not carried forward
- No persistence between workflow executions

9.6 Solution: Persistence

9.6.1 Concept Introduction

Persistence = Saving state between workflow executions

- **Without Persistence:** State erased after reaching END
- **With Persistence:** State saved and retrieved for future executions

9.6.2 Storage Options

1. **Database Storage:** Permanent, survives program restarts
2. **Memory Storage (RAM):** Temporary, lost when program stops

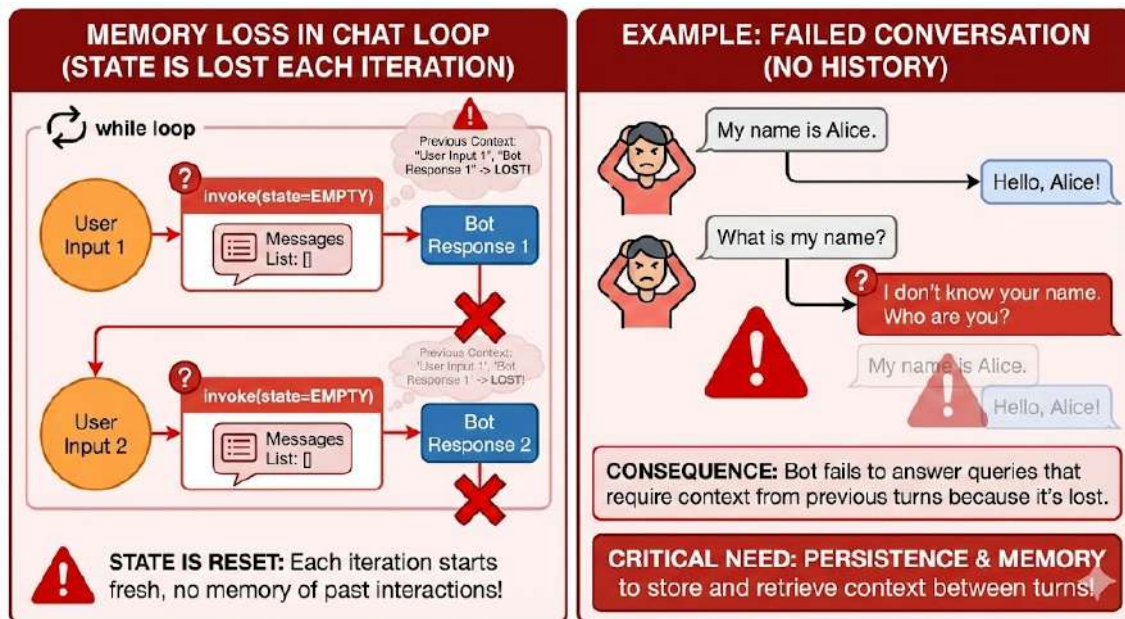


Figure 9.2. Storage Options

9.6.3 Implementation with Memory Saver

```
from langgraph.checkpoint.memory import MemorySaver

# Create checkpoint
checkpointer = MemorySaver()

# Compile with checkpointer
chatbot = graph.compile(checkpointer=checkpointer)
```

9.6.4 Thread Management

```
thread_id = "user_123"

config = {
    "configurable": {
        "thread_id": thread_id
    }
}

# Use config in invoke
response = chatbot.invoke({
    'messages': [HumanMessage(content=user_message)]
}, config=config)
```

9.6.5 Complete Working Example

```
thread_id = "user_123"
config = {"configurable": {"thread_id": thread_id}}

while True:
    user_message = input("Type here: ")
    if user_message.strip().lower() in ['exit', 'quit', 'bye']:
        break

    print("User:", user_message)
```

```
response = chatbot.invoke({
    'messages': [HumanMessage(content=user_message)]
}, config=config)

print("AI:", response['messages'][-1].content)
```

9.7 Behind the Scenes: How Persistence Works

1. First Interaction:

- User: "Hi, my name is Nitesh"
- State stores: [HumanMessage, AIMessage]
- Workflow ends → State saved to RAM

2. Second Interaction:

- User: "What is my name?"
- LangGraph retrieves previous state from RAM
- Appends new message to existing conversation
- LLM receives full conversation history
- Can answer: "Your name is Nitesh"

3. Subsequent Interactions:

- Each new message appended to growing conversation
- Full context maintained throughout session

9.8 Key Concepts Introduced

9.8.1 Threads

- **Thread** = One user's interaction session with chatbot
- Multiple users can have separate threads simultaneously
- Each thread maintains independent conversation history

9.8.2 Checkpointers

- Mechanism for saving/loading state
- MemorySaver: RAM-based storage
- Database checkpointers available for production

9.8.3 Configuration

- Required to specify thread_id for state management
- Links each interaction to correct conversation thread

9.9 State Inspection

```
# View current state
current_state = chatbot.get_state(config)
print(current_state) # Shows all messages in conversation
```

9.10 Production Considerations

9.10.1 Memory vs Database Storage

- **Development:** MemorySaver (RAM) - simple setup
- **Production:** Database checkpoints - persistent across restarts
- **Use Case:** WhatsApp bot needs to remember conversations across days/weeks

9.11 Next Steps Teased

Future videos will cover:

- Detailed persistence concepts
- Checkpointers deep dive
- Threads management
- Human-in-the-loop implementation
- Retry logic and fault tolerance
- Memory optimization techniques

9.12 Summary

- Built basic chatbot with LangGraph
- Identified and solved memory persistence problem
- Introduced checkpointers and thread management
- Demonstrated working chatbot with conversation memory
- Set foundation for advanced features in upcoming videos

LangGraph Chatbot Development

Building Memory-Enabled Conversational AI

OVERVIEW

Objective:

- Build chatbot with conversation memory
- Maintain context across interactions
- Use LangGraph persistence features

Architecture:

Sequential workflow with single chat node
User → Chat Node (LLM) → Response

STATE DEFINITION

```
class ChatState(TypedDict):
    messages: Annotated[
        list[BaseMessage],
        add_messages
    ]
    
```

- `add_messages` prevents replacement
- Maintains conversation history

THE MEMORY PROBLEM

Issues:

- Each `invoke()` starts fresh state
- Previous messages lost
- No conversation continuity

Example:

"My name is Nitesh" → "What's my name?"
Result: "I don't have access to your info"

PERSISTENCE SOLUTION

Features:

- State saved between executions
- Conversation history maintained
- Context-aware responses

Implementation:

MemorySaver + Thread Management

IMPLEMENTATION CODE

```
# Setup persistence
checkpointer = MemorySaver()
chatbot = graph.compile(checkpointer=checkpointer)

# Use with thread management
config = {"configurable": {"thread_id": "user_123"}}
```

KEY CONCEPTS

Threads:

- One user's conversation session
- Independent conversation histories

Checkpointers:

- Save/load state mechanism
- MemorySaver (RAM) vs Database

Configuration:

- Links interactions to threads

STORAGE OPTIONS

- Memory (RAM)**
 - Fast access
 - Lost on restart
 - Development use
- Database**
 - Persistent storage
 - Production ready

NEXT STEPS

- Detailed persistence concepts
- Advanced checkpointer features
- Human-in-the-loop implementation
- Retry logic and fault tolerance
- RAG integration
- Tool integration
- UI development
- LangSmith integration

ACHIEVEMENT UNLOCKED

Built a memory-enabled chatbot using LangGraph persistence
Foundation ready for advanced conversational AI features

Part 1 of LangGraph Chatbot Series
github.com/langchain-ai/langgraph

Figure 9.3. Summary

LangGraph Persistence: Complete Comprehensive Guide

10.1 Code

- https://github.com/campusx-official/langgraph-tutorials/blob/main/10_persistence.ipynb

10.2 1. Introduction

Persistence is one of the most **foundational and important concepts** in LangGraph. It serves as a building block upon which many other advanced features are built. Understanding persistence is crucial because:

- It enables **fault tolerance** in your workflows
- It powers **short-term memory** in chatbots
- It makes **Human-in-the-Loop (HITL)** workflows possible
- It enables **time travel** for debugging and replay

This guide provides a complete understanding of persistence from theory to practical implementation.

10.3 2. What is Persistence?

10.3.1 2.1 Definition

Persistence in LangGraph refers to the ability to save and restore the state of a workflow over time.

In simple terms, persistence allows you to:

1. **Save** the current state of your workflow at various points during execution
2. **Restore** that state later, even after the program has stopped
3. **Resume** execution from where it left off

10.3.2 2.2 Core Principles of LangGraph

Before understanding persistence, we need to recall two fundamental principles of LangGraph:

Principle 1: Concept of Graph

LangGraph allows you to decompose any high-level goal into a set of tasks and represent them as a **graph**:

- **Nodes:** Each node represents a task in your goal
- **Edges:** Edges represent the execution order (which task executes after which)

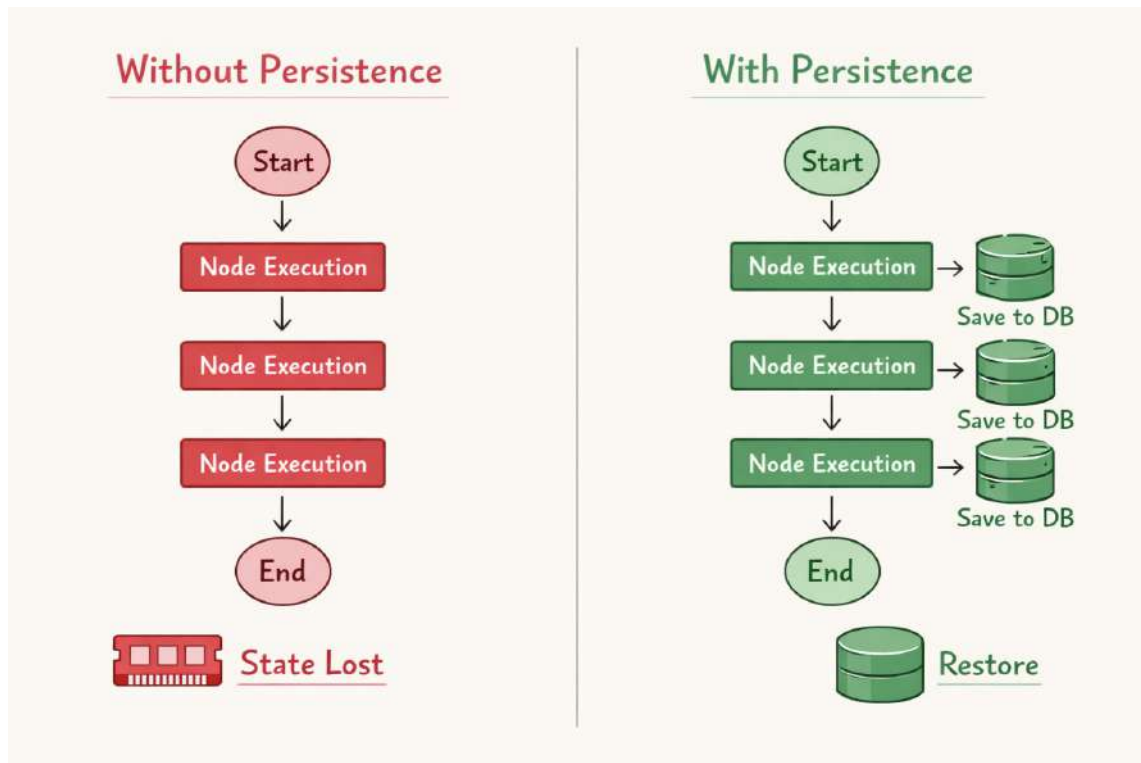


Figure 10.1. 1 Definition

Example: For a chatbot workflow, nodes might include:

- Receive user message
- Process with LLM
- Generate response
- Send response

Principle 2: Concept of State

Every LangGraph workflow requires a **State** definition:

- **State** is essentially a **dictionary** that stores important data needed during workflow execution
- Every node in your graph can **read from** and **write to** the state
- State holds information that needs to be shared across nodes

```
# Example State Definition
class ChatState(TypedDict):
    messages: list[str] # Important data: conversation history
    user_name: str # Important data: user information
    context: str # Important data: conversation context
```

State Access Pattern:

Table 10.1. Principle 2: Concept of State

Operation	Description
Read	Any node can access values stored in state
Write	Any node can modify values stored in state
Share	State acts as shared memory across all nodes

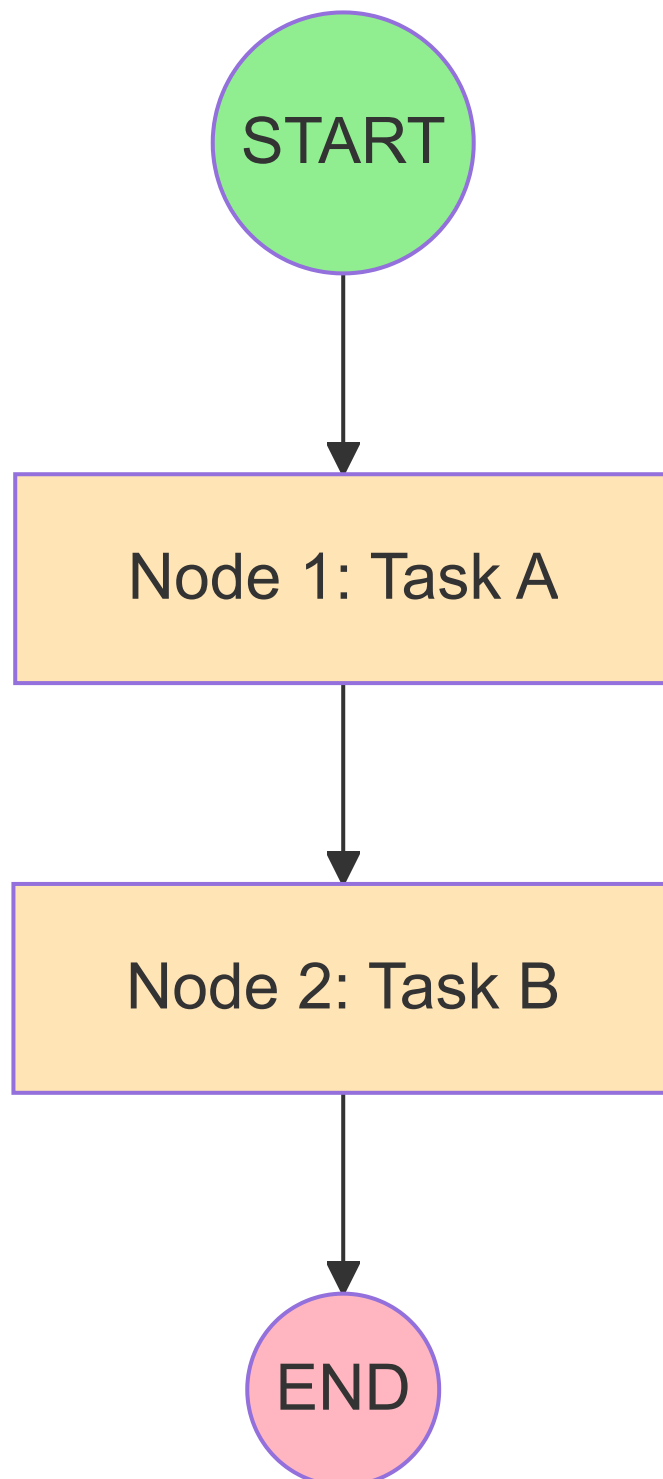


Figure 10.2. *Principle 1: Concept of Graph*

10.3.3 2.3 Default Behavior Without Persistence

By default, LangGraph exhibits the following behavior:

The Problem:

1. When you trigger a workflow with `invoke()`, execution begins
2. State changes as nodes execute
3. Once execution completes, **all state values are erased from RAM**
4. You **cannot access** those values in the future
5. If you need those values later, they are **unrecoverable**

This is where Persistence comes in - it changes this core behavior by allowing you to save state values so they can be recovered and used in the future.

10.4 3. How Persistence Works

10.4.1 3.1 Speciality: Intermediate State Storage

The most important characteristic of persistence is:

Persistence doesn't just store the final state values - it stores ALL intermediate state values at every checkpoint.

This is a crucial distinction that enables many powerful features.

Example: State Tracking Through Execution

Consider a workflow with a state variable `name`:

What Gets Stored:

Table 10.2. Example: State Tracking Through Execution

Checkpoint	Location	State Value	Stored
CP1	Before Node 1	name: 'A'	✓ Yes
CP2	After Node 1	name: 'B'	✓ Yes
CP3	After Node 2	name: 'C'	✓ Yes
CP4	At END	name: 'C'	✓ Yes

Without persistence: Only final value accessible, then lost **With persistence:** ALL values stored and retrievable

10.4.2 3.2 Crash Recovery Example

One powerful application of storing intermediate states is **crash recovery**:

Why this works:

- Since we save state at every checkpoint
- We know exactly where the crash occurred
- We know the state values at that point
- We can resume execution from the crash point, not from the beginning

This feature is called Fault Tolerance - one of the four major benefits of persistence.

10.5 4. Checkpointers: The Implementation Mechanism

10.5.1 4.1 What is a Checkpointer?

A **Checkpointer** is the component that implements persistence in LangGraph. It:

1. **Divides** your graph execution into checkpoints
2. **Saves** state values at each checkpoint
3. **Stores** them in some form of storage (memory, database, etc.)

10.5.2 4.2 Understanding Super Steps

How are checkpoints determined?

Checkpoints are created at each **Super Step** of your graph execution.

What is a Super Step?

A **Super Step** is a unit of parallel execution in the graph:

- When nodes execute **sequentially**, each node is one super step
- When nodes execute **in parallel**, all parallel nodes together form **one super step**

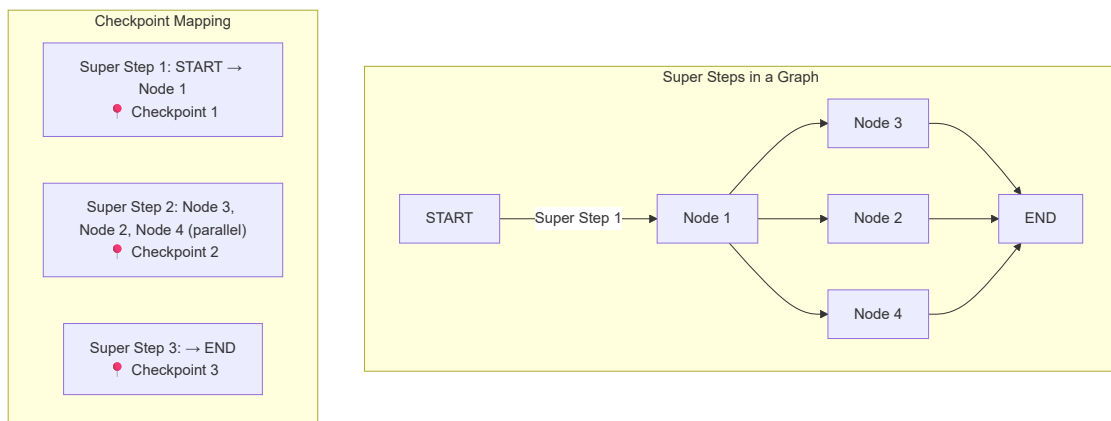


Figure 10.3. What is a Super Step?

Checkpoint Creation Rules:

Table 10.3. What is a Super Step?

Scenario	Super Steps	Checkpoints
3 sequential nodes	3 super steps	3 checkpoints
1 node → 3 parallel nodes → 1 node	3 super steps	3 checkpoints
All 5 nodes in parallel	1 super step	1 checkpoint

10.5.3 4.3 Types of Checkpointers

LangGraph provides different checkpointer implementations for different use cases:

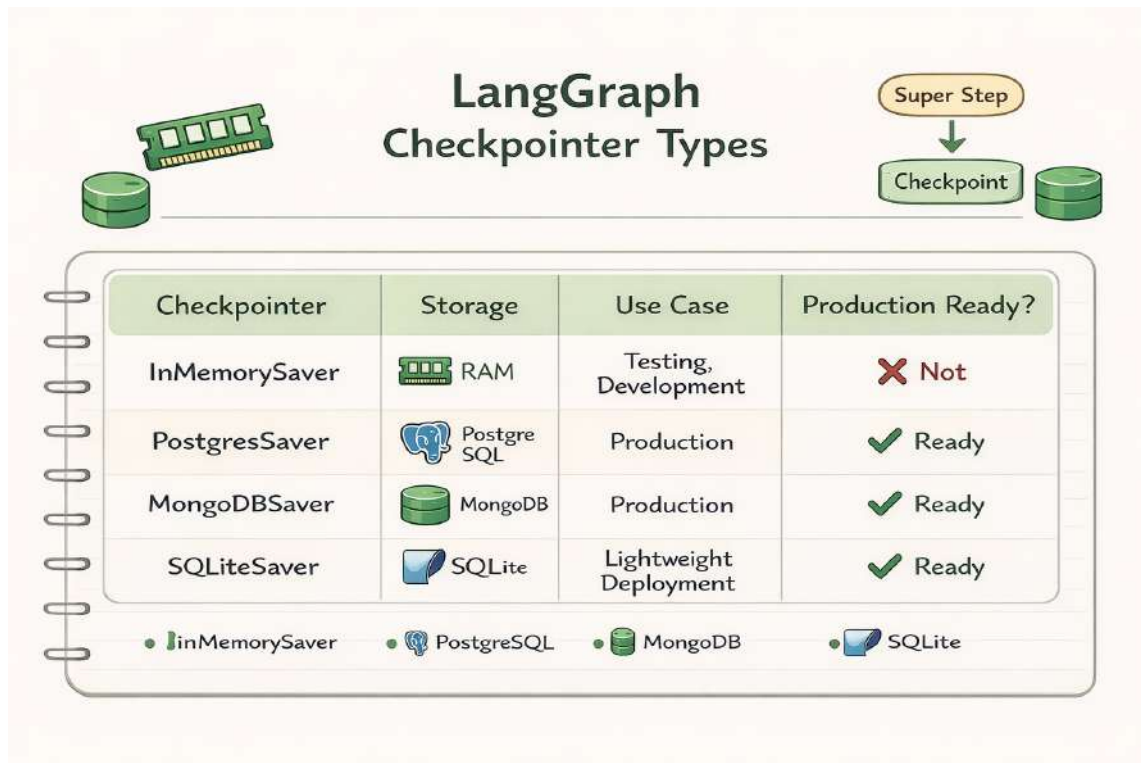


Figure 10.4. 3 Types of Checkpointers

Table 10.4. 3 Types of Checkpointers

Checkpointer	Storage	Use Case	Production Ready
InMemorySaver	RAM	Demos, Testing, Learning	✗ No
PostgresSaver	PostgreSQL Database	Production applications	✓ Yes
RedisSaver	Redis	Production applications	✓ Yes
SqliteSaver	SQLite Database	Small-scale production	✓ Yes

Important: InMemorySaver stores data in RAM, so:

- Data is lost when program stops
- Suitable only for learning and demos
- **Never use in production**

```
# Import statements for different checkpointers
from langgraph.checkpoint.memory import InMemorySaver      # For demos
from langgraph.checkpoint.postgres import PostgresSaver    # For production
from langgraph.checkpoint.redis import RedisSaver          # For production
```

10.5.4 4.4 Detailed Checkpoint Example

Let's trace through a complete example to understand how checkpoints work:

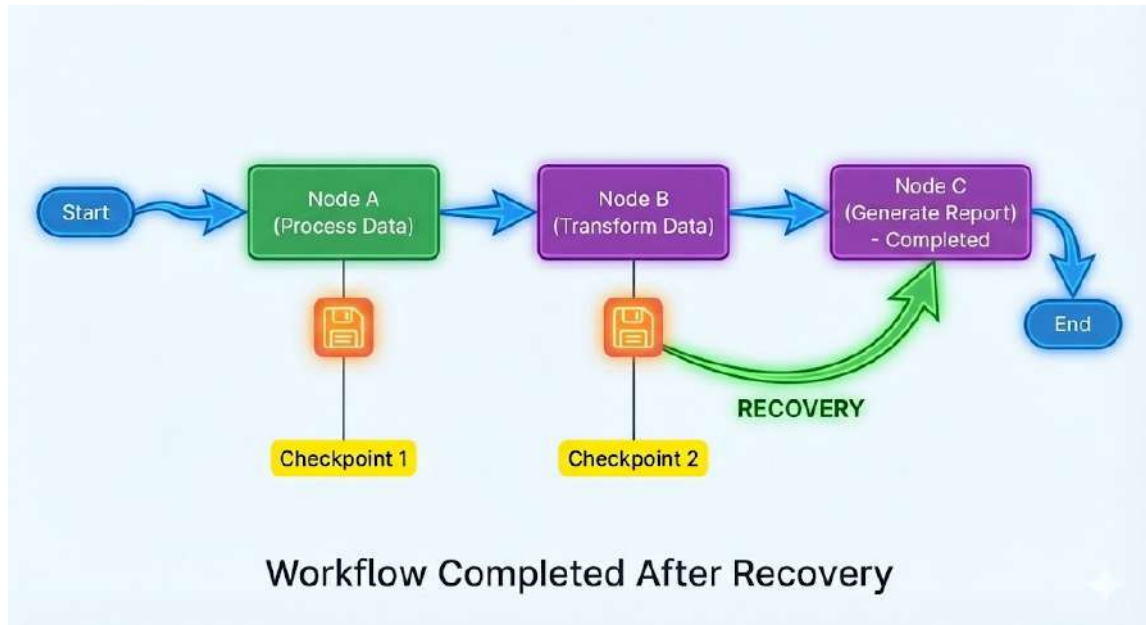
Setup:

- State has one attribute: numbers (list of integers)
- Reducer function: new values are **merged** (appended) to the list

```
# State Definition
class GraphState(TypedDict):
    numbers: Annotated[list[int], operator.add] # Reducer: merge lists
```

Checkpoint-by-Checkpoint State Values:**Table 10.5.** 4 Detailed Checkpoint Example

Checkpoint	Location	numbers Value	Action
CP1	At START	[1]	Initial input saved
CP2	After Node 1	[1, 2]	Node 1 added 2
CP3	After parallel nodes	[1, 2, 3, 4, 5]	Nodes 3,2,4 added 3,4,5
CP4	At END	[1, 2, 3, 4, 5]	Final state saved

Visualization:**Figure 10.5.** 4 Detailed Checkpoint Example**10.6 5. Threads: Managing Multiple Executions****10.6.1 5.1 What are Threads?**

When using persistence, you'll likely execute your workflow multiple times with different inputs. **Thread IDs** help you differentiate between these different executions.

The Problem Without Thread IDs:

If you execute a workflow twice:

- Execution 1: Input [1] → Final state [1,2,3,4,5]
- Execution 2: Input [6] → Final state [6,7,8,9,10]

Both get stored in the database. But when you want to retrieve **only** Execution 1's states, how do you differentiate?

10.6.2 5.2 Thread ID Mechanism

Solution: Assign a unique **Thread ID** to each execution.

How it works:

Table 10.6. 2 Thread ID Mechanism

Execution	Input	Thread ID	Stored Against
First	[1]	"1"	All checkpoints tagged with thread_id=1
Second	[6]	"2"	All checkpoints tagged with thread_id=2

Retrieval:

- Query with thread_id=1 → Get only first execution's data
- Query with thread_id=2 → Get only second execution's data

10.6.3 5.3 Use Case: Chatbot Conversations

Thread IDs are especially useful for chatbots:

Benefits for Chatbots:

1. Each conversation gets a unique thread ID
2. All messages in a conversation are stored against that thread ID
3. User can resume any past conversation by loading its thread ID
4. This is how "Resume Chat" feature works in ChatGPT and similar apps

10.7 6. Implementation in Code**10.7.1 6.1 Basic Setup**

To implement persistence in your LangGraph workflow:

```
# Step 1: Import the checkpointer
from langgraph.checkpoint.memory import InMemorySaver

# Step 2: Create checkpointer instance
checkerpoint = InMemorySaver()

# Step 3: Compile graph with checkpointer
workflow = graph.compile(checkpointer=checkerpoint)

# Step 4: Invoke with thread ID
config = {"configurable": {"thread_id": "1"}}
result = workflow.invoke({"input": "value"}, config=config)
```

10.7.2 6.2 Complete Joke Generation Example

Let's implement a complete example that generates a joke and its explanation:

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
from langgraph.checkpoint.memory import InMemorySaver

# Load environment variables
load_dotenv()

# Initialize LLM
llm = ChatOpenAI()

# Define State
class JokeState(TypedDict):
    topic: str # Input: topic for the joke
    joke: str # Generated joke
    explanation: str # Explanation of the joke
```

```
# Define Nodes
def generate_joke(state: JokeState):
    """Node 1: Generate a joke on the given topic"""
    prompt = f'Generate a joke on the topic: {state["topic"]}'
    response = llm.invoke(prompt).content
    return {'joke': response}

def generate_explanation(state: JokeState):
    """Node 2: Generate explanation for the joke"""
    prompt = f'Write an explanation for this joke: {state["joke"]}'
    response = llm.invoke(prompt).content
    return {'explanation': response}

# Build the Graph
graph = StateGraph(JokeState)

# Add nodes
graph.add_node('generate_joke', generate_joke)
graph.add_node('generate_explanation', generate_explanation)

# Add edges
graph.add_edge(START, 'generate_joke')
graph.add_edge('generate_joke', 'generate_explanation')
graph.add_edge('generate_explanation', END)

# Create checkpoint and compile
checkpoint = InMemorySaver()
workflow = graph.compile(checkpointer=checkpoint)
```

Executing the Workflow:

```
# First execution with thread_id "1"
config1 = {"configurable": {"thread_id": "1"}}
result1 = workflow.invoke({'topic': 'pizza'}, config=config1)

print(result1)
# Output:
# {
#     'topic': 'pizza',
#     'joke': 'Why did the pizza go to the doctor? Because it was feeling a little cheesy!',
#     'explanation': 'This joke plays on the double meaning of "cheesy"...'
# }

# Second execution with thread_id "2"
config2 = {"configurable": {"thread_id": "2"}}
result2 = workflow.invoke({'topic': 'pasta'}, config=config2)
```

10.7.3 6.3 Key API Methods

LangGraph provides several methods to interact with persisted state:

6.3.1 get_state() - Get Current/Final State

```
# Get final state for a specific thread
state = workflow.get_state(config1)
print(state.values)
# {'topic': 'pizza', 'joke': '...', 'explanation': '...'}
```

6.3.2 get_state_history() - Get All Intermediate States

```
# Get all checkpoints for a thread
history = list(workflow.get_state_history(config1))
```

```
for checkpoint in history:
    print(f"Values: {checkpoint.values}")
    print(f"Next node: {checkpoint.next}")
    print("---")
```

Example Output:

Table 10.7. 3.2 `getstatehistory()` - Get All Intermediate States

Checkpoint	Values	Next Node
1	{}	('__start__',)
2	{'topic': 'pizza'}	('generate_joke',)
3	{'topic': 'pizza', 'joke': '...'}	('generate_explanation',)
4	{'topic': 'pizza', 'joke': '...', 'explanation': '...'}	()

6.3.3 `update_state()` - Modify State at a Checkpoint

```
# Update state at a specific checkpoint
workflow.update_state(
    {"configurable": {"thread_id": "1", "checkpoint_id": "..."}},
    {'topic': 'samosa'} # New value
)
```

6.3.4 Summary of API Methods

Table 10.8. 3.4 Summary of API Methods

Method	Purpose	Parameters
<code>invoke()</code>	Execute workflow	input, config (with <code>thread_id</code>)
<code>get_state()</code>	Get current state	config (with <code>thread_id</code>)
<code>get_state_history()</code>	Get all checkpoints	config (with <code>thread_id</code>)
<code>update_state()</code>	Modify state	config (with <code>checkpoint_id</code>), new values

10.8 7. Four Benefits of Persistence

Persistence enables four major capabilities in LangGraph:

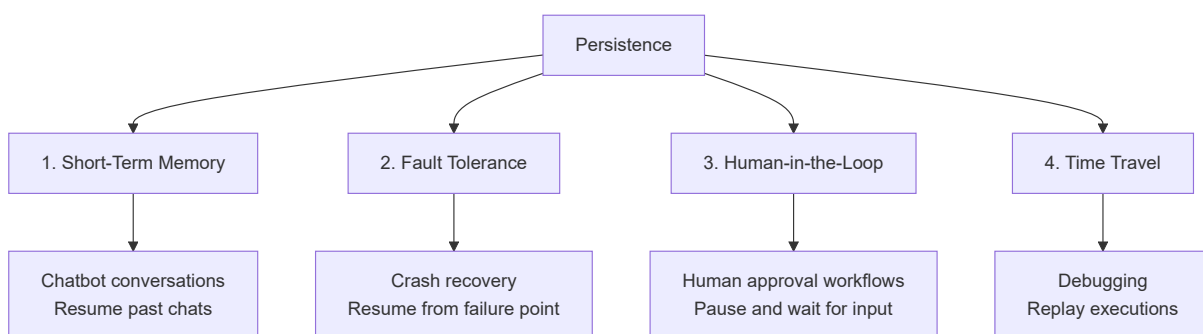


Figure 10.6. Four Benefits of Persistence

10.8.1 7.1 Benefit 1: Short-Term Memory

What it enables: Chatbots can remember conversation history within a session.

How it works:

1. Each conversation gets a unique thread ID
2. All messages are stored against that thread ID
3. When resuming, load all messages for that thread ID
4. Continue conversation with full context

Example: ChatGPT-like Resume Feature:

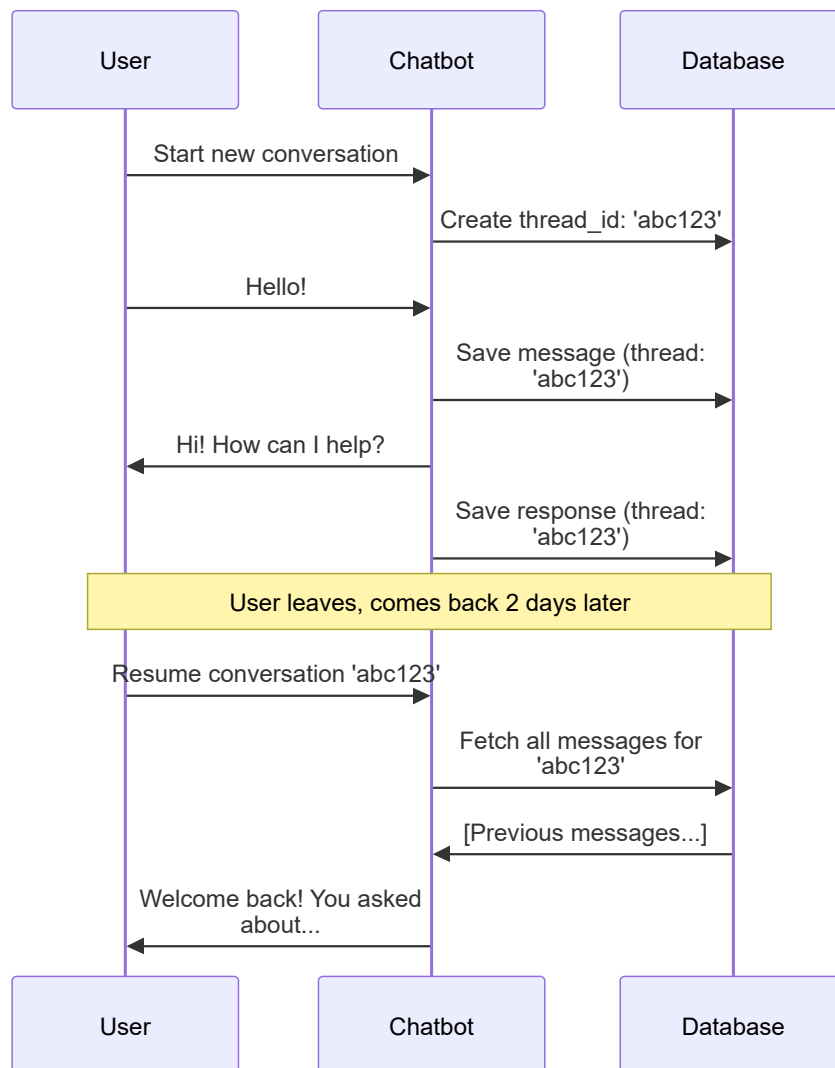


Figure 10.7. 1 Benefit 1: Short-Term Memory

Without Persistence: No way to resume past conversations **With Persistence:** Full conversation history available

10.8.2 7.2 Benefit 2: Fault Tolerance

What it enables: Workflows can recover from crashes and resume from the failure point.

How it works:

1. State is saved at every checkpoint

2. If crash occurs, we know exactly where it happened
3. We know the state values at that point
4. Resume execution from crash point, not from beginning

Key Points:

- Long-running workflows benefit most
- No need to re-execute completed steps
- Server failures, API timeouts, etc. don't lose progress

10.8.3 7.3 Benefit 3: Human-in-the-Loop (HITL)

What it enables: Workflows can pause, wait for human input, and resume.

Use Case Example: LinkedIn Post Workflow:

The Challenge:

- After generating post, workflow needs human approval
- Human might respond immediately or after 2 days
- Can't keep workflow active in memory for 2 days

How Persistence Solves This:

1. Generate post
2. **Interrupt** (suspend) execution
3. Save state to database
4. Wait for human input (minutes to days)
5. When human responds, **resume** from saved state
6. Continue with post or regenerate based on input

10.8.4 7.4 Benefit 4: Time Travel (Debugging)

What it enables: Replay workflow execution from any checkpoint with the same or modified inputs.

Use Cases:

1. **Debugging:** See exactly what happened at each step
2. **Experimentation:** Run same workflow with different state values
3. **Branching:** Create alternative execution paths

Operations Available:

Table 10.9. 4 Benefit 4: Time Travel (Debugging)

Operation	Description	Code Pattern
Replay	Re-execute from a checkpoint	<code>invoke(None, config_with_checkpoint_id)</code>
Branch	Create new execution path	Re-execute from old checkpoint
Modify	Change state and re-execute	<code>update_state()</code> then <code>invoke()</code>

10.9 8. Practical Code Examples

10.9.1 8.1 Fault Tolerance Implementation

```
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import InMemorySaver
from typing import TypedDict
```

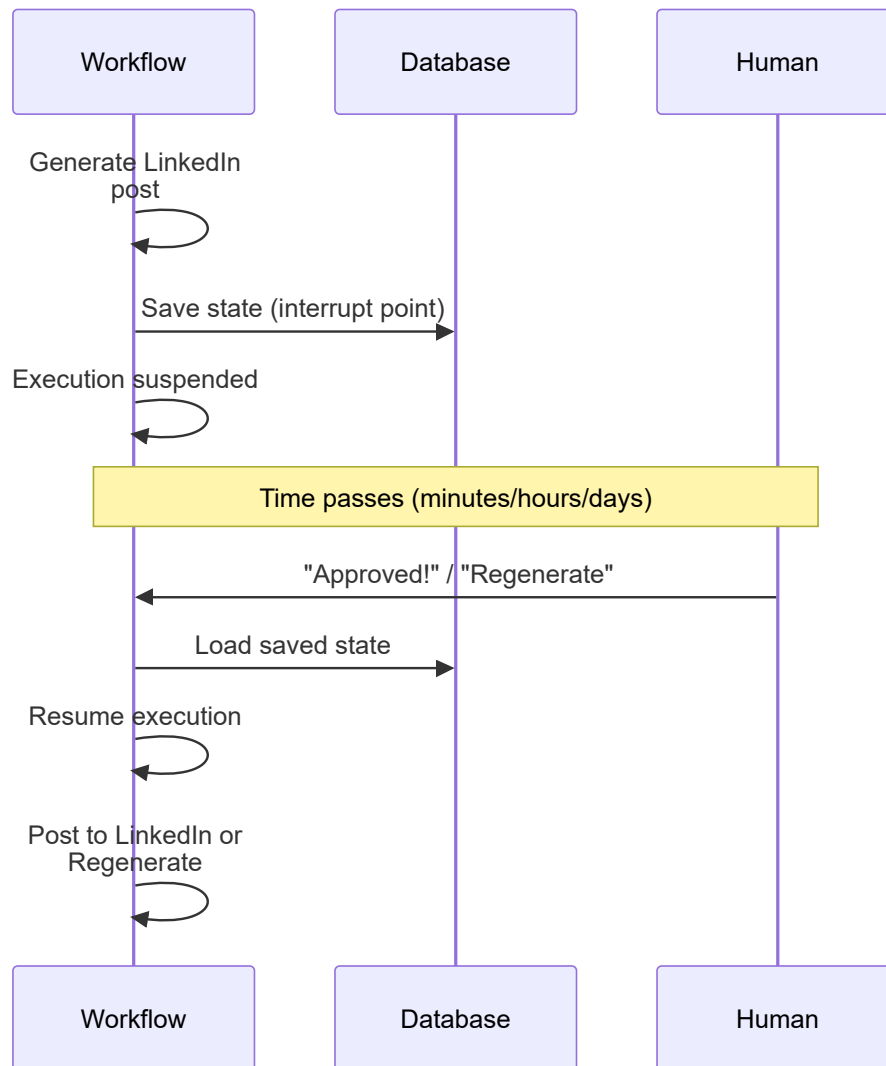


Figure 10.8. 3 Benefit 3: Human-in-the-Loop (HITL)

```

import time

# Define State
class CrashState(TypedDict):
    input: str
    step1: str
    step2: str
    step3: str

# Define Steps
def step_1(state: CrashState):
    print(" Step 1 executed")
    return {"step1": "done"}

def step_2(state: CrashState):
    print(" Step 2 executing... (simulating long operation)")
    time.sleep(30) # Simulates long-running task
    return {"step2": "done"}

def step_3(state: CrashState):
    print(" Step 3 executed")
    return {"step3": "done"}

# Build Graph
builder = StateGraph(CrashState)
  
```

```

builder.add_node("step_1", step_1)
builder.add_node("step_2", step_2)
builder.add_node("step_3", step_3)

builder.set_entry_point("step_1")
builder.add_edge("step_1", "step_2")
builder.add_edge("step_2", "step_3")
builder.add_edge("step_3", END)

checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)

# First execution (will be interrupted)
config = {"configurable": {"thread_id": "fault-demo"}}
try:
    print(" Starting execution...")
    graph.invoke({"input": "start"}, config=config)
except KeyboardInterrupt:
    print(" Execution interrupted (simulating crash)")

# Check current state
state = graph.get_state(config)
print(f"\n Current state: {state.values}")
print(f" Next node to execute: {state.next}")

# Resume execution
print("\n Resuming execution...")
final_state = graph.invoke(None, config=config) # None = resume
print(f"\n Final state: {final_state}")

```

Expected Output:

```

Starting execution...
Step 1 executed
Step 2 executing... (simulating long operation)
Execution interrupted (simulating crash)

Current state: {'input': 'start', 'step1': 'done'}
Next node to execute: ('step_2',)

Resuming execution...
Step 2 executing... (simulating long operation)
Step 3 executed

Final state: {'input': 'start', 'step1': 'done', 'step2': 'done', 'step3': 'done'}

```

10.9.2 8.2 Time Travel Implementation

```

# Assuming joke workflow is already set up and executed

# Step 1: Get state history
history = list(workflow.get_state_history(config1))
print(" State History:")
for i, checkpoint in enumerate(history):
    print(f" [{i}] Values: {checkpoint.values}")
    print(f"      Next: {checkpoint.next}")
    print(f"      Checkpoint ID: {checkpoint.config['configurable']['checkpoint_id']}")
    print()

# Step 2: Find checkpoint where topic was set but joke not generated
target_checkpoint_id = None
for checkpoint in history:
    if 'topic' in checkpoint.values and 'joke' not in checkpoint.values:
        target_checkpoint_id = checkpoint.config['configurable']['checkpoint_id']
        break

print(f" Target checkpoint: {target_checkpoint_id}")

# Step 3: Time travel - Re-execute from that checkpoint

```

```

replay_config = {
    "configurable": {
        "thread_id": "1",
        "checkpoint_id": target_checkpoint_id
    }
}

result = workflow.invoke(None, replay_config)
print(f"\n Replayed result: {result}")

```

10.9.3 8.3 State Update Implementation

```

# Step 1: Find checkpoint to modify
history = list(workflow.get_state_history(config1))
target_checkpoint = None
for cp in history:
    if cp.values.get('topic') == 'pizza' and 'joke' not in cp.values:
        target_checkpoint = cp
        break

checkpoint_id = target_checkpoint.config['configurable']['checkpoint_id']

# Step 2: Update state at that checkpoint
workflow.update_state(
    {
        "configurable": {
            "thread_id": "1",
            "checkpoint_id": checkpoint_id,
            "checkpoint_ns": ""
        }
    },
    {'topic': 'samosa'} # Change topic from 'pizza' to 'samosa'
)

# Step 3: Get the new checkpoint ID (from the update)
new_history = list(workflow.get_state_history(config1))
new_checkpoint_id = new_history[0].config['configurable']['checkpoint_id']

# Step 4: Execute from new state
result = workflow.invoke(
    None,
    {"configurable": {"thread_id": "1", "checkpoint_id": new_checkpoint_id}}
)

print(f" Result with modified state: {result}")
# Now joke will be about 'samosa' instead of 'pizza'

```

10.10 9. Best Practices and Production Considerations

10.10.1 9.1 Checkpointer Selection

Table 10.10. 1 Checkpointer Selection

Environment	Recommended Checkpointer	Reason
Development	InMemorySaver	Fast, no setup needed
Testing	InMemorySaver or SqliteSaver	Easy reset between tests
Production	PostgresSaver or RedisSaver	Persistent, scalable, reliable

10.10.2 9.2 Thread ID Best Practices

```
# Bad: Generic thread IDs
config = {"configurable": {"thread_id": "1"}}

# Good: Meaningful, unique thread IDs
import uuid
from datetime import datetime

# For chatbots: include user and timestamp
thread_id = f"user_{user_id}_chat_{datetime.now().isoformat()}"

# For batch processing: include job identifier
thread_id = f"batch_job_{job_id}_{uuid.uuid4()}"

# For debugging: include context
thread_id = f"debug_feature_xyz_{datetime.now().strftime('%Y%m%d_%H%M%S')}"
```

10.10.3 9.3 Error Handling

```
from langgraph.errors import GraphInterrupt

try:
    result = workflow.invoke(input_data, config)
except GraphInterrupt as e:
    # Handle planned interruptions (HITL)
    print(f"Workflow paused for human input: {e}")
except Exception as e:
    # Handle unexpected errors
    state = workflow.get_state(config)
    print(f"Error at node: {state.next}")
    print(f"Current state: {state.values}")
    # Log and potentially retry
```

10.10.4 9.4 Checkpoint Cleanup

For long-running applications, consider cleaning up old checkpoints:

```
# Pseudo-code for cleanup strategy
def cleanup_old_checkpoints(max_age_days=30):
    """Remove checkpoints older than max_age_days"""
    cutoff_date = datetime.now() - timedelta(days=max_age_days)
    # Implementation depends on your checkpointer storage
    # PostgreSQL: DELETE FROM checkpoints WHERE created_at < cutoff_date
```

10.11 10. Summary and Quick Reference

10.11.1 Key Concepts

Table 10.11. *Key Concepts*

Concept	Definition
Persistence	Ability to save and restore workflow state over time
Checkpoint	Component that implements persistence by saving state at checkpoints
Checkpoint	A saved snapshot of state at a super step
Super Step	Unit of parallel execution (sequential nodes = separate super steps, parallel nodes = one super step)

Concept	Definition
Thread ID	Unique identifier for each workflow execution

10.11.2 API Quick Reference

```
# Setup
checkpointer = InMemorySaver()
workflow = graph.compile(checkpointer=checkpointer)

# Execute with thread ID
config = {"configurable": {"thread_id": "unique_id"}}
result = workflow.invoke({"input": "data"}, config)

# Get current state
state = workflow.get_state(config)

# Get all checkpoints
history = list(workflow.get_state_history(config))

# Resume from crash
result = workflow.invoke(None, config)

# Time travel to specific checkpoint
replay_config = {"configurable": {"thread_id": "id", "checkpoint_id": "cp_id"}}
result = workflow.invoke(None, replay_config)

# Update state at checkpoint
workflow.update_state(replay_config, {"key": "new_value"})
```

10.11.3 Four Benefits Summary

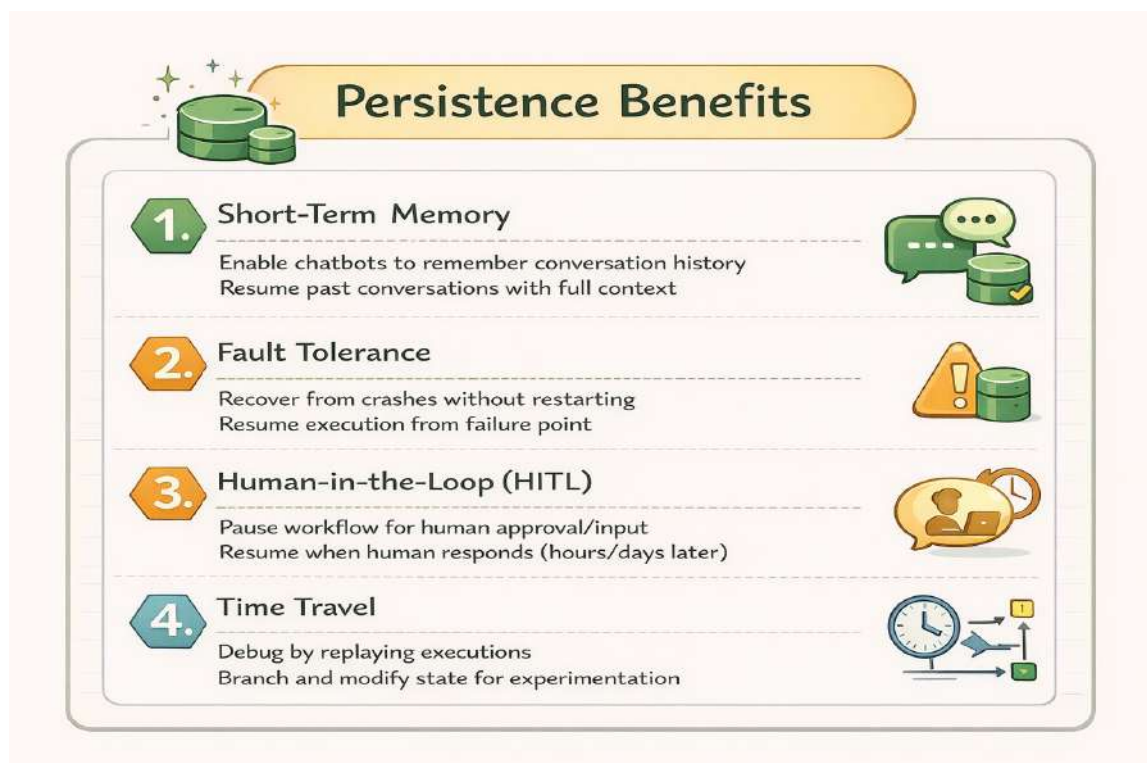


Figure 10.9. Four Benefits Summary

10.12 Conclusion

Persistence is a foundational concept in LangGraph that transforms simple workflows into robust, production-ready applications. By understanding and implementing persistence:

1. Your chatbots can maintain conversation history
2. Your workflows become fault-tolerant and resilient
3. You can implement human approval workflows
4. You gain powerful debugging capabilities through time travel

Key Takeaway: Always use persistence when building any production LangGraph application. Choose the appropriate checkpointer for your environment, and use meaningful thread IDs to organize your workflow executions.

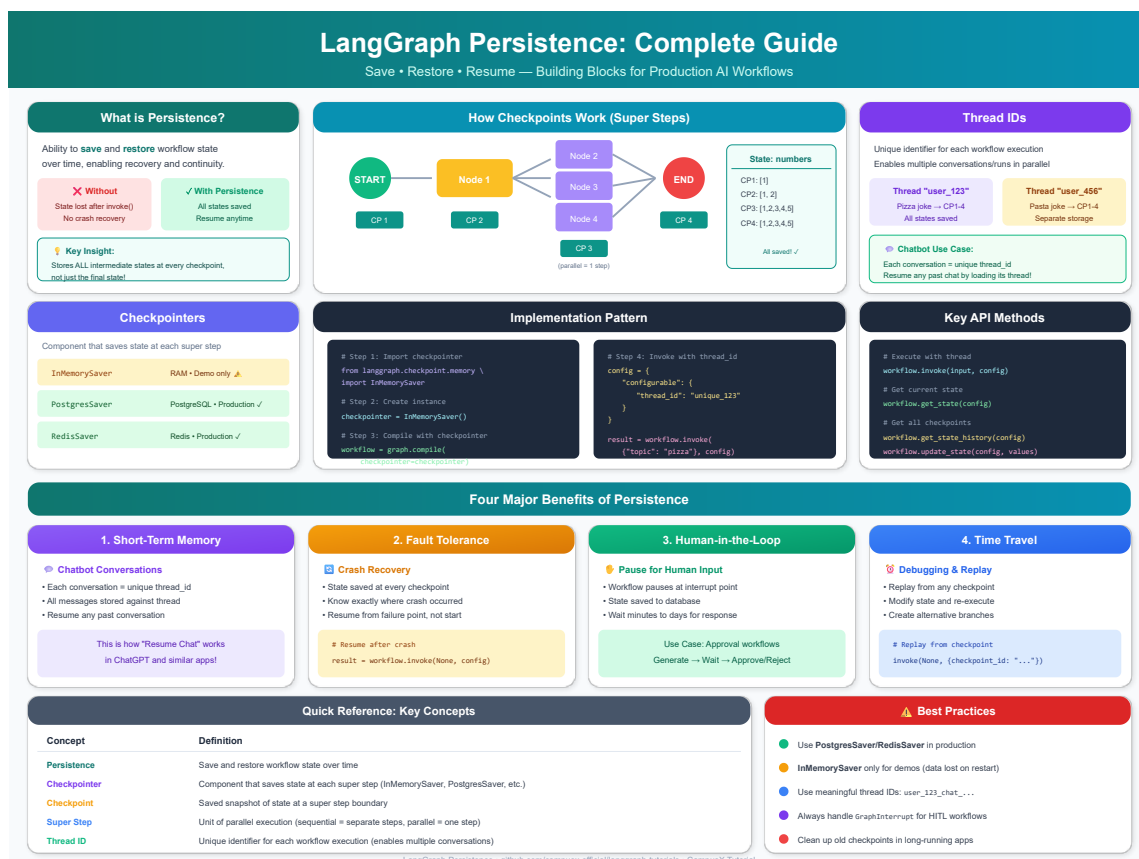


Figure 10.10. Conclusion

Building a LangGraph Chatbot with Streamlit UI

11.1 Code

- <https://github.com/campusx-official/langgraph-tutorials>

11.2 1. Introduction

When building AI-powered chatbots using LangGraph, developers often face a significant challenge: the chatbot works perfectly in a Jupyter notebook or command-line environment, but lacks a proper **User Interface (UI)** that end-users can interact with comfortably.

This guide addresses this limitation by teaching you how to create a **web-based chat interface** using **Streamlit** that connects seamlessly with your **LangGraph backend**. By the end of this guide, you'll have a fully functional chatbot with:

- A clean, modern chat interface
- Message history display with distinct user and assistant icons
- Short-term memory that remembers past conversations
- Real-time interaction capabilities

11.3 2. Understanding the Problem

When you build a chatbot using LangGraph in a Jupyter notebook, you typically interact with it through code cells. While this works for development and testing, it presents several issues for real-world usage:

Problems with Notebook-Based Chatbots:

Table 11.1. *Understanding the Problem*

Issue	Description
No Visual Interface	Users must interact through code, not a friendly UI
Poor User Experience	Technical barrier for non-developers
Limited Accessibility	Cannot be shared easily with end-users
No Visual Separation	Difficult to distinguish between user and AI messages
Non-Production Ready	Cannot be deployed as a standalone application

The Solution:

Create a web-based interface using Streamlit that:

1. Provides a clean chat UI similar to ChatGPT or Claude
2. Handles user input through a text box
3. Displays conversation history with proper formatting
4. Connects to the LangGraph backend for AI responses

11.4 3. Architecture Overview

The chatbot system follows a **two-component architecture** that separates concerns between the AI logic and the user interface.

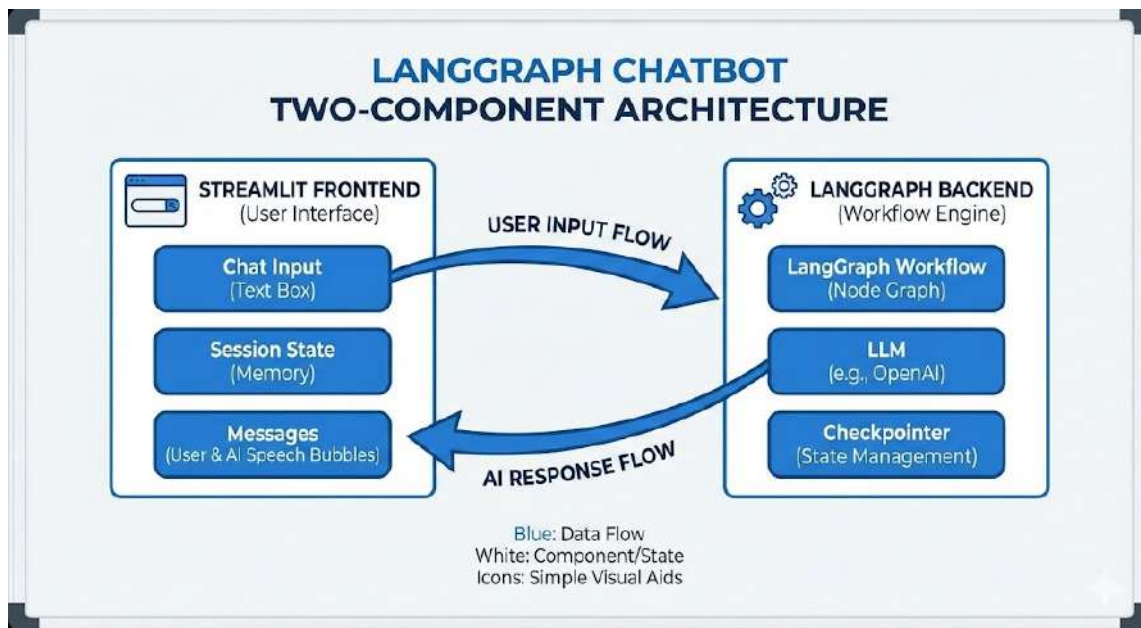


Figure 11.1. Architecture Overview

Component Responsibilities:

Backend (LangGraph):

- Contains the LangGraph workflow definition
- Handles state management and graph compilation
- Processes user messages through the LLM
- Maintains conversation memory using checkpointer
- All AI logic resides here

Frontend (Streamlit):

- Provides the web-based user interface
- Captures user input through chat input component
- Displays messages using chat message components
- Maintains conversation history in session state
- Communicates with the backend to get AI responses

11.5 4. Prerequisites

Before starting, ensure you have the following knowledge and tools:

Knowledge Requirements:

Table 11.2. Prerequisites

Skill	Level Required	Notes
Python	Intermediate	Basic syntax, functions, classes
Streamlit	Basic	15-20 minutes of fundamentals is sufficient
LangGraph	Basic	Understanding of nodes, edges, and workflows
LangChain	Basic	Message types and LLM integration

Skill	Level Required	Notes
-------	----------------	-------

Required Libraries:

```
# Install all required packages
pip install streamlit
pip install langchain
pip install langgraph
pip install langchain-openai # or your preferred LLM provider
pip install python-dotenv
```

Environment Setup:

Create a `.env` file in your project root:

```
OPENAI_API_KEY=your_openai_api_key_here
```

11.6 5. Project Setup

Create the following project structure:

```
chatbot-project/|—
  .env           # Environment variables (API keys)|—
  langgraph_backend.py # LangGraph workflow (Backend)|—
  streamlit_frontend.py # Streamlit UI (Frontend)|—
  requirements.txt  # Project dependencies
```

Backend File: `langgraph_backend.py`

This file contains the LangGraph workflow. If you've followed previous LangGraph tutorials, this code should be familiar:

```
from dotenv import load_dotenv
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import MemorySaver
from langchain_openai import ChatOpenAI

# Load environment variables
load_dotenv()

# Initialize the LLM
llm = ChatOpenAI(model='gpt-4o-mini')

# Define the State
class State(TypedDict):
    messages: Annotated[list, add_messages]

# Define the chat function (node)
def chat(state: State):
    """Process the current state and generate a response."""
    return {'messages': [llm.invoke(state['messages'])]}

# Build the graph
graph_builder = StateGraph(State)

# Add the chat node
graph_builder.add_node('chat', chat)

# Define edges
graph_builder.add_edge(START, 'chat')
graph_builder.add_edge('chat', END)

# Add checkpoint for memory
```

```

checkpoint = MemorySaver()

# Compile the graph
chatbot = graph_builder.compile(checkpointer=checkpoint)

```

Understanding the Backend Code:

1. **State Definition:** The State TypedDict defines what data flows through the graph. Here, it contains a list of messages.
2. **Chat Function:** This is the main node that:
 - Takes the current state (including all messages)
 - Invokes the LLM with those messages
 - Returns the LLM's response
3. **Graph Structure:**

```
START → chat → END
```

A simple linear flow where every message goes through the chat node.

4. **Checkpoint:** MemorySaver enables short-term memory by saving conversation state.

11.7 6. Streamlit Chat Interface Fundamentals

Before integrating LangGraph, you must understand the two essential Streamlit components for building chat interfaces.

11.7.1 6.1 Chat Message Component

The `st.chat_message` component creates a message bubble with an icon indicating who sent the message.

Basic Syntax:

```

import streamlit as st

# Display a user message
with st.chat_message('user'):
    st.text('Hello, how are you?')

# Display an assistant message
with st.chat_message('assistant'):
    st.text('I am doing well, thank you!')

```

How It Works:

Role Parameter Values:

Table 11.3. 1 Chat Message Component

Role	Icon	Use Case
'user'	Person icon	Messages typed by the user
'assistant'	AI/Sparkle icon	Responses from the AI
'human'	Person icon	Alternative to 'user'

Example Output:

When you run the code, you'll see:

- A message box with a **person icon** for user messages

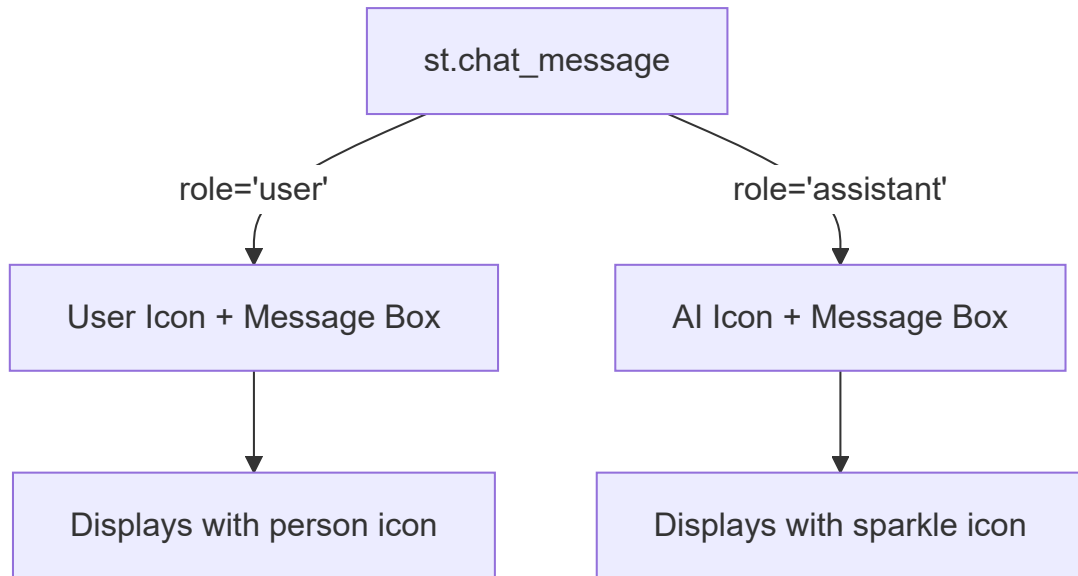


Figure 11.2. 1 Chat Message Component

- A message box with a **different icon** for assistant messages

Customization Options:

```
# Using custom avatar
with st.chat_message('user', avatar=''):
    st.text('Custom avatar message')

# Multiple content types in one message
with st.chat_message('assistant'):
    st.text('Here is some text')
    st.code('print("Hello World")', language='python')
```

11.7.2 6.2 Chat Input Component

The `st.chat_input` component creates an input text box at the bottom of the screen where users can type their messages.

Basic Syntax:

```
import streamlit as st

# Create chat input
user_input = st.chat_input('Type your message here...')

# Check if user submitted something
if user_input:
    st.write(f'You typed: {user_input}')
```

How It Works:

Key Characteristics:

1. **Fixed Position:** Always appears at the bottom of the page
2. **Enter to Submit:** Users press Enter to submit their message
3. **Returns String:** The input is captured as a string variable
4. **Triggers Rerun:** Submitting triggers a complete script rerun

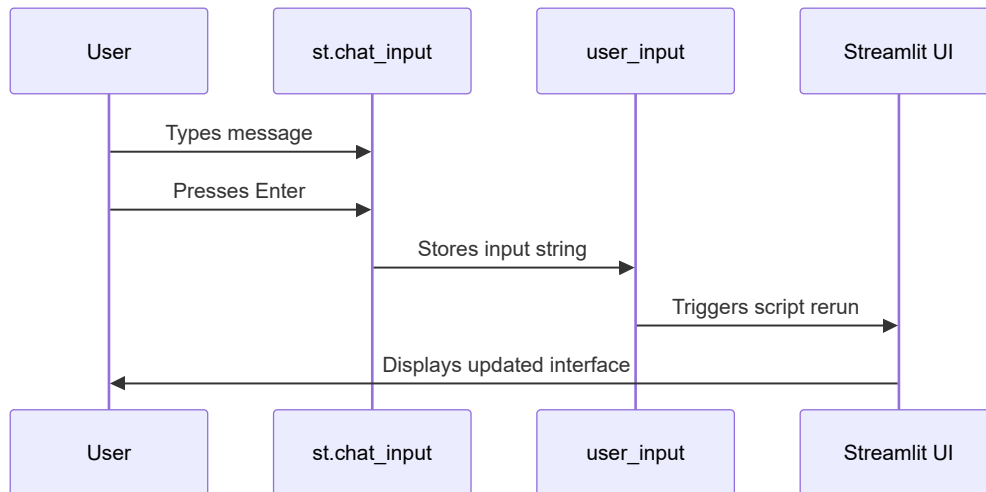


Figure 11.3. 2 Chat Input Component

11.8 7. Building a Basic Copy-Cat Chatbot

Before connecting to LangGraph, let's build a simple chatbot that echoes back whatever the user types. This helps understand the basic flow.

Initial (Broken) Attempt:

```

import streamlit as st

# Get user input
user_input = st.chat_input('Type here')

if user_input:
    # Display user message
    with st.chat_message('user'):
        st.text(user_input)

    # Display assistant message (copy-cat)
    with st.chat_message('assistant'):
        st.text(user_input) # Echo the same message
  
```

The Problem:

When you run this code and send multiple messages, you'll notice that **previous messages disappear**. Only the current message pair is visible.

Why This Happens:

Every time you press Enter in Streamlit:

1. The entire Python script runs from top to bottom
2. All previous variable values are lost
3. Only the current `user_input` value exists

The Solution: We need to store conversation history!

11.9 8. Understanding Streamlit's Execution Model

This is a critical concept that many developers miss when first working with Streamlit.

What Gets Reset:

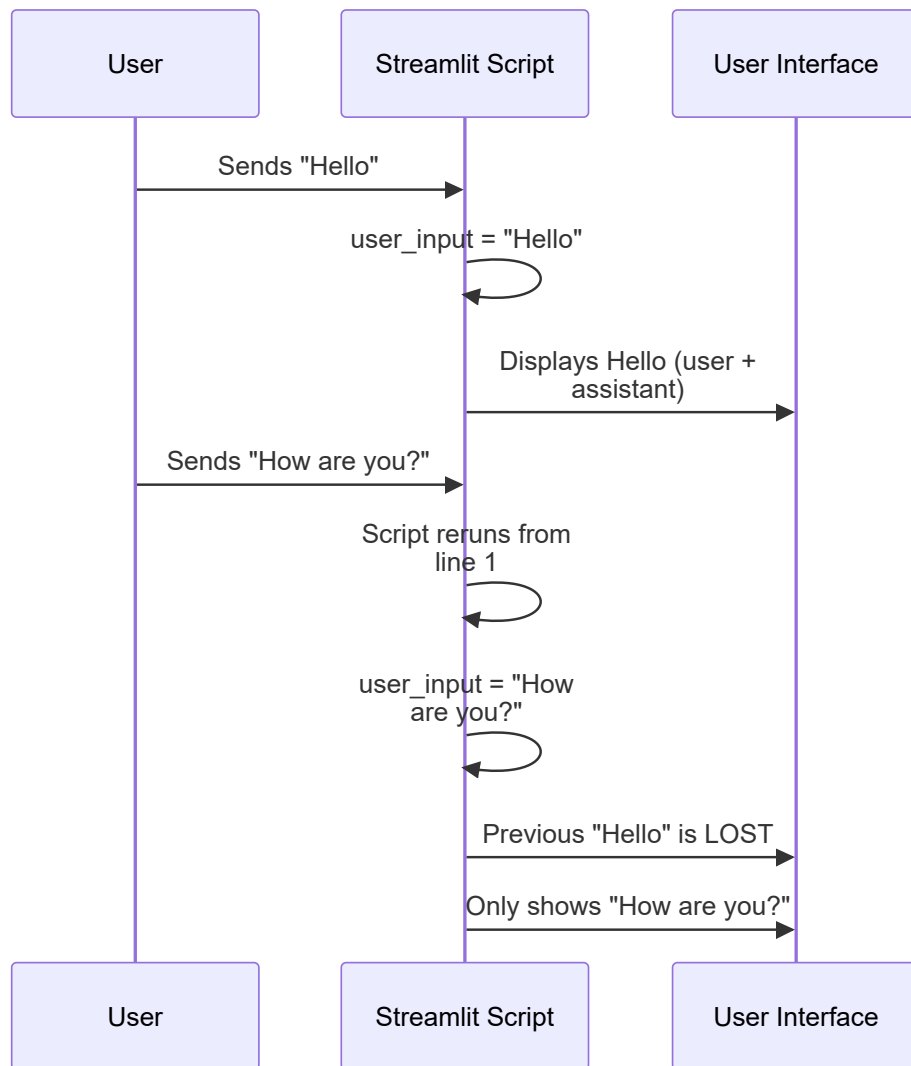


Figure 11.4. Building a Basic Copy-Cat Chatbot

Streamlit Execution Model: Rerun & Session State

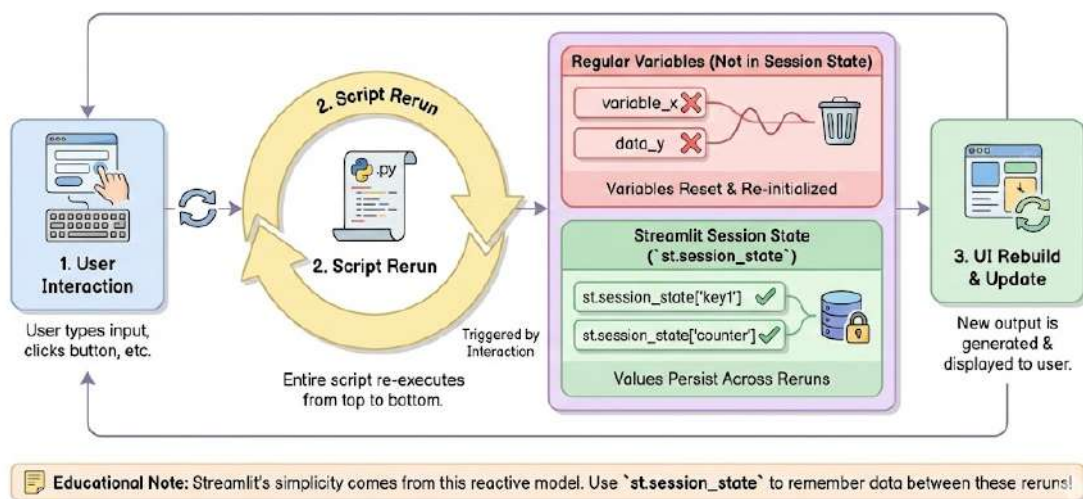


Figure 11.5. Building a Basic Copy-Cat Chatbot

Table 11.4. *Understanding Streamlit's Execution Model*

Item	Reset on Rerun?	Example
Regular variables	✓ Yes	<code>message_history = []</code>
Function calls	✓ Yes	Re-executed each time
UI components	✓ Yes	Rebuilt from code
Session State	✗ No	<code>st.session_state['key']</code>

Practical Example:

```
# This gets reset to [] on EVERY rerun
message_history = []

# User types "Hello"
message_history.append("Hello")
# message_history = ["Hello"]

# User types "Hi"
# Script reruns...
message_history = [] # Reset!
message_history.append("Hi")
# message_history = ["Hi"] # "Hello" is lost!
```

This is why we need **Session State**.

11.10 9. Session State: Persisting Conversation History

Session State is a special dictionary in Streamlit that **survives script reruns**. Data in session state only resets when:

- The user manually refreshes the browser page
- The session expires
- You explicitly clear it

How to Use Session State:

```
import streamlit as st

# Initialize session state (only runs once)
if 'message_history' not in st.session_state:
    st.session_state['message_history'] = []

# Add to session state
st.session_state['message_history'].append({'role': 'user', 'content': 'Hello'})

# Access session state
for msg in st.session_state['message_history']:
    print(msg)
```

Message Structure:

We store each message as a dictionary with two keys:

```
{
    'role': 'user',          # or 'assistant'
    'content': 'Hello World' # the actual message text
}
```

This structure allows us to:

1. Know who sent each message
2. Display the correct icon
3. Maintain the conversation order

Complete Copy-Cat Chatbot with Session State:

```
import streamlit as st

# Initialize session state for message history
if 'message_history' not in st.session_state:
    st.session_state['message_history'] = []

# Load and display conversation history
for message in st.session_state['message_history']:
    with st.chat_message(message['role']):
        st.text(message['content'])

# Get user input
user_input = st.chat_input('Type here')

if user_input:
    # Add user message to history
    st.session_state['message_history'].append({
        'role': 'user',
        'content': user_input
    })

    # Display user message
    with st.chat_message('user'):
        st.text(user_input)

    # Add assistant message to history (copy-cat)
    st.session_state['message_history'].append({
        'role': 'assistant',
        'content': user_input
    })

    # Display assistant message
    with st.chat_message('assistant'):
        st.text(user_input)
```

11.11 10. Integrating LangGraph with Streamlit

Now we connect our Streamlit frontend to the LangGraph backend. The key insight is: **we only need to change the assistant message generation logic.**

What Stays the Same:

- Loading conversation history
- Getting user input
- Storing and displaying user messages

What Changes:

- Instead of echoing the user's message, we call `chatbot.invoke()` from LangGraph

Step-by-Step Integration:

Step 1: Import the Backend

```
from langgraph_backend import chatbot
from langchain_core.messages import HumanMessage
```

- `chatbot` is the compiled LangGraph graph
- `HumanMessage` is required to format user input correctly

Step 2: Define Configuration

```
CONFIG = {'configurable': {'thread_id': 'thread-1'}}
```

The `thread_id` is essential because:

- Our backend uses a checkpointer (MemorySaver)
- The checkpointer requires a thread ID to track conversation state
- Without it, memory won't work

Step 3: Invoke the Chatbot

```
# Instead of echoing user_input...
response = chatbot.invoke(
    {'messages': [HumanMessage(content=user_input)]},
    config=CONFIG
)

# Extract the AI's response
ai_message = response['messages'][-1].content
```

Response Structure:

When you call `chatbot.invoke()`, you get back a dictionary:

```
response = {
    'messages': [
        HumanMessage(content='Hello'),           # User's message
        AIMessage(content='Hi! How can I help?') # AI's response
    ]
}
```

The AI's response is always the **last message** in the list, accessed via:

```
ai_message = response['messages'][-1].content
```

11.12 11. Complete Implementation

Here is the complete, production-ready code for both files.

File 1: `langgraph_backend.py`

```
"""
LangGraph Backend for Chatbot
This file contains the LangGraph workflow that processes user messages.
"""

from dotenv import load_dotenv
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import MemorySaver
from langchain_openai import ChatOpenAI

# Load environment variables (API keys)
load_dotenv()

# Initialize the Language Model
llm = ChatOpenAI(model='gpt-4o-mini')

class State(TypedDict):
    """
    State definition for the chatbot.

    Attributes:
        messages: List of messages in the conversation.
                   The add_messages annotation ensures messages
                   are appended rather than replaced.
    """
    messages: Annotated[list, add_messages]
```

```
def chat(state: State) -> dict:
    """
    Chat node function that processes messages through the LLM.

    Args:
        state: Current state containing message history

    Returns:
        Dictionary with the LLM's response message
    """
    return {'messages': [llm.invoke(state['messages'])]}

# Build the graph
graph_builder = StateGraph(State)

# Add the chat node
graph_builder.add_node('chat', chat)

# Define the workflow edges
graph_builder.add_edge(START, 'chat') # Start -> Chat
graph_builder.add_edge('chat', END) # Chat -> End

# Initialize checkpointer for conversation memory
checkpointer = MemorySaver()

# Compile the graph with checkpointer
chatbot = graph_builder.compile(checkpointer=checkpointer)
```

File 2: streamlit_frontend.py

```
"""
Streamlit Frontend for LangGraph Chatbot
This file creates the web-based user interface for the chatbot.
"""

import streamlit as st
from langgraph_backend import chatbot
from langchain_core.messages import HumanMessage

# Configuration for LangGraph
# thread_id is required for the checkpointer to maintain memory
CONFIG = {'configurable': {'thread_id': 'thread-1'}}

# =====
# STEP 1: Initialize Session State
# =====
# This only runs once when the session starts
if 'message_history' not in st.session_state:
    st.session_state['message_history'] = []

# =====
# STEP 2: Load and Display Conversation History
# =====
# Display all previous messages from the session
for message in st.session_state['message_history']:
    with st.chat_message(message['role']):
        st.text(message['content'])

# =====
# STEP 3: Get User Input
# =====
user_input = st.chat_input('Type here')

# =====
# STEP 4: Process User Input
# =====
if user_input:
```

```

# -----
# 4a: Handle User Message
# -----
# Add to history
st.session_state['message_history'].append({
    'role': 'user',
    'content': user_input
})

# Display on screen
with st.chat_message('user'):
    st.text(user_input)

# -----
# 4b: Get AI Response from LangGraph
# -----
# Invoke the chatbot with the user's message
response = chatbot.invoke(
    {'messages': [HumanMessage(content=user_input)]},
    config=CONFIG
)

# Extract the AI's response (last message in the list)
ai_message = response['messages'][-1].content

# -----
# 4c: Handle AI Message
# -----
# Add to history
st.session_state['message_history'].append({
    'role': 'assistant',
    'content': ai_message
})

# Display on screen
with st.chat_message('assistant'):
    st.text(ai_message)

```

Running the Application:

```

# Make sure you're in the project directory
cd chatbot-project

# Activate your virtual environment (if using one)
source venv/bin/activate # On Unix/Mac
# or
venv\Scripts\activate    # On Windows

# Run the Streamlit app
streamlit run streamlit_frontend.py

```

Expected Output:

The application will open in your browser at <http://localhost:8501> with:

- A clean chat interface
- An input box at the bottom
- Messages displayed with appropriate icons
- Memory that remembers previous messages in the conversation

11.13 12. Common Pitfalls and Solutions

11.13.1 Pitfall 1: Forgetting the Thread ID

Symptom: Error message about missing `thread_id` or memory not working.

Cause: When using a checkpointer, you must provide a `thread_id` in the config.

Solution:

```
# WRONG - No config
response = chatbot.invoke({'messages': [HumanMessage(content=user_input)]})

# CORRECT - With config
CONFIG = {'configurable': {'thread_id': 'thread-1'}}
response = chatbot.invoke(
    {'messages': [HumanMessage(content=user_input)]},
    config=CONFIG
)
```

11.13.2 Pitfall 2: Using Regular Dictionary Instead of Session State

Symptom: Conversation history disappears after each message.

Cause: Regular Python dictionaries reset on script rerun.

Solution:

```
# WRONG - Regular dictionary
message_history = [] # Resets every time!

# CORRECT - Session state
if 'message_history' not in st.session_state:
    st.session_state['message_history'] = []
```

11.13.3 Pitfall 3: Not Importing HumanMessage

Symptom: Error about HumanMessage not being defined.

Cause: Missing import statement.

Solution:

```
from langchain_core.messages import HumanMessage
```

11.13.4 Pitfall 4: Incorrect Response Extraction

Symptom: Getting the full response object instead of just the text.

Cause: Not extracting the content from the message object.

Solution:

```
# WRONG - Gets the message object
ai_message = response['messages'][-1]

# CORRECT - Gets the text content
ai_message = response['messages'][-1].content
```

11.13.5 Pitfall 5: Environment Variables Not Loading

Symptom: Authentication errors with the LLM provider.

Cause: .env file not loaded or incorrect path.

Solution:

```
# In your backend file
from dotenv import load_dotenv
load_dotenv() # Must be called before using environment variables
```

11.14 13. Summary

Key Concepts Covered:

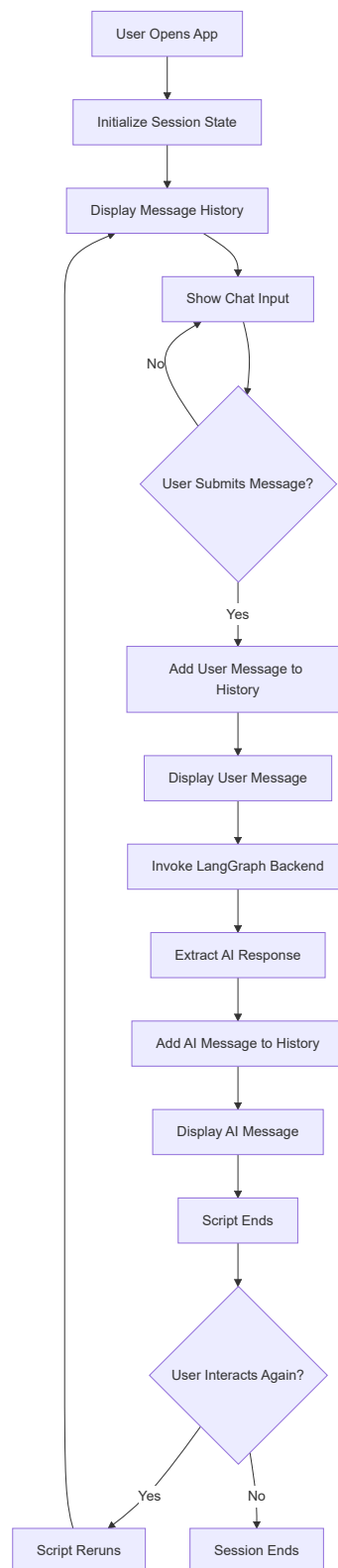
Table 11.5. *Summary*

Concept	Description
Two-Component Architecture	Separate backend (LangGraph) and frontend (Streamlit)
st.chat_message	Component for displaying messages with role-based icons
st.chat_input	Component for capturing user input
Session State	Persistent storage that survives script reruns
Execution Model	Streamlit reruns entire script on each interaction
Integration Pattern	Import backend, invoke with HumanMessage, extract content

The Complete Flow:

What You've Learned:

1. How to structure a chatbot application with separate backend and frontend
2. How to use Streamlit's chat components (`chat_message` and `chat_input`)
3. Why session state is crucial for maintaining conversation history
4. How to integrate a LangGraph workflow with a Streamlit UI
5. How to handle the conversation flow and state management
6. Common mistakes to avoid and how to fix them

**Figure 11.6.** *Summary*

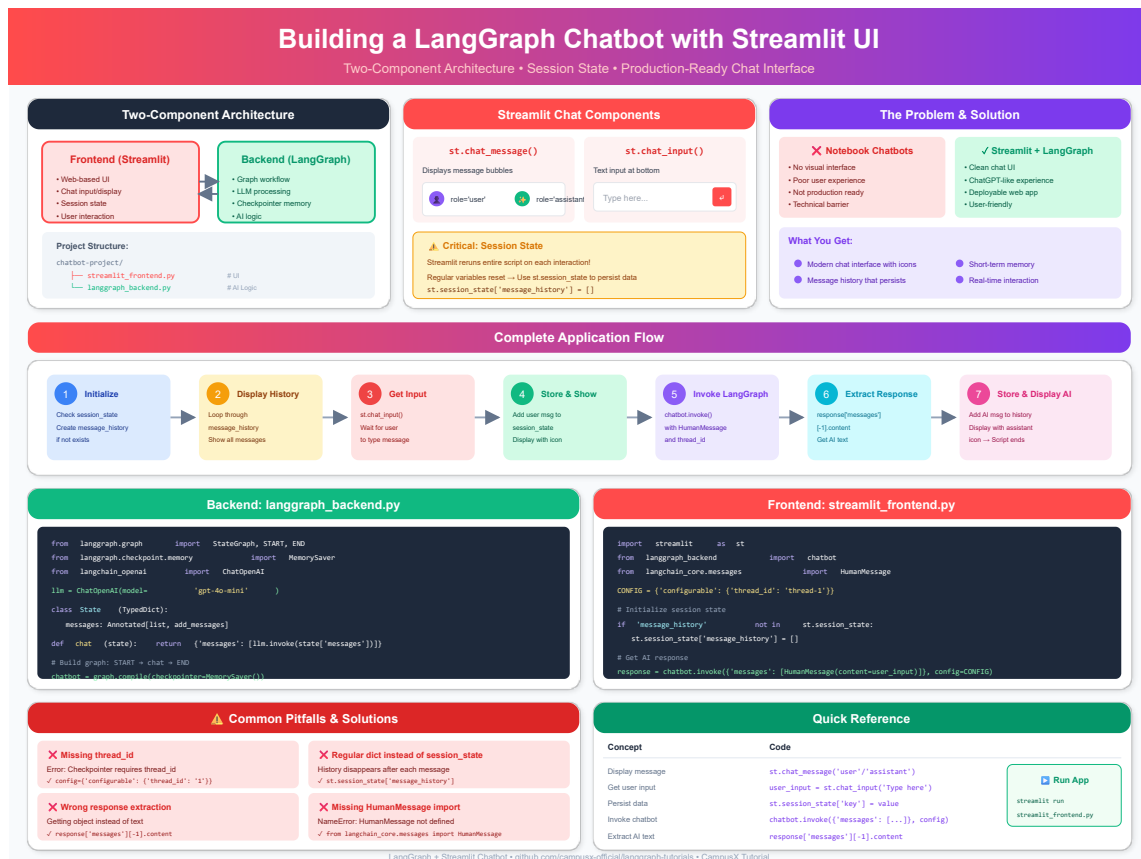


Figure 11.7. Summary

Streaming in LLM Applications using LangGraph

12.1 Code

- <https://github.com/campusx-official/chatbot-in-langgraph>

12.2 Introduction

When building LLM-based chatbots, one common problem developers face is the **delayed response experience**. When you ask an LLM to generate a long output (like a 500-word blog post), the entire response is generated first and then displayed all at once. This creates two significant problems:

1. **User Wait Time:** Users must wait 5-10 seconds with a blank screen
2. **Poor Readability:** Large chunks of text appearing suddenly are hard to read

The solution to this problem is **Streaming** - a technique that displays tokens (words) as they are generated, creating a typewriter effect similar to what you see in ChatGPT.

12.3 What is Streaming?

12.3.1 Definition

In LLMs, streaming means the model starts sending tokens (words) as soon as they're generated, instead of waiting for the entire response to be ready before returning it.

12.3.2 Visual Comparison: With vs Without Streaming

12.3.3 Two Ways of Response Delivery

When you send a query to an LLM (e.g., "What is the recipe for making pasta?"), there are two ways the response can be delivered:

Table 12.1. *Two Ways of Response Delivery*

Approach	How it Works	User Experience
Non-Streaming	LLM generates the complete answer first, then sends the entire response at once	User waits with blank screen, then sees everything suddenly
Streaming	LLM sends tokens as they are generated, one by one	User sees words appearing progressively like a typewriter

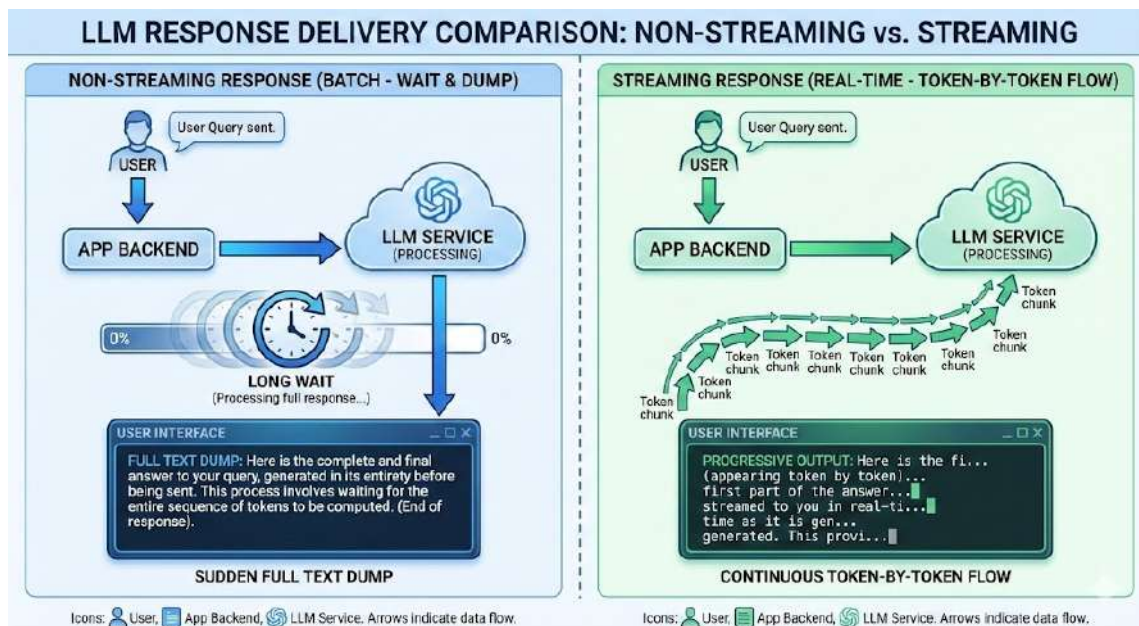


Figure 12.1. Visual Comparison: With vs Without Streaming

12.4 Why Streaming is Important

Streaming is considered essential for LLM-based applications. Here are the six key benefits:

12.4.1 1. Faster Response Time (Low Drop-off Rates)

Problem without streaming:

- When generating long content (essays, blogs, code), LLM takes 5-10 seconds
- Users see a blank screen during this time
- Non-technical users may think the app is frozen or broken
- This leads to **user drop-off** - users abandon the application

Solution with streaming:

- Response starts appearing immediately
- Users know the system is working
- Reduces anxiety and confusion
- Keeps users engaged with the application

12.4.2 2. Mimics Human-Like Conversation

Streaming creates a natural conversation experience because:

- **Builds Trust:** Users feel they're interacting with something intelligent
- **Feels Alive:** The response appears to be "thinking" and "typing"
- **Keeps User Engaged:** Users stay focused watching the next word appear

This is exactly what you experience with ChatGPT - it feels like talking to another person rather than waiting for a machine to compute.

12.4.3 3. Essential for Multi-Modal UIs

Consider voice-based interfaces like Alexa or Google Assistant:

Without Streaming:

User: "Give me the recipe to cook pasta"
 [10 seconds of silence]
 Device: [Speaks entire response at once]

This creates an awkward, unnatural conversation with uncomfortable silence gaps.

With Streaming:

User: "Give me the recipe to cook pasta"
 Device: [Starts speaking immediately, word by word]

This creates seamless, natural conversation flow.

12.4.4 4. Better UX for Long Outputs (Code, Documents)

When generating code or long documents:

Table 12.2. *Better UX for Long Outputs (Code, Documents)*

Without Streaming	With Streaming
Entire code block appears at once	Code appears line by line
Difficult to understand structure	Easy to follow logic flow
Overwhelming to read	Better comprehension
User can't identify what's happening	User sees the progression

12.4.5 5. Ability to Cancel Midway (Token Savings = Cost Savings)

A crucial benefit for production applications:

- **You can stop generation midway** if the response isn't what you wanted
- Stopping early means **fewer tokens are generated**
- Fewer tokens = **Lower API costs** (since LLM providers charge per token)

12.4.6 6. Interleave UI Updates (For AI Agents)

When building AI agents that perform multiple steps:

Without Streaming Updates:

User: "Book me a movie ticket"
 [1 minute of nothing]
 Agent: "Your ticket is booked"

User is anxious and uncertain during the wait.

With Streaming Updates:

User: "Book me a movie ticket"
 Agent: "Opening BookMyShow..."
 Agent: "Searching for movies..."
 Agent: "Found Avengers, selecting..."
 Agent: "Choosing seat A12..."
 Agent: "Processing payment..."
 Agent: "Ticket booked successfully!"

Each step provides feedback, keeping the user informed and engaged.



Figure 12.2. Interleave UI Updates (For AI Agents)

12.5 How Streaming Works - Technical Foundation

12.5.1 The Core Mechanism

12.5.2 Key Concept: Generator Objects

The streaming mechanism in Python relies on **Generators**. When you call `.stream()` instead of `.invoke()`, you receive a **generator object** instead of a complete response.

12.6 Python Generators - The Building Block

12.6.1 What is a Generator?

In Python, a generator is a special type of iterator that allows you to generate values on the fly, one at a time, using the `yield` keyword instead of `return`.

12.6.2 Generator vs Regular Function

```
# Regular Function - Returns all at once
def get_numbers_regular():
    result = []
    for i in range(5):
        result.append(i)
    return result # Returns complete list

numbers = get_numbers_regular()
print(numbers) # [0, 1, 2, 3, 4] - All at once

# Generator Function - Yields one at a time
def get_numbers_generator():
    for i in range(5):
        yield i # Yields one value at a time

gen = get_numbers_generator()
for num in gen:
    print(num) # 0, then 1, then 2, then 3, then 4 - One by one
```

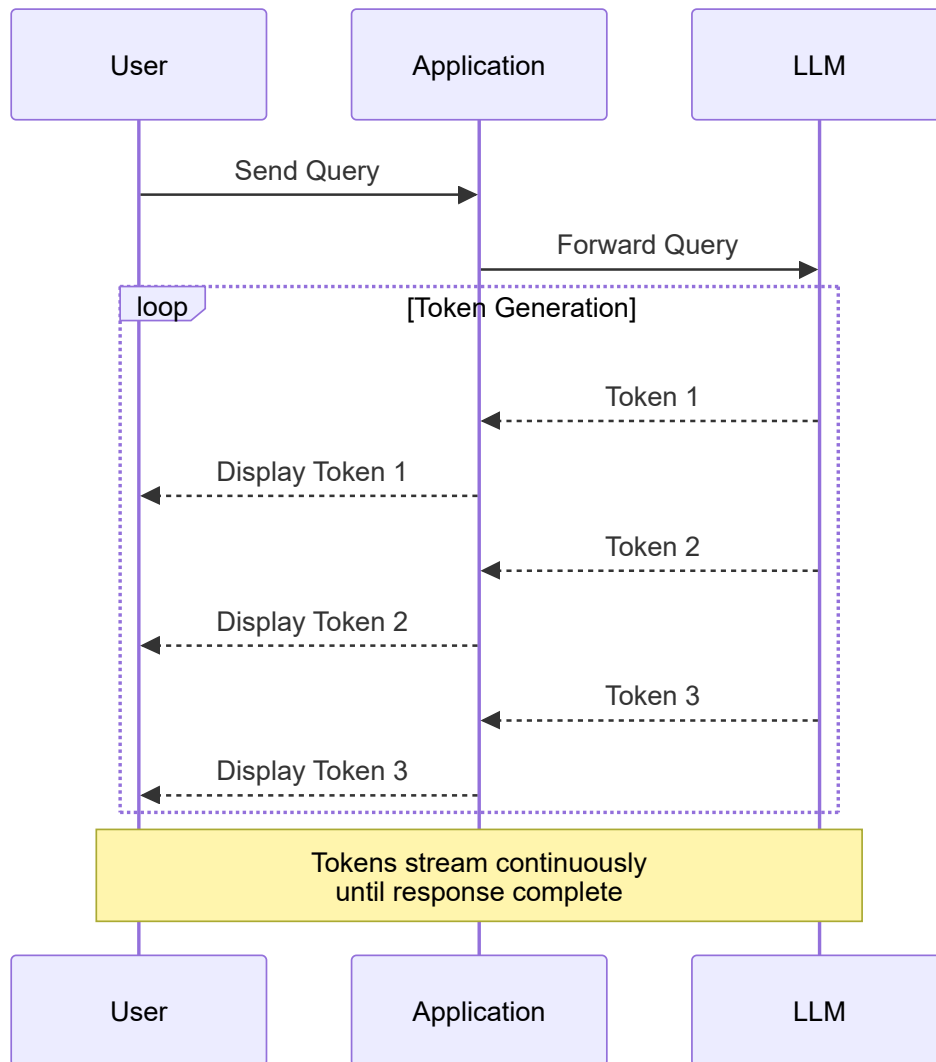


Figure 12.3. The Core Mechanism

12.6.3 Key Differences

Table 12.3. Key Differences

Regular Function	Generator Function
Uses return	Uses yield
Returns complete data at once	Yields data one item at a time
Stores entire result in memory	Memory efficient (one item at a time)
Execution completes, then returns	Pauses at yield, resumes on next call

12.6.4 Why Generators for Streaming?

Generators are perfect for streaming because:

1. They produce values **on-demand** (lazy evaluation)
2. They don't require storing the entire response in memory
3. They can be **iterated over** with a simple loop
4. They naturally support the **token-by-token** delivery pattern

12.7 Implementing Streaming in LangGraph

12.7.1 The Key Change: `.invoke()` vs `.stream()`

The fundamental change to enable streaming is simple:

```
# Without Streaming (Old Way)
response = graph.invoke(initial_state, config=config)

# With Streaming (New Way)
stream = graph.stream(initial_state, config=config, stream_mode='messages')
```

12.7.2 Understanding the Stream Object

When you call `.stream()`, you receive a generator that yields tuples containing:

1. **Message Chunk:** Contains the actual content (tokens)
2. **Metadata:** Additional information about the stream

```
for message_chunk, metadata in graph.stream(...):
    print(message_chunk.content) # Access the actual token text
```

12.7.3 Complete Streaming Implementation Pattern

```
from langgraph.graph import StateGraph, START, END
from langchain_core.messages import HumanMessage

# Step 1: Define your graph (same as before)
chatbot = graph.compile(checkpointer=checkpointer)

# Step 2: Configure the stream call
initial_state = {'messages': [HumanMessage(content="Your question here")]}
config = {'configurable': {'thread_id': 'thread-1'}}

# Step 3: Use .stream() with stream_mode='messages'
for message_chunk, metadata in chatbot.stream(
    initial_state,
    config=config,
    stream_mode='messages'
):
    # Step 4: Check if content exists and print
    if message_chunk.content:
        print(message_chunk.content, end=' ')
```

12.7.4 Step-by-Step Breakdown

12.8 Stream Modes in LangGraph

LangGraph provides multiple streaming modes for different use cases:

Table 12.4. Stream Modes in LangGraph

Stream Mode	Description	Use Case
messages	Streams LLM tokens as they are generated	Chatbots, text generation
values	Streams the complete state after each node	Debugging, state monitoring
updates	Streams only the changes/updates to state	Efficient state tracking
custom	Custom streaming logic	Advanced agent implementations

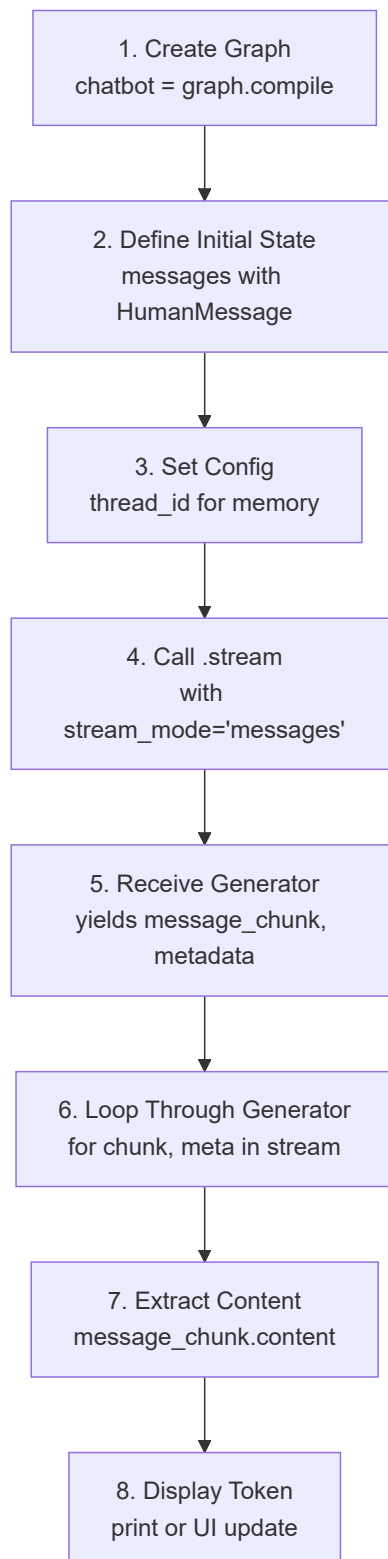


Figure 12.4. *Step-by-Step Breakdown*

12.8.1 When to Use Which Mode

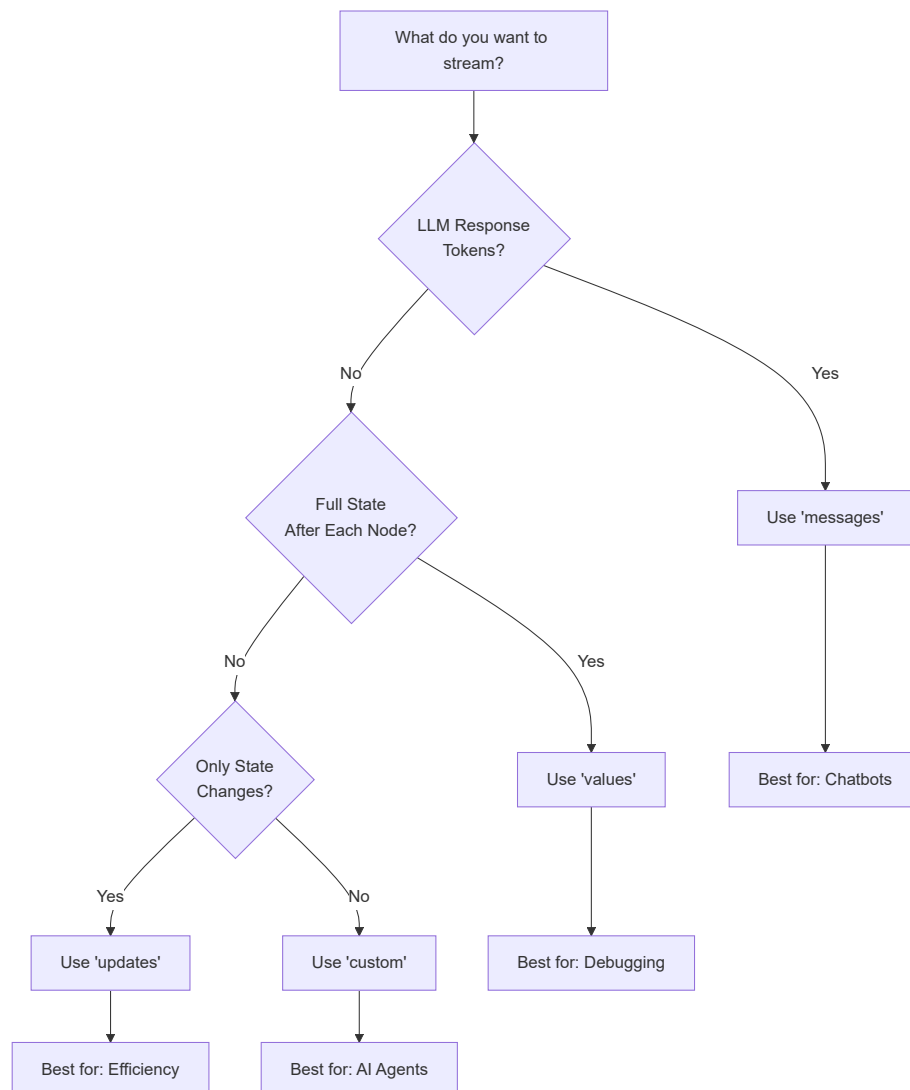


Figure 12.5. *When to Use Which Mode*

For displaying LLM responses with typewriter effect, always use `stream_mode='messages'`.

12.9 Frontend Integration with Streamlit

12.9.1 Streamlit's Chat Elements

Streamlit provides several elements for building chat interfaces:

Table 12.5. *Streamlit's Chat Elements*

Element	Purpose
<code>st.chat_input()</code>	Text input for user messages
<code>st.chat_message()</code>	Container for displaying messages
<code>st.write_stream()</code>	Displays generator content with typewriter effect
<code>st.status()</code>	Shows status updates (useful for agents)

12.9.2 The Magic of `st.write_stream()`

`st.write_stream()` is specifically designed for streaming:

- Takes a **generator** as input
- Automatically handles the **typewriter effect**
- Returns the **complete text** after streaming finishes
- No manual loop or printing required

```
# This one line handles all the streaming UI!
ai_message = st.write_stream(generator_object)
```

12.9.3 Integration Pattern

```
import streamlit as st
from langchain_core.messages import HumanMessage
from langgraph_backend import chatbot

# Inside your chat response logic
with st.chat_message('assistant'):
    ai_message = st.write_stream(
        message_chunk.content
        for message_chunk, metadata in chatbot.stream(
            {'messages': [HumanMessage(content=user_input)]},
            config={'configurable': {'thread_id': 'thread-1'}},
            stream_mode='messages'
        )
    )

# Store the complete response for history
st.session_state['message_history'].append({
    'role': 'assistant',
    'content': ai_message
})
```

12.10 Complete Code Implementation

12.10.1 Backend: `langgraph_backend.py`

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated
from langchain_core.messages import BaseMessage
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph.message import add_messages
from dotenv import load_dotenv

# Load environment variables (for OPENAI_API_KEY)
load_dotenv()

# Initialize the LLM
llm = ChatOpenAI()

# Define the State Structure
class ChatState(TypedDict):
    """
    State definition for the chatbot.
    messages: List of conversation messages with add_messages reducer
    """
    messages: Annotated[list[BaseMessage], add_messages]
```

```

def chat_node(state: ChatState):
    """
    Main chat node that processes messages and generates responses.

    Args:
        state: Current state containing message history

    Returns:
        Updated state with LLM response added to messages
    """
    messages = state['messages']
    response = llm.invoke(messages)
    return {"messages": [response]}

# Initialize Checkpointer for Memory
checkpointer = InMemorySaver()

# Build the Graph
graph = StateGraph(ChatState)
graph.add_node("chat_node", chat_node)
graph.add_edge(START, "chat_node")
graph.add_edge("chat_node", END)

# Compile with Checkpointer
chatbot = graph.compile(checkpointer=checkpointer)

```

12.10.2 Frontend: stream_frontend_streaming.py

```

import streamlit as st
from langgraph_backend import chatbot
from langchain_core.messages import HumanMessage

# Configuration for thread-based memory
CONFIG = {'configurable': {'thread_id': 'thread-1'}}

# Initialize session state for message history
if 'message_history' not in st.session_state:
    st.session_state['message_history'] = []

# Display existing conversation history
for message in st.session_state['message_history']:
    with st.chat_message(message['role']):
        st.text(message['content'])

# Get user input
user_input = st.chat_input('Type here')

if user_input:
    # Add user message to history
    st.session_state['message_history'].append({
        'role': 'user',
        'content': user_input
    })

    # Display user message
    with st.chat_message('user'):
        st.text(user_input)

    # Generate and stream AI response
    with st.chat_message('assistant'):
        # st.write_stream handles the typewriter effect
        ai_message = st.write_stream(
            message_chunk.content
            for message_chunk, metadata in chatbot.stream(
                {'messages': [HumanMessage(content=user_input)]},
                config=CONFIG,
                stream_mode='messages'
            )
        )

```

```
    )

    # Store AI response in history
    st.session_state['message_history'].append({
        'role': 'assistant',
        'content': ai_message
    })
```

12.10.3 Architecture Overview

12.11 Key Takeaways

12.11.1 Summary Points

- 1. **Streaming Definition:** Tokens are sent as they’re generated, not all at once
- 2. **Six Benefits of Streaming:**
 - Faster perceived response time
 - Human-like conversation feel
 - Essential for voice/multi-modal interfaces
 - Better UX for long outputs
 - Ability to cancel and save costs
 - Show real-time updates for agents
- 3. **Implementation is Simple:**
 - Change `.invoke()` to `.stream()`
 - Add `stream_mode='messages'`
 - Loop through the generator
- 4. **Streamlit Integration:**
 - Use `st.write_stream()` for automatic typewriter effect
 - It returns the complete text for storage
- 5. **Generator Objects** are the key mechanism enabling streaming

12.11.2 Quick Reference Card

Table 12.6. Quick Reference Card

Task	Code
Enable streaming	<code>chatbot.stream(..., stream_mode='messages')</code>
Loop through stream	<code>for chunk, meta in stream:</code>
Get token content	<code>message_chunk.content</code>
Streamlit streaming	<code>st.write_stream(generator)</code>

12.11.3 Common Mistakes to Avoid

Table 12.7. Common Mistakes to Avoid

Mistake	Correct Approach
Hardcoding user input in stream call	Use the variable <code>user_input</code>
Forgetting <code>stream_mode='messages'</code>	Always specify for LLM token streaming

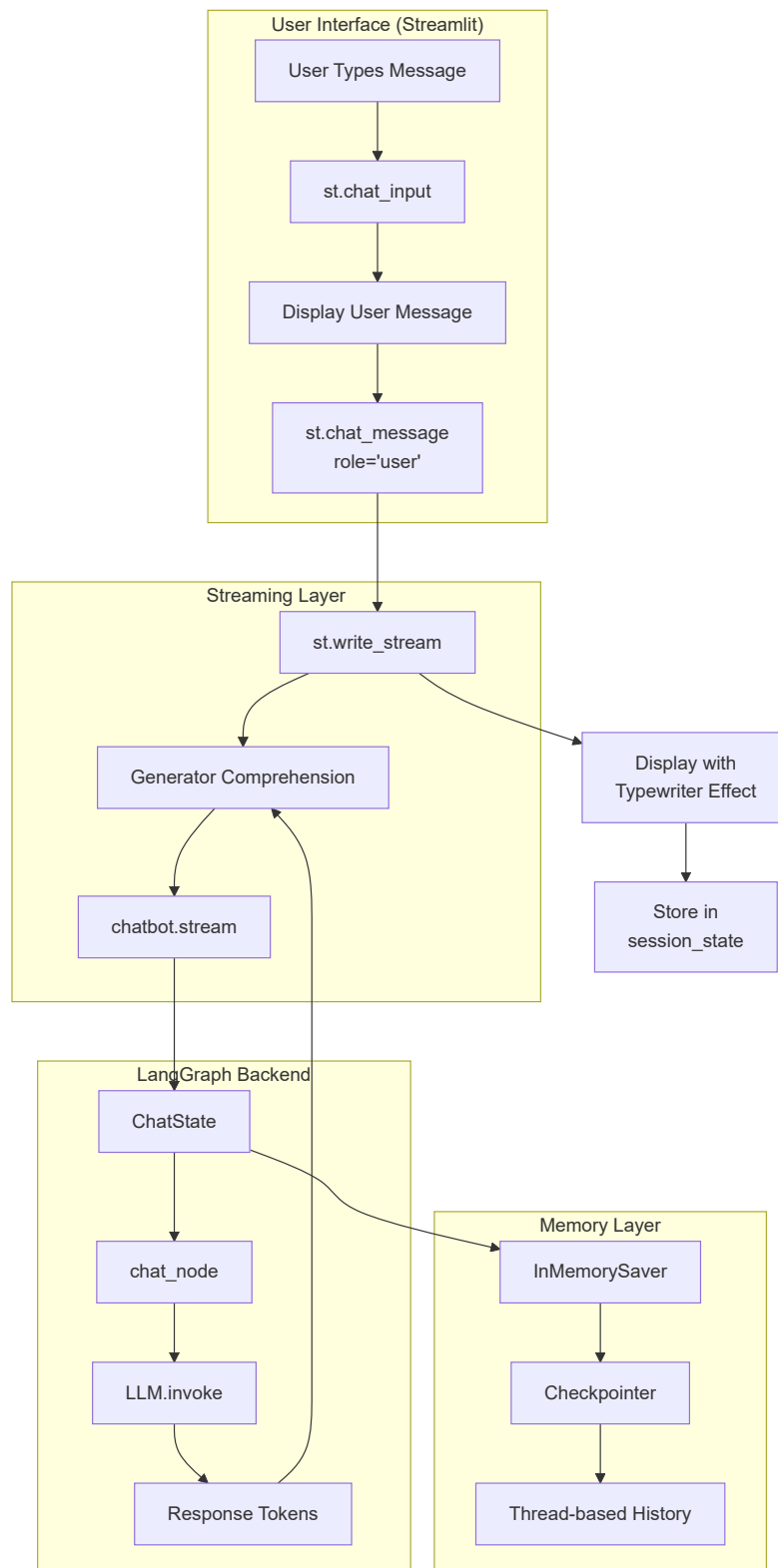
Mistake	Correct Approach
Not storing streamed response	Capture return value of <code>st.write_stream()</code>
Using <code>.invoke()</code> for streaming	Use <code>.stream()</code> method

12.12 Further Learning

To implement streaming effectively, ensure you understand:

- Python Generators (yield keyword, iterators)
- LangGraph State Management
- Streamlit Session State
- Async programming concepts (for advanced implementations)

Streaming is a **small change in code** but provides a **10x improvement in user experience** for LLM-based applications.

**Figure 12.6.** *Architecture Overview*

Streaming in LLM Applications using LangGraph

Token-by-Token Delivery • Typewriter Effect • Better UX

What is Streaming?

LLM sends tokens as they're generated, instead of waiting for the entire response to be ready.

❌ Without Streaming

- Wait 5-10 seconds
- Blank screen anxiety
- Everything appears at once
- User thinks app is frozen

✅ With Streaming

- Instant response start
- Typewriter effect
- User stays engaged
- Like ChatGPT experience

How Streaming Works

```

graph LR
    User((User)) -- Token --> App[App]
    App -- Token --> LLM[LLM]
    LLM -- Token --> Gen[Generator]
    Gen -- yield token1 --> Out1[ ]
    Gen -- yield token2 --> Out2[ ]
    Gen -- yield token3 --> Out3[ ]
  
```

Python Generators (The Key)

```

# Regular Function
def get_nums():
    return [1,2,3,4,5]
# Returns All at once
# Stores in memory
❌ Not for streaming

```

```

# Generator Function
def get_nums():
    yield 1
    yield 2
    yield 3...
# One at a time!
✅ Perfect for streaming

```

6 Key Benefits of Streaming

- 1 Faster Response**
Response starts immediately
Reduces user drop-off
No blank screen anxiety
10x better perceived speed
- 2 Human-Like Feel**
Mimics natural typing
Builds user trust
Feels "alive" and intelligent
ChatGPT-like experience
- 3 Voice/Multi-Modal**
Essential for voice UIs
No awkward silence gaps
Natural conversation flow
Alexa, Google Assistant
- 4 Long Output UX**
Code appears line by line
Easy to follow logic
Better comprehension
Code, docs, essays
- 5 Cancel = Save \$**
Stop generation midway
Fewer tokens = lower cost
LLMs charge per token
Cost optimization
- 6 Agent Updates**
Show step-by-step progress
"Searching...", "Found..."
Keep user informed
AI Agents, multi-step tasks

LangGraph Implementation: .invoke() → .stream()

❌ Old Way (No Streaming)

```

# Waits for complete response
response = graph.invoke(state)

```

✅ New Way (With Streaming)

```

# Returns generator immediately
stream = graph.stream(state,

```

```

# Complete streaming pattern
for message_chunk, metadata in chatbot.stream(state, config, stream_mode="messages"):
    if message_chunk.content:
        print(message_chunk.content, end="")

```

Stream Modes in LangGraph

'messages'	'values'	'updates'	'custom'
LLM tokens as generated Best for: Chatbots ★ Most Common	Full state after each node Best for: Debugging	Only state changes Best for: Efficiency	Custom logic Best for: AI Agents

Streamlit Integration (Frontend)

```

# One line handles typewriter effect!
ai_message = st.write_stream(generator_object) # Returns complete text

```

Complete Architecture: Backend + Frontend

Backend: langgraph_backend.py

```

from langgraph.graph import StateGraph
from langgraph.checkpoint.memory import InMemorySaver

# Define state, nodes, edges...
graph = StateGraph(ChatState)
graph.add_node("chat_node", chat_node)

# Compile with checkpoint
checkpointeer = InMemorySaver()
chatbot = graph.compile(checkpointer=checkpointer)

```

Frontend: streamlit_app.py

```

import streamlit as st
from langgraph_backend import chatbot

user_input = st.chat_input("Type here")

with st.chat_message("assistant"):
    ai_msg = st.write_stream(
        chatbot.stream(state, config, stream_mode="messages")
    )

```

Quick Reference

Key Code Changes:

- Enable streaming: `.stream()` instead of `.invoke()`
- Stream mode: `stream_mode="messages"`
- Get content: `message_chunk.content`
- Streamlit: `st.write_stream(generator)`

Common Mistakes:

- Forgetting `stream_mode="messages"`
- Using `.invoke()` instead of `.stream()`

LangGraph Streaming • Small code change = 10x better UX • Token-by-token delivery

Figure 12.7. Further Learning

LangGraph Chatbot: Resume Chat Feature - Comprehensive Guide

13.1 Code

- <https://github.com/campusx-official/chatbot-in-langgraph>

13.2 Introduction

This guide covers the implementation of a **Resume Chat** feature in a chatbot built using **LangGraph** (backend) and **Streamlit** (frontend). The Resume Chat feature allows users to:

- Start new conversations on different topics
- Switch between multiple ongoing conversations
- Resume previous conversations with full message history intact
- Maintain conversation context across different chat threads

This is similar to how modern AI chatbots like ChatGPT handle multiple conversations - you can start a new chat or go back to any previous conversation at any time.

13.3 Feature Overview

13.3.1 What the Feature Enables

Before implementing this feature, the chatbot only supported a single conversation thread. After implementation, users can:

1. **Create multiple conversations** - Each with its own unique thread ID
2. **Switch between conversations** - Click on any previous conversation to load it
3. **Maintain separate contexts** - Each conversation remembers its own history
4. **Visual conversation management** - Sidebar displays all available conversations

13.3.2 User Experience Flow

When a user loads the chatbot:

1. A sidebar appears on the left with:
 - Application title ("LangGraph Chatbot")
 - "New Chat" button to start fresh conversations
 - "My Conversations" section listing all previous chats
2. Users can:
 - Type messages in the main chat area
 - Click "New Chat" to start a new conversation

- Click on any thread ID in the sidebar to resume that conversation

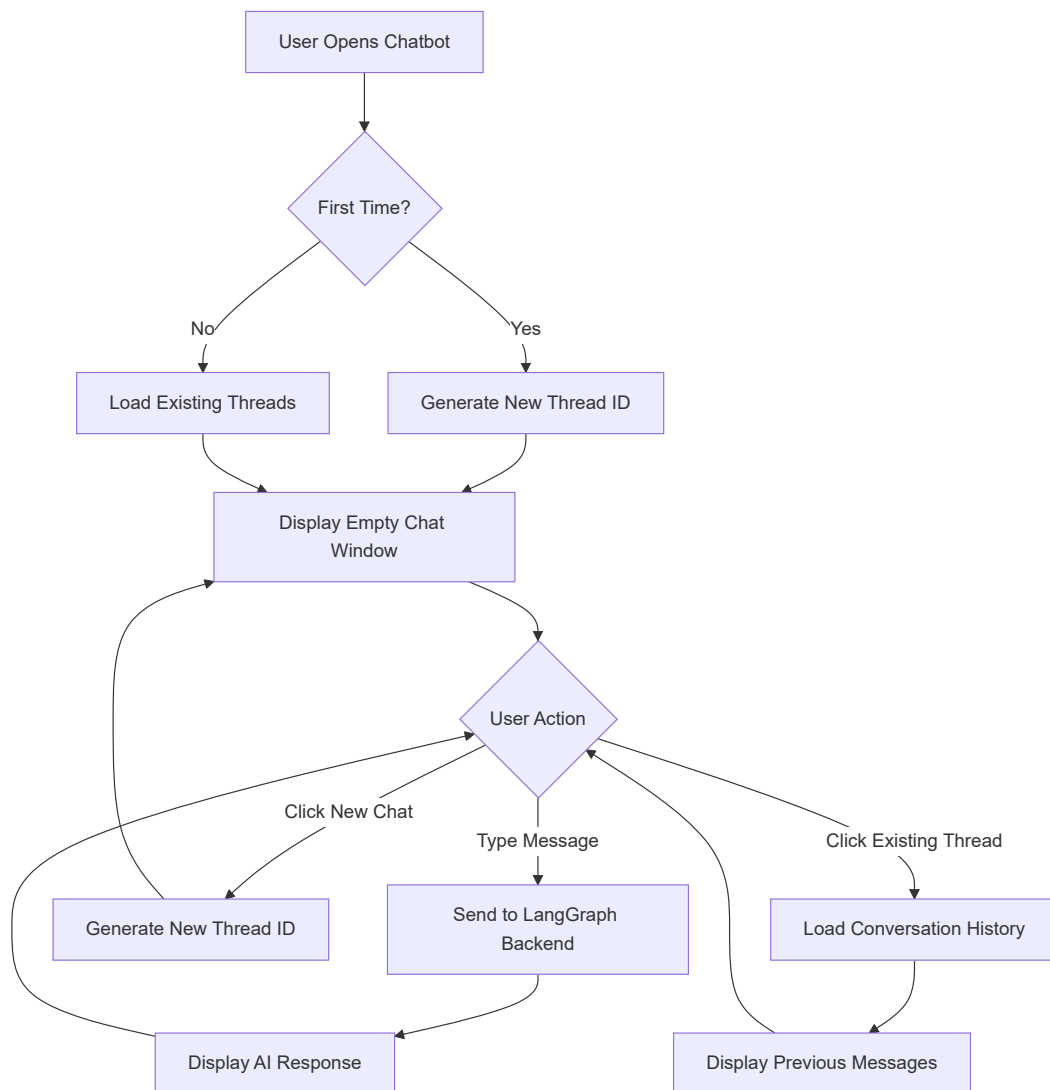


Figure 13.1. User Experience Flow

13.4 Architecture Understanding

13.4.1 System Components

The chatbot system consists of two main parts:

Backend (LangGraph):

- Handles AI conversation logic
- Manages state persistence using checkpoints
- Stores messages associated with thread IDs
- Processes user inputs and generates AI responses

Frontend (Streamlit):

- Provides user interface
- Manages session state for UI elements
- Handles user interactions (clicks, inputs)

- Displays conversation history

13.4.2 Component Interaction

13.4.3 Data Flow Overview

Table 13.1. *Data Flow Overview*

Component	Responsibility	Data Stored
Streamlit Session State	UI state management	message_history, thread_id, chat_threads
LangGraph Checkpointer	Conversation persistence	All messages per thread ID
UUID Library	ID generation	Unique thread identifiers

13.5 Implementation Roadmap

The implementation is broken down into **4 task sets**, following the principle of dividing large features into smaller, manageable tasks.

13.6 Detailed Implementation

13.6.1 Task Set 1: Basic UI Setup

Objective: Add a sidebar with title, “New Chat” button, and “My Conversations” section.

Understanding Streamlit Sidebar

Streamlit provides a built-in sidebar component. Any UI element prefixed with `st.sidebar.` will appear in the left sidebar instead of the main content area.

Code Implementation

```
# Sidebar UI Section
st.sidebar.title('LangGraph Chatbot')

if st.sidebar.button('New Chat'):
    reset_chat() # Function to be implemented later

st.sidebar.header('My Conversations')
```

Explanation:

- `st.sidebar.title('LangGraph Chatbot')` - Creates a large title in the sidebar
- `st.sidebar.button('New Chat')` - Creates a clickable button; returns True when clicked
- `st.sidebar.header('My Conversations')` - Creates a header text below the button

The if statement around the button ensures the `reset_chat()` function only runs when the button is actually clicked.

13.6.2 Task Set 2: New Chat Functionality

Objective: Enable creating new conversations with unique identifiers.

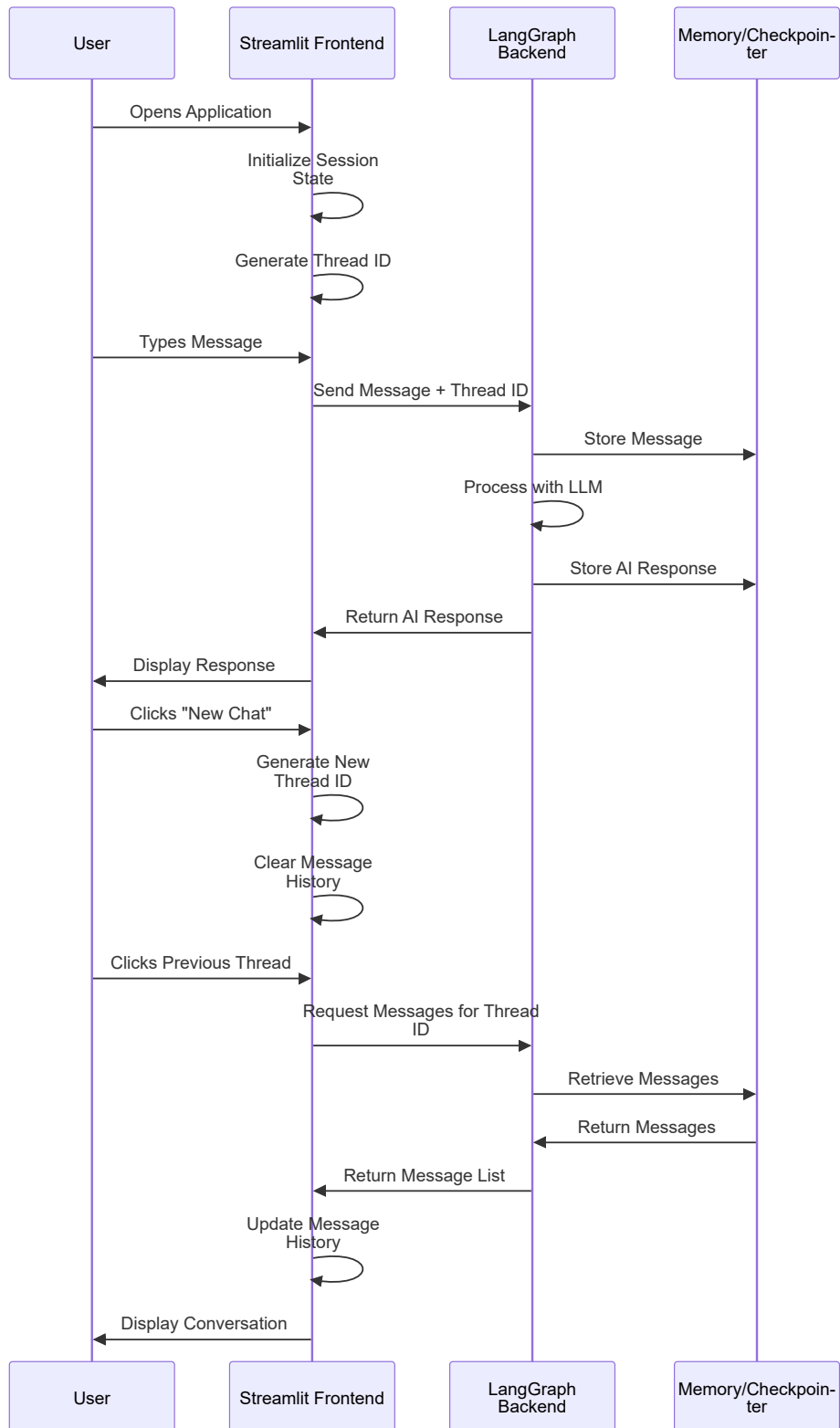


Figure 13.2. *Component Interaction*

Why Dynamic Thread IDs?

In the original implementation, thread ID was hardcoded:

```
# Old approach - problematic
CONFIG = {'configurable': {'thread_id': 'thread-1'}}
```

Problem: Users should be able to create unlimited new conversations. Hardcoded IDs don't allow this.

Solution: Generate unique thread IDs programmatically using Python's uuid library.

Understanding UUID

UUID (Universally Unique Identifier) generates unique identifiers. UUID4 specifically creates random UUIDs.

```
import uuid

# Example: Generate a UUID
print(uuid.uuid4()) # Output: something like 'f47ac10b-58cc-4372-a567-0e02b2c3d479'
```

Each call to `uuid.uuid4()` generates a different ID, making it perfect for unique conversation identifiers.

Code Implementation

Utility Function for Thread ID Generation:

```
import uuid

def generate_thread_id():
    """Generate a unique thread ID using UUID4."""
    thread_id = uuid.uuid4()
    return thread_id
```

Session Setup with Dynamic Thread ID:

```
# Session Setup Section
if 'message_history' not in st.session_state:
    st.session_state['message_history'] = []

if 'thread_id' not in st.session_state:
    st.session_state['thread_id'] = generate_thread_id()
```

Explanation of Session Setup Logic:

The `if 'key' not in st.session_state` pattern is crucial in Streamlit:

- Streamlit reruns the entire script on every user interaction
- Without this check, variables would reset on each rerun
- Session state persists data across reruns

Reset Chat Function:

```
def reset_chat():
    """Reset the chat by creating a new thread and clearing history."""
    # Step 1: Generate new thread ID
    thread_id = generate_thread_id()

    # Step 2: Update session with new thread ID
    st.session_state['thread_id'] = thread_id

    # Step 3: Add new thread to the threads list
    add_thread(st.session_state['thread_id'])
```

```
# Step 4: Clear message history for fresh UI
st.session_state['message_history'] = []
```

What happens when “New Chat” is clicked:

1. A new unique thread ID is generated
2. This ID replaces the current thread ID in session state
3. The new thread is added to the list of all threads
4. Message history is cleared (empty list)
5. User sees a fresh, empty chat window

13.6.3 Task Set 3: Thread Management

Objective: Store all thread IDs and display them in the sidebar as clickable buttons.

Why Store All Thread IDs?

When using only a single `thread_id` in session state:

- Creating a new chat replaces the old thread ID
- Previous conversation’s thread ID is lost
- User cannot access previous conversations

Solution: Maintain a list (`chat_threads`) that stores ALL thread IDs ever created.

Code Implementation

Session Setup for Thread List:

```
if 'chat_threads' not in st.session_state:
    st.session_state['chat_threads'] = []
```

Utility Function to Add Threads:

```
def add_thread(thread_id):
    """Add a thread ID to the chat_threads list if not already present."""
    if thread_id not in st.session_state['chat_threads']:
        st.session_state['chat_threads'].append(thread_id)
```

The duplicate check (`if thread_id not in`) prevents the same thread from being added multiple times, which could happen during page reruns.

When to Add Threads:

Thread addition happens in **two scenarios**:

1. **Initial page load** - When the first thread ID is generated
2. **New chat creation** - When user clicks “New Chat”

```
# After session setup, add current thread to list
add_thread(st.session_state['thread_id'])

# Inside reset_chat function
def reset_chat():
    thread_id = generate_thread_id()
    st.session_state['thread_id'] = thread_id
    add_thread(st.session_state['thread_id']) # Add to list
    st.session_state['message_history'] = []
```

Displaying Threads in Sidebar:

```
# Sidebar UI - Display all threads as buttons
for thread_id in st.session_state['chat_threads'][::-1]: # [::-1] reverses the list
    if st.sidebar.button(str(thread_id)):
        # Handle click - implemented in Task Set 4
        pass
```

Why Reverse the List ([::-1])?

- New conversations are appended to the end of the list
- Without reversal: oldest conversation appears at top
- With reversal: newest conversation appears at top (better UX, matches ChatGPT behavior)

13.6.4 Task Set 4: Resume Conversation

Objective: Load a previous conversation when its thread button is clicked.

How LangGraph Stores Conversations

LangGraph uses a **checkpoint** to persist conversation state. The checkpoint stores:

- All messages in a conversation
- Associated metadata
- State snapshots

Each conversation is identified by its **thread ID**. Given a thread ID, you can retrieve all associated messages.

Retrieving Messages from LangGraph

```
def load_conversation(thread_id):
    """Load all messages for a given thread ID from LangGraph."""
    state = chatbot.get_state(config={'configurable': {'thread_id': thread_id}})
    return state.values.get('messages', [])
```

Understanding the Response Structure:

When you call `chatbot.get_state()`:

```
# Response structure
state = StateSnapshot(
    values={
        'messages': [
            HumanMessage(content='Hi my name is Nitesh'),
            AIMessage(content='Hello Nitesh! How can I help you?')
        ],
        # ... other metadata
    )
```

- `state.values` - Dictionary containing all state keys
- `state.values['messages']` - List of message objects
- Each message is either `HumanMessage` or `AIMessage` type

Message Format Conversion Challenge

The Problem:

LangGraph returns messages in this format:

```
# LangGraph format
[
```

```
HumanMessage(content='Hi'),
AIMessage(content='Hello!')
]
```

But Streamlit's message history uses this format:

```
# Streamlit format
[
    {'role': 'user', 'content': 'Hi'},
    {'role': 'assistant', 'content': 'Hello!'}
]
```

Solution: Convert between formats when loading conversations.

Complete Click Handler Implementation

```
for thread_id in st.session_state['chat_threads'][::-1]:
    if st.sidebar.button(str(thread_id)):
        # Step 1: Update current thread ID in session
        st.session_state['thread_id'] = thread_id

        # Step 2: Load messages from LangGraph
        messages = load_conversation(thread_id)

        # Step 3: Convert message format
        temp_messages = []

        for msg in messages:
            # Determine role based on message type
            if isinstance(msg, HumanMessage):
                role = 'user'
            else:
                role = 'assistant'

            # Create dictionary in Streamlit format
            temp_messages.append({
                'role': role,
                'content': msg.content
            })

        # Step 4: Update session state
        st.session_state['message_history'] = temp_messages
```

Step-by-Step Explanation:

Table 13.2. Complete Click Handler Implementation

Step	Action	Purpose
1	Update thread_id	Ensure subsequent messages go to correct thread
2	Call load_conversation()	Retrieve messages from LangGraph backend
3	Loop through messages	Convert each message to Streamlit format
4	Update message_history	Trigger UI update with loaded messages

The isinstance() Function:

```
from langchain_core.messages import HumanMessage, AIMessage

msg = HumanMessage(content='Hello')

isinstance(msg, HumanMessage) # Returns: True
isinstance(msg, AIMessage)    # Returns: False
```

This function checks if an object is an instance of a particular class, allowing us to determine message types.

13.7 Code Walkthrough

13.7.1 Complete Frontend Code Structure

13.7.2 Backend Code (langgraph_backend.py)

```

from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated
from langchain_core.messages import BaseMessage
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph.message import add_messages
from dotenv import load_dotenv

load_dotenv()

# Initialize the LLM
llm = ChatOpenAI()

# Define the state structure
class ChatState(TypedDict):
    messages: Annotated[List[BaseMessage], add_messages]

# Define the chat node function
def chat_node(state: ChatState):
    messages = state['messages']
    response = llm.invoke(messages)
    return {"messages": [response]}

# Create checkpoint for state persistence
checkpoint = InMemorySaver()

# Build the graph
graph = StateGraph(ChatState)
graph.add_node("chat_node", chat_node)
graph.add_edge(START, "chat_node")
graph.add_edge("chat_node", END)

# Compile with checkpoint
chatbot = graph.compile(checkpointer=checkpoint)

```

Key Backend Components:

Table 13.3. Backend Code (langgraph_backend.py)

Component	Purpose
ChatState	Defines the structure of conversation state
chat_node	Processes messages through the LLM
InMemorySaver	Stores conversation history in RAM
chatbot	Compiled graph ready for use

13.7.3 Complete Frontend Code (streamlit_frontend_threading.py)

```

import streamlit as st
from langgraph_backend import chatbot
from langchain_core.messages import HumanMessage, AIMessage
import uuid

# ===== UTILITY FUNCTIONS =====

def generate_thread_id():
    """Generate a unique thread identifier."""
    thread_id = uuid.uuid4()
    return thread_id

```

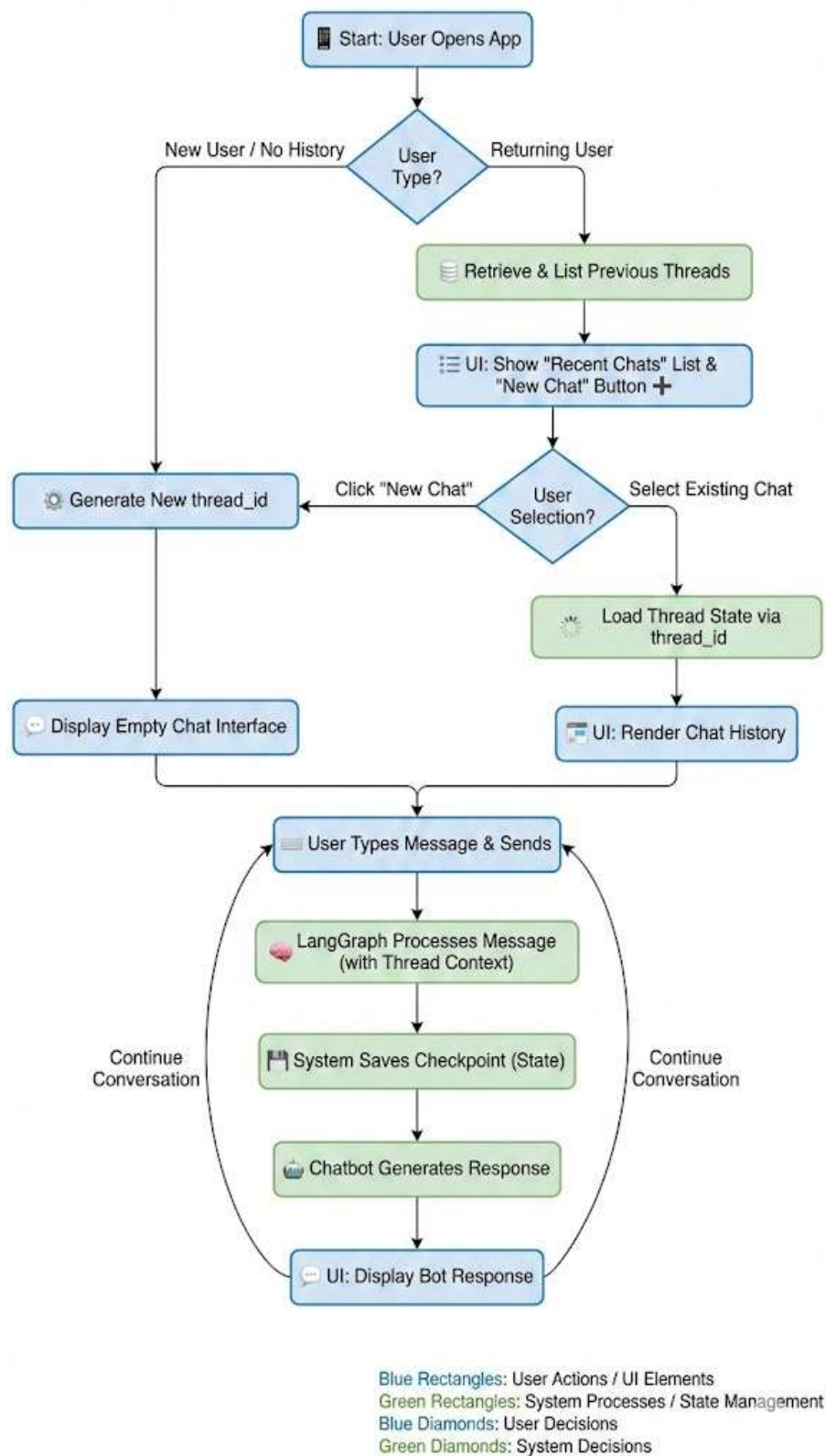


Figure 13.3. Complete Frontend Code Structure

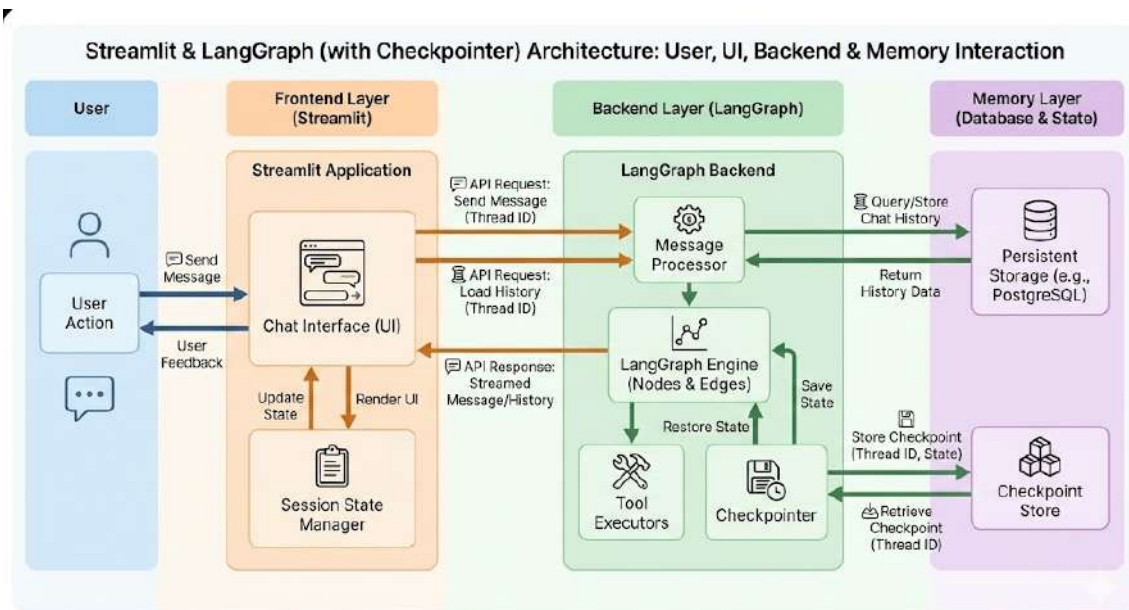


Figure 13.4. Complete Frontend Code Structure

```
def reset_chat():
    """Reset chat: new thread ID, clear history, add to thread list."""
    thread_id = generate_thread_id()
    st.session_state['thread_id'] = thread_id
    add_thread(st.session_state['thread_id'])
    st.session_state['message_history'] = []

def add_thread(thread_id):
    """Add thread to the threads list if not already present."""
    if thread_id not in st.session_state['chat_threads']:
        st.session_state['chat_threads'].append(thread_id)

def load_conversation(thread_id):
    """Load messages for a thread from LangGraph backend."""
    state = chatbot.get_state(config={'configurable': {'thread_id': thread_id}})
    return state.values.get('messages', [])

# ===== SESSION SETUP =====

if 'message_history' not in st.session_state:
    st.session_state['message_history'] = []

if 'thread_id' not in st.session_state:
    st.session_state['thread_id'] = generate_thread_id()

if 'chat_threads' not in st.session_state:
    st.session_state['chat_threads'] = []

add_thread(st.session_state['thread_id'])

# ===== SIDEBAR UI =====

st.sidebar.title('LangGraph Chatbot')

if st.sidebar.button('New Chat'):
    reset_chat()

st.sidebar.header('My Conversations')

for thread_id in st.session_state['chat_threads'][::-1]:
    if st.sidebar.button(str(thread_id)):
        st.session_state['thread_id'] = thread_id
        messages = load_conversation(thread_id)
```

```

temp_messages = []
for msg in messages:
    if isinstance(msg, HumanMessage):
        role = 'user'
    else:
        role = 'assistant'
    temp_messages.append({'role': role, 'content': msg.content})

st.session_state['message_history'] = temp_messages

# ===== MAIN UI =====

# Display conversation history
for message in st.session_state['message_history']:
    with st.chat_message(message['role']):
        st.text(message['content'])

# Chat input
user_input = st.chat_input('Type here')

if user_input:
    # Add user message to history and display
    st.session_state['message_history'].append({
        'role': 'user',
        'content': user_input
    })
    with st.chat_message('user'):
        st.text(user_input)

    # Configure with current thread ID
    CONFIG = {'configurable': {'thread_id': st.session_state['thread_id']}}

    # Stream AI response
    with st.chat_message("assistant"):
        def ai_only_stream():
            for message_chunk, metadata in chatbot.stream(
                {"messages": [HumanMessage(content=user_input)]},
                config=CONFIG,
                stream_mode="messages"
            ):
                if isinstance(message_chunk, AIMessage):
                    yield message_chunk.content

        ai_message = st.write_stream(ai_only_stream())

    # Add AI response to history
    st.session_state['message_history'].append({
        'role': 'assistant',
        'content': ai_message
    })

```

13.8 Key Concepts Explained

13.8.1 Session State in Streamlit

Streamlit reruns the entire Python script on every user interaction (button click, input, etc.). Without session state, all variables would reset.

Session State Behavior:

Why This Matters for Our Feature:

Without session state:

- Thread ID would regenerate on every click
- Message history would be lost on every interaction
- Thread list would reset continuously

With session state:

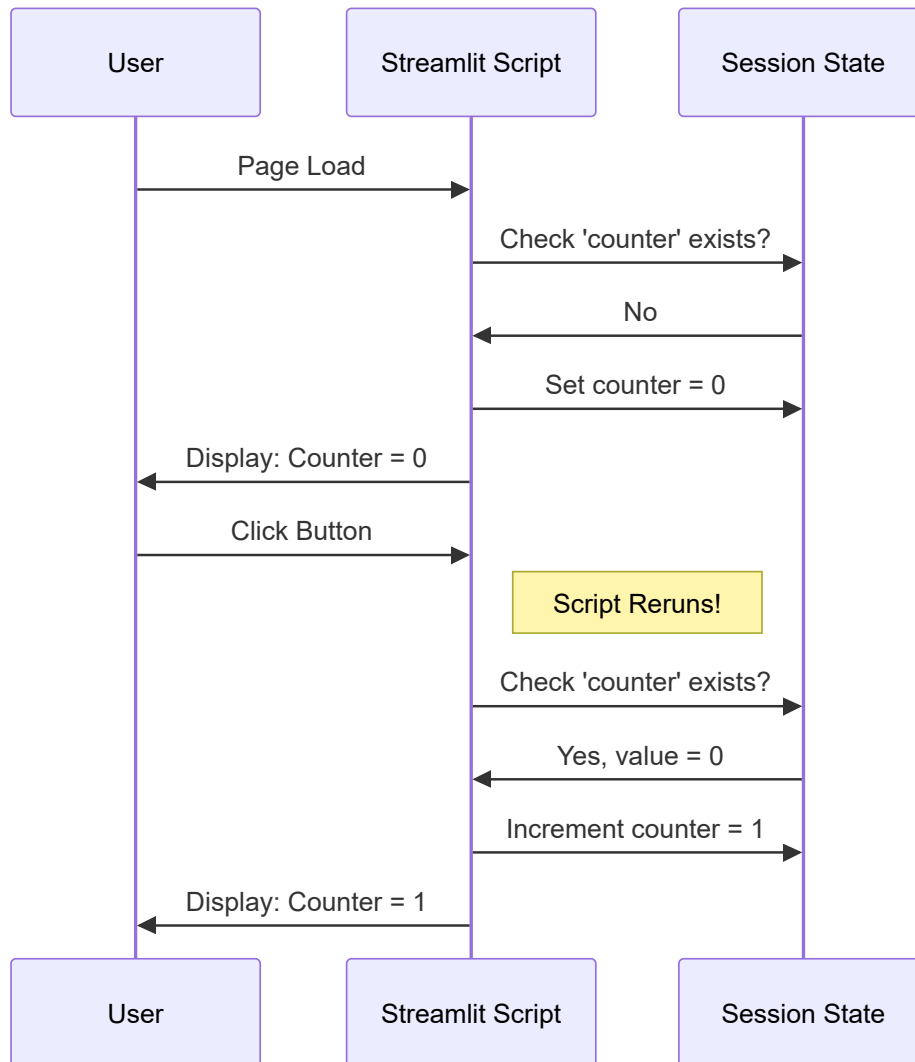


Figure 13.5. *Session State in Streamlit*

- Variables persist across reruns
- UI state is maintained
- Conversations are preserved during the session

13.8.2 Thread ID and Configuration

LangGraph uses configuration dictionaries to identify conversations:

```

config = {
  'configurable': {
    'thread_id': 'unique-identifier-here'
  }
}
  
```

How Thread ID Works:

1. When you send a message with a thread ID, LangGraph:
 - Checks if messages exist for that thread
 - Loads existing messages into context
 - Processes the new message

- Saves the updated conversation
2. The checkpoint maps: thread_id $\rightarrow$ [list of messages]
 3. Same thread ID = same conversation context = AI remembers previous messages

13.8.3 The Checkpointer (InMemorySaver)

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
chatbot = graph.compile(checkpointer=checkpointer)
```

What InMemorySaver Does:

- Stores conversation states in RAM
- Allows retrieval of conversation history
- Enables conversation resumption
- Persists data only while the program runs

Limitation:

Data is lost when the program terminates or restarts. For production applications, database-backed checkpointers are needed.

13.8.4 Message Types in LangChain

LangChain uses specific message classes:

Table 13.4. Message Types in LangChain

Class	Purpose	Example
HumanMessage	User's input	HumanMessage(content='Hello')
AIMessage	AI's response	AIMessage(content='Hi there!')
SystemMessage	System instructions	SystemMessage(content='You are helpful')
BaseMessage	Base class for all	Parent class

Checking Message Type:

```
from langchain_core.messages import HumanMessage, AIMessage

message = HumanMessage(content='Test')

# Using isinstance
if isinstance(message, HumanMessage):
    print("This is from the user")
elif isinstance(message, AIMessage):
    print("This is from the AI")
```

13.9 Current Limitations and Future Improvements

13.9.1 Current Limitations

1. **Data Persistence:**
 - Using InMemorySaver means all conversations are lost on refresh
 - Thread list (chat_threads) only exists in session state
 - No cross-session persistence

2. Thread Display:

- Threads shown as UUID strings (not user-friendly)
- No conversation titles or summaries
- No way to delete old conversations

3. UI Issues:

- Some formatting bugs with streaming
- No loading indicators
- No confirmation dialogs

13.9.2 Planned Improvements (Next Video)

Benefits of Database Checkpointer:

- Conversations persist across refreshes
- Conversations survive program restarts
- Cross-device conversation access (with proper auth)
- Scalable for production use

13.9.3 Homework Challenge

Task: Implement friendly conversation names instead of UUID thread IDs.

Hints:

1. Could use the first few words of the first message
2. Could generate a summary using the LLM
3. Could let users name their conversations
4. Could show timestamp-based names

13.10 Summary

The Resume Chat feature transforms a simple single-conversation chatbot into a multi-conversation application similar to ChatGPT. The implementation involves:

1. **Dynamic Thread ID Generation** - Using UUID for unique conversation identifiers
2. **Session State Management** - Persisting UI state across Streamlit reruns
3. **Thread List Maintenance** - Storing all conversation IDs for sidebar display
4. **Conversation Loading** - Retrieving and converting messages from LangGraph
5. **Format Conversion** - Translating between LangGraph and Streamlit message formats

The key insight is that **no backend changes were required** - the LangGraph backend already supports multiple threads through its checkpointing system. All changes were made in the Streamlit frontend to:

- Generate and manage thread IDs
- Maintain a list of all conversations
- Provide UI for switching between conversations
- Load and display historical messages

This modular approach demonstrates good software design - separating concerns between frontend and backend makes feature additions simpler and more maintainable.

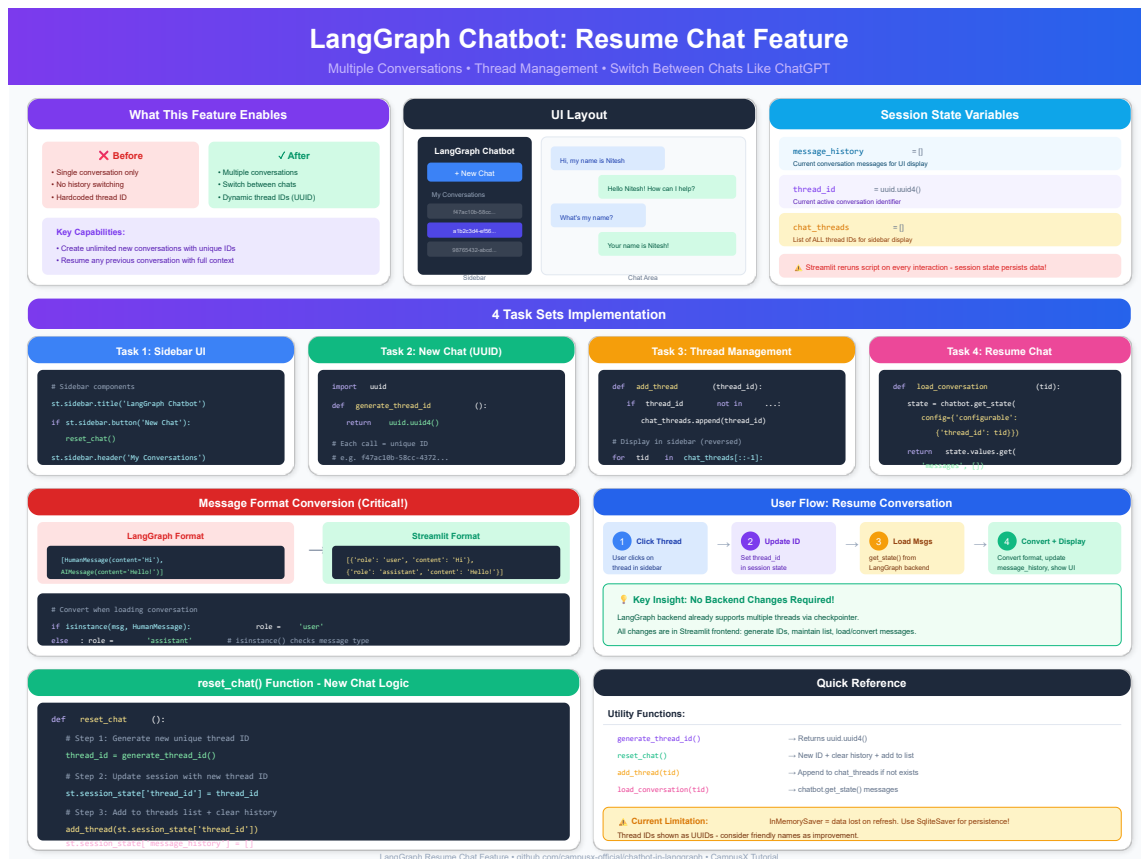


Figure 13.6. Summary

LangGraph Persistent Storage with SQLite Database

14.1 Code

- <https://github.com/campusx-official/chatbot-in-langgraph>

14.2 1. Introduction and Problem Statement

14.2.1 The Evolution of the Chatbot

This tutorial is part of a series on building Agentic AI using LangGraph. The chatbot has evolved through several iterations:

Table 14.1. *The Evolution of the Chatbot*

Version	Feature Added	Description
V1	Basic Chatbot	Simple LangGraph workflow with console interaction
V2	GUI Interface	Added Streamlit frontend for better user experience
V3	Streaming	Token-by-token response display for better UX
V4	Thread Support	Multiple conversation threads with history
V5	Persistent Storage	Database integration for permanent storage

14.2.2 The Core Problem

When using **InMemorySaver** (RAM-based checkpointer), all conversation data is lost when:

- The application is closed
- The page is refreshed/reloaded
- The server restarts
- The system crashes

This happens because **InMemorySaver** stores all conversation history in **RAM (Random Access Memory)**, which is volatile storage.

14.2.3 The Solution

Replace **InMemorySaver** with **SqliteSaver** to achieve:

- Permanent storage of conversations
- Resume conversations after application restart
- Data persistence across sessions
- Historical conversation access days/weeks later

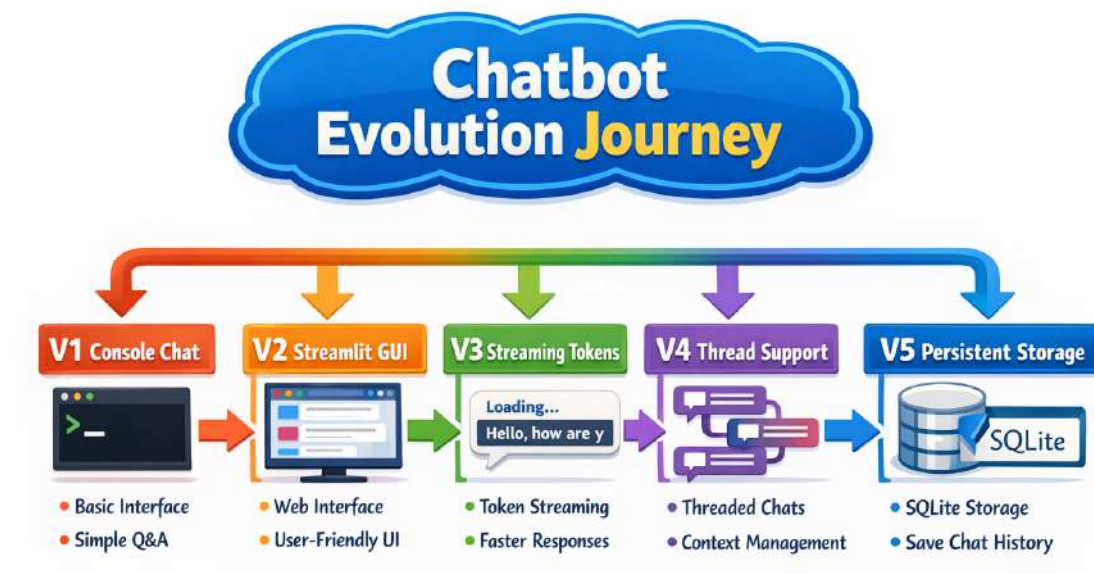


Figure 14.1. The Evolution of the Chatbot

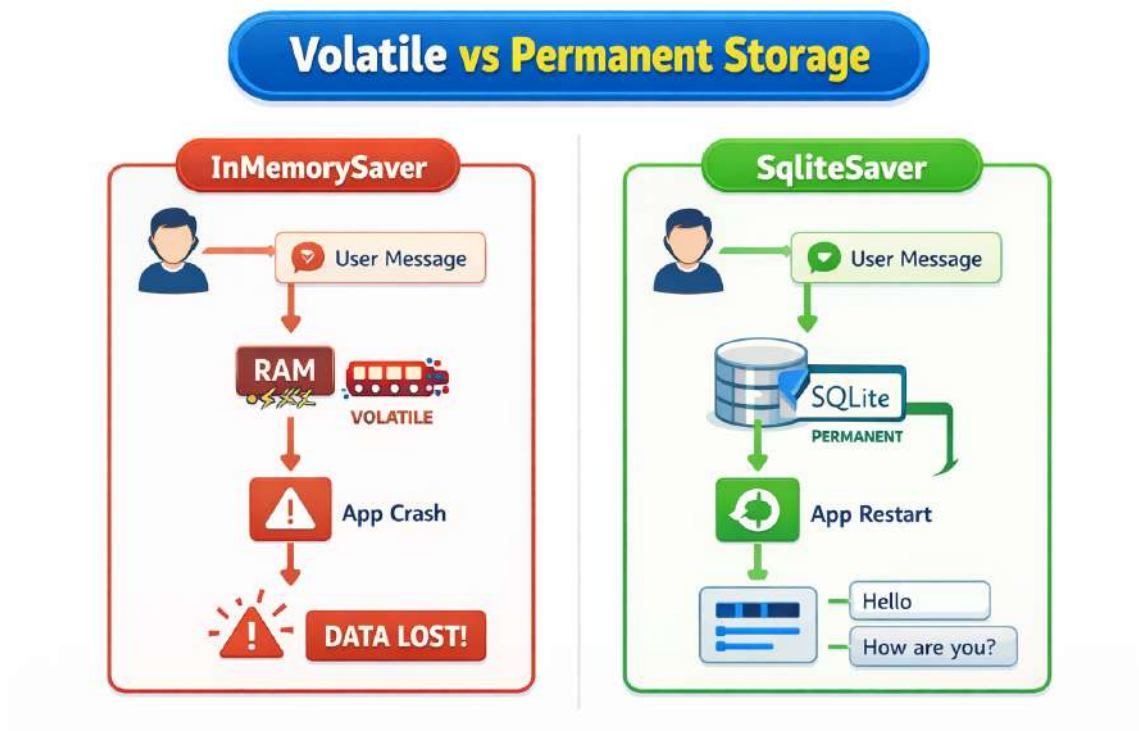


Figure 14.2. The Core Problem

14.3 2. Understanding Checkpointers in LangGraph

14.3.1 What is a Checkpointer?

A **checkpoint** in LangGraph is a component that saves the state of your workflow at specific points during execution. Think of it as an “auto-save” feature for your AI application’s state.

14.3.2 Types of Checkpointers Available

LangGraph provides three main types of checkpointers:

Table 14.2. *Types of Checkpointers Available*

Checkpoint	Storage Type	Use Case	Persistence
InMemorySaver	RAM	Development, Testing	✗ Volatile
SqliteSaver	SQLite Database	Prototyping, Learning	✓ Persistent
PostgresSaver	PostgreSQL Database	Production	✓ Persistent + Scalable

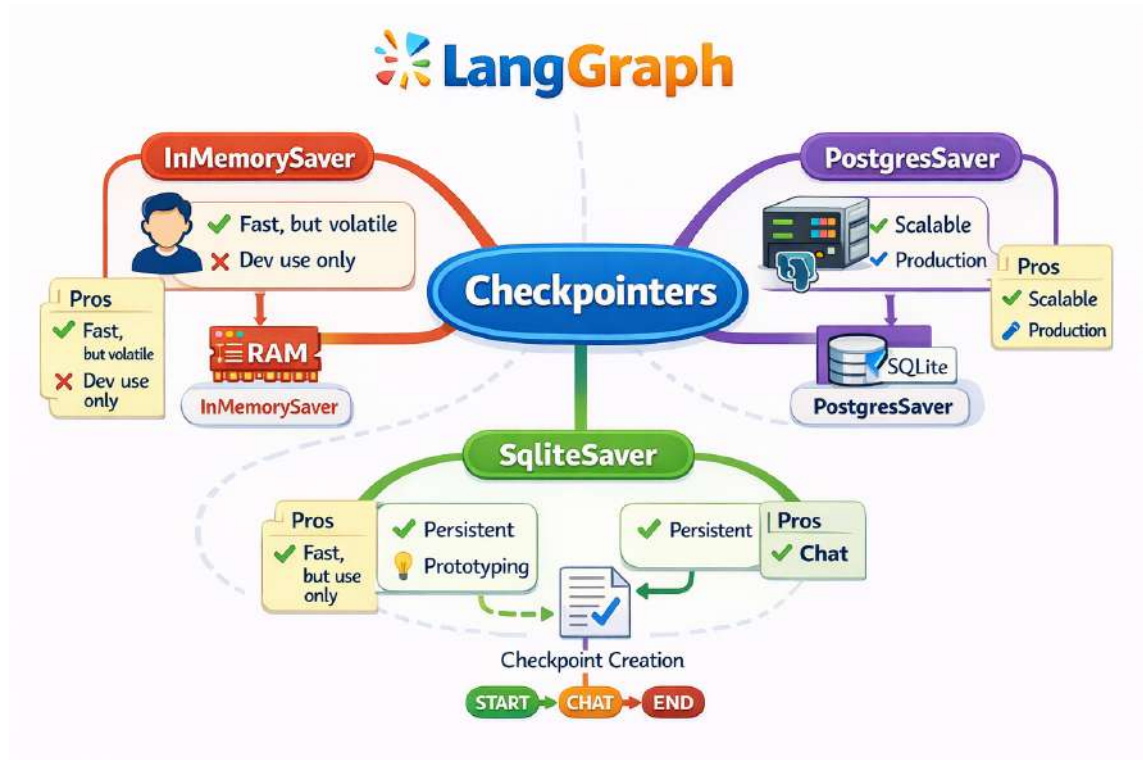


Figure 14.3. *Types of Checkpointers Available*

14.3.3 When Checkpoints are Created

In a typical chatbot workflow, checkpoints are created at three key moments:

1. **START Node:** When execution begins
2. **Chat Node:** After processing the message
3. **END Node:** When execution completes

For a single message exchange, **3 checkpoints** are created. So if you have 6 message exchanges, you’ll have **18 checkpoints** (6×3).

14.4 3. SQLite Saver Implementation

14.4.1 Installation

The SqliteSaver is not built into LangGraph core. You need to install an external library:

```
pip install langgraph-checkpoint-sqlite
```

You can find the package at: <https://pypi.org/project/langgraph-checkpoint-sqlite/>

14.4.2 Import Changes

Before (InMemorySaver):

```
from langgraph.checkpoint.memory import InMemorySaver
```

After (SqliteSaver):

```
from langgraph.checkpoint.sqlite import SqliteSaver
import sqlite3
```

14.4.3 Understanding SQLite

SQLite is a lightweight, file-based relational database that:

- Requires no separate server installation
- Stores data in a single .db file
- Is perfect for prototyping and learning
- Has Python built-in support via sqlite3 module

14.5 4. Backend Code Implementation

14.5.1 Step-by-Step Implementation Plan

Based on the tutorial, here's the implementation roadmap:

```
-> create new frontend and backend files
-> install https://pypi.org/project/langgraph-checkpoint-sqlite/
-> implement database in backend
-> chat in multiple threads
-> install and visualize
-> integrate to frontend
```

14.5.2 Complete Backend Code with SQLite

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated
from langchain_core.messages import BaseMessage, HumanMessage
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.graph.message import add_messages
from dotenv import load_dotenv
import sqlite3

# Load environment variables (API keys)
load_dotenv()
```

```

# Initialize the Language Model
llm = ChatOpenAI()

# Define the state structure for our chatbot
class ChatState(TypedDict):
    messages: Annotated[List[BaseMessage], add_messages]

# Define the chat node function
def chat_node(state: ChatState):
    messages = state['messages']
    response = llm.invoke(messages)
    return {"messages": [response]}

# Create SQLite database connection
conn = sqlite3.connect(database='chatbot.db', check_same_thread=False)

# Create the SqliteSaver checkpointer
checkpointer = SqliteSaver(conn=conn)

# Build the graph
graph = StateGraph(ChatState)
graph.add_node("chat_node", chat_node)
graph.add_edge(START, "chat_node")
graph.add_edge("chat_node", END)

# Compile with checkpointer
chatbot = graph.compile(checkpointer=checkpointer)

# Function to retrieve all existing threads from database
def retrieve_all_threads():
    all_threads = set()
    for checkpoint in checkpointer.list(None):
        all_threads.add(checkpoint.config['configurable']['thread_id'])
    return list(all_threads)

```

14.5.3 Code Comparison: Before vs After

Table 14.3. Code Comparison: Before vs After

Component	InMemorySaver Version	SqliteSaver Version
Import	from langgraph.checkpoint.memory import InMemorySaver	from langgraph.checkpoint.sqlite import SqliteSaver + import sqlite3
Connection	Not needed	conn = sqlite3.connect(database='chatbot.db', check_same_thread=False)
Checkpointers	checkpointer = InMemorySaver()	checkpointer = SqliteSaver(conn=conn)
Persistence	✗ Lost on restart	✓ Survives restart

14.6 5. Database Connection Deep Dive

14.6.1 The sqlite3.connect() Function

```
conn = sqlite3.connect(database='chatbot.db', check_same_thread=False)
```

This single line does multiple things:

Parameter 1: database='chatbot.db'

- Specifies the database file name
- If the file doesn't exist, it will be **automatically created**

- The file is created in your **project directory**
- You can specify a full path like `database='/path/to/chatbot.db'`

Parameter 2: `check_same_thread=False`

This is a **critical parameter** that requires detailed explanation.

The Problem: By default, SQLite enforces that database connections can only be used in the **same thread** where they were created. This is a safety feature to prevent data corruption.

Why it causes issues: In a chatbot application with multiple conversation threads, Python may use different OS threads to handle different conversations. If SQLite checks thread identity, you'll get an error:

```
sqlite3.ProgrammingError: SQLite objects created in a thread can only be used in that same thread.
```

The Solution: Setting `check_same_thread=False` disables this check, allowing the same database connection to be used across multiple threads.

14.7 6. Thread Management and Retrieval

14.7.1 What is a Thread?

A **thread** in LangGraph represents a unique conversation session. Each thread has:

- A unique **thread_id**
- Its own **message history**
- Isolated **state** from other threads

Think of threads as separate “chat rooms” where each room has its own conversation.

14.7.2 How Threads are Stored

When you chat in different threads, they are stored separately in the database:

14.7.3 The `retrieve_all_threads()` Function

This function extracts all unique thread IDs from the database:

```
def retrieve_all_threads():
    all_threads = set() # Use set to store unique values only

    # List all checkpoints (passing None means all checkpoints)
    for checkpoint in checkpointer.list(None):
        # Extract thread_id from each checkpoint's config
        thread_id = checkpoint.config['configurable']['thread_id']
        all_threads.add(thread_id)

    return list(all_threads) # Convert set to list before returning
```

Step-by-Step Explanation:

1. `checkpointer.list(None)`:

- Returns a generator of all checkpoints in the database
- Passing `None` means “don’t filter by `thread_id`, give me everything”
- If you pass a specific `thread_id`, it returns only that thread’s checkpoints

2. **Checkpoint Structure:** Each checkpoint contains a config object with this structure:

```
checkpoint.config = {
    'configurable': {
        'thread_id': 'thread_1' # or 'thread_2', etc.
    }
}
```

3. Why use a Set?:

- Each thread has multiple checkpoints (3 per message exchange)
- Using a set() automatically removes duplicates
- Thread “thread_1” appears 9 times? Set keeps only 1 instance

4. Convert to List:

- Sets don’t maintain order and some operations need lists
- list(all_threads) converts the set to a list for compatibility

14.7.4 Testing Thread Persistence

```
# Test: Send a message in Thread 1
CONFIG = {'configurable': {'thread_id': 'thread_1'}}
response = chatbot.invoke(
    {'messages': [HumanMessage(content='Hi, my name is Nitish')]},
    config=CONFIG
)
print(response) # Shows conversation with AI's response

# Close and restart the program...

# Test: Ask a follow-up question
response = chatbot.invoke(
    {'messages': [HumanMessage(content='What is my name?')]},
    config=CONFIG
)
print(response) # AI remembers: "Your name is Nitish!"
```

If using **InMemorySaver**, the second query would fail because the name was lost. With **SqliteSaver**, the AI remembers across restarts!

14.8 7. Frontend Integration

14.8.1 What Changes in Frontend?

The main change is **how we initialize the chat_threads list**:

Table 14.4. What Changes in Frontend?

Before (InMemorySaver)	After (SqliteSaver)
chat_threads = []	chat_threads = retrieve_all_threads()
Always starts empty	Loads existing threads from database

14.8.2 Updated Session Setup

```
import streamlit as st
from langgraph_database_backend import chatbot, retrieve_all_threads
from langchain_core.messages import HumanMessage
import uuid

# ===== Session Setup =====
```

```

if 'message_history' not in st.session_state:
    st.session_state['message_history'] = []

if 'thread_id' not in st.session_state:
    st.session_state['thread_id'] = generate_thread_id()

# KEY CHANGE: Load existing threads from database instead of empty list
if 'chat_threads' not in st.session_state:
    st.session_state['chat_threads'] = retrieve_all_threads() # ← This is the key change!

add_thread(st.session_state['thread_id'])

```

14.8.3 Complete Frontend Code

```

import streamlit as st
from langgraph_database_backend import chatbot, retrieve_all_threads
from langchain_core.messages import HumanMessage
import uuid

# ===== Utility Functions =====
def generate_thread_id():
    """Generate a unique thread ID using UUID"""
    thread_id = uuid.uuid4()
    return thread_id

def reset_chat():
    """Reset the chat by creating a new thread"""
    thread_id = generate_thread_id()
    st.session_state['thread_id'] = thread_id
    add_thread(st.session_state['thread_id'])
    st.session_state['message_history'] = []

def add_thread(thread_id):
    """Add a thread to the list if it doesn't already exist"""
    if thread_id not in st.session_state['chat_threads']:
        st.session_state['chat_threads'].append(thread_id)

def load_conversation(thread_id):
    """Load conversation history for a specific thread from the database"""
    state = chatbot.get_state(config={'configurable': {'thread_id': thread_id}})
    # Return messages if they exist, otherwise return empty list
    return state.values.get('messages', [])

# ===== Session Setup =====
if 'message_history' not in st.session_state:
    st.session_state['message_history'] = []

if 'thread_id' not in st.session_state:
    st.session_state['thread_id'] = generate_thread_id()

if 'chat_threads' not in st.session_state:
    # Load existing threads from database
    st.session_state['chat_threads'] = retrieve_all_threads()

add_thread(st.session_state['thread_id'])

# ===== Sidebar UI =====
st.sidebar.title('LangGraph Chatbot')

if st.sidebar.button('New Chat'):
    reset_chat()

st.sidebar.header('My Conversations')

# Display all threads in reverse order (newest first)
for thread_id in st.session_state['chat_threads'][::-1]:
    if st.sidebar.button(str(thread_id)):
        st.session_state['thread_id'] = thread_id
        messages = load_conversation(thread_id)

```

```

# Convert LangChain messages to streamlit format
temp_messages = []
for msg in messages:
    if isinstance(msg, HumanMessage):
        role = 'user'
    else:
        role = 'assistant'
    temp_messages.append({'role': role, 'content': msg.content})

st.session_state['message_history'] = temp_messages

# ===== Main Chat UI =====
# Display conversation history
for message in st.session_state['message_history']:
    with st.chat_message(message['role']):
        st.text(message['content'])

# Handle user input
user_input = st.chat_input('Type here')

if user_input:
    # Add user message to history and display
    st.session_state['message_history'].append({'role': 'user', 'content': user_input})
    with st.chat_message('user'):
        st.text(user_input)

    # Configure the request
    CONFIG = {
        "configurable": {"thread_id": st.session_state["thread_id"]},
        "metadata": {"thread_id": st.session_state["thread_id"]},
        "run_name": "chat_turn",
    }

    # Get AI response with streaming
    with st.chat_message('assistant'):
        ai_message = st.write_stream(
            message_chunk.content for message_chunk, metadata in chatbot.stream(
                {'messages': [HumanMessage(content=user_input)]},
                config=CONFIG,
                stream_mode='messages'
            )
        )

    # Add AI response to history
    st.session_state['message_history'].append({'role': 'assistant', 'content': ai_message})

```

14.8.4 Application Flow with Database

14.8.5 Installing SQLite Viewer Extension

To view the contents of your SQLite database in VS Code:

1. Open VS Code Extensions (Ctrl+Shift+X)
2. Search for “SQLite Viewer”
3. Install the extension by **Florian Klampfer**
4. Click on any .db file to view its contents

14.8.6 What You’ll See in the Database

When you open chatbot.db, you’ll see all checkpoints stored:

Table 14.5. What You’ll See in the Database

thread_id	checkpoint_id	data
thread_1	1	START checkpoint

thread_id	checkpoint_id	data
thread_1	2	Chat node checkpoint
thread_1	3	END checkpoint
thread_1	4	START checkpoint (2nd message)
thread_2	1	START checkpoint
...	...	...

14.8.7 Understanding Checkpoint Count

If you have:

- 2 threads (thread_1 and thread_2)
- 3 message exchanges in each thread

You'll have: $2 \times 3 \times 3 = 18$ checkpoints

Table 14.6. *Understanding Checkpoint Count*

Action	Checkpoints Created	Running Total
Thread 1: "Hi, my name is Nitish"	3	3
Thread 1: "What is my name?"	3	6
Thread 1: "Capital of India?"	3	9
Thread 2: "Hi, my name is Rahul"	3	12
Thread 2: "What is my name?"	3	15
Thread 2: "Capital of India?"	3	18

14.8.8 Navigating the Database Viewer

In the SQLite Viewer:

1. Click on a checkpoint row to see basic info
2. Click on the "configurable" column to see the checkpoint's config
3. Look for the thread_id in the config to identify which thread it belongs to
4. Examine the data column to see actual messages stored

The messages are stored in JSON format within the checkpoint data, showing:

- User messages: "Hi, my name is Nitish"
- AI responses: "Hello Nitish! How can I help you?"

14.9 9. Complete Code Reference

14.9.1 File Structure

```
project/|—
  langgraph_database_backend.py # Backend with SQLite|—
  streamlit_frontend_database.py # Frontend with Streamlit|—
  chatbot.db                  # SQLite database (auto-created)|—
  .env                        # API keys|—
  requirements.txt            # Dependencies
```

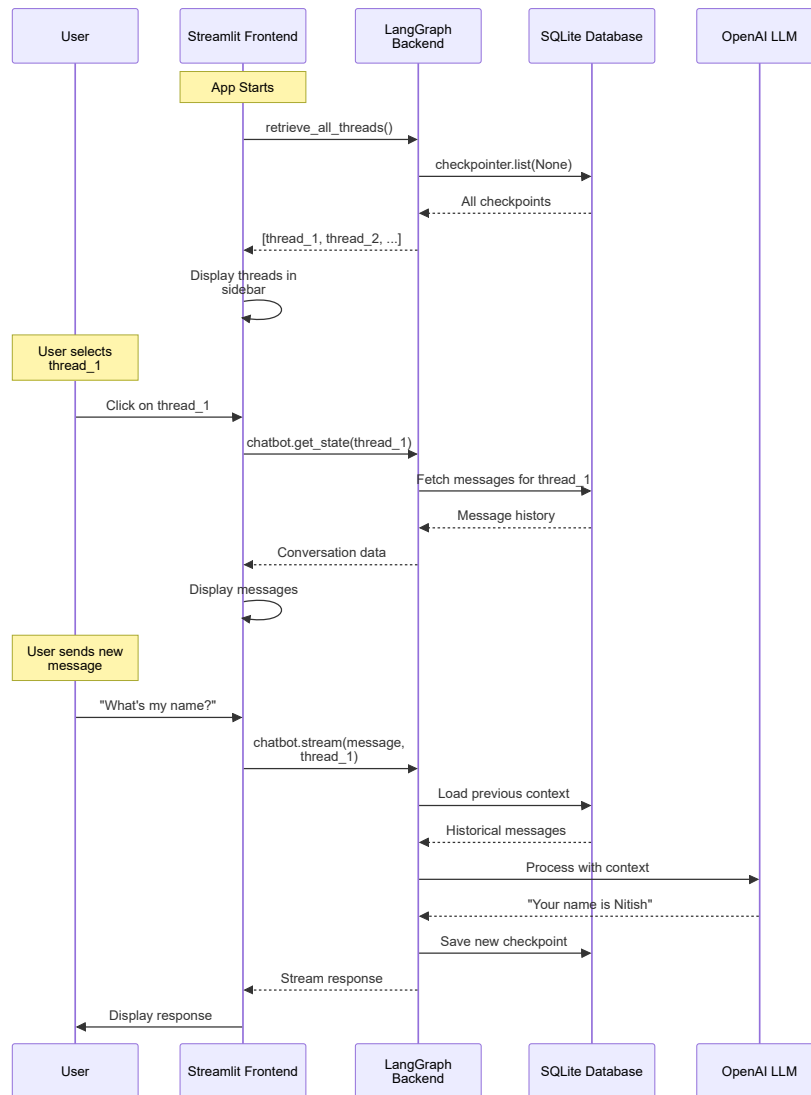



Figure 14.4. Application Flow with Database

14.9.2 requirements.txt

```

langgraph
langchain
langchain-openai
langgraph-checkpoint-sqlite
streamlit
python-dotenv

```

14.9.3 .env File

```

OPENAI_API_KEY=your_api_key_here

```

14.9.4 Backend File: langgraph_database_backend.py

```

from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated

```

```

from langchain_core.messages import BaseMessage, HumanMessage
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.graph.message import add_messages
from dotenv import load_dotenv
import sqlite3

# Load environment variables
load_dotenv()

# Initialize LLM
llm = ChatOpenAI()

# Define state structure
class ChatState(TypedDict):
    messages: Annotated[List[BaseMessage], add_messages]

# Chat node function
def chat_node(state: ChatState):
    messages = state['messages']
    response = llm.invoke(messages)
    return {"messages": [response]}

# Database connection
conn = sqlite3.connect(database='chatbot.db', check_same_thread=False)

# SQLite checkpointer
checkpointer = SqliteSaver(conn=conn)

# Build graph
graph = StateGraph(ChatState)
graph.add_node("chat_node", chat_node)
graph.add_edge(START, "chat_node")
graph.add_edge("chat_node", END)

# Compile chatbot with checkpointer
chatbot = graph.compile(checkpointer=checkpointer)

def retrieve_all_threads():
    """Retrieve all unique thread IDs from the database"""
    all_threads = set()
    for checkpoint in checkpointer.list(None):
        all_threads.add(checkpoint.config['configurable']['thread_id'])
    return list(all_threads)

```

14.9.5 Frontend File: streamlit_frontend_database.py

```

import streamlit as st
from langgraph_database_backend import chatbot, retrieve_all_threads
from langchain_core.messages import HumanMessage
import uuid

# Utility functions
def generate_thread_id():
    return uuid.uuid4()

def reset_chat():
    thread_id = generate_thread_id()
    st.session_state['thread_id'] = thread_id
    add_thread(st.session_state['thread_id'])
    st.session_state['message_history'] = []

def add_thread(thread_id):
    if thread_id not in st.session_state['chat_threads']:
        st.session_state['chat_threads'].append(thread_id)

def load_conversation(thread_id):
    state = chatbot.get_state(config={'configurable': {'thread_id': thread_id}})
    return state.values.get('messages', [])

```

```

# Session setup
if 'message_history' not in st.session_state:
    st.session_state['message_history'] = []

if 'thread_id' not in st.session_state:
    st.session_state['thread_id'] = generate_thread_id()

if 'chat_threads' not in st.session_state:
    st.session_state['chat_threads'] = retrieve_all_threads()

add_thread(st.session_state['thread_id'])

# Sidebar
st.sidebar.title('LangGraph Chatbot')

if st.sidebar.button('New Chat'):
    reset_chat()

st.sidebar.header('My Conversations')

for thread_id in st.session_state['chat_threads'][::-1]:
    if st.sidebar.button(str(thread_id)):
        st.session_state['thread_id'] = thread_id
        messages = load_conversation(thread_id)
        temp_messages = []
        for msg in messages:
            role = 'user' if isinstance(msg, HumanMessage) else 'assistant'
            temp_messages.append({'role': role, 'content': msg.content})
        st.session_state['message_history'] = temp_messages

# Main UI
for message in st.session_state['message_history']:
    with st.chat_message(message['role']):
        st.text(message['content'])

user_input = st.chat_input('Type here')

if user_input:
    st.session_state['message_history'].append({'role': 'user', 'content': user_input})
    with st.chat_message('user'):
        st.text(user_input)

    CONFIG = {
        "configurable": {"thread_id": st.session_state["thread_id"]},
        "metadata": {"thread_id": st.session_state["thread_id"]},
        "run_name": "chat_turn",
    }

    with st.chat_message('assistant'):
        ai_message = st.write_stream(
            message_chunk.content for message_chunk, metadata in chatbot.stream(
                {'messages': [HumanMessage(content=user_input)]},
                config=CONFIG,
                stream_mode='messages'
            )
        )

    st.session_state['message_history'].append({'role': 'assistant', 'content': ai_message})

```

14.9.6 Running the Application

```

# Run the Streamlit frontend
streamlit run streamlit_frontend_database.py

```

14.10 10. Summary and Best Practices

14.10.1 Key Takeaways

1. **InMemorySaver vs SqliteSaver:** InMemorySaver is for development/testing only; SqliteSaver provides persistent storage suitable for prototyping and learning.
2. **Database Connection:** Always set `check_same_thread=False` when using SQLite in multi-threaded applications like chatbots.
3. **Thread Retrieval:** Use `checkpointer.list(None)` to get all checkpoints, then extract unique thread IDs using a set.
4. **Frontend Changes:** The main change is initializing `chat_threads` from the database instead of an empty list.
5. **Checkpoint Count:** Each message exchange creates 3 checkpoints (START, chat_node, END).

14.10.2 When to Use Each Checkpointer

Table 14.7. *When to Use Each Checkpointer*

Scenario	Recommended Checkpointer
Quick testing during development	InMemorySaver
Learning LangGraph concepts	SqliteSaver
Building prototypes	SqliteSaver
Demo applications	SqliteSaver
Production applications	PostgresSaver
High-concurrency systems	PostgresSaver
Cloud deployments	PostgresSaver

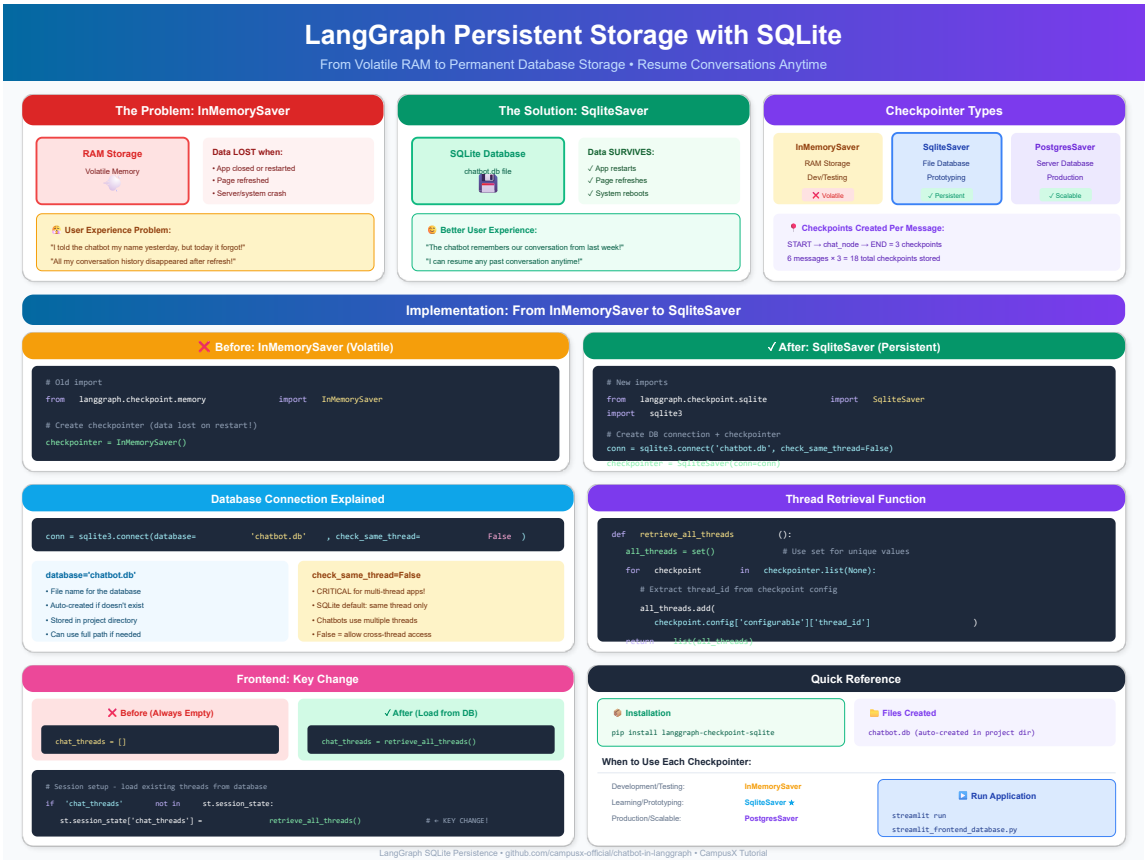
14.10.3 Best Practices

1. **Always close database connections** when your application shuts down
2. **Use meaningful thread IDs** (UUID is recommended)
3. **Handle missing messages gracefully** with `.get('messages', [])`
4. **Consider database backup** for important conversations
5. **Use PostgresSaver for production** - SQLite is single-file and has concurrency limitations

14.10.4 Potential Improvements

For production-ready applications, consider:

- Adding user authentication
- Implementing thread naming/labeling
- Adding search functionality for conversations
- Implementing conversation export/import
- Adding rate limiting and error handling
- Using PostgreSQL for better scalability



LangSmith: Complete Guide to LLM Observability and Debugging

15.1 Code

- <https://github.com/campusx-official/langsmith-masterclass>

15.2 1. Introduction

15.2.1 What is This Guide About?

LangSmith is a **unified observability and evaluation platform** developed by LangChain for debugging, testing, and monitoring AI applications. As LLM-based applications become increasingly complex with multiple components, chains, agents, and RAG systems, the need for proper observability becomes critical.

15.2.2 Prerequisites

Before diving into LangSmith, you should have:

Table 15.1. *Prerequisites*

Prerequisite	Why Needed
LangChain Knowledge	Understanding chains, prompts, models, and parsers
LangGraph Basics	Understanding graph-based workflows (if working with agents)
Python Experience	Writing and understanding Python code

15.2.3 Learning Objectives

By the end of this guide, you will understand:

1. **Why observability matters** in LLM applications through real-world scenarios
2. **What LangSmith is** and how it helps solve debugging challenges
3. **Core concepts** (Project, Trace, Run) that form the foundation
4. **Practical implementation** with hands-on code examples
5. **Best practices** for production-ready LLM observability

15.3 2. Why Do We Need LangSmith?

To understand the need for LangSmith, let's explore three real-world scenarios that demonstrate the debugging challenges in LLM-based systems.

15.3.1 Scenario 1: AI Job Application Assistant - Latency Debugging

The Problem Context

Imagine you work at a startup that has identified a problem faced by students:
The Manual Process Students Face Daily:



Figure 15.1. The Problem Context

Pain Point: This entire process must be repeated **10+ times daily**. Students don't want to send generic applications - they want customized cover letters that show effort for each specific job.

The LLM Solution Your Startup Built

Your team built an **AI Job Application Assistant** that automates this process:

Key Observation: This system uses **multiple LLM calls**:

- JD analysis → LLM
- Skill matching → LLM
- Cover letter generation → LLM
- Proofreading → LLM

The Latency Problem Emerges

Table 15.2. The Latency Problem Emerges

Metric	Normal Performance	Problem State
Total Latency	~2 minutes	7-10 minutes
User Experience	Acceptable	Frustrating
User Retention	Good	Users leaving!

What Happened?

One day, you start receiving complaints:

- “Your app is too slow!”
- “Takes forever to generate a cover letter!”
- “I’m switching to a competitor!”

The Debugging Challenge:**The Root Cause (Unknown Without Observability):**

In the last update, someone accidentally changed the Google Drive scan:

- **Before:** Scan only the “Portfolio” folder
- **After:** Scan the **entire** Google Drive

This single change caused the portfolio fetch step to take **8+ extra minutes**, but without observability, you can’t identify which component is the culprit!

15.3.2 Scenario 2: Research Assistant Agent - Cost Debugging**The Problem Context**

Your team built a **Research Assistant Agent** that helps researchers:

This is an AGENT - meaning it has autonomy to:

- Decide how many papers to fetch
- Determine when quality is “good enough”
- Loop back if results aren’t satisfactory

The Cost Problem Emerges

Table 15.3. *The Cost Problem Emerges*

Metric	Normal Performance	Problem State
Cost per Report	₹0.50 (~\$0.006)	₹2.00 (~\$0.024)
Cost Multiplier	1x	4x increase!
Monthly Impact	Budget OK	Exceeding budget!

What Happened?

Your OpenAI dashboard shows a massive spike in token usage:

- Some reports still cost ₹0.50 ✓
- Some reports cost ₹2.00 ✗
- Pattern seems random!

The Root Cause (Unknown Without Observability):

In a recent update, someone changed the prompt:

```
# Before (Original Prompt)
Generate a summary of the research papers.

# After (Problematic Prompt)
Generate a summary of the research papers.
Keep improving until the summary is excellent. ← This line causes issues!
```

What This Caused:

The agent became like a **perfectionist**:

- For some topics → Happy with first result → ₹0.50 ✓

- For other topics → “Not good enough, let me try again...” → Loops 3-4 times → ₹2.00 ✗

The Debugging Challenge:

- No error traces (code runs fine)
- Not consistent (happens sometimes, not always)
- Multiple components involved
- Agent behavior is autonomous and unpredictable

15.3.3 Scenario 3: RAG Chatbot - Hallucination Debugging

The Problem Context

You work at TCS (a large IT company) and built a **RAG-based HR Chatbot** for freshers:

Why This Was Needed:

Table 15.4. *The Problem Context*

Problem	Impact
Thousands of freshers join annually	Need constant HR support
Repetitive questions	HR overloaded
Same questions daily	“What’s the leave policy?” “What’s the notice period?”

The Solution: RAG Chatbot

The Hallucination Problem Emerges

Complaints Start Coming:

“Your chatbot told me we have **unlimited leave**! I packed my bags for Goa and got fired!”

“The chatbot said **notice period is 1 week**. HR says it’s 3 months!”

Table 15.5. *The Hallucination Problem Emerges*

Issue	Impact
Misinformation	Employees making wrong decisions
Trust Loss	No one believes the chatbot anymore
Serious Consequences	People could lose jobs based on wrong info

Two Possible Root Causes:

Table 15.6. *The Hallucination Problem Emerges*

Failure Point	What Went Wrong	How to Identify
Retriever	n=1 (only 1 doc fetched) instead of n=5	Check what docs were retrieved
Generator	Prompt doesn’t strictly enforce “answer from context only”	Check final prompt sent to LLM

The Debugging Challenge:

Without observability, you can’t see:

- What documents were retrieved?
- What was the final prompt?
- Did the LLM follow the context?

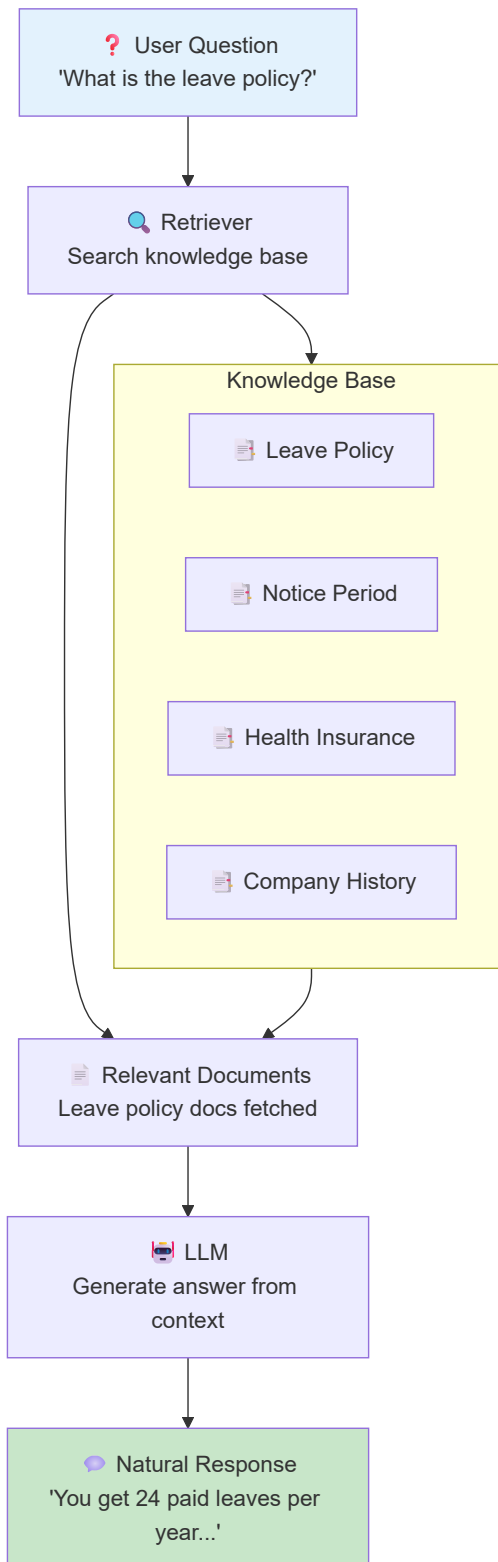


Figure 15.2. *The Problem Context*

15.4 3. The Core Problem: Non-Deterministic Nature of LLMs

15.4.1 Traditional Software vs LLM Systems

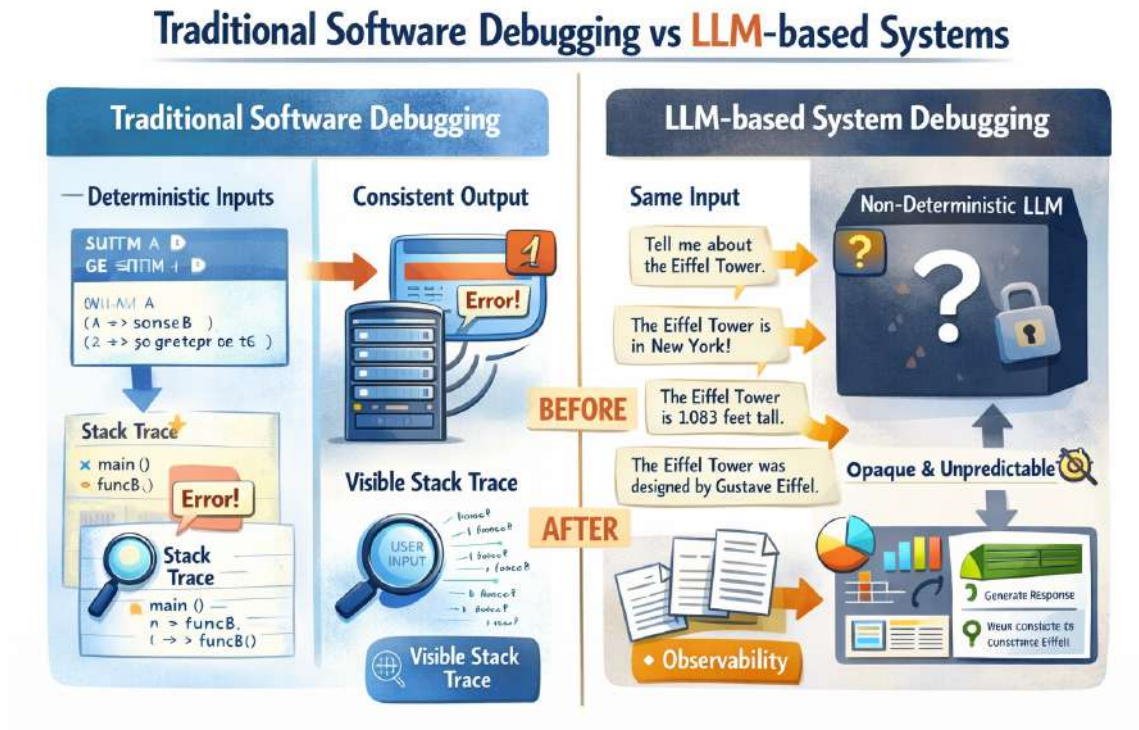


Figure 15.3. Traditional Software vs LLM Systems

15.4.2 Why This Matters for Debugging

Table 15.7. Why This Matters for Debugging

Aspect	Traditional Debugging	LLM Debugging
Error Detection	Clear error messages	No errors - just poor quality
Reproducibility	Easy to reproduce bugs	Hard to reproduce
Root Cause	Stack traces help	No stack traces for quality issues
Testing	Unit tests catch issues	Same test may pass/fail randomly

15.4.3 The Black Box Problem

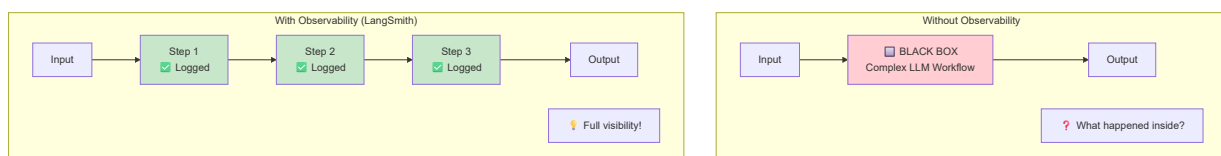


Figure 15.4. The Black Box Problem

15.5 4. Understanding Observability

15.5.1 Formal Definition

***Observability** is the ability to understand a system’s internal state by examining its external outputs, like logs, metrics, and traces.*

It allows you to diagnose issues, understand performance, and improve reliability by analyzing data generated by the system.

*Essentially, it’s about being able to answer “**WHY something is happening**” within a system, even if you didn’t anticipate the problem.*

15.5.2 Breaking Down the Definition

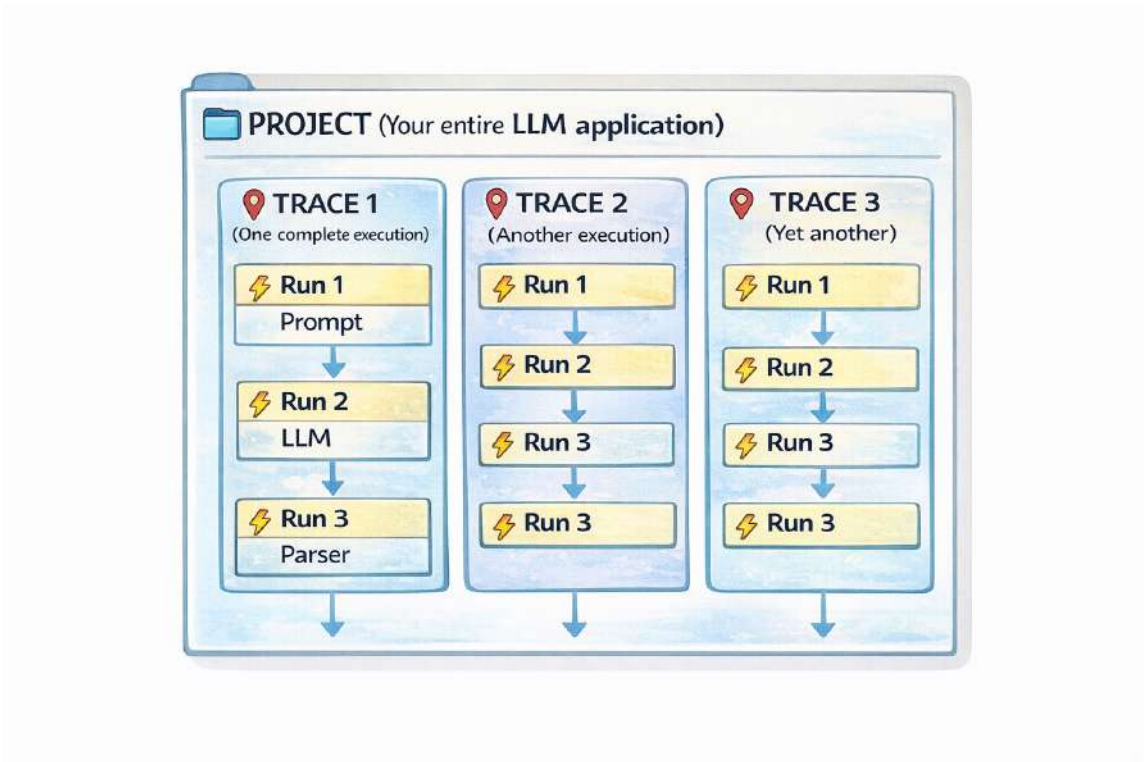


Figure 15.5. Breaking Down the Definition

15.5.3 Observability in LLM Context

For LLM applications specifically, observability means being able to:

Table 15.8. Observability in LLM Context

Need	Without Observability	With Observability
Debug latency	“It’s slow somewhere”	“Step 3 takes 8 minutes”
Control costs	“Bills are high”	“Agent loops 4 times here”
Fix hallucinations	“It makes stuff up”	“Retriever fetched wrong docs”
Understand behavior	Black box	Full transparency

15.6 5. What is LangSmith?

15.6.1 Formal Definition

*LangSmith is a unified observability and evaluation platform where teams can **debug**, **test**, and **monitor** AI application performance.*

15.6.2 What LangSmith Does

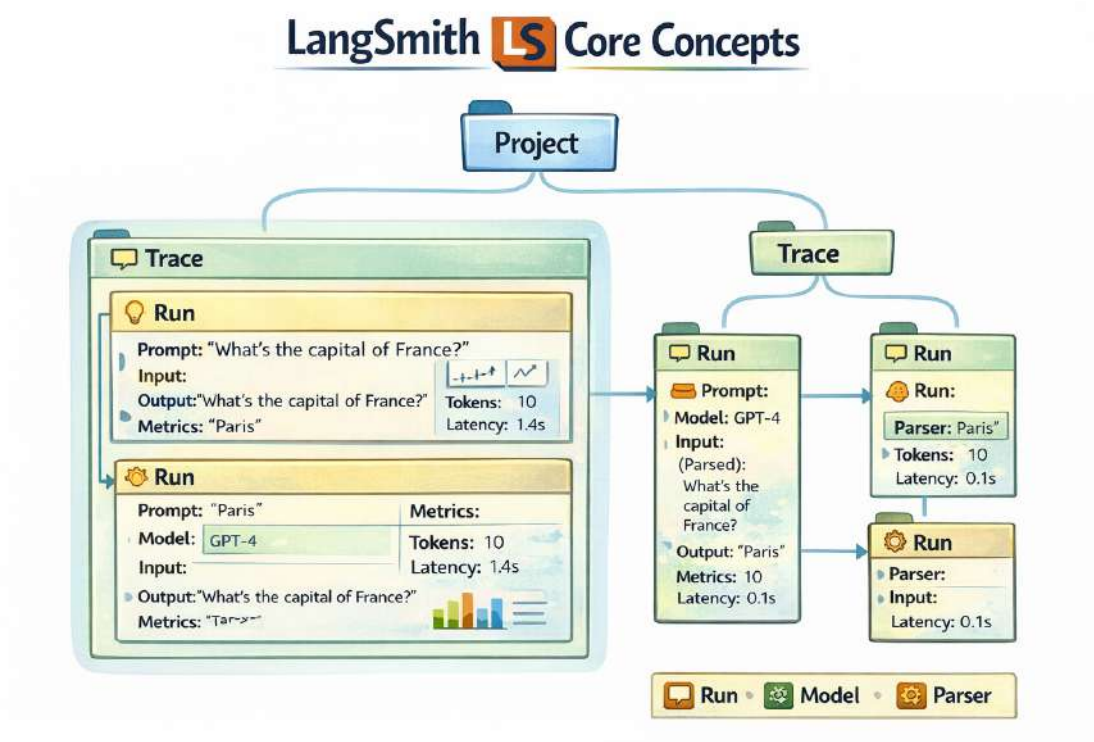


Figure 15.6. What LangSmith Does

15.6.3 Key Capabilities

Table 15.9. Key Capabilities

Capability	What It Does	Benefit
Tracing	Records every component execution	See what happened at each step
Debugging	Visualize component inputs/outputs	Find exactly where issues occur
Evaluation	Test response quality	Ensure LLM outputs meet standards
Monitoring	Track production metrics	Catch issues before users complain

15.6.4 Integration Philosophy

Zero Code Changes Required!

```
# Your existing code - NO CHANGES NEEDED
from langchain_openai import ChatOpenAI

chain = prompt | model | parser
result = chain.invoke({"question": "Hello"})
```

```
# LangSmith traces automatically IF:  
# - LANGCHAIN_TRACING_V2=true in environment  
# - LANGCHAIN_API_KEY is set
```

15.7 6. What Does LangSmith Trace?

LangSmith captures comprehensive information about every execution:

15.7.1 Complete List of Traced Data

15.7.2 Detailed Breakdown

Table 15.10. Detailed Breakdown

#	Data Type	Description	Example
1	Input & Output	User query and final response	Q: "Capital of India?" A: "New Delhi"
2	Intermediate Steps	Every component's input/output	Prompt template output, LLM response, parser result
3	Latency	Time taken at each level	Total: 1.5s, LLM: 1.2s, Parser: 0.01s
4	Token Usage	Tokens consumed	Input: 150 tokens, Output: 50 tokens
5	Cost	Money spent	\$0.0015 based on model pricing
6	Errors	Exceptions caught	"Rate limit exceeded" or "None"
7	Tags	Labels for filtering	["production", "user-query", "gpt-4o"]
8	Metadata	Additional context	{"model": "gpt-4o", "temperature": 0.7}
9	Feedback	User satisfaction	or , comments

15.8 7. Core Concepts: Project, Trace, and Run

Understanding these three concepts is **essential** for working with LangSmith.

15.8.1 Visual Hierarchy

15.8.2 Detailed Definitions

1. Project

A *Project* represents your entire LLM application.

Table 15.11. Project

Aspect	Description
What it is	Container for all traces from one application
Examples	"Job Application Assistant", "Research Agent", "HR Chatbot"

Aspect	Description
Contains	Multiple traces (all executions of this application)
Best Practice	One project per application

2. Trace

A *Trace* represents one single end-to-end execution of your application.

Table 15.12. Trace

Aspect	Description
What it is	Record of one complete user interaction
Examples	User asked "What is the capital of Peru?" → System responded
Contains	Multiple runs (one per component)
Timing	Start time, end time, total duration

3. Run

A *Run* represents the execution of one single component within a trace.

Table 15.13. Run

Aspect	Description
What it is	Record of one component doing its job
Examples	PromptTemplate formatting, LLM generating, Parser extracting
Contains	Input, output, latency, errors for that component
Parent	Belongs to exactly one trace

15.8.3 Example with Simple Chain

Consider this simple LLM application:

```
chain = prompt | model | parser
result = chain.invoke({"question": "What is AI?"})
```

Hierarchy Breakdown:

```
PROJECT: "Simple Q&A App" |
├── TRACE: User asked "What is AI?"
│   ├── Total Latency: 1.5 seconds
│   ├── Input: {"question": "What is AI?"}
│   └── Output: "AI is artificial intelligence..."
│   ├── RUN 1: PromptTemplate
│   │   ├── Input: {"question": "What is AI?"}
│   │   ├── Output: "What is AI?"
│   │   └── Latency: 0.001s
│   ├── RUN 2: ChatOpenAI (gpt-4o-mini)
│   │   ├── Input: HumanMessage("What is AI?")
│   │   ├── Output: AIMessage("AI is artificial intelligence...")
│   │   ├── Latency: 1.49s
│   │   └── Tokens: 15 in, 50 out
│   └── RUN 3: StrOutputParser
```

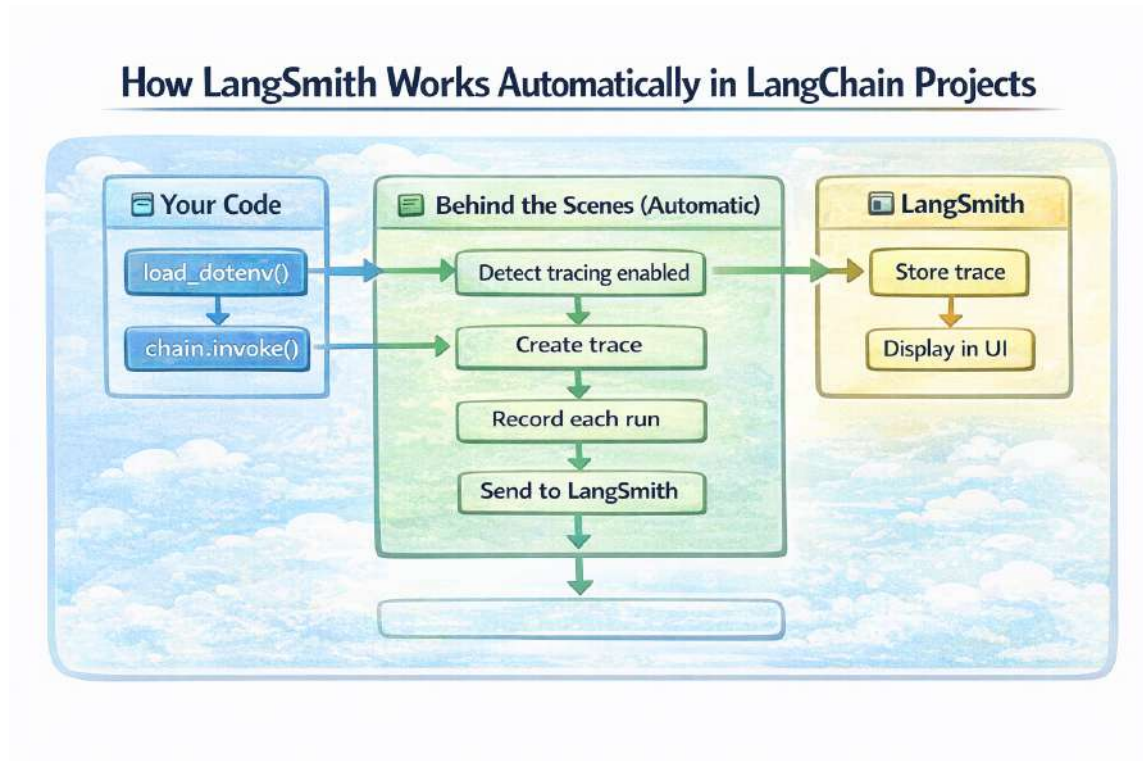



Figure 15.7. Visual Hierarchy

```

|— Input: AIMessage object
|— Output: "AI is artificial intelligence..."
|— Latency: 0.001s
  
```

15.8.4 Relationship Summary

Table 15.14. Relationship Summary

Concept	Contains	Contained By	Quantity
Project	Many Traces	-	1 per application
Trace	Many Runs	1 Project	1 per user interaction
Run	Input/Output/Metrics	1 Trace	1 per component

15.9 8. Setup and Configuration

15.9.1 Step-by-Step Setup Process

15.9.2 Step 1: Clone the Repository

```

# Clone the project repository
git clone https://github.com/campusx-official/langsmith-masterclass.git

# Navigate to the project directory
cd langsmith-masterclass
  
```

15.9.3 Step 2: Create Virtual Environment

```
# Create a new virtual environment
python -m venv myenv

# Activate the virtual environment
# On Windows:
myenv\Scripts\activate

# On macOS/Linux:
source myenv/bin/activate
```

15.9.4 Step 3: Install Dependencies

```
# Install all required packages
pip install -r requirements.txt
```

Key Dependencies:

Table 15.15. Step 3: Install Dependencies

Package	Purpose
langchain	Core LLM framework
langchain-openai	OpenAI integration
langsmith	LangSmith SDK
python-dotenv	Environment variable management

15.9.5 Step 4: Create LangSmith Account

1. Navigate to: <https://smith.langchain.com>
2. Click **Sign Up**
3. Create your account (free tier available)

15.9.6 Step 5: Generate API Key

1. After logging in, go to **Settings** (gear icon)
2. Navigate to **API Keys**
3. Click the **+** (**Plus**) button to create new key
4. Fill in details:
 - **Description:** “Personal Project” (or any identifier)
 - **Key Type:** Personal Access Token
 - **Expiry Date:** Set as needed (or “Never”)
5. Click **Create API Key**
6. **! IMPORTANT: Copy the key immediately** - you won’t see it again!

15.9.7 Step 6: Configure Environment Variables

Create a `.env` file in your project root:

```
# OpenAI Configuration
OPENAI_API_KEY='your-openai-api-key-here'

# LangSmith Configuration
LANGCHAIN_TRACING_V2=true
```

```
LANGCHAIN_ENDPOINT='https://api.smith.langchain.com'
LANGCHAIN_API_KEY='your-langsmith-api-key-here'
LANGCHAIN_PROJECT='langsmith-demo'
```

15.9.8 Configuration Parameters Explained

Table 15.16. Configuration Parameters Explained

Parameter	Purpose	Required	Example
OPENAI_API_KEY	Authenticate with OpenAI	Yes (if using OpenAI)	sk-proj-abc123...
LANGCHAIN_TRACING_V2	Enable/disable tracing	Yes	true or false
LANGCHAIN_ENDPOINT	LangSmith API URL	Yes	https://api.smith.langchain.com
LANGCHAIN_API_KEY	Your LangSmith key	Yes	lsv2_pt_abc123...
LANGCHAIN_PROJECT	Project name in LangSmith	Yes	my-project-name

Key Insight: Setting `LANGCHAIN_TRACING_V2=true` automatically enables tracing for **all** LangChain operations. No code changes required!

15.10 9. Practical Implementation

15.10.1 Example 1: Simple LLM Call

This example demonstrates the most basic LangSmith integration.

The Complete Code

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Load environment variables (including LangSmith config)
load_dotenv()

# Create components
prompt = PromptTemplate.from_template("{question}")
model = ChatOpenAI()
parser = StrOutputParser()

# Build the chain: prompt → model → parser
chain = prompt | model | parser

# Execute the chain
result = chain.invoke({"question": "What is the capital of Peru?"})
print(result)
```

Output:

```
The capital of Peru is Lima.
```

Understanding the Magic

Key Point: Notice there's **NO LangSmith-specific code** in the application!

The tracing happens automatically because:

1. `.env` file has `LANGCHAIN_TRACING_V2=true`

2. `load_dotenv()` loads these environment variables
3. LangChain automatically sends trace data to LangSmith

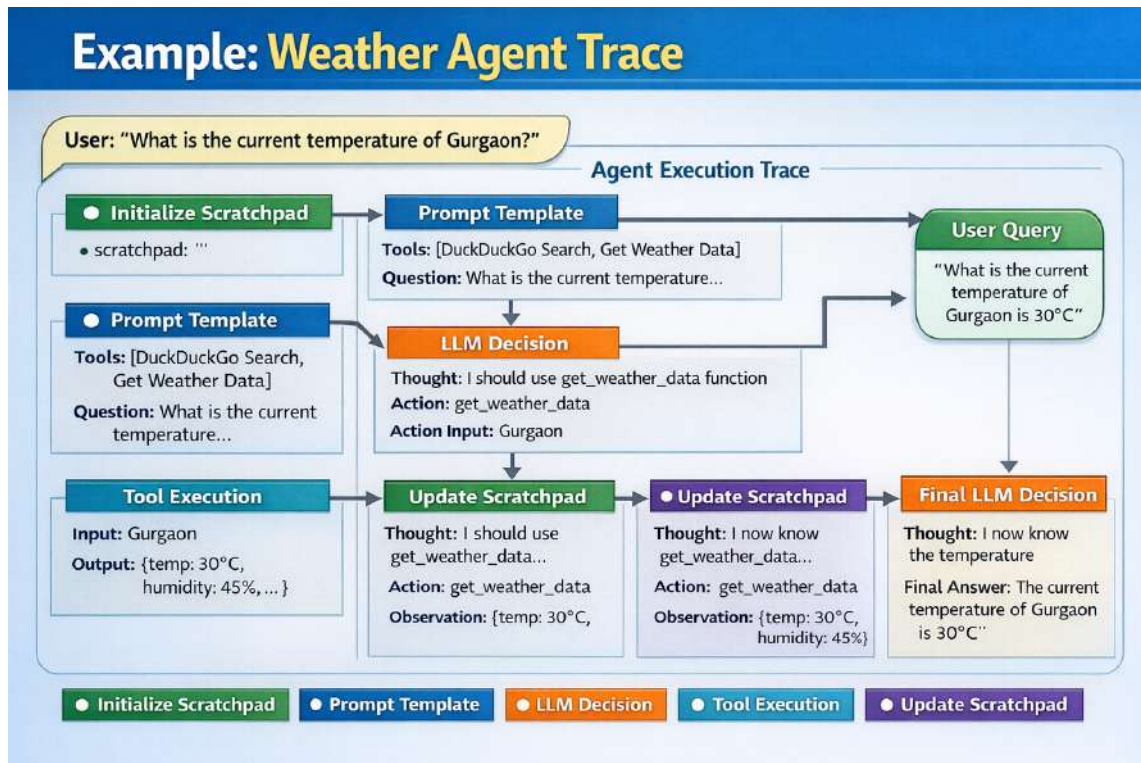


Figure 15.8. Understanding the Magic

What You See in LangSmith Dashboard

After running this code:

1. Go to smith.langchain.com
2. Click **Tracing Projects**
3. Find your project (e.g., "langsmith-demo")
4. Click on the trace

Dashboard View:

```
langsmith-demo (Project) |
Trace: RunnableSequence
| Status: Success
| Input: {"question": "What is the capital of Peru?"}
| Output: "The capital of Peru is Lima."
| Total Latency: 1.12s
| Total Tokens: 23 (15 in, 8 out)
| Cost: $0.0001
|
| Run 1: PromptTemplate
| Input: {"question": "What is the capital of Peru?"}
| Output: StringPromptValue("What is the capital of Peru?")
| Latency: 0.001s
|
| Run 2: ChatOpenAI
| Input: HumanMessage("What is the capital of Peru?")
| Output: AIMessage("The capital of Peru is Lima.")
| Latency: 1.11s
| Model: gpt-3.5-turbo (auto-tagged)
| Tokens: 15 in, 8 out
```

```

└─ Run 3: StrOutputParser
   Input: AIMessage object
   Output: "The capital of Peru is Lima."
   Latency: 0.001s

```

Metrics Summary

Table 15.17. *Metrics Summary*

Metric	Value	Notes
Total Latency	1.12s	Most time in LLM call
LLM Latency	1.11s	~99% of total time
Token Usage	15 input, 8 output	Billed tokens
Estimated Cost	~\$0.0001	Based on model pricing
Status	Success	No errors

15.10.2 Example 2: Sequential Chain with Custom Configuration

This example shows a more complex workflow with **custom tags**, **metadata**, and **project settings**.

The Complete Code

```

import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Load environment variables
load_dotenv()

# Override project name in code (instead of .env)
os.environ['LANGCHAIN_PROJECT'] = 'sequential-llm-app'

# Create prompt templates
prompt1 = PromptTemplate(
    template='Generate a detailed report on {topic}',
    input_variables=['topic']
)

prompt2 = PromptTemplate(
    template='Generate a 5 pointer summary from the following text \n {text}',
    input_variables=['text']
)

# Create models with explicit settings
model1 = ChatOpenAI(model='gpt-4o-mini', temperature=0.7)
model2 = ChatOpenAI(model='gpt-4o', temperature=0.5)

# Create parser
parser = StrOutputParser()

# Build the sequential chain
# prompt1 → model1 → parser → prompt2 → model2 → parser
chain = prompt1 | model1 | parser | prompt2 | model2 | parser

# Create custom configuration
config = {
    'tags': ['llm-app', 'report-generation', 'summarization'],
    'metadata': {
        'model1': 'gpt-4o-mini',

```

```
        'model1_temperature': 0.7,
        'model2': 'gpt-4o',
        'model2_temperature': 0.5,
        'parser': 'StrOutputParser'
    },
    'run_name': 'Sequential Report Generator'
}

# Execute the chain with config
result = chain.invoke({'topic': 'Unemployment in India'}, config=config)
print(result)
```

Understanding Custom Configuration

1. Project Name Override:

```
# Method 1: In .env file
LANGCHAIN_PROJECT='langsmith-demo'

# Method 2: In code (overrides .env)
os.environ['LANGCHAIN_PROJECT'] = 'sequential-llm-app'
```

2. Tags:

Tags are labels that help you **filter and organize** traces.

```
'tags': ['llm-app', 'report-generation', 'summarization']
```

Table 15.18. *Understanding Custom Configuration*

Use Case	Example Tags
Environment	production, staging, development
Feature	report-gen, summarization, qa
Version	v1.0, v2.0, experiment-A
Priority	critical, high, low

3. Metadata:

Metadata stores **additional context** about the execution.

```
'metadata': {
    'model1': 'gpt-4o-mini',
    'model1_temperature': 0.7,
    'model2': 'gpt-4o',
    'model2_temperature': 0.5,
    'parser': 'StrOutputParser'
}
```

4. Run Name:

Custom name instead of auto-generated “RunnableSequence”.

```
'run_name': 'Sequential Report Generator'
```

What You See in LangSmith Dashboard

```
sequential-llm-app (Project) | —
Trace: Sequential Report Generator ← Custom name!
|   Status: Success
|   Tags: [llm-app, report-generation, summarization] ← Custom tags!
```

```
| Metadata: {model: gpt-4o-mini, ...} ← Custom metadata!
|
| Run 1: PromptTemplate
| Tag: seq:step:1 (auto-generated)
| Input: {topic: "Unemployment in India"}
| Output: "Generate a detailed report on Unemployment in India"
|
| Run 2: ChatOpenAI (gpt-4o-mini)
| Tag: seq:step:2
| Model: gpt-4o-mini ← Auto-detected!
| Temperature: 0.7
| Output: [Full detailed report...]
|
| Run 3: StrOutputParser
| Tag: seq:step:3
|
| Run 4: PromptTemplate
| Tag: seq:step:4
| Input: {text: [Full report from step 2]}
|
| Run 5: ChatOpenAI (gpt-4o)
| Tag: seq:step:5
| Model: gpt-4o ← Different model!
| Temperature: 0.5
|
| Run 6: StrOutputParser
| Tag: seq:step:6
| Output: [5-point summary]
```

Key Observations

Table 15.19. Key Observations

Feature	What You See
6 Runs	2 prompts + 2 LLMs + 2 parsers
Auto Tags	Each run tagged with sequence step (seq:step:1, etc.)
Model Detection	LangSmith automatically identifies gpt-4o-mini vs gpt-4o
Custom Name	“Sequential Report Generator” instead of “RunnableSequence”
Metadata Access	Click “Metadata” tab to see your custom data

15.11 10. Solving the Initial Scenarios

Now that you understand LangSmith, let’s see how it solves our three scenarios:

15.11.1 Scenario 1 Solution: Latency Debugging

With LangSmith:

```
Trace: Cover Letter Generation
| Total Latency: 10 minutes (T00 SLOW!)
|
| JD Analysis: 15 seconds
| Portfolio Fetch: 8 MINUTES ← CULPRIT FOUND!
| Skill Matching: 30 seconds
| Cover Letter Gen: 45 seconds
| Proofreading: 30 seconds
```

Root Cause Identified: Portfolio fetch taking 8 minutes instead of 30 seconds. Someone changed the Google Drive scan scope!

15.11.2 Scenario 2 Solution: Cost Debugging

With LangSmith:

```
Trace 1: Solar Energy Report ₹(0.50)
|   Total Tokens: 5000
|   Agent Loops: 1
└─ Normal

Trace 2: Quantum Computing Report ₹(2.00)
|   Total Tokens: 20000 ← 4x more!
|   Agent Loops: 4 ← Agent keeps retrying!
└─ Agent perfectionism issue
```

Root Cause Identified: Prompt change (“keep improving until excellent”) causes agent to loop multiple times for certain topics.

15.11.3 Scenario 3 Solution: Hallucination Debugging

With LangSmith:

```
Trace: Leave Policy Question
|
├─ Retriever Run
|   Query: "What is leave policy?"
|   Documents Retrieved: 1 (n=1!) ← TOO FEW!
|   Document: "Company History.pdf" ← WRONG DOC!
└─ Generator Run
    Context: [Company history info]
    Output: "Unlimited leave!" ← HALLUCINATION!
```

Root Cause Identified:

1. n=1 is too low - should be n=5
2. Retriever fetched wrong document
3. LLM had no relevant context, so it hallucinated

15.12 11. Best Practices

15.12.1 Project Organization

Table 15.20. Project Organization

Practice	Recommendation
One project per application	Don't mix different apps in one project
Descriptive project names	hr-chatbot-prod not test123
Environment separation	hr-chatbot-dev, hr-chatbot-staging, hr-chatbot-prod

15.12.2 Effective Tagging

```
# Good tagging strategy
config = {
  'tags': [
    'production',      # Environment
    'hr-chatbot',      # Application
    'leave-policy',    # Feature/topic
    'v2.1',            # Version
  ]
}
```


15.12.3 Metadata Best Practices

```
# Useful metadata to track
config = {
    'metadata': {
        # Model configuration
        'model': 'gpt-4o',
        'temperature': 0.7,
        'max_tokens': 1000,

        # Application context
        'user_id': 'user_123', # For filtering
        'session_id': 'sess_456',
        'feature_flag': 'new_retriever_v2',

        # Debugging info
        'code_version': '2.1.0',
        'deployment_id': 'deploy_789'
    }
}
```

! WARNING: Never store PII (Personally Identifiable Information) in metadata!

15.12.4 Run Naming

```
# Default: RunnableSequence (not helpful)
# Better: Descriptive names

config = {
    'run_name': 'Leave Policy Q&A - User Query'
}

# Even better: Dynamic naming
config = {
    'run_name': f'HR Query: {query_type} - {timestamp}'
}
```

15.12.5 LangSmith Dashboard Navigation

Table 15.21. LangSmith Dashboard Navigation

Location	What You Find
Tracing Projects	List of all your projects
Project → Traces	All executions of that project
Trace → Runs	All components in that execution
Run → Details	Input, output, latency, tokens
Settings → API Keys	Create/manage API keys

15.13 Summary

15.13.1 Key Takeaways

1. **LLM systems are non-deterministic** - Same input can produce different outputs, making traditional debugging inadequate.
2. **Observability is essential** - You need to see inside the “black box” to debug latency, cost, and quality issues.

3. **LangSmith provides unified observability** - One platform for tracing, debugging, and monitoring.
4. **Three core concepts:**
 - **Project** = Your entire application
 - **Trace** = One complete execution
 - **Run** = One component's execution
5. **Zero code changes for basic tracing** - Just set environment variables!
6. **Custom configuration adds power** - Tags, metadata, and run names help organize and debug.

15.13.2 What's Next?

After mastering these basics, you can explore:

- RAG application tracing
- Agent workflow debugging
- LangGraph integration
- Evaluation and testing features
- Production monitoring dashboards

15.14 1. Introduction to RAG and LangSmith Integration

RAG (Retrieval-Augmented Generation) applications combine the power of Large Language Models with external knowledge sources. While the concept seems straightforward, production deployments often face quality issues that are difficult to diagnose without proper observability tools.

LangSmith provides the observability layer that makes debugging and optimizing RAG applications possible by tracing every step of the pipeline.

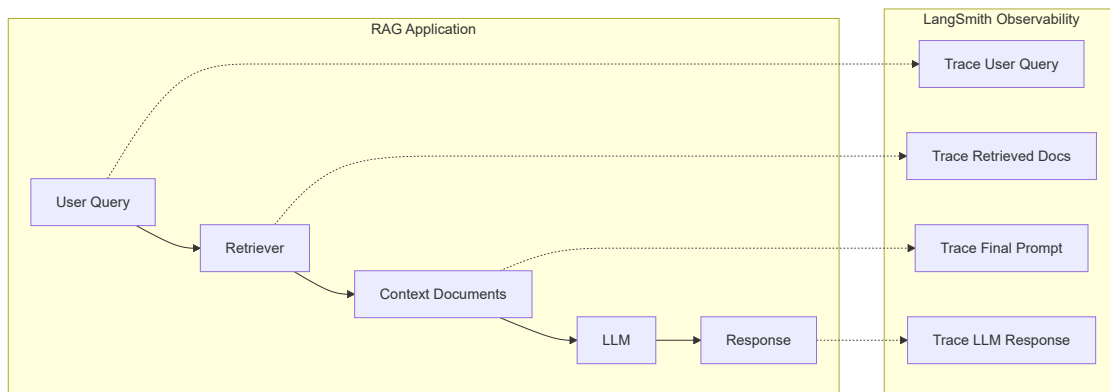


Figure 15.9. Introduction to RAG and LangSmith Integration

15.15 2. Understanding RAG Applications

15.15.1 What is RAG?

RAG (Retrieval-Augmented Generation) is an application pattern where you provide your LLM with:

1. **A Query/Question** - What the user wants to know

2. **Additional Context** - Relevant information retrieved from your knowledge base

The LLM then combines both inputs to generate a final answer based on your specific data.

15.15.2 Why RAG Sounds Simple But Is Complex in Production

Table 15.22. *Why RAG Sounds Simple But Is Complex in Production*

Aspect	Theory	Reality
Retrieval Quality	Get relevant documents	Often retrieves wrong or insufficient context
LLM Response	Generate accurate answer	May hallucinate even with correct context
Debugging	Identify issues easily	Hard to pinpoint whether retriever or generator failed
Performance	Fast responses	Can be slow without proper optimization
Monitoring	Track what's happening	No visibility into intermediate steps

15.16 3. The Two Critical Failure Modes in RAG

RAG applications can fail in two distinct ways, and understanding these failure modes is crucial for effective debugging.

15.16.1 Failure Mode 1: Retriever Errors

Definition: The retriever fails to fetch the correct or relevant chunks of information from the knowledge source.

Causes:

- Poor quality embeddings
- Inappropriate chunk size
- Query doesn't match document vocabulary
- Insufficient documents in knowledge base

Example Scenario:

User Question: "What is the author's opinion on neural networks?"
 Expected Retrieval: Chapter discussing neural network perspectives
 Actual Retrieval: Unrelated content about linear regression
 Result: Wrong or incomplete answer

15.16.2 Failure Mode 2: Generator Errors

Definition: Even when the retriever brings correct documents, the LLM hallucinates or generates incorrect responses.

Causes:

- Context too long for LLM to process effectively
- Prompt not guiding LLM properly
- LLM misinterprets the context
- Temperature setting too high

Example Scenario:

User Question: "Who wrote this book?"
 Retrieved Context: Contains author information correctly
 LLM Response: Makes up a different author name (hallucination)

15.16.3 The Debugging Dilemma in Production

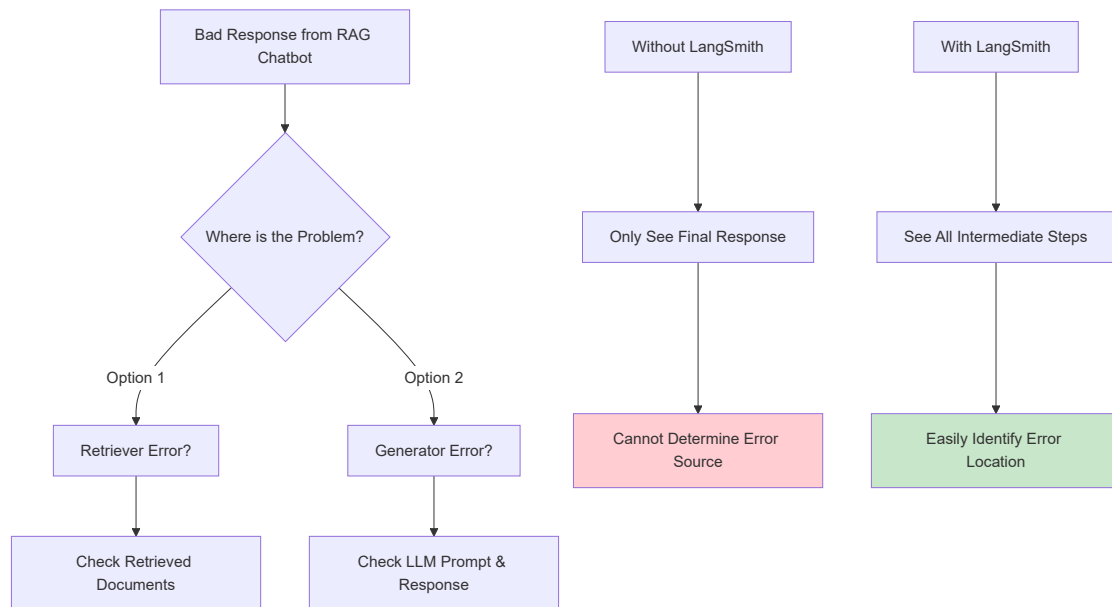


Figure 15.10. *The Debugging Dilemma in Production*

The Core Problem: In production, you only see the final response. Without intermediate step visibility, you cannot determine whether the retriever or generator is at fault.

15.17 4. Why LangSmith is Essential for RAG Applications

15.17.1 What LangSmith Traces Automatically

LangSmith captures the complete lifecycle of your RAG application:

Table 15.23. *What LangSmith Traces Automatically*

Component	What is Traced
User Query	The original question asked
Retriever Operation	Documents fetched from vector store
Final Prompt	Complete prompt sent to LLM (question + context)
LLM Response	The generated answer
Timing Information	Latency for each component
Token Usage	Input and output tokens consumed
Cost	Estimated API costs

Key Benefits:

1. **Error Identification** - Instantly see where failures occur
2. **Performance Optimization** - Identify bottlenecks in the pipeline
3. **Cost Monitoring** - Track token usage and API costs
4. **Quality Assurance** - Review retrieved documents for relevance
5. **Debugging** - Step through each component's input/output

15.18 5. Basic RAG Implementation (Version 1)

15.18.1 The Problem with Basic Implementation

The basic RAG implementation works but has two significant issues:

1. **Partial Tracing** - Only LangChain runnables are traced (where `.invoke()` is called)
2. **Repeated Processing** - PDF loading, chunking, and embedding happen on every run

15.18.2 Basic RAG Code Structure

```
# Required installations:
# pip install -U langchain langchain-openai langchain-community faiss-cpu pypdf python-dotenv

import os
from dotenv import load_dotenv
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableParallel, RunnablePassthrough, RunnableLambda
from langchain_core.output_parsers import StrOutputParser

load_dotenv() # expects OPENAI_API_KEY in .env

PDF_PATH = "islr.pdf" # Your PDF file path
```

15.18.3 Step-by-Step Pipeline

Step 1: Load the PDF

```
# Load PDF - creates one Document object per page
loader = PyPDFLoader(PDF_PATH)
docs = loader.load()
```

Step 2: Chunk the Documents

```
# Split documents into smaller chunks for better retrieval
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, # Maximum characters per chunk
    chunk_overlap=150 # Overlap between consecutive chunks
)
splits = splitter.split_documents(docs)
```

Step 3: Create Embeddings and Vector Store

```
# Generate embeddings and store in FAISS vector database
emb = OpenAIEmbeddings(model="text-embedding-3-small")
vs = FAISS.from_documents(splits, emb)
retriever = vs.as_retriever(search_type="similarity", search_kwargs={"k": 4})
```

Step 4: Create the Prompt Template

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer ONLY from the provided context. If not found, say you don't know."),
    ("human", "Question: {question}\n\nContext:\n{context}")
])
```

Step 5: Build the Chain

```
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
```

```
def format_docs(docs):
    """Merge all document objects into a single text string"""
    return "\n\n".join(d.page_content for d in docs)

# Parallel chain: processes question and context simultaneously
parallel = RunnableParallel({
    "context": retriever | RunnableLambda(format_docs),
    "question": RunnablePassthrough()
})

# Complete chain: parallel processing -> prompt -> LLM -> output parser
chain = parallel | prompt | llm | StrOutputParser()
```

15.18.4 Understanding the Chain Architecture

Chain Flow Explanation:

1. **RunnableParallel** processes two paths simultaneously:
 - **Path 1:** Question passes through unchanged (**RunnablePassthrough**)
 - **Path 2:** Question goes to retriever → retrieves documents → formats into single text
2. **Output of Parallel:** Two values - question and context
3. **Prompt Template:** Combines question and context into final prompt
4. **LLM:** Generates response based on prompt
5. **StrOutputParser:** Extracts clean string output

15.18.5 Running the Basic RAG

```
# Interactive query loop
print("PDF RAG ready. Ask a question (or Ctrl+C to exit).")
q = input("\nQ: ")
ans = chain.invoke(q.strip())
print("\nA:", ans)
```

15.18.6 Issues with Basic Implementation

Table 15.24. *Issues with Basic Implementation*

Issue	Description	Impact
Partial Tracing	Only chain execution is traced	Cannot see PDF loading, chunking, embedding steps
High Latency	Every run reprocesses the entire PDF	20-30 seconds per query
No Caching	Embeddings regenerated each time	Wasteful compute and API costs
Limited Observability	Missing critical pipeline information	Harder debugging

15.19 6. Adding Tracing with @traceable Decorator (Version 2)

15.19.1 The Solution: @traceable Decorator

LangSmith provides the `@traceable` decorator that allows you to trace any Python function, not just LangChain runnables.

```
from langsmith import traceable
```

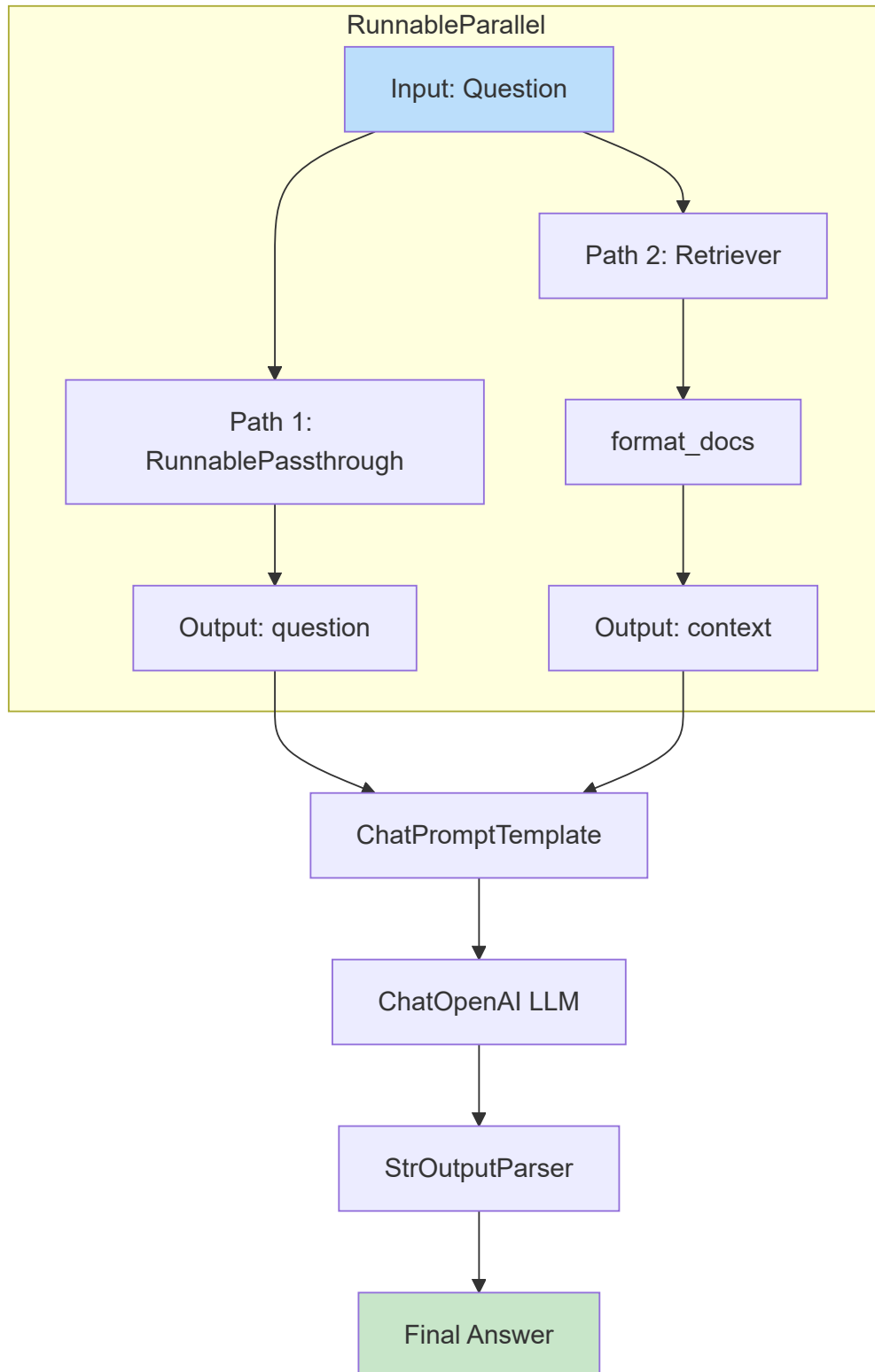


Figure 15.11. Understanding the Chain Architecture

15.19.2 Converting Code to Traced Functions

Step 1: Create Traceable Functions

```
@traceable(name="load_pdf")
def load_pdf(path: str):
    """Load PDF and return list of Document objects"""
    loader = PyPDFLoader(path)
    return loader.load()

@traceable(name="split_documents")
def split_documents(docs, chunk_size=1000, chunk_overlap=150):
    """Split documents into chunks"""
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap
    )
    return splitter.split_documents(docs)

@traceable(name="build_vectorstore")
def build_vectorstore(splits):
    """Create vector store from document chunks"""
    emb = OpenAIEmbeddings(model="text-embedding-3-small")
    return FAISS.from_documents(splits, emb)
```

Step 2: Create Parent Setup Function

```
@traceable(name="setup_pipeline", tags=["setup"])
def setup_pipeline(pdf_path: str, chunk_size=1000, chunk_overlap=150):
    """
    Parent function that orchestrates all setup steps.
    All child functions are traced under this parent.
    """
    docs = load_pdf(pdf_path)
    splits = split_documents(docs, chunk_size=chunk_size, chunk_overlap=chunk_overlap)
    vs = build_vectorstore(splits)
    return vs
```

15.19.3 Understanding the Tracing Hierarchy

When you nest traceable functions:

1. **Parent Function:** setup_pipeline becomes the root trace
2. **Child Functions:** load_pdf, split_documents, build_vectorstore appear as nested children
3. **Automatic Context:** LangSmith automatically links parent-child relationships

15.19.4 Creating a Single Root Trace for Entire Application

To ensure your entire RAG application appears as ONE trace (not two separate traces):

```
@traceable(name="pdf_rag_full_run")
def setup_pipeline_and_query(pdf_path: str, question: str):
    """
    Single root trace for entire RAG operation.
    Setup and query are both children of this root.
    """
    # Child trace: setup
    vectorstore = setup_pipeline(pdf_path, chunk_size=1000, chunk_overlap=150)
    retriever = vectorstore.as_retriever(search_type="similarity", search_kwargs={"k": 4})

    # Build chain
    parallel = RunnableParallel({
        "context": retriever | RunnableLambda(format_docs),
        "question": RunnablePassthrough(),
    })
    chain = parallel | prompt | llm | StrOutputParser()
```



```
# Child trace: query execution
return chain.invoke(question, config={"run_name": "pdf_rag_query"})
```

15.20 7. Solving the Latency Problem with Caching (Version 4)

15.20.1 The Latency Problem

Every time you run the RAG application:

Table 15.25. *The Latency Problem*

Step	Time	Repeated?
Load PDF	~15 seconds	Yes (wasteful)
Split Documents	~0.5 seconds	Yes (wasteful)
Generate Embeddings	~10 seconds	Yes (wasteful)
Query Processing	~2 seconds	Required
Total First Run	~27 seconds	
Subsequent Runs	~27 seconds	Same! (wasteful)

15.20.2 The Solution: Index Caching

Instead of regenerating embeddings every time, we:

1. **First Run:** Build index and save to disk
2. **Subsequent Runs:** Load existing index from disk

15.20.3 Performance Comparison

Table 15.26. *Performance Comparison*

Metric	Without Caching	With Caching
First Run	~27 seconds	~27 seconds
Second Run	~27 seconds	~1.5 seconds
Third Run	~27 seconds	~1.5 seconds
Improvement	-	94% faster

15.20.4 Implementation: Cache Key Generation

The cache key ensures index is rebuilt when necessary:

```
import hashlib
import json
from pathlib import Path

def _file_fingerprint(path: str) -> dict:
    """Generate fingerprint based on file content and metadata"""
    p = Path(path)
    h = hashlib.sha256()

    # Hash the actual file content
    with p.open("rb") as f:
        for chunk in iter(lambda: f.read(1024 * 1024), b''):
            h.update(chunk)

    return {
```

```

        "sha256": h.hexdigest(),    # Content hash
        "size": p.stat().st_size,   # File size
        "mtime": int(p.stat().st_mtime) # Last modified time
    }

def _index_key(pdf_path: str, chunk_size: int, chunk_overlap: int, embed_model_name: str) -> str:
    """Generate unique cache key based on all parameters"""
    meta = {
        "pdf_fingerprint": _file_fingerprint(pdf_path),
        "chunk_size": chunk_size,
        "chunk_overlap": chunk_overlap,
        "embedding_model": embed_model_name,
        "format": "v1",
    }
    return hashlib.sha256(json.dumps(meta, sort_keys=True).encode("utf-8")).hexdigest()

```

15.20.5 Implementation: Load or Build Decision

```

INDEX_ROOT = Path(".indices")
INDEX_ROOT.mkdir(exist_ok=True)

def load_or_build_index(
    pdf_path: str,
    chunk_size: int = 1000,
    chunk_overlap: int = 150,
    embed_model_name: str = "text-embedding-3-small",
    force_rebuild: bool = False,
):
    """
    Smart dispatcher: loads existing index or builds new one.
    """
    # Generate unique key for this configuration
    key = _index_key(pdf_path, chunk_size, chunk_overlap, embed_model_name)
    index_dir = INDEX_ROOT / key

    # Check if valid cache exists
    cache_hit = index_dir.exists() and not force_rebuild

    if cache_hit:
        # Fast path: load existing index
        return load_index_run(index_dir, embed_model_name)
    else:
        # Slow path: build new index
        return build_index_run(pdf_path, index_dir, chunk_size, chunk_overlap, embed_model_name)

```

15.20.6 Traced Load and Build Functions

```

@traceable(name="load_index", tags=["index"])
def load_index_run(index_dir: Path, embed_model_name: str):
    """Load existing index from disk (fast)"""
    emb = OpenAIEmbeddings(model=embed_model_name)
    return FAISS.load_local(
        str(index_dir),
        emb,
        allow_dangerous_deserialization=True
    )

@traceable(name="build_index", tags=["index"])
def build_index_run(pdf_path: str, index_dir: Path, chunk_size: int,
                    chunk_overlap: int, embed_model_name: str):
    """Build new index and save to disk (slow)"""
    # These are traced as children
    docs = load_pdf(pdf_path)
    splits = split_documents(docs, chunk_size=chunk_size, chunk_overlap=chunk_overlap)
    vs = build_vectorstore(splits, embed_model_name)

```

```
# Save to disk for future runs
index_dir.mkdir(parents=True, exist_ok=True)
vs.save_local(str(index_dir))

# Save metadata for debugging
(index_dir / "meta.json").write_text(json.dumps({
    "pdf_path": os.path.abspath(pdf_path),
    "chunk_size": chunk_size,
    "chunk_overlap": chunk_overlap,
    "embedding_model": embed_model_name,
}, indent=2))

return vs
```

15.21 8. Index Rebuild Trigger Conditions

The caching system is smart about when to rebuild the index. Here are the exact conditions:

15.21.1 When Index is Rebuilt (Slow Path)

Table 15.27. *When Index is Rebuilt (Slow Path)*

Condition	Description	Example
First Run	No cache exists yet	.indices/ folder empty
PDF Content Changes	File content hash changes	Edit the PDF file
PDF Metadata Changes	File size or modification time changes	Touch the file
Chunk Size Changes	Different chunk_size parameter	Change from 1000 to 500
Chunk Overlap Changes	Different chunk_overlap parameter	Change from 150 to 200
Embedding Model Changes	Different embedding model	Switch to ada-002
Force Rebuild	Explicitly requested	force_rebuild=True

15.21.2 When Index is Reused (Fast Path)

Index is loaded from disk when ALL of the following are true:

1. Index directory exists
2. PDF content hash matches
3. PDF metadata (size, mtime) matches
4. Chunk parameters match
5. Embedding model matches
6. force_rebuild=False

15.22 9. Adding Tags and Metadata for Rich Tracing

15.22.1 Why Tags and Metadata Matter

Tags and metadata make your traces:

1. **Searchable** - Find specific traces across thousands of runs
2. **Filterable** - Group traces by component, model, or configuration
3. **Informative** - Understand what settings were used without opening each trace

15.22.2 Adding Tags and Metadata to @traceable

```
@traceable(
    name="load_pdf",
    tags=["pdf", "loader"], # Categorization tags
    metadata={
        "loader": "PyPDFLoader" # Component-specific info
    }
)
def load_pdf(path: str):
    return PyPDFLoader(path).load()

@traceable(
    name="build_vectorstore",
    tags=["embedding", "vectorstore"],
    metadata={
        "embedding_model": "text-embedding-3-small",
        "dimensions": 1536,
        "vectorstore": "FAISS"
    }
)
def build_vectorstore(splits, embed_model_name: str):
    emb = OpenAIEmbeddings(model=embed_model_name)
    return FAISS.from_documents(splits, emb)
```

15.22.3 Practical Use Cases for Tags and Metadata

Table 15.28. Practical Use Cases for Tags and Metadata

Use Case	Tags	Metadata
A/B Testing	["model_v1"], ["model_v2"]	{"model": "gpt-4o-mini"}
Component Tracking	["retriever"], ["generator"]	{"k": 4}
Environment	["production"], ["staging"]	{"env": "prod"}
User Segments	["free_tier"], ["premium"]	{"user_type": "premium"}
Debugging	["debug"], ["slow_query"]	{"input_length": 1500}

15.22.4 Searching by Metadata in LangSmith

You can click on any metadata value in LangSmith to:

1. **Filter traces** - Show only traces with that specific value
2. **Compare runs** - See how different configurations perform
3. **Track changes** - Monitor when configurations were changed

15.23 10. Tracing Agentic Applications

15.23.1 Why Agent Tracing is Different

Agents operate autonomously with a thought-action-observation loop. This creates:

- **Multiple LLM calls** per query
- **Dynamic tool selection** based on context
- **Iterative refinement** until task completion
- **Complex decision trees** that are hard to debug

15.23.2 What LangSmith Traces for Agents

Table 15.29. What LangSmith Traces for Agents

Component	What is Traced
Scratchpad Initialization	Empty scratchpad at start
Prompt Formation	Question + available tools + format instructions
LLM Reasoning	Thought, action, action input
Tool Execution	Tool name, input, output
Scratchpad Updates	Accumulated context
Final Answer	Complete response with reasoning



Figure 15.12. What LangSmith Traces for Agents

15.23.3 Multi-Tool Agent Example

For complex queries requiring multiple tools:

Query: "Identify the birthplace of Kalpana Chawla and give its current temperature"

Trace shows:

1. Tool 1: DuckDuckGo Search → "Kalpana Chawla birthplace" → Karnal
2. Tool 2: Get Weather Data → Karnal → 27C
3. Final Answer: Combines both pieces of information

15.23.4 Benefits of Agent Tracing

Table 15.30. *Benefits of Agent Tracing*

Benefit	Description
Transparency	See exactly how agent reasons through problems
Debugging	Identify where reasoning goes wrong
Cost Tracking	Monitor token usage across multiple LLM calls
Optimization	Find redundant tool calls or reasoning loops
Quality Assurance	Verify tools return correct information

15.24 11. Complete Code Reference

15.24.1 Version 4: Production-Ready RAG with Caching

```
# pip install -U langchain langchain-openai langchain-community faiss-cpu pypdf python-dotenv langsmith

import os
import json
import hashlib
from pathlib import Path
from dotenv import load_dotenv

from langsmith import traceable

from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableParallel, RunnablePassthrough, RunnableLambda
from langchain_core.output_parsers import StrOutputParser

# Load environment variables
load_dotenv()

# Configuration
PDF_PATH = "islr.pdf" # Change to your file
INDEX_ROOT = Path(".indices")
INDEX_ROOT.mkdir(exist_ok=True)

# ===== TRACEABLE HELPER FUNCTIONS =====

@traceable(name="load_pdf", tags=["pdf", "loader"], metadata={"loader": "PyPDFLoader"})
def load_pdf(path: str):
    """Load PDF and return list of Document objects"""
    return PyPDFLoader(path).load()

@traceable(name="split_documents", tags=["chunking"])
def split_documents(docs, chunk_size=1000, chunk_overlap=150):
    """Split documents into chunks for embedding"""
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap
    )
    return splitter.split_documents(docs)

@traceable(name="build_vectorstore", tags=["embedding", "vectorstore"])
def build_vectorstore(splits, embed_model_name: str):
    """Create vector store from document chunks"""
    emb = OpenAIEmbeddings(model=embed_model_name)
    return FAISS.from_documents(splits, emb)

# ===== CACHING UTILITIES =====

def _file_fingerprint(path: str) -> dict:
    """Generate fingerprint based on file content and metadata"""
    p = Path(path)
```

```

h = hashlib.sha256()
with p.open("rb") as f:
    for chunk in iter(lambda: f.read(1024 * 1024), b''):
        h.update(chunk)
return {
    "sha256": h.hexdigest(),
    "size": p.stat().st_size,
    "mtime": int(p.stat().st_mtime)
}

def _index_key(pdf_path: str, chunk_size: int, chunk_overlap: int, embed_model_name: str) -> str:
    """Generate unique cache key based on all parameters"""
    meta = {
        "pdf_fingerprint": _file_fingerprint(pdf_path),
        "chunk_size": chunk_size,
        "chunk_overlap": chunk_overlap,
        "embedding_model": embed_model_name,
        "format": "v1",
    }
    return hashlib.sha256(json.dumps(meta, sort_keys=True).encode("utf-8")).hexdigest()

# ===== INDEX LOAD/BUILD =====

@traceable(name="load_index", tags=["index", "cache_hit"])
def load_index_run(index_dir: Path, embed_model_name: str):
    """Load existing index from disk (fast path)"""
    emb = OpenAIEmbeddings(model=embed_model_name)
    return FAISS.load_local(
        str(index_dir),
        emb,
        allow_dangerous_deserialization=True
    )

@traceable(name="build_index", tags=["index", "cache_miss"])
def build_index_run(pdf_path: str, index_dir: Path, chunk_size: int,
                    chunk_overlap: int, embed_model_name: str):
    """Build new index and save to disk (slow path)"""
    docs = load_pdf(pdf_path)
    splits = split_documents(docs, chunk_size=chunk_size, chunk_overlap=chunk_overlap)
    vs = build_vectorstore(splits, embed_model_name)

    # Save to disk
    index_dir.mkdir(parents=True, exist_ok=True)
    vs.save_local(str(index_dir))

    # Save metadata
    (index_dir / "meta.json").write_text(json.dumps({
        "pdf_path": os.path.abspath(pdf_path),
        "chunk_size": chunk_size,
        "chunk_overlap": chunk_overlap,
        "embedding_model": embed_model_name,
    }, indent=2))

    return vs

def load_or_build_index(
    pdf_path: str,
    chunk_size: int = 1000,
    chunk_overlap: int = 150,
    embed_model_name: str = "text-embedding-3-small",
    force_rebuild: bool = False,
):
    """Smart dispatcher: loads existing index or builds new one"""
    key = _index_key(pdf_path, chunk_size, chunk_overlap, embed_model_name)
    index_dir = INDEX_ROOT / key
    cache_hit = index_dir.exists() and not force_rebuild

    if cache_hit:
        return load_index_run(index_dir, embed_model_name)
    else:
        return build_index_run(pdf_path, index_dir, chunk_size, chunk_overlap, embed_model_name)

# ===== MODEL AND PROMPT =====

```

```

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer ONLY from the provided context. If not found, say you don't know."),
    ("human", "Question: {question}\n\nContext:\n{context}")
])

def format_docs(docs):
    """Merge all documents into single text"""
    return "\n\n".join(d.page_content for d in docs)

# ===== MAIN PIPELINE =====

@traceable(name="setup_pipeline", tags=["setup"])
def setup_pipeline(pdf_path: str, chunk_size=1000, chunk_overlap=150,
                  embed_model_name="text-embedding-3-small", force_rebuild=False):
    """Setup pipeline with caching"""
    return load_or_build_index(
        pdf_path=pdf_path,
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        embed_model_name=embed_model_name,
        force_rebuild=force_rebuild,
    )

@traceable(name="pdf_rag_full_run")
def setup_pipeline_and_query(
    pdf_path: str,
    question: str,
    chunk_size: int = 1000,
    chunk_overlap: int = 150,
    embed_model_name: str = "text-embedding-3-small",
    force_rebuild: bool = False,
):
    """Complete RAG pipeline: setup + query in single trace"""
    vectorstore = setup_pipeline(pdf_path, chunk_size, chunk_overlap, embed_model_name, force_rebuild)
    retriever = vectorstore.as_retriever(search_type="similarity", search_kwargs={"k": 4})

    parallel = RunnableParallel({
        "context": retriever | RunnableLambda(format_docs),
        "question": RunnablePassthrough(),
    })
    chain = parallel | prompt | llm | StrOutputParser()

    return chain.invoke(
        question,
        config={"run_name": "pdf_rag_query", "tags": ["qa"], "metadata": {"k": 4}}
    )

# ===== CLI =====

if __name__ == "__main__":
    print("PDF RAG ready. Ask a question (or Ctrl+C to exit).")
    q = input("\nQ: ").strip()
    ans = setup_pipeline_and_query(PDF_PATH, q)
    print("\nA:", ans)

```

15.24.2 Environment Variables Required

Create a `.env` file:

```

# OpenAI API Key
OPENAI_API_KEY=sk-your-openai-api-key

# LangSmith Configuration
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=lsv2-your-langsmith-api-key
LANGCHAIN_PROJECT=rag_chatbot

```


15.25 12. Best Practices and Summary

15.25.1 Summary of Key Concepts

Table 15.31. *Summary of Key Concepts*

Concept	Description
RAG Failure Modes	Retriever errors (wrong docs) and Generator errors (hallucination)
LangSmith Tracing	Full visibility into every pipeline step
@traceable Decorator	Trace custom Python functions, not just LangChain runnables
Index Caching	Save embeddings to disk, reload on subsequent runs
Cache Invalidation	Rebuild when PDF, params, or model changes
Tags & Metadata	Make traces searchable and informative
Agent Tracing	Track thought-action-observation loops

15.25.2 Best Practices Checklist

For RAG Applications:

- Use @traceable on all major pipeline functions
- Create a single root trace for the entire operation
- Implement index caching for production deployments
- Add meaningful tags for filtering traces
- Include metadata for searchability (model names, parameters)
- Set run_name in chain.invoke() config for clear identification

For Debugging:

- Check retriever output first when answers are wrong
- Verify prompt contains correct context
- Monitor token usage to identify cost issues
- Track latency for each component

For Production:

- Cache embeddings to reduce latency
- Invalidate cache on PDF changes
- Monitor for retriever quality degradation
- Set up alerts for high latency or error rates

15.25.3 Quick Reference: Decorator Usage

```
# Basic tracing
@traceable(name="function_name")
def my_function():
    pass

# With tags (for filtering)
@traceable(name="function_name", tags=["category1", "category2"])
def my_function():
    pass

# With metadata (for searchability)
@traceable(name="function_name", metadata={"key": "value"})
def my_function():
    pass

# Complete example
```

```
@traceable(
    name="build_vectorstore",
    tags=["embedding", "vectorstore"],
    metadata={
        "embedding_model": "text-embedding-3-small",
        "vectorstore": "FAISS"
    }
)
def build_vectorstore(splits, embed_model_name: str):
    # implementation
    pass
```

15.26 Key Takeaways

1. **RAG applications fail in two ways:** retriever errors and generator errors. Without visibility into intermediate steps, debugging is nearly impossible.
2. **LangSmith provides complete observability** by tracing every step of the pipeline, including user queries, retrieved documents, prompts, and LLM responses.
3. **The @traceable decorator** extends tracing beyond LangChain runnables to any Python function, enabling full end-to-end visibility.
4. **Index caching dramatically improves performance** by avoiding redundant PDF loading, chunking, and embedding generation on subsequent runs.
5. **Smart cache invalidation** ensures indexes are rebuilt when necessary (PDF changes, parameter changes, model changes) while reusing cached indexes otherwise.
6. **Tags and metadata** make traces searchable and filterable, essential for production monitoring and debugging.
7. **Agent tracing** provides transparency into the thought-action-observation loop, critical for debugging autonomous AI systems.

15.27 LangGraph Fundamentals

15.27.1 What is LangGraph?

LangGraph is a library for building LLM applications that treats them as **workflows** represented in **graph form**.

15.27.2 Key Characteristics

Table 15.32. Key Characteristics

Feature	Description
Graph-Based	LLM applications represented as directed graphs
State Management	Built-in state feature for maintaining context
Nodes	Each node represents a task/function
Edges	Connect nodes to define execution flow
Flexibility	Supports conditional branching, parallel execution, subgraphs

15.27.3 The Challenge with LangGraph

When building complex workflows in LangGraph, the graph structure becomes:

- **Complex and difficult to understand** - Multiple nodes, edges, and conditional logic

- **Hard to debug** - When something fails, identifying the problematic node is challenging
- **Opaque execution flow** - Difficult to see which path was taken during execution

This is where LangSmith becomes essential.

15.28 LangGraph + LangSmith Integration

LangGraph and LangSmith are both developed by the same team (LangChain), resulting in **tight integration** between the two products.

15.28.1 Two Fundamental Principles

Principle 1: Graph Execution → Trace

When you execute your entire LangGraph workflow from start to finish, that **complete execution becomes ONE trace** in LangSmith.

Graph.invoke(input) → Creates Single Trace in LangSmith

Principle 2: Each Node → Run

Every node in your LangGraph becomes a **separate run (component)** inside that trace.

Graph with 5 Nodes → Trace with 5 Runs

15.28.2 Tracing in LangGraph

What LangSmith Captures

Table 15.33. *What LangSmith Captures*

Element	LangSmith Representation
Full graph execution	One trace
Each node	One run inside trace
Path taken	Visualized execution flow
Conditional branches	Shows which branch executed
Parallel execution	Shows concurrent runs
Subgraphs	Nested runs

Path Visualization

LangSmith allows you to see exactly which path was taken during execution:

START → Retriever → Reranker → LLM Answer → END

If your workflow has branching (conditional, parallel, or subgraph), LangSmith captures **which specific path** was executed.

15.28.3 Practical Example: Essay Evaluation System

Let's examine a real-world example: an LLM application that evaluates essays on three parameters.

Application Overview

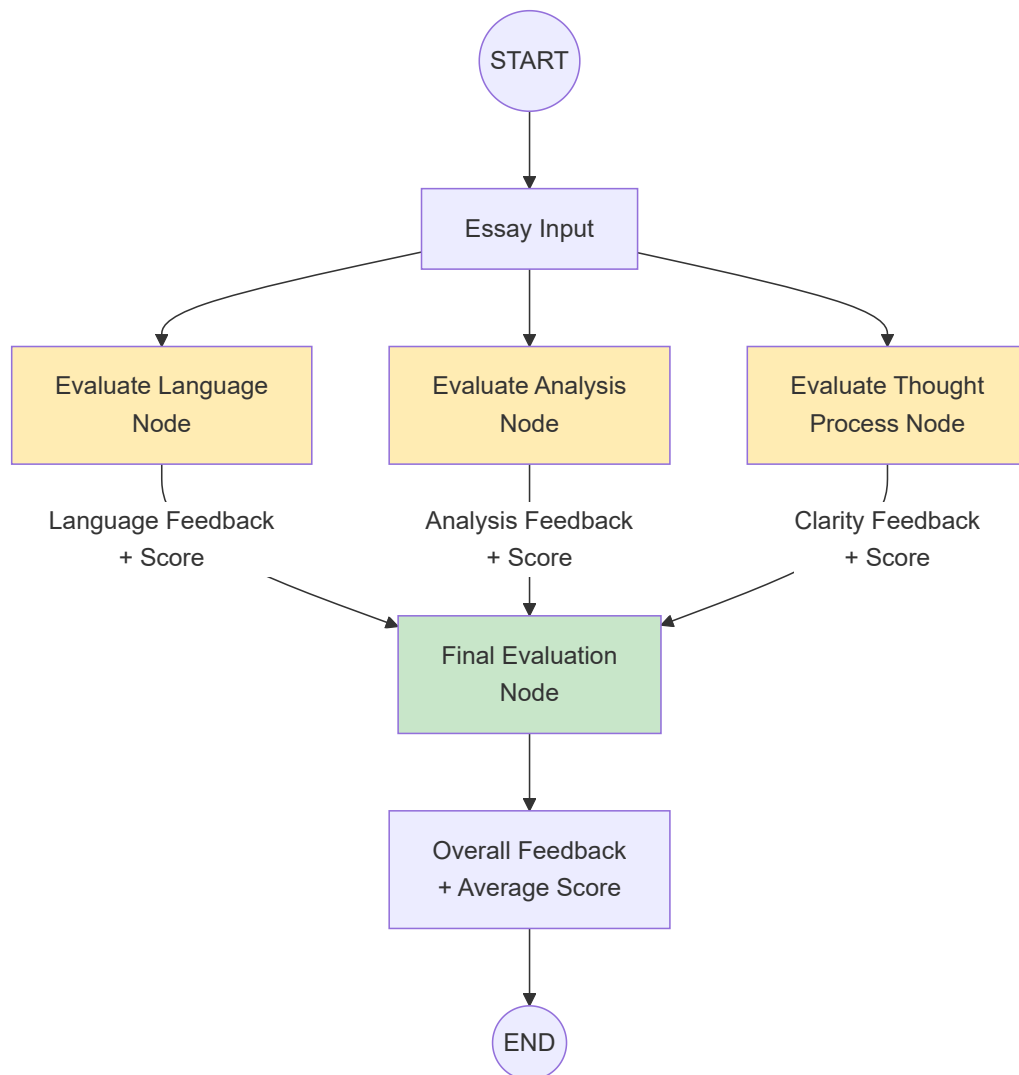


Figure 15.13. Application Overview

Workflow Description

1. **Input:** An essay is provided to the graph
2. **Parallel Evaluation:** Essay is evaluated on three dimensions simultaneously
 - **Language Quality:** Grammar, vocabulary, sentence structure
 - **Analysis Depth:** Quality of arguments and reasoning
 - **Thought Clarity:** Clear expression of ideas
3. **Individual Outputs:** Each evaluation node produces:
 - Feedback (text)
 - Score (numeric)
4. **Final Evaluation:** Combines all three feedbacks and scores
 - Generates overall feedback summary
 - Calculates average score

Code Structure Overview

```

from langgraph.graph import StateGraph
from langsmith import traceable

# Define state schema
class EssayState(TypedDict):
    essay: str
    language_feedback: str
    language_score: int
    analysis_feedback: str
    analysis_score: int
    clarity_feedback: str
    clarity_score: int
    overall_feedback: str
    average_score: float

# Node functions with @traceable for function-level tracing
@traceable(name="evaluate_language_function")
def evaluate_language(state: EssayState) -> dict:
    """Evaluate essay language quality"""
    # Uses structured LLM with schema
    model = ChatOpenAI(model="gpt-4o-mini")
    structured_model = model.with_structured_output(EvaluationSchema)

    result = structured_model.invoke(prompt)
    return {
        "language_feedback": result.feedback,
        "language_score": result.score
    }

@traceable(name="evaluate_analysis_function")
def evaluate_analysis(state: EssayState) -> dict:
    """Evaluate essay analysis depth"""
    # Similar structure
    pass

@traceable(name="evaluate_thought_function")
def evaluate_thought(state: EssayState) -> dict:
    """Evaluate clarity of thought"""
    # Similar structure
    pass

@traceable(name="final_evaluation_function")
def final_evaluation(state: EssayState) -> dict:
    """Generate overall feedback and average score"""
    # Uses regular LLM (not structured)
    model = ChatOpenAI(model="gpt-4o-mini")

    # Calculate average
    avg = (state["language_score"] +
           state["analysis_score"] +
           state["clarity_score"]) / 3

    # Generate overall feedback
    feedback = model.invoke(summary_prompt)

    return {
        "overall_feedback": feedback.content,
        "average_score": avg
    }

# Build the graph
graph = StateGraph(EssayState)

# Add nodes
graph.add_node("evaluate_language", evaluate_language)
graph.add_node("evaluate_analysis", evaluate_analysis)
graph.add_node("evaluate_thought", evaluate_thought)
graph.add_node("final_evaluation", final_evaluation)

# Add edges (parallel then converge)

```

```
graph.add_edge(START, "evaluate_language")
graph.add_edge(START, "evaluate_analysis")
graph.add_edge(START, "evaluate_thought")
graph.add_edge("evaluate_language", "final_evaluation")
graph.add_edge("evaluate_analysis", "final_evaluation")
graph.add_edge("evaluate_thought", "final_evaluation")
graph.add_edge("final_evaluation", END)
```

```
# Compile and run
app = graph.compile()
```

Running with LangSmith Tracing

```
# Execute with config for better trace identification
result = app.invoke(
    {"essay": essay_text},
    config={
        "run_name": "evaluate_upsc_essay",
        "tags": ["essay", "evaluation"],
        "metadata": {"essay_type": "upsc"}
    }
)
```

What You See in LangSmith Dashboard

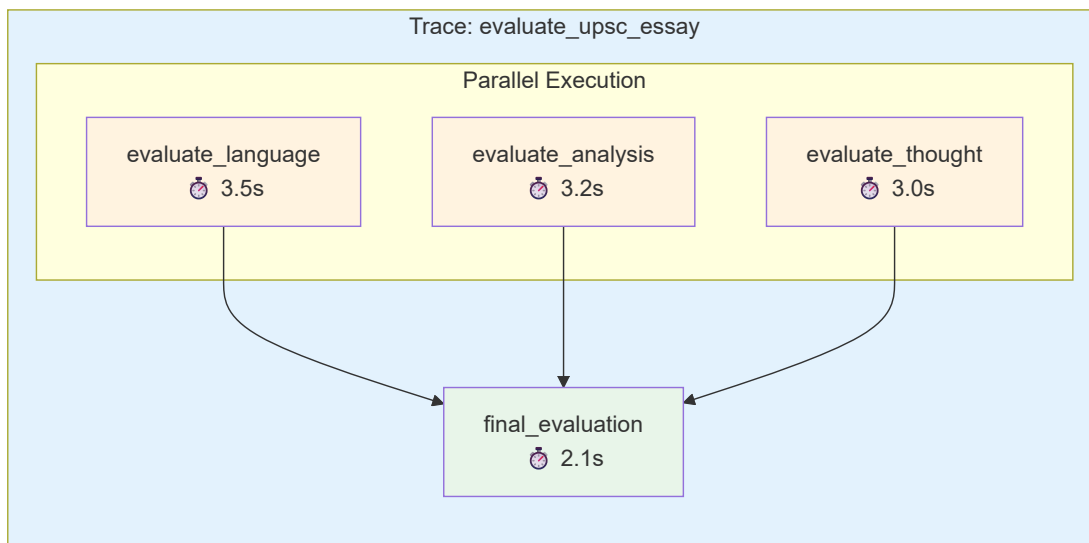


Figure 15.14. What You See in LangSmith Dashboard

Trace Hierarchy Deep Dive

When you expand each node in LangSmith:

For Structured Output Nodes (evaluate_language, evaluate_analysis, evaluate_thought):

```
evaluate_language (Node) |—
  evaluate_language_function (Traceable Function) |
    |— Input: state (with essay) |
    |— Output: {feedback, score} |—
  RunnableSequence
    |— ChatOpenAI (Structured)
    |   |— Input: prompt
```

```

└─ Output: structured response
└─ RunnableLambda
  └─ Input: LLM response
  └─ Output: extracted schema values

```

For Regular LLM Nodes (`final_evaluation`):

```

final_evaluation (Node)└─
  final_evaluation_function (Traceable Function)|
    └─ Input: state (with all feedbacks/scores)|
    └─ Output: {overall_feedback, average_score}└─
  ChatOpenAI (Simple)
    └─ Input: summary prompt
    └─ Output: text response

```

Key Insight: Structured vs Regular LLM

Table 15.34. *Key Insight: Structured vs Regular LLM*

Aspect	Structured LLM	Regular LLM
Definition	<code>model.with_structured_output(Schema)</code>	<code>ChatOpenAI(model=...)</code>
Output	Follows predefined schema	Free-form text
Trace Structure	Shows <code>RunnableSequence</code> + <code>RunnableLambda</code>	Simple LLM call
Use Case	When you need specific fields	When you need narrative text

What Information is Available Per Node

Table 15.35. *What Information is Available Per Node*

Metric	Description
Latency	Time taken for node execution
Cost	Token cost for that specific node
Input	State received by the node
Output	State modifications made by node
Token Usage	Input/output token counts
Function Details	If <code>@traceable</code> used, function-level visibility

15.29 Other Features of LangSmith

Beyond observability, LangSmith offers a comprehensive suite of features for production LLM applications.

15.29.1 Monitoring and Alerting

What It Does

Monitoring in LangSmith looks across **many traces at once** to track the **overall health** of your LLM system. Unlike observability (single trace analysis), monitoring aggregates data across all executions.

Key Metrics Tracked

Table 15.36. Key Metrics Tracked

Metric	Description	Percentiles
Latency	Response time	P50, P95, P99
Token Usage	Input/output tokens	Total, per trace
Cost	Dollar cost of LLM calls	Per trace, cumulative
Error Rates	Failed executions	Percentage
Success Rates	Successful completions	Percentage

Alerting System

You can set up alerts to notify you when metrics drift outside acceptable ranges:

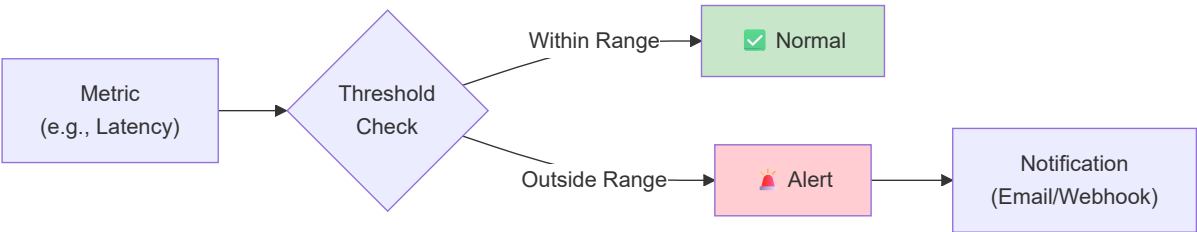


Figure 15.15. Alerting System

Example Alert Scenarios:

Table 15.37. Alerting System

Scenario	Alert Condition	Action
Latency Spike	Latency > 5 seconds	Investigate slow components
Cost Growth	Daily cost > \$100	Review token usage
Error Rate	Errors > 5%	Debug failing traces
Success Drop	Success < 95%	Check system health

Why Monitoring Matters

The Key Insight:

*In production, issues often appear first as **patterns across multiple runs** rather than in a single trace.*

A single slow trace might be a one-off event. But if you see latency increasing across hundreds of traces, that’s a **systemic issue** requiring attention.

Monitoring Dashboard in LangSmith

Available graphs and charts:

Table 15.38. Monitoring Dashboard in LangSmith

Chart	Purpose
Traces per Day	Usage patterns

Chart	Purpose
Trace Latency	Performance trends
Error Rate	System reliability
Total LLM Calls	Volume tracking
LLM Call Latency	Model performance
Cost & Tokens	Financial tracking
Cost per Trace	Unit economics
Output/Input Tokens	Token distribution
Tool Usage	Function call frequency

Setting Up Alerts

Navigation: Monitoring → Alerts → Select Project → Create Alert

Configuration Options:

- Metric selection (latency, cost, errors, etc.)
- Threshold value
- Comparison operator ($>$, $<$, $=$)
- Notification channel (email, webhook)

15.29.2 Evaluation

What It Does

Evaluation in LangSmith helps you **systematically measure the quality** of your LLM outputs.

The Problem with LLM Behavior

LLMs are **non-deterministic** (probabilistic) - same input can produce different outputs.

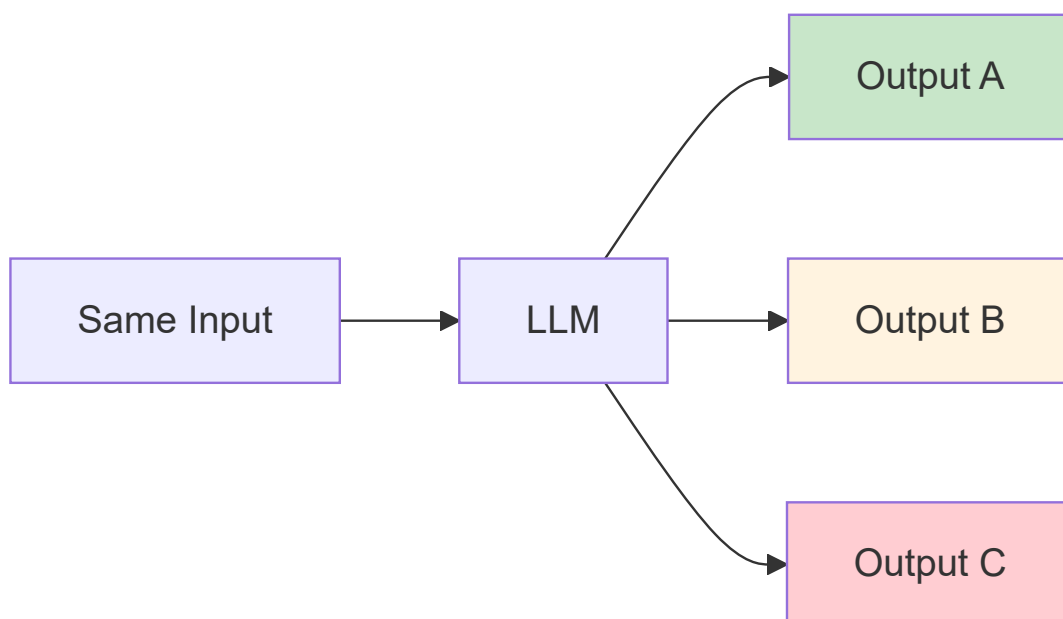


Figure 15.16. *The Problem with LLM Behavior*

A small change in your system can dramatically affect outputs:

Table 15.39. *The Problem with LLM Behavior*

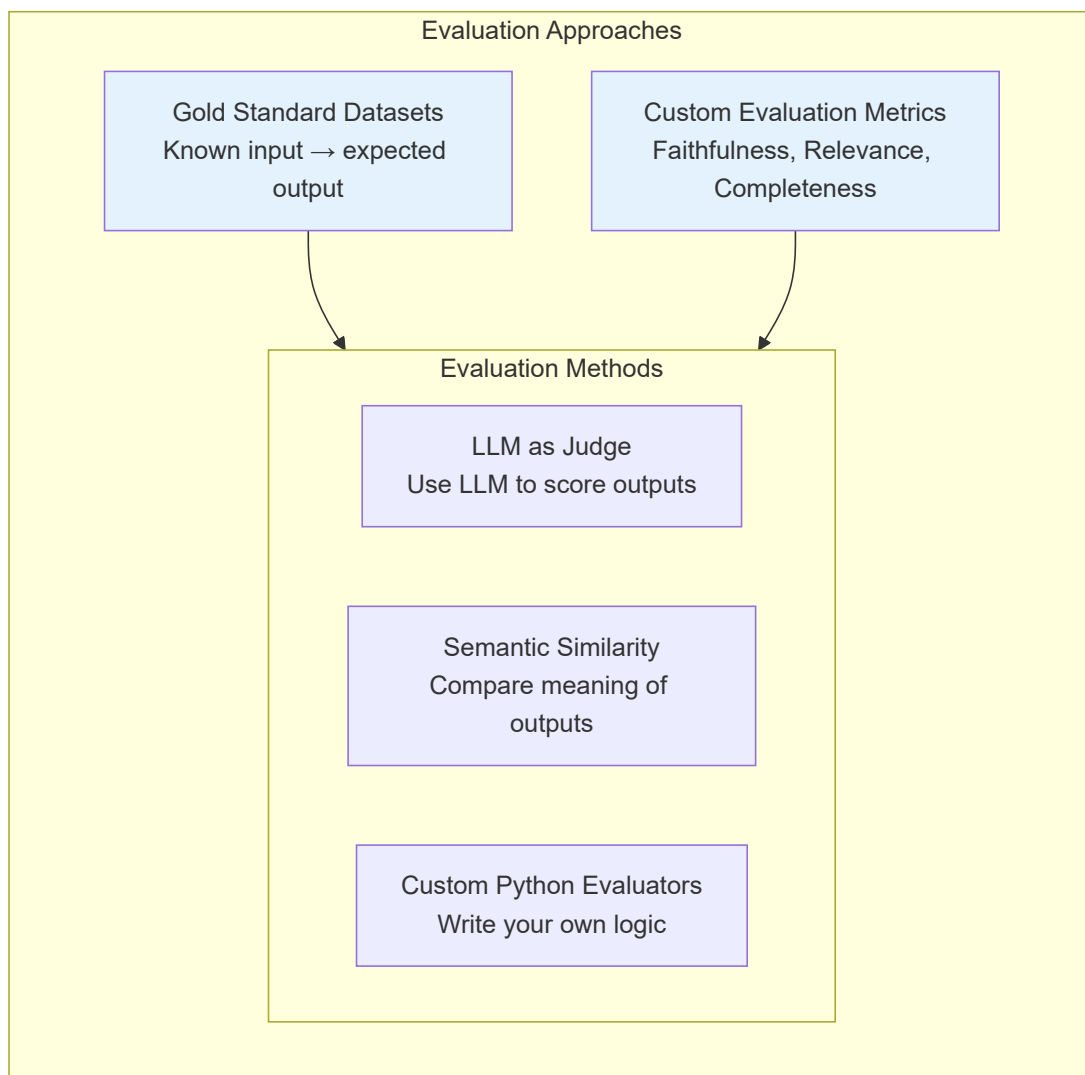
Change	Potential Impact
Prompt modification	Complete change in response style
Model swap (A → B)	Different reasoning patterns
Retriever change	Different context provided
Temperature adjustment	Varied creativity levels

Why Evaluation Matters

How do you know that Version 2 of your LLM system is better than Version 1?

Without systematic evaluation, you can't objectively answer this question. Evaluation provides an **objective, repeatable way** to track performance over time.

Evaluation Approaches in LangSmith

**Figure 15.17.** *Evaluation Approaches in LangSmith*

Evaluation Types

Table 15.40. *Evaluation Types*

Type	Description	Use Case
Offline	Batch tests before deployment	Pre-release validation
Online	Continuous checks on live traffic	Production monitoring

RAG Chatbot Evaluation Example

For a RAG chatbot, you might evaluate:

Table 15.41. *RAG Chatbot Evaluation Example*

Metric	Question
Faithfulness	Are answers grounded in retrieved documents?
Relevance	Did the response actually address the user's question?
Completeness	Was all necessary information included?
Factuality	Are the facts stated correct?

Model Comparison

By running the same dataset across different models, you can directly compare performance:

Pre-built Evaluators in LangSmith

Table 15.42. *Pre-built Evaluators in LangSmith*

Evaluator	Purpose
Hallucination Checker	Detect made-up information
Conciseness Checker	Evaluate response brevity
Code Checker	Validate code correctness
Custom	Define your own criteria

15.29.3 Prompt Experimentation

The Importance of Prompts

In generative AI, prompts are crucial. The quality of your LLM output directly depends on prompt quality. This has led to an entire field: **Prompt Engineering**.

The Challenge

How do you know if Prompt A is better than Prompt B?

Running it once in ChatGPT doesn't provide conclusive evidence. You need **systematic experimentation**.

What LangSmith Offers

A/B Testing Prompts

LangSmith allows you to:

1. **Define multiple prompt versions**
2. **Run them on the same standardized dataset**
3. **Track performance against evaluation metrics**
4. **Record outcomes over time**
5. **Build history of what works under which conditions**

Playground Features

Table 15.43. *Playground Features*

Feature	Description
Compare Mode	Test multiple prompts side-by-side
Schema Setup	Define expected output structure
Evaluation Criteria	Set up automated scoring
Model Testing	Try same prompt on different models

Prompt Versioning

LangSmith provides prompt management capabilities:

Table 15.44. *Prompt Versioning*

Feature	Benefit
Version Control	Track changes over time
Team Collaboration	Multiple people can work on prompts
Public Prompts	Browse community prompts via LangChain Hub
Hosting	Centralized storage for all prompts

15.29.4 Dataset Creation and Annotation

Purpose

Evaluation requires **standardized datasets**. These can be:

- Publicly available datasets
- Custom datasets for your specific use case

LangSmith provides tools to build and manage datasets.

Creating Datasets

From Existing Data:

- Import CSV, JSON files
- Upload existing evaluation sets

From Traces:

- Select interesting traces from production
- Add them to datasets for future testing

- Build evaluation sets from real user interactions

From Scratch:

- Create empty dataset
- Manually add examples
- Define input/output pairs

Annotation Workflow

Annotation = adding labels to data to indicate quality or correctness.

Example: Customer Chatbot Dataset

Building a dataset for a customer support chatbot:

Table 15.45. *Example: Customer Chatbot Dataset*

Input (Question)	Expected Output	Quality Label
"What are your hours?"	"We're open 9-5 M-F"	Good
"Cancel my order"	"I can help with that..."	Good
"Is product X in stock?"	"Let me check..."	Good

This dataset can test **all future versions** of your chatbot.

Dataset Features

Table 15.46. *Dataset Features*

Feature	Description
Versioning	Track dataset changes over time
Reusability	Use across multiple projects
Annotation Queue	Systematic labeling workflow
Organization	Proper structure and metadata

15.29.5 User Feedback Integration

What It Is

Just like ChatGPT's thumbs up/down feature, LangSmith lets you capture user feedback on LLM responses.

Feedback Types

Table 15.47. *Feedback Types*

Type	Description
Binary	Thumbs up/down
Ratings	1-5 stars
Structured	Specific feedback categories
Text	Free-form comments

How It's Stored

Feedback is **logged alongside traces**, tied to the exact:

- Prompt used
- Model configuration
- State at execution time

This creates a complete picture of what users liked/disliked.

Viewing Feedback

Per Trace: Each trace has a “Feedback” tab showing user responses.

Aggregate: Monitoring dashboard shows feedback scores across all traces.

Why It Matters

User feedback provides:

- **Direct signal** of satisfaction
- **Data for improvement** - understand what works
- **Early warning** - spot degrading performance
- **Prioritization** - focus on frequently disliked responses

15.29.6 Collaboration

Team-First Design

LangSmith is designed for teams working together on LLM applications.

Collaboration Features

Table 15.48. *Collaboration Features*

Feature	Description
Share Links	One-click sharing of any trace
Team Access	Multiple users per organization
Custom Dashboards	Create shareable monitoring views
Prompt Collaboration	Team members can edit prompts
Annotations	Multiple annotators per dataset

Sharing a Trace

1. Open trace in LangSmith
2. Click “Copy Link” button
3. Share link with teammate
4. Teammate sees **exact same trace** on their machine

Custom Dashboards

Create dashboards for:

- Specific metrics your team cares about
- Project-specific views
- Executive summaries

- Engineering deep-dives

15.30 LLMOps Overview

All these features fall under a broader discipline: **LLMOps** (LLM Operations).

15.30.1 LLMOps Definition

LLMOps is the practice of managing the entire lifecycle of LLM applications in production:

Table 15.49. *LLMOps Definition*

Phase	Activities
Build	Develop prompts, integrate models, create workflows
Test	Evaluate outputs, run benchmarks, compare versions
Deploy	Release to production, manage versions
Monitor	Track health, detect issues, alert on anomalies
Improve	Collect feedback, update datasets, refine prompts

15.30.2 LangSmith's Role in LLMOps

LangSmith provides tools for the entire lifecycle:

Table 15.50. *LangSmith's Role in LLMOps*

LLMOps Phase	LangSmith Feature
Build	Prompt Playground, Versioning
Test	Evaluation, Datasets
Deploy	N/A (integration with deployment tools)
Monitor	Monitoring, Alerting
Improve	User Feedback, Annotation

15.31 Best Practices Summary

15.31.1 For LangGraph Integration

1. **Always integrate LangSmith** when building complex workflows
2. **Use @traceable** on node functions for deeper visibility
3. **Add meaningful names** to runs via config
4. **Use tags and metadata** for searchability
5. **Study traces** to understand graph execution flow

15.31.2 For Monitoring

1. **Set up alerts early** - don't wait for problems
2. **Monitor latency trends** - catch degradation before users notice
3. **Track costs** - avoid surprise bills
4. **Review error rates daily**

15.31.3 For Evaluation

1. **Build datasets** from real user interactions
2. **Run evaluations** before every deployment
3. **Compare model versions** systematically
4. **Use multiple evaluation metrics**

15.31.4 For Prompt Engineering

1. **Version all prompts** - never lose a working prompt
2. **A/B test changes** - don't guess, measure
3. **Document conditions** - record when prompts work best
4. **Collaborate** - get team input on prompt design

15.31.5 LangSmith Navigation Quick Reference

Table 15.51. *LangSmith Navigation Quick Reference*

Goal	Navigation Path
View traces	Observability → Tracing Projects → [Project]
Set up monitoring	Observability → Monitoring → [Project]
Create alerts	Monitoring → Alerts → Create
Add evaluators	Tracing Projects → [Project] → Evaluators
Test prompts	Prompt Engineering → Playground
Manage prompts	Prompt Engineering → Prompts
Create datasets	Evaluation → Datasets & Experiments
View feedback	[Trace] → Feedback tab

15.32 Conclusion

LangSmith is much more than just a tracing tool. It's a comprehensive platform for building, testing, monitoring, and improving LLM applications at scale.

Key Takeaways:

1. **LangGraph + LangSmith** = Essential combination for complex workflows
2. **Observability** = Single trace debugging
3. **Monitoring** = Multi-trace health tracking
4. **Evaluation** = Systematic quality measurement
5. **Prompt Experimentation** = Data-driven prompt optimization
6. **Dataset Creation** = Build your evaluation foundation
7. **User Feedback** = Direct signal from production
8. **Collaboration** = Team-first design

As you build more sophisticated LLM applications, these features become increasingly important for maintaining quality, reliability, and team productivity.

LangSmith: LLM Observability and Debugging

Debug • Test • Monitor • Evaluate — Unified Platform for AI Applications

Why Observability Matters: Real-World Scenarios

Scenario 1: Latency Issues

User: "App is slow!"
Without LangSmith: Guessing game

With LangSmith:

- See exact latency per step
- Identify LLM call? Retrieval? Planning?
- Pinpoint bottleneck instantly

Scenario 2: Cost Spike

Boss: "Why is our bill so high?"
Without LangSmith: Check logs manually

With LangSmith:

- Token usage per request
- Cost breakdown by operation
- Find expensive prompts quickly

Scenario 3: Hallucination

User: "AI gave wrong answer!"
Without LangSmith: Reproduce manually

With LangSmith:

- See exact input that caused issue
- View retrieved context (RAG)
- Debug prompt + response chain

Core Concepts: Project → Trace → Run

PROJECT (Container for all traces, e.g., "RAG-App") → **TRACE** (One user request/query, e.g., "What is AI?") → **RUN** (Individual operation, e.g., "embed")

Trace: "What is AI?" → Runs: [embed, retrieve, llm_call]

What LangSmith Traces (Automatically)

Inputs: User queries, prompts | Outputs: LLM responses | Latency: Time per operation | Tokens: Input/output counts | Cost: \$ per request | Errors: Exceptions, failures | Tags: Custom labels | Metadata: User ID, session, etc.

All traced automatically with LangChain integration — zero code changes needed!

Setup and Configuration

Step 1: Get API Key

- Go to smith.langchain.com
- Create account → Settings → API Keys
- Copy your LANGCHAIN_API_KEY

Step 2: Environment Variables

```
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=sk-1234567890abcdef
```

.env file or set in code

```
import os
os.environ["LANGCHAIN_TRACING_V2"] = "true"
```

@traceable Decorator: Custom Function Tracing

Why Use @traceable?

- LangChain auto-traces LLM calls
- Custom functions need explicit tracing
- See your entire pipeline in one trace

```
from langsmith import traceable
@traceable
def my_rag_pipeline(query):
    # All nested calls traced!
```

Trace Hierarchy in LangSmith UI:

```
my_rag_pipeline → embed_query → retrieve_docs → no_results → respond
```

RAG Application Tracing Example

```
@traceable
def rag_pipeline(query: str):
    # Step 1: Embed query
    query_embedding = embeddings.embed_query(query)
    # Step 2: Retrieve relevant docs
    docs = vectordb.similarity_search(query, k=5)
    # Step 3: Generate response
    return llm.invoke(format_prompt(query, docs))
```

Index Caching for Performance

X Problem: Rebuilding index every request
Embedding + indexing = expensive and slow!

```
# Cache index to disk
if os.path.exists("cache_index"):
    vectordb = FAISS.load_index(...)
else:
    vectordb = create_and_save_index()
```

LangGraph Integration

✓ Automatic Tracing
LangGraph traces all nodes, edges, and state transitions

```
# Just set env vars - that's all!
graph = StateGraph(State)
graph.add_node("agent", agent_fn)
# Each node execution = separate run
# Full graph = one trace
```

Additional LangSmith Features

- Monitoring:** Dashboards, alerts, metrics
- Evaluation:** Test LLM output quality
- Prompts:** Version control, A/B test
- Datasets:** Create test sets from traces
- User Feedback:** Collect thumbs up/down
- Collaboration:** Team sharing, annotations

Complete RAG Application with LangSmith Tracing

Full Code Example

```
from langsmith import traceable
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

# Initialize components
llm = ChatOpenAI(model="gpt-4o")
embeddings = OpenAIEmbeddings()

@traceable # This traces the entire pipeline!
def rag_answer(question: str) -> str:
    # Load cached index or create new
    vectordb = load_or_create_index()
    # Retrieve relevant documents
    docs = vectordb.similarity_search(question, k=5)
```

What You See in LangSmith UI

```
Trace rag_answer 2.2s / 1,245 tokens / $0.024
```

- rag_answer: 2.2s / 1,245 tokens / \$0.024
- OpenAIEmbeddings.embed_query: 0.0s
- FAISS.similarity_search: 0.0s
- ChatOpenAI.invoke: 2.2s / 1,245 tokens

Trace Details:
Input: "What is machine learning?"
Output: "Machine learning is a subset of AI that enables..."

Quick Reference: LangSmith Essentials

- Required Env Vars:** LANGCHAIN_TRACING_V2=true
- Dashboard URL:** smith.langchain.com
- Custom Tracing:** @traceable decorator
- Install:** pip install langsmith
- Pro Tip:** Cache indexes for performance

LangSmith Observability Guide • smith.langchain.com • Debug, Test, Monitor, and Evaluate LLM Applications

Figure 15.18. Conclusion

325

LangSmith Observability for LangGraph Chatbot

16.1 Code

- <https://github.com/campusx-official/chatbot-in-langgraph>

16.2 1. Introduction and Context

16.2.1 Journey Till Now (Playlist Recap)

This video is part of an **Agentic AI using LangGraph** playlist. Before adding observability, the chatbot has evolved through several stages:

Previous Features Implemented:

Table 16.1. *Journey Till Now (Playlist Recap)*

Feature	Description	Benefit
GUI	Streamlit-based user interface	User-friendly interaction
Streaming	Real-time token display	No waiting for complete response
Database Persistence	SQLite checkpoint storage	Chats survive restarts, accessible after days

16.2.2 Today's Focus

Adding Observability - The ability to trace and monitor every aspect of chatbot execution using LangSmith.

16.3 2. What is Observability?

16.3.1 Definition in Context

Observability refers to the ability to **understand and monitor what's happening inside a system** by examining its outputs and behaviors. In the context of LLM applications:

Observability = End-to-End Tracing of Chatbot Execution

16.3.2 What Gets Traced?

16.3.3 Practical Meaning

When a user interacts with your chatbot:

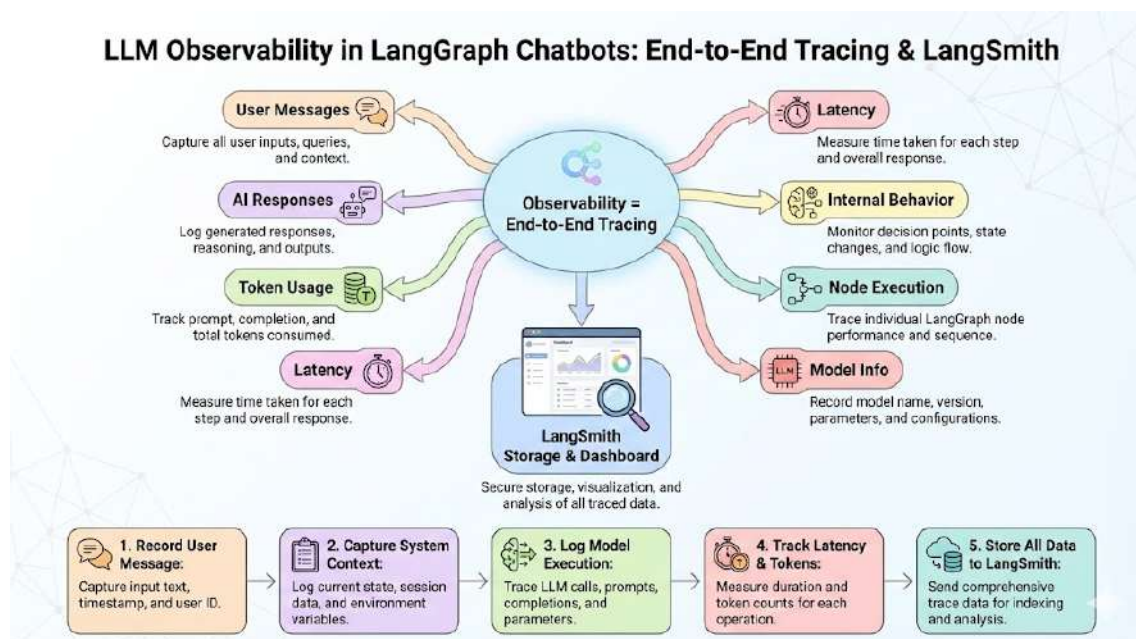


Figure 16.1. What Gets Traced?

1. **Every message sent** by the user is recorded
2. **Every response** from the AI is captured
3. **Performance metrics** (time, tokens) are logged
4. **Internal workings** (which nodes executed, which models used) are visible

All this information is stored in **LangSmith**, a software platform for LLM observability.

16.4 3. Why LangSmith for Observability?

16.4.1 The Tool: LangSmith

LangSmith is developed by the same team behind LangChain and LangGraph, making it the natural choice for observability in LangGraph applications.

16.4.2 Key Benefits

Table 16.2. Key Benefits

Benefit	Description
Zero Code Changes	Once configured, LangSmith automatically traces everything
Comprehensive Tracing	Captures inputs, outputs, tokens, latency, and more
Organized Dashboard	Projects → Threads → Traces hierarchy
Debugging Support	Helps identify issues in complex workflows
Production Ready	Essential for monitoring deployed applications

16.4.3 When Observability Becomes Critical

Observability becomes increasingly valuable when:

- Adding **Tools** to your chatbot
- Implementing **RAG** (Retrieval-Augmented Generation)

- Working with **MCP** (Model Context Protocol)
- Debugging **complex multi-step workflows**
- Running in **production environments**

16.5 4. Prerequisites

16.5.1 Required Knowledge

Before proceeding, you should understand:

1. Basic LangGraph concepts
2. Chatbot development with LangGraph
3. Threading concept in LangGraph
4. Database persistence in LangGraph

16.5.2 Recommended Prior Viewing

The instructor recommends watching the **LangSmith Crash Course** (2-hour video) for:

- Deep understanding of observability concepts
- Detailed LangSmith tool usage
- Additional features like monitoring, datasets, experiments, prompts, and playground

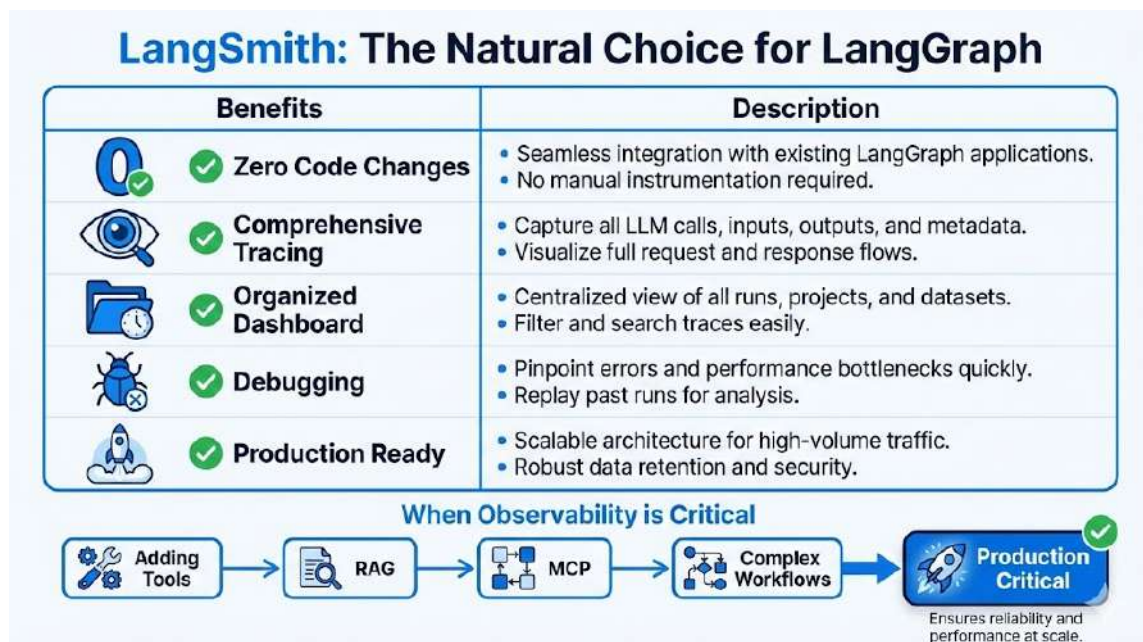


Figure 16.2. Recommended Prior Viewing

16.6 5. LangSmith Setup Guide

16.6.1 Step 1: Create LangSmith Account

1. Navigate to <https://smith.langchain.com>
2. Create a new account or log in to existing account

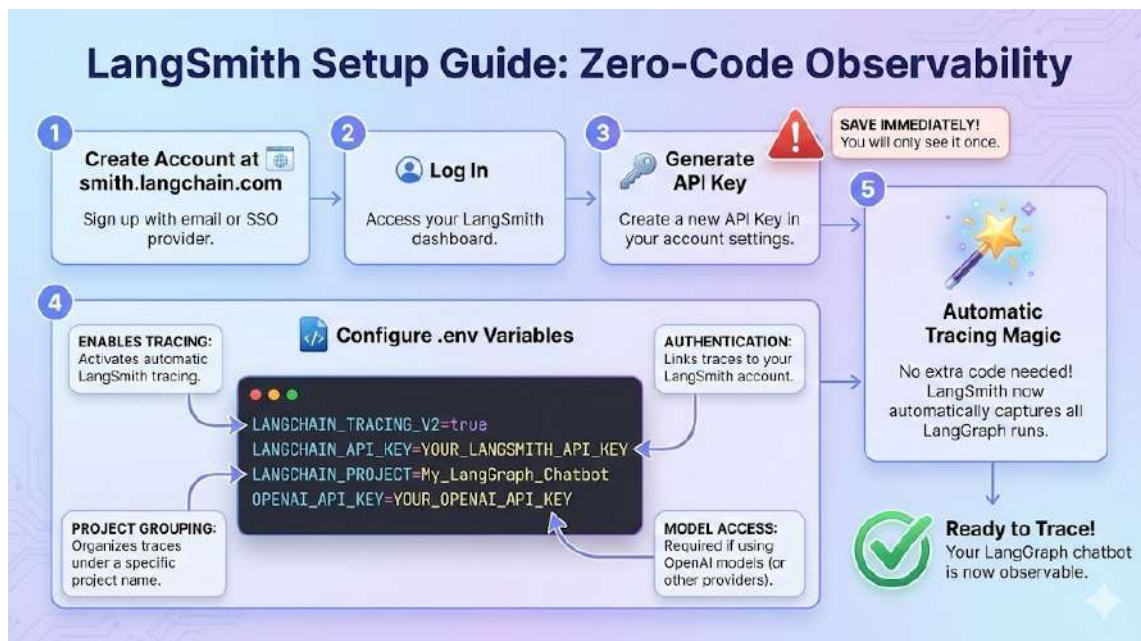


Figure 16.3. Step 2: Generate API Key

16.6.2 Step 2: Generate API Key

Important: Copy and save your API key immediately - you won't be able to see it again!

16.6.3 Step 3: Configure Environment Variables

Add the following variables to your `.env` file:

```
# Enable LangSmith Tracing
LANGCHAIN_TRACING_V2=true

# LangSmith API Endpoint
LANGCHAIN_ENDPOINT='https://api.smith.langchain.com'

# Your LangSmith API Key (replace with your actual key)
LANGCHAIN_API_KEY='lsv2_pt_your_api_key_here'

# Project Name (creates a project in LangSmith)
LANGCHAIN_PROJECT='Chatbot Project'
```

16.6.4 Environment Variables Explained

Table 16.3. Environment Variables Explained

Variable	Purpose	Value
LANGCHAIN_TRACING_V2	Enables/disables tracing	true
LANGCHAIN_ENDPOINT	LangSmith API URL	https://api.smith.langchain.com
LANGCHAIN_API_KEY	Authentication key	Your generated API key
LANGCHAIN_PROJECT	Project name in LangSmith	Custom name (e.g., "Chatbot Project")

16.6.5 The Magic of Zero Code Changes

Once these environment variables are set, **LangSmith automatically starts tracing** your LangGraph application. No modifications to your main code are required!

16.7 6. Understanding LangSmith Architecture

16.7.1 Hierarchical Organization

LangSmith organizes data in a three-level hierarchy:

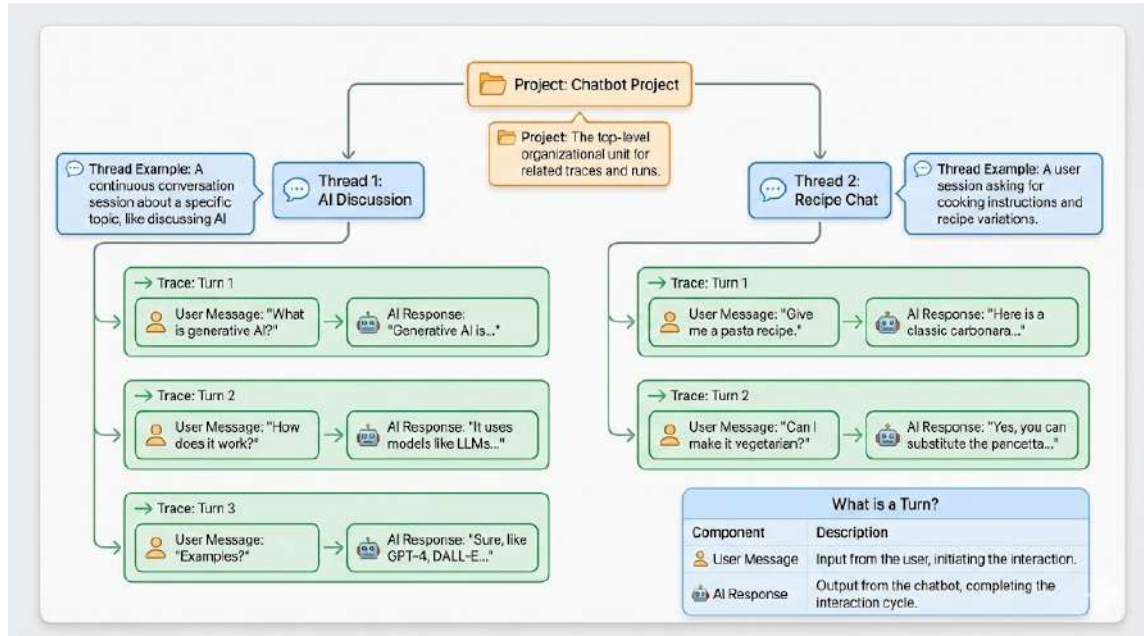


Figure 16.4. Hierarchical Organization

16.7.2 Terminology

Table 16.4. Terminology

Term	Definition	Example
Project	Top-level container for all tracing data	"Chatbot Project"
Thread	A single conversation/session	User's chat session
Trace	One turn of conversation	User asks, AI responds

16.7.3 What is a "Turn"?

A **turn** represents one complete interaction cycle:

Turn = User Message → AI Response

Each turn generates one trace in LangSmith.

16.8 7. Basic Tracing Implementation

16.8.1 Automatic Tracing

With environment variables configured, tracing happens automatically:

```
# No special code needed - just run your chatbot normally
# LangSmith captures everything behind the scenes
```

16.8.2 What Gets Captured in Each Trace

When you view a trace in LangSmith, you'll see:

Table 16.5. *What Gets Captured in Each Trace*

Information	Description
Node Name	The LangGraph node that executed (e.g., "chat_node")
LLM Model	The model used (e.g., "ChatOpenAI")
Input	User's message
Output	AI's response
Execution Start	When processing began
Execution End	When processing completed
First Token Time	Time to first token (for streaming)
Status	Success/failure status
Total Tokens	Input tokens + Output tokens
Latency	Total response generation time

16.9 8. The Thread Organization Problem

16.9.1 The Issue

With basic setup, **all traces from all conversations are stored together** without organization:

16.9.2 Why This is Problematic

1. **No Conversation Context:** Can't see which messages belong to which conversation
2. **Difficult to Debug:** Hard to trace issues in specific user sessions
3. **Poor Organization:** Different conversations mixed randomly
4. **Lacks Logical Grouping:** No separation between users or sessions

16.9.3 The Expected Behavior

Different conversations (threads) should be **separately organized**:

16.10 9. Implementing Thread-Based Organization

16.10.1 The Solution

LangSmith provides a feature to organize traces into threads. To enable this, you need to **explicitly pass the thread_id in the metadata**.

16.10.2 Accepted Identifiers

LangSmith accepts any of these identifiers:

- thread_id
- session_id
- conversation_id

16.10.3 Code Modification Required

Original CONFIG (without thread organization):

```
CONFIG = {
    'configurable': {
        'thread_id': st.session_state['thread_id']
    }
}
```

New CONFIG (with thread organization):

```
CONFIG = {
    "configurable": {
        "thread_id": st.session_state["thread_id"]
    },
    "metadata": {
        "thread_id": st.session_state["thread_id"]
    },
    "run_name": "chat_turn",
}
```

16.10.4 Understanding the Changes

Table 16.6. *Understanding the Changes*

Component	Purpose	Required?
configurable.thread_id	For LangGraph's internal threading	Yes (existing)
metadata.thread_id	For LangSmith's thread organization	Yes (new)
run_name	Custom name for traces in dashboard	Optional

16.10.5 The run_name Parameter

Without run_name:

- Traces appear as “LangGraph” in the dashboard
- Not very informative

With run_name: “chat_turn”:

- Traces appear as “chat_turn”
- Clearly indicates each trace represents one conversation turn

16.11 10. Complete Code Implementation

16.11.1 Full Streamlit Frontend Code

```
import streamlit as st
from langgraph_database_backend import chatbot, retrieve_all_threads
from langchain_core.messages import HumanMessage
import uuid

# ===== UTILITY FUNCTIONS =====

def generate_thread_id():
    """Generate a unique thread ID using UUID4."""
    thread_id = uuid.uuid4()
    return thread_id
```

```

def reset_chat():
    """Reset chat by creating a new thread and clearing message history."""
    thread_id = generate_thread_id()
    st.session_state['thread_id'] = thread_id
    add_thread(st.session_state['thread_id'])
    st.session_state['message_history'] = []

def add_thread(thread_id):
    """Add a new thread to the list of chat threads if not already present."""
    if thread_id not in st.session_state['chat_threads']:
        st.session_state['chat_threads'].append(thread_id)

def load_conversation(thread_id):
    """Load conversation history for a specific thread from the chatbot state."""
    state = chatbot.get_state(config={'configurable': {'thread_id': thread_id}})
    # Check if messages key exists in state values, return empty list if not
    return state.values.get('messages', [])

# ===== SESSION SETUP =====

# Initialize message history if not present
if 'message_history' not in st.session_state:
    st.session_state['message_history'] = []

# Initialize thread ID if not present
if 'thread_id' not in st.session_state:
    st.session_state['thread_id'] = generate_thread_id()

# Initialize chat threads list from database
if 'chat_threads' not in st.session_state:
    st.session_state['chat_threads'] = retrieve_all_threads()

# Add current thread to the list
add_thread(st.session_state['thread_id'])

# ===== SIDEBAR UI =====

st.sidebar.title('LangGraph Chatbot')

# New Chat button - creates a fresh conversation
if st.sidebar.button('New Chat'):
    reset_chat()

# Display all conversation threads
st.sidebar.header('My Conversations')
for thread_id in st.session_state['chat_threads'][::-1]: # Reverse for newest first
    if st.sidebar.button(str(thread_id)):
        # Switch to selected thread
        st.session_state['thread_id'] = thread_id
        messages = load_conversation(thread_id)

        # Convert messages to display format
        temp_messages = []
        for msg in messages:
            if isinstance(msg, HumanMessage):
                role = 'user'
            else:
                role = 'assistant'
            temp_messages.append({'role': role, 'content': msg.content})

        st.session_state['message_history'] = temp_messages

# ===== MAIN UI =====

# Display conversation history
for message in st.session_state['message_history']:
    with st.chat_message(message['role']):
        st.text(message['content'])

# Chat input field
user_input = st.chat_input('Type here')

if user_input:

```

```

# Add user message to history and display
st.session_state['message_history'].append({
    'role': 'user',
    'content': user_input
})

with st.chat_message('user'):
    st.text(user_input)

# =====
# KEY CHANGE: CONFIG with metadata for LangSmith threads
# =====
CONFIG = {
    "configurable": {
        "thread_id": st.session_state["thread_id"]
    },
    "metadata": {
        "thread_id": st.session_state["thread_id"] # For LangSmith
    },
    "run_name": "chat_turn", # Custom name for traces
}

# Stream AI response
with st.chat_message('assistant'):
    ai_message = st.write_stream(
        message_chunk.content
        for message_chunk, metadata in chatbot.stream(
            {'messages': [HumanMessage(content=user_input)]},
            config=CONFIG,
            stream_mode='messages'
        )
    )

# Add AI response to history
st.session_state['message_history'].append({
    'role': 'assistant',
    'content': ai_message
})

```

16.11.2 Key Code Section Explained

The critical change is in the CONFIG dictionary:

```

CONFIG = {
    "configurable": {
        "thread_id": st.session_state["thread_id"]
    },
    "metadata": {
        "thread_id": st.session_state["thread_id"]
    },
    "run_name": "chat_turn",
}

```

16.12 11. LangSmith Dashboard Navigation

16.12.1 Accessing Your Traces

1. Log in to smith.langchain.com
2. Navigate to **Tracing Projects**
3. Find your project (e.g., “Chatbot Project”)
4. Click to view traces

16.12.2 Traces View vs Threads View

Table 16.7. *Traces View vs Threads View*

View	Shows	Best For
Traces	All traces in chronological order	Quick overview of all activity
Threads	Traces grouped by conversation	Understanding user journeys

16.12.3 Trace Details Information

When clicking on a specific trace, you'll see:

Table 16.8. *Trace Details Information*

Field	Description	Example
Node Name	LangGraph node that processed	"chat_node"
Model	LLM model used	"ChatOpenAI"
Input	User's message	"Give me a roadmap to study AI"
Output	AI's complete response	Full response text
Execution Start	Timestamp	2025-08-19 17:45:23
Execution End	Timestamp	2025-08-19 17:45:28
First Token Time	Time to first token	0.8s
Status	Result	Success
Input Tokens	Tokens in prompt	150
Output Tokens	Tokens in response	450
Total Tokens	Sum of both	600
Latency	Response time	5.2s

16.13 12. Benefits and Use Cases

16.13.1 Current Benefits

1. **Full Visibility:** See exactly what users are asking and getting
2. **Performance Monitoring:** Track latency and token usage
3. **Debugging:** Identify issues in specific conversations
4. **Historical Record:** Access past conversations anytime

16.13.2 Production Use Cases

Table 16.9. *Production Use Cases*

Use Case	How LangSmith Helps
Cost Management	Monitor token usage across users
Performance Optimization	Identify slow responses
Quality Assurance	Review AI responses for accuracy
User Behavior Analysis	Understand what users ask about
Error Tracking	Catch and diagnose failures

16.14 13. Quick Reference

16.14.1 Environment Variables Checklist

```
# Add to .env file
LANGCHAIN_TRACING_V2=true
LANGCHAIN_ENDPOINT='https://api.smith.langchain.com'
LANGCHAIN_API_KEY='your_api_key_here'
LANGCHAIN_PROJECT='Your Project Name'
```

16.14.2 CONFIG Template for Thread Organization

```
CONFIG = {
  "configurable": {
    "thread_id": your_thread_id
  },
  "metadata": {
    "thread_id": your_thread_id # Required for LangSmith threads
  },
  "run_name": "chat_turn", # Optional but recommended
}
```

16.14.3 LangSmith URLs

Table 16.10. *LangSmith URLs*

Resource	URL
Dashboard	https://smith.langchain.com
API Endpoint	https://api.smith.langchain.com
Documentation	https://docs.smith.langchain.com

16.15 Summary

Adding observability to your LangGraph chatbot using LangSmith provides:

1. **Automatic Tracing:** Once configured, no code changes needed for basic tracing
2. **Comprehensive Monitoring:** Captures inputs, outputs, tokens, latency, and more
3. **Thread Organization:** With metadata configuration, conversations are properly grouped
4. **Future-Proof:** Essential foundation for advanced features like tools, RAG, and MCP

The key implementation change is adding `metadata.thread_id` to your `CONFIG` when invoking the chatbot, which enables LangSmith to organize traces by conversation threads.

16.16 Additional Resources

- **LangSmith Crash Course:** Detailed 2-hour video covering all LangSmith features
- **LangSmith Documentation:** <https://docs.smith.langchain.com>
- **LangGraph Documentation:** <https://langchain-ai.github.io/langgraph/>

LangSmith Observability for LangGraph Chatbot

End-to-End Tracing • Thread Organization • Zero Code Changes • Production Ready

Chatbot Journey So Far

- Streamlit Interface
- Streaming (Realtime Token Chipping)
- SQLite Persistence (Chat Service Rebuild)
- TODAY: Observability** (LangSmith Tracing)
- Observability = End-to-End Tracing of Chatbot Execution

What Gets Traced?

- Messages** (User inputs, AI responses)
- Performance** (Latency, Token counts)
- Internals** (Nodes executed, Models used)

Why LangSmith?

- Same team as LangChain/LangGraph
- Zero code changes for basic tracing
- Essential for Tools, RAG, MCP
- Production monitoring ready

LangSmith Hierarchy

PROJECT (Crossed Thread) → THREAD (Conversations) → TRACE (One Step)

What is a "Turn"?

Turn = User Message + AI Response

LangSmith Setup Guide

- Create Account** (smith.langchain.com, Sign up or log in)
- Generate API Key** (Settings → API Keys, Copy immediately)
- Configure env** (Add environment vars, See code below)

```

LANGSMITH_TRACING_ID = "trace"
LANGSMITH_ENVIRONMENT = "https://api.smith.langchain.com"
LANGSMITH_API_KEY = "lsd_jwt_your_key_here"
LANGSMITH_PROJECT = "Observability"

```

What Gets Captured in Each Trace

Node Name	LLM Model	Input/Output	Status
Execution Time	First Token Time	Total Tokens	Latency

🔮 The Magic of Zero Code Changes

Once environment variables are set, LangSmith automatically starts tracing your LangGraph application. No modifications to your main code required!

✗ The Thread Organization Problem

Without Thread Organization

```

Trace 1: "hello" (user: A)
Trace 2: "hello is ai?" (user: A)
Trace 3: "hello" (user: A)
Trace 4: "help me" (user: A)
Trace 5: "help me!" (user: A)

```

Why This is Problematic:

- No conversation context
- Difficult to debug specific sessions
- Different conversations mixed randomly
- Can't trace user journeys
- Poor organization for production

✓ The Solution: Thread-Based Organization

With Thread Organization

```

Thread: user-a-welc
  - "hello" → "hi, there!"
  - "hello is ai?" → "No, I'm not!"
  - "help me!" → "I'll help you!"
Thread: user-b-welc
  - "hello is ai?" → "No, I'm not!"

```

The Key: Pass thread_id in metadata

LangSmith accepts:

```

{
  "thread_id": "user-a-welc",
  "conversation_id": "user-a-welc"
}

```

The Critical Code Change: CONFIG with Metadata

✗ Before (No Thread Organization)

```

# Original CONFIG
CONFIG = {
  "configurable": {
    "thread_id": thread_id
  }
}

```

✓ After (With Thread Organization)

```

CONFIG = {
  "configurable": {
    "thread_id": tid,
    "metadata": {
      "thread_id": tid # For LangSmith!
    },
    "run_name": "chat_turn" # Optional
  }
}

```

Understanding the Changes

- `configurable.thread_id` For LangSmith thread tracing
- `metadata.thread_id` For LangSmith thread organization
- `run_name` Custom name for trace (optional)

Complete Streamlit Implementation

```

# Chat input handling with LangSmith tracing
user_input = st.chat_input("Type here")

if user_input:
    # Add user message to history
    st.session_state["message_history"].append(...)

    # KEY: CONFIG with metadata for LangSmith
    CONFIG = {
      "configurable": {
        "thread_id": st.session_state["thread_id"],
        "metadata": {
          "thread_id": st.session_state["thread_id"],
          "run_name": "chat_turn"
        }
      }
    }

    # Stream AI response
    with st.chat_message("assistant"):
        ai_message = st.write_stream(
            chatbot_streamer(
                messages = [user_message,...],

```

LangSmith Dashboard View

Chatbot Project [Trace] [Threads]

Threads View (Organized by Conversation)

Thread: 4f7ac18b-58cc-4372-a687-...

```

chat_turn: "hello" → "Hi! How can I help?" 1.2s
chat_turn: "What is AI?" → "No, I'm not!" 2.5s

```

Thread: 87b2b3d4-af56-7888-...

```

chat_turn: "Give me a reading!" → "Here's a reading..." 3.5s

```

Trace Details (click any trace):

- Node: chat_node (Model: ChatOpenAI)
- Input Tokens: 100 | Output Tokens: 400 | Total: 500
- Latency: 2.5s | First Token: 1.5s | Status: ✓ Success
- Full Input/Output text visible for debugging

Benefits and Production Use Cases

- Full Visibility** (See what users ask and what they get)
- Performance** (Track latency and token usage)
- Cost Control** (Monitor token usage per user)
- Debugging** (Identify issues in specific sessions)

Quick Reference

Environment Variables Checklist:

- ✓ LANGSMITH_TRACING_ID=trace
- ✓ LANGSMITH_API_KEY=your_api_key
- ✓ LANGSMITH_PROJECT=trace

LangSmith URLs:

- Dashboard: smith.langchain.com
- API: api.smith.langchain.com
- Docs: docs.smith.langchain.com

Summary: Adding Observability to Your LangGraph Chatbot

- Set 4 environment variables
- Add metadata.thread_id to CONFIG
- Optional: Add run_name
- View organized traces

LangSmith Observability for LangGraph Chatbot • github.com/camptocamp/official-chatbot-in-langgraph • Camptocamp, Toulouse

Figure 16.5. Additional Resources

Tools in LangGraph: Comprehensive Guide

17.1 Code

- <https://github.com/campusx-official/chatbot-in-langgraph>

17.2 1. Introduction

This guide covers how to add **Tools** to a LangGraph-based chatbot. Tools are capabilities that allow your chatbot to go beyond simple conversation and actually **perform actions** on behalf of users.

What we're building: A chatbot that can:

- Have normal conversations (Q&A, general chat)
- Perform mathematical calculations using a Calculator tool
- Search the internet using DuckDuckGo Search tool
- Fetch real-time stock prices using a Stock Price API tool

Prerequisites:

- Basic understanding of LangGraph (StateGraph, nodes, edges)
- Familiarity with LangChain concepts
- Understanding of what tools and tool calling are in LLMs

17.3 2. Understanding the Problem

17.3.1 The Limitation of Basic Chatbots

A standard chatbot connected to an LLM (like OpenAI's GPT) can only:

- Answer questions from its training data
- Engage in conversation
- Generate text based on patterns it learned

What it CANNOT do:

- Perform real-time calculations with guaranteed accuracy
- Search the internet for current information
- Access external APIs or databases
- Execute any real-world actions

17.3.2 Example Scenarios

Table 17.1. *Example Scenarios*

Query	Can Basic Chatbot Handle?	Why?
"What is the capital of India?"	✓ Yes	Static knowledge
"What is 654×713 ?"	! Unreliable	LLMs can make arithmetic errors
"What's today's top news in India?"	✗ No	Requires internet access
"What's Tesla's current stock price?"	✗ No	Requires API call

17.3.3 The Solution: Tools

By adding tools to your chatbot, you enable it to:

1. **Recognize** when a user's request requires an action
2. **Select** the appropriate tool for the task
3. **Execute** the tool with correct parameters
4. **Return** refined results to the user

17.4 3. What are Tools?

A **Tool** in the context of LLMs and LangGraph is a function that the language model can call to perform a specific action. Think of tools as the "hands and eyes" of your AI—they allow it to interact with the real world.

17.4.1 Tool Anatomy

Every tool has three essential components:

TOOL		
1. NAME	Unique identifier	
2. DESCRIPTION	What the tool does (LLM reads)	
3. FUNCTION	Actual code that executes	

Critical Point: The description (docstring) is extremely important because the LLM reads it to decide which tool to use for a given task.

17.4.2 How Tool Calling Works

17.5 4. Core Concepts

17.5.1 4.1 ToolNode

Definition: > In LangGraph, a ToolNode is a **prebuilt node type** that acts as a bridge between your graph and external tools (functions, APIs, utilities).

Key Characteristics of ToolNode:

Table 17.2. *Key Characteristics of ToolNode*

Feature	Description
Prebuilt	You don't write it yourself; LangGraph provides it
Ready-made	Knows how to handle a list of LangChain tools

Feature	Description
Automatic Routing	Automatically routes requests to the correct tool
Listens for Tool Calls	Monitors when the LLM requests a tool execution

What ToolNode Does:

1. **Listens** for tool calls from the LLM
2. **Identifies** which tool needs to be executed
3. **Extracts** the input parameters from the LLM's request
4. **Executes** the appropriate tool
5. **Returns** the tool's output back to the graph

How ToolNode Differs from Regular Nodes:

```
# Regular Node - You write the function
def my_custom_node(state):
    # Your custom logic
    return {"messages": [...]}

# ToolNode - Prebuilt, just pass your tools
tool_node = ToolNode(tools=[tool1, tool2, tool3])
```

ToolNode as Tool Executor:

Think of ToolNode as the **Tool Executor** in your system. It's the place where:

- All your tools reside
- Tool execution happens
- Results are collected

17.5.2 4.2 tools_condition

Definition: > `tools_condition` is a **prebuilt conditional edge function** that helps your graph decide: "Should the flow go to the ToolNode next, or to the END node?"

Purpose of `tools_condition`:

The `tools_condition` function examines the LLM's response and determines:

- Did the LLM request a tool call? → Route to ToolNode
- Did the LLM provide a final answer? → Route to END

Decision Logic:

How It Works Internally:

```
# Simplified conceptual view of what tools_condition does
def tools_condition(state):
    last_message = state["messages"][-1]

    if hasattr(last_message, "tool_calls") and last_message.tool_calls:
        return "tools" # Route to ToolNode
    else:
        return END # Route to END node
```

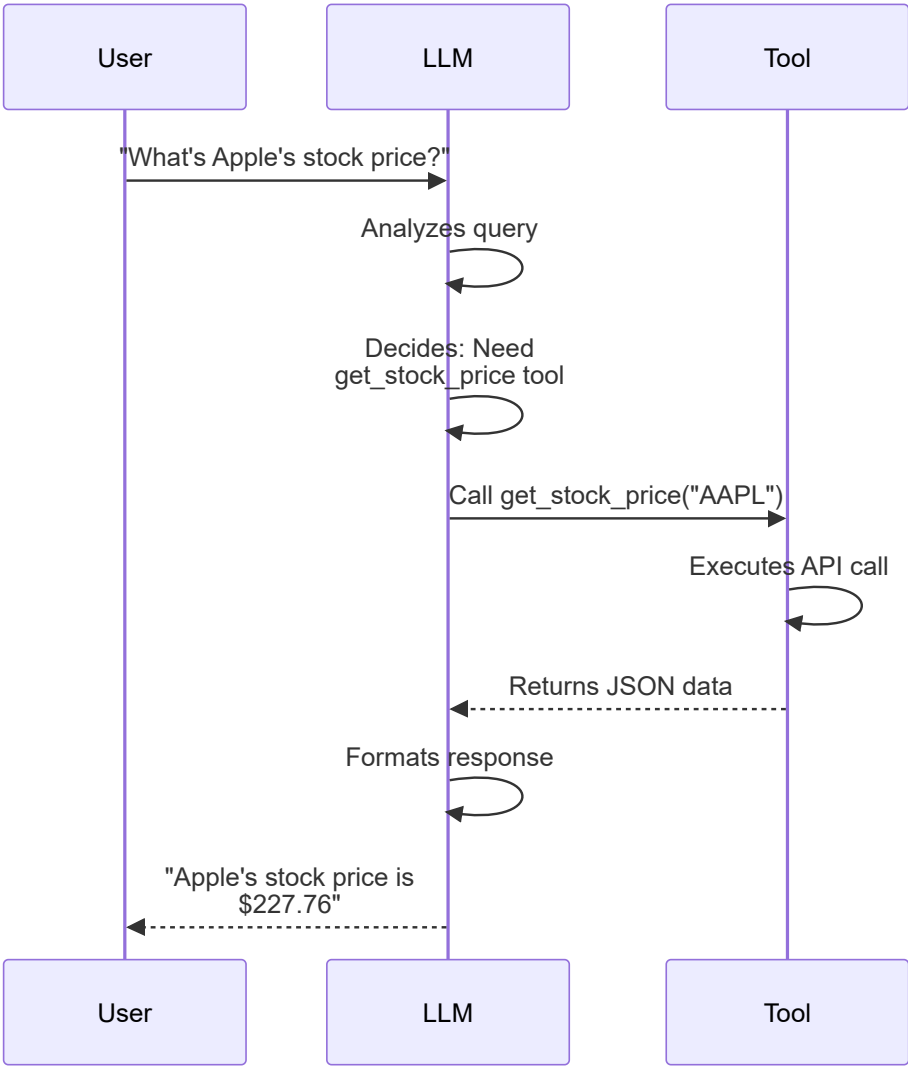


Figure 17.1. How Tool Calling Works

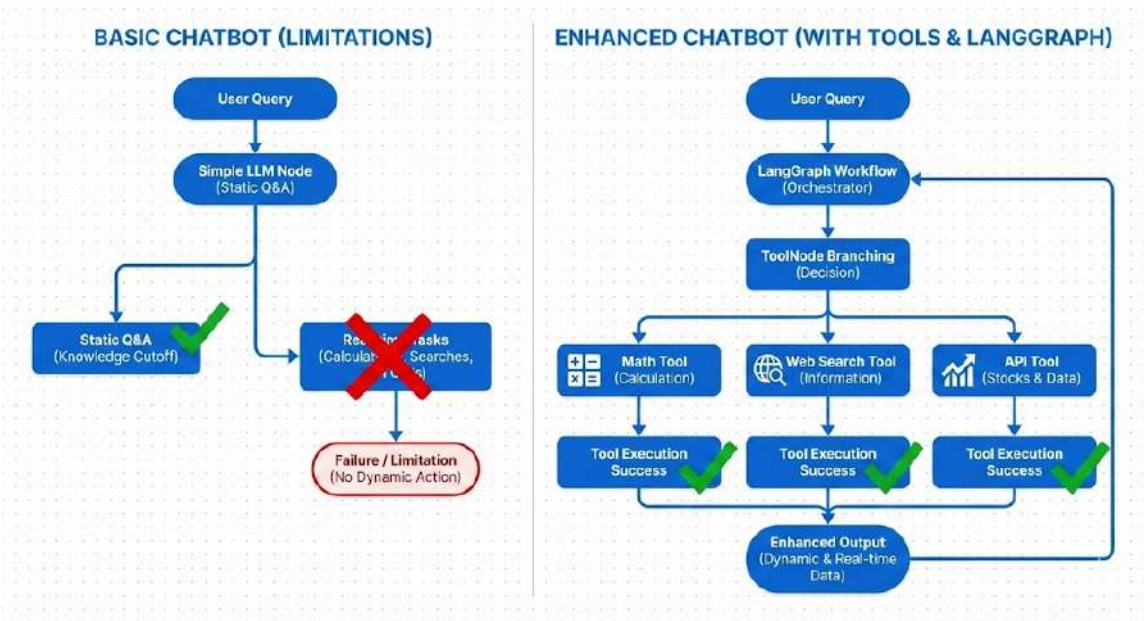


Figure 17.2. ToolNode as Tool Executor

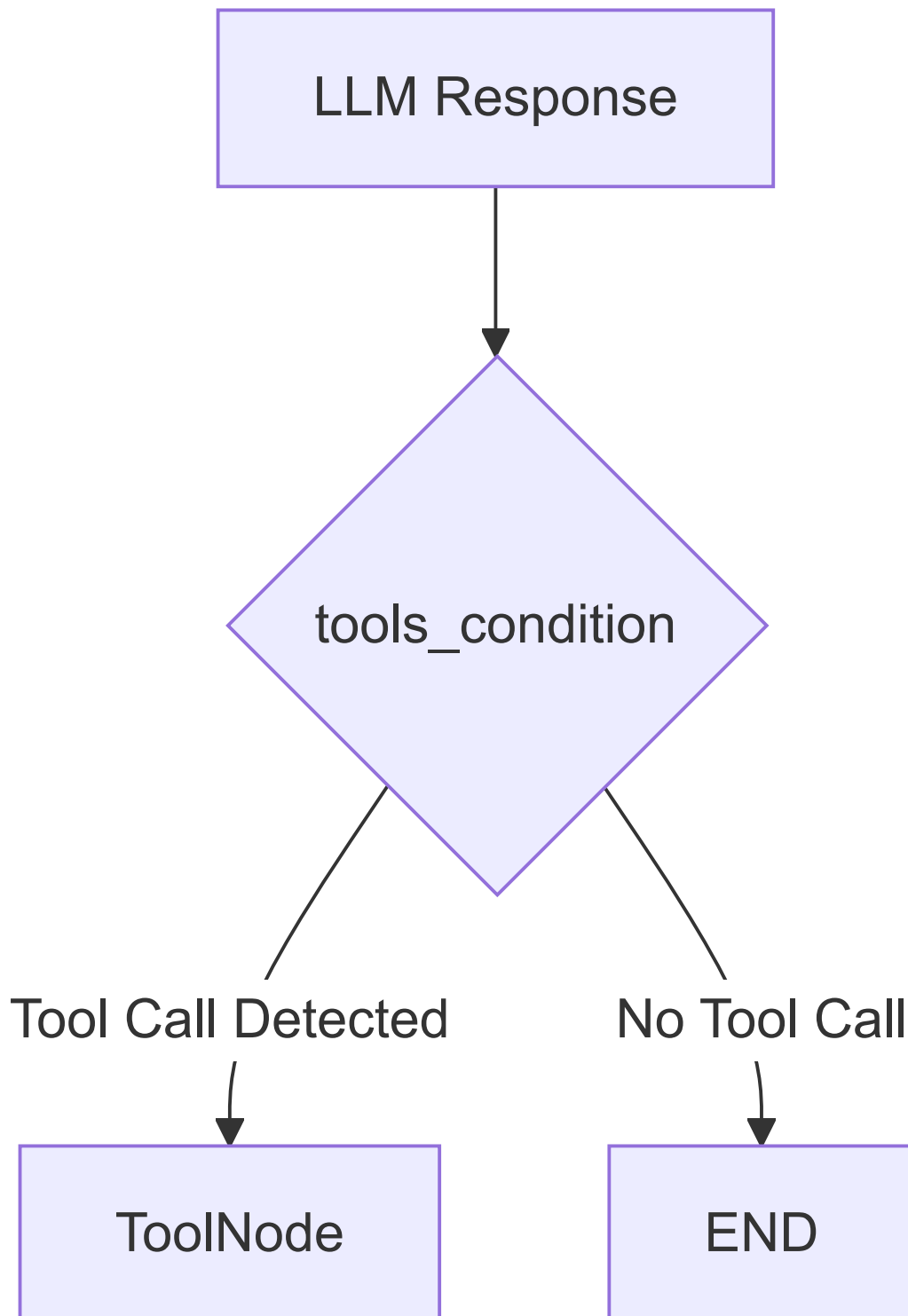


Figure 17.3. *Decision Logic*

17.6 5. Architecture and Workflow

17.6.1 5.1 Simple Workflow (Without Tools)

This is the most basic LangGraph workflow where the chatbot only handles conversations.

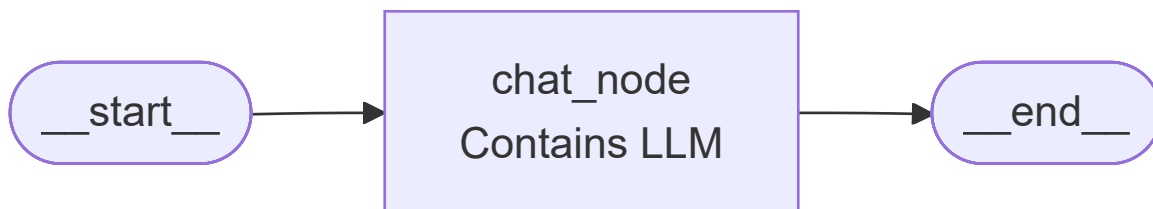


Figure 17.4. 1 Simple Workflow (Without Tools)

Flow:

1. User sends a question: "What is the capital of India?"
2. Question goes to chat_node (LLM)
3. LLM generates answer: "New Delhi"
4. Workflow ends

Limitations:

- Cannot perform actions
- Cannot access external data
- Limited to LLM's training knowledge

17.6.2 5.2 Enhanced Workflow (With Tools)

When tools are added, the workflow becomes more sophisticated with decision-making capabilities.

What Changed:

1. The chat_node now includes **decision-making logic**
2. A **conditional edge** (tools_condition) determines the path
3. A **ToolNode** handles all tool executions

17.6.3 5.3 The Feedback Loop Pattern

Important: The workflow shown above has a critical flaw. Let's understand why.

Problem with Direct Tool → END Flow:

Issues:

1. **Unpolished Output:** Tool outputs are often technical (JSON, raw data) and not user-friendly
2. **No Multi-Step Reasoning:** Cannot chain multiple tool calls together
3. **No Refinement:** LLM cannot interpret or summarize tool results

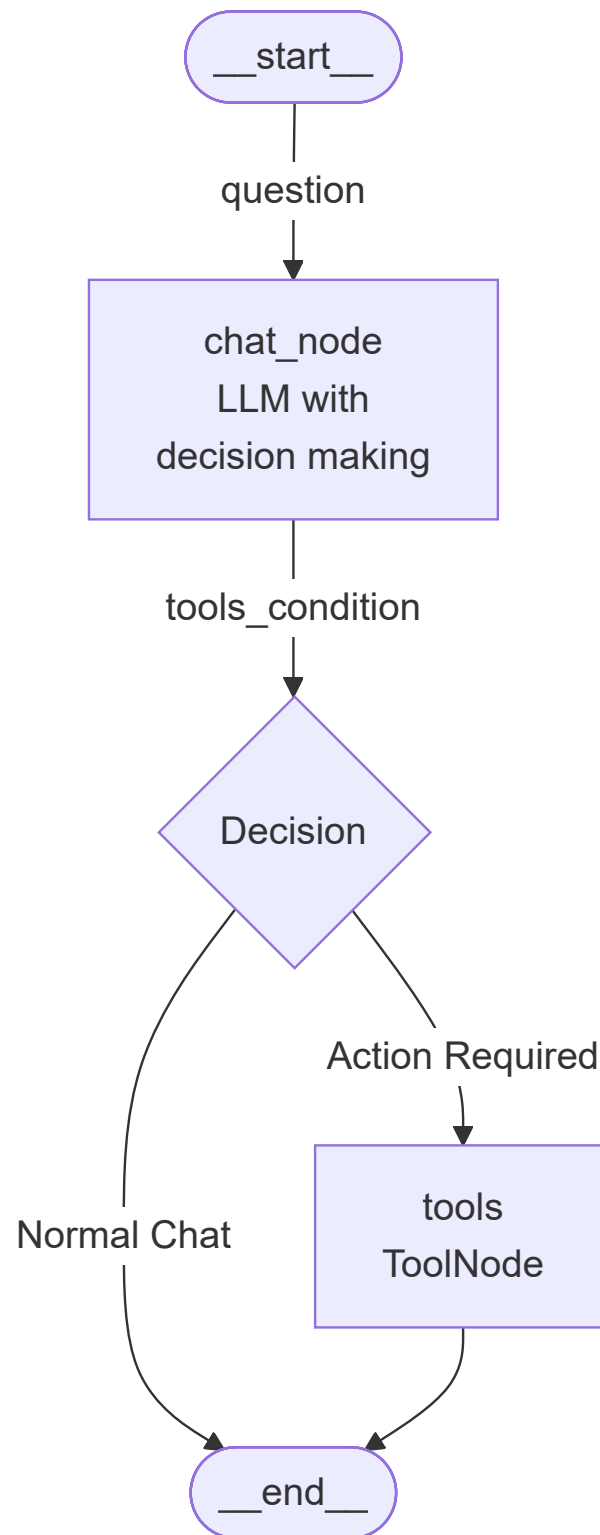


Figure 17.5. 2 Enhanced Workflow (With Tools)

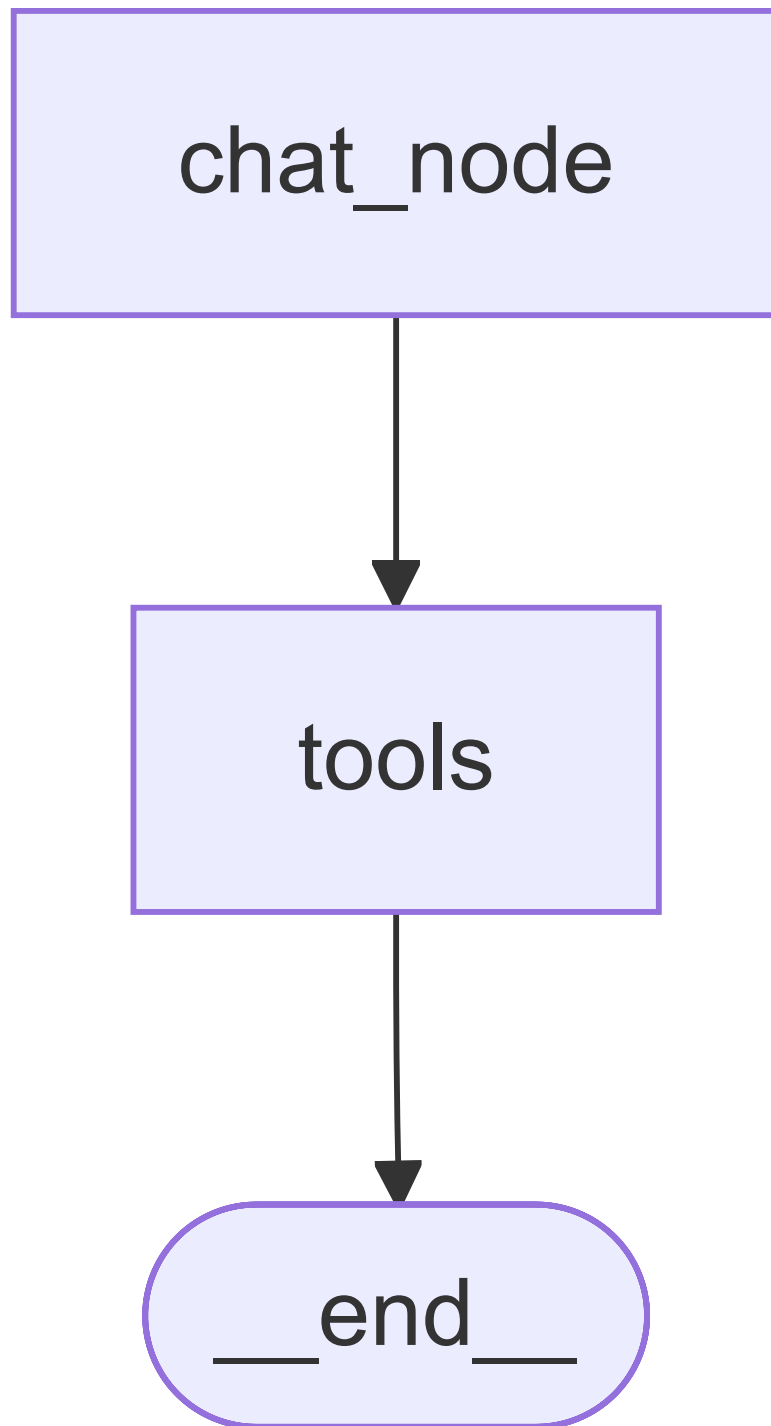


Figure 17.6. Problem with Direct Tool $\rightarrow$ END Flow

Example of the Problem:

User asks: “What’s Apple’s stock price?”

Table 17.3. Problem with Direct Tool $\rightarrow$ END Flow

Without Loop	With Loop
Raw JSON output with technical data	“Apple’s current stock price is \$227.76”

The Correct Pattern: Feedback Loop

The solution is to always route tool output **back to the chat_node** for refinement.

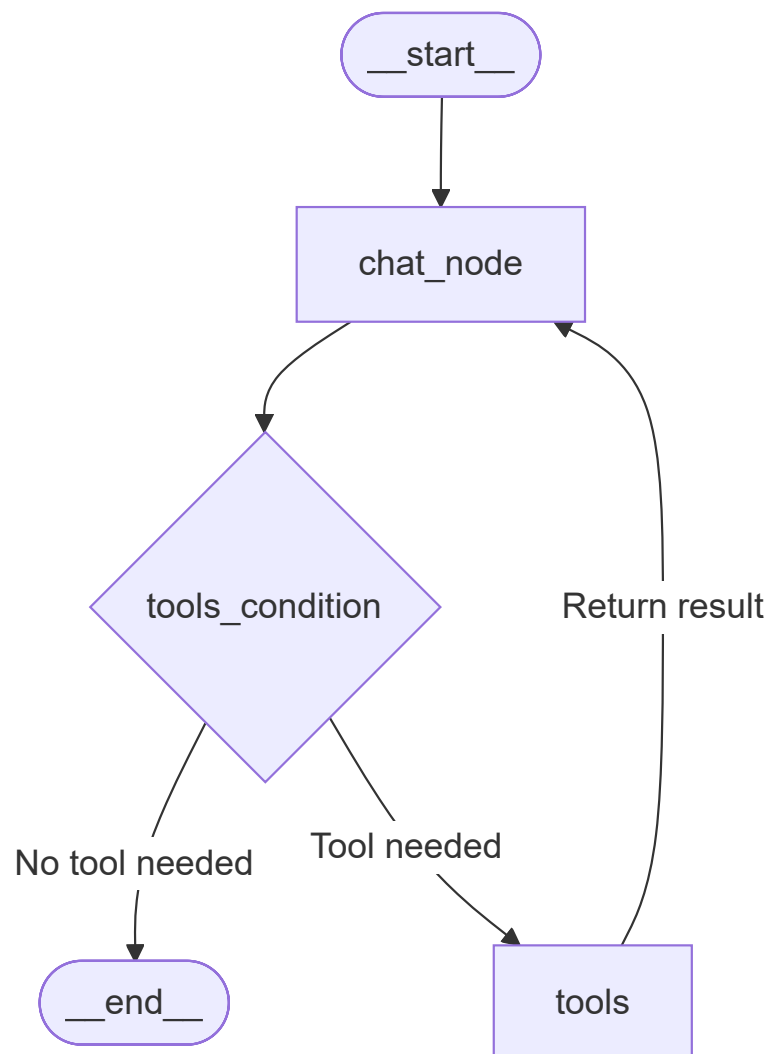


Figure 17.7. The Correct Pattern: Feedback Loop

Why This Works:

1. **Step 1:** User asks “What’s Apple’s stock price and how much for 50 shares?”
2. **Step 2:** chat_node decides → needs stock price tool
3. **Step 3:** ToolNode executes `get_stock_price(“AAPL”)`
4. **Step 4:** Result returns to chat_node
5. **Step 5:** chat_node analyzes → now needs calculator tool

6. **Step 6:** ToolNode executes calculator(price, 50, "mul")
7. **Step 7:** Result returns to chat_node
8. **Step 8:** chat_node has all info → generates refined response
9. **Step 9:** Flow goes to END

Multi-Step Tool Usage Flow:

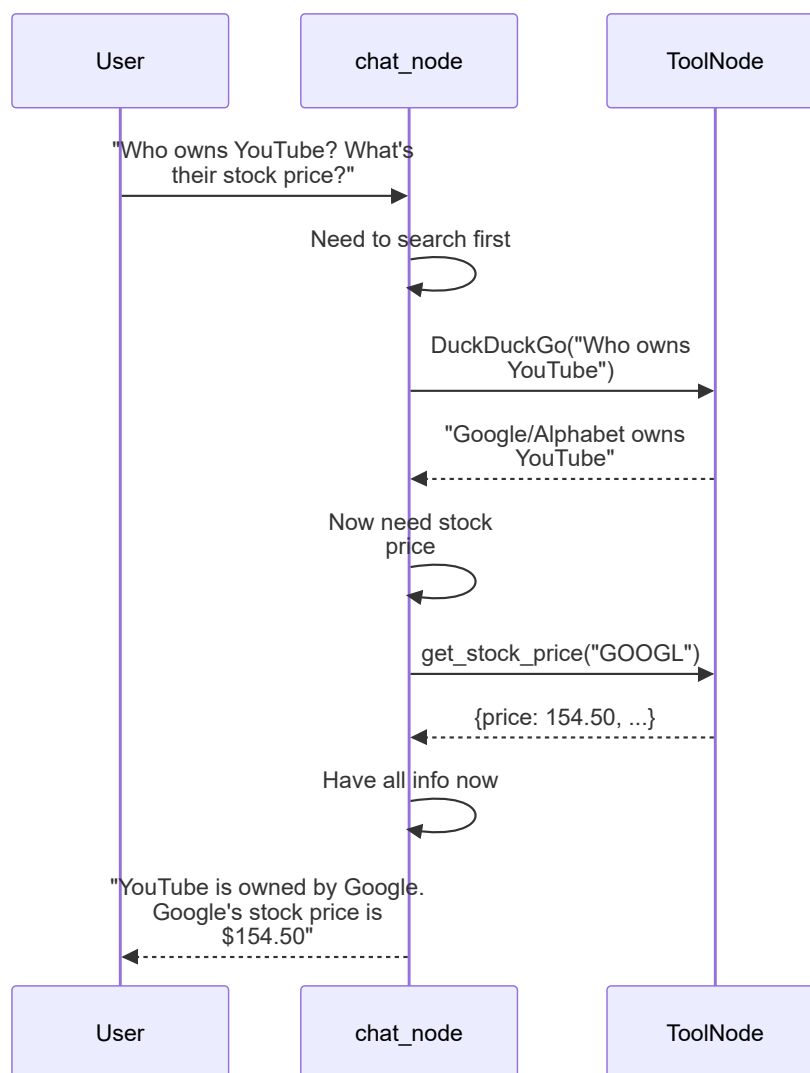


Figure 17.8. Multi-Step Tool Usage Flow

17.7 6. Types of Tools

In LangChain/LangGraph, there are two main categories of tools:

17.7.1 6.1 Prebuilt Tools

These are tools that LangChain provides out-of-the-box for common use cases.

Examples:

- DuckDuckGoSearchRun - Internet search
- WikipediaQueryRun - Wikipedia queries
- ArxivQueryRun - Academic paper search
- PythonREPL - Execute Python code

Usage:

```
from langchain_community.tools import DuckDuckGoSearchRun

search_tool = DuckDuckGoSearchRun(region="us-en")
```

Advantages:

- No coding required
- Tested and maintained by LangChain community
- Easy to integrate

17.7.2 6.2 Custom Tools

Tools you create yourself for specific use cases.

When to Create Custom Tools:

- No prebuilt tool exists for your need
- You need to integrate with your own APIs
- You want specific behavior or formatting

How to Create:

```
from langchain_core.tools import tool

@tool
def my_custom_tool(param1: str, param2: int) -> dict:
    """
    Description of what this tool does.
    This docstring is CRITICAL - the LLM reads it!
    """
    # Your implementation
    result = do_something(param1, param2)
    return result
```

17.7.3 Comparison Table:

Table 17.4. Comparison Table

Aspect	Prebuilt Tools	Custom Tools
Setup Time	Minutes	Hours
Flexibility	Limited	Unlimited
Maintenance	Community	You
Use Case	Common tasks	Specific needs
Example	DuckDuckGo Search	Your company's API

17.8 7. Complete Code Implementation

17.8.1 7.1 Required Imports

```
# LangGraph imports
from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import ToolNode, tools_condition
from langgraph.graph.message import add_messages

# LangChain imports
from langchain_core.messages import BaseMessage, HumanMessage
from langchain_openai import ChatOpenAI
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_core.tools import tool

# Standard library
from typing import TypedDict, Annotated
from dotenv import load_dotenv
import requests
```

Explanation of Key Imports:

Table 17.5. 1 Required Imports

Import	Purpose
ToolNode	Prebuilt node to handle tool execution
tools_condition	Conditional routing function
DuckDuckGoSearchRun	Prebuilt internet search tool
tool	Decorator to create custom tools

17.8.2 7.2 Environment Setup

```
# Load environment variables (API keys, etc.)
load_dotenv()

# Initialize the LLM
llm = ChatOpenAI()
```

17.8.3 7.3 Creating the Tools

Tool 1: DuckDuckGo Search (Prebuilt)

```
search_tool = DuckDuckGoSearchRun(region="us-en")
```

This tool allows the chatbot to search the internet for current information.

Tool 2: Calculator (Custom)

```
@tool
def calculator(first_num: float, second_num: float, operation: str) -> dict:
    """
    Perform a basic arithmetic operation on two numbers.
    Supported operations: add, sub, mul, div
    """
    try:
        if operation == "add":
            result = first_num + second_num
        elif operation == "sub":
            result = first_num - second_num
        elif operation == "mul":
            result = first_num * second_num
        elif operation == "div":
            if second_num == 0:
                raise ValueError("Cannot divide by zero")
            result = first_num / second_num
    except Exception as e:
        return {"error": str(e)}
```

```

        return {"error": "Division by zero is not allowed"}
    result = first_num / second_num
    else:
        return {"error": f"Unsupported operation '{operation}'"}

    return {
        "first_num": first_num,
        "second_num": second_num,
        "operation": operation,
        "result": result
    }
except Exception as e:
    return {"error": str(e)}

```

Key Points:

- The `@tool` decorator converts a regular function into a LangChain tool
- The docstring is critical—the LLM reads it to understand what the tool does
- Parameter types are specified for proper parsing
- Returns a structured dictionary for clear results

Tool 3: Stock Price Fetcher (Custom)

```

@tool
def get_stock_price(symbol: str) -> dict:
    """
    Fetch latest stock price for a given symbol (e.g. 'AAPL', 'TSLA')
    using Alpha Vantage with API key in the URL.
    """
    url = f"https://www.alphavantage.co/query?function=GLOBAL_QUOTE&symbol={symbol}&apikey=YOUR_API_KEY"
    r = requests.get(url)
    return r.json()

```

Note: Replace `YOUR_API_KEY` with your actual Alpha Vantage API key. You can get a free key from [alphavantage.co](https://www.alphavantage.co).

17.8.4 7.4 Binding Tools to LLM

```

# Create a list of all tools
tools = [get_stock_price, search_tool, calculator]

# Bind tools to the LLM
llm_with_tools = llm.bind_tools(tools)

```

What `bind_tools` Does:

1. Makes the LLM aware of available tools
2. Provides tool schemas to the LLM
3. Enables the LLM to generate tool calls in its responses

17.8.5 7.5 Defining the State

```

class ChatState(TypedDict):
    messages: Annotated[List[BaseMessage], add_messages]

```

Explanation:

- `ChatState` defines the structure of data flowing through the graph
- `messages` stores the conversation history
- `add_messages` is a reducer that appends new messages to existing ones

17.8.6 7.6 Creating Graph Nodes

Chat Node (LLM with Decision Making)

```
def chat_node(state: ChatState):
    """LLM node that may answer or request a tool call."""
    messages = state['messages']
    response = llm_with_tools.invoke(messages)
    return {"messages": [response]}
```

What Happens Inside:

1. Receives current state with all messages
2. Passes messages to the LLM (which has tools bound)
3. LLM either:
 - Returns a regular response (for normal chat)
 - Returns a tool call request (for action needed)
4. Returns the response to be added to state

Tool Node

```
tool_node = ToolNode(tools) # Executes tool calls
```

This single line creates a node that:

- Contains all three tools
- Automatically executes the requested tool
- Returns tool results

17.8.7 7.7 Building the Graph

```
# Initialize the graph
graph = StateGraph(ChatState)

# Add nodes
graph.add_node("chat_node", chat_node)
graph.add_node("tools", tool_node)

# Add edges
graph.add_edge(START, "chat_node") # START → chat_node
graph.add_conditional_edges("chat_node", tools_condition) # Conditional routing
graph.add_edge("tools", "chat_node") # tools → chat_node (feedback loop!)

# Compile the graph
chatbot = graph.compile()
```

Edge Breakdown:

Table 17.6. 7 Building the Graph

Edge	From	To	Type	Purpose
1	START	chat_node	Normal	Entry point
2	chat_node	END/tools	Conditional	Route based on LLM response
3	tools	chat_node	Normal	Feedback loop for refinement

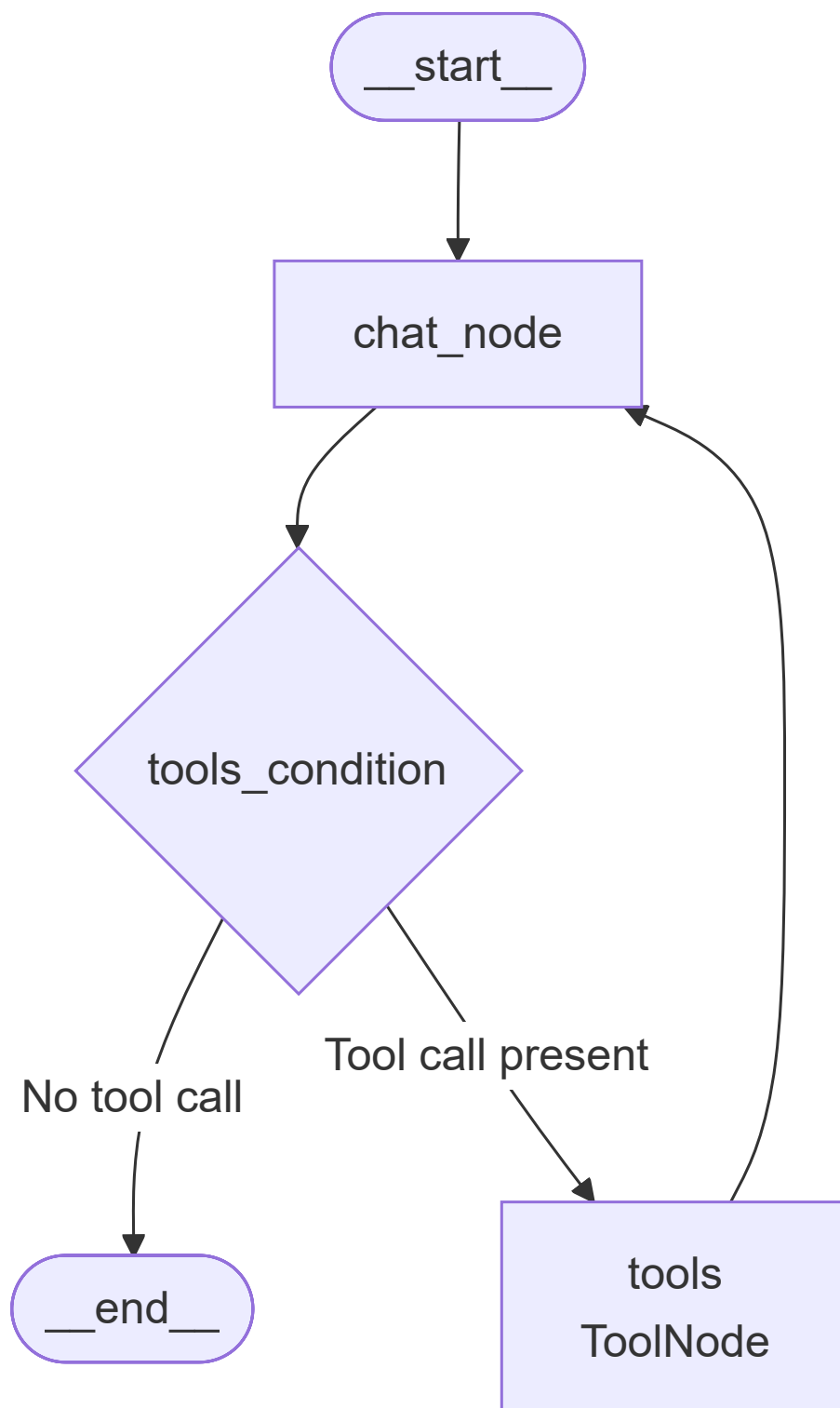


Figure 17.9. 8 Visual Representation of Compiled Graph

17.8.8 7.8 Visual Representation of Compiled Graph

17.9 8. Execution Flow Analysis

17.9.1 8.1 Normal Chat Flow

Query: "Hello!"
Output: "Hello! How can I assist you today?"

17.9.2 8.2 Single Tool Usage Flow

Query: "What is 2×3 ?"

17.9.3 8.3 Multi-Tool Usage Flow

Query: "What is Apple's stock price? How much would 50 shares cost?"

17.9.4 8.4 Internet Search Flow

Query: "Who owns YouTube? Find their stock price."
Step-by-step Execution:

Table 17.7. 4 Internet Search Flow

Step	Node	Action	Result
1	chat_node	Analyze query	Need search first
2	tools	DuckDuckGo("Who owns YouTube")	Google/Alphabet
3	chat_node	Process search result	Now need stock price
4	tools	get_stock_price("GOOGL")	\$154.50
5	chat_node	Compile final answer	Complete response
6	END	Return to user	"YouTube is owned by Google. Google's stock price is \$154.50"

17.10 9. Common Problems and Solutions

17.10.1 Problem 1: Raw Tool Output Displayed

Symptom: User sees JSON or technical data instead of human-friendly response.

Cause: Flow goes directly from ToolNode to END without LLM refinement.

Wrong Architecture:

```
chat_node → tools → END
```

Correct Architecture:

```
chat_node → tools → chat_node → END
```

Solution: Always add the feedback loop edge:

```
graph.add_edge("tools", "chat_node")
```

17.10.2 Problem 2: Multi-Step Queries Fail

Symptom: Complex queries requiring multiple tools only execute one tool.

Example: “Find Apple’s stock price and calculate cost of 50 shares”

Cause: Same as Problem 1—no feedback loop.

Solution: The feedback loop allows the LLM to:

1. Execute first tool
2. Analyze result
3. Decide if more tools needed
4. Execute additional tools
5. Compile final response

17.10.3 Problem 3: Tool Messages Streaming to User

Symptom: Both AI messages and raw tool messages appear in chat.

Cause: Streaming code doesn’t filter message types.

Wrong Code:

```
# Streams ALL messages
for msg in response:
    st.write(msg.content)
```

Correct Code:

```
from langchain_core.messages import AIMessage

# Only stream AI messages
for msg in response:
    if isinstance(msg, AIMessage):
        st.write(msg.content)
```

17.10.4 Problem 4: Wrong Tool Selected

Symptom: LLM chooses incorrect tool for the task.

Cause: Poor or ambiguous tool descriptions.

Solution: Write clear, specific docstrings:

```
# BAD - Vague description
@tool
def search(query: str):
    """Search for something"""
    ...

# GOOD - Specific description
@tool
def search(query: str):
    """
    Search the internet for current information, news, and real-time data.
    Use this when the user asks about recent events, current prices,
    or information that may have changed since my training data.
    """
    ...
```

17.11 10. Frontend Integration

17.11.1 10.1 Basic Streamlit Integration

```
import streamlit as st
from langchain_core.messages import AIMessage, HumanMessage

# Display chat history
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.write(message["content"])

# Get user input
if prompt := st.chat_input("What's on your mind?"):
    # Add to history
    st.session_state.messages.append({"role": "user", "content": prompt})

# Get response (with streaming)
response = chatbot.invoke({"messages": [HumanMessage(content=prompt)]})

# Filter and display only AI messages
for msg in response["messages"]:
    if isinstance(msg, AIMessage):
        st.session_state.messages.append({"role": "assistant", "content": msg.content})
```

17.11.2 10.2 Status Container for Tool Usage

To show users which tool is being used:

```
with st.status("Processing...", expanded=True) as status:
    for event in chatbot.stream({"messages": messages}):
        for key, value in event.items():
            if key == "tools":
                # Extract tool name from the message
                tool_name = extract_tool_name(value)
                status.update(label=f"Using {tool_name}...")
            elif key == "chat_node":
                if isinstance(value["messages"][-1], AIMessage):
                    status.update(label="Complete!", state="complete")
```

User Experience:

Table 17.8. 2 Status Container for Tool Usage

State	Display
Searching	"Using DuckDuckGo Search..."
Calculating	"Using Calculator..."
Fetching Stock	"Using Get Stock Price..."
Complete	"Complete!"

17.12 11. Key Takeaways

17.12.1 Conceptual Understanding

1. **Tools extend LLM capabilities** from pure text generation to real-world actions
2. **ToolNode is the executor** that handles all tool operations
3. **tools_condition is the router** that decides chat vs. action paths
4. **The feedback loop is essential** for refined outputs and multi-step reasoning

17.12.2 Implementation Checklist

- ☐ Import ToolNode and tools_condition from langgraph.prebuilt
- ☐ Create tools (prebuilt or custom with @tool decorator)
- ☐ Write clear, descriptive docstrings for custom tools
- ☐ Create a tools list and bind to LLM with llm.bind_tools()
- ☐ Add both chat_node and tool_node to the graph
- ☐ Use add_conditional_edges with tools_condition
- ☐ **Always add the feedback loop:** tools $\rightarrow$ chat_node
- ☐ Filter AI messages when streaming to avoid showing tool outputs

17.12.3 Architecture Diagram Summary

17.12.4 Best Practices

Table 17.9. *Best Practices*

Practice	Description
Descriptive Docstrings	LLM reads these to choose tools
Type Annotations	Help with parameter parsing
Error Handling	Return clear error messages
Feedback Loop	Always route tool output back to LLM
Message Filtering	Don't show tool messages to users
Status Updates	Show users which tool is being used

17.13 Quick Reference Code

17.13.1 Minimal Working Example

```

from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import ToolNode, tools_condition
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages

# 1. Create LLM
llm = ChatOpenAI()

# 2. Create Tool
@tool
def add_numbers(a: float, b: float) -> float:
    """Add two numbers together."""
    return a + b

# 3. Bind Tools
tools = [add_numbers]
llm_with_tools = llm.bind_tools(tools)

# 4. Define State
class State(TypedDict):
    messages: Annotated[list, add_messages]

# 5. Create Nodes
def chat(state):
    return {"messages": [llm_with_tools.invoke(state["messages"])]}

```

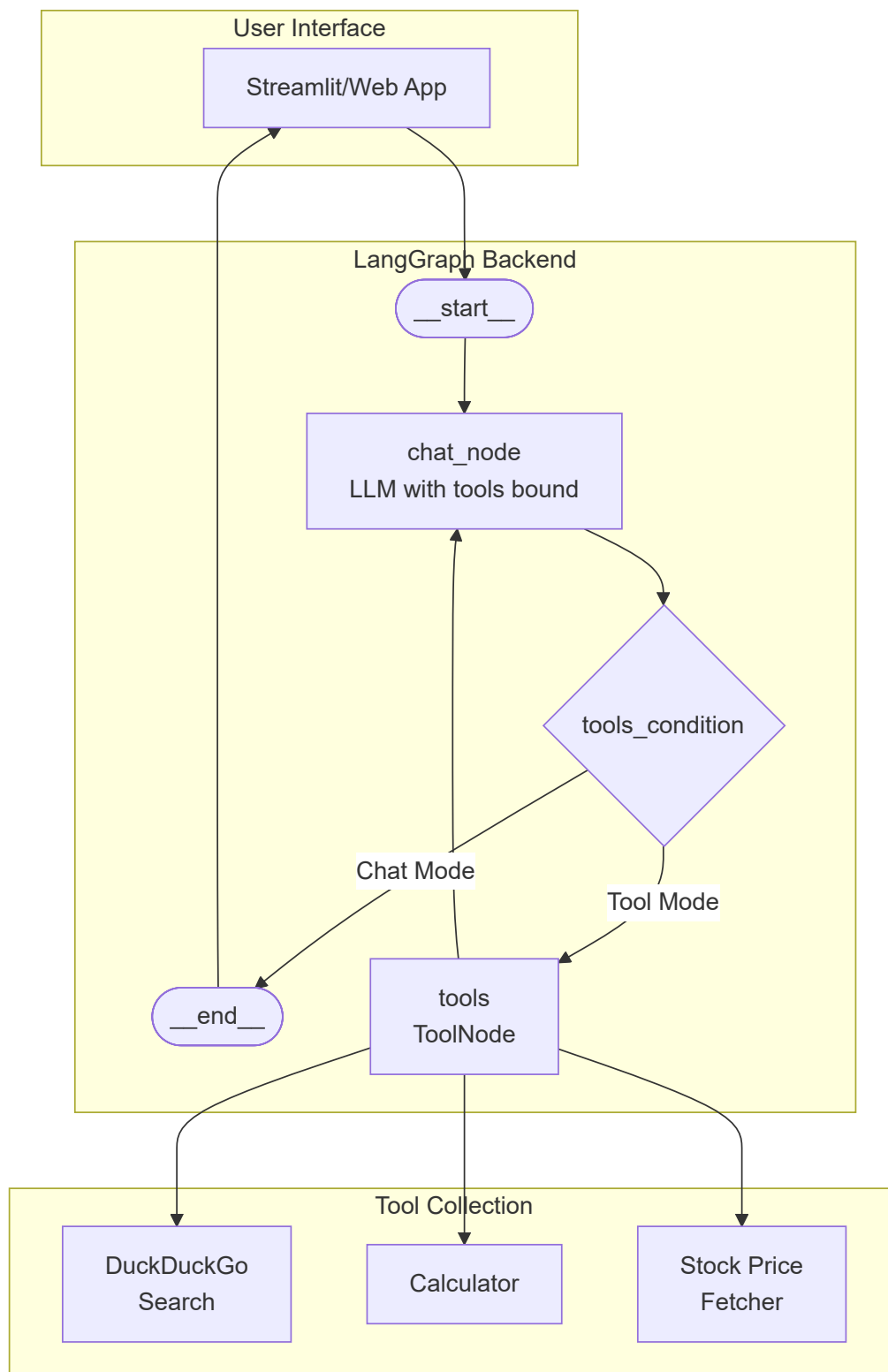


Figure 17.10. Architecture Diagram Summary

```
# 6. Build Graph
graph = StateGraph(State)
graph.add_node("chat", chat)
graph.add_node("tools", ToolNode(tools))
graph.add_edge(START, "chat")
graph.add_conditional_edges("chat", tools_condition)
graph.add_edge("tools", "chat") # FEEDBACK LOOP!

# 7. Compile & Use
chatbot = graph.compile()
result = chatbot.invoke({"messages": [("user", "What is 5 + 3?")]}))
print(result["messages"][-1].content)
```

17.14 Glossary

Table 17.10. *Glossary*

Term	Definition
Tool	A function that an LLM can call to perform actions
ToolNode	Prebuilt LangGraph node that executes tools
tools_condition	Function that decides whether to route to tools or end
bind_tools	Method to make LLM aware of available tools
Tool Call	LLM's request to execute a specific tool
Feedback Loop	Pattern of routing tool output back to LLM
Docstring	Tool description that LLM reads for decision making

Tools in LangGraph: Comprehensive Guide

Calculator • Internet Search • Stock Prices • Custom APIs — Extend Your Chatbot's Capabilities

The Problem: Basic Chatbot Limitations

What Basic Chatbots CANNOT Do:

- ❌ "What is 654 * 713?" → LLMs make arithmetic errors
- ❌ "Today's top news in India?" → No internet access
- ❌ "Tesla's current stock price?" → No API access
- ❌ Execute any real-world actions

✓ Solution: Add TOOLS to your chatbot!

What Are Tools?

Functions that LLM can call to perform actions
Think of tools as the "hands and eyes" of your AI

Tool Anatomy:

1. NAME → Unique identifier
2. DESCRIPTION → LLM reads that
3. FUNCTION → Actual code

⚠️ Description (docstrings) is CRITICAL for tool selection!

Core Concepts

ToolNode

- Prebuilt node from LangGraph
- Executes tool calls
- Routes to correct tool

tools_condition

- Prebuilt conditional edge
- Decides tools or END?
- Checks for tool_calls

tools_condition Decision Logic:

- Tool call detected? → Route to ToolNode
- No tool call? → Route to END

Architecture: Simple vs Enhanced Workflow

Simple Workflow (Without Tools)

```
graph LR; start((start)) --> chat(chat_node); chat --> end((end))
```

Limitations:

- Cannot perform actions
- Cannot access external data
- Limited to LLM's training knowledge

Enhanced Workflow (With Tools + Feedback Loop)

```
graph LR; start((start)) --> chat(chat_node); chat --> tools{tools_condition}; tools --> tool_calls[Tool Calls]; tool_calls --> tools; tools --> end((end));
```

★ FEEDBACK LOOP (Critical!)

Why Feedback Loop?

1. Refine raw tool output
2. Enable multi-step reasoning
3. Chain multiple tool calls
4. Human-friendly responses

Without loop: Raw JSON output
With loop: "Apple's price is \$227"

Types of Tools

Prebuilt Tools

LangChain provides out-of-the-box:

- DuckDuckGoSearchRun
- WikipediaQueryRun
- ArxivQueryRun
- PythonREPL

```
search = DuckDuckGoSearchRun()
```

Custom Tools

Create your own with @tool decorator:

```
@tool
def my_tool(param: str):
    """Toolstring is CRITICAL!
    LLM reads this to decide."""
    return result
```

Three Tools We're Building

DuckDuckGo Search

Type: Prebuilt
Purpose: Internet search

```
search_tool = DuckDuckGoSearchRun()

"Who owns YouTube?"
→ Google/Alphabet
```

Calculator

Type: Custom
Purpose: Math operations

```
@tool
def calculator(a, b, op):
    """Perform arithmetic"""
    if op == "add": return a + b
    elif op == "sub": return a - b
    elif op == "mul": return a * b
    elif op == "div": return a / b
```

Stock Price

Type: Custom (API)
Purpose: Real-time prices

```
@tool
def get_stock_price(symbol):
    """Fetch stock price"""
    return 227.76
```

Complete Implementation

```
# Key imports
from langgraph.prebuilt import ToolNode, tools_condition
from langchain_core.tools import Tool

# Step 1: Create Tools
@tool
def calculator(a: float, b: float, op: str):
    """Perform arithmetic"""
    if op == "add": return a + b
    elif op == "sub": return a - b
    elif op == "mul": return a * b
    elif op == "div": return a / b

# Step 2: Bind Tools to LLM
tools = [calculator, search, get_stock_price]
llm = bind_to_llm(model, tools)

# Step 3: Build Graph with Feedback Loop
graph = StateGraph(State)
graph.add_node("chat_node", chat_node)
graph.add_node("tools", ToolNode(tools))
graph.add_edge("start", "chat_node")
graph.add_conditional_edge("chat_node", "tools", tools_condition)
graph.add_node("end", lambda: "END")
graph.add_edge("tools", "end")
graph.compile()
```

Multi-Tool Execution Flow Example

Query: "Who owns YouTube? What's their stock price?"

- 1 chat_node → Analyzes query → Need search first
tools_condition returns "tools"
- 2 tools (ToolNode) → DuckDuckGo("Who owns YouTube")
Returns: "Google/Alphabet owns YouTube"
- 3 chat_node (loop back) → Now need stock price
tools_condition returns "tools" again
- 4 tools (ToolNode) → get_stock_price("GOOGL")
Returns: (price: \$154.50, ...)
- 5 chat_node (final) → Has all info, generates response
tools_condition returns END

✓ "YouTube is owned by Google. Google's stock price is \$154.50"

Common Problems and Solutions

Problem 1: Raw Tool Output

User sees JSON instead of friendly text

✓ Solution: Add feedback loop
graph.add_edge("tools", "chat_node")

Problem 2: Multi-Step Fails

Complex queries only execute one tool

✓ Solution: Same - feedback loop!
Loop enables chaining multiple tools

Problem 3: Tool Messages Shown

Raw tool messages appear in chat UI

✓ Solution: Filter message types
if isinstance(msg, AIMessage): show()

Problem 4: Wrong Tool Selected

LLM chooses incorrect tool for task

✓ Solution: Better docstrings!
Be specific about when to use tool

Best Practices

Descriptive Docstrings

LLM reads these to choose tools
Be specific about use cases!

Always Add Feedback Loop

tools → chat_node edge is essential
Enables refinement + multi-step

Type Annotations

Help with parameter parsing
def tool(a: float, b: str) -> dict

Error Handling

Return clear error messages
Handle edge cases gracefully

Frontend: Filter Messages + Show Status

Only show AIMessages to users • Use st.status() to show "Using Calculator..." during tool execution

Quick Reference: Minimal Working Example

```
from langgraph.graph import StateGraph, START
from langgraph.prebuilt import ToolNode, tools_condition
from langchain_core.tools import Tool

# 1. Create LLM and Tool
llm = ChatOpenAI()

@tool
def add(a: float, b: float):
    """Add two numbers"""
    return a + b

# 2. Bind tools to LLM
tools = [add]
llm_with_tools = llm.bind_tools(tools)

# 3. Build graph with FEEDBACK LOOP
graph = StateGraph(State)
graph.add_node("chat", llm_with_tools)
graph.add_node("tools", ToolNode(tools))
graph.add_conditional_edge("chat", "tools", tools_condition)
graph.add_node("end", lambda: "END")
graph.add_edge("tools", "end")
graph.compile()
```

graph.add_edge("tools", "chat") ★ FEEDBACK LOOP - DON'T FORGET!

Glossary

Tool	Function LLM can call for actions	ToolNode	Prebuilt node that executes tools
tools_condition	Routes to tools or END	bind_tools()	Makes LLM aware of tools
Tool Call	LLM's request to execute a tool	Feedback Loop	tools → chat_node edge

Implementation Checklist:

- Import ToolNode, tools_condition from langgraph.prebuilt
- Create tools with @tool decorator + clear docstrings
- Bind tools to LLM: llm.bind_tools(tools)
- Add conditional edges with tools_condition
- ADD FEEDBACK LOOP: graph.add_edge("tools", "chat_node")

Tools in LangGraph Guide • [github.com/campusx-official/chatbot-in-langgraph](#) • CampusX Tutorial

Figure 17.11. Glossary

Agentic AI using LangGraph: Adding MCP (Model Context Protocol)

18.1 Code

-<https://github.com/campusx-official/chatbot-in-langgraph>

18.2 Playlist Recap

18.2.1 Progress So Far: 18 Videos

Table 18.1. *Progress So Far: 18 Videos*

Category	Topics Covered
Fundamentals	• What is Agentic AI? • Generative AI vs Agentic AI • LangGraph vs LangChain
LangGraph Core	• Core concepts • Building workflows (Parallel, Sequential, Conditional, Iterative)
Chatbot Project	• Basic chatbot creation • UI integration • Streaming feature • Resume chat capability • Database integration • Observability with LangSmith • Tools integration (Stock market, Web search, Calculator)

18.3 What is MCP?

18.3.1 Definition

MCP = Model Context Protocol

Simple Explanation: MCP is an improved version of Tools

MCP = Standardized way to connect Tools to LLM applications

18.3.2 Key Concept

Table 18.2. *Key Concept*

Traditional Tools	MCP
✗ Direct coupling	✓ Standardized protocol
✗ Brittle integration	✓ Robust separation
✗ High maintenance	✓ Future-proof

18.4 ! Problems with Traditional Tools

18.4.1 Example Scenario

Current Setup:

- ✓ Search Tool (Internet search)
- ✓ Calculator Tool (Math operations)
- ✓ Stock Price Tool (Get current stock prices)

New Requirement:

- Connect to GitHub
- Answer questions about repositories

18.4.2 Traditional Tool Implementation

```
@tool
def get_pull_requests(
    owner: str,
    repo_url: str,
    state: str = "open",
    per_page: int = 5
):
    """Fetch pull requests from GitHub repository"""

    # GitHub access token
    headers = {
        "Authorization": f"token {GITHUB_TOKEN}",
        "Accept": "application/vnd.github.v3+json"
    }

    # API endpoint
    url = f"https://api.github.com/repos/{owner}/{repo}/pulls"

    # Fetch data
    response = requests.get(url, headers=headers)
    data = response.json()

    # Extract information
    for pr in data:
        print(f"PR #{pr['number']}")
        print(f>Title: {pr['title']}")
        print(f"Author: {pr['user']['login']}")
        print(f"State: {pr['state']}")
        print(f"URL: {pr['html_url']}")
```

18.4.3 The Brittleness Problem

18.4.4 Maintenance Nightmare

Table 18.3. *Maintenance Nightmare*

Scenario	Impact
1 Tool × 1 API Change	Manual update required
10 Tools × 1 API Change	Update in 10 places
5 Chatbots × 10 Tools	n × m maintenance problem
+ Multiple Services	Gmail, Slack, Jira, GitHub...

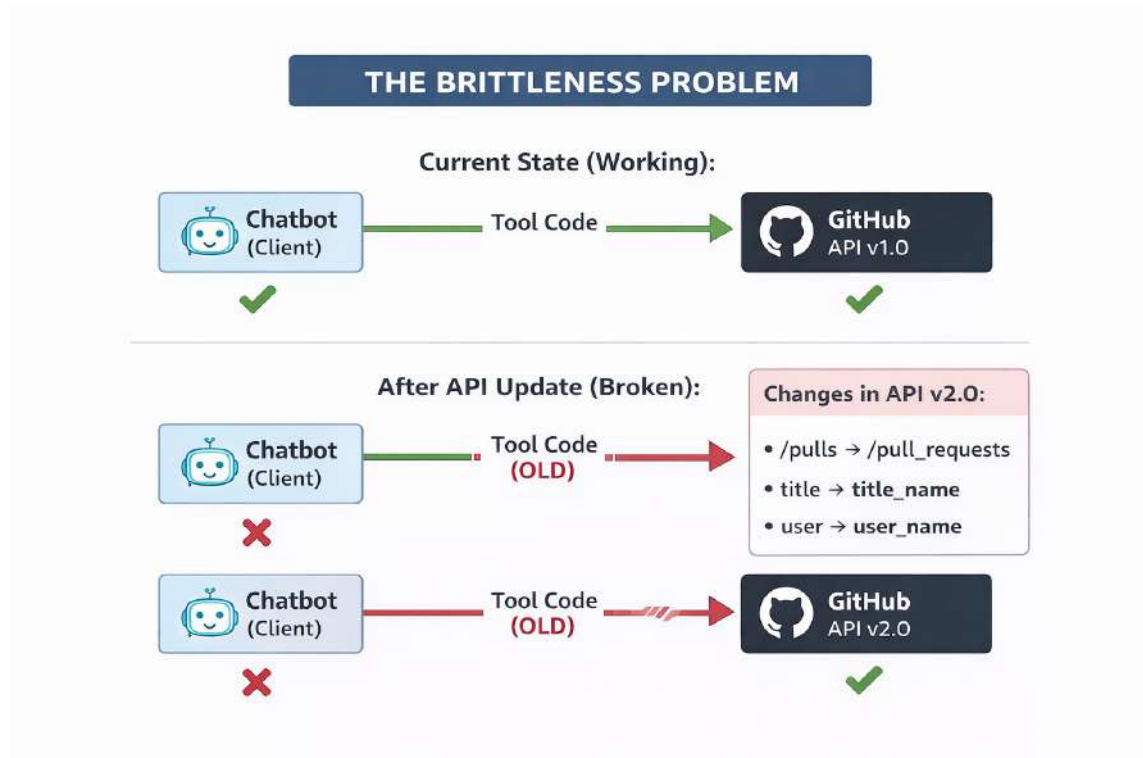


Figure 18.1. The Brittleness Problem

18.4.5 Real Problems

1. API Version Changes

- Endpoints change
- Parameters change
- Response structure changes

2. Code Maintenance

- Must study new documentation
- Update multiple files
- Test everything again

3. Time Waste

- Constantly updating tool code
- No time for new features

4. Tight Coupling

- Server changes → Client breaks
- No separation of concerns

18.5 ✓ MCP Solution

18.5.1 Core Principle: Separation of Concerns

18.5.2 Key Benefits

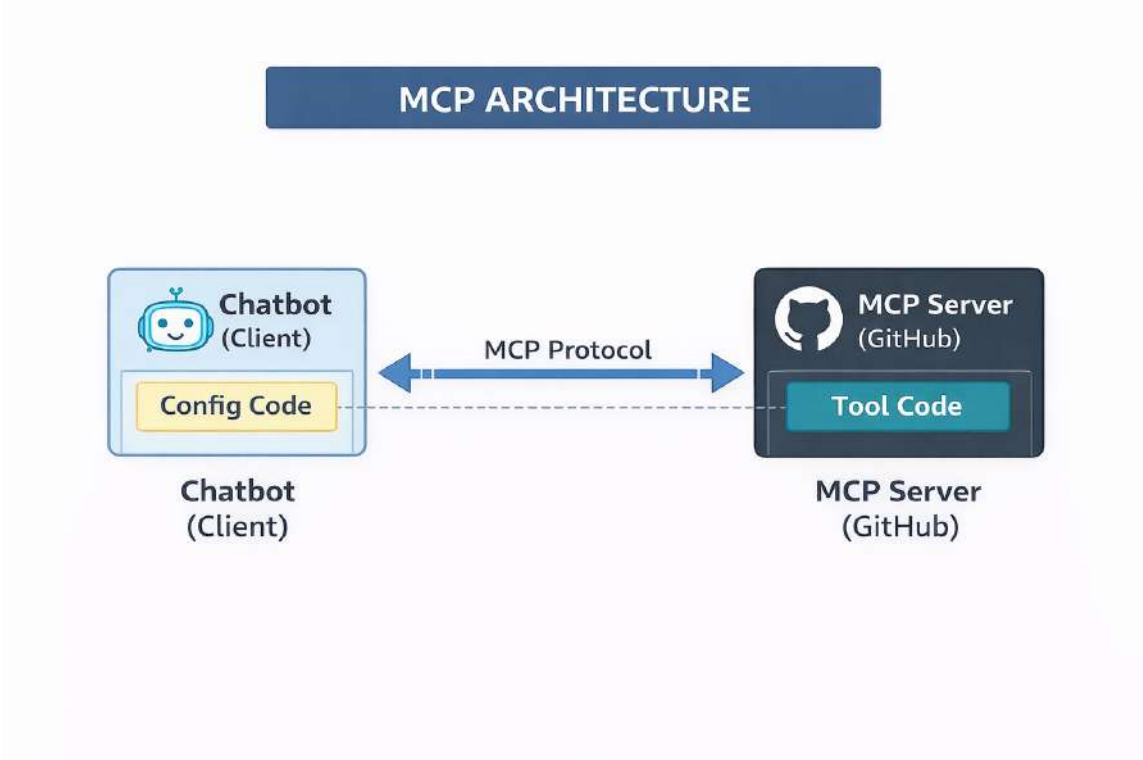


Figure 18.2. Core Principle: Separation of Concerns

Table 18.4. Key Benefits

Aspect	Traditional Tools	MCP
Code Location	Client-side	Server-side
When API Changes	Must update client	Server updates itself
Client Code	Complex tool logic	Simple config
Maintenance	High effort	Low effort
Future-proof	✗ Brittle	✓ Robust

18.5.3 Code Comparison

Traditional Tool Approach:

```
# 50+ lines of complex code
@tool
def get_pull_requests(...):
    # GitHub token
    # Headers
    # API endpoint
    # Request handling
    # Response parsing
    # Error handling
    # etc...
```

MCP Approach:

```
# Simple config code
client = MultiServerMCPClient(
    servers={
        "github": {
            "transport": "stdio",
            "command": "python",
            "args": ["/path/to/github_mcp_server.py"]
        }
    }
)
```



```

    }
  }
)

```

18.5.4 The Big Advantage

Server changes? No problem!

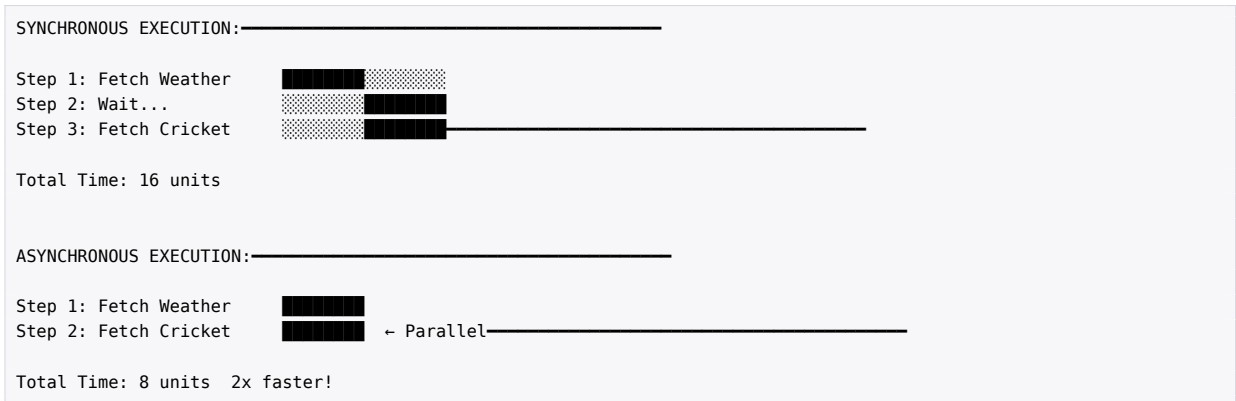
✓ *GitHub updates API v1.0 → v2.0* ✓ *Server code automatically updated* ✗ *No client code changes needed!*

18.6 Converting to Async

18.6.1 Why Async?

MCP libraries only work in async mode

18.6.2 Sync vs Async Execution



18.6.3 Key Changes for Async

Table 18.5. *Key Changes for Async*

Component	Change Required
Functions	Add <code>async</code> keyword
Function Calls	Add <code>await</code> keyword
LangGraph Nodes	Convert to <code>async</code>
Tool Nodes	Already <code>async</code> (no change)
Main Function	Use <code>asyncio.run()</code>

18.6.4 Code Transformation

Before (Sync):

```

def chat_node(state: State):
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response]}

# Build graph
graph = StateGraph(State)
graph.add_node("chat", chat_node)

```

```
chatbot = graph.compile()

# Run
result = chatbot.invoke({"messages": [user_message]})
```

After (Async):

```
async def chat_node(state: State):
    response = await llm_with_tools.ainvoke(state["messages"])
    return {"messages": [response]}

# Build graph
async def build_graph():
    graph = StateGraph(State)
    graph.add_node("chat", chat_node)
    return graph.compile()

# Run
async def main():
    chatbot = await build_graph()
    result = await chatbot.ainvoke({"messages": [user_message]})

if __name__ == "__main__":
    asyncio.run(main())
```

18.7 Building MCP Client in LangGraph

18.7.1 Required Library

```
# Install MCP adapter
pip install langchain-mcp-adapters

# Or with uv
uv add langchain-mcp-adapters
```

18.7.2 Two Components Needed

MCP SYSTEM COMPONENTS	
1. MCP Server (Tool implementation)	
2. MCP Client (Connects to server)	

Today's Focus: Building MCP Client

18.7.3 Local MCP Server Example

File: math_mcp_server.py

```
from fastmcp import FastMCP

mcp = FastMCP("Math Operations Server")

@mcp.tool()
async def add(a: float, b: float) -> float:
    """Add two numbers"""
    return a + b

@mcp.tool()
```

```

async def subtract(a: float, b: float) -> float:
    """Subtract b from a"""
    return a - b

@mcp.tool()
async def multiply(a: float, b: float) -> float:
    """Multiply two numbers"""
    return a * b

@mcp.tool()
async def divide(a: float, b: float) -> float:
    """Divide a by b"""
    return a / b

@mcp.tool()
async def power(a: float, b: float) -> float:
    """Raise a to power b"""
    return a ** b

@mcp.tool()
async def modulus(a: float, b: float) -> float:
    """Get remainder of a divided by b"""
    return a % b

```

18.7.4 MCP Client Implementation

```

import asyncio
from langchain_mcp_adapters.client import MultiServerMCPClient
from langgraph.graph import StateGraph
from langchain_openai import ChatOpenAI

# Import MCP client library
from langchain_mcp_adapters.client import MultiServerMCPClient

# Initialize LLM
llm = ChatOpenAI(model="gpt-4", temperature=0)

# Create MCP Client with server configuration
client = MultiServerMCPClient(
    servers={
        "math-server": {
            "transport": "stdio", # Local server uses stdio
            "command": "python",
            "args": ["/path/to/math_mcp_server.py"]
        }
    }
)

# Build graph function
async def build_graph():
    # Fetch tools from MCP server
    tools = await client.get_tools()

    # Bind tools to LLM
    llm_with_tools = llm.bind_tools(tools)

    # Create graph
    graph = StateGraph(State)
    # ... add nodes and edges

    return graph.compile()

# Main execution
async def main():
    chatbot = await build_graph()
    result = await chatbot.ainvoke({
        "messages": [HumanMessage(content="Add 5 and 3")]
    })
    print(result)

```

```
if __name__ == "__main__":
    asyncio.run(main())
```

18.7.5 Transport Types

Table 18.6. *Transport Types*

Type	Use Case	Example
stdio	Local servers on same machine	"transport": "stdio"
SSE	Remote servers over network	"transport": "sse"

18.8 Demo & Implementation

18.8.1 Demo 1: Local Math Server

Query: “Perform modulus operation between 25 and 13 and explain like a cricket commentator”

Output:

```
Tools Available:
- add
- subtract
- multiply
- divide
- power
- modulus

Result: "And that's a pitch! Misses a few...
and the remainder is 12!"
```

18.8.2 Demo 2: Adding Remote Expense Tracker

Server Configuration:

```
client = MultiServerMCPClient(
    servers={
        # Local math server
        "math-server": {
            "transport": "stdio",
            "command": "python",
            "args": ["/Desktop/mcp_math_server/main.py"]
        },
        # Remote expense tracker server
        "expense-tracker": {
            "transport": "sse", # Remote uses SSE
            "url": "https://expense-tracker-mcp.example.com"
        }
    }
)
```

Available Tools:

- ✓ add_expense - Add new expense
- ✓ list_expenses - List all expenses
- ✓ summarize_expenses - Summarize by category

18.8.3 Usage Examples

Table 18.7. *Usage Examples*

Command	Tool Used	Result
"Add expense ₹500 for Udemty course on Nov 10"	add_expense	✓ Expense added
"List all November expenses"	list_expenses	Shows all expenses
"Summarize November spending"	summarize_expenses	Category breakdown

18.8.4 Full Chatbot Integration

Features:

- ✓ Normal chat
- ✓ Traditional tools (Stock price)
- ✓ MCP tools (Math operations)
- ✓ MCP tools (Expense tracking)

Code Structure:

```
chatbot_mcp/|—
  langgraph_mcp_backend.py|
    |— MCP client config|
    |— Traditional tools|
    |— Tool merging|
    |— Graph building| —
  streamlit_frontend_mcp.py
    |— Async stream handling
    |— UI components
    |— User interaction
```

18.9 Key Takeaways

18.9.1 MCP Advantages

Table 18.8. *MCP Advantages*

Benefit	Description
Standardization	Universal protocol for tool integration
Robustness	Server changes don't break client
Scalability	Easy to add multiple servers
Maintenance	Low overhead, future-proof
Flexibility	Mix local & remote servers

18.9.2 ! Important Notes

1. **Async Required:** MCP libraries only work with async code
2. **Library:** Use langchain-mcp-adapters
3. **Transport:**
 - Local servers → stdio
 - Remote servers → sse
4. **Tool Mixing:** Can combine traditional tools + MCP tools

18.9.3 Frontend Considerations

Current Setup (Hacky):

- ✗ Streamlit (not ideal for async)
- ! Works but not production-ready

Recommended for Production:

- ✓ FastAPI backend (native async)
- ✓ React/Next.js frontend
- ✓ Better async handling

18.10 Next Steps

18.10.1 Coming Up: RAG (Retrieval Augmented Generation)

Next video: Adding capability to answer questions from internal documents

18.10.2 Resources

- MCP Playlist: 8 videos covering MCP concepts in detail
- Code Repository: Full implementation examples
- Recommended: Watch MCP playlist before this video for better understanding

18.11 Complete Architecture

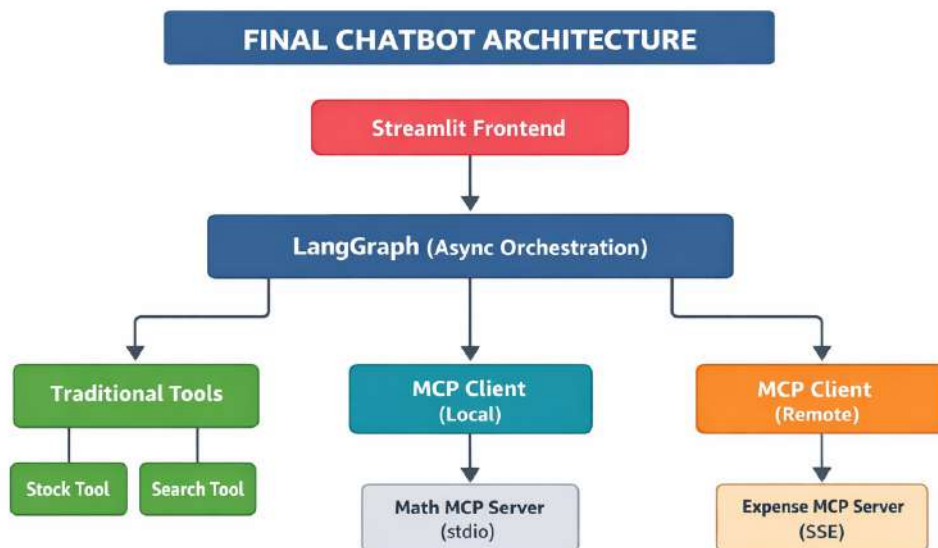


Figure 18.3. Complete Architecture

18.12 Pro Tips

1. **Learn MCP First:** Watch the dedicated MCP playlist (8 videos)
2. **Start with Async:** Convert to async before adding MCP
3. **Test Incrementally:** Add one server at a time
4. **Use Config:** Keep server configs separate and clean
5. **Plan for Scale:** Think about $n \times m$ maintenance problem early

End of Notes | Happy Coding! | See you in the next video!

RAG (Retrieval Augmented Generation) Integration with LangGraph

19.1 Code

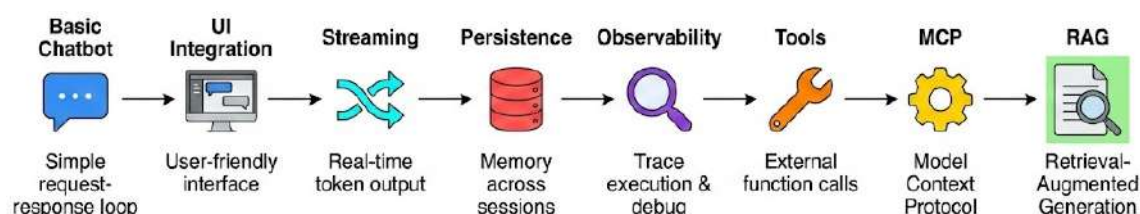
- <https://github.com/campusx-official/langgraph-tutorials>
- <https://github.com/campusx-official/chatbot-in-langgraph>

19.2 Introduction and Context

This guide covers the integration of RAG (Retrieval Augmented Generation) into a LangGraph-based chatbot. This is part of an incremental learning approach where a chatbot has been progressively enhanced with various capabilities.

19.2.1 Evolution of the Chatbot Project

The chatbot has evolved through multiple stages, each adding new capabilities:



Stage	Feature	LangGraph Concept
Basic Chatbot	Stateful message passing	StateGraph, Nodes, Edges
UI Integration	API endpoints for UI	Deployment, API Wrapper
Streaming	Stream tokens/updates	Streaming support, Callbacks
Persistence	Save/load conversation state	Checkpointers (e.g., MemorySaver, SqliteSaver)
Observability	Monitor and trace workflows	LangSmith Integration
Tools	Invoke external functions	StructuredTool, ToolNode
MCP	Standardized tool interfaces	MCP Integration, Tool abstractions
RAG	Retrieve relevant context	VectorStore, Retrieval Tools, State management

Figure 19.1. Evolution of the Chatbot Project

Each stage represents a core LangGraph concept:

Table 19.1. Evolution of the Chatbot Project

Stage	Feature	LangGraph Concept
1	Basic Chatbot	State Management, Nodes
2	UI	Frontend Integration
3	Streaming	Token Streaming
4	Persistence	Checkpointing
5	Observability	Tracing with LangSmith
6	Tools	Tool Binding
7	MCP	Model Context Protocol
8	RAG	Document Retrieval

19.2.2 What We’re Building

After RAG integration, the chatbot becomes a **Multi-Utility Chatbot** capable of:

- Normal conversational chat
- Tool usage (web search, calculator, stock prices)
- Document-based question answering (upload PDFs and query them)

19.3 Understanding the Problem: Why RAG?

RAG addresses three fundamental limitations of Large Language Models (LLMs):

19.3.1 1. Outdated Knowledge (Knowledge Cutoff)

Every LLM has a **knowledge cutoff date** - the date when its training was completed. The model cannot answer questions about events or information after this date.

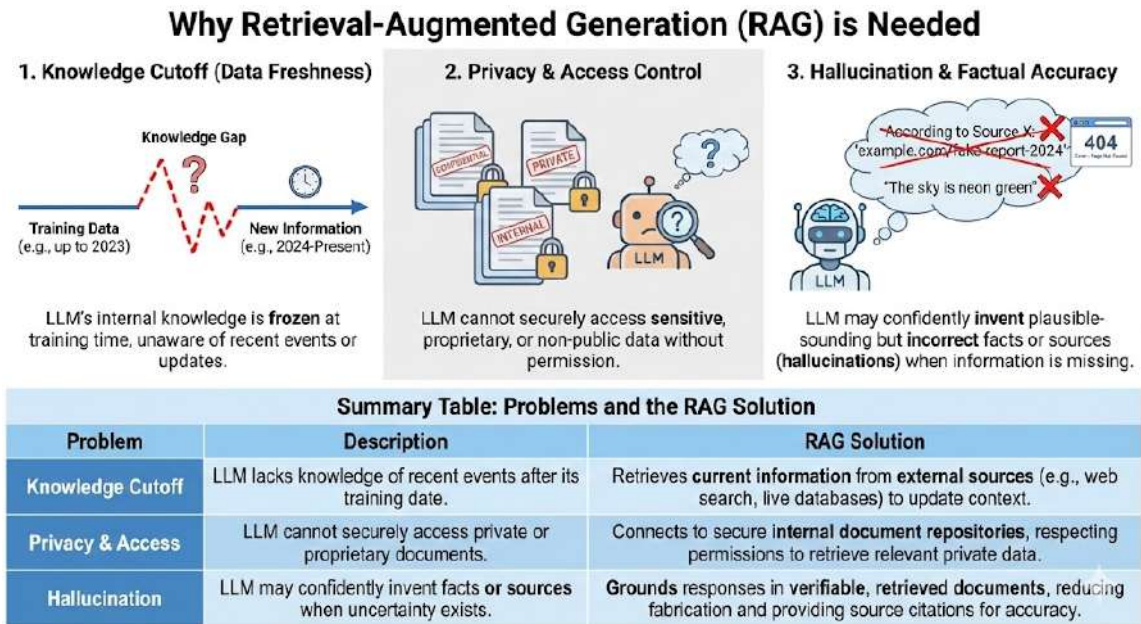


Figure 19.2. Outdated Knowledge (Knowledge Cutoff)

How ChatGPT and similar systems handle this: They perform web searches behind the scenes, retrieve current information, and then process it to answer your question. This is essentially RAG in action.

19.3.2 2. Privacy (Private/Personal Data)

LLMs are trained on publicly available data. They have no knowledge of:

- Your personal expense sheets
- Company financial reports
- Internal documentation
- Private codebases
- Personal notes or journals

The Problem: If you ask about data the LLM has never seen, it cannot help you.

RAG Solution: Connect your private documents to the LLM so it can answer questions based on your specific data.

19.3.3 3. Hallucination

Hallucination occurs when an LLM generates false information with high confidence.

Example Scenario:

User: "What are the top AI research papers in 2025?"
 LLM Response: [Provides 10 links]
 Reality: 4 of those links return 404 errors - they don't exist!

RAG Solution: Ground the LLM's responses in actual retrieved documents. By instructing the LLM to answer only based on provided context, we significantly reduce hallucination.

19.3.4 Summary of RAG Benefits

Table 19.2. *Summary of RAG Benefits*

Problem	Description	RAG Solution
Outdated Knowledge	LLM training has a cutoff date	Retrieve current information on-demand
Privacy	LLM doesn't know your private data	Connect private documents to LLM
Hallucination	LLM generates false information	Ground responses in retrieved facts

19.4 What is RAG? The Core Concept

RAG operates on a fundamental principle called **In-Context Learning**.

19.4.1 In-Context Learning Explained

In-context learning means that if you provide additional context to an LLM along with your query, the LLM can use that context to generate answers - even if that information was never part of its training.

19.4.2 Simple Chat Flow vs RAG-Enhanced Flow

Traditional LLM Flow:

The LLM relies solely on its **parametric knowledge** (what it learned during training).

RAG-Enhanced Flow:

Now the LLM has access to:

1. Your query
2. External context (your documents)

3. Its parametric knowledge

19.4.3 Practical Example

Scenario: You have an expense sheet for the current month.

Without RAG:

```
User: "What was my highest expense this month?"
LLM: "I don't have access to your personal expense data."
```

With RAG:

```
User Query + Expense Sheet Data (as context)
↓
LLM: "Based on your expense sheet, your highest expense
      was ₹15,000 on Electronics on the 15th."
```

19.4.4 The Context Window Limitation

There's a critical constraint: **Context Window Size**.

Every LLM has a limited context window (measured in tokens). You cannot paste unlimited amounts of data.

Problem Scenarios:

Table 19.3. *The Context Window Limitation*

Scenario	Issue
200-page book	Exceeds context window
Folder with 100 ebooks	Far exceeds context window
Large codebase (thousands of files)	Cannot fit all code

Solution: Don't paste the entire context. Instead, **filter and retrieve only the relevant portions**.

Key Insight: RAG requires intelligent context filtering - you only provide the chunks that are relevant to the user's question.

19.5 RAG Architecture: How It Works

The RAG architecture consists of two main phases:

1. **Indexing Phase** (one-time setup)
2. **Retrieval Phase** (per-query)

19.5.1 Complete RAG Pipeline**19.5.2 Step-by-Step Breakdown****Phase 1: Indexing****Step 1: Load the Document**

- Take your knowledge source (PDF, book, webpage)
- Load it into memory using a document loader

Step 2: Split into Chunks

- Divide the document into smaller, manageable pieces

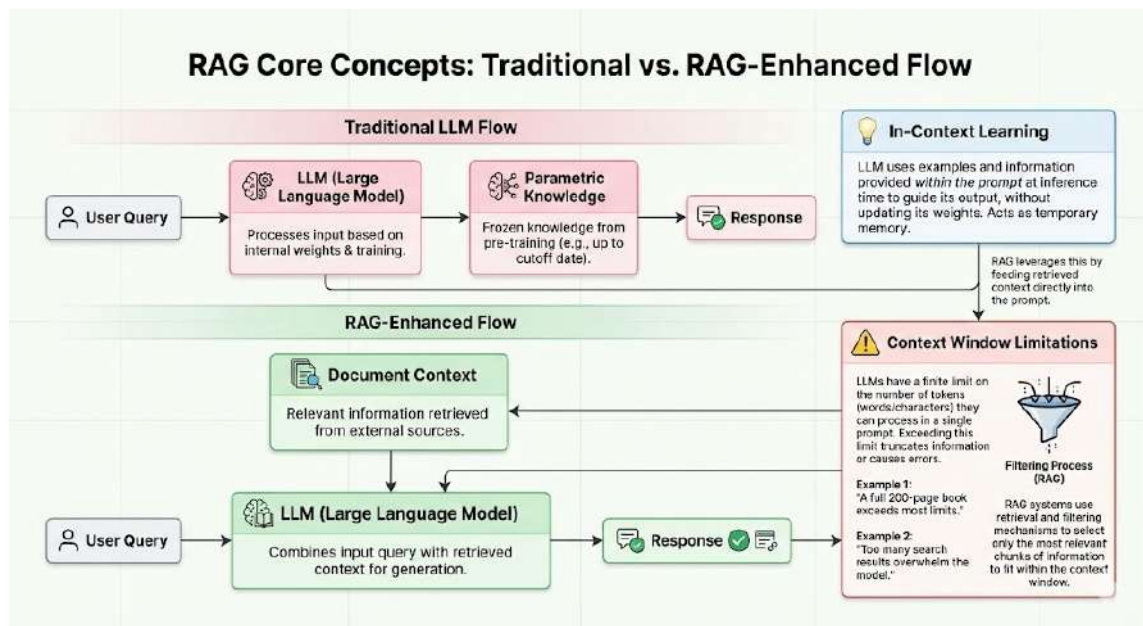


Figure 19.3. Complete RAG Pipeline

- Example: A 100-page book → 100 chunks (one per page, or smaller)

Step 3: Generate Embeddings

- Use an embedding model to convert each chunk into a vector (array of numbers)
- These vectors capture the **semantic meaning** of the text
- Similar content will have similar vectors

Step 4: Store in Vector Database

- Save the embeddings in a specialized database (Vector Store)
- Examples: FAISS, Chroma, Pinecone, Weaviate
- Store both the vector AND the original text

Phase 2: Retrieval

Step 1: Receive User Query

- User asks: "What is Machine Learning?"

Step 2: Convert Query to Embedding

- Use the same embedding model
- Transform the question into a vector

Step 3: Similarity Search

- Compare the query vector against all stored vectors
- Find the top-K most similar vectors
- This identifies which chunks are most relevant to the question

Step 4: Retrieve Original Text

- Get the text content associated with the top-K vectors
- These become your "context"

Step 5: Build Prompt and Generate Response

- Combine: Original Query + Retrieved Context

- Send to LLM
- LLM uses both the context AND its parametric knowledge
- Generate a grounded response

19.5.3 Understanding Embeddings

Embeddings are numerical representations that capture semantic meaning.

Properties of Embeddings:

- Similar concepts have similar vectors
- Enables mathematical comparison of text meaning
- Typical dimensions: 384, 768, 1536 (depending on model)

Example:

```
Text: "Machine Learning is a subset of AI"
↓ Embedding Model
Embedding: [0.12, -0.45, 0.78, 0.23, -0.56, ...]
(hundreds or thousands of dimensions)
```

19.5.4 Similarity Search

When searching, we compare vectors using distance metrics:

Table 19.4. Similarity Search

Metric	Description	Use Case
Cosine Similarity	Measures angle between vectors	Most common for text
Euclidean Distance	Measures straight-line distance	General purpose
Dot Product	Measures alignment	When vectors are normalized

Cosine Similarity Formula:

$$\text{similarity}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n A_i^2} \times \sqrt{\sum_{i=1}^n B_i^2}}$$

Where:

- A and B are embedding vectors
- $A \cdot B$ is the dot product
- $\|A\|$ and $\|B\|$ are the magnitudes (lengths) of the vectors

19.6 Implementation Strategy: RAG as a Tool

In LangGraph, the recommended approach is to implement RAG as a **tool**. This provides several advantages:

19.6.1 Why RAG as a Tool?

1. **Unified Architecture:** Treats RAG like any other tool (web search, calculator, etc.)
2. **Agent Decision Making:** LLM decides when to use RAG based on the query
3. **Composability:** Easy to combine with other tools
4. **Standard Pattern:** Common in production agentic AI applications

Flow Explanation:

1. Query enters at chat_node
2. LLM decides if tools are needed
3. If RAG is needed, control goes to tools node
4. RAG tool retrieves context from vector store
5. Context returns to chat_node
6. LLM generates final answer using query + context
7. Response goes to END

19.7 Step-by-Step Code Implementation

19.7.1 Prerequisites and Setup

```
# Install required packages
!pip install langchain langchain-openai langchain-community langgraph python-dotenv faiss-cpu pypdf
```

19.7.2 Required Imports

```
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_core.tools import tool
from langgraph.graph import StateGraph, START
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode, tools_condition
from typing import Annotated, TypedDict
from langchain_core.messages import HumanMessage, BaseMessage
from dotenv import load_dotenv

load_dotenv()
```

19.7.3 Step 1: Initialize the LLM

```
llm = ChatOpenAI(model='gpt-4o-mini')
```

19.7.4 Step 2: Load the Document

```
# Load PDF document
loader = PyPDFLoader("intro-to-ml.pdf")
docs = loader.load()

print(f"Loaded {len(docs)} pages")
# Output: Loaded 392 pages
```

The PyPDFLoader:

- Takes a PDF file path
- Returns a list of Document objects
- Each Document represents one page

19.7.5 Step 3: Split into Chunks

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,      # Maximum characters per chunk
    chunk_overlap=200     # Overlap between chunks (for context retention)
)

chunks = splitter.split_documents(docs)
print(f"Created {len(chunks)} chunks")
# Output: Created 973 chunks
```

Understanding the Parameters:

Table 19.5. Step 3: Split into Chunks

Parameter	Value	Purpose
chunk_size	1000	Maximum characters in each chunk
chunk_overlap	200	Characters shared between consecutive chunks

Why Overlap? Overlap ensures that context isn't lost at chunk boundaries. If an important concept spans two chunks, the overlap ensures it's captured in at least one chunk completely.

```
Chunk 1: [...text about topic A... | overlap zone ]
Chunk 2:          [ overlap zone | ...continuation of A and topic B...]
```

19.7.6 Step 4: Generate Embeddings and Create Vector Store

```
# Initialize embedding model
embeddings = OpenAIEmbeddings(model='text-embedding-3-small')

# Create vector store and generate embeddings in one step
vector_store = FAISS.from_documents(chunks, embeddings)
```

What happens here:

1. Each chunk is passed to the embedding model
2. Model converts text → vector (array of numbers)
3. Vectors are stored in FAISS (Facebook AI Similarity Search)
4. Original text is also stored alongside vectors

19.7.7 Step 5: Create the Retriever

```
retriever = vector_store.as_retriever(
    search_type='similarity',    # Use similarity-based search
    search_kwargs={'k': 4}      # Return top 4 most similar chunks
)
```

Retriever Parameters:

Table 19.6. Step 5: Create the Retriever

Parameter	Value	Meaning
search_type	'similarity'	Find semantically similar chunks
k	4	Number of chunks to retrieve

19.7.8 Step 6: Test the Retriever

```
# Test retrieval
results = retriever.invoke("What is a decision tree?")

print(f"Retrieved {len(results)} documents")
# Output: Retrieved 4 documents

# Examine first result
print(results[0].page_content)
print(results[0].metadata)
```

Document Object Structure:

```
{
  'id': 'unique-identifier',
  'page_content': 'The actual text content...',
  'metadata': {
    'source': 'intro-to-ml.pdf',
    'page': 45,
    'producer': 'Author Name',
    ...
  }
}
```

19.7.9 Step 7: Create the RAG Tool

```
@tool
def rag_tool(query: str) -> dict:
    """
    Retrieve relevant information from the PDF document.
    Use this tool when the user asks factual/conceptual questions
    that might be answered from the stored documents.
    """
    # Invoke retriever with the query
    result = retriever.invoke(query)

    # Extract page content from each document
    context = [doc.page_content for doc in result]

    # Extract metadata (source info, page numbers, etc.)
    metadata = [doc.metadata for doc in result]

    return {
        'query': query,
        'context': context,
        'metadata': metadata
    }
```

Key Points:

- The @tool decorator converts the function into a LangChain tool
- The docstring is crucial - LLM reads it to understand when to use this tool
- Returns structured data: query, context, and metadata

19.7.10 Step 8: Bind Tools to LLM

```
tools = [rag_tool]
llm_with_tools = llm.bind_tools(tools)
```

This tells the LLM that it has access to the RAG tool and can call it when needed.

19.7.11 Step 9: Define State

```
class ChatState(TypedDict):  
    messages: Annotated[list[BaseMessage], add_messages]
```

The state holds the conversation history. `add_messages` reducer appends new messages to the existing list.

19.7.12 Step 10: Create Chat Node

```
def chat_node(state: ChatState):  
    """LLM node that may answer directly or request a tool call."""  
    messages = state['messages']  
    response = llm_with_tools.invoke(messages)  
    return {'messages': [response]}
```

19.7.13 Step 11: Create Tool Node

```
tool_node = ToolNode(tools)
```

The `ToolNode` from `LangGraph` automatically handles:

- Parsing tool call requests from LLM
- Executing the appropriate tool
- Formatting the response

19.7.14 Step 12: Build the Graph

```
# Initialize graph  
graph = StateGraph(ChatState)  
  
# Add nodes  
graph.add_node('chat_node', chat_node)  
graph.add_node('tools', tool_node)  
  
# Add edges  
graph.add_edge(START, 'chat_node')  
graph.add_conditional_edges('chat_node', tools_condition)  
graph.add_edge('tools', 'chat_node')  
  
# Compile  
chatbot = graph.compile()
```

Understanding `tools_condition`:

This is a prebuilt function that:

- Checks if the LLM response contains tool calls
- If yes → routes to 'tools' node
- If no → routes to END

19.7.15 Step 13: Test the RAG Chatbot

```
result = chatbot.invoke({  
    "messages": [  
        HumanMessage(  
            content="Using the pdf notes, explain how to find the ideal value of K in KNN"  
        )  
    ]  
})
```

```

    ]
  })

  print(result['messages'][-1].content)

```

Expected Flow:

1. Query goes to chat_node
2. LLM sees “Using the pdf notes” and decides to call rag_tool
3. rag_tool retrieves relevant chunks about KNN and K selection
4. Context returns to chat_node
5. LLM generates answer based on query + retrieved context
6. Final answer is returned

19.8 Production-Ready Integration

For production use, additional considerations are needed:

19.8.1 Enhanced RAG Tool with Thread Support

```

# Global storage for per-thread retrievers
_THREAD_RETRIEVERS: Dict[str, Any] = {}
_THREAD_METADATA: Dict[str, dict] = {}

def ingest_pdf(file_bytes: bytes, thread_id: str, filename: str = None) -> dict:
    """
    Build a FAISS retriever for the uploaded PDF and store it for the thread.
    """
    # Save bytes to temporary file
    with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as temp_file:
        temp_file.write(file_bytes)
        temp_path = temp_file.name

    try:
        # Load and process
        loader = PyPDFLoader(temp_path)
        docs = loader.load()

        splitter = RecursiveCharacterTextSplitter(
            chunk_size=1000,
            chunk_overlap=200
        )
        chunks = splitter.split_documents(docs)

        # Create vector store
        vector_store = FAISS.from_documents(chunks, embeddings)
        retriever = vector_store.as_retriever(
            search_type="similarity",
            search_kwargs={"k": 4}
        )

        # Store for this thread
        _THREAD_RETRIEVERS[str(thread_id)] = retriever
        _THREAD_METADATA[str(thread_id)] = {
            "filename": filename,
            "documents": len(docs),
            "chunks": len(chunks),
        }

    return {
        "filename": filename,
        "documents": len(docs),
        "chunks": len(chunks),
    }

finally:
    os.remove(temp_path)

```

19.8.2 Thread-Aware RAG Tool

```
@tool
def rag_tool(query: str, thread_id: Optional[str] = None) -> dict:
    """
    Retrieve relevant information from the uploaded PDF for this chat thread.
    Always include the thread_id when calling this tool.
    """
    retriever = _get_retriever(thread_id)

    if retriever is None:
        return {
            "error": "No document indexed for this chat. Upload a PDF first.",
            "query": query,
        }

    result = retriever.invoke(query)
    context = [doc.page_content for doc in result]
    metadata = [doc.metadata for doc in result]

    return {
        "query": query,
        "context": context,
        "metadata": metadata,
        "source_file": _THREAD_METADATA.get(str(thread_id), {}).get("filename"),
    }
```

19.8.3 Complete Tool Set

```
# Search tool for web queries
search_tool = DuckDuckGoSearchRun(region="us-en")

# Calculator tool
@tool
def calculator(first_num: float, second_num: float, operation: str) -> dict:
    """Perform basic arithmetic: add, sub, mul, div"""
    operations = {
        "add": lambda a, b: a + b,
        "sub": lambda a, b: a - b,
        "mul": lambda a, b: a * b,
        "div": lambda a, b: a / b if b != 0 else "Error: Division by zero"
    }

    if operation not in operations:
        return {"error": f"Unsupported operation '{operation}'"}

    result = operations[operation](first_num, second_num)
    return {"first_num": first_num, "second_num": second_num,
            "operation": operation, "result": result}

# Stock price tool
@tool
def get_stock_price(symbol: str) -> dict:
    """Fetch latest stock price for a given symbol (e.g. 'AAPL', 'TSLA')"""
    url = f"https://www.alphavantage.co/query?function=GLOBAL_QUOTE&symbol={symbol}&apikey=YOUR_API_KEY"
    response = requests.get(url)
    return response.json()

# All tools combined
tools = [search_tool, get_stock_price, calculator, rag_tool]
llm_with_tools = llm.bind_tools(tools)
```

19.8.4 Enhanced Chat Node with System Message

```
def chat_node(state: ChatState, config=None):
```

```

"""LLM node that may answer or request a tool call."""
thread_id = None
if config and isinstance(config, dict):
    thread_id = config.get("configurable", {}).get("thread_id")

system_message = SystemMessage(
    content=(
        "You are a helpful assistant. For questions about the uploaded PDF, "
        f"call the `rag_tool` and include the thread_id `{thread_id}`. "
        "You can also use web search, stock price, and calculator tools when helpful. "
        "If no document is available, ask the user to upload a PDF."
    )
)

messages = [system_message, *state["messages"]]
response = llm_with_tools.invoke(messages, config=config)
return {"messages": [response]}

```

19.8.5 Adding Persistence

```

import sqlite3
from langgraph.checkpoint.sqlite import SqliteSaver

# Create connection and checkpointer
conn = sqlite3.connect(database="chatbot.db", check_same_thread=False)
checkpointer = SqliteSaver(conn=conn)

# Compile graph with checkpointer
chatbot = graph.compile(checkpointer=checkpointer)

```

19.9 Observability and Debugging

19.9.1 Using LangSmith for Tracing

LangSmith provides visibility into the RAG flow:

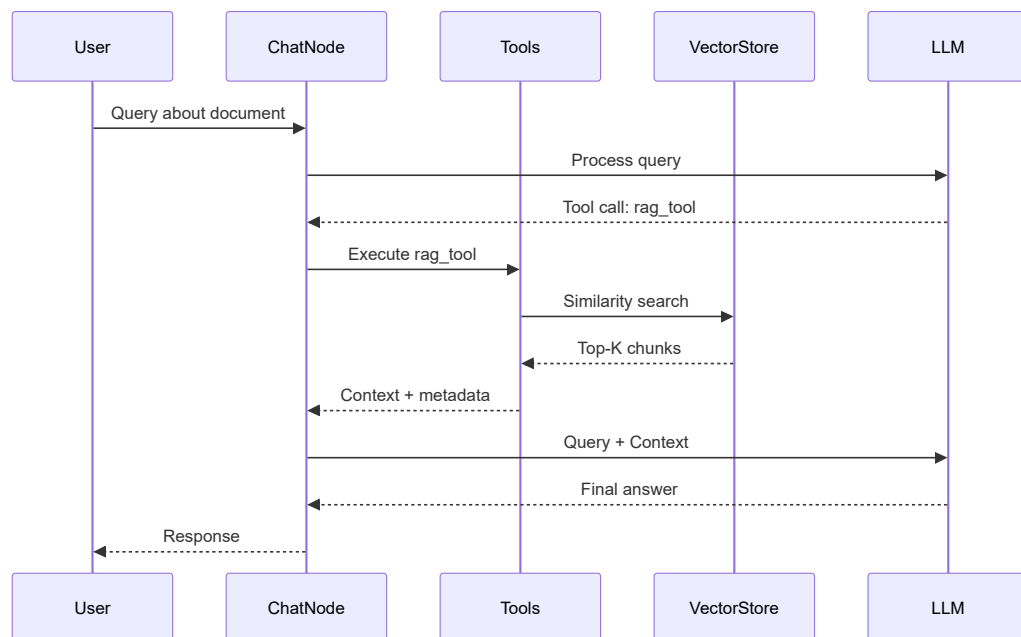


Figure 19.4. Using LangSmith for Tracing

19.9.2 What LangSmith Shows

For each trace, you can see:

1. **Initial Input:** The user's question
2. **First Chat Node:**
 - LLM's decision to call RAG tool
 - Tool call parameters
3. **Tools Node:**
 - RAG tool execution
 - Retrieved chunks (context)
 - Metadata
4. **Second Chat Node:**
 - Full context available
 - Final answer generation

19.9.3 Example Trace Analysis

Table 19.7. *Example Trace Analysis*

Step	Component	Input	Output
1	chat_node	User query	Tool call: rag_tool
2	tools	Query string	4 context chunks
3	chat_node	Query + Context	Final answer

19.10 Key Takeaways

19.10.1 Core Concepts

1. **RAG = In-Context Learning + Retrieval**
 - Provide relevant context to LLM at query time
 - LLM uses context + parametric knowledge to answer
2. **RAG Solves Three Problems:**
 - Outdated knowledge
 - Private data access
 - Hallucination reduction
3. **Context Window Limitation Requires Filtering**
 - Cannot paste entire documents
 - Must retrieve only relevant chunks

19.10.2 Implementation Patterns

1. **RAG as a Tool is the Recommended Pattern**
 - Integrates naturally with other tools
 - LLM decides when to use RAG
 - Standard in production agentic AI

2. The Indexing Pipeline:

Document → Load → Split → Embed → Store in Vector DB

3. The Retrieval Pipeline:

Query → Embed → Search → Top-K Chunks → Prompt → LLM → Response

19.10.3 Best Practices

Table 19.8. Best Practices

Practice	Reason
Use appropriate chunk size (500-1500 chars)	Balance between context and precision
Include chunk overlap (10-20%)	Preserve context across boundaries
Store metadata with embeddings	Enable source attribution
Use thread-specific retrievers	Support multiple concurrent users
Implement proper error handling	Handle missing documents gracefully



Figure 19.5. Best Practices

Human-in-the-Loop (HITL) in LangGraph - Complete Guide

20.1 Code

- <https://github.com/campusx-official/langgraph-tutorials>

20.2 1. Introduction to HITL

Human-in-the-Loop (HITL) is one of the most important and commonly used concepts in modern Agentic AI systems. As AI agents become more autonomous, the need for human oversight at critical decision points becomes paramount. This guide provides a comprehensive understanding of HITL, its implementation in LangGraph, and practical code examples.

20.3 2. What is HITL?

20.3.1 2.1 Definition

HITL (Human-in-the-Loop) is a design approach in AI systems where a human actively participates at critical points of the AI workflow — either to **supervise**, **approve**, **correct**, or **guide** the model’s output.

20.3.2 2.2 Simple Explanation

Think of HITL as putting a **human checkpoint** inside an AI pipeline so that important decisions are **not made autonomously** by the model. It’s like having a human supervisor who reviews and approves critical actions before the AI system proceeds.

20.3.3 2.3 Visual Representation

20.3.4 2.4 Real-World Analogy

Imagine you’re booking a flight using an AI chatbot:

Table 20.1. 4 Real-World Analogy

Step	Action	HITL Status
1	You say “Show me flights from Delhi to Mumbai”	AI handles autonomously
2	AI displays available flights	AI handles autonomously
3	You ask “Show cheap flights between 6-9 AM”	AI handles autonomously
4	You say “Book this ticket”	HITL Checkpoint Triggered

Step	Action	HITL Status
5	AI asks “I’ve selected this flight for ₹5,000. Should I proceed with payment?”	Human decision required
6	You confirm “Yes”	AI proceeds with booking

20.4 3. Why HITL Exists in Agentic AI Systems

20.4.1 3.1 The Autonomy Challenge

Agentic AI systems were created to provide **autonomy** — the ability to complete tasks automatically without human intervention. However, current LLMs (Large Language Models) are **not perfect**. They can:

- **Misinterpret** user goals
- Struggle with **ambiguous** queries
- **Hallucinate** incorrect information
- Make **wrong decisions** in complex scenarios

20.4.2 3.2 Two Primary Reasons for HITL

Reason 1: To Help Agentic Systems

Since LLMs are not perfect, they may:

Table 20.2. Reason 1: To Help Agentic Systems

Issue	Example
Misinterpretation	User says “Book flight for next Friday” — Does this mean this week’s Friday or next week’s Friday?
Ambiguity	“Transfer money to John” — Which John? What amount?
Hallucination	AI confidently provides incorrect flight prices or unavailable options

Solution: Bring a human into the picture to provide judgment and resolve these issues.

Reason 2: To Add Accountability

No matter how powerful or intelligent AI systems become, they cannot provide **accountability**.

Example Scenario:

- Google implements an AI system in Gmail that auto-generates and sends email replies
- If the AI sends an inappropriate or incorrect reply, you cannot blame the AI
- A human must be in the loop to approve the response before sending

Key Principle: You can blame a human for mistakes, but you cannot hold an AI system legally or ethically accountable.

20.5 4. Benefits of HITL

20.5.1 4.1 Comprehensive Benefits Overview

20.5.2 4.2 Detailed Benefits Table

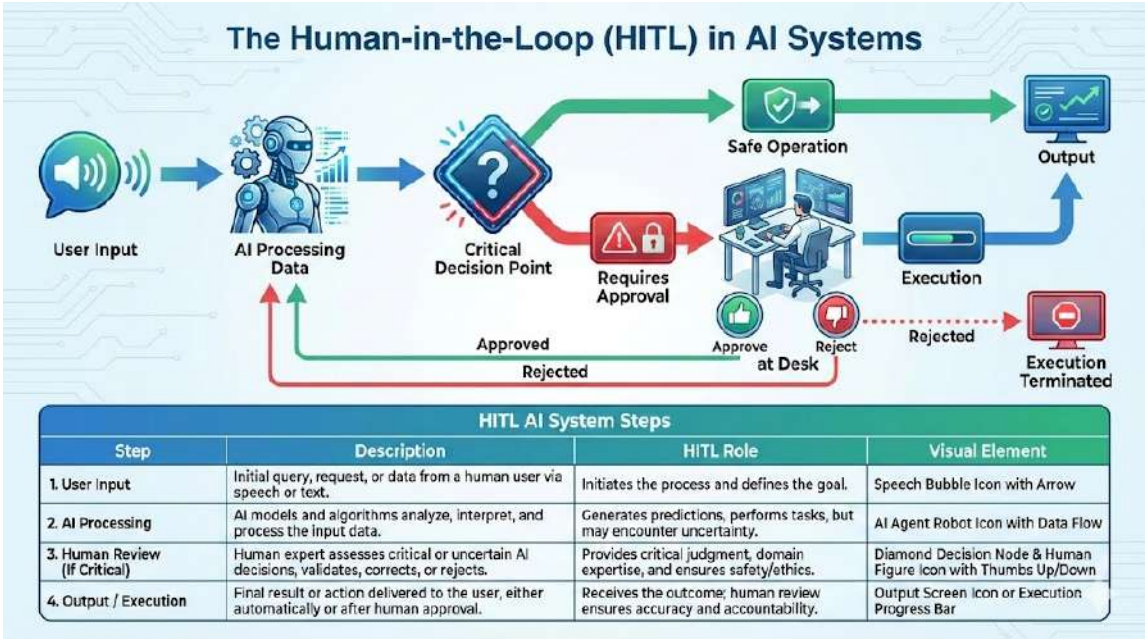


Figure 20.1. 3 Visual Representation

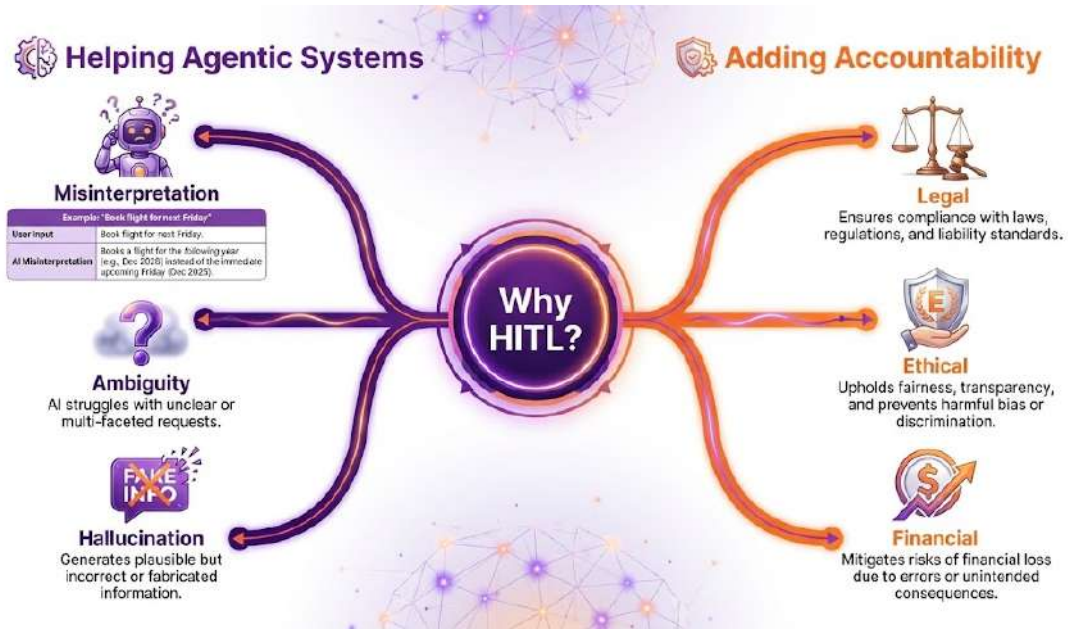


Figure 20.2. 2 Two Primary Reasons for HITL

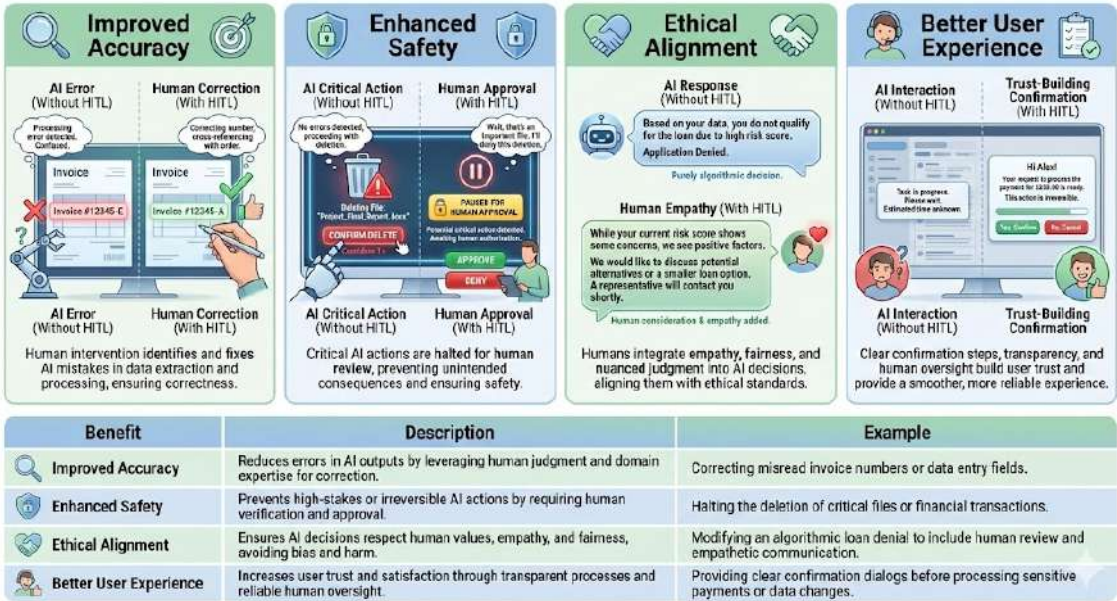


Figure 20.3. 1 Comprehensive Benefits Overview

Table 20.3. 2 Detailed Benefits Table

Benefit	Description	Example
Accuracy	Human judgment catches AI mistakes	Invoice scanning: AI reads ₹1,200 as ₹12,000 — Human corrects it
Safety	Prevents potentially harmful actions	“Delete all files not used in 30 days” — AI asks before deleting important project files
Ethical Alignment	Ensures responses match company values	Customer complaint: AI adds empathy to technically correct but cold response
User Experience	Builds trust through transparency	Payment confirmation before processing

20.5.3 4.3 Accuracy Example - Invoice Processing

Scenario: User uploads invoice and asks AI to make payment

Without HITL:—

AI scans invoice—
AI misreads ₹1,200 as ₹12,000—
AI makes incorrect payment

With HITL:—

AI scans invoice—
AI asks: “I extracted ₹12,000 from the invoice. Is this correct?”—
Human responds: “No, it should be ₹1,200”—
AI corrects and processes correctly

20.5.4 4.4 Safety Example - File Deletion

User Request: “Delete all files I haven't used in 30 days”

Without HITL:—

AI identifies 50 unused files|—
 AI deletes all files immediately|—
 Critical project files lost

With HITL:|—

AI identifies 50 unused files|—
 AI asks: "10 of these files belong to your current project. Delete them too?"|—
 Human responds: "No, keep project files"|—
 AI deletes only safe files

20.6 5. Common HITL Patterns

20.6.1 5.1 Pattern Overview

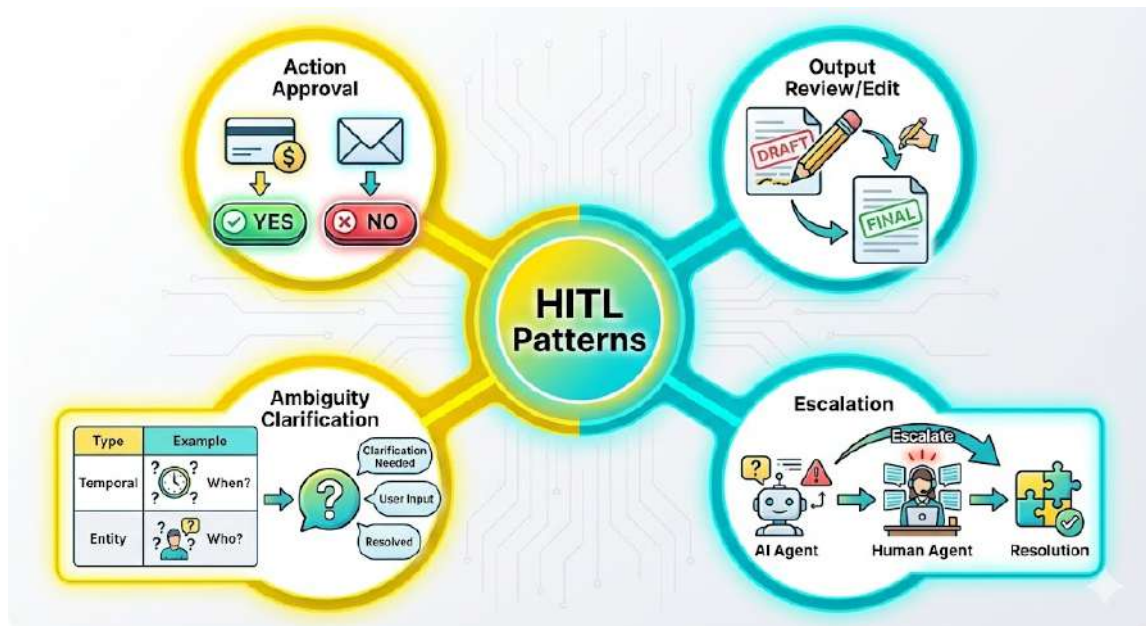


Figure 20.4. 1 Pattern Overview

20.6.2 5.2 Detailed Pattern Descriptions

Pattern 1: Action Approval Pattern (Most Common)

Definition: Before executing any crucial action, the AI asks for human approval.

Table 20.4. Pattern 1: Action Approval Pattern (Most Common)

Use Case	AI Action	Human Decision
Payment Processing	"I'm about to pay ₹5,000. Proceed?"	Yes/No
Email Sending	"Send this email to the client?"	Approve/Edit/Reject
File Deletion	"Delete 50 files from server?"	Confirm/Cancel
Data Modification	"Update database record?"	Allow/Deny

Code Pattern:

```
decision = interrupt({
  "type": "approval",
  "action": "payment",
```

```
"amount": 5000,  
"instruction": "Approve this payment? yes/no"  
})  
if decision == "yes":  
    execute_payment()  
else:  
    cancel_operation()
```

Pattern 2: Output Review/Edit Pattern

Definition: AI generates output (draft), human reviews and potentially edits before final action.

Example Flow:

1. AI researches topic for blog post
2. AI generates draft
3. Human reviews draft
4. Human approves or requests changes
5. AI publishes (if approved) or revises (if changes requested)

Use Cases:

- Blog post publishing
- Social media content
- Report generation
- Email drafting

Pattern 3: Ambiguity Clarification Pattern

Definition: When AI encounters unclear or ambiguous input, it asks for clarification.

Example:

```
User: "Book flight tickets for next Friday"  
Today: Monday  
  
AI Response: "I noticed some ambiguity in your request:  
- Did you mean this Friday (in 4 days)?  
- Or next week's Friday (in 11 days)?  
Please clarify."
```

Common Ambiguity Types:

Table 20.5. Pattern 3: Ambiguity Clarification Pattern

Type	Example	Clarification Needed
Temporal	"Next Friday"	Which Friday?
Entity	"Send to John"	Which John?
Quantity	"Buy some stocks"	How many?
Context	"Continue the project"	Which project?

Pattern 4: Escalation Pattern

Definition: AI handles routine tasks but escalates complex issues to human agents.

Common in: Customer support systems

Example: Swiggy/Zomato customer support

- Simple queries (order status, refund policy) → AI handles
- Complex issues (payment disputes, serious complaints) → Escalated to human

20.7 6. HITL in LangGraph - Architecture Overview

20.7.1 6.2 Social Media Manager Example

Consider a Social Media Manager Agent that:

1. Researches a topic
2. Generates a tweet draft
3. Posts to Twitter/X

20.8 7. Core Components for HITL Implementation

20.8.1 7.1 Essential Components

Table 20.6. 1 Essential Components

Component	Purpose	Required
<code>interrupt()</code>	Pauses execution and sends message to frontend	Yes
<code>Command(resume=...)</code>	Resumes execution with user's decision	Yes
MemorySaver / Checkpointer	Saves state during pause	Yes
<code>thread_id</code>	Identifies conversation/session	Yes

20.8.2 7.2 The `interrupt()` Function

Definition: The `interrupt()` function pauses graph execution and sends a message to the frontend for human input.

Syntax:

```
from langgraph.types import interrupt

decision = interrupt({
    "type": "approval",
    "reason": "Description of why approval is needed",
    "data": "Relevant data for decision",
    "instruction": "What action is expected from user"
})
```

What happens when `interrupt()` is called:

20.8.3 7.3 The Command Function

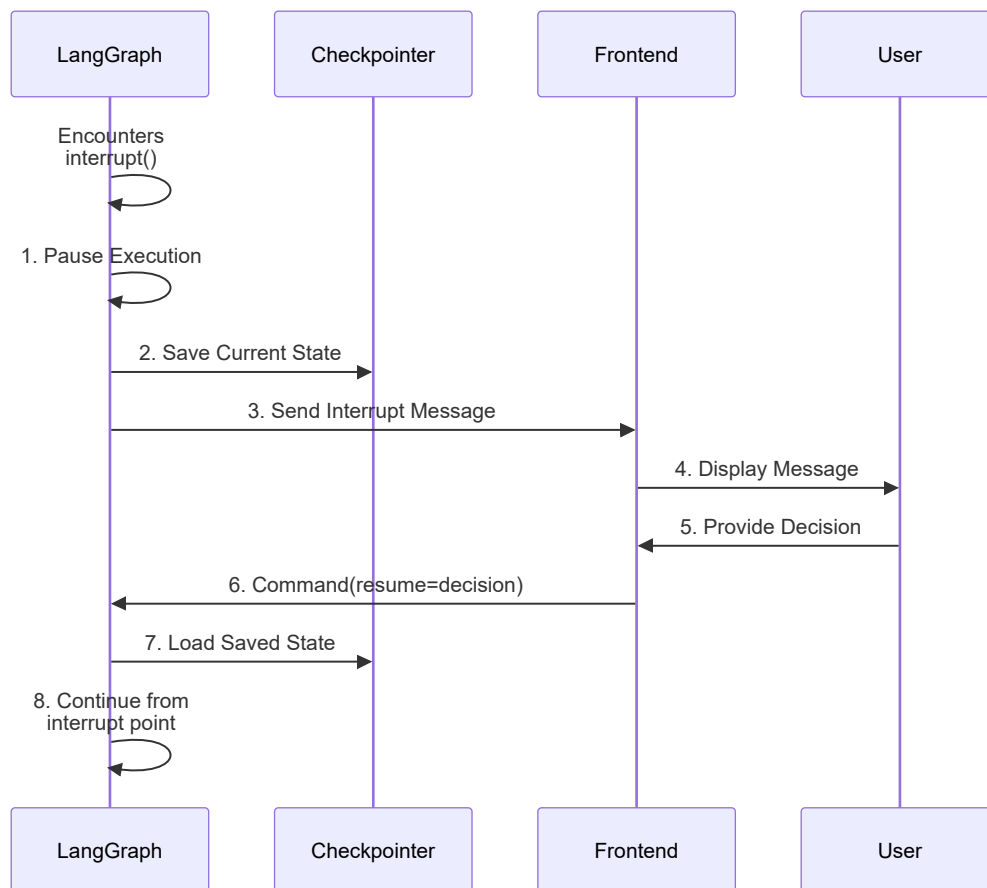
Definition: The Command function is used to resume graph execution with the user's decision.

Syntax:

```
from langgraph.types import Command

# Resume with dictionary
result = graph.invoke(
    Command(resume={"approved": "yes"}),
    config={"configurable": {"thread_id": "123"}}
)

# Resume with simple value
result = graph.invoke(
    Command(resume="yes"),
    config={"configurable": {"thread_id": "123"}}
)
```

Figure 20.5. 2 The `interrupt()` Function

20.8.4 7.4 Checkpointer (Memory Saver)

Definition: Checkpointer saves the graph state when execution is paused, allowing it to resume from the exact same point.

Why Required for HITL:

1. Graph execution pauses at `interrupt()`
2. State must be persisted somewhere
3. When `Command(resume=...)` is called, state is loaded back
4. Execution continues from where it paused

Types of Checkpointers:

Table 20.7. 4 Checkpointer (Memory Saver)

Type	Use Case	Persistence
MemorySaver	Development/Testing	In-memory (lost on restart)
SqliteSaver	Production (simple)	SQLite database
PostgresSaver	Production (scalable)	PostgreSQL database

Example:

```

from langgraph.checkpoint.memory import MemorySaver

checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)
  
```


20.9 8. Step-by-Step HITL Workflow in LangGraph

20.9.1 8.1 Complete Workflow Diagram

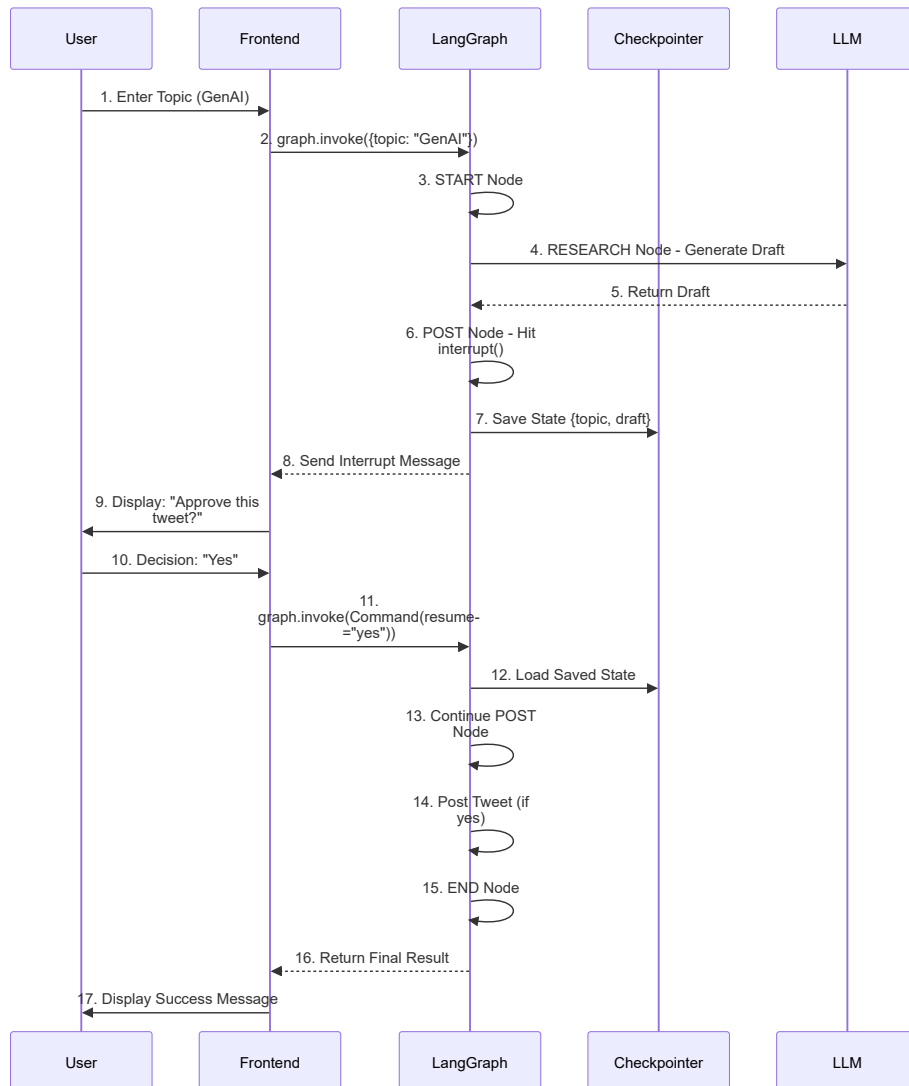


Figure 20.6. 1 Complete Workflow Diagram

20.9.2 8.2 Step-by-Step Explanation

Step 1-2: Initial Invocation

```

# User provides topic
initial_input = {"messages": [{"user", "Generate tweet about GenAI"}]}

# Frontend invokes the graph
result = graph.invoke(initial_input, config={"configurable": {"thread_id": "123"}})
  
```

Step 3-5: Processing Phase

```
def research_node(state):  
    # Research and generate draft  
    draft = llm.invoke(f"Generate tweet about {state['topic']}")  
    return {"draft": draft}
```

Step 6-8: Interrupt Phase

```
def post_node(state):  
    # HITL Checkpoint  
    decision = interrupt({  
        "type": "approval",  
        "message": f"I prepared this draft: {state['draft']}",  
        "instruction": "Should I post it? yes/no"  
    })  
  
    # This code executes AFTER user resumes  
    if decision == "yes":  
        post_tweet(state['draft'])  
        return {"status": "posted"}  
    else:  
        return {"status": "rejected"}
```

Step 9-11: User Decision

```
# Frontend receives interrupt message  
interrupt_message = result['__interrupt__'][0].value  
  
# Display to user and get decision  
user_decision = input(f"{interrupt_message['instruction']}: ")  
  
# Resume graph with decision  
final_result = graph.invoke(  
    Command(resume=user_decision),  
    config={"configurable": {"thread_id": "123"}}  
)
```

20.9.3 8.3 State Flow During HITL

20.10 9. Code Implementation - Basic Example

20.10.1 9.1 Scenario Description

A simple chatbot that asks for user approval before answering questions with an LLM.

Flow:

1. User asks a question
2. AI asks: "Do you really want me to answer this?"
3. If Yes → AI answers using LLM
4. If No → AI responds "Not approved"

20.10.2 9.2 Complete Code

```
# =====  
# IMPORTS  
# =====  
from typing import Annotated, TypedDict  
from langchain_openai import ChatOpenAI
```

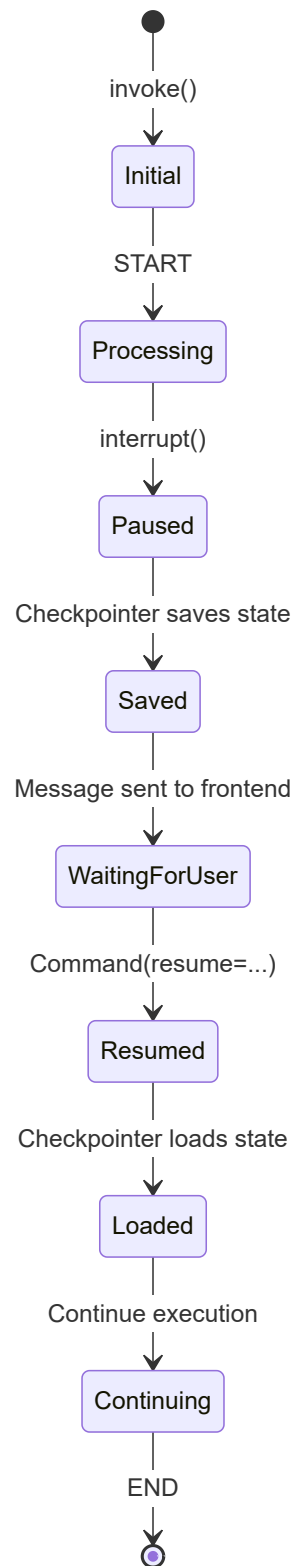


Figure 20.7. 3 State Flow During HITL

```

from langchain_core.messages import BaseMessage, AIMessage
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import interrupt, Command
from dotenv import load_dotenv

# Load environment variables
load_dotenv()

# =====
# INITIALIZE LLM
# =====
llm = ChatOpenAI(model="gpt-4.1-mini")

# =====
# DEFINE STATE
# =====
class ChatState(TypedDict):
    """
    State definition for the chat graph.

    Attributes:
        messages: List of conversation messages with add_messages reducer
    """
    messages: Annotated[list[BaseMessage], add_messages]

# =====
# DEFINE HITL NODE
# =====
def chat_node(state: ChatState):
    """
    Chat node with HITL implementation.

    This node:
    1. Triggers an interrupt to ask for approval
    2. Waits for user decision
    3. Proceeds based on approval status

    Args:
        state: Current graph state containing messages

    Returns:
        Updated state with AI response
    """
    # STEP 1: Trigger interrupt and wait for human decision
    decision = interrupt({
        "type": "approval",
        "reason": "Model is about to answer a user question.",
        "question": state["messages"][-1].content,
        "instruction": "Approve this question? yes/no"
    })

    # STEP 2: Check user's decision (this runs AFTER resume)
    if decision.get("approved") == 'no':
        return {"messages": [AIMessage(content="Not approved.")]}
    else:
        # STEP 3: If approved, invoke LLM and return response
        response = llm.invoke(state["messages"])
        return {"messages": [response]}

# =====
# BUILD GRAPH
# =====
# Create state graph
builder = StateGraph(ChatState)

# Add nodes
builder.add_node("chat", chat_node)

# Add edges
builder.add_edge(START, "chat")
builder.add_edge("chat", END)

```

```

# CRITICAL: Checkpointer is REQUIRED for interrupts
checkpointer = MemorySaver()

# Compile the graph
app = builder.compile(checkpointer=checkpointer)

# =====
# EXECUTION - STEP 1: INITIAL INVOKE
# =====
# Configuration with thread ID
config = {"configurable": {"thread_id": "1234"}}

# User's question
initial_input = {
    "messages": [
        ("user", "Explain gradient descent in very simple terms.")
    ]
}

# First invocation - will pause at interrupt
result = app.invoke(initial_input, config=config)

# =====
# EXECUTION - STEP 2: HANDLE INTERRUPT
# =====
# Extract interrupt message
interrupt_data = result['__interrupt__'][0].value
print(f"Interrupt received: {interrupt_data}")

# Simulate frontend: Get user input
user_input = input(f"\n{interrupt_data['instruction']}: ")

# =====
# EXECUTION - STEP 3: RESUME WITH DECISION
# =====
# Resume the graph with user's decision
final_result = app.invoke(
    Command(resume={"approved": user_input}),
    config=config,
)

# Print final response
print(f"\nAI Response: {final_result['messages'][-1].content}")

```

20.10.3 9.3 Graph Visualization

20.10.4 9.4 Expected Output

Scenario 1: User approves

```

Interrupt received: {'type': 'approval', 'reason': 'Model is about to answer...',
                    'question': 'Explain gradient descent...',
                    'instruction': 'Approve this question? yes/no'}

```

Approve this question? yes/no: yes

AI Response: Gradient descent is like finding the lowest point in a valley by taking small steps downhill. You keep moving in the direction that goes down the most until you reach the bottom...

Scenario 2: User rejects

Approve this question? yes/no: no

AI Response: Not approved.

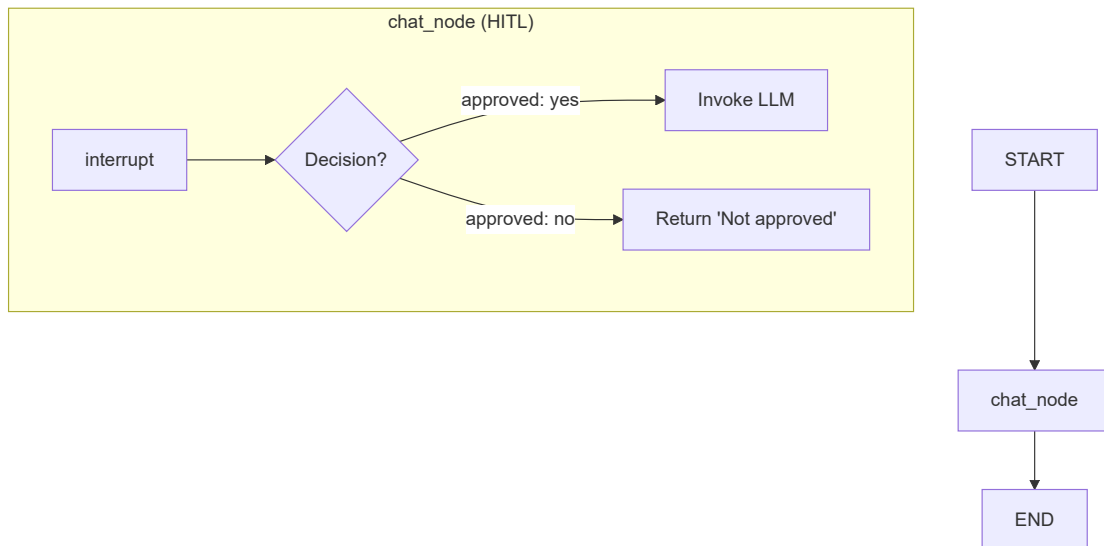


Figure 20.8. 3 Graph Visualization

20.11 10. Code Implementation - Advanced Chatbot Example

20.11.1 10.1 Scenario Description

A stock trading chatbot with two tools:

1. **get_stock_price**: Fetches current stock price (safe, no HITL needed)
2. **purchase_stocks**: Buys stocks (risky, HITL required)

20.11.2 10.3 Complete Code

```

# =====
# IMPORTS
# =====
from typing import Annotated, TypedDict, Literal
from langchain_openai import ChatOpenAI
from langchain_core.messages import BaseMessage, AIMessage, ToolMessage
from langchain_core.tools import tool
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import interrupt, Command
from langgraph.prebuilt import ToolNode
import requests
from dotenv import load_dotenv

load_dotenv()

# =====
# DEFINE TOOLS
# =====

@tool
def get_stock_price(company: str) -> str:
    """
    Fetches the current stock price for a given company.

    This is a SAFE tool - no HITL required.

    Args:
        company: Stock ticker symbol (e.g., 'AAPL', 'GOOGL')

```

```

Returns:
    Current stock price as a string
    """
url = f"https://query1.finance.yahoo.com/v8/finance/chart/{company}"
headers = {'User-Agent': 'Mozilla/5.0'}

try:
    response = requests.get(url, headers=headers)
    data = response.json()
    price = data['chart']['result'][0]['meta']['regularMarketPrice']
    return f"The current stock price of {company} is ${price:.2f}"
except Exception as e:
    return f"Error fetching stock price: {str(e)}"

@tool
def purchase_stocks(company: str, quantity: int) -> str:
    """
    Purchases stocks for the specified company.

    This is a RISKY tool - HITL is REQUIRED.

    Args:
        company: Stock ticker symbol
        quantity: Number of shares to purchase

    Returns:
        Purchase confirmation or rejection message
    """
    # ===== HITL IMPLEMENTATION =====
    # Step 1: Trigger interrupt and wait for approval
    decision = interrupt(
        f"Approve buying {quantity} shares of {company}? (yes/no)"
    )

    # Step 2: Process decision (executes after resume)
    if isinstance(decision, str) and decision.lower() == 'yes':
        # Simulate successful purchase
        return f"Purchase successful! Bought {quantity} shares of {company}."
    else:
        return f"Purchase cancelled. No shares of {company} were bought."

# List of all tools
tools = [get_stock_price, purchase_stocks]

# =====
# INITIALIZE LLM WITH TOOLS
# =====
llm = ChatOpenAI(model="gpt-4.1-mini")
llm_with_tools = llm.bind_tools(tools)

# =====
# DEFINE STATE
# =====
class ChatState(TypedDict):
    """State for the stock trading chatbot."""
    messages: Annotated[List[BaseMessage], add_messages]

# =====
# DEFINE NODES
# =====

def chat_node(state: ChatState):
    """
    Main chat node that processes user messages.

    Uses LLM with bound tools to determine response.
    """
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response]}

```

```

def should_continue(state: ChatState) -> Literal["tools", END]:
    """
    Conditional edge function to determine next step.

    Returns:
        "tools" if tool calls are present, END otherwise
    """
    last_message = state["messages"][-1]

    if hasattr(last_message, 'tool_calls') and last_message.tool_calls:
        return "tools"
    return END

# =====
# BUILD GRAPH
# =====
# Create state graph
builder = StateGraph(ChatState)

# Add nodes
builder.add_node("chat", chat_node)
builder.add_node("tools", ToolNode(tools))

# Add edges
builder.add_edge(START, "chat")
builder.add_conditional_edges("chat", should_continue)
builder.add_edge("tools", "chat")

# Checkpointer for HITL
checkpointer = MemorySaver()

# Compile
graph = builder.compile(checkpointer=checkpointer)

# =====
# MAIN EXECUTION LOOP
# =====
def run_chatbot():
    """
    Main function to run the stock trading chatbot.

    Implements a CLI-based chat interface with HITL support.
    """
    thread_id = "stock-trading-001"
    config = {"configurable": {"thread_id": thread_id}}

    print("=" * 50)
    print("Stock Trading Chatbot with HITL")
    print("Commands: 'exit' to quit")
    print("=" * 50)

    while True:
        # Get user input
        user_message = input("\nYou: ").strip()

        if user_message.lower() == 'exit':
            print("Goodbye!")
            break

        if not user_message:
            continue

        # Create input state
        input_state = {
            "messages": [("user", user_message)]
        }

        # Invoke graph
        result = graph.invoke(input_state, config=config)

        # Check for interrupt (HITL checkpoint)
        while '__interrupt__' in result and result['__interrupt__']:
            # Extract interrupt message

```



```

interrupt_info = result['__interrupt__'][0]
interrupt_message = interrupt_info.value

print(f"\n HITL Checkpoint: {interrupt_message}")
user_decision = input("Your decision: ").strip()

# Resume with decision
result = graph.invoke(
    Command(resume=user_decision),
    config=config
)

# Display final response
if result["messages"]:
    last_message = result["messages"][-1]
    if hasattr(last_message, 'content') and last_message.content:
        print(f"\nAssistant: {last_message.content}")

# Run the chatbot
if __name__ == "__main__":
    run_chatbot()

```



Human-in-the-Loop (HITL) in LangGraph • github.com/camptocamp/langgraph-tutorials • Campux, Tutorial

Figure 20.9. 3 Complete Code

LangGraph Subgraphs - Complete Guide

21.1 Video Overview

Instructor: Nitesh **Series:** Agentic AI using LangGraph **Topic:** Subgraphs - Building Multi-Agent Systems **Importance:** Critical for complex GenAI workflows and multi-agent architectures

21.2 Plan of Action

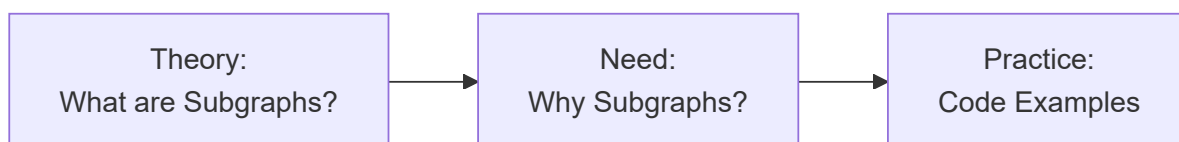


Figure 21.1. Plan of Action

21.2.1 Three-Part Structure:

1. **Theoretical Level** - Understanding subgraphs conceptually
2. **Need Analysis** - Why subgraphs matter in AI workflows
3. **Practical Implementation** - Code examples in LangGraph

21.3 Part 1: What Are Subgraphs?

21.3.1 Core Concept:

```

Traditional Graph: ──
Node 1 (Task A) ──
Node 2 (Task B) ──
Node 3 (Task C)

Subgraph Architecture: ──
Node 1 (Task A) ──
Node 2 → [Complete Graph Inside] ← This is a SUBGRAPH ──
Node 3 (Task C)
  
```

21.3.2 Definition:

A Subgraph in LangGraph is a graph that is embedded and executed as a node inside another graph.

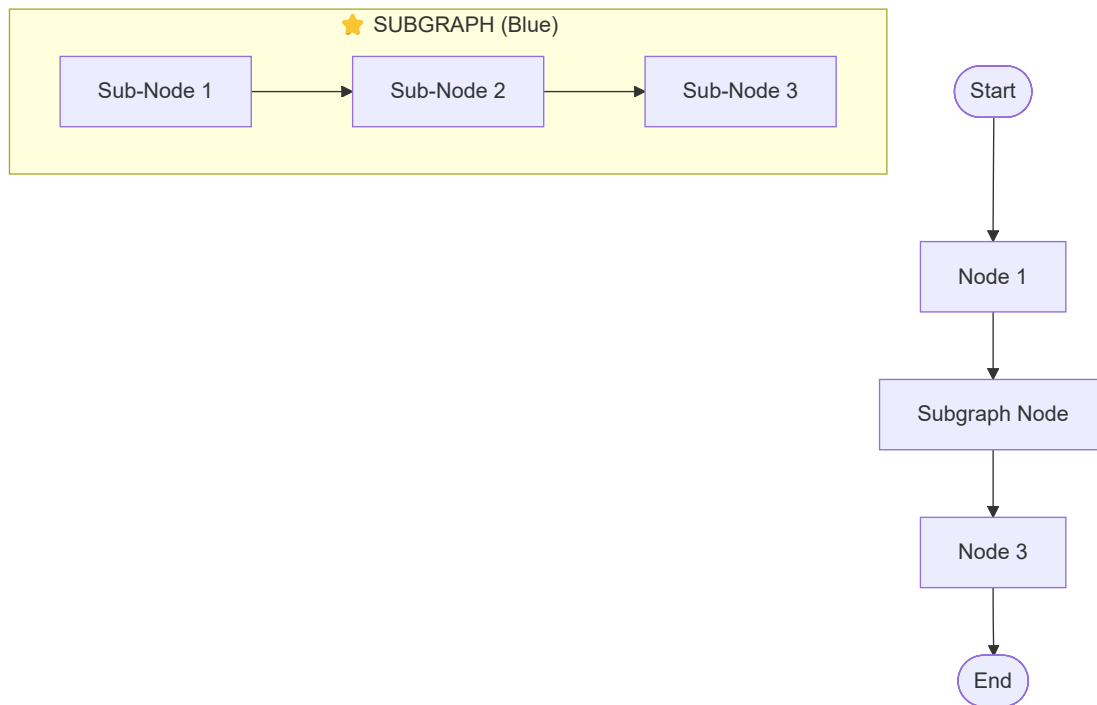


Figure 21.2. Visual Representation

21.3.3 Visual Representation:

Key Points:

- The entire outer structure is a graph
- Nodes inside can themselves be graphs
- Inner graphs = **Subgraphs**
- Creates hierarchical architecture

21.4 Part 2: Why Are Subgraphs Needed?

21.4.1 Evolution of GenAI Applications

Simple Application (Past):

User Query → LLM → Output

Characteristics: Linear, single-purpose, straightforward

Complex Application (Present):

Modern GenAI systems can include:|—
 Tool Integration|—
 RAG (Retrieval Augmented Generation)|—
 Conditional Routing|—
 Retry Logic|—
 Memory Management|—
 HITL (Human-in-the-Loop)|—
 Evaluation|—
 Guardrails

21.5 Real-World Example: Software Development Agent

21.5.1 Scenario: Build an agent that develops complete software

21.5.2 Development Workflow:

21.5.3 Without Subgraphs:

```
Problems:|—
One massive, complex graph|—
Difficult to maintain|—
Hard to debug|—
Single point of failure|—
Shared state confusion|—
Impossible to scale
```

21.5.4 With Subgraphs:

Solution: Break into specialized agents

```
Software Dev Agent (Parent)|—
Planning Agent (Subgraph 1)|
  |— Own tools|
  |— Own memory|
  |— Own guardrails|—
Backend Coding Agent (Subgraph 2)|
  |— Own tools|
  |— Own retry logic|
  |— Own evaluation|—
Frontend Coding Agent (Subgraph 3)|—
Testing Agent (Subgraph 4)|—
Code Review Agent (Subgraph 5)|—
DevOps Agent (Subgraph 6)
```

Architecture Transformation:

```
Complex Monolithic Agent
  ↓
Multiple Specialized Agents
  ↓
Multi-Agent Architecture
  ↓
Each Agent = One Subgraph
```

21.6 Core Benefits of Subgraphs

21.6.1 1 Modularity

```
# Software Engineering Principle Applied
Code Should Be Modular
  ↓
AI Workflows Should Be Modular
  ↓
Breaking Down Entire System into Functions
```

Benefits:

- Easier to understand
- Simpler to develop
- Clear separation of concerns

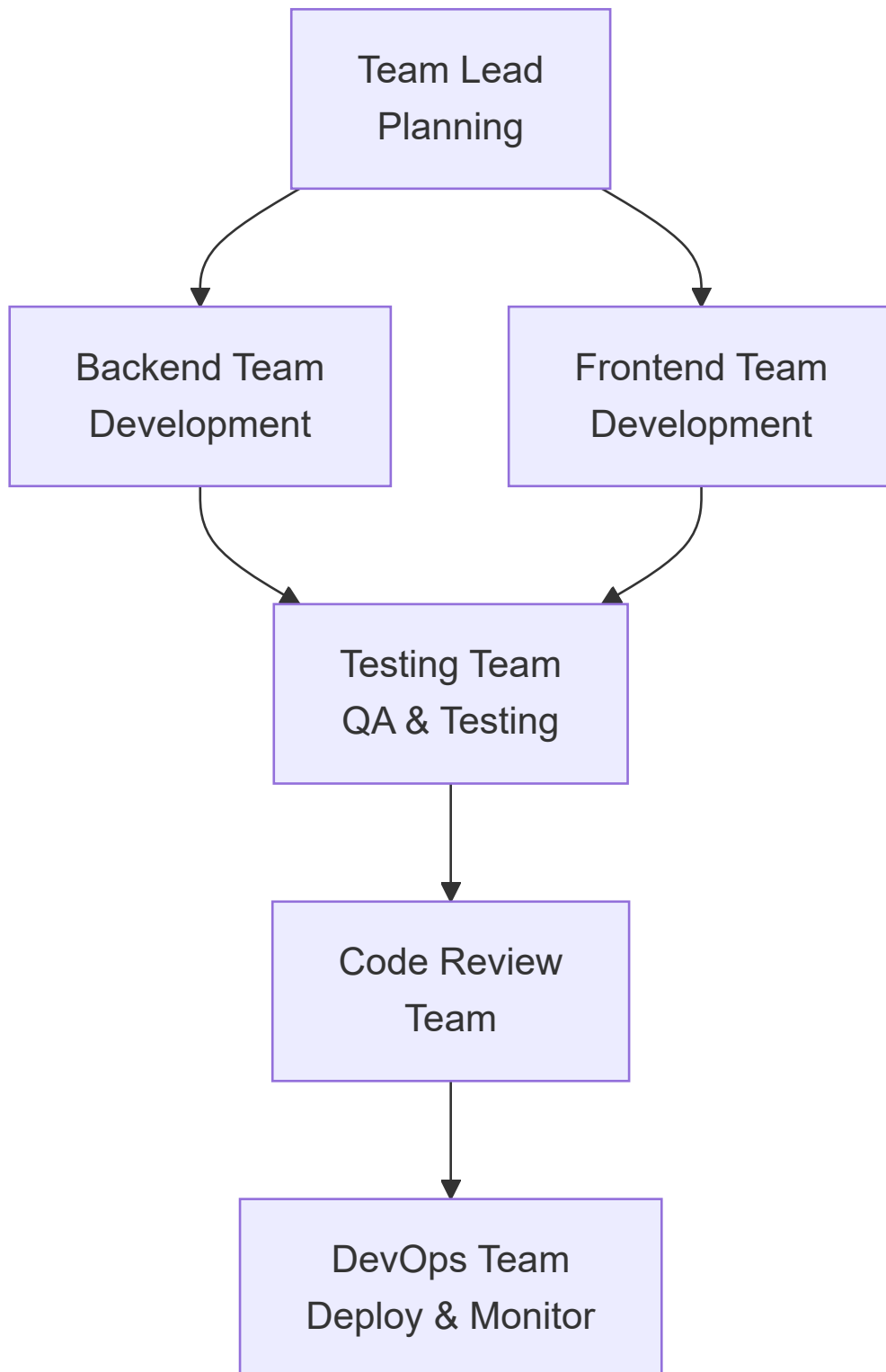


Figure 21.3. *Development Workflow*

- Independent development

21.6.2 2 Reusability

Example:

```
Coding Agent (Subgraph) |—
  Used for Backend Team |—
    Used for Frontend Team
```

Same logic, different contexts
Essentially both are coding
Just different file sets

Benefits:

- Write once, use multiple times
- Consistent behavior across uses
- Reduced code duplication
- Faster development

21.6.3 3 Maintainability

Debugging Scenario:

Without Subgraphs:
"Something broke!" → Search entire massive graph

With Subgraphs:
"Testing Agent failed!" → Debug only Testing subgraph

Benefits:

- Easier debugging
- Isolated testing
- Pinpoint error location
- Faster fixes

21.7 LangGraph-Specific Benefits

21.7.1 1 Failure Isolation

Scenario: Testing Agent Fails

Without Subgraphs:
Testing fails → Entire graph crashes

With Subgraphs:
Testing subgraph fails → Rest continues with warnings

How It Works:

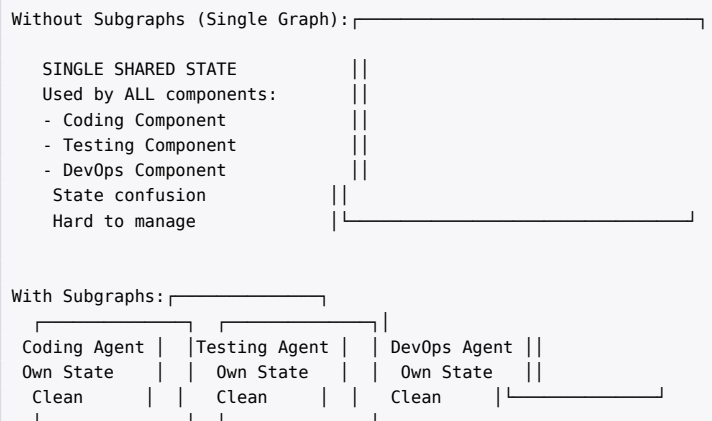
```
LangGraph Design: |—
  Subgraph failure is contained |—
  Parent graph continues execution |—
  Warnings issued but not fatal |—
  Other subgraphs unaffected
```

Benefits:

- Graceful degradation

- Partial success possible
- Better error handling
- System resilience

21.7.2 2 State Separation



State Isolation:

```

Coding Agent State:
  code_files: []
  current_module: "backend"
  dependencies: []

Testing Agent State:
  test_results: []
  coverage_score: 0.85
  failed_tests: []

Code Review Agent State:
  review_comments: []
  severity_levels: {}
  approved: false

```

Benefits:

- No state collision
- Clear data boundaries
- Easier reasoning
- Reduced bugs

21.7.3 3 Observability

LangSmith Integration:

Without Subgraphs: Monitor entire application (all-or-nothing)

With Subgraphs:

- Monitor Coding Agent specifically
 - Token consumption
 - Average latency
 - Success rate
- Monitor Testing Agent separately
- Monitor Code Review Agent independently

Granular Tracing:


```
# Track specific subgraph metrics
```

```
Coding Agent: |—
Tokens Used: 15,234 |—
Avg Latency: 2.3s |—
Success Rate: 94% |—
Error Rate: 6%
```

```
Testing Agent: |—
Tokens Used: 8,456 |—
Avg Latency: 1.8s |—
Success Rate: 98% |—
Error Rate: 2%
```

Benefits:

- Precise performance monitoring
- Identify bottlenecks
- Optimize specific components
- Better debugging

21.8 Part 3: Implementation Mechanisms

21.8.1 Two Ways to Implement Subgraphs

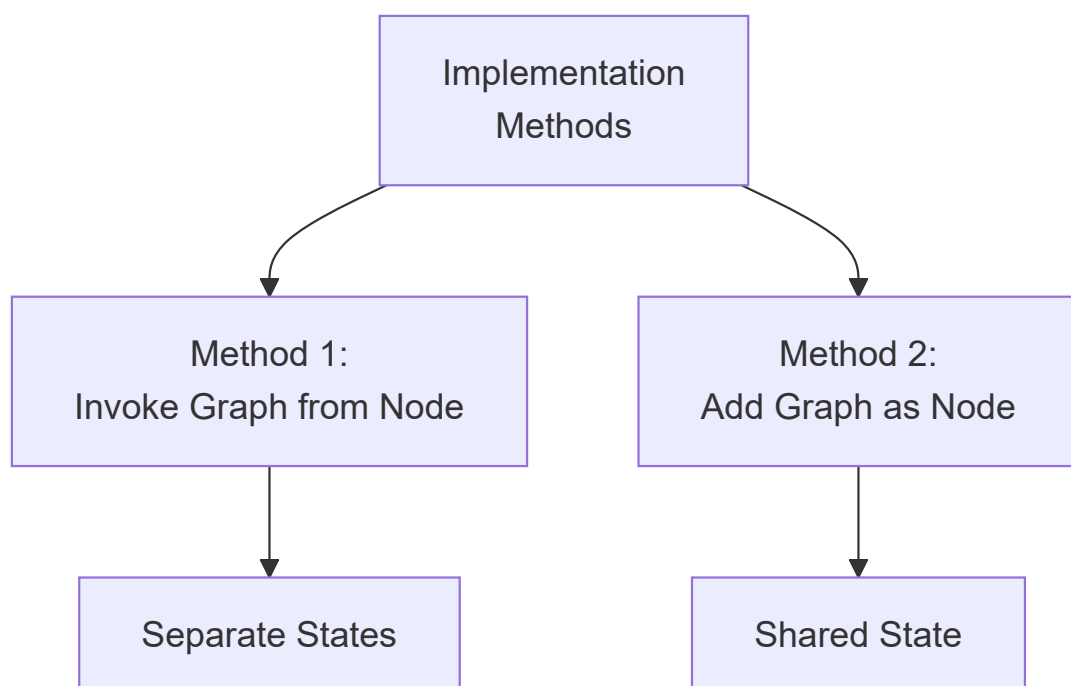


Figure 21.4. *Two Ways to Implement Subgraphs*

21.8.2 Method 1: Invoke Graph from Node

Architecture:

Characteristics:

Two independent graphs

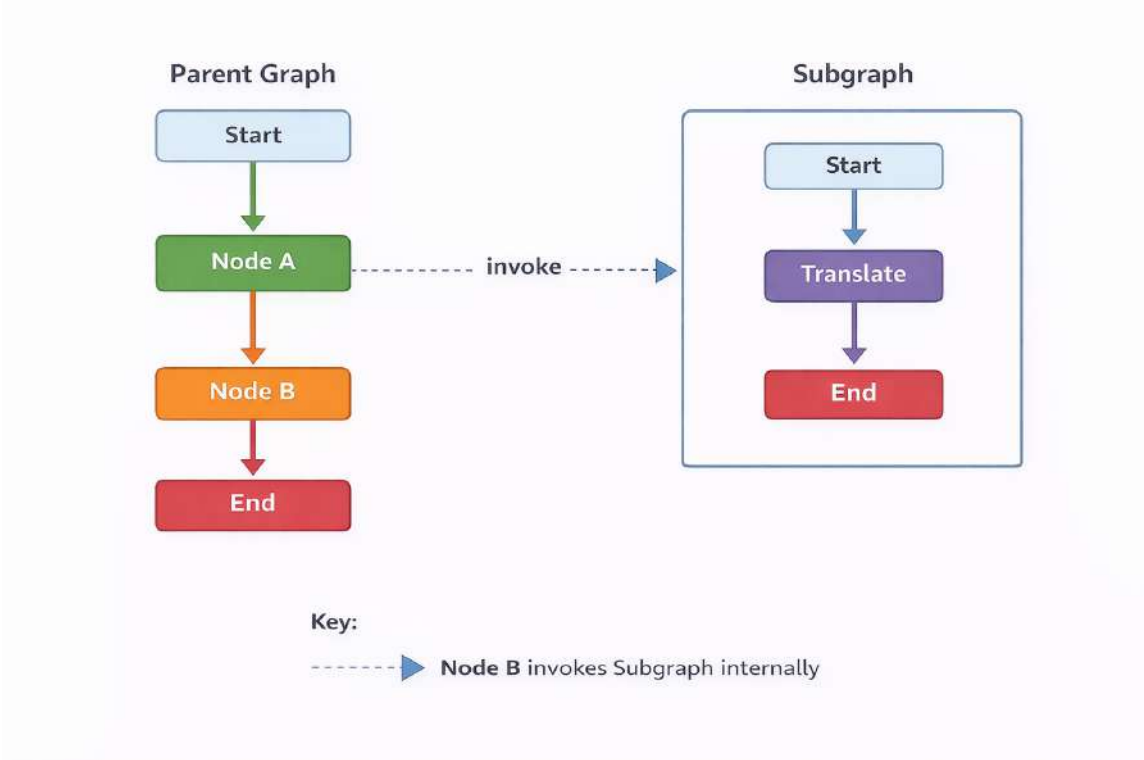


Figure 21.5. Architecture

Built separately
Each has own state
Explicit invocation in node code

21.8.3 Method 2: Add Graph as Node

Architecture:

Characteristics:

Subgraph added directly
Shares state keys with parent
Cleaner integration
More tightly coupled

21.9 Comparison: Two Methods

Table 21.1. Comparison: Two Methods

Feature	Method 1: Invoke from Node	Method 2: Add as Node
State	Separate States	Shared State Keys
Coupling	Loose	Tight
Independence	High	Medium
Complexity	More code	Less code
Use Case	Complex isolation needed	Simple shared workflows
Invocation	Manual in node code	Automatic

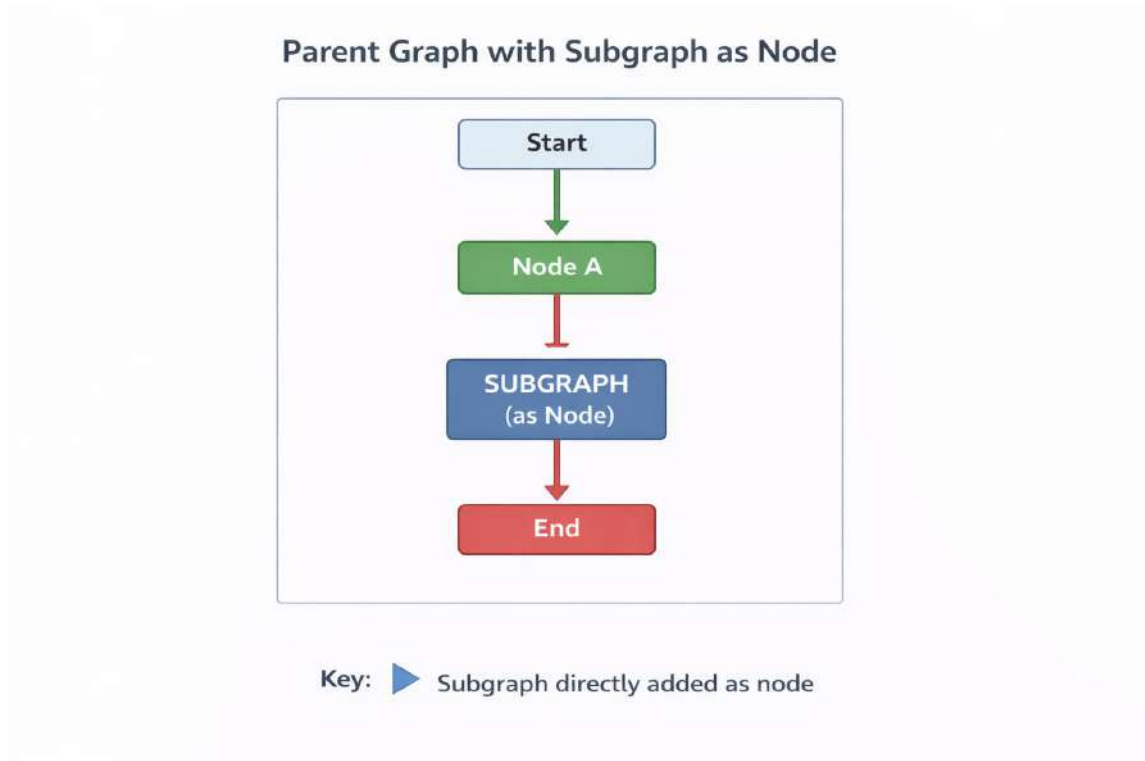


Figure 21.6. Architecture

21.10 Practical Example: Translation Workflow

21.10.1 Use Case:

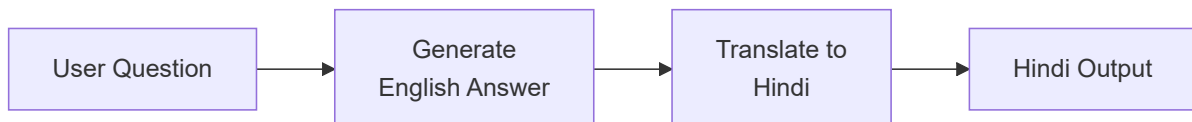


Figure 21.7. Use Case

Workflow:

1. User asks question
2. LLM generates English answer
3. Translate to Hindi
4. Return Hindi answer

21.11 Implementation: Method 1 (Separate States)

21.11.1 Architecture Design:

21.11.2 Code Implementation:

Step 1: Define Subgraph State

```

from langgraph.graph import StateGraph
from langchain_openai import ChatOpenAI
  
```

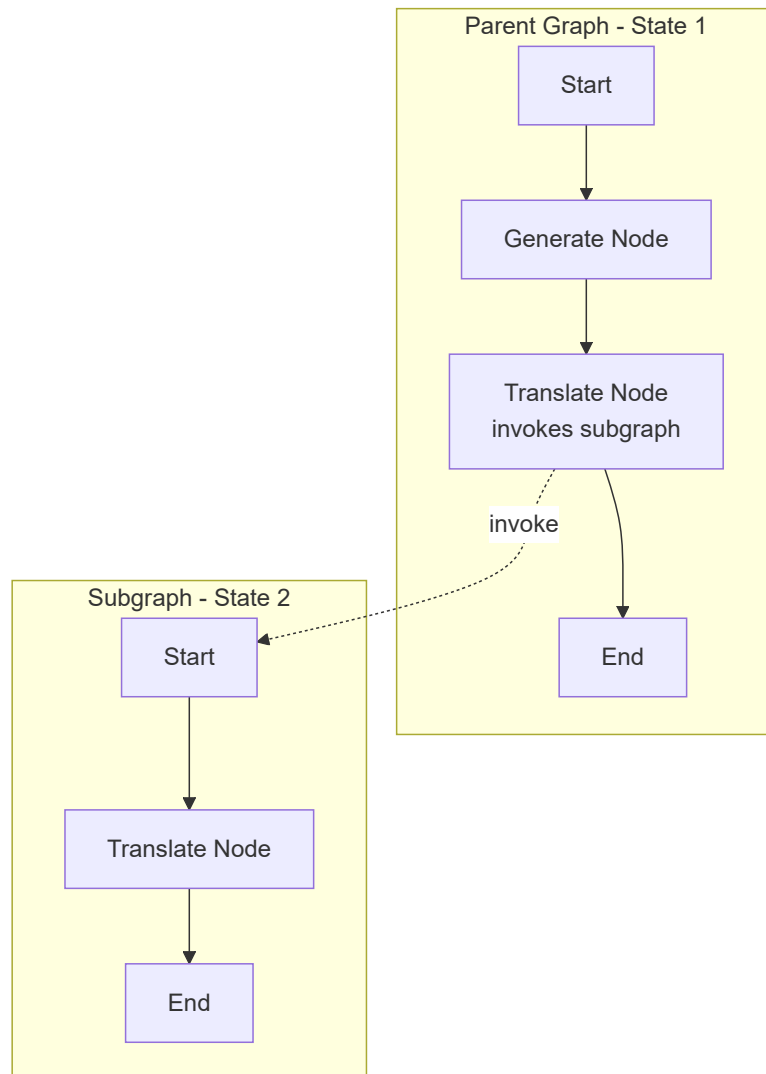


Figure 21.8. Architecture Design

```

from typing import TypedDict

# Subgraph State
class SubState(TypedDict):
    input_text: str # English answer input
    translated_text: str # Hindi translated output
  
```

State Purpose:

Input: English text to translate
 Output: Hindi translated text

Step 2: Create Subgraph LLM & Node

```

# Subgraph LLM
subgraph_llm = ChatOpenAI(model="gpt-4o-mini")

# Subgraph Node: Translate
def translate_text_node(state: SubState):
    """
    Translates English text to Hindi
  
```

```

"""
prompt = f"""
Translate the following text to Hindi.
Keep it natural and clear.
Do not add extra content.

Text: {state['input_text']}
"""

response = subgraph_llm.invoke(prompt)
hindi_text = response.content

return {"translated_text": hindi_text}

```

Node Function:

- Takes English text from state
- Creates translation prompt
- Invokes LLM
- Returns Hindi text

Step 3: Build & Compile Subgraph

```

# Build Subgraph
subgraph = StateGraph(SubState)

# Add node
subgraph.add_node("translate", translate_text_node)

# Add edges
subgraph.add_edge("__start__", "translate")
subgraph.add_edge("translate", "__end__")

# Compile
compiled_subgraph = subgraph.compile()

```

Subgraph Complete:

Start → Translate → End

Step 4: Define Parent Graph State

```

# Parent State
class ParentState(TypedDict):
    question: str # User's question
    english_answer: str # Generated English answer
    hindi_answer: str # Final Hindi answer

```

State Keys:

1. **question** - Original user query
2. **english_answer** - Generated by first node
3. **hindi_answer** - Translated by subgraph

Step 5: Create Parent Graph Nodes

```

# Parent LLM
parent_llm = ChatOpenAI(model="gpt-4o-mini")

# Node 1: Generate English Answer
def generate_answer_node(state: ParentState):

```

```

"""
Generates English answer to user's question
"""
prompt = f"""
You are a helpful assistant.
Answer this question: {state['question']}
"""

response = parent_llm.invoke(prompt)
english_answer = response.content

return {"english_answer": english_answer}

```

Step 6: Translate Node (Invokes Subgraph)

```

# Node 2: Translate (Invokes Subgraph)
def translate_node(state: ParentState):
    """
    Invokes subgraph to translate English answer to Hindi
    """
    # Invoke subgraph with English answer
    subgraph_result = compiled_subgraph.invoke({
        "input_text": state['english_answer']
    })

    # Extract Hindi translation from subgraph final state
    hindi_answer = subgraph_result['translated_text']

    return {"hindi_answer": hindi_answer}

```

Critical Logic:

```

# What happens:
1. Get english_answer from parent state
2. Pass to subgraph as input_text
3. Subgraph returns complete final state:
{
    "input_text": "English text...",
    "translated_text": "Hindi text..."
}
4. Extract only translated_text
5. Save to parent state as hindi_answer

```

Step 7: Build & Compile Parent Graph

```

# Build Parent Graph
parent_graph = StateGraph(ParentState)

# Add nodes
parent_graph.add_node("generate", generate_answer_node)
parent_graph.add_node("translate", translate_node)

# Add edges
parent_graph.add_edge("__start__", "generate")
parent_graph.add_edge("generate", "translate")
parent_graph.add_edge("translate", "__end__")

# Compile
compiled_parent = parent_graph.compile()

```

Parent Graph Flow:

```
Start → Generate → Translate → End
```

Step 8: Execute & Results

```
# Invoke parent graph
result = compiled_parent.invoke({
    "question": "What is artificial intelligence?"
})

# Access final state
print("Question:", result['question'])
print("English Answer:", result['english_answer'])
print("Hindi Answer:", result['hindi_answer'])
```

Example Output:

```
Question: "What is artificial intelligence?"

English Answer: "Artificial intelligence (AI) is the simulation
of human intelligence processes by machines, especially computer
systems..."

Hindi Answer: "आर्टिफिशियल इंटेलिजेंस (AI) मशीनों द्वारा मनुष्य की
इंटेलिजेंस प्रक्रियाओं का अनुकरण है, विशेष रूप से कंप्यूटर
सिस्टम..."
```

21.12 Implementation: Method 2 (Shared State)

21.12.1 Key Differences:

```
Method 1:
- Two separate states
- Manual invocation in node
- State mapping required

Method 2:
- Single shared state
- Subgraph added as node directly
- Automatic state sharing
```

21.12.2 Code Implementation:

Step 1: Define Single Shared State

```
# Shared State (used by both parent and subgraph)
class ParentState(TypedDict):
    question: str # User's question
    english_answer: str # English answer
    hindi_answer: str # Hindi answer
```

Important: Only ONE state definition needed!

Step 2: Define LLMs

```
# Parent LLM
parent_llm = ChatOpenAI(model="gpt-4o-mini")

# Subgraph LLM
subgraph_llm = ChatOpenAI(model="gpt-4o-mini")
```

Step 3: Build Subgraph First

```
# Subgraph Node
def translate_text_node(state: ParentState):
    """
    Translates english_answer to Hindi
    """
    prompt = f"""
    Translate the following text to Hindi.
    Keep it natural and clear.

    Text: {state['english_answer']}
    """

    response = subgraph_llm.invoke(prompt)
    hindi_text = response.content

    return {"hindi_answer": hindi_text}

# Build subgraph
subgraph = StateGraph(ParentState) # Uses same state!
subgraph.add_node("translate", translate_text_node)
subgraph.add_edge("__start__", "translate")
subgraph.add_edge("translate", "__end__")

# Compile
compiled_subgraph = subgraph.compile()
```

Notice: Subgraph uses ParentState directly!

Step 4: Build Parent Graph

```
# Parent Node: Generate Answer
def generate_answer_node(state: ParentState):
    """
    Generates English answer
    """
    prompt = f"""
    You are a helpful assistant.
    Answer: {state['question']}
    """

    response = parent_llm.invoke(prompt)
    return {"english_answer": response.content}

# Build parent graph
parent_graph = StateGraph(ParentState)

# Add generate node
parent_graph.add_node("generate", generate_answer_node)

# Add SUBGRAPH as NODE (not a function!)
parent_graph.add_node("translate", compiled_subgraph)

# Add edges
parent_graph.add_edge("__start__", "generate")
parent_graph.add_edge("generate", "translate")
parent_graph.add_edge("translate", "__end__")

# Compile
compiled_parent = parent_graph.compile()
```

Critical Difference:

```
# Method 1:
parent_graph.add_node("translate", translate_function)

# Method 2:
```



```
parent_graph.add_node("translate", compiled_subgraph)
                        ~~~~~
                        Graph object, not function!
```

Step 5: Execute

```
# Invoke
result = compiled_parent.invoke({
    "question": "What is machine learning?"
})

# Same output structure
print(result['question'])
print(result['english_answer'])
print(result['hindi_answer'])
```

21.13 Complete Comparison

21.13.1 Method 1 vs Method 2

```
# =====
# METHOD 1: INVOKE FROM NODE
# =====

# Subgraph State (Separate)
class SubState(TypedDict):
    input_text: str
    translated_text: str

# Parent State (Separate)
class ParentState(TypedDict):
    question: str
    english_answer: str
    hindi_answer: str

# Build both independently
subgraph = StateGraph(SubState)
parent = StateGraph(ParentState)

# Node manually invokes subgraph
def translate_node(state: ParentState):
    result = compiled_subgraph.invoke({
        "input_text": state['english_answer']
    })
    return {"hindi_answer": result['translated_text']}

parent.add_node("translate", translate_node)

# =====
# METHOD 2: ADD GRAPH AS NODE
# =====

# Single Shared State
class ParentState(TypedDict):
    question: str
    english_answer: str
    hindi_answer: str

# Both use same state
subgraph = StateGraph(ParentState)
parent = StateGraph(ParentState)

# Subgraph added directly as node
parent.add_node("translate", compiled_subgraph)
                        ~~~~~
```

Subgraph **object!**

21.14 Best Practices & Additional Features

21.14.1 1 Persistence in Subgraphs

```
from langgraph.checkpoint.memory import MemorySaver

# Add checkpointer to PARENT only
checkerpoint = MemorySaver()

compiled_parent = parent_graph.compile(
    checkerpoint=checkerpoint
)
```

How It Works:

```
Add checkerpoint to parent
↓
LangGraph automatically checkpoints subgraphs
↓
Both parent and child states persisted
```

Key Point: You only need to add checkerpoint to parent!

21.14.2 2 Streaming from Subgraphs

```
# Stream output from subgraph
for chunk in compiled_parent.stream({
    "question": "What is AI?"
}):
    print(chunk)

# Output includes both parent and subgraph updates
```

Stream Output:

```
{'generate': {'english_answer': 'AI is...'}}
{'translate': {'hindi_answer': 'AI ...'}}
```

21.14.3 3 Viewing Subgraph State

```
# Get state at any point
state = compiled_parent.get_state(config)

# State includes subgraph states
print(state['values']) # Complete state tree
```

21.15 Documentation Resources

21.15.1 Official LangGraph Docs:

```
Key Topics to Study:|—
  Subgraph basics (covered in video)|—
  Two implementation methods (covered)|—
```

Adding persistence to subgraphs|—
 Streaming subgraph outputs|—
 Viewing subgraph states

Study These:

- How to add persistence to subgraphs
- Streaming subgraph outputs
- Viewing subgraph state
- Advanced subgraph patterns

Link: [LangGraph Subgraphs Documentation]

21.16 When to Use Which Method?

21.16.1 Use Method 1 (Invoke from Node) When:

Need complete state isolation
 Subgraph has different data structure
 Want maximum independence
 Building loosely coupled systems
 Different teams working on different graphs

Example Use Cases:

- Complex multi-agent systems
- Microservice-like architecture
- Different state requirements
- Maximum modularity needed

21.16.2 Use Method 2 (Add as Node) When:

Need to share state data
 Simpler workflow
 Tightly integrated operations
 Same data structure throughout
 Sequential processing preferred

Example Use Cases:

- Linear workflows with shared data
- Simple multi-step processes
- When state sharing is beneficial
- Less complex architectures

21.17 Future Applications

21.17.1 Multi-Agent Systems:

When building multi-agent systems:|—
 Each agent = One subgraph|—
 Agents work independently|—
 Can have own tools, memory, guardrails|—
 Easy to debug individual agents|—
 Scalable architecture

Coming Soon in Series:

- Advanced multi-agent patterns
- Agent collaboration strategies
- Complex workflow orchestration
- Production multi-agent systems

21.18 Key Takeaways

21.18.1 Core Concepts:

Subgraph = Graph embedded as node in another graph
Enables hierarchical architecture
Critical for complex GenAI applications
Provides modularity, reusability, maintainability
Two methods: Invoke from node vs Add as node
Method 1: Separate states, loose coupling
Method 2: Shared state, tight integration
Essential for multi-agent systems

21.18.2 Best Practices:

1. **Start Simple** - Use Method 2 for simple workflows
2. **Scale with Method 1** - Use when complexity increases
3. **Document State Flow** - Clear understanding of data movement
4. **Test Independently** - Test subgraphs in isolation first
5. **Monitor Granularly** - Use LangSmith for observability
6. **Plan State Structure** - Design states before coding

21.19 Architecture Patterns Summary

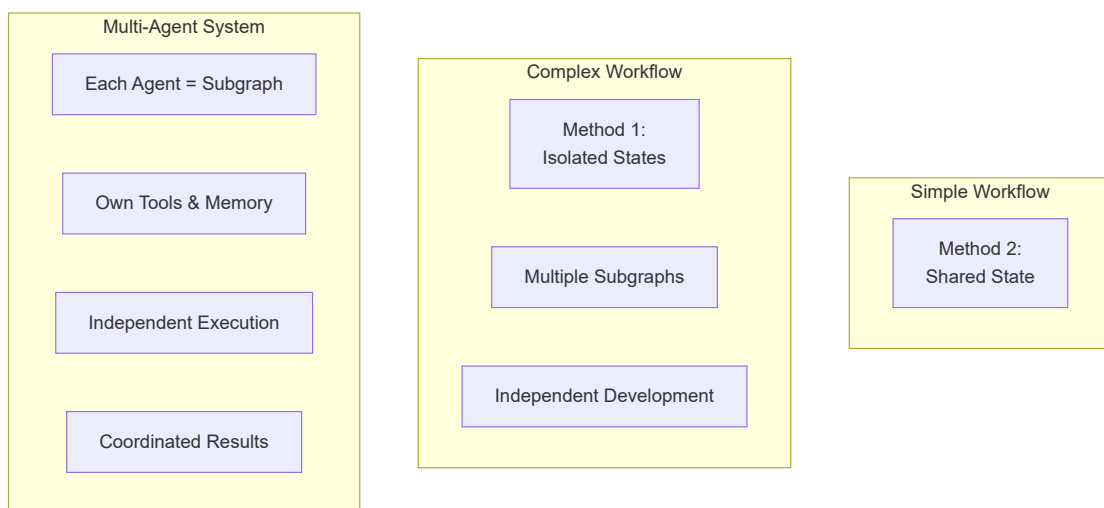


Figure 21.9. *Architecture Patterns Summary*

21.20 Example Architectures

21.20.1 1. Customer Service System:

```

Main Graph|—
  Intent Classification (Subgraph)|
    |— Text preprocessing|
    |— Classification|
    |— Confidence scoring|—
  Response Generation (Subgraph)|
    |— Context retrieval|
    |— Answer generation|
    |— Quality check|—
  Escalation Handler (Subgraph)
    |— Priority assessment
    |— Agent assignment
    |— Ticket creation

```

21.20.2 2. Content Creation Pipeline:

```

Main Graph|—
  Research Agent (Subgraph)|
    |— Web search|
    |— Source validation|
    |— Information extraction|—
  Writing Agent (Subgraph)|
    |— Outline creation|
    |— Content generation|
    |— Style application|—
  Review Agent (Subgraph)
    |— Grammar check
    |— Fact verification
    |— Final approval

```

21.21 Debugging Tips

21.21.1 Subgraph-Specific Debugging:

```

# 1. Test subgraph independently
subgraph_result = compiled_subgraph.invoke(test_input)
print(subgraph_result)

# 2. Use LangSmith for tracing
# Each subgraph appears as separate trace

# 3. Check state at each step
state_snapshot = graph.get_state(config)

# 4. Add logging in nodes
def my_node(state):
    print(f"Current state: {state}")
    # ... node logic

```

21.22 Performance Considerations

21.22.1 State Size:

Keep states minimal
Only necessary data
Avoid duplicating data across states
Use references where possible

21.22.2 Subgraph Depth:

Avoid deep nesting (subgraphs within subgraphs within subgraphs)
Typically 2-3 levels maximum
Flatten if getting too complex

21.22.3 Execution Time:

Each subgraph adds overhead
Monitor with LangSmith
Optimize critical paths
Consider parallel execution where possible

21.23 Quick Reference

21.23.1 Method 1 Template:

```
# Separate states
class SubState(TypedDict): ...
class ParentState(TypedDict): ...

# Build subgraph
subgraph = StateGraph(SubState)
# ... add nodes
compiled_sub = subgraph.compile()

# Parent node invokes subgraph
def parent_node(state: ParentState):
    result = compiled_sub.invoke(data)
    return mapped_result

# Build parent
parent = StateGraph(ParentState)
parent.add_node("node", parent_node)
```

21.23.2 Method 2 Template:

```
# Shared state
class ParentState(TypedDict): ...

# Build subgraph with same state
subgraph = StateGraph(ParentState)
# ... add nodes
compiled_sub = subgraph.compile()

# Build parent - add subgraph as node
parent = StateGraph(ParentState)
parent.add_node("subgraph_node", compiled_sub)
```

You're now ready to build complex multi-agent systems with LangGraph Subgraphs!
Next in series: Advanced multi-agent architectures and agent collaboration patterns

21.24 Additional Study Resources

Must Read in LangGraph Docs:

1. Subgraph persistence mechanisms
2. Streaming from nested subgraphs
3. State inspection and debugging
4. Advanced subgraph patterns
5. Multi-agent coordination strategies

Practice Exercises:

1. Build a 3-subgraph workflow
2. Implement both methods for same use case
3. Add persistence and streaming
4. Create a mini multi-agent system
5. Implement error handling across subgraphs

LangGraph Subgraphs: Building Multi-Agent Systems

Modularity • Reusability • State Isolation • Hierarchical Architecture

What Are Subgraphs?

Definition:
A Subgraph is a complete graph that is embedded and executed as a node inside another graph.

Traditional Graph

```

graph LR
    Node1 --> Node2 --> Node3
            
```

All nodes are simple functions

Subgraph Architecture

```

graph LR
    Node1 --> SubGraph[SubGraph] --> Node3
            
```

Node 2 is a complete graph inside!

Why Subgraphs? The Evolution of GenAI Apps

Without Subgraphs (Monolithic)

- One massive, complex graph
- Hard to maintain, debug, scale
- Single point of failure

With Subgraphs (Multi-Agent)

- Each agent = specialized subgraph
- Open tools, memory, guardrails
- Independent development and testing

Example: Software Development Agent

```

graph LR
    Planning[Planning Agent] --> Backend[Backend Agent]
    Backend --> Frontend[Frontend Agent]
    Frontend --> Testing[Testing Agent]
    Testing --> Review[Review Agent]
    Review --> Deploy[Deploy Agent]
            
```

Core Benefits of Subgraphs

Modularity

- Break system into functions
- Clear separation of concerns
- Independent development

Reusability

- Write once, use multiple times
- Same logic, different contexts
- Reduced code duplication

Maintainability

- Easier debugging
- Isolated testing
- Precise error location

Example: Coding Agent subgraph used for BOTH Backend Team AND Frontend Team - same logic, different file sets!

Debugging: "Testing Agent failed!" → Debug only Testing subgraph, not entire system

LangGraph-Specific Benefits

Failure Isolation

- Subgraph failure contained
- Parent graph continues
- Graceful degradation

State Separation

- Each subgraph owns state
- No state collision
- Clear data boundaries

Observability

- Monitor each agent
- Consider LangSmith tracing
- Identify bottlenecks

State Isolation Example:

```

Coding: {code_files: []} | Testing: {test_results: []} | Review: {comments: []}
            
```

Two Ways to Implement Subgraphs

Method 1: Invoke Graph from Node (Separate States)

Architecture: Two Independent Graphs

```

graph LR
    Parent[Parent Graph / ParentState] -- invoke --> Sub[Subgraph / SubState - Different]
            
```

✓ Characteristics:

- Two separate state definitions
- Manual invocation in node code
- Loose coupling, high independence

Use When:

- Need complete state isolation
- Different data structures needed
- Maximum modularity required

```

# Node manually invokes subgraph
def translate_node(state: ParentState):
    result = compiled_subgraph.invoke({
        "input_text": state["input_text"],
        "state": {"language": "english"}
    })
    return { "hindi": result["translated_text"] }
            
```

Method 2: Add Graph as Node (Shared State)

Architecture: Single Shared State

```

graph LR
    Parent[Parent Graph / ParentState] -- add_node --> Sub[Subgraph / ParentState - Same!]
            
```

✓ Characteristics:

- Single shared state definition
- Subgraph added directly as node
- Tight coupling, cleaner integration

Use When:

- Need to share state data
- Simpler, linear workflows
- Less complex architectures

```

# Subgraph added directly as node (not a function!)
parent_graph.add_node("translate", compiled_subgraph)

# Graph object, not function!
            
```

Practical Example: Translation Workflow (Question → English → Hindi)

Method 1: Separate States Implementation

```

# Subgraph State (Different)
class SubState(TypedDict):
    input_text: str
    translated_text: str
            
```

```

# Build Subgraph
def translate_text_node(state: SubState):
    response = llm.invoke(f"Translate to Hindi: {state['input_text']}")
    return { "translated_text": response.content }
            
```

```

# Parent Node - Invokes Subgraph
def translate_node(state: ParentState):
    # Invoke subgraph with mapping
    result = compiled_subgraph.invoke({
        "input_text": state["english_answer"],
        "state": {"language": "english"}
    })
    return { "hindi_answer": result["translated_text"] }
            
```

```

# Parent State (Different)
class ParentState(TypedDict):
    question: str
    english_answer: str
    hindi_answer: str
            
```

Method 2: Shared State Implementation

```

# Single Shared State (used by BOTH parent and subgraph)
class ParentState(TypedDict):
    question: str
    english_answer: str
    hindi_answer: str
            
```

```

# Subgraph uses ParentState directly!
def translate_text_node(state: ParentState):
    response = llm.invoke(f"Translate: {state['english_answer']}")
    return { "hindi_answer": response.content }
            
```

```

# Build subgraph with same state
subgraph = StateGraph(ParentState) # Same state!
compiled_subgraph = subgraph.compile()

# Add subgraph directly as node
parent_graph.add_node("translate", compiled_subgraph)
            
```

Method Comparison Table

Feature	Method 1: Invoke from Node	Method 2: Add as Node
State	Separate States	Shared State Keys
Coupling	Loose	Tight
Independence	High	Medium
Code Complexity	More code (mapping)	Less code (automatic)
Best For	Complex multi-agent systems	Simple shared workflows

Best Practices and Additional Features

Persistence in Subgraphs

Add checkpoint to PARENT only!

LangGraph auto-checkpoints subgraphs

```

parent.config.checkpoint_config.checkpoint_id
            
```

Streaming from Subgraphs

Stream includes both parent and subgraph

```

for chunk in parent.stream(input):
    # Both parent & subgraph content
            
```

Performance Considerations

- Keep states minimal - only necessary data • Avoid deep nesting (2-3 levels max)
- Monitor with LangSmith • Test subgraphs independently first • Consider parallel execution

Quick Reference: Subgraph Templates

```

# Method 1 Template (Separate States)
class SubState(TypedDict):
    input_text: str
    output: str
class ParentState(TypedDict):
    question: str
    answer: str

subgraph = StateGraph(SubState); compiled_sub = subgraph.compile()

def parent_node(state: ParentState):
    result = compiled_sub.invoke({
        "input_text": state["question"]
    })
    return { "answer": result["output"] }

parent = StateGraph(ParentState)
parent.add_node("node", parent_node) # Function that invokes subgraph
            
```

```

# Method 2 Template (Shared State)
class ParentState(TypedDict):
    question: str
    answer: str

# Both use same state!
subgraph = StateGraph(ParentState); compiled_sub = subgraph.compile()
parent = StateGraph(ParentState)
parent.add_node("subgraph_node", compiled_sub)

# Add subgraph as node
            
```

Key Takeaways

- Subgraph = Graph embedded as node in another graph
- Method 2: Shared state, tight integration
- Enables hierarchical multi-agent architecture
- Benefits: Modularity, Reusability, Maintainability
- Method 1: Separate states, loose coupling
- LangGraph: Failure isolation, State separation, Observability

LangGraph Subgraphs Guide • github.com/campusx-official/langgraph-tutorials • CampusX Tutorial

Figure 21.10. Additional Study Resources

428

Memory in Generative AI — Complete Notes

22.1 1. Introduction: Why Memory Matters in GenAI

Memory is one of the most fundamental and critical concepts in Generative AI. Without memory, no GenAI application — whether it is a chatbot, an AI assistant, or an autonomous agent — can function properly. Imagine a chatbot that forgets everything you just said in the previous message. Every interaction would feel like talking to a stranger. That is exactly what happens when there is no memory system in place.

This set of notes covers memory as a **framework-agnostic concept**. It is not tied to any specific library like LangChain or LangGraph. Instead, it builds understanding from first principles — as if we are inventing the concept of memory ourselves. By the end, you will have a deep foundational understanding of how memory works in LLM-based systems and how to implement it in your own applications.

22.2 2. How LLMs Work at Inference — The Mathematical View

22.2.1 2.1 LLM as a Parameterized Math Function

The starting point of our discussion is a simple but powerful statement:

An LLM at inference is just a parameterized math function.

This is represented by the equation:

$$y = f_{\theta}(x)$$

Let us break this down piece by piece.

22.2.2 2.2 What is a Parameterized Math Function?

In mathematics, a **parameterized function** is a function whose output depends not only on its input but also on some additional **parameters**.

Simple Example:

$$y = a \cdot x^2$$

Here, to calculate y , you need both the input x **and** the parameter a . The user provides x , but a comes from somewhere else (in machine learning, it comes from training on data).

22.2.3 2.3 Concrete Example — Linear Regression

In linear regression, you have scattered data points and you want to find the best-fit line through them. The equation of that line is:

$$y = mx + b$$

Here:

- m (slope) and b (intercept) are the **parameters**.
- Their values are learned from the **training data**.
- The user provides x (the input), and the model computes y (the output).

We can rewrite this in our parameterized form:

$$y = f_{m,b}(x)$$

This is exactly the same representation as $y = f_{\theta}(x)$.

22.2.4 2.4 Applying This to LLMs

Since an LLM is also a machine learning model (specifically a deep learning model), the same setup applies:

$$y = f_{\theta}(x)$$

Table 22.1. 4 Applying This to LLMs

Component	What It Represents	Details
f	The mathematical function	A very complex function hidden inside the neural network (we don't need to know the details)
θ (theta)	The parameters	Represents billions of parameter values (e.g., a 70B parameter model, a 100B parameter model). These values are learned during training.
x	Input tokens	The tokens you send to the LLM during inference. Simply put, this is your prompt .
y	Output tokens	The response generated by the LLM, based on both θ and x .

22.2.5 2.5 Key Properties of θ and x

y depends on both θ and x

However:

- θ is **FIXED** — Its values were decided during training. The user has **no control** over it at inference time.
- x is **NOT FIXED** — The user can change it freely. This is exactly why changing your prompt gives you a different output from the LLM.

22.3 3. LLMs Are Stateless by Design

22.3.1 3.1 What Does “Stateless” Mean?

A system is stateless if its output depends only on the current input, and not on anything that happened before.

This is a critical property of the LLM math function. Let us see what this means practically.

22.3.2 3.2 Stateless Behavior — Step by Step

Suppose you call the LLM twice:

First call:

$$y_1 = f_{\theta}(x_1)$$

You send prompt x_1 , the LLM uses its parameters θ , and produces output y_1 .

Second call:

$$y_2 = f_{\theta}(x_2)$$

You send a new prompt x_2 , and you get output y_2 .

The critical point: When computing y_2 , the LLM uses **only** x_2 and θ . It has **no knowledge** of x_1 or y_1 . The previous interaction is completely invisible to it. Each function call is **independent** and **unrelated** to the previous one.

22.3.3 3.3 Practical Demonstration

```
# First call
llm.invoke('My name is Nitish').content
# Output: 'Nice to meet you, Nitish. How can I assist you today?'

# Second call (independent – no memory of the first call)
llm.invoke('What is my name?').content
# Output: "I'm sorry, I do not know your name."
```

The LLM could not remember the name because it is **stateless**. It treated the second call as a completely fresh, independent request.

22.4 4. The Core Problem: LLMs Have No Intrinsic Memory

From the stateless nature of LLMs, we can establish two important facts:

Fact 1: LLMs do not have any intrinsic memory. They cannot remember past conversations on their own.

Fact 2: Almost every GenAI application in the world (chatbots, agents, assistants) **requires** memory to function properly. Without memory, the user experience is highly frustrating — you would have to re-explain everything from scratch every single time.

This creates a **deadlock**: LLMs need memory to be useful, but they don’t have any built-in memory.

Fact 3 (The Solution Direction): Since LLMs don’t have intrinsic memory, we must **develop memory externally**. We need to build a system **around** the LLM that acts like memory.

22.5 5. Two Prerequisite Concepts for Building Memory

Before we can build a memory system, we need to understand two foundational concepts.

22.5.1 5.1 Context Window

The context window is the amount of text an LLM can read and remember at one time before answering.

Camera Analogy: Think of the LLM as a camera and the context window as its **lens**. A smaller lens captures only a small part of the scene. A bigger lens captures a much larger portion. Similarly, a bigger context window means the LLM can process more text before generating an answer.

Modern LLM context windows:

- Most modern LLMs have a context window of **128K tokens** or more.
- Some models (like Google’s Gemini) have a context window of **1 million tokens**.
- A 128K token context window can roughly hold a **200-page PDF**.

Why this matters for memory: The fact that we can send a **lot of tokens** in the input x is a very important power that will help us build memory systems.

22.5.2 5.2 In-Context Learning (ICL)

When an LLM is trained, it absorbs knowledge from massive amounts of data. This knowledge gets stored in the LLM’s parameters and is called **parametric knowledge**. At inference time, when you ask a question, the LLM searches its parametric knowledge for the answer.

However, as LLMs grew larger, researchers observed an **emergent phenomenon**:

In-context learning is an emergent ability that allows an LLM to use information and patterns present in the prompt itself, in addition to its trained parametric knowledge, to generate an answer.

Example: You upload a 100-page private company PDF into the prompt and ask a question about it. The LLM has never seen this data during training (it’s not in its parametric knowledge). But it can **read the PDF content from the prompt** and answer the question. This is in-context learning.

Table 22.2. 2 In-Context Learning (ICL)

Knowledge Source	Description	Example
Parametric Knowledge	Learned during training, stored in model parameters	General facts like “Paris is the capital of France”
In-Context Knowledge	Provided in the prompt at inference time	A private company report uploaded in the prompt

22.6 6. Short-Term Memory — The First Solution

22.6.1 6.1 The Simple Elegant Solution

By combining the two concepts above (large context window + in-context learning), we can build a memory system. The plan is:

*Every time we invoke the LLM, we **concatenate the entire conversation history** into the input x .*

Step-by-step:**First message:**

$$y_1 = f_{\theta}(x_1)$$

No history exists yet, so we just send the first user message.

Second message (with memory):

Instead of:

$$y_2 = f_{\theta}(x_2) \quad (\text{stateless — no memory})$$

We do:

$$y_2 = f_{\theta}(\text{concat}(x_1, y_1, x_2))$$

We concatenate the entire past conversation (x_1, y_1) along with the new message (x_2) and send it all as input. This way:

- The **large context window** ensures we can fit the conversation history.
- **In-context learning** allows the LLM to read the past conversation and use it to answer the current question.

22.6.2 6.2 Code Implementation

```
# Step 1: Initialize the conversation buffer
messages = ['My name is Nitish']

# Step 2: First LLM call
output = llm.invoke(messages).content
# Output: "Hello Nitish! How can I assist you today?"

# Step 3: Append the LLM's response to messages
messages.append(output)
# messages now has: [x1, y1]

# Step 4: User asks a new question
messages.append('What is my name?')
# messages now has: [x1, y1, x2]

# Step 5: Second LLM call with full history
output = llm.invoke(messages).content
# Output: "Your name is Nitish"
```

22.6.3 6.3 Understanding the Mechanism

The messages variable acts as the **state** of our system. Before this variable, the system was stateless. Now, with this conversation buffer, the system has become **stateful**.

This messages variable is also called a **conversation buffer** because it dynamically loads the entire conversation history.

Important: If you restart the code/file, the messages variable is wiped clean, and the LLM will no longer recognize you. This is why this memory is **temporary** — hence the name **Short-Term Memory (STM)**.

22.6.4 6.4 How Chatbots Implement Short-Term Memory

In any chatbot (ChatGPT, Gemini, etc.), conversations are organized into **separate sessions** called **conversations** (also called **threads**).

Key rule: Short-term memory is **conversation-scoped** (or **thread-scoped**). Each conversation gets its own independent short-term memory.

- When you open “Conversation A,” the messages buffer contains only the messages from Conversation A.
- When you switch to “Conversation B,” the buffer is emptied and reloaded with messages from Conversation B.
- The two conversations do not share memory.

Why not use a single buffer for all conversations? If you have 1000 conversations and dump all their messages into one buffer, it becomes far too long and incoherent. The LLM would not be able to process it properly. That is why a **logical boundary** (one thread = one memory scope) is created.

Terminology: “Thread” is often used interchangeably with “Conversation.” So when you read “STM is thread-scoped,” it means the same as “STM is conversation-scoped.”

22.7 7. Problems with Short-Term Memory

22.7.1 7.1 Problem 1: STM is Fragile

The messages variable lives in the application’s runtime memory (RAM). If the code resets, the server crashes, or you switch to a new conversation, all stored messages are **lost**.

Scenario: You are chatting in Conversation A, accumulating messages. You click “New Chat” to start Conversation B. The messages buffer is cleared for the new chat. Later, you go back to Conversation A — but there is **nothing there** because the old messages were never saved persistently.

Solution — Persistence:

Connect the short-term memory to a **database**. The workflow becomes:

1. While chatting in a conversation, messages accumulate in the messages variable.
2. Before switching away, **save** all messages to a database under a unique **Thread ID** (e.g., Thread ID = 1).
3. When starting a new conversation, create a new Thread ID (e.g., Thread ID = 2) and save those messages separately.
4. When returning to an old conversation, **load** the messages from the database using the Thread ID back into the messages variable.

This way, conversation context is never lost, even across restarts or switches.

22.7.2 7.2 Problem 2: Context Window Overflow

As a conversation grows very long (hundreds of messages), the total token count of the messages buffer can **exceed** the LLM’s context window limit. When this happens, the LLM may produce incoherent replies or start hallucinating.

Solution — Trimming + Summarization (Hybrid Approach):

This is the most widely used solution and it combines two techniques:

Technique 1: Trimming — Keep only the most recent n messages. For example, instead of sending all 500 messages, send only the last 50. The rationale is that the most recent context is usually the most relevant.

Technique 2: Summarization — Take the older messages that were trimmed away and generate a **summary** of them using another LLM call. For example, summarize the first 450 messages into a short paragraph.

Hybrid approach: Send the summary of older messages **plus** the most recent 50 messages to the LLM. The total token count stays within the context window, and the LLM can still understand the broader context through the summary.

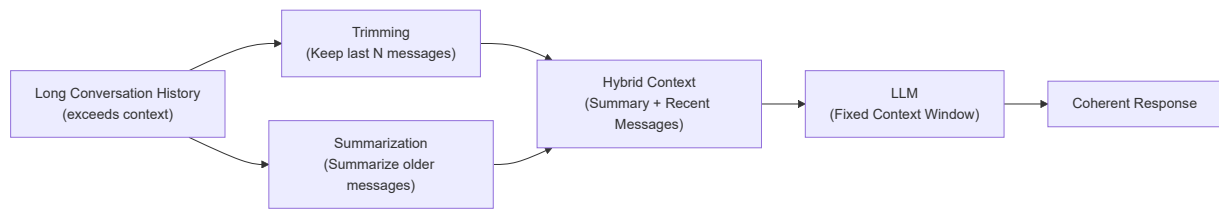


Figure 22.1. 2 Problem 2: Context Window Overflow

22.7.3 7.3 Problem 3: STM is Thread-Scoped (The Biggest Problem)

Since STM only exists within a single conversation, it creates three major sub-problems:

A. Loss of User Continuity Across Conversations

In one conversation, you tell the LLM you prefer Python. It remembers — but only for that session. In the next conversation, it forgets and might give you Java code again. STM cannot retain **user preferences** across conversations.

B. Learning Never Compounds Over Time

Suppose in one conversation, you teach the LLM to use window functions instead of subqueries for SQL. Next conversation, it goes back to writing subqueries. All the effort you invested is lost. The LLM never **adapts** or **evolves** with you.

C. Cross-Thread Reasoning is Impossible

You can never ask: “What did we discuss yesterday?” or “Last time we faced this problem, what solution did we use?” The LLM has no access to previous conversations, so it simply cannot answer such questions.

The fundamental issue: Every time you start a new conversation, you become a **stranger** to the LLM all over again. Personalization is impossible with STM alone.

22.8 8. The Need for Long-Term Memory (LTM)

Since STM cannot solve the cross-conversation problems, we need a completely **new type of memory** with two essential properties:

Property 1 — Store Special Information for a Long Time

This memory should store information whose relevance extends **beyond a single conversation** — sometimes for days, weeks, or even months. Examples include who the user is, their preferences, ongoing projects, and past decisions.

The kinds of information that define continuity across sessions include:

- Who the user is (e.g., male, Indian, teacher, YouTuber)
- How the system should behave for them
- What tends to work and what usually fails
- What decisions were already made in the past

Property 2 — Be Highly Selective

We do NOT blindly store the entire conversation. We extract only the most **important, stable, useful, and reusable** pieces of information. Everything else is noise and should be ignored.

Only information that proves to be stable, useful, and reusable should survive beyond a single thread. Everything else should naturally fade away.

This new type of memory is called **Long-Term Memory (LTM)** because it exists **outside** and **beyond** individual conversations, for a long period of time.

22.9 9. Types of Long-Term Memory

There are **three distinct types** of information stored in Long-Term Memory:

22.9.1 9.1 Episodic Memory — “What Happened”

Episodic memory stores **past events and experiences**, usually tied to time or context. It tells the system what occurred in previous sessions.

Examples:

- “Yesterday, the user rejected solution A.”
- “The last deployment failed due to missing credentials.”
- “In the previous session, this approach worked.”
- “User approved plan X in the last conversation.”

Why it exists: It answers questions like “What did we do last time?”, “Has this happened before?”, “Have we tried this already?” — questions that STM simply cannot answer because they span multiple threads.

22.9.2 9.2 Semantic Memory — “What is True”

Semantic memory stores **facts, knowledge, and stable information** that are not tied to a single event. This is the most common and most important type of LTM.

Examples:

- “User prefers Python.”
- “This user is a beginner.”
- “The system uses PostgreSQL.”
- “Budget constraint is ₹10,000.”

Why it exists: It answers questions like “What do we know about this user?”, “What constraints always apply?”, “What facts should persist?”

22.9.3 9.3 Procedural Memory — “How to Do Things”

Procedural memory stores **strategies, rules, and learned behaviors**. It tells the system the preferred way to accomplish tasks for a particular user or context.

Examples:

- “When solving SQL problems for this user, avoid subqueries — use window functions instead.”
- “If tool X fails, always try tool Y.”
- “For this user, explain step by step.”
- “Preferred workflow for task Z is [specific workflow].”

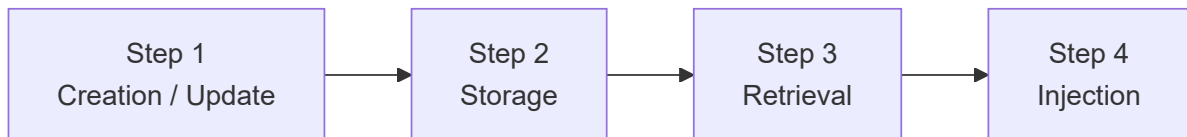
Why it exists: It answers questions like “What approach usually works?”, “How should I behave in this situation?”, “What strategy should I follow?” This is the memory that enables the agent to **compound its learning** over time and feel progressively better to the user.

Table 22.3. 3 *Procedural Memory* — “How to Do Things”

Type	Core Question	What It Stores	Example
Episodic	What happened?	Past events, experiences, context tied to time	“Last session, the user rejected approach A”
Semantic	What is true?	Facts about the user, system, and task	“User prefers Python”, “Budget is ₹10K”
Procedural	How to do things?	Strategies, rules, learned behaviors	“Avoid subqueries; use window functions”

22.10 10. How Long-Term Memory Works — High-Level Architecture

The entire LTM workflow operates in **four steps**:

**Figure 22.2.** *How Long-Term Memory Works — High-Level Architecture*

22.10.1 Step 1: Creation / Update

During a conversation between the user and the AI, the system must identify whether anything being discussed is **worth remembering beyond this conversation**.

What the system looks at:

- User messages
- Model responses
- Tool outcomes (if the system uses tools/agents)

What happens in this step:

1. **Extract memory candidates** — e.g., the user says “I prefer Python over Java.” This is a memory candidate.
2. **Filter out noise** — Remove irrelevant chatter and extract the core fact or information.
3. **Decide scope** — Is this memory at the user level, agent level, or app level? Tag it accordingly.
4. **Decide action** — Should the system: create a **new** memory, **update** an existing memory, or **ignore** it entirely?

22.10.2 Step 2: Storage

Once a piece of information has been identified and processed as a memory, it must be **stored durably** so it survives restarts and crashes.

What happens in this step:

1. Save the memory to a **durable store** (a database).
2. Attach **identifiers and metadata** tags so that future retrieval is efficient.

Types of storage options:

Table 22.4. Step 2: Storage

Storage Type	Best For
Relational Database (SQL)	Structured facts (Semantic Memory)
Key-Value Store	Simple lookups, preferences
Log / Text File	Chronological event records (Episodic Memory)
Vector Database	Semantic search over memories (when you need to find memories by meaning rather than exact match)

The choice of storage depends on the type of memory being stored (episodic, semantic, or procedural).

22.10.3 Step 3: Retrieval

When the user starts a **new conversation**, the system needs to determine whether any stored long-term memories are relevant to the current discussion.

What happens in this step:

1. Examine the current user input.
2. Decide: does this query require any long-term memory?
3. If yes, **search** the memory store.
4. Extract a **small, relevant subset** of memories — not everything.

Critical principle: Retrieval is **selective, not exhaustive**. You do not dump all memories into the context. You perform a targeted search and bring back only what is relevant to the current situation.

22.10.4 Step 4: Injection

This is where long-term memory actually reaches the LLM. However, it does **not** interact with the LLM directly.

The flow is:



The retrieved memory is **inserted into the short-term memory** (the conversation buffer / prompt). The model sees it as just **more input tokens**. It has no idea these tokens came from a long-term memory store — it simply processes them as part of the prompt.

Key takeaway: LTM never directly touches the LLM. LTM → STM → Context Window → LLM. The LLM always works the same way: $y = f_{\theta}(x)$. We just make x smarter by injecting relevant memories into it.

22.11 11. Complete Memory Architecture — End-to-End Flow

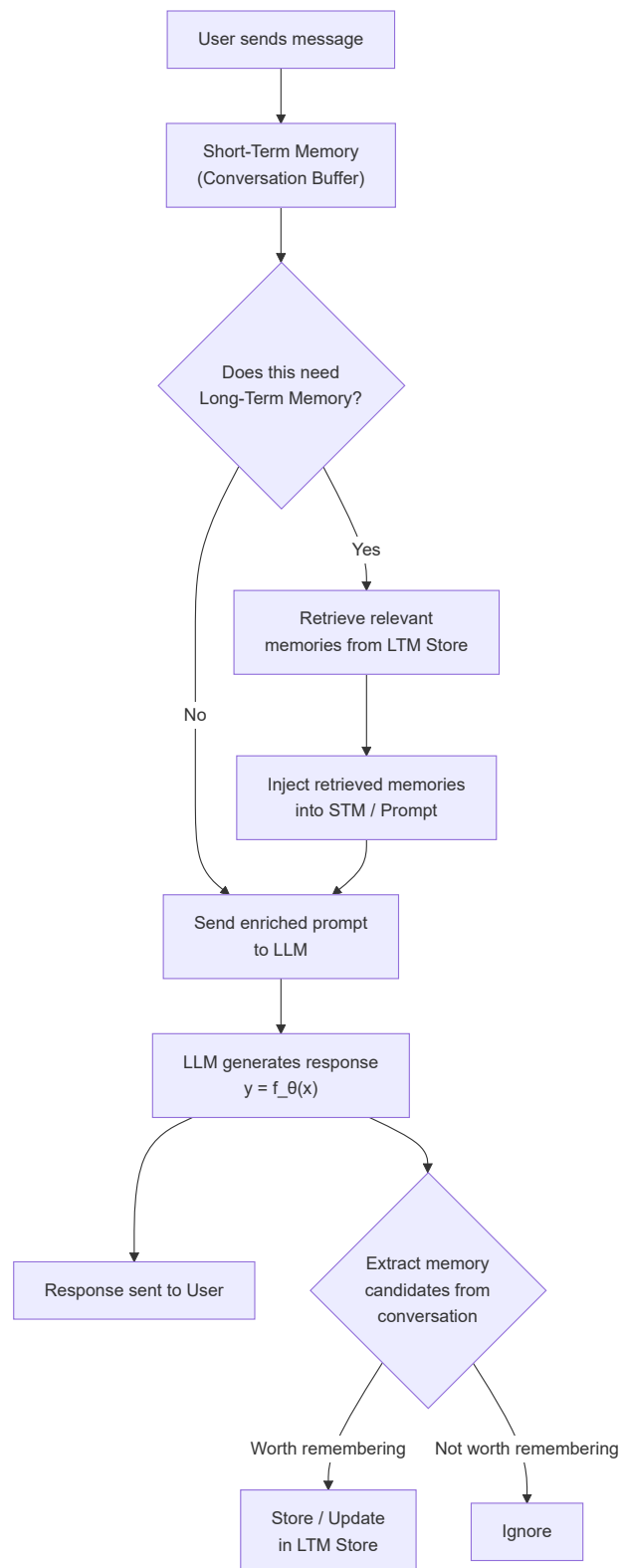


Figure 22.3. Complete Memory Architecture — End-to-End Flow

22.12 12. Challenges in Building Memory Systems

Building a robust LTM system is non-trivial. Here are the three key challenges:

Challenge 1: Deciding What is Worth Remembering

During a normal conversation, there is a lot of noise. Determining whether a particular statement should become a long-term memory — and whether it will remain relevant in the future — is extremely difficult. This requires sophisticated judgment.

Challenge 2: Retrieving the Right Memory at the Right Time

Even if you have correctly stored memories, figuring out **which** memories are relevant to the **current** conversation in real-time is a hard problem. Fetching irrelevant memories adds noise; missing relevant ones defeats the purpose.

Challenge 3: Orchestrating the Entire System

You are already building a complex AI system (an agent, a chatbot). On top of that, integrating the entire memory pipeline — creation, storage, retrieval, injection — adds many moving parts. Connecting memory stores, managing thread IDs, handling concurrent users, and ensuring everything works smoothly is an engineering challenge.

22.13 13. Tools and Libraries for Memory Management

To address these challenges, several managed solutions and libraries have emerged:

Table 22.5. *Tools and Libraries for Memory Management*

Tool	Description
LangMem	Part of the LangChain family. Easily integrates with LangGraph for building AI agents with memory.
Memo (MemZero)	A popular platform that acts as a memory layer for your GenAI applications. Handles creation, storage, and retrieval of long-term memories.
SuperMemory	A platform for managing long-term memory for GenAI apps. Built by a 15-year-old Indian developer and gaining traction with strong funding.

These tools essentially say: “You focus on building your application. We will handle the entire long-term memory system — creation, storage, retrieval, and injection — for you. Just call our functions.”

22.14 14. The Future: Intrinsic Memory in LLMs

All the problems discussed so far arise because LLMs **do not have built-in memory**. Researchers are now working on building LLM architectures that have **intrinsic memory** built into the model itself, so that external memory systems become less necessary.

A notable example is Google Research’s **Titans + Mirage** paper, which explores a different kind of Transformer architecture with built-in intrinsic memory. This is an active and rapidly growing area of research.

22.15 15. Summary — Complete Memory Landscape

Table 22.6. *Summary — Complete Memory Landscape*

Aspect	Short-Term Memory (STM)	Long-Term Memory (LTM)
What it is	Conversation buffer holding chat history	External store holding extracted facts, events, and strategies
Scope	Thread-scoped (one conversation)	Cross-thread (spans multiple conversations)
Lifetime	Exists only during a conversation	Persists for days, weeks, or months
How it works	Concatenate all messages and send to LLM	Search, retrieve relevant memories, inject into STM
Storage	In-memory variable (RAM)	Database (SQL, Key-Value, Vector DB, etc.)
Personalization	Not possible	Enables deep personalization
Key equation	$y_2 = f_\theta(\text{concat}(x_1, y_1, x_2))$	Retrieved LTM → injected into STM → becomes part of x

22.15.1 The Core Equation That Drives Everything

$$y = f_\theta(x)$$

- θ is fixed (from training) — we cannot change it.
- x is our lever — we make it smarter by injecting conversation history (STM) and retrieved long-term memories (LTM).
- y (the output) improves as x becomes richer and more informed.

Memory does not change the LLM. Memory changes the input to the LLM. And a smarter input produces a smarter output.

Short-Term Memory in LangGraph — Detailed Notes

23.1 Table of Contents

1. Prerequisites & Recap: LLMs Are Stateless
 2. What is Short-Term Memory (STM)?
 3. Implementing STM in LangGraph — Checkpointers & Threads
 4. Without Short-Term Memory (Code Example)
 5. With Short-Term Memory Using InMemorySaver (Code Example)
 6. Limitation of InMemorySaver — Volatile Storage
 7. Persistence Using PostgreSQL
 8. Setting Up PostgreSQL via Docker
 9. Persistent Memory with PostgresSaver (Code Example)
 10. Context Overflow Problem
 11. Technique 1: Trimming
 12. Technique 2: Deletion
 13. Technique 3: Summarization
 14. Summarization Workflow Implementation (Code Example)
 15. Comparison Table — Trimming vs Deletion vs Summarization
 16. Key Takeaways
-

23.2 1. Prerequisites & Recap: LLMs Are Stateless

Before diving into short-term memory in LangGraph, it is essential to understand a fundamental fact about Large Language Models (LLMs):

LLMs do not have any intrinsic memory. Every single call to `llm.invoke()` is treated as a completely fresh conversation. The LLM has no idea what you told it 5 seconds ago in a previous call. This property is called **statelessness**.

Think of it like a person who forgets everything the moment you stop talking. Every time you come back, you have to remind them of the entire conversation from the beginning.

This means:

- LLMs cannot remember your name across calls.
- LLMs cannot recall what topic you were discussing.
- Each `llm.invoke()` is an independent, isolated interaction.

So how do chatbots like ChatGPT “remember” conversations? They use a trick — they maintain a **conversation buffer**. Every time a new message is sent to the LLM, the entire previous

conversation history is also attached along with the new message. This way, at any moment, the LLM has the full context of the conversation inside a single `invoke()` call.

This entire mechanism of maintaining and sending conversation history is what we call **Short-Term Memory (STM)**.

23.3 2. What is Short-Term Memory (STM)?

Short-Term Memory in the context of LLMs refers to the technique of **maintaining a conversation buffer** that stores all the messages exchanged between the human and the AI during a session. When a new message is sent, the full buffer (all past messages + the new message) is passed to the LLM so that it can respond with awareness of the entire conversation.

Key Characteristics of STM:

- It exists only during a conversation session (or thread).
- It grows with every new message exchanged.
- It gives the LLM the illusion of “remembering” previous messages.
- It is fundamentally different from Long-Term Memory (which persists across sessions and conversations).

Analogy: Imagine you are at a meeting and someone is taking notes on a whiteboard. Every time someone speaks, the note-taker writes it down. When the next person needs to respond, they can look at the entire whiteboard to understand the full context. That whiteboard is your Short-Term Memory. If you erase the whiteboard (end the session), everything is lost.

23.4 3. Implementing STM in LangGraph — Checkpointers & Threads

LangGraph provides two core concepts to implement short-term memory:

23.4.1 3.1 Checkpointer

A **Checkpoint** is a mechanism that saves the state of your graph (including all messages) at every super-step. Think of it as an automatic save system — after every node execution, the current state is saved to some storage backend.

LangGraph offers different checkpointer backends:

Table 23.1. 1 Checkpointer

Checkpoint	Storage Location	Persistence	Use Case
InMemorySaver	RAM	Lost on program restart	Development / Testing
PostgresSaver	PostgreSQL Database	Survives restarts	Production systems

23.4.2 3.2 Thread ID

A **Thread ID** is a unique identifier for a conversation. Each conversation gets its own thread ID, and all messages within that conversation are stored against that specific thread ID.

Think of Thread IDs like separate chat windows. Thread 1 might be your conversation about cooking, Thread 2 might be about physics. The LLM keeps these conversations isolated — messages in Thread 1 do not leak into Thread 2.

How they work together:

1. You create a checkpoint (e.g., `InMemorySaver()`).
2. You compile your graph with this checkpoint.
3. For every `graph.invoke()` call, you pass a config dict that contains the `thread_id`.
4. The checkpoint automatically saves and retrieves messages for that specific thread.

23.5 4. Without Short-Term Memory (Code Example)

Let us first see what happens when we do NOT use any checkpoint or thread ID.

```
from langgraph.graph import StateGraph, START, MessagesState
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI()

# Define a single node
def call_model(state: MessagesState):
    response = model.invoke(state["messages"])
    return {"messages": [response]}

# Build the graph
builder = StateGraph(MessagesState)
builder.add_node("call_model", call_model)
builder.add_edge(START, "call_model")

# Compile WITHOUT a checkpoint
graph = builder.compile()
```

Graph structure:

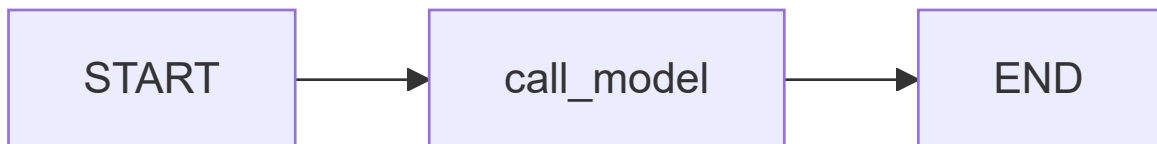


Figure 23.1. Without Short-Term Memory (Code Example)

Now let us test:

```
# First message
graph.invoke({"messages": [{"role": "user", "content": "Hi! My name is Nitish."}]})
# Response: "Nice to meet you, Nitish! How can I assist you today?"

# Second message
graph.invoke({"messages": [{"role": "user", "content": "What is my name?"}]})
# Response: "I'm sorry, I cannot know your name as I am a computer program."
```

What happened? The second call has no idea about the first call. Each `graph.invoke()` is independent. The LLM treats every call as a completely new conversation. This is the statelessness problem.

23.6 5. With Short-Term Memory Using InMemorySaver (Code Example)

Now let us add a checkpointer and thread ID to enable STM.

```
from langgraph.checkpoint.memory import InMemorySaver

# Same node definition as before
def call_model(state: MessagesState):
    response = model.invoke(state["messages"])
    return {"messages": [response]}

builder = StateGraph(MessagesState)
builder.add_node("call_model", call_model)
builder.add_edge(START, "call_model")

# Create an InMemorySaver checkpointer
checkpointer = InMemorySaver()

# Compile the graph WITH the checkpointer
graph = builder.compile(checkpointer=checkpointer)
```

Now we define a thread config and invoke:

```
# Define thread config
config = {"configurable": {"thread_id": "thread-1"}}

# First message - pass config
graph.invoke(
    {"messages": [{"role": "user", "content": "Hi! My name is Nitish."}]},
    config
)
# Response: "Hi Nitish! Nice to meet you."

# Second message - same thread
graph.invoke(
    {"messages": [{"role": "user", "content": "What is my name?"}]},
    config
)
# Response: "Your name is Nitish!"
```

It remembers now because both calls share thread-1. The checkpointer saved all messages from the first call, and when the second call happened on the same thread, those previous messages were automatically loaded and sent along.

Proving thread isolation:

```
config2 = {"configurable": {"thread_id": "thread-2"}}

graph.invoke(
    {"messages": [{"role": "user", "content": "What is my name?"}]},
    config2
)
# Response: "I'm sorry, I do not have the ability to know your name."
```

Thread 2 is a brand new conversation. It has no knowledge of Thread 1's messages.

Inspecting the state:

```
snap = graph.get_state(config)
for m in snap.values.get("messages", []):
    print("-", type(m).__name__, ":", m.content)
```

This will show all four messages stored in Thread 1 (2 human + 2 AI messages).

23.7 6. Limitation of InMemorySaver — Volatile Storage

InMemorySaver stores all state in **RAM (Random Access Memory)**. RAM is volatile — when the program stops, all data in RAM is lost.

What this means in practice:

1. You run your program and have a conversation (Thread 1 with 4 messages).
2. You restart the program (kernel restart, server reboot, etc.).
3. You try to access Thread 1's state — it is empty. All messages are gone.

This is acceptable for development and testing, but completely unacceptable for production systems where users expect their conversation history to persist.

The solution: Use a production-grade database to store the state, so that even if the application restarts, all conversation data is preserved. LangGraph recommends **PostgreSQL** for this purpose.

23.8 7. Persistence Using PostgreSQL

Persistence means that your conversation data **survives beyond the lifetime of the application process**. Even if the server crashes and restarts, all thread data (messages, summaries, etc.) remain intact in the database.

The architecture looks like this:



Figure 23.2. Persistence Using PostgreSQL

Two possible setups exist:

Table 23.2. Persistence Using PostgreSQL

Setup	Description	Recommended?
PostgreSQL installed directly on machine	Native install, can have OS-specific installation issues	Not ideal due to setup complexity
PostgreSQL via Docker container	Consistent, isolated, easy to start/stop	Yes — recommended approach

23.9 8. Setting Up PostgreSQL via Docker

Docker allows you to run PostgreSQL in an isolated container without installing it directly on your OS.

Step-by-step setup:

Step 1: Install Docker Desktop from <https://www.docker.com/products/docker-desktop/>

Step 2: Launch Docker Desktop and verify installation:

```
docker --version
# Output: Docker version 28.x.x, build xxxxxx
```

Step 3: Create a `docker-compose.yml` file:

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: postgres
    ports:
      - "5442:5432"
```

This file tells Docker to:

- Download the PostgreSQL 16 image.
- Set the username, password, and database name all to postgres.
- Map port 5442 on your machine to port 5432 inside the container (5432 is PostgreSQL's default port).

Step 4: Start the container:

```
docker compose up -d
```

The `-d` flag runs it in detached mode (in the background).

Step 5: Verify it is running:

```
docker ps
# You should see the postgres container running
```

Step 6: Install Python dependencies:

```
pip install langgraph langgraph-checkpoint-postgres "psycopg[binary,pool]" langchain-openai
```

***Note:** If you are using `zsh` (macOS default shell), the square brackets in `psycopg[binary,pool]` need to be quoted with double quotes: `"psycopg[binary,pool]"`. Otherwise, `zsh` interprets them as glob patterns and throws no matches found.*

23.10 9. Persistent Memory with PostgresSaver (Code Example)

Now let us replace `InMemorySaver` with `PostgresSaver` to achieve persistence.

```
from langgraph.graph import StateGraph, START, MessagesState
from langgraph.checkpoint.postgres import PostgresSaver
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

llm = ChatOpenAI(model="gpt-4o-mini")

def call_model(state: MessagesState):
    response = llm.invoke(state["messages"])
    return {"messages": [response]}

builder = StateGraph(MessagesState)
builder.add_node("call_model", call_model)
builder.add_edge(START, "call_model")
```

The key difference — using `PostgresSaver` inside a **context manager** (with statement):

```
DB_URI = "postgresql://postgres:postgres@localhost:5442/postgres"
```

```

with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup() # Creates necessary tables (run once)

    graph = builder.compile(checkpointer=checkpointer)

    # Thread 1 - two messages
    t1 = {"configurable": {"thread_id": "thread-1"}}
    graph.invoke({"messages": [{"role": "user", "content": "Hi, my name is Nitish"}]}, t1)
    out1 = graph.invoke({"messages": [{"role": "user", "content": "What is my name?"}]}, t1)
    print("Thread-1:", out1["messages"][-1].content)
    # Output: "Your name is Nitish."

```

Testing persistence — restart the program and retrieve state:

```

# After restarting the kernel / program:
with PostgresSaver.from_conn_string(DB_URI) as cp:
    g = builder.compile(checkpointer=cp)

    snap = g.get_state({"configurable": {"thread_id": "thread-1"}})
    msgs = snap.values.get("messages", [])
    print("Last message:", msgs[-1].content if msgs else None)
    # Output: "Your name is Nitish. What else would you like to know?"

```

The messages survived the restart because they are stored in PostgreSQL, not in RAM.

Key differences from InMemorySaver:

Table 23.3. Persistent Memory with PostgresSaver (Code Example)

Aspect	InMemorySaver	PostgresSaver
Storage	RAM	PostgreSQL Database
Persistence	Lost on restart	Survives restarts
Usage pattern	Direct instantiation	Context manager (with block)
Setup	None	Requires checkpointer.setup() once
Production-ready	No	Yes

23.11 10. Context Overflow Problem

Every LLM has a **context window** — the maximum number of tokens it can process at one time. This includes both the input tokens (your messages) and the output tokens (the LLM's response).

The context window can be thought of using a camera analogy:

- A **smaller lens** camera captures less of the scene → an LLM with a smaller context window can process fewer tokens.
- A **bigger lens** camera captures more → a bigger context window processes more tokens.

Mathematically, an LLM can be modeled as:

$$y = f_{\theta}(x)$$

Where:

- x = input tokens (your conversation history + new message)
- y = output tokens (the LLM's response)
- θ = model parameters
- The context window sets a hard upper bound on $|x| + |y|$

The Problem with STM:

In short-term memory, we concatenate the entire conversation history and send it with every new message. As the conversation grows (imagine 500 or 1000 messages), the total token count of the input can **exceed the context window**.

When this happens:

- The LLM may start **hallucinating** (producing incorrect or nonsensical responses).
- The LLM may give **incomplete or incoherent answers**.
- You fundamentally **cannot trust** the LLM's output anymore.

Rule of thumb: Your input tokens should always be well below the context window limit.

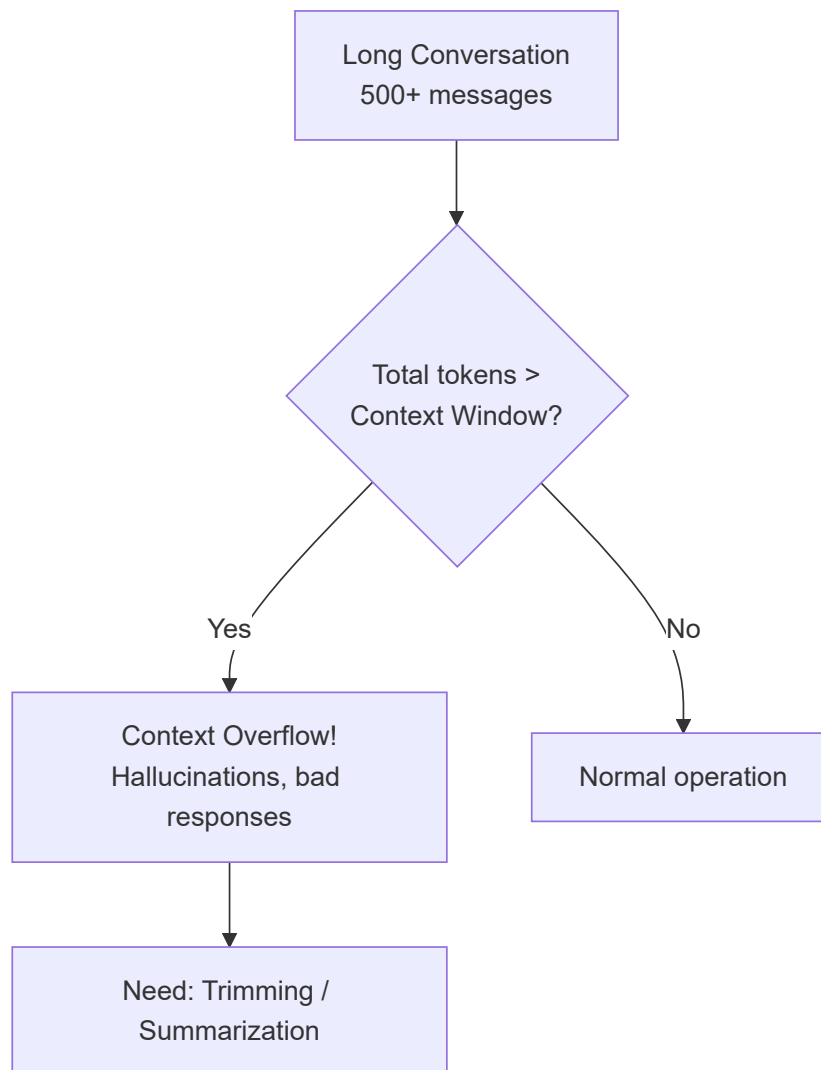


Figure 23.3. *Context Overflow Problem*

To fight this problem, three techniques are used: **Trimming**, **Deletion**, and **Summarization**.

23.12 11. Technique 1: Trimming

Trimming is the simplest technique to prevent context overflow.

23.12.1 How Trimming Works

1. You set a **maximum token limit** (e.g., 150 tokens).
2. Before every LLM call, you check the total token count of all messages in the conversation.
3. If the total is within the limit → send all messages as-is.
4. If the total exceeds the limit → keep only the **last N messages** whose combined token count fits within the limit. Remove (ignore) all older messages.

Critical point: Trimming does **not** delete messages from the state. The messages still exist in memory/database. Trimming only controls which messages are **shown to the LLM** during a particular invoke call.

23.12.2 Trimming — Visual Walkthrough

Suppose `MAX_TOKENS = 150`:

Table 23.4. *Trimming — Visual Walkthrough*

Turn	Action	Token Count	Trimming Needed?
1	User: "Hi, my name is Nitish"	10	No
2	User: "I am learning LangGraph" + past	40	No
3	User: "Can you explain short term memory?" + past	108	No
4	User: "What is my name?" + past	> 150	Yes — only the last message (8 tokens) is sent

At Turn 4, the combined history exceeds 150 tokens. So trimming kicks in — it keeps only the most recent message(s) that fit within 150 tokens. Since even the previous AI response alone was very large (the explanation of short-term memory), it had to be excluded. Only "What is my name?" was sent, so the LLM could not remember the name.

23.12.3 Trimming Code Example

```
from langchain_core.messages.utils import trim_messages, count_tokens_approximately

MAX_TOKENS = 150

def call_model(state: MessagesState):
    # Trim before sending to LLM
    messages = trim_messages(
        state["messages"],
        strategy="last",  # Keep last messages
        token_counter=count_tokens_approximately,
        max_tokens=MAX_TOKENS
    )

    print('Current Token Count ->', count_tokens_approximately(messages=messages))

    response = model.invoke(messages)
    return {"messages": [response]}
```

The `trim_messages` function does all the heavy lifting:

- It takes the full message list.
- It counts tokens using `count_tokens_approximately` (LangChain's built-in heuristic).

- With `strategy="last"`, it starts from the most recent message and works backward, including messages until the token budget is exhausted.
- All messages that don't fit are excluded (but NOT deleted from state).

23.12.4 Inherent Flaw of Trimming

Trimming assumes that **only the most recent N messages matter** and that older messages are no longer useful. This assumption fails in many real-world scenarios:

- A user mentioned their preferences 200 messages ago — still relevant.
- A critical piece of context (like a project name or a constraint) was discussed early in the conversation.
- The LLM completely loses awareness of anything that happened before the trimming window.

This flaw leads us to a better technique: **Summarization**.

23.13 12. Technique 2: Deletion

Deletion is the act of **permanently removing messages from the state**. Unlike trimming (which only hides messages from the LLM), deletion actually erases them from the state store.

Deletion is not typically used alone — it is a **prerequisite step for summarization**. In summarization, after generating a summary of old messages, those old messages are deleted from the state so that only the summary + recent messages remain.

23.13.1 How Deletion Works in LangGraph

LangGraph provides the `RemoveMessage` function to permanently delete messages from the state by their ID.

```
from langchain.messages import RemoveMessage

def delete_old_messages(state: MessagesState):
    msgs = state["messages"]

    # If more than 10 messages, delete the earliest 6
    if len(msgs) > 10:
        to_remove = msgs[:6]
        return {"messages": [RemoveMessage(id=m.id) for m in to_remove]}

    return {}
```

23.13.2 Deletion Workflow

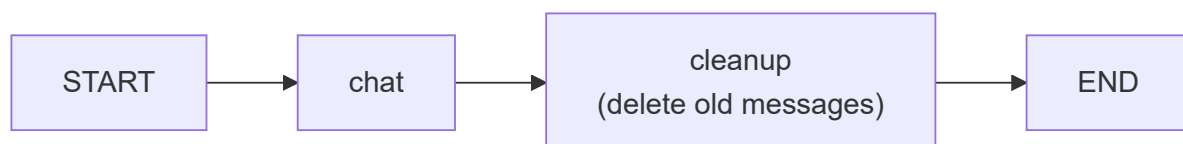


Figure 23.4. Deletion Workflow

The cleanup node runs after every chat response. It checks if the total number of messages exceeds a threshold (e.g., 10). If yes, it deletes the oldest 6 messages, keeping only the most recent 8 (the 4 that were already there plus any that were not removed).

23.13.3 Deletion Code Example

```
builder = StateGraph(MessagesState)
builder.add_node("chat", chat)
builder.add_node("cleanup", delete_old_messages)

builder.add_edge(START, "chat")
builder.add_edge("chat", "cleanup")
builder.add_edge("cleanup", "__end__")

graph = builder.compile(checkpointer=InMemorySaver())
```

After 7 invocations (14 messages total — 7 human + 7 AI), since $14 > 10$, the first 6 messages are permanently deleted. The state then contains only 8 messages.

Important: Deletion is permanent. Once removed via `RemoveMessage`, those messages are gone from the state (and from the database if using `PostgresSaver`). This is fundamentally different from trimming, which only hides messages during a single LLM call.

23.14 13. Technique 3: Summarization

Summarization is the most popular and effective technique for managing long conversations. It combines the best aspects of trimming (keeping recent messages) with context preservation (keeping a summary of older messages).

23.14.1 How Summarization Works

1. At any moment, you send the **most recent N messages** to the LLM (similar to trimming).
2. But instead of completely ignoring older messages, you take those older messages and pass them to a **separate LLM call** that generates a **summary**.
3. This summary is stored in the state.
4. The next time the LLM is called, it receives: **summary of old messages + most recent N messages**.
5. The old messages (the ones that were summarized) are **deleted** from the state.

23.14.2 Summarization — Worked Example

Suppose we decide to always keep the last 100 messages:

Stage 1: 500 messages in conversation.

- Last 100 messages → kept as-is.
- First 400 messages → summarized.
- LLM receives: `Summary(400)` + Last 100 messages

Stage 2: Conversation continues, 100 more messages added (total 600 now, but state has Summary + 200 messages).

- Last 100 messages → kept as-is.
- The 100 messages that were previously “recent” → summarized and merged with the existing summary.
- LLM receives: `Summary(500)` + Last 100 messages

This process repeats indefinitely. The summary keeps growing (but remains compact), and only the most recent messages are kept verbatim.

23.14.3 Why Summarization is Better Than Trimming

Table 23.5. *Why Summarization is Better Than Trimming*

Aspect	Trimming	Summarization
Old context	Completely lost	Preserved as a summary
LLM awareness	Only recent messages	Recent messages + summary of all past
Conversation coherence	Breaks down in long conversations	Maintained across very long conversations
Computational cost	Low (just counting tokens)	Higher (requires an LLM call for summarization)
State management	Messages stay in state (just hidden)	Old messages deleted after summarization

23.14.4 Key Detail: Deletion After Summarization

After generating a summary, the old messages must be **deleted** from the state. If you keep both the old messages AND the summary, you create redundancy and confusion. The summary replaces the old messages.

23.15 14. Summarization Workflow Implementation (Code Example)

23.15.1 Custom State with Summary Field

Since we need to store a summary alongside messages, we define a custom state:

```
from langgraph.graph import MessagesState

class ChatState(MessagesState):
    summary: str
```

This extends the built-in MessagesState (which has a messages key) with an additional summary key.

23.15.2 Workflow Structure

- If the state has 6 or fewer messages → no summarization needed, go to END.
- If more than 6 → trigger the summarize node, then go to END.

23.15.3 The Chat Node

The chat node handles two cases:

Case 1: No summary exists yet (conversation just started, ≤ 6 messages)

- Simply send the messages to the LLM.

Case 2: A summary already exists (summarization has happened before)

- Prepend the summary as a system message, then append the current messages.
- Send the combined list to the LLM.

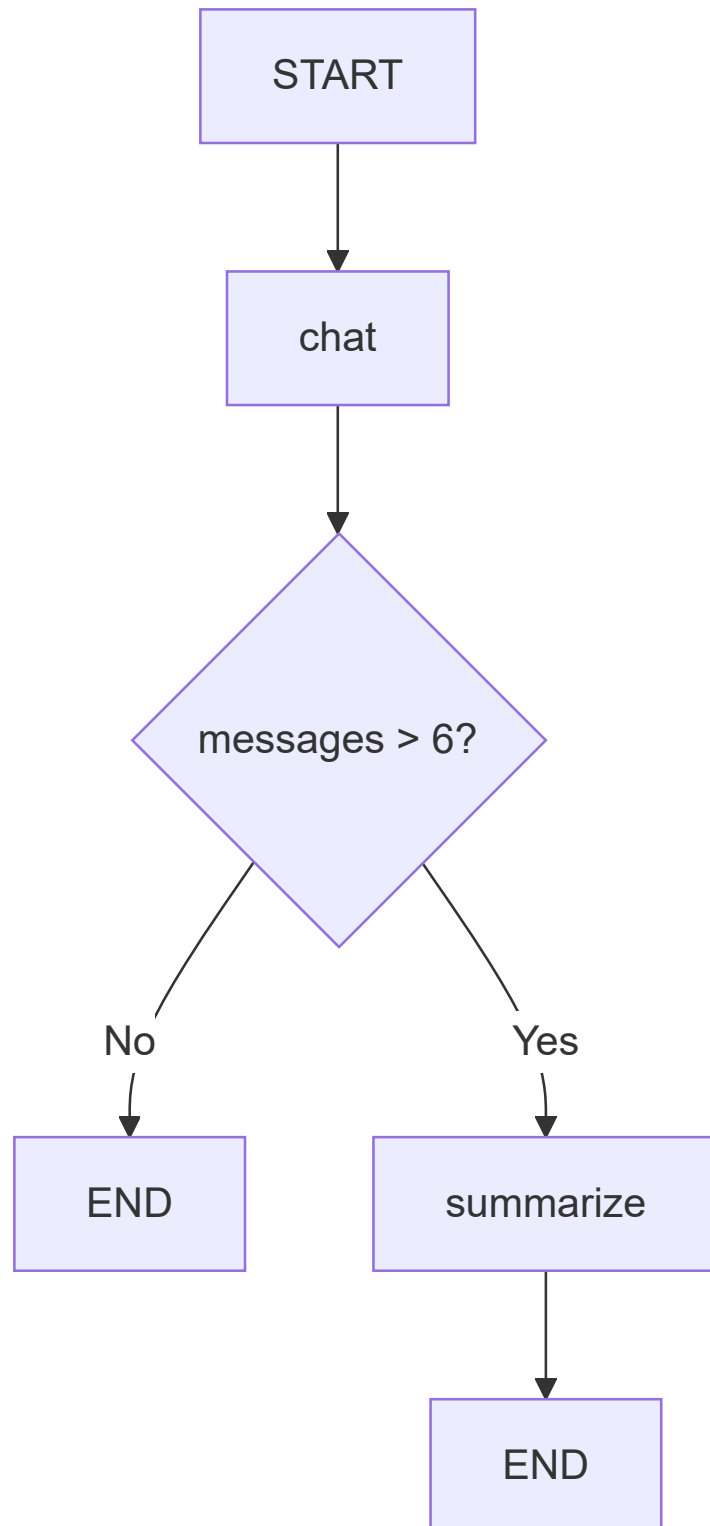


Figure 23.5. *Workflow Structure*

```
def chat_node(state: ChatState):
    messages = []

    # If summary exists, prepend it as system message
    if state["summary"]:
        messages.append({
            "role": "system",
            "content": f"Conversation summary:\n{state['summary']}"
        })

    # Add current messages
    messages.extend(state["messages"])

    response = model.invoke(messages)
    return {"messages": [response]}
```

23.15.4 The Summarize Node

This node does two things: (a) generates/extends a summary, and (b) deletes old messages.

```
from langchain_core.messages import HumanMessage, RemoveMessage

def summarize_conversation(state: ChatState):
    existing_summary = state["summary"]

    # Build the appropriate prompt
    if existing_summary:
        # Summary exists - extend it
        prompt = (
            f"Existing summary:\n{existing_summary}\n\n"
            "Extend the summary using the new conversation above."
        )
    else:
        # First time summarization
        prompt = "Summarize the conversation above."

    # Send messages + prompt to LLM for summarization
    messages_for_summary = state["messages"] + [HumanMessage(content=prompt)]
    response = model.invoke(messages_for_summary)

    # Keep only the last 2 messages, delete the rest
    messages_to_delete = state["messages"][:-2]

    return {
        "summary": response.content,
        "messages": [RemoveMessage(id=m.id) for m in messages_to_delete],
    }
```

How the prompt adapts:

Table 23.6. The Summarize Node

Scenario	Prompt Used
First-time summary	"Summarize the conversation above."
Extending existing summary	"Existing summary: {old_summary}. Extend the summary using the new conversation above."

23.15.5 Conditional Edge — Should We Summarize?

```
def should_summarize(state: ChatState):
    return len(state["messages"]) > 6
```

23.15.6 Building the Full Graph

```
builder = StateGraph(ChatState)

builder.add_node("chat", chat_node)
builder.add_node("summarize", summarize_conversation)

builder.add_edge(START, "chat")

builder.add_conditional_edges(
    "chat",
    should_summarize,
    {
        True: "summarize",
        False: "__end__",
    }
)

builder.add_edge("summarize", "__end__")

checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)
```

23.15.7 Testing the Summarization Workflow

```
config = {"configurable": {"thread_id": "t1"}}

def run_turn(text):
    out = graph.invoke(
        {"messages": [HumanMessage(content=text)], "summary": ""},
        config=config
    )
    return out

def show_state():
    snap = graph.get_state(config)
    vals = snap.values
    print("summary:", vals.get("summary", ""))
    print("num_messages:", len(vals.get("messages", [])))
```

Table 23.7. *Testing the Summarization Workflow*

Turn	User Message	Messages in State	Summary
1	"Quantum Physics"	2	Empty
2	"How is Albert Einstein related?"	4	Empty
3	"What are some of Einstein's famous works?"	6	Empty
4	"Explain special theory of relativity"	2 (only the last pair)	Generated! Summary of all 6 previous messages

At Turn 4, the state had 8 messages (6 existing + 2 new), which triggered summarization. The first 6 messages were summarized into a compact paragraph, then deleted. Only the last 2 messages remained in state alongside the new summary.

23.16 15. Comparison Table — Trimming vs Deletion vs Summarization

Table 23.8. Comparison Table — Trimming vs Deletion vs Summarization

Feature	Trimming	Deletion	Summarization
What it does	Hides old messages from LLM	Permanently removes messages from state	Generates summary of old messages, then deletes them
State impact	No change to state	Messages removed from state	Old messages removed, summary added
Context preservation	None — old context is lost	None — old messages are gone	Yes — summary preserves the essence
When to use	Quick & simple token control	As a sub-step within summarization	Production-grade long conversations
LangGraph function	<code>trim_messages()</code>	<code>RemoveMessage()</code>	Custom node with LLM call + <code>RemoveMessage()</code>
Computational cost	Very low	Very low	Higher (requires extra LLM call)
Best for	Short conversations, prototyping	Combined with summarization	Long, production conversations

23.17 16. Key Takeaways

1. **LLMs are stateless** — every `llm.invoke()` is a fresh conversation. Memory is an external construct, not built into the model.
2. **Short-Term Memory = Conversation Buffer** — you maintain a list of all messages and send the full list with every new call.
3. **Checkpointing + Thread ID** — these are the two building blocks for STM in LangGraph. The checkpointer saves state, and the thread ID identifies which conversation the state belongs to.
4. **InMemorySaver for development, PostgresSaver for production** — RAM storage is volatile and will lose data on restart. Use a database for any real deployment.
5. **Context Overflow is a real threat** — as conversations grow, you risk exceeding the LLM's context window, leading to hallucinations and bad outputs.
6. **Trimming is simple but lossy** — it keeps recent messages within a token budget but completely discards older context.
7. **Summarization is the recommended production approach** — it preserves old context as a compact summary while keeping recent messages verbatim. Combined with deletion, it keeps the state clean and manageable.
8. **Deletion supports summarization** — old messages are deleted from the state after being summarized, avoiding redundancy.
9. **All these techniques can be combined** — in a production system, you might use PostgresSaver for persistence, summarization for long-term context, and trimming as an additional safety net.

Note: All code examples in these notes are from the [campusx-official/memory-in-llms](#) repository and the associated YouTube playlist on Agentic AI using LangGraph by Nitesh (CampusX).

Long Term Memory in LangGraph — Complete Notes

24.1 1. Introduction: Why Long Term Memory Matters

When you build an LLM-based chatbot (like ChatGPT), users interact across **multiple conversation threads**. In one thread they discuss a technical topic, in another they plan travel, and in a third they explore philosophical ideas. The key insight is:

A user doesn't reveal everything about themselves in a single chat. They share bits and pieces across many separate conversations over time.

The Problem: Each conversation thread is isolated. The chatbot in thread #2 has no idea what the user said in thread #1 or thread #3.

The Goal: As a chatbot builder, you want to **personalize responses** over time. To do this, you need to **remember important facts about the user** across all conversations — their name, preferences, profession, ongoing projects, etc.

The Solution: Extract important pieces of information from every conversation and store them in a **persistent database** (called a **Memory Store**). Before answering any new question, the LLM first checks this store for relevant user information and uses it to personalize its response.

This persistent, cross-conversation memory is what we call **Long Term Memory (LTM)**.

24.1.1 How Long Term Memory Differs from Short Term Memory

Table 24.1. How Long Term Memory Differs from Short Term Memory

Feature	Short Term Memory	Long Term Memory
Scope	Within a single conversation thread	Across all conversation threads
Lifetime	Exists only while the thread is active	Persists indefinitely (even after restart)
Storage	The conversation's message history (context window)	External database (Postgres, Redis, etc.)
Purpose	Maintain context within one chat	Personalize responses using user-specific facts
Example	"You asked about Fibonacci 2 messages ago"	"User prefers Python and teaches AI on YouTube"

24.1.2 Real-World Example of LTM in Action

Imagine a user has had many chats with your bot. In the past, they mentioned:

- Their name is Nitish

- They prefer Python
- They are a teacher on YouTube

Now, when they ask: *“Write a code for Fibonacci series”*, the LLM:

1. Checks the Memory Store
2. Finds: *“User prefers Python”*
3. Writes the Fibonacci code **in Python** without the user having to specify

This is **personalization powered by Long Term Memory**.

24.2 2. LangGraph Memory Store Architecture

In LangGraph, Long Term Memory is implemented through the concept of a **Store**. The entire architecture follows a clean object-oriented design.

24.2.1 Class Hierarchy

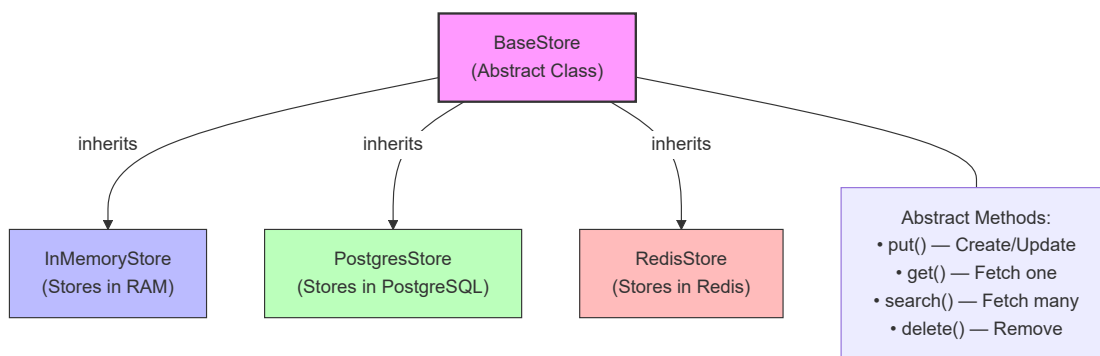


Figure 24.1. *Class Hierarchy*

24.2.2 BaseStore (Abstract Class)

`BaseStore` is an abstract class (inherits from Python’s `ABC`). It defines **what** a memory store can do but not **how**. The abstract methods it declares are:

- `put(namespace, key, value)` — Create or update a memory
- `get(namespace, key)` — Fetch a specific memory by its key
- `search(namespace)` — Fetch all memories (or semantically search)
- `delete(namespace, key)` — Remove a memory

24.2.3 InMemoryStore

- Inherits from `BaseStore`
- Stores all memories **in RAM**
- **Not persistent** — data is lost when the program restarts
- **Use case:** Quick prototyping and testing
- **Not suitable for production**

24.2.4 PostgresStore

- Inherits from BaseStore
- Stores all memories in a **PostgreSQL database**
- **Persistent** — data survives restarts, shutdowns, and deployments
- **Use case:** Production-grade chatbot systems

24.2.5 RedisStore

- Inherits from BaseStore
- Stores memories in **Redis** (an in-memory data structure store with persistence options)
- **Use case:** Production systems needing very fast read/write with optional persistence

24.3 3. Namespaces — Organizing Memories

24.3.1 What is a Namespace?

Think of your Memory Store as a **Google Drive**. A Namespace is like a **folder** inside that Drive. You use namespaces to organize memories logically.

Namespaces are always represented as **tuples of strings**.

24.3.2 Namespace Examples

Table 24.2. *Namespace Examples*

Namespace (Tuple)	Meaning
("user", "u1")	Top-level folder user → subfolder u1 (all memories for user u1)
("user", "u2")	Top-level folder user → subfolder u2 (all memories for user u2)
("user", "u1", "profile")	user → u1 → profile (profile-specific memories for u1)
("user", "u1", "preferences")	user → u1 → preferences (preference-specific memories for u1)
("user", "u1", "details")	user → u1 → details (general details for u1)

24.3.3 Visual Folder Analogy

```

Memory Store (Google Drive) |—
user/|
  |— u1/|
  |   |— profile/      → Name, Age, Gender, Profession|
  |   |— preferences/  → Dark mode, Python language, Bullet points|
  |   |— details/      → All general memories|
  |— u2/|
  |   |— profile/|
  |   |— preferences/

```

Key Rule: Every memory you create lives inside a specific namespace. You must always specify the namespace when creating, reading, or searching memories.

24.4 4. Working with InMemoryStore — CRUD Operations

24.4.1 Step 1: Import and Create the Store

```
from langgraph.store.memory import InMemoryStore

# Create an in-memory store object
store = InMemoryStore()
```

The store object now has access to all methods: `put()`, `get()`, `search()`, `delete()`.

24.4.2 Step 2: Define a Namespace

```
namespace = ("user", "u1")
```

This creates a logical folder path: `user` $\rightarrow$ `u1`. All memories stored under this namespace belong to user `u1`.

24.4.3 Step 3: Create Memories (put)

The `put()` method requires three arguments:

Table 24.3. Step 3: Create Memories (`put`)

Argument	Description	Example
namespace	The folder where the memory goes	<code>("user", "u1")</code>
key	A unique identifier for this memory	<code>"1", "profile_1",</code> UUID string
value	The actual data (a dictionary)	<code>{"data": "User likes pizza"}</code>

```
# Create two memories for user u1
store.put(namespace, "1", {"data": "User likes pizza"})
store.put(namespace, "2", {"data": "User prefers dark mode"})

# Create memories for a second user
namespace2 = ("user", "u2")
store.put(namespace2, "1", {"data": "User likes pasta"})
store.put(namespace2, "2", {"data": "User prefers grid style navigation"})
```

24.4.4 Step 4: Retrieve a Specific Memory (get)

The `get()` method needs only the namespace and the key.

```
result = store.get(namespace2, "2")
# Output:
# Item(namespace=['user', 'u2'], key='2',
#       value={'data': 'User prefers grid style navigation'},
#       created_at=..., updated_at=...)
```

When to use `get()`: When you know the exact key of the memory you want.

24.4.5 Step 5: Retrieve All Memories (search)

The `search()` method takes only the namespace and returns **all** memories in that namespace.

```
items = store.search(namespace2)

for item in items:
    print(item.value)

# Output:
# {'data': 'User likes pasta'}
# {'data': 'User prefers grid style navigation'}
```

When to use `search()`: When you want to fetch every memory for a user (or namespace).

24.5 5. Semantic Search in Memory Store

24.5.1 The Problem with `get()` and `search()`

- `get()` requires the exact key — you must know *which* memory you want.
- `search()` returns *all* memories — dumping 100 memories into the LLM’s context window causes confusion and wastes tokens.

Real Scenario: A user has 100 stored memories. They’re currently chatting about their Mumbai travel plan. Only 2–3 memories relate to Mumbai. You need to fetch *only those relevant memories*.

24.5.2 The Solution: Semantic Search

Semantic search matches memories based on **meaning**, not exact keywords. It uses an **embedding model** to:

1. Convert each memory into a vector (when stored)
2. Convert the search query into a vector (at query time)
3. Find the memories whose vectors are closest to the query vector

24.5.3 Implementation

Two changes are needed compared to basic usage:

Change 1: Pass an embedding model when creating the store.

```
from langchain_openai import OpenAIEmbeddings

embedding_model = OpenAIEmbeddings(model='text-embedding-3-small')

store = InMemoryStore(
    index={
        'embed': embedding_model,
        'dims': 1536 # dimension of the embedding vectors
    }
)
```

Change 2: Pass a query and `limit` to the `search()` method.

```
# Create 10 diverse memories
namespace = ('users', 'u1')

store.put(namespace, "1", {"data": "User prefers concise answers over long explanations"})
store.put(namespace, "2", {"data": "User likes examples in Python"})
store.put(namespace, "3", {"data": "User usually works late at night"})
store.put(namespace, "4", {"data": "User prefers dark mode in applications"})
store.put(namespace, "5", {"data": "User is learning machine learning"})
store.put(namespace, "6", {"data": "User dislikes overly theoretical explanations"})
```

```
store.put(namespace, "7", {"data": "User prefers step-by-step reasoning"})
store.put(namespace, "8", {"data": "User is based in India"})
store.put(namespace, "9", {"data": "User likes real-world analogies"})
store.put(namespace, "10", {"data": "User prefers bullet points over paragraphs"})
```

Example 1: Find what user is learning

```
items = store.search(namespace, query="what is the user currently learning", limit=1)
for item in items:
    print(item.value)
# Output: {'data': 'User is learning machine learning'}
```

Example 2: Find user preferences (top 3)

```
items = store.search(namespace, query="what are user's preferences", limit=3)
for item in items:
    print(item.value)
# Output:
# {'data': 'User prefers dark mode in applications'}
# {'data': 'User prefers bullet points over paragraphs'}
# {'data': 'User prefers concise answers over long explanations'}
```

24.5.4 How Semantic Search Works Internally

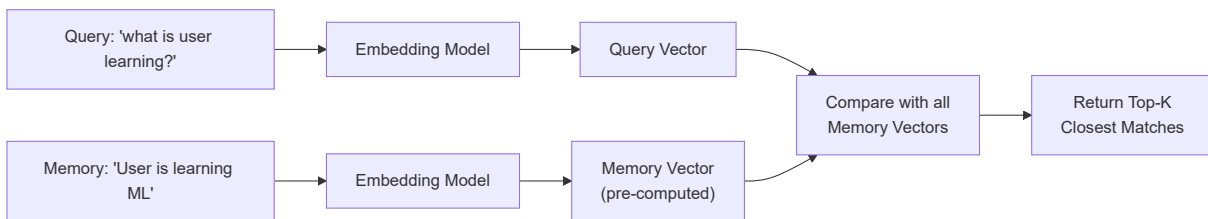


Figure 24.2. *How Semantic Search Works Internally*

The `limit` parameter controls how many top matches are returned. Setting `limit=1` gives the single most relevant memory; `limit=3` gives the top three.

24.6 6. Part 1 — Chatbot That READS Existing Memories

24.6.1 Goal

Build a simple LangGraph chatbot (START → chat → END) that:

- Has pre-filled memories in the store
- Reads those memories before responding
- Uses them to personalize its answers

Important: This chatbot can only READ memories. It does NOT create new ones.

24.6.2 Workflow Diagram

24.6.3 Complete Code

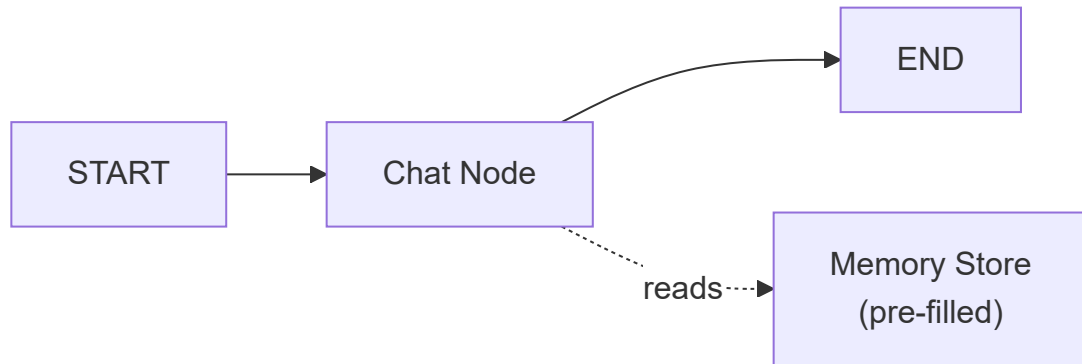


Figure 24.3. Workflow Diagram

```

from dotenv import load_dotenv
load_dotenv()

from langchain_openai import ChatOpenAI
from langchain_core.runnables import RunnableConfig
from langchain_core.messages import SystemMessage
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.store.memory import InMemoryStore
from langgraph.store.base import BaseStore

# 1) Create store and seed memories
store = InMemoryStore()
user_id = "u1"
user_details = ("user", user_id, "details")

store.put(user_details, "profile_1", {"data": "Name: Nitish"})
store.put(user_details, "profile_2", {"data": "Profession: Teaches AI on YouTube"})
store.put(user_details, "preference_1", {"data": "Prefers concise answers"})
store.put(user_details, "preference_2", {"data": "Likes examples in Python"})
store.put(user_details, "project_1",
          {"data": "Building MCP servers (Python-based project)"})

# 2) System prompt template
SYSTEM_PROMPT_TEMPLATE = """You are a helpful assistant with memory capabilities.
If user-specific memory is available, use it to personalize
your responses based on what you know about the user.
...
The user's memory (which may be empty) is provided as: {user_details_content}
"""

# 3) LLM
llm = ChatOpenAI(model="gpt-4o-mini")

# 4) Chat node
def chat_node(state: MessagesState, config: RunnableConfig, store: BaseStore):
    user_id = config["configurable"]["user_id"]
    user_details = ("user", user_id, "details")

    items = store.search(user_details)

    if items:
        user_details_content = "\n".join(
            f"- {it.value.get('data', '')}" for it in items
        )
    else:
        user_details_content = ""

    system_prompt = SYSTEM_PROMPT_TEMPLATE.format(
        user_details_content=user_details_content
    )
    system_msg = SystemMessage(content=system_prompt)

```

```
response = llm.invoke([system_msg] + state["messages"])
return {"messages": [response]}

# 5) Build and compile graph
builder = StateGraph(MessagesState)
builder.add_node("chat", chat_node)
builder.add_edge(START, "chat")
builder.add_edge("chat", END)
graph = builder.compile(store=store)

# 6) Run
config = {"configurable": {"user_id": "u1"}}
result = graph.invoke(
    {"messages": [{"role": "user", "content": "Explain gen ai in simple terms."}]},
    config,
)
print(result["messages"][-1].content)
```

24.6.4 How the Chat Node Works — Step by Step

1. Extract `user_id` from the `config` parameter (passed during `graph.invoke`)
2. Build the namespace from the `user_id`: ("user", "u1", "details")
3. Search the memory store for all memories under that namespace
4. Format memories into a string — each memory on a new line with a hyphen prefix
5. Inject the memory string into the system prompt template via `.format()`
6. Convert to `SystemMessage` and prepend it to the conversation history
7. Invoke the LLM with the full message list (system + conversation)
8. Return the response as a new message

24.6.5 Key Concept: Where Do `config` and `store` Come From?

The `chat_node` function receives three parameters:

Table 24.4. Key Concept: Where Do `config` and `store` Come From?

Parameter	Type	Source
<code>state</code>	<code>MessagesState</code>	The current conversation state (all messages so far)
<code>config</code>	<code>RunnableConfig</code>	Passed during <code>graph.invoke(..., config)</code> — contains <code>user_id</code>
<code>store</code>	<code>BaseStore</code>	Passed during <code>builder.compile(store=store)</code> — the memory store

24.6.6 Result

When asked “Explain gen ai in simple terms”, the LLM responds:

- Starts with “Sure, **Nitish!**” (read from memory)
- Suggests examples in **Python** (read from memory)
- Asks follow-up about **teaching materials** (read from memory about YouTube teaching)

24.7 7. Part 2 — Chatbot That CREATES New Memories

24.7.1 Goal

Build a workflow that extracts memorable information from the user’s message and stores it in the memory store.

Important: This workflow ONLY creates memories. It does NOT chat or use existing memories (kept simple for learning).

24.7.2 Workflow Diagram

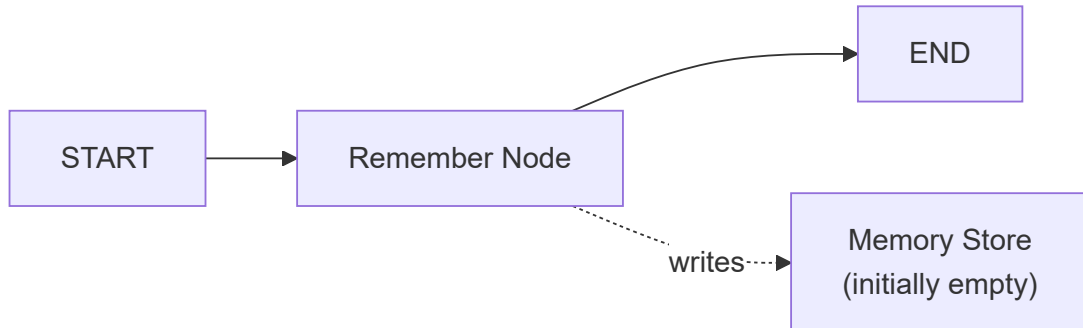


Figure 24.4. Workflow Diagram

24.7.3 Structured Output for Memory Extraction

We use a **Pydantic model** to force the LLM to return structured output:

```

from pydantic import BaseModel, Field
from typing import List

class MemoryDecision(BaseModel):
    should_write: bool = Field(
        description="Whether to store any memories"
    )
    memories: List[str] = Field(
        default_factory=list,
        description="Atomic user memories to store"
    )
  
```

should_write: A boolean flag. If True, the message contains something worth remembering. If False, skip writing entirely.

memories: A list of short, atomic sentences extracted from the user's message. Each sentence captures one piece of information.

24.7.4 The Remember Node

```

def remember_only_node(state: MessagesState, config: RunnableConfig,
                       store: BaseStore):
    user_id = config["configurable"]["user_id"]
    namespace = ("user", user_id, "details")
    last_msg = state["messages"][-1].content

    decision: MemoryDecision = memory_extractor.invoke([
        SystemMessage(content=(
            "Extract LONG-TERM memories from the user's message.\n"
            "Only store stable, user-specific info "\
            "(identity, preferences, ongoing projects).\n"
            "Do NOT store transient info.\n"
            "Return should_write=false if nothing is worth storing.\n"
            "Each memory should be a short atomic sentence."
        )),
        {"role": "user", "content": last_msg},
    ])
  
```

```
if decision.should_write:
    for mem in decision.memories:
        store.put(namespace, str(uuid.uuid4()), {"data": mem})

return {"messages": [{"role": "assistant", "content": "Noted."}]}
```

24.7.5 The Duplication Problem

If you send the same message twice (e.g., “My name is Nitish”), the node creates **duplicate memories**. After running the same three messages twice, you’d have 6 memories instead of 3.

Root Cause: The node doesn’t check whether a memory already exists before storing it.

24.8 8. De-Duplication Strategy

24.8.1 Approach

Instead of just sending the user’s latest message to the extractor LLM, also send **all existing memories**. Ask the LLM to mark each extracted memory as either:

- `is_new = True` → This is genuinely new information
- `is_new = False` → This already exists in the store (duplicate)

Then, only store memories where `is_new` is `True`.

24.8.2 Updated Pydantic Models

```
class MemoryItem(BaseModel):
    text: str = Field(description="Atomic user memory as a short sentence")
    is_new: bool = Field(
        description="True if this memory is NEW and should be stored. "
        "False if duplicate/already known."
    )

class MemoryDecision(BaseModel):
    should_write: bool = Field(description="Whether to store any memories")
    memories: List[MemoryItem] = Field(
        default_factory=list,
        description="Atomic user memories to store"
    )
```

24.8.3 Updated System Prompt

```
MEMORY_PROMPT = """You are responsible for updating and maintaining accurate
user memory.

CURRENT USER DETAILS (existing memories):
{user_details_content}

TASK:
- Review the user's latest message.
- Extract user-specific info worth storing long-term.
- For each extracted item, set is_new=true ONLY if it adds NEW information
  compared to CURRENT USER DETAILS.
- If it is basically the same meaning as something already present,
  set is_new=false.
- Keep each memory as a short atomic sentence.
```



```
- No speculation; only facts stated by the user.
- If there is nothing memory-worthy, return an empty list.
"""
```

24.8.4 Updated Node Logic

```
def remember_node(state, config, *, store):
    user_id = config["configurable"]["user_id"]
    ns = ("user", user_id, "details")

    # A) Load existing memories
    existing_items = store.search(ns)
    existing_texts = [it.value.get("data", "") for it in existing_items]
    user_details_content = (
        "\n".join(f"- {t}" for t in existing_texts)
        if existing_texts else "(empty)"
    )

    # B) Extract from latest message
    last_text = state["messages"][-1].content
    decision = memory_extractor.invoke([
        SystemMessage(content=MEMORY_PROMPT.format(
            user_details_content=user_details_content
        )),
        {"role": "user", "content": last_text},
    ])

    # C) Store ONLY new memories
    if decision.should_write:
        for mem in decision.memories:
            if mem.is_new: # ← the critical check
                store.put(ns, str(uuid.uuid4()), {"data": mem.text})

    return {}
```

24.8.5 Before vs After De-Duplication

Table 24.5. Before vs After De-Duplication

Scenario	Without De-Duplication	With De-Duplication
Send “My name is Nitish” once	1 memory created	1 memory created
Send “My name is Nitish” again	2nd duplicate created (total: 2)	LLM marks is_new=False, no duplicate
Send “I like Python” after “I like Python”	Duplicate stored	Existing detected, skipped

24.9 9. Merged Workflow — Read + Write Memories Together

24.9.1 Goal

Combine both capabilities into a single chatbot that:

1. **Extracts and stores** new memories from the user’s message
2. **Reads existing memories** to personalize its response

24.9.2 Workflow Diagram

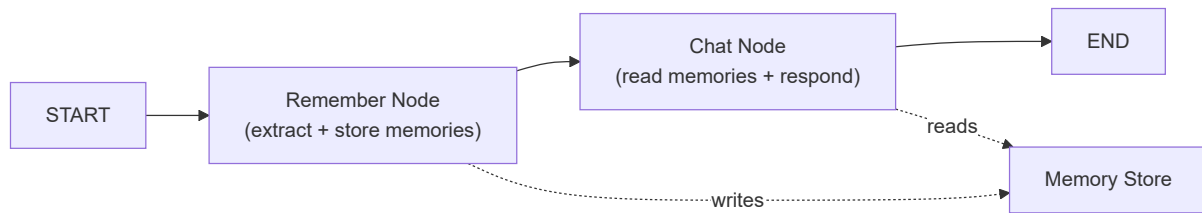


Figure 24.5. Workflow Diagram

24.9.3 Two Separate LLMs

Table 24.6. Two Separate LLMs

LLM	Purpose	Model
memory_llm	Extract memories from user messages	gpt-4o-mini (temperature=0 for deterministic extraction)
chat_llm	Generate personalized conversational responses	gpt-4o-mini (default temperature for creativity)

24.9.4 Complete Merged Code

```

# 1) Store (starts empty)
store = InMemoryStore()

# 2) Memory extraction LLM + Pydantic models
memory_llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

class MemoryItem(BaseModel):
    text: str = Field(description="Atomic user memory")
    is_new: bool = Field(description="True if new, false if duplicate")

class MemoryDecision(BaseModel):
    should_write: bool
    memories: List[MemoryItem] = Field(default_factory=list)

memory_extractor = memory_llm.with_structured_output(MemoryDecision)

# 3) Remember Node
def remember_node(state, config, *, store):
    user_id = config["configurable"]["user_id"]
    ns = ("user", user_id, "details")
    items = store.search(ns)
    existing = "\n".join(it.value["data"] for it in items) if items else "(empty)"
    last_msg = state["messages"][-1].content

    decision = memory_extractor.invoke([
        SystemMessage(content=MEMORY_PROMPT.format(
            user_details_content=existing)),
        {"role": "user", "content": last_msg},
    ])
    if decision.should_write:
        for mem in decision.memories:
            if mem.is_new:
                store.put(ns, str(uuid.uuid4()), {"data": mem.text})
    return {} # no message change - just memory work

# 4) Chat Node
chat_llm = ChatOpenAI(model="gpt-4o-mini")

```

```
def chat_node(state, config, *, store):
    user_id = config["configurable"]["user_id"]
    ns = ("user", user_id, "details")
    items = store.search(ns)
    user_details = "\n".join(it.value["data"] for it in items) if items else ""

    system_msg = SystemMessage(
        content=SYSTEM_PROMPT_TEMPLATE.format(
            user_details_content=user_details or "(empty)")
    )
    response = chat_llm.invoke([system_msg] + state["messages"])
    return {"messages": [response]}

# 5) Build Graph
builder = StateGraph(MessagesState)
builder.add_node("remember", remember_node)
builder.add_node("chat", chat_node)
builder.add_edge(START, "remember")
builder.add_edge("remember", "chat")
builder.add_edge("chat", END)
graph = builder.compile(store=store)
```

24.9.5 Execution Flow for a Single Message

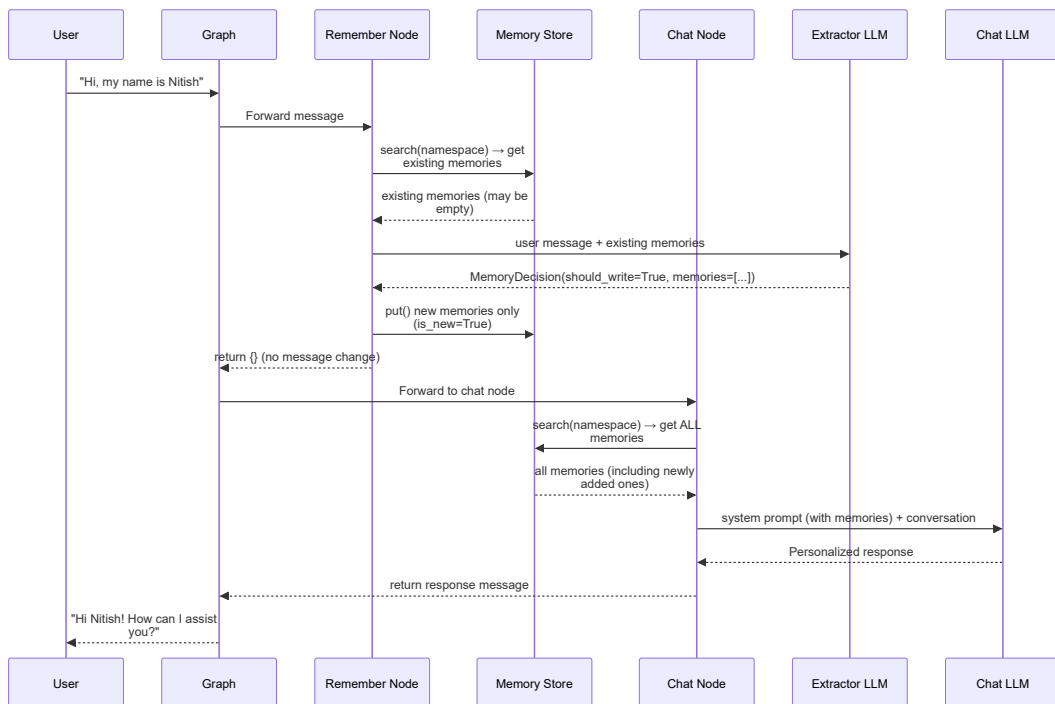


Figure 24.6. Execution Flow for a Single Message

24.9.6 Demo Results

Message 1: “Hi, my name is Nitish”

- Memory Store after: User's name is Nitish.
- Response: “Hi Nitish! It’s great to meet you. How can I assist you today?”

Message 2: “I teach AI on YouTube”

- Memory Store after: User's name is Nitish. + Nitish teaches AI on YouTube.
- Response: “That’s great, Nitish! Teaching AI on YouTube sounds exciting...”

Message 3: “Explain GenAI simply”

- Memory Store after: Still 2 memories (nothing new to store)
 - Response: “Sure, Nitish!” + explanation + follow-up questions about **teaching** and **Python**
-

24.10 10. Production Setup with PostgresStore

24.10.1 The InMemoryStore Problem

InMemoryStore keeps everything in RAM. When you restart the program, kernel, or machine — **all memories are gone**. This is unacceptable for production.

24.10.2 PostgresStore — Persistent Memory

PostgresStore stores memories in a PostgreSQL database. Data persists across restarts, shutdowns, and even server migrations.

24.10.3 Setup Steps

Step 1: Install Docker Desktop on your machine.

Step 2: Verify Docker is running:

```
docker --version
```

Step 3: Start a PostgreSQL container:

```
docker run --name langgraph-postgres \  
-e POSTGRES_USER=postgres \  
-e POSTGRES_PASSWORD=postgres \  
-e POSTGRES_DB=postgres \  
-p 5442:5432 \  
-d postgres:16
```

```
ER=postgres  
-e POSTGRES_PASSWORD=postgres  
-e POSTGRES_DB=postgres  
-p 5442:5432  
-d postgres:16 \end{lstlisting}
```

Step 4: Install required Python libraries:

```
pip install -U "psycopg[binary,pool]" langgraph langgraph-checkpoint-postgres
```

24.10.4 Code Changes

The only difference from the InMemoryStore version is **how you create and use the store**:

```
from langgraph.store.postgres import PostgresStore  
  
DB_URI = "postgresql://postgres:postgres@localhost:5442/postgres?sslmode=disable"  
  
with PostgresStore.from_conn_string(DB_URI) as store:  
    # Run ONCE the first time you use this database  
    store.setup()  
  
    # Compile graph with PostgresStore  
    graph = builder.compile(store=store)
```

```
# Use exactly the same way
config = {"configurable": {"user_id": "u1"}}
graph.invoke(
    {"messages": [{"role": "user", "content": "Hi, my name is Nitish"}]},
    config
)
```

24.10.5 Persistence Proof

After running the chatbot and storing memories, **restart the kernel entirely**, then:

```
from langgraph.store.postgres import PostgresStore

DB_URI = "postgresql://postgres:postgres@localhost:5442/postgres?sslmode=disable"

with PostgresStore.from_conn_string(DB_URI) as store:
    items = store.search(("user", "u1", "details"))

for it in items:
    print(it.value["data"])

# Output (data survived the restart):
# Nitish teaches AI on YouTube.
# User's name is Nitish.
```

24.10.6 InMemoryStore vs PostgresStore

Table 24.7. InMemoryStore vs PostgresStore

Feature	InMemoryStore	PostgresStore
Storage	RAM	PostgreSQL database
Persistence	Volatile (lost on restart)	Persistent (survives restarts)
Speed	Very fast	Slightly slower (network + disk)
Use case	Prototyping, testing	Production systems
Setup	None	Requires PostgreSQL (Docker or hosted)
Import	from langgraph.store.memory import InMemoryStore	from langgraph.store.postgres import PostgresStore

24.11 11. Summary — Complete Mental Model

24.11.1 Key Takeaways

- 1. **Long Term Memory** = persistent storage of user-specific facts extracted across multiple conversations.
- 2. **Namespaces** organize memories like folders — typically structured as ("user", "<user_id>", "details").
- 3. **Three core operations:** put() to create, get() to fetch one, search() to fetch many (with optional semantic search).
- 4. **Semantic search** uses an embedding model to match memories by meaning instead of exact keys — essential when users have many memories.

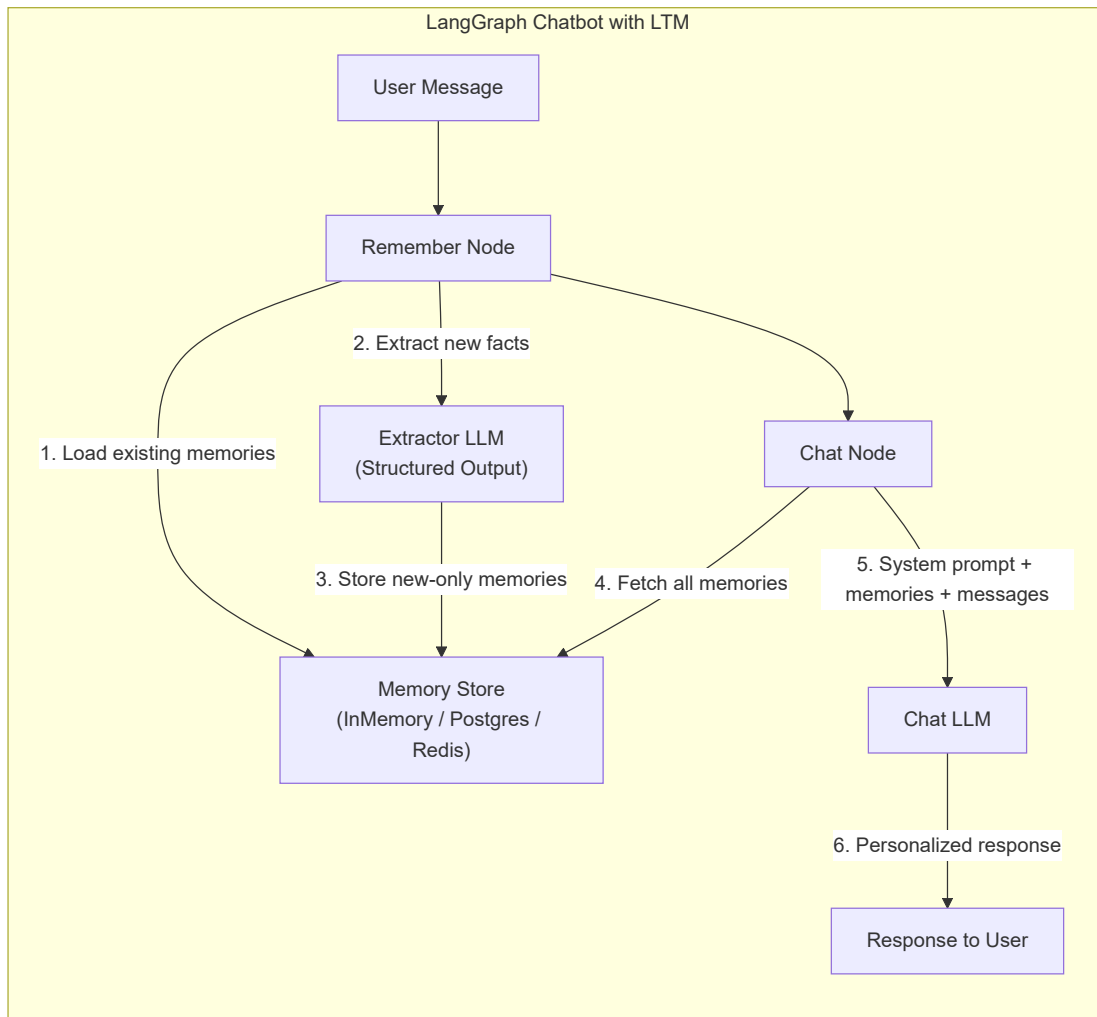


Figure 24.7. Summary — Complete Mental Model

5. **De-duplication** is critical. Send existing memories along with the new message to the extractor LLM, and only store memories marked as `is_new=True`.
6. **The merged workflow** has two nodes: `remember` (write) → `chat` (read + respond). Both interact with the same memory store.
7. **For production**, replace `InMemoryStore` with `PostgresStore` (or `RedisStore`). The rest of the code stays identical — thanks to the `BaseStore` abstraction.
8. **Two LLMs** work in tandem: one for memory extraction (structured, deterministic) and one for conversation (creative, personalized).

Notes based on the LangGraph Long Term Memory tutorial from the Agentic AI using LangGraph playlist.

Planning AI Agent — Blog Writing Agent with LangGraph

25.1 Table of Contents

1. What is a Planning Agent?
 2. Planning Agent vs Direct Execution
 3. Project Overview — Blog Writing Agent
 4. High-Level Architecture
 5. Orchestrator–Worker Pattern
 6. Stage 1 — Basic Blog Writing Agent
 7. Improving with Better Prompts & Pydantic Models
 8. Stage 2 — Adding Internet Research
 9. Tavily — Search Engine for LLMs
 10. Complete Data Flow Summary
-

25.2 1. What is a Planning Agent?

A **Planning Agent** is an AI agent that does **not** immediately jump to answering or acting. Instead, it first creates a **structured plan** of what needs to be done, and then **executes that plan step-by-step**.

Planning agents are useful when tasks are:

- **Long or multi-step** — Tasks that involve multiple stages of work.
- **Require structure** — Tasks like writing blogs, building projects, developing apps, etc.

A planning agent works in **two phases**:

Phase 1 — Plan: The agent receives a task and breaks it down into clear steps or sub-tasks. It figures out what needs to be done, in what order, and how each piece should be handled.

Phase 2 — Execute: The agent completes each step systematically, often checking progress along the way. Each sub-task is handled one by one (or in parallel) until the full task is complete.

***Key Insight:** For complex tasks, planning before execution leads to significantly better results than jumping straight into action.*

25.3 2. Planning Agent vs Direct Execution

To understand why planning agents matter, consider two approaches to a complex task like “Build an Expense Tracker Website”:

Approach 1 — Direct Execution (No Planning): The AI agent immediately starts writing code as soon as it receives the task. There is a good chance the outcome will have problems — missing features, inconsistent design, or incomplete logic — because the agent never stopped to think about the full scope.

Approach 2 — Planning First, Then Execution: The AI agent first understands the task, creates a structured plan (e.g., what pages to build, what database schema to use, what features to include), and then executes the plan step-by-step. This approach produces far better outcomes for complex tasks.

Table 25.1. *Planning Agent vs Direct Execution*

Aspect	Direct Execution	Planning Agent
First action	Immediately starts working	Creates a structured plan
Risk of missing things	High	Low
Quality of output	Inconsistent	More consistent and thorough
Best for	Simple, one-step tasks	Complex, multi-step tasks
Phases	Single phase	Two phases (Plan → Execute)

25.4 3. Project Overview — Blog Writing Agent

The project builds a **Blog Writing Agent** that takes a topic as input and generates a detailed, well-structured blog post. This is a perfect use case for a planning agent because writing a blog involves multiple sections, coherent structure, and sometimes external research.

25.4.1 Features of the Final Agent

1. **Planning** — Automatically creates a blog plan with sections, word targets, and structure.
2. **Internet Research** — Can go online using Tavily to fetch current information when needed.
3. **Image Generation** — Automatically identifies where images are needed and generates them (using Gemini).
4. **Blog History** — Saves previously generated blogs for future reference.
5. **Evidence & Citations** — When research is used, sources and links are tracked and displayed.
6. **Downloadable Output** — The final blog (with images) can be downloaded.

25.4.2 Four Development Stages

The project is built incrementally in **four stages**:

Stage 1 — Basic Blog Writing Agent: A simple text-only blog generator. No research, no images. The agent plans the sections and writes them.

Stage 2 — Adding Research Capability: The agent can now search the internet (using Tavily) to fetch up-to-date information and incorporate it into the blog.

Stage 3 — Adding Image Generation: The agent identifies where images are needed in the blog and generates them automatically.

Stage 4 — GUI (Graphical User Interface): The entire system is wrapped in a web-based GUI where users can input topics, see progress, view the plan, read the blog, and download it.

25.5 4. High-Level Architecture

The full agent architecture is built as a **LangGraph graph** with the following nodes:

```
__start__ → router → research → orchestrator → worker(s) → reducer → __end__
```

25.5.1 Node-by-Node Breakdown

1. **Start Node:** Receives the blog topic from the user. For example: “Self Attention” or “Top AI News of January 2026”.

2. **Router Node:** Decides whether the given topic needs internet research or not.

- If the topic is conventional (e.g., “Self Attention”, “Transformer Architecture”), the LLM already has the knowledge → **no research needed**.
- If the topic depends on recent/current information (e.g., “Top AI News of January 2026”), the LLM does not have this in its parametric knowledge → **research needed**.

3. **Research Node (conditional):** Only triggered if the router says research is needed. This node takes the topic, breaks it down into specific search queries, and uses **Tavily** (a search engine for LLMs) to fetch information from the internet. The results are stored in the state.

4. **Orchestrator Node (Planner):** This is the **planning node**. It takes the topic (and any research results, if available) and generates a structured plan for the blog. The plan includes which sections to write, how many words per section, and what each section should cover.

5. **Worker Nodes (Writers):** These are the execution nodes. Based on the plan, multiple worker nodes are dynamically spawned — one per section. Each worker independently writes its assigned section. All workers operate **in parallel**.

6. **Reducer Node:** Collects all completed sections from the workers, stitches them together in order, optionally adds images where needed, and produces the final markdown blog file.

25.6 5. Orchestrator–Worker Pattern

The Orchestrator–Worker pattern is a core design pattern used in this project. It allows **dynamic parallelism** — the number of workers is not fixed at design time but determined at runtime based on the plan.

25.6.1 How It Works

1. The **Orchestrator** (planner) creates a plan with N sections.
2. A **Fan-out** function reads the plan and dynamically spawns N worker nodes.
3. Each **Worker** receives its assigned task (one section) and works independently.
4. All workers run **in parallel**, so a blog with 7 sections takes roughly the same time as writing 1 section.
5. The **Reducer** collects all outputs and merges them into the final blog.

25.6.2 Fan-out Mechanism in LangGraph

LangGraph provides a Send API that enables this dynamic fan-out. The fan-out function loops through all tasks in the plan and sends each task to a worker node:

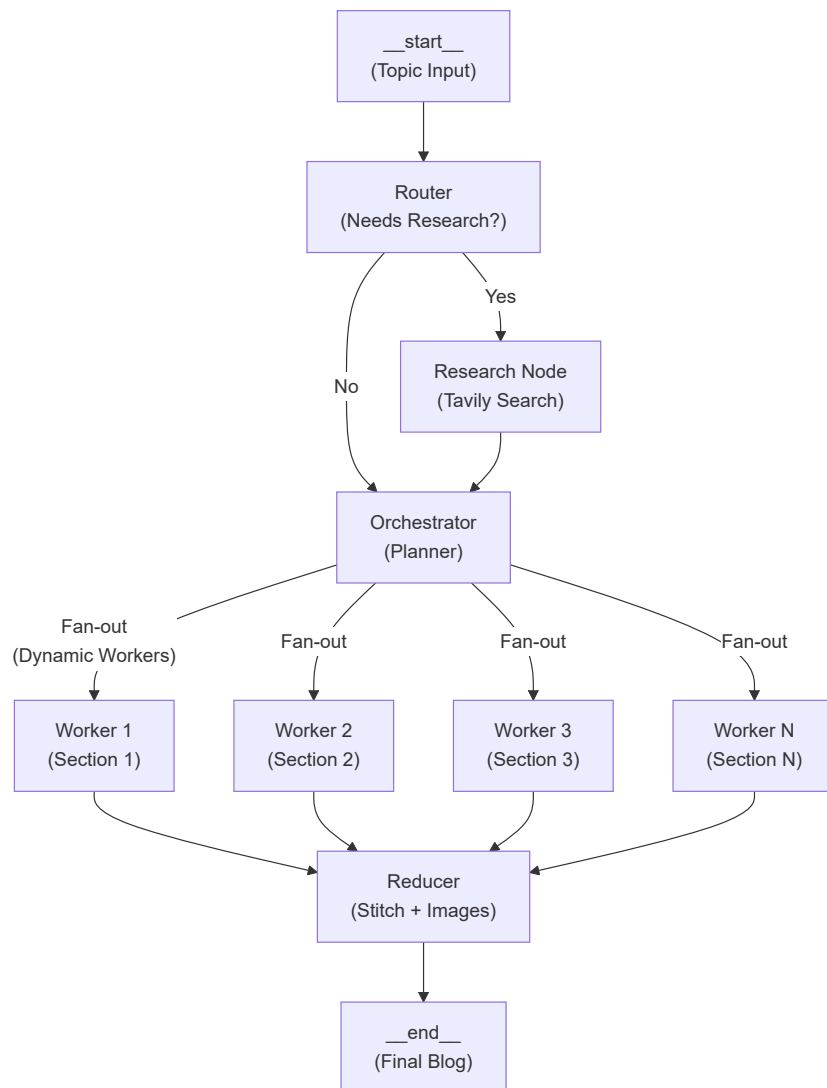


Figure 25.1. Node-by-Node Breakdown

```

from langgraph.types import Send

def fanout(state: State):
    return [
        Send("worker", {
            "task": task,
            "topic": state["topic"],
            "plan": state["plan"]
        })
        for task in state["plan"].tasks
    ]

```

Each Send call triggers one worker node with a specific payload containing the task details, the original topic, and the overall plan (for coherence).

25.6.3 Why Pass the Full Plan to Each Worker?

Even though each worker only writes one section, it receives the **entire plan**. This is intentional — knowing the overall plan helps each worker write its section in a way that is **coherent** with the rest of the blog. For example, if Section 3 is about “Implementation”, it helps to know that Section 2 already covered “Theory”.

25.7 6. Stage 1 — Basic Blog Writing Agent

Stage 1 builds the simplest version of the blog agent — no research, no images, just text generation using the orchestrator–worker–reducer pattern.

25.7.1 Architecture (Stage 1)

```
__start__ → orchestrator → worker(s) → reducer → __end__
```

25.7.2 Pydantic Models (Stage 1 — Basic)

Pydantic models define the **structured output** format that the LLM must follow.

Task Model — Represents one section of the blog:

```
class Task(BaseModel):
    id: int
    title: str
    brief: str = Field(..., description="What to cover")
```

Each task has an ID (section number), a title (section heading), and a brief description telling the LLM what to write in that section.

Plan Model — Represents the overall blog plan:

```
class Plan(BaseModel):
    blog_title: str
    tasks: List[Task]
```

The plan contains the blog’s title and a list of Task objects. If the planner decides the blog needs 5 sections, there will be 5 Task objects in this list.

25.7.3 State Definition

The LangGraph state holds all shared information across nodes:

```
class State(TypedDict):
    topic: str                # User-provided topic
    plan: Plan                # Generated plan
    sections: Annotated[List[str], operator.add] # Worker outputs (reduced)
    final: str                # Final stitched blog
```

The sections field uses `operator.add` as a **reducer**. This means when multiple workers write to sections, their outputs are automatically concatenated into a single list. Without this reducer, each worker’s output would overwrite the previous one.

***What is operator.add here?** In LangGraph, when multiple nodes write to the same state key in parallel, you need to tell the system how to combine those writes. `operator.add` means “append all results into one list.” So if Worker 1 returns `["section1_text"]` and Worker 2 returns `["section2_text"]`, the final sections list becomes `["section1_text", "section2_text"]`.*

25.7.4 LLM Setup

```
llm = ChatOpenAI(model="gpt-4.1-mini")
```

The project uses OpenAI's GPT-4.1-mini model. This is used by both the orchestrator (for planning) and the workers (for writing).

25.7.5 Orchestrator Node (Planner)

The orchestrator uses **structured output** to force the LLM to return a valid `Plan` object:

```
def orchestrator(state: State) -> dict:
    plan = llm.with_structured_output(Plan).invoke(
        [
            SystemMessage(
                content="Create a blog plan with 5-7 sections on the following topic."
            ),
            HumanMessage(content=f"Topic: {state['topic']}"),
        ]
    )
    return {"plan": plan}
```

How this works step-by-step:

1. `llm.with_structured_output(Plan)` — Forces the LLM to output a response that matches the `Plan` Pydantic schema. The output will always be a `Plan` object with a `blog_title` and a list of `Task` objects.
2. A `SystemMessage` tells the LLM its role: create a blog plan with 5–7 sections.
3. A `HumanMessage` provides the actual topic from the user.
4. The returned plan is stored in the state under the key "plan".

25.7.6 Worker Node (Writer)

Each worker receives a payload (not the full state) and writes one section:

```
def worker(payload: dict) -> dict:
    task = payload["task"]          # The specific Task object for this section
    topic = payload["topic"]        # Original user topic
    plan = payload["plan"]          # Full plan (for context)
    blog_title = plan.blog_title

    section_md = llm.invoke(
        [
            SystemMessage(content="Write one clean Markdown section."),
            HumanMessage(
                content=(
                    f"Blog: {blog_title}\n"
                    f"Topic: {topic}\n\n"
                    f"Section: {task.title}\n"
                    f"Brief: {task.brief}\n\n"
                    "Return only the section content in Markdown."
                )
            ),
        ]
    ).content.strip()

    return {"sections": [section_md]}
```

Key points about the worker:

- It receives a **payload** dictionary (not the state), containing the task, topic, and full plan.
- The LLM is instructed to write **only one section** in Markdown format.
- The output is wrapped in a list `[section_md]` because the sections state field expects a list (due to the `operator.add_reducer`).

25.7.7 Reducer Node (Stitcher)

The reducer combines all sections into one final blog document:

```

from pathlib import Path

def reducer(state: State) -> dict:
    title = state["plan"].blog_title
    body = "\n\n".join(state["sections"]).strip()

    final_md = f"# {title}\n\n{body}\n"

    # Save to file
    filename = title.lower().replace(" ", "_") + ".md"
    output_path = Path(filename)
    output_path.write_text(final_md, encoding="utf-8")

    return {"final": final_md}

```

What the reducer does:

1. Extracts the blog title from the plan.
2. Joins all sections with double newlines (`\n\n`) between them.
3. Creates a final Markdown string with the title as an H1 heading (`# Title`) followed by all the section content.
4. Saves the blog to a `.md` file on disk.
5. Returns the final Markdown as the `final` state variable.

25.7.8 Building the Graph

```

g = StateGraph(State)

# Add nodes
g.add_node("orchestrator", orchestrator)
g.add_node("worker", worker)
g.add_node("reducer", reducer)

# Add edges
g.add_edge(START, "orchestrator")
g.add_conditional_edges("orchestrator", fanout, ["worker"])
g.add_edge("worker", "reducer")
g.add_edge("reducer", END)

# Compile
app = g.compile()

```

Edge connections explained:

Table 25.2. *Building the Graph*

Edge	Meaning
START $\rightarrow$ orchestrator	When the graph starts, the orchestrator runs first
orchestrator $\rightarrow$ fanout $\rightarrow$ worker	After planning, fan-out dynamically creates N workers
worker $\rightarrow$ reducer	After all workers finish, the reducer collects results
reducer $\rightarrow$ END	After stitching, the graph ends

25.7.9 Running the Agent

```

out = app.invoke({
    "topic": "Write a blog on Self Attention",
    "sections": []
})

```

The `sections` is initialized as an empty list because it will be populated by the workers.

25.8 7. Improving with Better Prompts & Pydantic Models

The basic agent works but produces mediocre output. Two key improvements dramatically increase quality:

25.8.1 Improvement 1 — More Detailed Pydantic Models

The basic Task model only had id, title, and brief. The improved version adds much more detail:
Improved Task Model:

```
class Task(BaseModel):
    id: int
    title: str
    goal: str # One sentence: what the reader should understand after this section
    bullets: List[str] # 3-5 concrete, non-overlapping sub-points to cover
    target_words: int # How many words this section should have
    tags: List[str] # Tags categorizing this section
    requires_research: bool # Does this section need internet research?
    requires_citations: bool # Does this section need source citations?
    requires_code: bool # Does this section need code examples?
```

Improved Plan Model:

```
class Plan(BaseModel):
    blog_title: str
    audience: str # Target audience (e.g., "ML beginners", "senior engineers")
    tone: str # Writing tone (e.g., "technical", "casual", "formal")
    blog_kind: Literal['explainer', 'tutorial', 'comparison',
                      'news_roundup', 'system_design']
    constraints: List[str]
    tasks: List[Task]
```

The plan now includes audience, tone, blog kind, and constraints — giving the LLM far more context for generating a coherent plan.

25.8.2 Improvement 2 — Elaborate System Prompts

Instead of simple one-line prompts like “Create a blog plan with 5-7 sections”, the improved version uses **detailed, multi-paragraph system prompts** that clearly specify:

- How to structure sections
- What kind of content to include
- How to maintain coherence across sections
- Word count targets
- Formatting guidelines

***Key Takeaway:** The code structure remains identical — same state, same graph, same nodes. The only changes are in the Pydantic models (more fields) and the system prompts (more detailed instructions). Yet these changes produce significantly better output.*

25.9 8. Stage 2 — Adding Internet Research

Stage 2 adds the ability to search the internet when the blog topic requires current or recent information.

25.9.1 New Nodes Added

Two new nodes are introduced: **Router** and **Research**.

25.9.2 Router Node

The router is an **LLM-based decision maker** that examines the topic and decides whether internet research is needed.

RouterDecision Pydantic Schema:

```
class RouterDecision(BaseModel):
    needs_research: bool  # True/False
    mode: Literal['closed_book', 'hybrid', 'open_book']  # Category
    queries: List[str]  # Suggested search queries
```

The router outputs three things:

1. **needs_research (bool)**: A simple yes/no — does this topic need internet research?
2. **mode (Literal)**: Categorizes the topic into one of three modes:

Table 25.3. Router Node

Mode	Description	Example Topic	Needs Research?
closed_book	All needed knowledge is already in the LLM	“Self Attention”, “Transformer Architecture”	No
open_book	Most needed knowledge must come from the internet	“Top AI News of January 2026”	Yes
hybrid	LLM knows the concepts but needs internet for recent details	“Open Source LLMs” — concepts are known, but current models need lookup	Yes

3. **queries (List[str])**: If research is needed, the router generates 4–5 search queries that would help write the blog. For example, for the topic “Evolution of ChatGPT from 2022 to 2026”:

- “ChatGPT version releases and updates from 2022 to 2026”
- “New features introduced in ChatGPT”
- “Performance improvements in ChatGPT”
- “Changes in ChatGPT usage patterns and applications”
- “OpenAI announcements about ChatGPT”

25.9.3 Research Node

If the router says research is needed, the Research node executes the search queries using **Tavily**.

How the Research Node works:

1. Takes the list of search queries from the router.
2. For each query, calls the Tavily API.
3. Each Tavily search returns 2–10 results (configurable via `max_results`).
4. Each result is converted into a standardized **EvidenceItem** object.
5. All EvidenceItems are collected into an **EvidencePack** object.
6. The EvidencePack is stored in the state.

EvidenceItem Pydantic Schema:

```
class EvidenceItem(BaseModel):
    title: str  # Title of the search result
    url: str  # URL of the source
```

```
published_at: Optional[str]      # Publication date
source: Optional[str]           # Source name
snippet: Optional[str]          # Content/text from the page
```

EvidencePack Pydantic Schema:

```
class EvidencePack(BaseModel):
    evidence: List[EvidenceItem]      # Collection of all evidence items
```

Example calculation: If the router generates 5 queries, and each query returns 2 results from Tavily, the EvidencePack will contain 10 EvidenceItem objects.

25.9.4 How Research Feeds into Planning and Writing

Once the research is complete, the flow continues to the **Orchestrator (Planner)**. The planner now receives two inputs:

1. The original **topic** from the user.
2. The **EvidencePack** containing all research results.

The planner uses both to create a better, more informed plan. It can now plan sections that reference recent data, current events, or up-to-date statistics.

Similarly, when the **Worker nodes** write their sections, they also have access to the EvidencePack in the state. If a particular section requires internet-sourced knowledge (as indicated by `requires_research: True` in the Task object), the worker can pull relevant evidence items and incorporate them into the section content.

25.10 9. Tavily — Search Engine for LLMs

Tavily is a specialized search engine designed for AI agents and LLMs. Unlike traditional search engines that return web pages, Tavily returns **clean, structured text content** that LLMs can directly use.

25.10.1 Setting Up Tavily

1. Go to tavily.com and create an account.
2. Get your API key from the dashboard.
3. Store the API key in your `.env` file.

25.10.2 Using Tavily with LangChain

```
from langchain_community.tools.tavily_search import TavilySearchResults

# Create the search tool (max 2 results per query)
tavily_tool = TavilySearchResults(max_results=2)

# Execute a search
results = tavily_tool.invoke("ChatGPT version releases and updates from 2022 to 2026")
```

Each result contains: title, url, and content (the actual text from the webpage).

25.10.3 Why Tavily Instead of Regular Web Search?

Tavily is purpose-built for LLMs. It returns clean, extracted text content rather than HTML pages. This means you don't need to scrape, parse, or clean web pages — Tavily handles all of that and gives you ready-to-use text that can be directly fed into your LLM prompts.

25.11 10. Complete Data Flow Summary

Here is the complete end-to-end data flow of the Blog Writing Agent:

```

Step 1: User provides a TOPIC
↓
Step 2: ROUTER examines the topic
→ Outputs: needs_research (bool), mode, queries[]
↓
Step 3: If needs_research = True:
    RESEARCH NODE runs Tavily searches
    → Each query → Tavily → Results → EvidenceItems
    → All EvidenceItems → EvidencePack → stored in State
    If needs_research = False:
        Skip directly to Orchestrator
↓
Step 4: ORCHESTRATOR (Planner) creates a Plan
→ Uses: topic + EvidencePack (if available)
→ Outputs: Plan object with blog_title, audience, tone, tasks[]
↓
Step 5: FAN-OUT reads the Plan
→ Counts N tasks (sections)
→ Dynamically spawns N Worker nodes
→ Each Worker receives: its Task + topic + full Plan
↓
Step 6: N WORKERS run in PARALLEL
→ Each writes one section in Markdown
→ Each can access EvidencePack from state if needed
→ All outputs collected into sections[] via operator.add reducer
↓
Step 7: REDUCER stitches everything together
→ Joins all sections with newlines
→ Adds title as H1 heading
→ (In Stage 3) Identifies where images are needed and generates them
→ Saves final blog as .md file
↓
Step 8: Final blog is ready!

```

25.11.1 State at Each Stage

Table 25.4. *State at Each Stage*

State Field	Set By	Description
topic	User Input	The blog topic string
needs_research	Router	Boolean — whether internet search is needed
queries	Router	List of search queries (if research needed)
evidence_pack	Research Node	Collection of EvidenceItem objects from Tavily
plan	Orchestrator	Plan object with title, sections, metadata
sections	Workers (parallel)	List of markdown strings, one per section
final	Reducer	The complete stitched blog in Markdown

25.12 Key Concepts Recap

Planning Agent: Think first, act second. Break complex tasks into sub-tasks before executing them.

Orchestrator–Worker Pattern: One node plans (orchestrator), many nodes execute (workers), one node combines results (reducer).

Dynamic Fan-out with Send: LangGraph’s Send API lets you spawn a variable number of worker nodes at runtime based on the plan.

Structured Output: Using `llm.with_structured_output(PydanticModel)` forces the LLM to return data in a specific schema, making it reliable for programmatic use.

Reducer (operator.add): When multiple parallel workers write to the same state key, `operator.add` concatenates all their outputs into a single list.

Router Pattern: An LLM-based decision node that examines input and routes the flow to different paths (research vs. no research).

Tavily: A search engine designed for LLMs that returns clean text content from the web, ideal for AI agents that need internet research capability.

25.13 Blog Writing Agent — Stage 3 (Image Generation) & Stage 4 (Streamlit UI)

25.14 Table of Contents

1. Research-Aware Code Implementation (Recap)
2. Stage 3 — Adding Automatic Image Generation
3. Reducer Subgraph — The Three-Node Pipeline
4. Pydantic Schemas for Image Generation
5. Gemini API Setup for Image Generation
6. Stage 4 — Building the Streamlit GUI
7. Complete System Summary

25.15 1. Research-Aware Code Implementation (Recap)

Before diving into image generation, let us consolidate how the research-aware pipeline (Stage 2) is implemented in code. This section walks through the actual code structure that powers the router, research node, and evidence-aware orchestrator.

25.15.1 All Pydantic Schemas in One Place

The project defines several Pydantic schemas that work together. Here is the complete list and what each one does:

Table 25.5. All Pydantic Schemas in One Place

Schema	Purpose	Used By
Task	Defines one blog section (id, title, goal, bullets, target_words, tags, requires_research, requires_citations, requires_code)	Orchestrator → Worker
Plan	Full blog plan (blog_title, audience, tone, blog_kind, constraints, tasks[])	Orchestrator output

Schema	Purpose	Used By
RouterDecision	Router's output (needs_research, mode, queries[])	Router → conditional edge
EvidenceItem	One search result standardized (title, url, published_at, source, snippet)	Research Node
EvidencePack	Collection of EvidenceItem objects	Research Node → State

25.15.2 Updated State for Research

The state now holds additional fields to support routing and research:

```
class State(TypedDict):
    topic: str          # From user
    mode: str           # From router ("closed_book" / "hybrid" / "open_book")
    needs_research: bool # From router
    queries: List[str]   # From router (search queries)
    evidence: EvidencePack # From research node (Tavily results)
    plan: Plan           # From orchestrator
    sections: Annotated[List[str], operator.add] # From workers
    final: str           # From reducer
```

Each field maps to a specific node that populates it. The evidence field is particularly important — it holds all internet research results that both the planner and workers can access.

25.15.3 Router Node Code

The router uses structured output to force the LLM to return a RouterDecision object:

```
def router(state: State) -> dict:
    topic = state["topic"]
    decision = llm.with_structured_output(RouterDecision).invoke(
        [
            SystemMessage(content=ROUTER_SYSTEM_PROMPT),
            HumanMessage(content=topic),
        ]
    )
    return {
        "needs_research": decision.needs_research,
        "mode": decision.mode,
        "queries": decision.queries,
    }
```

The router system prompt explains the three modes (closed_book, hybrid, open_book), and instructs the LLM to generate 3–10 high-signal search queries if research is needed. It also says queries should be scoped and specific — for example, if the user asks for “last week’s news”, the queries should reflect that time constraint.

25.15.4 Conditional Routing Function

After the router, a simple function decides which node to visit next:

```
def route_next(state: State):
    if state["needs_research"]:
        return "research"
    else:
        return "orchestrator"
```

If needs_research is True, go to the research node. If False, skip research and go directly to the orchestrator (planner).

25.15.5 Research Node Code

The research node loops through all queries from the router, performs Tavily searches, and collects results:

```
def research(state: State) -> dict:
    queries = state["queries"]
    per_query_limit = 6  # Max 6 results per query
    raw_results = []

    for query in queries:
        results = tavily_search(query, max_results=per_query_limit)
        raw_results.extend(results)

    # Convert raw results to EvidenceItem objects
    # De-duplicate by URL
    # Package into EvidencePack
    evidence_pack = llm.with_structured_output(EvidencePack).invoke(
        [
            SystemMessage(content=RESEARCH_SYSTEM_PROMPT),
            HumanMessage(content=str(raw_results)),
        ]
    )
    return {"evidence": evidence_pack}
```

The research system prompt instructs the LLM to act as a “research synthesizer for technical writing.” Its rules are: only include items with a non-empty URL, prefer relevant and authoritative sources (company blogs, docs, reputed outlets), if published_at is explicitly present keep it as-is (if missing set to null — do not guess), keep snippets short, and de-duplicate by URL.

Example calculation: If the router generates 5 queries and per_query_limit is 6, the research node fetches up to 30 raw results from Tavily. After de-duplication and filtering, these become EvidenceItem objects stored in an EvidencePack.

25.15.6 Evidence-Aware Orchestrator

The orchestrator (planner) now receives the evidence along with the topic:

```
def orchestrator(state: State) -> dict:
    plan = llm.with_structured_output(Plan).invoke(
        [
            SystemMessage(content=ORCHESTRATOR_SYSTEM_PROMPT),
            HumanMessage(
                content=(
                    f"Topic: {state['topic']}\n\n"
                    f"Evidence (may be empty):\n{state.get('evidence', '')}"
                )
            ),
        ]
    )
    return {"plan": plan}
```

The key addition is the evidence section in the human message. If the topic was “closed_book” (like “Self Attention”), the evidence will be empty — the planner just uses the LLM’s parametric knowledge. If the topic was “open_book” or “hybrid”, the evidence will contain all the Tavily research results, and the planner uses them to create a more informed and accurate plan.

25.15.7 Evidence-Aware Fan-out and Workers

The fan-out now also passes evidence to each worker:

```
def fanout(state: State):
    return [
        Send("worker", {
            "task": task,
```

```

        "topic": state["topic"],
        "plan": state["plan"],
        "evidence": state.get("evidence", None), # NEW
    })
    for task in state["plan"].tasks
1

```

Each worker can access the evidence pack. If a particular section's `requires_research` flag is `True`, the worker uses the evidence items to write that section with real, cited information from the internet.

25.15.8 Graph Construction with Router

```

g = StateGraph(State)

g.add_node("router", router)
g.add_node("research", research)
g.add_node("orchestrator", orchestrator)
g.add_node("worker", worker)
g.add_node("reducer", reducer)

g.add_edge(START, "router")
g.add_conditional_edges("router", route_next, ["research", "orchestrator"])
g.add_edge("research", "orchestrator")
g.add_conditional_edges("orchestrator", fanout, ["worker"])
g.add_edge("worker", "reducer")
g.add_edge("reducer", END)

app = g.compile()

```

This creates the full graph: `start` $\rightarrow$ `router` $\rightarrow$ `(research?)` $\rightarrow$ `orchestrator` $\rightarrow$ `workers` $\rightarrow$ `reducer` $\rightarrow$ `end`.

25.16 2. Stage 3 — Adding Automatic Image Generation

Stage 3 adds the ability to automatically identify where images should appear in the blog and generate them using Google's Gemini model.

25.16.1 What Changes and What Stays the Same

Everything from `__start__` through the worker node remains **exactly the same** as Stage 2. All changes happen exclusively in the **Reducer** node.

Table 25.6. *What Changes and What Stays the Same*

Component	Changed?	Details
Router Node	No change	Same routing logic
Research Node	No change	Same Tavily search
Orchestrator Node	No change	Same planning logic
Fan-out	No change	Same dynamic worker spawning
Worker Nodes	No change	Same section writing
Reducer Node	Yes — major changes	Now a subgraph with 3 internal nodes

25.16.2 The Old Reducer vs The New Reducer

Old Reducer (Stages 1 & 2): A single node that merges all sections and saves the markdown file. It does two things — stitch sections together and write to file.

New Reducer (Stage 3): A **subgraph** containing three nodes that form a pipeline. It merges sections, identifies image locations, generates images, places them, and then saves the final blog.

25.17 3. Reducer Subgraph — The Three-Node Pipeline

The reducer is now a subgraph with three sequential nodes:

```
merge_content → decide_images → generate_and_place_images
```

25.17.1 Node 1: merge_content

This node does what the old reducer’s first step did — it takes all the sections from the workers and merges them into a single markdown string.

```
def merge_content(state: State) -> dict:
    title = state["plan"].blog_title
    body = "\n\n".join(state["sections"]).strip()
    merged_md = f"# {title}\n\n{body}\n\n"
    return {"merged_markdown": merged_md}
```

At this point, we have a complete textual blog in markdown format, but no images yet. This is just a text string, not a file.

Example output at this stage:

```
# Open Source vs Proprietary LLMs

## 1. What Are Open Source and Proprietary LLMs?

Open source LLMs provide weights and code, while proprietary ones are API-only...

## 2. Key Differences: Control vs Convenience

Proprietary models are easy to use but closed...

## 3. When Should You Choose Each?

It depends on cost, privacy, infrastructure...
```

25.17.2 Node 2: decide_images

This is the most interesting node. It sends the merged markdown to an LLM and asks: “Where in this blog should images be placed, and what kind of images should they be?”

The LLM does two things simultaneously:

First, it inserts **placeholders** into the markdown at locations where images would improve the blog. A placeholder looks like `[[IMAGE_1]]`, `[[IMAGE_2]]`, etc.

Second, for each placeholder, it generates a detailed **ImageSpec** object containing the filename, a prompt for image generation, the desired size, and other metadata.

```
def decide_images(state: State) -> dict:
    image_plan = llm.with_structured_output(GlobalImagePlan).invoke(
        [
            SystemMessage(content=DECIDE_IMAGES_SYSTEM_PROMPT),
```

```

        HumanMessage(content=state["merged_markdown"]),
    ]
)
return {
    "markdown_with_placeholders": image_plan.markdown_with_placeholders,
    "image_specs": image_plan.images,
}

```

The system prompt for this node says: “You are an expert technical editor. Decide if images/diagrams are needed for this blog.” It also enforces rules like maximum 3 images per blog, and each image should materially improve understanding (diagrams, flows, table-like visuals, etc.).

Example output at this stage:

The markdown now has placeholders:

```

# Open Source vs Proprietary LLMs

## 1. What Are Open Source and Proprietary LLMs?

Open source LLMs provide weights and code, while proprietary ones are API-only...

[[IMAGE_1]]

## 2. Key Differences: Control vs Convenience

Proprietary models are easy to use but closed...

```

And for each placeholder, there is an ImageSpec object:

```

ImageSpec(
    placeholder="[[IMAGE_1]]",
    filename="comparison_open_vs_proprietary.png",
    alt="Comparison of Open Source vs Proprietary LLMs",
    caption="Open vs Proprietary LLM Comparison",
    prompt="A clean technical diagram showing a two-column comparison...",
    size="1024",
)

```

25.17.3 Node 3: generate_and_place_images

This final node takes each ImageSpec, calls Google Gemini to generate the actual image, saves it to disk, and replaces the placeholder in the markdown with the image path.

Step-by-step process:

1. Loop through each ImageSpec object.
2. Extract the prompt from the ImageSpec.
3. Call Gemini’s image generation API with that prompt.
4. Receive the generated image bytes.
5. Save the image to a folder (e.g., images/comparison_open_vs_proprietary.png).
6. In the markdown string, replace [[IMAGE_1]] with “
7. After all images are processed, save the final markdown as a .md file.

```

def generate_and_place_images(state: State) -> dict:
    md = state["markdown_with_placeholders"]
    specs = state["image_specs"]

    for spec in specs:
        # Generate image using Gemini
        image_bytes = gemini_generate_image_bytes(prompt=spec.prompt)

        # Save to disk
        image_path = f"images/{spec.filename}"

```

```
save_image(image_bytes, image_path)

# Replace placeholder with markdown image syntax
md = md.replace(
    spec.placeholder,
    f"![{spec.alt}]({image_path})"
)

# Save final markdown file
filename = state["plan"].blog_title.lower().replace(" ", "_") + ".md"
Path(filename).write_text(md, encoding="utf-8")

return {"final": md}
```

Example — Final markdown after image placement:

```
# Open Source vs Proprietary LLMs

## 1. What Are Open Source and Proprietary LLMs?

Open source LLMs provide weights and code, while proprietary ones are API-only...

![[Comparison of Open Source vs Proprietary LLMs](images/comparison_open_vs_proprietary.png)]

## 2. Key Differences: Control vs Convenience

Proprietary models are easy to use but closed...
```

When you open this markdown file, the text is displayed normally, and the images appear at the exact positions the LLM decided they should go.

25.18 4. Pydantic Schemas for Image Generation

Two new Pydantic schemas are introduced in Stage 3:

25.18.1 ImageSpec

Represents the specification for one image to be generated:

```
class ImageSpec(BaseModel):
    placeholder: str # e.g., "[[IMAGE_1]]"
    filename: str # e.g., "comparison_diagram.png"
    alt: str # Alt text for accessibility
    caption: str # Caption displayed below the image
    prompt: str # Detailed prompt for Gemini to generate the image
    size: str # Image resolution, e.g., "1024"
```

The prompt field is the most important — it contains a detailed description that is sent to Gemini’s image generation API. The LLM crafts this prompt based on the blog content, so the generated image is contextually relevant.

25.18.2 GlobalImagePlan

The combined output of the `decide_images` node:

```
class GlobalImagePlan(BaseModel):
    markdown_with_placeholders: str # Full blog markdown with [[IMAGE_N]] placeholders
    images: List[ImageSpec] # One ImageSpec per placeholder
```

This schema cleanly packages both the modified markdown and all the image specifications, making it easy to pass between nodes.

25.18.3 Updated State Fields

The state gets these new fields to support image generation:

```
class State(TypedDict):
    # ... all previous fields ...
    merged_markdown: str          # Output of merge_content node
    markdown_with_placeholders: str # Output of decide_images node
    image_specs: List[ImageSpec]   # Output of decide_images node
```

25.18.4 Complete Schema Relationship

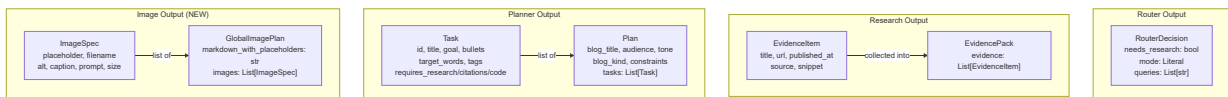


Figure 25.2. Complete Schema Relationship

25.19 5. Gemini API Setup for Image Generation

25.19.1 Getting the API Key

1. Go to ai.google.dev (Google AI Studio).
2. Click “Get API Key”.
3. Click “Create API Key”.
4. Select an existing Google Cloud project or create a new one.
5. Copy the generated API key.
6. Paste it into your `.env` file:

```
G00GLE_API_KEY=your_api_key_here
```

25.19.2 Cost Considerations

The free tier may have limited image generation capacity. The pay-as-you-go model is very affordable — approximately ₹135 (less than \$2) for around 30–40 blog generations with images during the entire tutorial development. For typical usage, expect costs around ₹100 or less.

Important warning: Never share your API key publicly. If someone else uses your key, your bill can increase significantly. Always keep keys in `.env` files and add `.env` to your `.gitignore`.

25.19.3 Gemini Image Generation Helper Function

The helper function that calls Gemini’s API:

```
from google import genai

def gemini_generate_image_bytes(prompt: str) -> bytes:
    """
    Sends a prompt to Gemini's image generation model
    and returns the raw image bytes.
    """
    client = genai.Client()
    response = client.models.generate_content(
```

```
model="gemini-2.0-flash-exp",      # or appropriate model
contents=prompt,
config=genai.types.GenerateContentConfig(
    response_modalities=["image"]
),
)
# Extract image bytes from response
image_bytes = response.candidates[0].content.parts[0].inline_data.data
return image_bytes
```

This function takes a prompt string, sends it to Gemini, and returns the generated image as raw bytes. The calling code then saves these bytes to a file.

25.20 6. Stage 4 — Building the Streamlit GUI

Stage 4 wraps the entire backend into a user-friendly web interface using **Streamlit**.

25.20.1 Why Streamlit?

Streamlit is one of the easiest ways to create a GUI in Python. It requires minimal frontend knowledge and lets you build interactive web apps with just Python code. While production environments in companies might use React or other frontend frameworks, Streamlit is perfect for learning projects and prototypes.

25.20.2 How the GUI Was Built

The approach used was **vibe coding** — planning the UI specifications clearly and then using ChatGPT to generate the entire Streamlit code:

1. **Plan the UI layout:** What components should the page have? A text input for the topic, a “Generate Blog” button, progress indicators, tabs for Plan/Blog/Evidence/Logs/Images.
2. **Write clear specifications:** Describe exactly what each component should do, how the layout should look, and what data it displays.
3. **Feed specifications + backend code to ChatGPT:** Give it the spec and show the backend code (the compiled LangGraph app).
4. **Receive the complete Streamlit code:** ChatGPT generates the entire `app.py` file (around 200–300 lines).

25.20.3 Connecting Backend to Frontend

The backend code (everything from router through reducer with images) is placed in a single Python file, e.g., `agent.py`. The compiled LangGraph app is imported into the Streamlit file:

```
# In streamlit_app.py
from agent import app # The compiled LangGraph graph

# Use app.invoke() or app.stream() to run the agent
result = app.invoke({"topic": user_topic, "sections": []})
```

25.20.4 Running the Application

```
streamlit run streamlit_app.py
```

This launches a local web server, and the GUI opens in your browser.

25.20.5 GUI Features

The Streamlit GUI provides these features:

Left Panel — Input: A text input field where you type the blog topic (just a topic name, or a topic with detailed description of what you want).

Generate Blog Button: Triggers the agent. Shows real-time progress as the agent moves through nodes (Router → Research → Orchestrator → Worker → Reducer).

Tabs in the Output Area:

Table 25.7. GUI Features

Tab	Content
Plan	Shows the generated plan — title, audience, tone, sections breakdown, target words per section, tags, research/citation/code flags
Blog	The final rendered blog with text and images, plus a download button
Evidence	All citations and source links used (populated only when research was performed)
Logs	Behind-the-scenes agent decision-making log
Images	Gallery of all generated images

Blog History: Previously generated blogs are saved and accessible from a sidebar. You can revisit any blog you generated in the past.

25.21 7. Complete System Summary

25.21.1 Full Architecture — All Four Stages Combined

25.21.2 What Each Stage Added

Table 25.8. What Each Stage Added

Stage	What Was Added	Key Components
Stage 1	Basic text-only blog generation	Orchestrator → Workers → Reducer (simple merge)
Stage 2	Internet research capability	Router + Research Node (Tavily) + Evidence-aware planning
Stage 3	Automatic image generation	Reducer subgraph (merge → decide_images → generate_and_place) + Gemini API
Stage 4	Web-based GUI	Streamlit frontend connecting to LangGraph backend

25.21.3 The Three-Step Reducer Pipeline in Detail

This is the core addition in Stage 3. Here is the complete flow:

Step 1 — merge_content: Input: List of section strings from workers. Process: Join all sections with newlines, add the title as H1 heading. Output: A single markdown string (text only, no images).

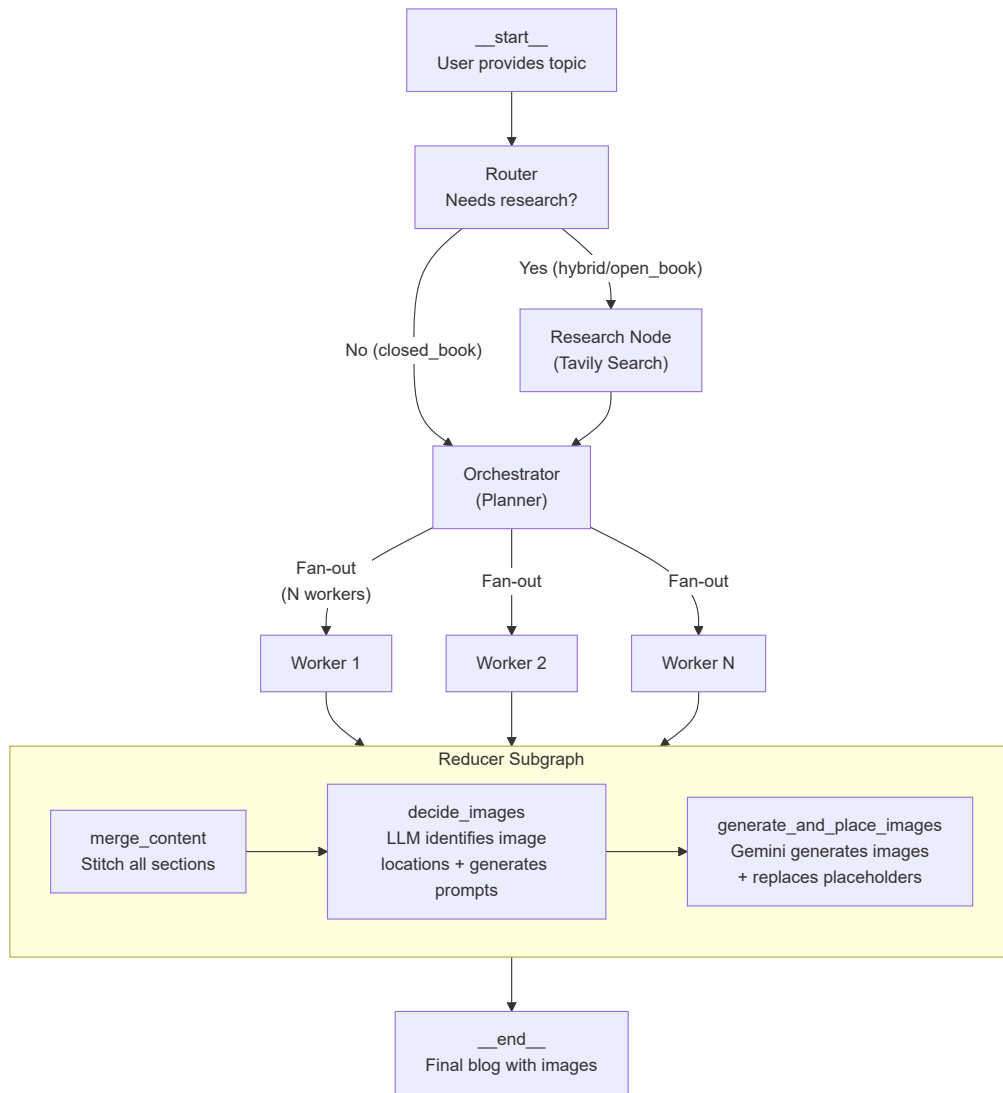


Figure 25.3. Full Architecture — All Four Stages Combined

Step 2 — decide_images: Input: The merged markdown string. Process: Send to LLM with a prompt asking it to identify 1–3 locations where images would improve the blog. The LLM returns a `GlobalImagePlan` containing the markdown with `[[IMAGE_N]]` placeholders inserted at chosen locations, and a list of `ImageSpec` objects (one per placeholder) with detailed generation prompts. Output: Markdown with placeholders + List of `ImageSpec` objects.

Step 3 — generate_and_place_images: Input: Markdown with placeholders + `ImageSpec` list. Process: For each `ImageSpec`, call Gemini API with the prompt, save the generated image to disk, and replace the `[[IMAGE_N]]` placeholder with the markdown image syntax `'`. Output: Final complete markdown with embedded image references, saved as `a.md` file.

25.21.4 Technology Stack

Table 25.9. Technology Stack

Technology	Role
LangGraph	Graph-based agent orchestration framework
OpenAI GPT-4.1-mini	LLM for planning, routing, writing, and image decision-making

Technology	Role
Tavily	Search engine for LLMs (internet research)
Google Gemini	Image generation from text prompts
Pydantic	Structured output schemas for reliable LLM responses
Streamlit	Python-based web GUI framework
LangChain	LLM interaction layer (ChatOpenAI, SystemMessage, HumanMessage)

25.21.5 Key Design Decisions

Why a Subgraph for the Reducer? The reducer's job grew from 2 steps (merge + save) to 5 steps (merge + identify image spots + generate prompts + generate images + replace placeholders + save). Converting it to a subgraph keeps the main graph clean and makes the reducer's internal logic modular and testable.

Why Does the LLM Decide Image Placement (Not the Workers)? Individual workers only see their own section. They cannot make good decisions about image placement for the overall blog. The reducer sees the entire merged blog, so it can make holistic decisions about where images add the most value and avoid over-cluttering.

Why Maximum 3 Images? Too many images slow down generation (each requires a Gemini API call), increase costs, and can make the blog feel cluttered. Three images is a practical limit that balances visual richness with efficiency.

Why Gemini for Images Instead of DALL-E? Gemini was chosen for its cost-effectiveness and quality of technical diagrams. The pay-as-you-go pricing is very affordable (under ₹135 for 30–40 complete blog generations).

25.21.6 Portfolio Project Potential

This project has significant potential as a portfolio piece for interviews. Areas for improvement include: adding more blog types (non-technical), supporting different output formats (HTML, PDF), improving image quality with better prompts, adding user feedback loops, implementing caching for repeated queries, and building a more polished frontend with React.